



pvx

User Guide

Colby Leider

Time stretch, pitch shift, automation, augmentation, and reference workflows

Command-line practice, quality tuning, multistage rendering, and machine-learning pipelines

pvx repository handbook

Compiled from project documentation and repository source

August 01, 2026

About This Guide

This book is a curated, book-length guide to **pvx** built from the repository’s user-facing documentation, examples, and command reference. It is meant to be read in layers: a practical first pass for musicians and engineers who want useful results quickly, a deeper pass for readers who want to understand the phase-vocoder model, and a reference pass for people building repeatable workflows, augmentation pipelines, or release processes around the tool.

The supported public surface for the current alpha remains intentionally narrow: **pvx**, **pvxvoc**, **pvxfreeze**, **pvxwarp**, **pvxformant**, **pvxfilter**, **pvxretune**, and **pvxanalysis**. Where the appendices document broader flags and exploratory surfaces, they should be read as inventory rather than a stronger compatibility promise.

Preface

[TBA]

This preface marks the beginning of a longer editorial layer for the pvx project. The technical reference can tell a reader what a flag accepts; it cannot by itself answer the question that arises in a session: what should I listen for, what should I change first, and when is an artifact useful rather than merely accidental? This book exists to put those questions beside the commands.

Phase-vocoder work rewards both curiosity and restraint. A small time change can be nearly invisible. An extreme stretch can make a cymbal bloom into weather. A voice can be moved through an impossible register while still carrying a recognisable vowel. The same transform that yields a clean utility render can become an instrument when it is pushed past its transparent operating range. pvx is built for that continuum, from sensible production repair to deliberate spectral invention.

The guide therefore has two jobs. One is pragmatic: it should help someone reach a dependable result without treating digital signal processing as a private language. The other is historical and musical: it should make clear that these tools come from a long line of experiments in speech, tape, synthesis, and computer music. The algorithms are not neutral buttons. They embody choices about continuity, frequency resolution, timing, and what a listener is willing to hear as a coherent sound.

The present volume is intentionally a living document. Its public contract follows the alpha surface described in the release material. Some pages record exploratory capabilities because understanding the surrounding landscape is useful, but a printed description is not a promise that every experiment will remain stable. The supported commands are the ones to carry into a session, a batch pipeline, or a release workflow.

The chapters can be read in order, but they do not demand it. A producer can start with the guided workflows and return to the mathematics only when a decision needs explanation. A researcher can begin with the first chapter and then move directly to architecture, windows, and the API material. A person building a dataset pipeline can treat the cookbook and flag reference as a working bench. The same book should be useful at three speeds: a quick answer during a render, a careful study at a desk, and a deeper rereading after one has heard the artifacts for oneself.

The remaining preface will eventually include a fuller account of the project’s origin, contributors, and the listening practices that informed its defaults. Until then, [TBA] is kept here deliberately: it is an invitation to distinguish the finished technical apparatus from the still-forming human story.

Introduction

[TBA]

pvx is a command-line environment for changing how recorded sound occupies time, pitch, spectrum, and space. Its core is the phase vocoder, but the experience of using it is not reducible to one algorithm. The command line holds choices about analysis windows, phase propagation, transient preservation, automation curves, channels, format, loudness, and output discipline. This introduction offers a map of those choices before the later chapters name every option.

0.1 The central promise

The ordinary recording presents pitch and duration as linked facts. Play a tape faster and the event becomes shorter and higher. Play it slower and it becomes longer and lower. Digital analysis makes it possible to loosen that link. Time can be stretched while pitch remains broadly stable; pitch can move while duration remains broadly stable. The word *broadly* matters. Audio is not a set of independent knobs. A transform must decide what continuity to preserve, and every decision leaves a trace.

pvx gives that trace a place in the workflow. Some material wants transparency: dialogue, a solo instrument, a finished mix, or a restoration pass. Other material wants character: a frozen chord, a voice that turns granular at the edges, or an attack that opens into a spectral cloud. Both are valid uses. The practical skill is to name the goal before selecting the settings.

0.2 A workflow begins with listening

Before choosing an FFT size or a phase-locking mode, listen to the source as if it were a score. Ask where the important attacks are, whether the sound is dominated by pitched partials or noise, whether the stereo image carries musical information, and how much change the listener is expected to accept. A drum loop and a sustained organ chord may both be called audio, but they make almost opposite demands on a transform. The loop asks for sharp onset identity; the chord asks for stable harmonic motion.

One useful first pass is to render only a short region. Keep the source untouched. Make a conservative version and an ambitious version. Compare them at the level where the finished work will be heard. Headphones reveal phase movement and high-frequency fizz. Speakers reveal whether a transient still has weight. A mono fold-down reveals whether a stereo treatment has undermined the image. This is not extra ceremony. It is how a parameter becomes audible rather than theoretical.

0.3 The four levers

Most pvx work can be understood through four levers:

1. **Time.** The stretch ratio changes the output duration. It also changes the distance between reconstructed analysis frames, which is why large ratios make phase and transient decisions more audible.
2. **Pitch.** Semitone and cent controls map musical intervals to frequency ratios. Pitch moves are often paired with formant treatment when a voice needs to retain its apparent vocal-tract identity.
3. **Continuity.** Window length, hop size, phase locking, and transient modes determine how the analysis observes motion and how the resynthesis joins it together.
4. **Change over time.** Automation lets a parameter follow a curve. Instead of treating a file as a single static setting, a render can grow, bend, open, narrow, or shift across its duration.

The later reference material separates these controls because the command line needs precise names. While learning, it is better to remember their relationship. Time and pitch describe the intended transformation. Continuity describes the cost of making it sound whole. Automation describes how the transformation moves through the piece.

0.4 Transparent, expressive, and extreme rendering

There are three useful attitudes toward phase-vocoder output. Transparent rendering aims to leave the listener unsure that anything happened except the requested change. It favours moderate ratios, appropriate windows, protected transients, and careful comparison against the source. Expressive rendering lets a little process become part of the timbre. It can work beautifully on pads, sampled voices, guitars, and ambient material. Extreme rendering makes the artifacts themselves the subject: the granular haze, the stretched breath, the unstable pitch edge, and the strange persistence of a single moment.

None of these attitudes is inherently more advanced than another. The mistake is to treat an extreme result as failed transparency, or a clean result as timid creativity. The command is only half the gesture. The context in which it is heard completes it.

0.5 The stable surface and the experimental edge

For the current alpha, this guide treats `pvx`, `pvxvoc`, `pvxfreeze`, `pvxwarp`, `pvxformant`, `pvxfilter`, `pvxtune`, and `pvxanalysis` as the supported command surface. They are the commands to learn first and the commands around which fixtures and release checks are organised. Other names may appear in source, examples, or larger inventories. Those are useful for orientation, but they do not widen the compatibility promise.

This distinction is a kindness to future work. It lets the project explore without pretending every experiment is ready to be depended upon. It also gives readers a clear place to stand. Learn the supported tools deeply. Use the exploratory material with curiosity, a disposable output directory, and a willingness to verify the result.

0.6 Reading and working with this book

Each major chapter has a different role. The first chapter establishes the vocabulary and history. Getting Started gives a dependable first render. The Quality Guide teaches diagnostic listening and parameter tuning. Mathematical Foundations explains why windows, hops, and phase updates behave as they do. The cookbook chapters turn recurring musical and production tasks into named recipes. The appendix is for the moment when a descriptive explanation is no longer enough and an exact flag is needed.

The document is also designed to be used with a terminal open. Read a small section, run a small command, and listen. Preserve the command line that produced a result you like. Keep input and output names unambiguous. For longer work, use checkpoints, manifests, or explicit notes about the settings. The best render is hard to value if it cannot be reproduced.

The introduction is deliberately fuller than a command synopsis, and it will grow with the project. The [TBA] marker remains because this is still an alpha handbook. The technical route is present now; the final account of its audience, practice, and evolution is still being written.

Reading Paths

If you are new to **pvx**, read this book in this order:

1. *Getting Started with pvx*
2. *pvx Quality Guide*
3. *pvx Example Cookbook*
4. The appendices when you need exact flags or deployment details

If you are using **pvx** as a production or research tool, add:

1. *Mathematical Foundations*
2. *pvx Application Programming Interface (API) Overview*
3. *ML Integration*
4. *CLI Flags Reference*

List of Symbols

This register collects the mathematical notation used throughout the guide. A symbol may acquire a more specific meaning inside a local derivation; when that happens, the accompanying where clause takes precedence. Frequencies are in hertz unless angular frequency is indicated, phase is in radians, time is in seconds unless indexed by samples or frames, and ratios are dimensionless.

Table 1: Symbols and notation used in this guide.

Symbol	Meaning	Units or domain
Indices, sets, and elementary quantities		
n	Discrete-time sample index	integer
m	Frame, segment, or synthesis-grain index	integer
t	Continuous time or, when subscripted, frame index	seconds or integer
k	Fourier-bin index	$0 \leq k < N$
i, j	Generic item, control-point, partial, or matrix indices	integer
c	Audio-channel index	integer
q	Quantization, scale-step, or transform-order index	integer
ℓ	Summation index or generalized-cosine coefficient index	integer
$\mathbb{R}$	Set of real numbers	set
$\mathbb{C}$	Set of complex numbers	set
$j = \sqrt{-1}$	Imaginary unit used in complex exponentials	complex
$\lceil z \rceil$	Smallest integer greater than or equal to z	integer
$ z $	Absolute value or complex magnitude of z	nonnegative real
$\arg z$	Principal complex angle of z	radians
Signals, sampling, and duration		
$x[n]$	Input discrete-time signal	amplitude
$y[n]$	Output discrete-time signal	amplitude
$x_c[n]$	Input signal in channel c	amplitude
$y_c[n]$	Output signal in channel c	amplitude
$x(t)$	Continuous-time signal model	amplitude
f_s	Sampling frequency	samples per second
$T_s = 1/f_s$	Sampling interval	seconds
L	Signal length	samples
T	Signal or requested render duration	seconds
N_s	Total number of samples in a signal or render	samples
$x_{\max}$	Peak absolute sample magnitude	amplitude
x_{rms}	Root-mean-square signal magnitude	amplitude
$\delta[n]$	Unit impulse	sequence
$*$	Convolution operator	operation
Frames, transforms, and the STFT		
N	FFT or transform size	samples
M	Window length when distinguished from N	samples
H_a	Analysis hop size	samples per frame
H_s	Synthesis hop size	samples per frame
$R = N/H_a$	Analysis overlap factor when N is divisible by H_a	dimensionless
F	Number of analysis frames or transforms, as defined locally	count
$w[n]$	Analysis-window coefficient	dimensionless
$g[n]$	Synthesis-window coefficient	dimensionless
$X_t[k]$	Complex STFT coefficient at frame t , bin k	complex amplitude

Symbol	Meaning	Units or domain
$Y_t[k]$	Modified or synthesized complex spectrum	complex amplitude
$A_t[k] = X_t[k] $	STFT magnitude	amplitude
$P_t[k] = X_t[k] ^2$	STFT power	amplitude squared
$\phi_t[k]$	Observed input phase	radians
$\hat{\phi}_t[k]$	Reconstructed output phase	radians
$\Delta\phi_t[k]$	Wrapped phase residual after expected advance is removed	radians
$\text{princarg}(\theta)$	Wrapping of angle θ to a principal interval	radians
$\omega_k = 2\pi k/N$	Nominal digital angular frequency of bin k	radians per sample
$\hat{\omega}_t[k]$	Estimated instantaneous angular frequency	radians per sample
$f_k = kf_s/N$	Nominal frequency of bin k	hertz
$f_{\text{Nyq}} = f_s/2$	Nyquist frequency	hertz
$\mathcal{F}\{x\}$	Fourier transform of x	transform
$\mathcal{F}^{-1}\{X\}$	Inverse Fourier transform of X	transform
DFT_N	Length- N discrete Fourier transform	transform
IDFT_N	Length- N inverse discrete Fourier transform	transform
Time scaling, remapping, and reconstruction		
α	Global output-duration or time-scale ratio	$\alpha > 0$
$a(t)$	Time-varying stretch ratio	positive real
$\tau(t)$	Mapping between source and output time, as stated locally	seconds
$d\tau/dt$	Local slope of a time map	dimensionless
$\rho = H_s/H_a$	Hop ratio in a basic phase-vocoder schedule	positive real
$C[n]$	Overlap-add normalization sum	dimensionless
$\sum_m w[n - mH]$	Constant-overlap-add window sum	dimensionless
$\sum_m w^2[n - mH]$	Squared-window overlap normalization sum	dimensionless
ϵ	Small positive numerical floor	positive real
B	Chunk, block, or streaming-buffer length	samples
D	Analysis frame rate in historical PVC notation	frames per second
Window analysis and design		
$W(\omega)$	Discrete-time Fourier transform of a window	complex response
CG	Coherent gain of a window	dimensionless
$ENBW$	Equivalent noise bandwidth	FFT bins
SL	Scalloping loss	decibels
β	Kaiser-window shape parameter	nonnegative real
σ	Gaussian-window standard deviation	samples or ratio
α_w	Window-specific taper or shape parameter	dimensionless
a_ℓ	Generalized-cosine window coefficient	dimensionless
I_0	Modified Bessel function of the first kind, order zero	function
$\Gamma(z)$	Gamma function	function
K	Highest cosine-series order in a generalized window	integer
$\Delta f = f_s/N$	FFT-bin spacing	hertz
B_{ML}	Main-lobe width	FFT bins or hertz
A_{SL}	Peak sidelobe level	decibels
Pitch, tuning, partials, and formants		
$f_0(t)$	Fundamental-frequency trajectory	hertz
f_{ref}	Reference or tuning frequency	hertz
f_{A4}	Frequency assigned to concert A4	hertz
r	Pitch-shift or frequency ratio	positive real
s	Pitch displacement	semitones
c_{cent}	Pitch displacement	cents
$r = 2^{s/12}$	Equal-tempered ratio for s semitones	dimensionless
$r = 2^{c_{\text{cent}}/1200}$	Ratio for a displacement in cents	dimensionless
p/q	Rational tuning interval	positive rational

Symbol	Meaning	Units or domain
E	Number of equal divisions of an octave	integer
$f_i(t)$	Frequency of partial or tracked component i	hertz
$A_i(t)$	Amplitude trajectory of partial i	amplitude
F_i	Center frequency of formant i	hertz
B_i	Bandwidth of formant i	hertz
$E_t[k]$	Spectral-envelope estimate	amplitude
h	Harmonic number	positive integer
η	Inharmonicity coefficient	nonnegative real
$v(t)$	Voicing probability or voiced-state measure	$0 \leq v \leq 1$
$\kappa(t)$	Pitch-confidence measure	$0 \leq \kappa \leq 1$
Control curves, routing, and interpolation		
$u(t)$	Generic time-varying control signal	parameter-dependent
u_i	Control value at point i	parameter-dependent
t_i	Timestamp of control point i	seconds
λ	Normalized position between two control points	$0 \leq \lambda \leq 1$
$h(\lambda)$	Easing or interpolation-shape function	$[0, 1] \rightarrow [0, 1]$
$3\lambda^2 - 2\lambda^3$	Smoothstep interpolation polynomial	dimensionless
$6\lambda^5 - 15\lambda^4 + 10\lambda^3$	Smooterstep interpolation polynomial	dimensionless
p	Polynomial interpolation order in that context	integer, $p \geq 1$
a, b	Scale and offset in an affine route $au + b$	parameter-dependent
$u_{\min}, u_{\max}$	Lower and upper control clamps	parameter-dependent
r_c	Control sampling rate	values per second
$\mathcal{R}(u)$	Generic route or control transformation	function
Spectral filtering, morphing, and cross-synthesis		
$H[k]$	Static complex frequency response	complex gain
$R_t[k]$	Time-varying response or reference spectrum	complex gain
$M_t[k]$	Spectral mask	nonnegative real
γ	Mask exponent, spectral-warp exponent, or coherence value as defined locally	dimensionless
μ	Morph, mix, or phase-interpolation amount	$0 \leq \mu \leq 1$
X_A, X_B	Spectra of source A and source B	complex amplitude
A_A, A_B	Magnitudes of source A and source B	amplitude
ϕ_A, ϕ_B	Phases of source A and source B	radians
$G_t[k]$	Applied spectral gain	nonnegative real
$\theta_t[k]$	Output phase angle after phase mixing	radians
$\mathcal{P}$	Set of detected spectral peaks	index set
k_p	Bin index of a spectral peak	integer
$\mathcal{N}(k_p)$	Bin neighborhood associated with peak k_p	index set
Q	Resonator quality factor	positive real
f_r	Resonance center frequency	hertz
τ_r	Resonance or feedback decay time	seconds
Dynamics, levels, and mastering		
A	Linear amplitude or gain when unambiguous locally	nonnegative real
A_{dB}	Amplitude level in decibels	dB
$20 \log_{10} A $	Linear-amplitude to decibel conversion	dB
P_{dB}	Power level in decibels	dB
$10 \log_{10} P$	Linear-power to decibel conversion	dB
θ	Compressor, expander, limiter, or gate threshold	dB
R_c	Compression ratio	ratio
R_e	Expansion ratio	ratio
G_{dB}	Gain reduction or makeup gain	dB
τ_a	Attack time constant	seconds

Symbol	Meaning	Units or domain
τ_d	Decay or release time constant	seconds
L_K	K-weighted loudness quantity	LUFS-related
L_{target}	Requested integrated loudness target	LUFS
P_{true}	True-peak level	dBTP
c_{clip}	Soft-clip or hard-clip ceiling	amplitude
Stereo, multichannel, and spatial quantities		
$L[n], R[n]$	Left and right channel signals	amplitude
$M[n] = (L[n] + R[n])/2$	Mid signal	amplitude
$S[n] = (L[n] - R[n])/2$	Side signal	amplitude
$\Gamma_{xy}(k)$	Complex coherence between channels x and y	complex; magnitude $[0, 1]$
$\text{IPD}(k)$	Interchannel phase difference	radians
ILD	Interaural or interchannel level difference	decibels
ITD	Interaural or interchannel time difference	seconds
c_{ref}	Reference channel used for phase or coherence locking	channel index
ζ	Coherence-strength control	$0 \leq \zeta \leq 1$
C	Number of channels when used as a count	positive integer
Features, statistics, and evaluation		
RMS	Root-mean-square level or feature	amplitude
SNR	Signal-to-noise ratio	decibels
SISDR	Scale-invariant signal-to-distortion ratio	decibels
LSD	Log-spectral distance	decibels
MAE	Mean absolute error	parameter-dependent
RMSE	Root-mean-square error	parameter-dependent
$\bar{x}$	Arithmetic mean of observations	same as x
$\tilde{x}$	Median or robust central estimate	same as x
σ_x	Standard deviation of x	same as x
$\text{cov}(x, y)$	Covariance of x and y	product units
$\text{corr}(x, y)$	Correlation coefficient	$[-1, 1]$
$z_d(t)$	Feature dimension d at time t	feature-dependent
$\mathbf{z}(t)$	Feature vector at time t	real vector
d	Feature-vector dimension	positive integer
$\mathcal{D}$	Dataset, corpus, or evaluation collection	set
$\mathcal{L}$	Loss or objective function	nonnegative real
Computational and reproducibility notation		
$\mathcal{O}(\cdot)$	Asymptotic computational complexity	order notation
$N \log_2 N$	Characteristic FFT operation-count scale	operations
s_{seed}	Deterministic random or dither seed	integer
h_{SHA256}	SHA-256 content digest	256-bit value
v_{schema}	Artifact or manifest schema version	identifier
b	Output bit depth in export context	bits per sample
r_b	Bit rate	bits per second
Δt_c	Streaming chunk duration	seconds
N_c	Samples per chunk	samples

Contents

About This Guide	ii
Preface	iii
Introduction	iv
0.1 The central promise	iv
0.2 A workflow begins with listening	iv
0.3 The four levers	iv
0.4 Transparent, expressive, and extreme rendering	v
0.5 The stable surface and the experimental edge	v
0.6 Reading and working with this book	v
Reading Paths	vi
List of Symbols	vii
 I Foundations	 1
1 What Is a Phase Vocoder?	2
1.1 Vocoder versus phase vocoder	2
1.2 A quick mental model	3
1.3 From waveform to WOLA frames	3
1.4 The STFT creates a time-frequency lattice	4
1.5 Reading frequency from phase advance	5
1.6 Moving the frames without moving their pitch	6
1.7 Inverse STFT and weighted overlap-add	7
1.8 Why basic phase propagation can still sound wrong	8
1.9 The historical thread	8
Speech analysis before digital audio	8
Tape, television, and the wish to uncouple time from pitch	10
Flanagan’s digital formulation	10
From research technique to musical tool	11
A chronology of ideas, not merely machines	11
Flanagan and Golden: phase as transmitted information	12
Portnoff: the FFT makes the method practical	12
Computer music and the widening of purpose	13
Mark Dolson and the tutorial synthesis	13
Horizontal coherence and vertical coherence	14
Jean Laroche and Mark Dolson: explaining phasiness	14
Pitch shifting, harmonising, and spectral translation	15
Transients expose the limits of stationarity	16
Stereo and multichannel coherence	16
1.10 A deeper history of spectral time	17

Harmonic analysis before electronic sound	17
Telegraphy, telephony, and the filter-bank imagination	18
Homer Dudley, the vocoder, and the Voder	18
Secure speech and large-scale vocoder engineering	19
Tape music and independent control by segmentation	19
Sampling, digital audio, and the FFT threshold	24
Flanagan and Golden in the Bell Labs setting	25
Portnoff and the practical digital formulation	25
James A. Moorer and the computer-music turn	25
UC San Diego, CARL, and a software culture	26
Mark Dolson: research, software, and pedagogy	26
Sinusoidal modeling and a neighboring lineage	26
IRCAM and the institutionalization of spectral transformation	27
Jonathan Harvey and spectral ambiguity	27
Roger Reynolds and the genealogy of <i>Transfigured Wind</i>	28
Trevor Wishart and sound morphing	28
JoAnn Kuchera-Morin and expanded duration	28
Puckette, phase locking, and real-time environments	29
Laroche and Dolson: from artifact to explanatory model	29
Transients become a separate research object	29
WSOLA, PSOLA, granular methods, and productive rivalry	30
Spectral envelopes, formants, and the source-filter return	30
Noise, residuals, and the limits of sinusoidal phase	31
Stereo, spatial hearing, and multichannel history	31
Open tools, standards, and software migration	31
From offline render to interactive instrument	32
Adaptive and nonstationary time-frequency analysis	32
Machine learning enters the surrounding workflow	32
A historiography of claims and cautions	33
Reading the foundational papers as historical documents	33
The hidden history of computation time	35
A fuller genealogy of phase-vocoder software	36
The analysis file as a new kind of score	38
Compositional histories beyond the canonical examples	39
Artifact history as listening history	40
Teaching, diagrams, and the public life of the algorithm	41
Bell Laboratories as a long technical precondition	41
Stanford, CCRMA, and model-based sound	42
UCSD, CARL, and the command-line studio in detail	42
IRCAM as a meeting of research and commission	43
Speech as the first difficult material	43
Sustained instrumental sound and the discovery of coherence	44
Percussion and the historical challenge of the onset	45
Noise, ambience, and environmental recordings	45
Extreme duration and the change of listening scale	46
Preservation, reconstruction, and obsolete systems	47
A decade-by-decade change in the imagined user	47
Annotated chronology	48
Reading the history through software design	50
Source notes for the expanded history	50
What the historical work leaves us	51

1.11	What the transform sees	51
1.12	A small comparison	52
1.13	How pvx puts the idea to work	52
II	Orientation and Core Workflow	53
2	Getting Started with pvx	54
2.1	Quick Setup (Install + PATH)	54
2.2	Launch-Ready Helper Commands	54
2.3	Running Any Command with uv	55
2.4	First 3 Commands to Run	55
2.5	What Problem pvx Solves	55
2.6	Analog Tape Methods for Pitch/Time Shifting	56
2.7	Basic DSP Terms (Plain Language)	56
2.8	Supported File Types	56
2.9	Stretch vs Pitch Shift	56
2.10	Denoising with Phase-Consistent STFT Processing	57
2.11	Time-Varying Parameter Control (CSV/JSON)	57
2.12	Visual Mental Model of STFT	59
2.13	Example Audio Scenarios	59
	Use-Case Matrix (Where to start quickly)	60
2.14	Step-by-Step Walkthrough (One Small WAV)	60
	Step A: Baseline stretch	60
	Step B: Add transient protection	61
	Step C: Pitch shift with formant preservation	61
	Step D: Auto profile planning	61
2.15	What Outputs Should Sound Like	61
2.16	New Quality Modes (Beginner Version)	61
	Hybrid transient mode (recommended on speech/drums)	61
	Stereo coherence lock (recommended for wide stereo sources)	62
2.17	Intent Presets	62
2.18	Simpler One-Line Pipelines	62
2.19	Estimate Stretch Budget Before Extreme Jobs	62
2.20	Feature Tracking for Sidechain Control	63
2.21	Output Policy Controls	63
2.22	Next Steps	63
2.23	Extra Beginner Recipes (Quick Wins)	64
3	pvx Quality Guide	66
3.1	Fast Starting Presets	66
3.2	Artifact -> Fix Table	66
3.3	Practical Recipes	67
	Speech stretch with transient protection	67
	Drum-safe stretch	67
	Stereo coherence lock	67
	Denoise without musical-noise artifacts	67
3.4	Parameter Ranges That Usually Work	68
3.5	Debug Workflow	68
4	Supported Surface	69

4.1	Stable in 0.1.x	69
4.2	Beta in 0.1.x	69
4.3	Experimental in 0.1.x	69
4.4	Compatibility Shims	70
4.5	Docs Guide	70
5	Supported File Types	71
5.1	I/O Behavior by Category	71
5.2	Full Audio Container List (Current Build)	71
5.3	Verify on Your Machine	72
III	Theory and Internal Model	73
6	pvx Mathematical Foundations	74
6.1	Notation	74
6.2	STFT Analysis and Synthesis	74
6.3	Transform Backend Families and Tradeoffs	74
	Sample Transform Use Cases	76
6.4	Phase-Vocoder Frequency and Phase Update	76
6.5	Time Stretch and Pitch Ratio	76
6.6	Dynamic Control Interpolation (CSV/JSON)	76
6.7	Microtonal Retune Mapping	77
6.8	Loudness and Dynamics	77
6.9	Spatial Coherence and Channel Alignment	78
6.10	Function Graph Gallery	78
7	pvx Architecture	79
7.1	Runtime and CLI Flow	79
7.2	Algorithm Registry and Dispatch	79
7.3	Documentation Build Graph	79
7.4	CI + Pages	79
8	pvx Window Reference	80
8.1	Notation	80
8.2	Formula Families	80
	W0	80
	W1	80
	W2	80
	W3	81
	W4	81
	W5	81
	W6	81
	W7	81
	W8	81
	W9	82
	W10	82
	W11	82
	W12	82
	W13	82
	W14	82
	W15	82

	W16	83
	W17	83
	W18	83
8.3	Quantitative Metrics	83
8.4	Comparative Window Summary	83
8.5	Complete Window Atlas	87
	hann	87
	hamming	88
	blackman	89
	blackmanharris	90
	nuttall	91
	flattop	92
	blackman nuttall	93
	exact blackman	94
	sine	95
	bartlett	96
	boxcar	97
	triangular	98
	bartlett hann	99
	tukey	100
	tukey 0p1	101
	tukey 0p25	102
	tukey 0p75	103
	tukey 0p9	104
	parzen	105
	lanczos	106
	welch	107
	gaussian 0p25	108
	gaussian 0p35	109
	gaussian 0p45	110
	gaussian 0p55	111
	gaussian 0p65	112
	general gaussian 1p5 0p35	113
	general gaussian 2p0 0p35	114
	general gaussian 3p0 0p35	115
	general gaussian 4p0 0p35	116
	exponential 0p25	117
	exponential 0p5	118
	exponential 1p0	119
	cauchy 0p5	120
	cauchy 1p0	121
	cauchy 2p0	122
	cosine power 2	123
	cosine power 3	124
	cosine power 4	125
	hann poisson 0p5	126
	hann poisson 1p0	127
	hann poisson 2p0	128
	general hamming 0p50	129
	general hamming 0p60	130
	general hamming 0p70	131

	general hamming 0p80	132
	bohman	133
	cosine	134
	kaiser	135
	rect	136
9	Lessons from Paul Koonce’s PVC for pvx	137
9.1	PVC as a working environment	137
9.2	Lesson one: small commands can form a broad language	138
9.3	Lesson two: time-varying control belongs in the foundation	138
9.4	Lesson three: intermediate artifacts support musical memory	139
9.5	Lesson four: expose the processing choice that changes the sound	140
9.6	Lesson five: scripts are part of the instrument	140
9.7	Lesson six: stable releases need an experimental edge	140
9.8	What pvx should not copy	141
9.9	Three PVC-inspired laboratories	141
	Laboratory one: a reusable spectral imprint	141
	Laboratory two: control data as compositional material	141
	Laboratory three: spectral resonance and harmonic mapping	142
9.10	A continuing design checklist	142
10	PVC-to-pvx Parity Matrix	143
10.1	Scope	143
10.2	Capability Matrix	143
10.3	Implemented in Phase 1 and Phase 2	144
10.4	Schema Notes	144
	PVXAN	144
	PVXRF	144
10.5	Implemented in Phase 3, Phase 4, and Phase 5	144
10.6	Implemented in Phase 6 and Phase 7	145
10.7	Immediate Next Steps	145
	Sorted Top-Level Command Roadmap	145
IV	Workflow Cookbook	147
11	pvx Example Cookbook	148
11.1	Starter Path (Run These First)	148
11.2	Use-Case Index (By Theme)	148
11.3	Slow Down Speech	149
11.4	Time-Compress Speech (Faster Playback)	149
11.5	Raise Pitch Without Changing Speed	149
11.6	Lower Pitch With Formant Preservation	150
11.7	Stretch + Preserve Formants	150
11.8	Extreme Time Stretch Ambient Pad	150
11.9	Freeze a Harmonic Moment	150
11.10	Retune Vocal to A440	151
11.11	Morph Two Sounds	151
11.12	Create Chorus/Unison Thickness	152
11.13	De-Noise Field Recording	152
11.14	De-Reverb a Room Recording	153

11.15	Segment-Based Dynamic Stretch via CSV	153
11.16	Sidechain Pitch Trajectory (A Controls B)	153
11.17	GPU Acceleration	154
11.18	Batch Processing Folder	154
11.19	Pipe Multiple Tools in One Line	154
11.20	Auto Profile + Auto Transform (Planning)	155
11.21	Multi-Resolution Fusion Render	155
11.22	Checkpoint + Resume Long Render	155
11.23	Harmonize Triad Stack	156
11.24	Timeline Warp with pvxwarp	156
11.25	Layer Harmonic and Percussive Paths	156
11.26	JSON Manifest + Automation-Friendly Logging	157
11.27	Speech Natural Stretch (Hybrid Transients)	157
11.28	Drums Safe Stretch	157
11.29	Stereo Coherent Stretch	157
11.30	Extreme Ambient Stretch (10-minute target)	158
11.31	Pitch Shift with Formant Preservation	158
11.32	Hybrid Transient Mode (Manual Tuning)	158
11.33	Benchmark Command (pvx vs Rubber Band vs librosa)	159
11.34	Transient Reset Mode for Cleaner Attacks	159
11.35	WSOLA-Only Transient Handling	159
11.36	Mid/Side Stereo Coherence Stretch	160
11.37	Reference-Channel Lock for Multichannel Content	160
11.38	Irrational Pitch Ratio Input	160
11.39	Just-Ratio Pitch Input	160
11.40	N-TET CSV Map (19-TET) with pvxconform	161
11.41	Just-Intonation CSV Conform Map	161
11.42	Transform A/B: FFT vs DCT	161
11.43	CZT for Non-Power-of-Two Frame Setup	161
11.44	Full Mastering Chain in One Render	162
11.45	Safety Limiter + Hard Clip Guard	162
11.46	Plan-Only Diagnostic (No Render)	162
11.47	Checkpointed Long Render Workflow	163
11.48	Multi-Tool Pipe with Streaming I/O	163
11.49	Batch Tree Processing with find + xargs	163
11.50	Vocal Retune by Scale/Root (pvxretune)	164
11.51	Harmonize Then Widen in One Pipeline	164
11.52	Ambient Macro-Chain (Stretch + Cleanup + Loudness)	164
11.53	Broadcast-Style Speech Finishing (Stretch + Loudness + Safety)	165
11.54	Extreme Stretch with Checkpointed Recovery	165
11.55	Room Recording Rehab (De-Reverb Then Stretch)	165
11.56	Scale-Quantized Retune Followed by Unison Widening	166
11.57	Morph Then Freeze for Hybrid Pads	166
11.58	CUDA Attempt with Deterministic CPU Fallback Workflow	166
11.59	5.1/Multichannel Reference-Lock Stretch	166
11.60	Mid/Side Lock with Formant-Preserving Pitch Shift	167
11.61	Confidence-Gated External Pitch Control via Pipe	167
11.62	Microtonal Just-Ratio Map Playback	167

11.63	Speech Clarity Pass (Identity Lock + Conservative Window/Hop)	168
11.64	Transform Research A/B (FFT vs Hartley)	168
11.65	Multi-Resolution Fusion With Explicit Weights	168
11.66	Conservative Field-Restoration Chain	169
11.67	Reproducible Ambient Session With Manifest Logging	169
11.68	Managed One-Line Chain (No Manual Pipe Wiring)	169
11.69	Chunked Stream Wrapper + Output Policy Controls	169
11.70	Dynamic Stretch via Direct CSV Flag Input	170
11.71	Dynamic Pitch Ratio via JSON + Polynomial Interpolation	170
11.72	Stairstep (Sample-and-Hold) Control Mode	171
11.73	Dynamic Analysis Resolution (N-FFT + Hop Size Maps)	172
11.74	Stateful Stream Mode with Dynamic Stretch CSV	172
11.75	Generate a Stretch Envelope (Function Stream)	173
11.76	Reshape and Densify a Control Map	173
11.77	Run PVC Parity Benchmark Gate	174
11.78	Persist STFT Analysis Artifact (PVXAN)	174
11.79	Reusable Response Artifact (PVXRF)	175
11.80	Response-Profile Denoising with <code>pvx noisefilter</code>	175
11.81	Time-Varying Spectral Filtering with <code>pvx tvfilter</code>	175
11.82	Spectral Peak Emphasis with <code>pvx bandamp</code>	176
11.83	Response-Shaped Spectral Dynamics with <code>pvx spec-compander</code>	176
11.84	Ring Modulation Fundamentals with <code>pvx ring</code>	177
11.85	Resonant Ring Filtering with <code>pvx ringfilter</code>	177
11.86	Harmonic Chord Mapping with <code>pvx chordmapper</code>	177
11.87	Inharmonic Spectral Warping with <code>pvx inharmonator</code>	178
11.88	Diagnose Environment and Install Issues (<code>pvx doctor</code>)	178
11.89	Minimal Public-Demo Sequence (<code>pvx quickstart</code>)	178
11.90	Quality-First First Pass (<code>pvx safe</code>)	179
11.91	Transform Availability and Selection (<code>pvx transforms</code>)	179
11.92	Fast End-to-End Health Check (<code>pvx smoke</code>)	179
11.93	Deterministic AI Dataset Augmentation (<code>pvx augment</code>)	180
11.94	Speaker-Balanced Split Assignment for Augmentation	180
11.95	Contrastive Paired Views for Self-Supervised Learning	180
11.96	Resume + Append for Long Augmentation Runs	181
11.97	Manifest Validation and Merge	181
11.98	Augmentation Benchmark Gate	181
12	Advanced Session Recipes	183
12.1	Restoration and delivery	183
	Recipe 1: dialogue reconstruction from matched room tone	183
	Recipe 2: archival speech with two-stage timing repair	183
	Recipe 3: field recording preservation with multiresolution stretch	184
	Recipe 4: coherent multichannel restoration	184
12.2	Automation as composition	184
	Recipe 5: exponential expansion with a smoothed alternate take	184
	Recipe 6: polynomial pitch arc with formant protection	185
	Recipe 7: rhythmic sample-and-hold time folding	185
	Recipe 8: section-adaptive analysis resolution	186
12.3	Sidechain and feature routing	186

	Recipe 9: inverse melodic breathing	186
	Recipe 10: MFCC pitch with flux-driven time	186
	Recipe 11: transient-protected feature following	187
	Recipe 12: two guides with separate musical roles	187
12.4	Persistent spectral artifacts	188
	Recipe 13: reusable timbral imprint with an automated entrance	188
	Recipe 14: response-informed noise removal before extreme stretch	188
	Recipe 15: morph, freeze, and finish a hybrid pad	188
	Recipe 16: response-shaped dynamics and resonant afterimage	189
12.5	Harmonic, inharmonic, and spatial design	189
	Recipe 17: retuned harmony choir with controlled width	189
	Recipe 18: chord-constrained resonant cloud	190
	Recipe 19: inharmonic bell transformed into a frozen bed	190
	Recipe 20: stereo vocal transposition and harmony with image control	190
12.6	Long renders, comparison, and reproducibility	191
	Recipe 21: four-hour resumable installation render	191
	Recipe 22: deterministic delivery and null-test pair	191
	Recipe 23: matched transform and resolution suite	191
	Recipe 24: complex session with plan, render, and regression gate	192
12.7	Adapting the recipes	192
13	Feature-Driven Sidechain Examples	193
13.1	Quick Start	193
13.2	Inspect Feature Columns Before Routing	193
13.3	Single-Feature Recipes	194
	<code>f0_hz</code> -> pitch ratio	194
	<code>pitch_ratio</code> -> stretch (inverse motion)	194
	<code>confidence</code> -> subtle pitch bend depth	194
	<code>voicing_prob</code> -> stretch	194
	<code>rms_norm</code> -> stretch	194
	<code>spectral_flux</code> -> stretch	195
	<code>onset_strength</code> -> attack-reactive stretch	195
	<code>transientness</code> -> transient-safe stretch variation	195
	<code>spectral_centroid_hz</code> -> brightness-linked pitch	195
	<code>spectral_flatness</code> -> noisy/tonal pitch response	195
	<code>harmonic_ratio</code> -> harmonicity-linked pitch	195
	<code>inharmonicity</code> -> anti-harmonic pitch detune	196
	<code>rolloff_norm</code> -> pitch	196
	<code>pitch_stability</code> -> micro-variation amount	196
13.4	Multi-Feature Recipes (Different Features -> Different Targets)	196
	MFCC + MPEG-7 spectral flux	196
	Formant + onset	196
	Beat phase + downbeat phase	197
	Tempo + centroid	197
	Stereo cues: interaural level difference + interaural time difference	197
	Noise-aware: hiss + hum	197
	Speech vs music probabilities	197
	Formant 2 + spectral spread	198
	Loudness proxy + transient mask	198
	MPEG-7 centroid + MPEG-7 spread	198
	MPEG-7 attack-time + MFCC driver	198

13.5	Route Operator Patterns (<code>const</code> , <code>inv</code> , <code>pow</code> , <code>mul</code> , <code>add</code> , <code>affine</code> , <code>clip</code>)	198
	Start from a feature, then scale and bias	199
	Exaggerate movement with power law	199
	Invert and clamp	199
13.6	Feature-Vector Recipes (MFCC + MPEG-7)	199
	MFCC second coefficient driver	199
	MFCC + MPEG-7 audio spectrum envelope band	199
	MPEG-7 spectral crest and decrease	200
	Increase MFCC dimensionality to 20	200
	Build custom vector features, then route them	200
13.7	Multiple Guide Files (A and B both influence target)	201
	Blend pitch behavior from two guides	201
	Guide A controls pitch, guide B controls stretch	201
13.8	Manual Pipe Variants	202
13.9	Reuse a Saved Feature Map for Fast Iteration	202
13.10	Compact Feature Set (Smaller CSV)	202
13.11	Safe Starting Ranges	203
14	Follow Workflow Migration	204
14.1	Why This Change	204
14.2	Old vs New	204
14.3	When to Keep Manual Pipes	204
14.4	Rollout Guidance	204
15	100 Crazy Things To Try With pvx	205
15.1	Extreme Time-Scale Madness	205
15.2	Freeze + Morph Experiments	206
15.3	Pitch, Harmony, and Tuning Extremes	207
15.4	Transients, Stereo, and Layer Split	207
15.5	Sidechain Follow and Feature-Driven Control	208
15.6	Control-Map Authoring With Envelope + Reshape	209
15.7	Analysis/Response Artifact and Spectral Operator Ideas	210
15.8	Ring, Resonator, Chordmapper, Inharmonator	211
15.9	Multi-Stage Pipelines, Streaming, and Batch Tricks	212
15.10	QA, Stress, and Reproducibility Challenges	213
V	Python, ML, and Dataset Work	215
16	pvx Application Programming Interface (API) Overview	216
16.1	Import Paths	216
16.2	Minimal Time-Stretch Example	216
16.3	Minimal Pitch-Shift (Duration-Preserving)	216
16.4	Jupyter Notebook Snippets	217
	Spectrogram visualization snippet	217
	Compare phase locking modes	218
16.5	Batch Processing Example (Python)	218
16.6	Segment Processing Example (Python)	218
16.7	Related CLI Equivalents	219
17	ML/DL Framework Integration Guide	220
17.1	Contents	220

17.2	Installation	220
17.3	Quick Start , Pure Python	221
17.4	Intent Preset Pipelines	221
17.5	PyTorch Integration	222
	PvxAugmentDataset	222
	PvxAugmentTransform (torchvision-style)	222
	AudioCollator for variable-length batches	223
17.6	HuggingFace Datasets Integration	223
	Basic map() usage	223
	Contrastive view generation (SSL)	224
	One-liner convenience	224
	Saving augmented datasets	224
17.7	TensorFlow / tf.data Integration	225
	Tensor map function	225
	Dict-based map for keyed datasets	225
17.8	Determinism and Reproducibility	225
17.9	Performance Tips	226
	Offline pre-computation (recommended for large datasets)	226
	Online augmentation with worker processes	226
	Caching expensive transforms	226
	TimeStretch and PitchShift , Engine Selection	226
	Streaming Augmentation for Long-Form Audio	227
	Impulse Response Database	228
	Mix online and offline augmentation	228
17.10	GPU-Accelerated Transforms (PyTorch)	229
	Mixing GPU and NumPy transforms	229
	Available GPU transforms	229
17.11	Available Transforms	230
18	AI Research and Data Augmentation with pvx	232
18.1	Core Command	232
18.2	Intent Profiles	232
18.3	Reproducibility Controls	232
18.4	Manifest Fields	233
18.5	Example Workflows	234
	Speech Robustness Set	234
	Music Information Retrieval Set	234
	Contrastive Planning-Only Pass	234
	With explicit manifest paths	234
	Resume and append	235
	Manifest merge + stats	235
18.6	Policy File Template	235
18.7	Why Augmentation Works	236
18.8	Invariance, Equivariance, and Invalid Transformations	236
18.9	Designing the Augmentation Distribution	237
18.10	Transform Order Is Part of the Model	237
18.11	Phase-Vocoder Augmentation as a Special Case	238
18.12	Severity, Coverage, and Curriculum	239
18.13	Determinism and Seed Architecture	239
18.14	Split Hygiene and Family Leakage	240

18.15	Online, Offline, and Hybrid Augmentation	241
18.16	Contrastive and Self-Supervised Views	241
18.17	Paired and Structured Targets	242
18.18	Task-Specific Policy Reasoning	242
	Automatic speech recognition	242
	Speaker and paralinguistic modeling	242
	Music information retrieval	243
	Pitch and melody estimation	243
	Enhancement and source separation	243
	Generative and neural-vocoder training	243
18.19	Class Imbalance and Conditional Policies	243
18.20	Ablation Studies	243
18.21	Evaluation Beyond One Aggregate Score	244
18.22	Detecting Shortcuts and Augmentation Artifacts	245
18.23	Quality Control Before Training	245
18.24	A Reproducible Experiment Layout	245
18.25	Complex Policy Example: Robust Speech with Auditable Views	246
18.26	Complex Policy Example: Equivariant Music Labels	247
18.27	Fairness, Consent, and Dataset Governance	247
18.28	Publication and Handoff Checklist	247
18.29	Research Questions Worth Testing	248
18.30	Benchmarking Augmentation Pipelines	248
18.31	Running with uv	248
18.32	Research Notes	248
19	Audio Augmentation Cookbook	250
19.1	Contents	250
19.2	ASR , Automatic Speech Recognition	250
19.3	Keyword Spotting / Wake-Word Detection	251
19.4	Speaker Verification / Identification	252
19.5	Speech Enhancement and Denoising	252
19.6	Music Classification and Tagging	253
19.7	Beat Tracking and Tempo Estimation	253
19.8	Pitch Estimation and Melody Extraction	254
19.9	Audio Event Detection / Sound Classification	255
19.10	Self-Supervised / Contrastive Learning	255
19.11	Music Source Separation	256
19.12	Augmentation Intensity Guide	256
20	pvx Pipeline Cookbook	257
20.1	Analysis/QA	257
	Quality metrics on processed speech	257
20.2	Automation	257
	A/B report generation	257
	Benchmark matrix sweep	257
	Quality regression check	257
20.3	Batch	258
	Batch stretch over folder	258
	Dry-run output validation	258
20.4	Mastering	258

	Integrated loudness targeting with limiter	258
	Soft clip and hard safety ceiling	258
20.5	Microtonal	258
	Custom cents map retune	259
	Conform CSV with per-segment ratios	259
20.6	Phase-vocoder core	259
	Moderate vocal stretch with formant preservation	259
	Independent cents retune	259
	Extreme stretch with multistage strategy	259
	Ultra-smooth speech stretch (600x)	259
	Auto-profile plan preview	260
	Multi-resolution fusion stretch	260
	Checkpointed long render with manifest	260
20.7	Pipelines	260
	Time-stretch -> denoise -> dereverb in one pipe	260
	Morph -> formant -> unison	260
	Pitch-follow sidechain map (A controls B)	261
20.8	Spatial	261
	VBAP adaptive panning via algorithm dispatcher	261
20.9	Transform selection	261
	Default production backend (FFT + transient protection)	261
	Reference Fourier baseline using explicit DFT mode	261
	Prime-size frame experiment with CZT backend	261
	DCT timbral compaction for smooth harmonic material	262
	DST odd-symmetry color pass	262
	Hartley real-basis exploratory render	262
	A/B sweep of transform backends from shell loop	262
20.10	Notes	262
VI	Operations and Delivery	263
21	pvx Benchmarks	264
21.1	Reproduce	264
21.2	Benchmark Spec	264
21.3	Host	264
21.4	Backend Summary	264
21.5	Per-Signal Results	265
22	Installing and Managing pvx with Homebrew	267
22.1	Stable Installation	267
22.2	Development Installation	267
22.3	Upgrading	268
22.4	Uninstalling	268
22.5	What the Formula Installs	268
22.6	Shell Path	269
22.7	Troubleshooting	269
22.8	Maintainer Release Flow	270
22.9	Formula Version Policy	270
23	Alpha Release Guide	271
23.1	Supported Surface	271

23.2	Release Checklist	271
23.3	Alpha Principles	271
A	pvx Command-Line Interface (CLI) Flags Reference	272
A.1	Unique Long Flags	272
A.2	hps-pitch-track	273
A.3	pvx	274
A.4	pvxanalysis	279
A.5	pvxcodec	279
A.6	pvxconform	280
A.7	pvxdenoise	280
A.8	pvxdeverb	280
A.9	pvxenvelope	280
A.10	pvxfilter	282
A.11	pvxformant	283
A.12	pvxfreeze	283
A.13	pvxgain	283
A.14	pvxharmmap	284
A.15	pvxharmonize	284
A.16	pvxlayer	285
A.17	pvxmorph	285
A.18	pvxnoise	286
A.19	pvxreshape	286
A.20	pvxresponse	287
A.21	pvxretune	288
A.22	pvxring	288
A.23	pvxrir	289
A.24	pvxspecaugment	290
A.25	pvxtrajectoryreverb	290
A.26	pvxtransient	291
A.27	pvxunison	291
A.28	pvxvoc	292
A.29	pvxwarp	299
B	pvx Algorithm Limitations and Applicability	300
B.1	Group-Level Summary	300
B.2	analysis_qa_and_automation	301
B.3	creative_spectral_effects	302
B.4	denoise_and_restoration	303
B.5	dereverb_and_room_correction	304
B.6	dynamics_and_loudness	305
B.7	granular_and_modulation	306
B.8	pitch_detection_and_tracking	307
B.9	retune_and_intonation	308
B.10	separation_and_decomposition	309
B.11	spatial_and_multichannel	310
B.12	spectral_time_frequency_transforms	313
B.13	time_scale_and_pitch_core	314
C	pvx Diagram Atlas	315

C.1	End-to-End pvxvoc Flow	315
C.2	STFT Analysis / Synthesis Timeline	315
C.3	Phase Propagation and Locking	315
C.4	Hybrid Transient Engine	315
C.5	Transient Feature Stack	316
C.6	WSOLA Similarity Search	316
C.7	Stereo / Multichannel Coherence Modes	316
C.8	Microtonal Map Processing	316
C.9	Control-Rate Interpolation Graph Examples	316
C.10	Function Graph Gallery	317
C.11	Extreme Stretch: Multistage Strategy	317
C.12	Transform Backend Decision Path	317
C.13	Mastering Chain Block Diagram	317
C.14	Metrics Printing Path	317
C.15	Benchmark Runner + CI Gate	318
C.16	Pipe Chaining Pattern	318
C.17	Checkpoint / Resume State Machine	318
C.18	Troubleshooting Decision Tree	318
C.19	Data Product Map	318
C.20	Window/Overlap Tradeoff Map	318
C.21	Phase Reset vs Hybrid Boundary Behavior	318
C.22	Multi-Resolution Fusion Fan-In	319
C.23	Pitch-Map Confidence Gate	319
C.24	Microtonal CSV Interpolation Flow	319
C.25	Audio Metrics Table Generation	319
C.26	Quality-First Optimization Loop	319
C.27	Benchmark Metric Taxonomy	319
C.28	Documentation Build Pipeline	320
D	pvx Phasiness Implementation Plan	321
D.1	Why this paper matters	321
D.2	Benefits we can extract for pvx	321
D.3	Phased implementation	321
	Phase 1: Coherence-aware phase propagation in core	321
	Phase 2: Adaptive coherence policy	321
	Phase 3: Stereo/multichannel coherence extension	322
	Phase 4: Benchmark and regression gates	322
	Phase 5: UX and docs	322
D.4	Validation targets	322
D.5	Source paper and cited references to track	322
E	pvx Citation Quality Report	323
E.1	Link-Type Summary	323
E.2	Entries Still Using Scholar Links	323
F	PVC Phase 3-5 Architecture	328
F.1	Module Layout	328
F.2	Processing Contracts	328
F.3	Phase 3 Design	328
F.4	Phase 4 Design	329

F.5	Phase 5 Design	329
F.6	Backward Compatibility and UX	329
G	A Phase-Vocoder Listening Library: 100 Musical Works	330
G.1	How to use the library	330
G.2	Documented phase-vocoder practice	330
	1. Trevor Wishart: <i>Vox 5</i>	330
	2. Roger Reynolds: <i>Transfigured Wind I</i>	331
	3. JoAnn Kuchera-Morin: <i>Dreampaths</i>	331
	4. Jonathan Harvey: <i>Mortuos Plango, Vivos Voco</i>	331
	5. Trevor Wishart: <i>Supernova</i>	331
	6. Trevor Wishart: <i>Vox Cycle</i>	331
G.3	Spectral-composition context	331
	7. Trevor Wishart: <i>Red Bird</i>	331
	8. Trevor Wishart: <i>Tongues of Fire</i>	332
	9. Tristan Murail: <i>Desintegrations</i>	332
	10. Wendy Carlos: <i>Digital Moonscapes</i>	332
	11. Jean-Claude Risset: <i>Inharmonique</i>	332
	12. Jean-Claude Risset: <i>Songes</i>	332
	13. Kaija Saariaho: <i>Lichtbogen</i>	332
	14. Kaija Saariaho: <i>NoaNoa</i>	332
	15. Paul Lansky: <i>Six Fantasies on a Poem by Thomas Campion</i>	333
G.4	Voice, text, and formants	333
	16. Hildegard von Bingen: <i>O vis aeternitatis</i>	333
	17. Anonymous: <i>Dies irae</i>	333
	18. Josquin des Prez: <i>Ave Maria virgo serena</i>	333
	19. Thomas Tallis: <i>Spem in alium</i>	333
	20. Claudio Monteverdi: <i>Lamento d'Arianna</i>	333
	21. Henry Purcell: <i>Dido's Lament</i>	333
	22. Johann Sebastian Bach: <i>Erbarme dich</i>	334
	23. George Frideric Handel: <i>Lascia ch'io pianga</i>	334
	24. Wolfgang Amadeus Mozart: <i>Der Holle Rache</i>	334
	25. Franz Schubert: <i>Erlkonig</i>	334
	26. Richard Wagner: <i>Mild und leise</i>	334
	27. Gustav Mahler: <i>Ich bin der Welt abhanden gekommen</i>	334
	28. Arnold Schoenberg: <i>Mondestrunken</i>	334
	29. Luciano Berio: <i>Sequenza III</i>	334
	30. Steve Reich: <i>Different Trains</i>	335
G.5	Sustained tone and resonance	335
	31. Antonio Vivaldi: <i>Winter, Largo</i>	335
	32. Johann Sebastian Bach: <i>Air on the G String</i>	335
	33. Ludwig van Beethoven: <i>Symphony No. 7, Allegretto</i>	335
	34. Franz Schubert: <i>String Quintet in C, Adagio</i>	335
	35. Frederic Chopin: <i>Nocturne in E-flat major, Op. 9 No. 2</i>	335
	36. Richard Wagner: <i>Prelude to Das Rheingold</i>	335
	37. Anton Bruckner: <i>Symphony No. 7, Adagio</i>	336
	38. Claude Debussy: <i>Prelude a l'apres-midi d'un faune</i>	336
	39. Maurice Ravel: <i>Daphnis et Chloe, Lever du jour</i>	336
	40. Ralph Vaughan Williams: <i>Fantasia on a Theme by Thomas Tallis</i>	336
	41. Gustav Holst: <i>Neptune, the Mystic</i>	336
	42. Olivier Messiaen: <i>Louange a l'Eternite de Jesus</i>	336
	43. Gyorgy Ligeti: <i>Lux Aeterna</i>	336

	44. Arvo Part: <i>Spiegel im Spiegel</i>	336
	45. John Tavener: <i>The Protecting Veil</i>	337
G.6	Transient and rhythmic stress tests	337
	46. Johann Sebastian Bach: <i>Prelude in C major, BWV 846</i>	337
	47. Ludwig van Beethoven: <i>Piano Sonata No. 14, third movement</i>	337
	48. Niccolò Paganini: <i>Caprice No. 24</i>	337
	49. Frederic Chopin: <i>Etude Op. 10 No. 4</i>	337
	50. Nikolai Rimsky-Korsakov: <i>Flight of the Bumblebee</i>	337
	51. Igor Stravinsky: <i>The Rite of Spring, Augurs of Spring</i>	337
	52. Edgard Varese: <i>Ionisation</i>	338
	53. John Cage: <i>First Construction in Metal</i>	338
	54. Dave Brubeck Quartet: <i>Take Five</i>	338
	55. James Brown: <i>Funky Drummer</i>	338
	56. Kraftwerk: <i>Numbers</i>	338
	57. Public Enemy: <i>Bring the Noise</i>	338
	58. Aphex Twin: <i>Vordhosbn</i>	338
	59. Squarepusher: <i>My Red Hot Car</i>	338
	60. Autechre: <i>Gantz Graf</i>	339
G.7	Polyphony and orchestral density	339
	61. Giovanni Pierluigi da Palestrina: <i>Missa Papae Marcelli, Kyrie</i>	339
	62. Johann Sebastian Bach: <i>The Art of Fugue, Contrapunctus I</i>	339
	63. Wolfgang Amadeus Mozart: <i>Symphony No. 40, first movement</i>	339
	64. Ludwig van Beethoven: <i>Symphony No. 9, finale</i>	339
	65. Hector Berlioz: <i>Symphonie fantastique, Marche au supplice</i>	339
	66. Johannes Brahms: <i>Symphony No. 4, finale</i>	339
	67. Gustav Mahler: <i>Symphony No. 2, finale</i>	340
	68. Claude Debussy: <i>La mer, Dialogue du vent et de la mer</i>	340
	69. Charles Ives: <i>The Unanswered Question</i>	340
	70. Igor Stravinsky: <i>The Firebird, Infernal Dance</i>	340
	71. Bela Bartok: <i>Music for Strings, Percussion and Celesta, third movement</i>	340
	72. Benjamin Britten: <i>The Young Person's Guide to the Orchestra</i>	340
	73. Iannis Xenakis: <i>Metastaseis</i>	340
	74. György Ligeti: <i>Atmospheres</i>	340
	75. John Adams: <i>Short Ride in a Fast Machine</i>	341
G.8	Electronic, studio, and ambient listening	341
	76. Karlheinz Stockhausen: <i>Gesang der Junglinge</i>	341
	77. Karlheinz Stockhausen: <i>Kontakte</i>	341
	78. Delia Derbyshire: <i>Doctor Who Theme</i>	341
	79. The Beatles: <i>Tomorrow Never Knows</i>	341
	80. Pink Floyd: <i>On the Run</i>	341
	81. Brian Eno: <i>An Ending (Ascent)</i>	341
	82. Laurie Anderson: <i>O Superman</i>	342
	83. Kate Bush: <i>Running Up That Hill</i>	342
	84. Cocteau Twins: <i>Lorelei</i>	342
	85. My Bloody Valentine: <i>Only Shallow</i>	342
	86. The Orb: <i>Little Fluffy Clouds</i>	342
	87. Boards of Canada: <i>Roygbiv</i>	342
	88. Björk: <i>All Is Full of Love</i>	342
	89. Radiohead: <i>Everything in Its Right Place</i>	342
	90. Oneohtrix Point Never: <i>Replica</i>	343
G.9	Extreme duration and temporal form	343

91. Johann Pachelbel: <i>Canon in D</i>	343
92. Ludwig van Beethoven: <i>Symphony No. 5, opening</i>	343
93. Richard Wagner: <i>Prelude to Tristan und Isolde</i>	343
94. Maurice Ravel: <i>Bolero</i>	343
95. Samuel Barber: <i>Adagio for Strings</i>	343
96. John Cage: <i>Organ2/ASLSP</i>	343
97. Steve Reich: <i>Piano Phase</i>	344
98. Alvin Lucier: <i>I Am Sitting in a Room</i>	344
99. William Basinski: <i>The Disintegration Loops 1.1</i>	344
100. Max Richter: <i>Sleep</i>	344
G.10 Comparative listening worksheet	344
H Transfer-Function and Automation Graph Atlas	345
H.1 Transfer functions	345
pitch ratio from semitones	345
pitch ratio from cents	346
dynamics transfer curves	346
soft-clip transfer functions	347
morph blend magnitude curves	347
mask exponent response	348
phase-mix angle response	348
H.2 Automation and interpolation	348
sample and hold	349
nearest neighbour	349
linear	349
cubic	350
exponential	350
smoothstep S-curve	350
smootherstep	351
polynomial order 1	351
polynomial order 2	351
polynomial order 3	352
polynomial order 5	352
Glossary	353
Bibliography	362
Open Media Credits	367

List of Figures

1.1	The phase-vocoder loop. Each output frame is rebuilt from a spectrum whose phase has been advanced on a new time grid.	3
1.2	Time stretching changes the distance between reconstructed frames while preserving their spectral content.	3
1.3	WOLA analysis framing. The waveform remains continuous, while overlapping windows select smoothly weighted local views.	4
1.4	The STFT lattice. Each cell $X_t[k]$ contains both magnitude and phase, even though a spectrogram normally displays only magnitude.	5
1.5	Phase advance as a frequency measurement. The residual between predicted and observed rotation estimates the offset from the nominal bin center.	6
1.6	A complete phase-vocoder frame path. Magnitude follows the current analysis frame, while frequency estimates drive a new phase trajectory on the synthesis timeline.	6
1.7	A two-times stretch. The same sequence of analyzed states is reconstructed at twice the frame spacing.	7
1.8	Weighted overlap-add reconstruction. Individual frame weights rise and fall, while their normalized sum remains level through the well-covered region.	7
1.9	Vertical phase coherence. Phase locking treats the bins around a spectral peak as a related group rather than as unrelated oscillators.	8
1.10	A Voder demonstration at the 1939 New York World's Fair, reproduced from Homer Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	9
1.11	Dudley's 1940 block diagram of the voice mechanism. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	9
1.12	A historical vocoder schematic from Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	10
1.13	Dudley's companion Voder schematic, showing the related synthesis-oriented system. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.	11
1.14	A selected chronology of representations and implementation ideas leading to modern phase-vocoder practice.	12
1.15	The FFT-oriented analysis and synthesis loop associated with the practical digital phase vocoder.	12
1.16	A conceptual parameter trajectory, the representation that makes time remapping and spectral editing intelligible.	14
1.17	Horizontal and vertical phase relationships in the STFT lattice.	14
1.18	Identity phase locking assigns neighbouring bins to spectral peaks and preserves their observed relative phases.	15
1.19	A conceptual spectral translation used for pitch shifting and harmonisation.	16
1.20	A transient can occupy only a small part of a frame, making onset preservation a separate design problem.	16
1.21	The phase vocoder grew from a loop of mathematical, acoustical, communications, and musical practices rather than a single linear ancestry.	18

1.22	Operating principle of a Springer-type rotating-head regulator. The drawing is schematic: transport velocity and head rotation provide two mechanical degrees of freedom that ordinary fixed-head varispeed does not have.	21
1.23	Anton Marian Springer's rotating multipolar transducer, left, and its geared tape-playback application, right. Figures 4 through 6 from United States Patent 3,064,088, filed in 1959 and issued in 1962. United States patent drawings, public domain.	21
1.24	Phase-vocoder, sinusoidal-modeling, and residual-processing lineages increasingly converged in hybrid systems.	27
1.25	Successive research eras enlarged the set of sound properties represented explicitly. Earlier questions survived inside later systems rather than being discarded.	34
1.26	The shrinking audition loop altered compositional method as profoundly as increases in nominal signal quality.	35
1.27	A conservative software genealogy. Solid lines indicate broad institutional or interface continuities; dashed lines indicate contemporary exchange rather than direct descent.	38
1.28	Musical technology develops through feedback among models, implementations, artistic practice, and learned perception.	40
1.29	Institutions contributed different working emphases. The diagram records exchange and convergence, not exclusive ownership of ideas.	44
1.30	Extreme stretching changes which temporal layer behaves as musical form. The boundaries are material-dependent rather than fixed ratio thresholds.	46
1.31	Automation treats a parameter as a continuous curve across the render, not as a sequence of unrelated settings.	51
1.32	A conceptual window response. Window choice balances temporal detail against frequency selectivity.	52
8.1	Time-domain shape of the hann window.	87
8.2	Magnitude spectrum of the hann window.	87
8.3	Time-domain shape of the hamming window.	88
8.4	Magnitude spectrum of the hamming window.	88
8.5	Time-domain shape of the blackman window.	89
8.6	Magnitude spectrum of the blackman window.	89
8.7	Time-domain shape of the blackmanharris window.	90
8.8	Magnitude spectrum of the blackmanharris window.	90
8.9	Time-domain shape of the nuttall window.	91
8.10	Magnitude spectrum of the nuttall window.	91
8.11	Time-domain shape of the flattop window.	92
8.12	Magnitude spectrum of the flattop window.	92
8.13	Time-domain shape of the blackman nuttall window.	93
8.14	Magnitude spectrum of the blackman nuttall window.	93
8.15	Time-domain shape of the exact blackman window.	94
8.16	Magnitude spectrum of the exact blackman window.	94
8.17	Time-domain shape of the sine window.	95
8.18	Magnitude spectrum of the sine window.	95
8.19	Time-domain shape of the bartlett window.	96
8.20	Magnitude spectrum of the bartlett window.	96
8.21	Time-domain shape of the boxcar window.	97
8.22	Magnitude spectrum of the boxcar window.	97
8.23	Time-domain shape of the triangular window.	98
8.24	Magnitude spectrum of the triangular window.	98
8.25	Time-domain shape of the bartlett hann window.	99
8.26	Magnitude spectrum of the bartlett hann window.	99
8.27	Time-domain shape of the tukey window.	100

8.28	Magnitude spectrum of the tukey window.	100
8.29	Time-domain shape of the tukey 0p1 window.	101
8.30	Magnitude spectrum of the tukey 0p1 window.	101
8.31	Time-domain shape of the tukey 0p25 window.	102
8.32	Magnitude spectrum of the tukey 0p25 window.	102
8.33	Time-domain shape of the tukey 0p75 window.	103
8.34	Magnitude spectrum of the tukey 0p75 window.	103
8.35	Time-domain shape of the tukey 0p9 window.	104
8.36	Magnitude spectrum of the tukey 0p9 window.	104
8.37	Time-domain shape of the parzen window.	105
8.38	Magnitude spectrum of the parzen window.	105
8.39	Time-domain shape of the lanczos window.	106
8.40	Magnitude spectrum of the lanczos window.	106
8.41	Time-domain shape of the welch window.	107
8.42	Magnitude spectrum of the welch window.	107
8.43	Time-domain shape of the gaussian 0p25 window.	108
8.44	Magnitude spectrum of the gaussian 0p25 window.	108
8.45	Time-domain shape of the gaussian 0p35 window.	109
8.46	Magnitude spectrum of the gaussian 0p35 window.	109
8.47	Time-domain shape of the gaussian 0p45 window.	110
8.48	Magnitude spectrum of the gaussian 0p45 window.	110
8.49	Time-domain shape of the gaussian 0p55 window.	111
8.50	Magnitude spectrum of the gaussian 0p55 window.	111
8.51	Time-domain shape of the gaussian 0p65 window.	112
8.52	Magnitude spectrum of the gaussian 0p65 window.	112
8.53	Time-domain shape of the general gaussian 1p5 0p35 window.	113
8.54	Magnitude spectrum of the general gaussian 1p5 0p35 window.	113
8.55	Time-domain shape of the general gaussian 2p0 0p35 window.	114
8.56	Magnitude spectrum of the general gaussian 2p0 0p35 window.	114
8.57	Time-domain shape of the general gaussian 3p0 0p35 window.	115
8.58	Magnitude spectrum of the general gaussian 3p0 0p35 window.	115
8.59	Time-domain shape of the general gaussian 4p0 0p35 window.	116
8.60	Magnitude spectrum of the general gaussian 4p0 0p35 window.	116
8.61	Time-domain shape of the exponential 0p25 window.	117
8.62	Magnitude spectrum of the exponential 0p25 window.	117
8.63	Time-domain shape of the exponential 0p5 window.	118
8.64	Magnitude spectrum of the exponential 0p5 window.	118
8.65	Time-domain shape of the exponential 1p0 window.	119
8.66	Magnitude spectrum of the exponential 1p0 window.	119
8.67	Time-domain shape of the cauchy 0p5 window.	120
8.68	Magnitude spectrum of the cauchy 0p5 window.	120
8.69	Time-domain shape of the cauchy 1p0 window.	121
8.70	Magnitude spectrum of the cauchy 1p0 window.	121
8.71	Time-domain shape of the cauchy 2p0 window.	122
8.72	Magnitude spectrum of the cauchy 2p0 window.	122
8.73	Time-domain shape of the cosine power 2 window.	123
8.74	Magnitude spectrum of the cosine power 2 window.	123
8.75	Time-domain shape of the cosine power 3 window.	124
8.76	Magnitude spectrum of the cosine power 3 window.	124
8.77	Time-domain shape of the cosine power 4 window.	125
8.78	Magnitude spectrum of the cosine power 4 window.	125

8.79	Time-domain shape of the hann poisson 0p5 window.	126
8.80	Magnitude spectrum of the hann poisson 0p5 window.	126
8.81	Time-domain shape of the hann poisson 1p0 window.	127
8.82	Magnitude spectrum of the hann poisson 1p0 window.	127
8.83	Time-domain shape of the hann poisson 2p0 window.	128
8.84	Magnitude spectrum of the hann poisson 2p0 window.	128
8.85	Time-domain shape of the general hamming 0p50 window.	129
8.86	Magnitude spectrum of the general hamming 0p50 window.	129
8.87	Time-domain shape of the general hamming 0p60 window.	130
8.88	Magnitude spectrum of the general hamming 0p60 window.	130
8.89	Time-domain shape of the general hamming 0p70 window.	131
8.90	Magnitude spectrum of the general hamming 0p70 window.	131
8.91	Time-domain shape of the general hamming 0p80 window.	132
8.92	Magnitude spectrum of the general hamming 0p80 window.	132
8.93	Time-domain shape of the bohman window.	133
8.94	Magnitude spectrum of the bohman window.	133
8.95	Time-domain shape of the cosine window.	134
8.96	Magnitude spectrum of the cosine window.	134
8.97	Time-domain shape of the kaiser window.	135
8.98	Magnitude spectrum of the kaiser window.	135
8.99	Time-domain shape of the rect window.	136
8.100	Magnitude spectrum of the rect window.	136
18.1	Two policy-design concepts. The left graph motivates a measured severity search. The right graph shows one possible curriculum that widens the allowed range during training.	239
H.1	Pitch ratio from semitones.	345
H.2	Pitch ratio from cents.	346
H.3	Dynamics transfer curves.	346
H.4	Soft-clip transfer functions.	347
H.5	Morph blend magnitude curves.	347
H.6	Mask exponent response.	348
H.7	Phase-mix angle response.	348
H.8	Sample and hold interpolation.	349
H.9	Nearest neighbour interpolation.	349
H.10	Linear interpolation.	349
H.11	Cubic interpolation.	350
H.12	Exponential interpolation.	350
H.13	Smoothstep s-curve interpolation.	350
H.14	Smoootherstep interpolation.	351
H.15	Polynomial order 1 interpolation.	351
H.16	Polynomial order 2 interpolation.	351
H.17	Polynomial order 3 interpolation.	352
H.18	Polynomial order 5 interpolation.	352

List of Tables

1	Symbols and notation used in this guide.	vii
1.1	The practical differences between a conventional channel vocoder and a phase vocoder.	2
1.2	Mechanical, analog, and digital approaches to controlling recorded pitch and duration.	23
1.3	Representational thresholds in the long history of the phase vocoder.	24
1.4	Artifacts as historical research questions.	30
1.5	Representative PVC tool families and their working representations.	37
1.6	Selected chronology of phase-vocoder history and its neighboring lineages.	48
1.7	Three ways to think about changing recorded sound.	52
8.1	Comparison of all pvx analysis windows.	84
8.2	Measured properties of the hann window.	87
8.3	Measured properties of the hamming window.	88
8.4	Measured properties of the blackman window.	89
8.5	Measured properties of the blackmanharris window.	90
8.6	Measured properties of the nuttall window.	91
8.7	Measured properties of the flattop window.	92
8.8	Measured properties of the blackman nuttall window.	93
8.9	Measured properties of the exact blackman window.	94
8.10	Measured properties of the sine window.	95
8.11	Measured properties of the bartlett window.	96
8.12	Measured properties of the boxcar window.	97
8.13	Measured properties of the triangular window.	98
8.14	Measured properties of the bartlett hann window.	99
8.15	Measured properties of the tukey window.	100
8.16	Measured properties of the tukey 0p1 window.	101
8.17	Measured properties of the tukey 0p25 window.	102
8.18	Measured properties of the tukey 0p75 window.	103
8.19	Measured properties of the tukey 0p9 window.	104
8.20	Measured properties of the parzen window.	105
8.21	Measured properties of the lanczos window.	106
8.22	Measured properties of the welch window.	107
8.23	Measured properties of the gaussian 0p25 window.	108
8.24	Measured properties of the gaussian 0p35 window.	109
8.25	Measured properties of the gaussian 0p45 window.	110
8.26	Measured properties of the gaussian 0p55 window.	111
8.27	Measured properties of the gaussian 0p65 window.	112
8.28	Measured properties of the general gaussian 1p5 0p35 window.	113
8.29	Measured properties of the general gaussian 2p0 0p35 window.	114
8.30	Measured properties of the general gaussian 3p0 0p35 window.	115
8.31	Measured properties of the general gaussian 4p0 0p35 window.	116
8.32	Measured properties of the exponential 0p25 window.	117

8.33	Measured properties of the exponential 0p5 window.	118
8.34	Measured properties of the exponential 1p0 window.	119
8.35	Measured properties of the cauchy 0p5 window.	120
8.36	Measured properties of the cauchy 1p0 window.	121
8.37	Measured properties of the cauchy 2p0 window.	122
8.38	Measured properties of the cosine power 2 window.	123
8.39	Measured properties of the cosine power 3 window.	124
8.40	Measured properties of the cosine power 4 window.	125
8.41	Measured properties of the hann poisson 0p5 window.	126
8.42	Measured properties of the hann poisson 1p0 window.	127
8.43	Measured properties of the hann poisson 2p0 window.	128
8.44	Measured properties of the general hamming 0p50 window.	129
8.45	Measured properties of the general hamming 0p60 window.	130
8.46	Measured properties of the general hamming 0p70 window.	131
8.47	Measured properties of the general hamming 0p80 window.	132
8.48	Measured properties of the bohman window.	133
8.49	Measured properties of the cosine window.	134
8.50	Measured properties of the kaiser window.	135
8.51	Measured properties of the rect window.	136

Part I

Foundations

CHAPTER 1

What Is a Phase Vocoder?

A phase vocoder is an analysis-and-resynthesis instrument for recorded sound. It listens to a signal through a sequence of short, overlapping windows, estimates how each frequency component is moving, and rebuilds the signal on a new time grid. That one change of grid is what makes duration and pitch independently controllable. The result may be transparent, deliberately unreal, or somewhere wonderfully in between.

The name can be misleading. A conventional vocoder divides speech into broad frequency bands and transfers their envelopes to a carrier. A phase vocoder also measures frequency content, but its central concern is the phase advance of thousands of narrow spectral bins from one frame to the next. This lets it infer local frequency and re-schedule time without throwing away continuity.

1.1 Vocoder versus phase vocoder

The shared word *vocoder* describes an important family resemblance: both systems analyze sound into changing spectral information and use that information during synthesis. Their signal paths, purposes, and characteristic sounds are nevertheless different. A conventional channel vocoder is principally a source-filter instrument. It extracts slowly varying envelopes from a *modulator*, often speech, and applies those envelopes to a separate *carrier*, often a synthesizer or noise source. The carrier supplies excitation while the modulator supplies broad spectral articulation. This is the familiar talking-synthesizer effect.

A phase vocoder does not normally require a separate carrier. It converts successive overlapping frames of the input into complex short-time spectra, tracks phase change between frames, modifies the spectral representation, and resynthesizes a waveform. Its best-known uses are time stretching, pitch shifting, spectral freezing, and transformations that preserve or deliberately alter the input's own partials. The following comparison keeps the two meanings distinct.

Question	Conventional vocoder	Phase vocoder
What enters the system?	A modulator plus a separate carrier.	Usually one signal to analyze, modify, and resynthesize.
What is measured?	Broad-band amplitude envelopes, often with voiced and unvoiced information.	Complex STFT bins containing magnitude and phase, with phase change used to estimate local frequency.
What drives synthesis?	The measured envelopes shape bands of the carrier.	Modified magnitudes and propagated phases reconstruct the input on a synthesis timeline.
What is the usual goal?	Transfer speech-like articulation or spectral shape to another source.	Change duration, pitch, phase behavior, or spectral structure.
What is the characteristic sound?	A carrier that speaks, sings, or follows another sound's articulation.	The original sound stretched or transformed, sometimes with phasiness or transient blur.
Is phase central?	Not usually in the classic channel-vocoder design.	Yes. Inter-frame phase advance is the defining measurement.

Table 1.1: The practical differences between a conventional channel vocoder and a phase vocoder.

The quickest practical test is to ask where the excitation comes from. If speech envelopes make a synthesizer pad appear to speak, the process is conventional vocoding. If one recording is made longer while its pitch remains stable, the process is phase-vocoder time scaling. Cross-synthesis can blur this boundary

because a phase-vocoder system may combine spectral information from two sounds, but its complex-bin analysis and phase propagation still distinguish the underlying method from a classic envelope-controlled filter bank. Product names are not reliable evidence: plug-ins sometimes use *vocoder* as a broad label for any spectral resynthesis effect, so the documented signal path matters more than the name on the interface.

1.2 A quick mental model

Imagine a recording as a roll of film. Each frame is not an image but a tiny spectrum: how much bass, midrange, brightness, noise, and harmonic detail is present at one moment. A phase vocoder overlaps those frames so the film has no visible joins. It can space the reconstructed frames farther apart to lengthen time, closer together to compress time, or use an additional resampling step to move pitch.



Figure 1.1: The phase-vocoder loop. Each output frame is rebuilt from a spectrum whose phase has been advanced on a new time grid.

The practical cycle is simple:

1. Window a short slice of audio.
2. Transform it into magnitude and phase.
3. Compare phase with the preceding slice to estimate true frequency.
4. Advance a new output phase at the desired synthesis hop.
5. Invert the spectrum and overlap-add the result.

The word *phase* matters because magnitude alone is not enough to make a stable waveform. A sine wave can have the same magnitude at many moments while occupying different positions in its cycle. When thousands of partials are rebuilt without a coherent phase story, the ear hears blur, shimmer, or the familiar metallic wash called phasiness.

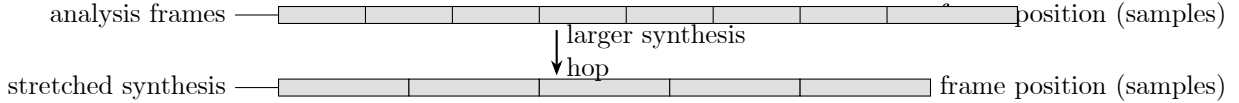


Figure 1.2: Time stretching changes the distance between reconstructed frames while preserving their spectral content.

1.3 From waveform to WOLA frames

The working core of a phase vocoder is a weighted overlap-add short-time Fourier transform, usually abbreviated WOLA STFT. The phrase names three operations at once. *Short-time* means that the signal is examined in finite frames. *Weighted* means that each frame is multiplied by a smooth analysis window and, in a general WOLA system, by a synthesis window after the inverse transform. *Overlap-add* means that neighboring reconstructed frames are placed on an output timeline and summed where they overlap. The FFT is important, but the FFT alone is not the phase vocoder. The surrounding frame schedule, phase measurements, and reconstruction weights make the transform usable for sound.

Let the frame beginning at analysis time tH_a be written as:

$$x_t[n] = x[n + tH_a]w_a[n], \quad 0 \leq n < N$$

where $x[n]$ is the input signal, t is the frame index, H_a is the analysis hop in samples, $w_a[n]$ is the analysis window, n is the local sample index, and N is the transform size.

The frame is local in two senses. It covers only N samples, and its samples are measured relative to the frame origin. Successive origins are separated by H_a , which is usually smaller than N . If $N = 2048$ and

$H_a = 512$, each new frame advances by one quarter of a window and therefore overlaps the preceding frame by 75 percent.

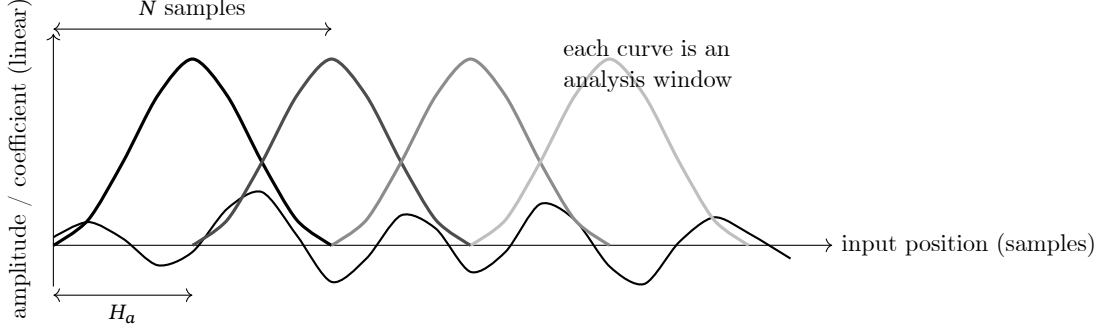


Figure 1.3: WOLA analysis framing. The waveform remains continuous, while overlapping windows select smoothly weighted local views.

The picture also explains why a hard rectangular cut is rarely the default. A discontinuity at either frame edge would look to the transform like broadband energy. A tapered window lowers the edge toward zero, reducing spectral leakage. Overlap then ensures that samples suppressed near one window edge receive useful weight from neighboring windows.

1.4 The STFT creates a time-frequency lattice

Each weighted frame is transformed into complex Fourier coefficients. The analysis STFT is:

$$X_t[k] = \sum_{n=0}^{N-1} x[n + tH_a]w_a[n]e^{-j2\pi kn/N}$$

where $X_t[k]$ is the complex coefficient at frame t and bin k , $x[n + tH_a]$ is the input sample under the frame, $w_a[n]$ is the analysis-window coefficient, N is the transform size, and $j = \sqrt{-1}$ is the imaginary unit.

Every coefficient has a magnitude and an angle:

$$X_t[k] = A_t[k]e^{j\phi_t[k]}, \quad A_t[k] = |X_t[k]|, \quad \phi_t[k] = \arg X_t[k]$$

where $A_t[k]$ is the nonnegative magnitude of bin k , $\phi_t[k]$ is its wrapped phase in radians, and $X_t[k]$ is the complex coefficient being represented in polar form.

Stacking the frames produces a lattice. Moving horizontally changes time. Moving vertically changes nominal frequency. Magnitude can be imagined as the darkness or height of each cell, while phase is an angle stored inside the same cell. A conventional spectrogram displays mostly magnitude, so it hides the information that a phase vocoder must preserve and propagate.

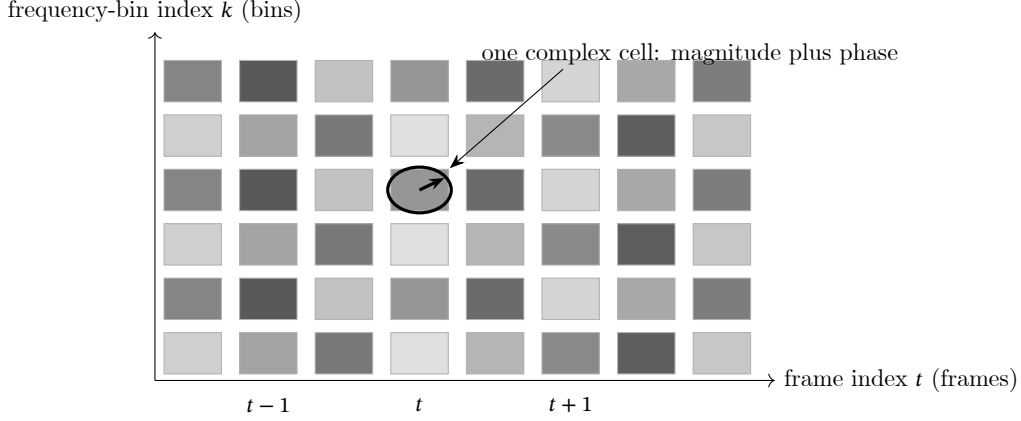


Figure 1.4: The STFT lattice. Each cell $X_t[k]$ contains both magnitude and phase, even though a spectrogram normally displays only magnitude.

The nominal center frequency of bin k is:

$$f_k = \frac{k f_s}{N}$$

where f_k is the bin-center frequency in hertz, k is the bin index, f_s is the sampling frequency, and N is the transform size.

Real musical partials seldom land exactly at those centers. A sinusoid at 443 Hz, for example, may distribute energy among bins whose centers lie on either side of 443 Hz. Its true frequency is revealed by how phase changes from one frame to the next. This is the key step that distinguishes a phase vocoder from a sequence of unrelated spectral snapshots.

1.5 Reading frequency from phase advance

Suppose a sinusoid sits exactly at bin center k . Advancing the input by H_a samples should advance its phase by the predictable amount:

$$\Omega_k H_a = \frac{2\pi k H_a}{N}$$

where $\Omega_k = 2\pi k/N$ is the nominal angular frequency in radians per sample, H_a is the analysis hop, k is the bin index, and N is the transform size.

The measured phase difference contains that expected rotation plus a residual caused by the component's displacement from the bin center. Because an angle reported by $\arg$ is known only modulo 2π , the residual must be wrapped into a principal interval:

$$\Delta\phi_t[k] = \text{princarg}(\phi_t[k] - \phi_{t-1}[k] - \Omega_k H_a)$$

where $\Delta\phi_t[k]$ is the wrapped residual phase advance, $\phi_t[k]$ and $\phi_{t-1}[k]$ are consecutive observed phases, $\Omega_k H_a$ is the expected bin-center advance, and princarg maps an angle to $(-\pi, \pi]$.

The estimated instantaneous angular frequency is then:

$$\hat{\omega}_t[k] = \Omega_k + \frac{\Delta\phi_t[k]}{H_a}$$

where $\hat{\omega}_t[k]$ is the estimated angular frequency in radians per sample, Ω_k is the nominal bin-center angular frequency, $\Delta\phi_t[k]$ is the wrapped residual, and H_a is the analysis hop.

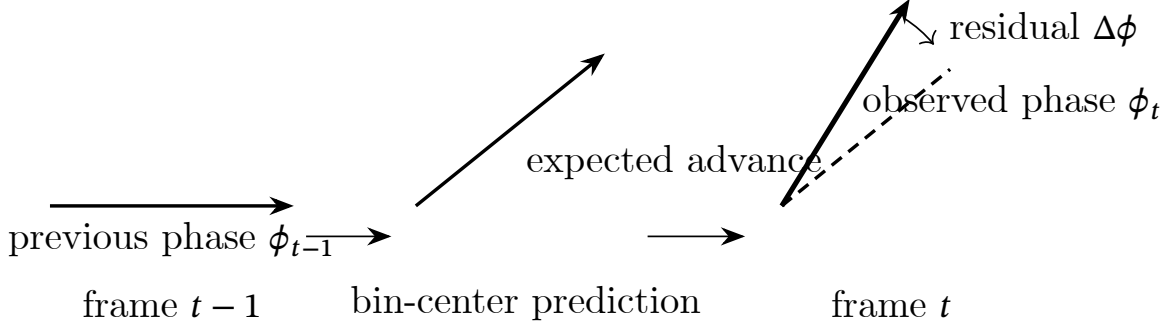


Figure 1.5: Phase advance as a frequency measurement. The residual between predicted and observed rotation estimates the offset from the nominal bin center.

Wrapping is not an optional cosmetic operation. Without it, the same physical angle could appear to jump by nearly 2π when the reported phase crosses from $+\pi$ to $-\pi$. The principal-argument operation chooses the locally plausible residual. That choice relies on sufficient temporal sampling: if the phase can move too far between frames, the estimate becomes ambiguous. Smaller analysis hops reduce that risk at the cost of more frames and more computation.

1.6 Moving the frames without moving their pitch

Time stretching begins when analysis and synthesis use different frame schedules. The input is read every H_a samples, but reconstructed frames are written every H_s samples. For a constant stretch ratio α , the basic schedule is:

$$H_s = \alpha H_a$$

where H_s is the synthesis hop, H_a is the analysis hop, and α is the output-duration ratio, with $\alpha > 1$ producing a longer output and $0 < \alpha < 1$ producing a shorter output.

Simply copying the input phase into frames placed at the new spacing would change or disorder frequency. Instead, the measured instantaneous frequency is integrated over the synthesis hop:

$$\theta_t[k] = \theta_{t-1}[k] + \hat{\omega}_t[k]H_s$$

where $\theta_t[k]$ is the accumulated synthesis phase, $\theta_{t-1}[k]$ is its previous value, $\hat{\omega}_t[k]$ is the estimated instantaneous angular frequency, and H_s is the synthesis hop.

The output spectrum combines the current magnitude with that newly propagated phase:

$$Y_t[k] = A_t[k]e^{j\theta_t[k]}$$

where $Y_t[k]$ is the output spectrum, $A_t[k]$ is the selected analysis magnitude, and $\theta_t[k]$ is the accumulated synthesis phase.

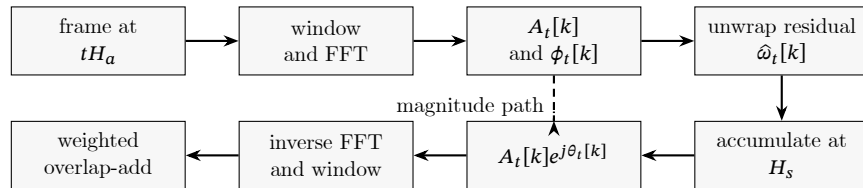


Figure 1.6: A complete phase-vocoder frame path. Magnitude follows the current analysis frame, while frequency estimates drive a new phase trajectory on the synthesis timeline.

For a concrete example, take $H_a = 256$ samples and request a two-times stretch. Then $H_s = 512$ samples. Analysis frame 10 begins at input sample 2560, while synthesis frame 10 begins at output sample 5120. A

component estimated at 440 Hz is still advanced as a 440 Hz component, but it is advanced across the longer 512-sample synthesis interval. This is how the algorithm expands time without halving pitch.

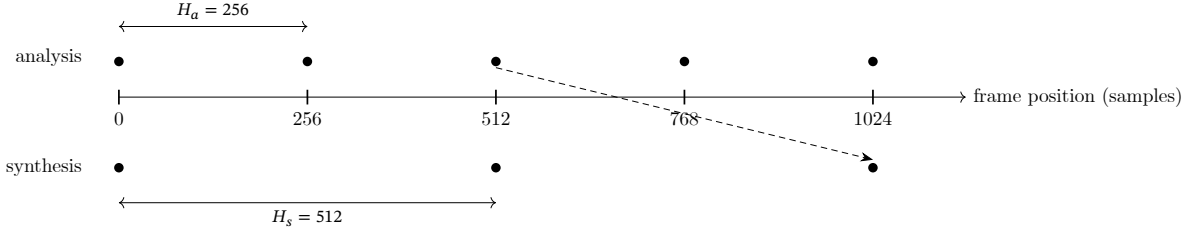


Figure 1.7: A two-times stretch. The same sequence of analyzed states is reconstructed at twice the frame spacing.

1.7 Inverse STFT and weighted overlap-add

After phase propagation, each output spectrum is transformed back into a time-domain frame:

$$\tilde{x}_t[n] = \frac{1}{N} \sum_{k=0}^{N-1} Y_t[k] e^{j2\pi kn/N}$$

where $\tilde{x}_t[n]$ is the inverse-transformed frame, $Y_t[k]$ is the complex output spectrum, N is the transform size, k is the bin index, and n is the local time index.

Those frames still have local coordinates. WOLA places frame t at output position tH_s , multiplies it by a synthesis window, and adds every overlapping contribution. A robust normalized form is:

$$\hat{x}[n] = \frac{\sum_t \tilde{x}_t[n - tH_s] w_s[n - tH_s]}{\sum_t w_a[n - tH_s] w_s[n - tH_s] + \epsilon}$$

where $\hat{x}[n]$ is the reconstructed output sample, $\tilde{x}_t$ is inverse-transformed frame t , w_a and w_s are the analysis and synthesis windows, H_s is the synthesis hop, and ϵ is a small positive value that prevents division by zero near boundaries.

When the same window is used for analysis and synthesis, the denominator becomes a sum of squared window values. This normalization is what keeps overlap from causing a periodic level ripple. In a compatible constant-overlap-add configuration, the accumulated weighting is constant through the interior of the signal. Near the beginning and end, padding and explicit normalization handle the incomplete set of neighbors.

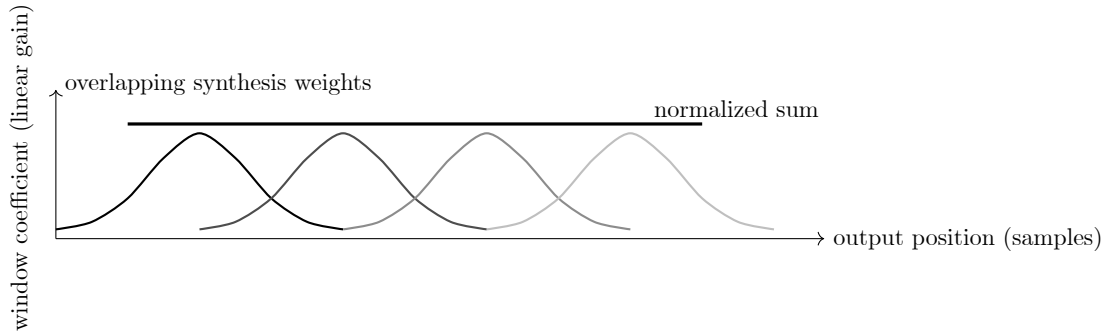


Figure 1.8: Weighted overlap-add reconstruction. Individual frame weights rise and fall, while their normalized sum remains level through the well-covered region.

Perfect reconstruction is easiest to understand as a bookkeeping promise. If the spectra are left unchanged, the phase path is left unchanged, the windows and hops are compatible, and normalization is correct, the reconstructed samples should match the input apart from numerical roundoff and boundary policy. Once

magnitudes, phases, or the synthesis schedule are edited, WOLA still provides smooth assembly, but it cannot guarantee perceptual transparency by itself.

1.8 Why basic phase propagation can still sound wrong

The derivation treats every bin as an independent sinusoidal tracker. Musical sounds are not independent bins. One partial spreads across several bins because of the window response, an attack excites many bins at once, and stereo channels encode a shared acoustic event. Independent phase propagation can preserve each bin’s horizontal continuity while damaging the vertical relationships among neighboring bins.

The most common improvements therefore restore relationships that the basic model ignores. Peak or identity phase locking makes bins around a spectral peak move with a common reference. Transient resets prevent old steady-state phase trajectories from being dragged through a new attack. Hybrid processing may route attacks through waveform-similarity overlap-add while reserving the phase vocoder for sustained regions. Stereo locking preserves selected interchannel phase differences. None of these changes replaces WOLA; each changes the spectra or phase trajectories that WOLA reconstructs.

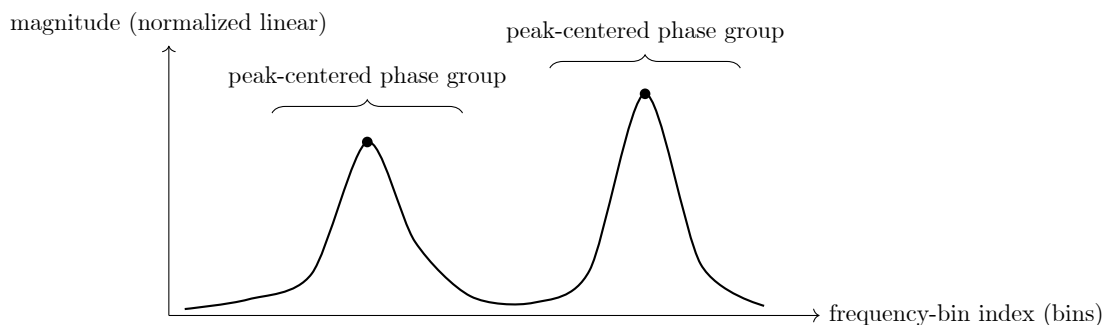


Figure 1.9: Vertical phase coherence. Phase locking treats the bins around a spectral peak as a related group rather than as unrelated oscillators.

The complete mental model is therefore larger than “FFT, modify, inverse FFT.” A phase vocoder observes overlapping local spectra, estimates frequency from phase motion, writes those frequencies onto a new time grid, repairs important within-frame and cross-channel relationships, and reconstructs the weighted frames through overlap-add. The quality of a result depends on every link in that chain.

1.9 The historical thread

The phase vocoder belongs to a longer attempt to separate the identity of sound from the speed at which it unfolds. Mechanical and tape methods came first. If tape runs faster, a recording becomes shorter and its pitch rises. If it runs slower, it becomes longer and its pitch falls. The basic artistic wish, especially in speech, film, and music production, was to change one of those dimensions while keeping the other stable.

Speech analysis before digital audio

In the 1930s and 1940s, Bell Telephone Laboratories developed systems that analyzed speech into spectral control information and reconstructed it with a separate source. Homer Dudley’s vocoder and Voder were built for speech research and demonstration, not for the later practice of digital time scaling. Still, they established the crucial conceptual split: a sound can be described by time-varying spectral structure and then recreated by another mechanism.

The historical vocoder used filter banks and envelope followers. It was not yet a short-time Fourier transform, but it trained a generation of engineers to think of speech as a continuously changing spectrum rather than a single indivisible waveform.



Figure 1.10: A Voder demonstration at the 1939 New York World's Fair, reproduced from Homer Dudley's 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

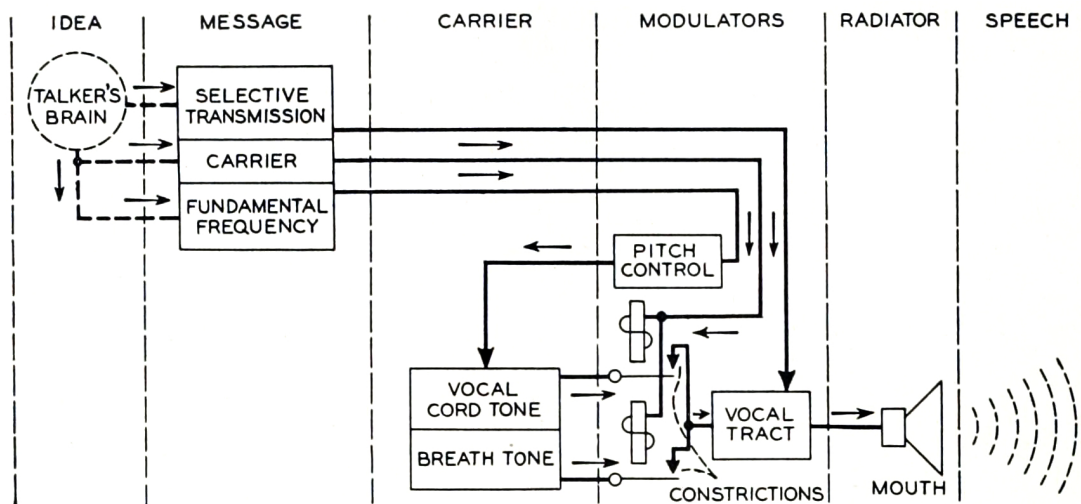


Fig. 6—Block diagram of the voice mechanism.

Figure 1.11: Dudley's 1940 block diagram of the voice mechanism. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

Tape, television, and the wish to uncouple time from pitch

During the tape era, machines such as Anton Springer’s rotating-head systems and the later Eltro information rate changer attacked the same problem by slicing and reassembling material mechanically. The techniques made duration and pitch more independent than ordinary varispeed, but editing points, tape-head geometry, and repeated segments imposed their own audible fingerprint. Film and studio work adopted these systems precisely because the problem was already urgent: dialogue had to fit picture, voices had to become strange, and musical phrases had to land where an edit demanded.

The digital phase vocoder inherits that ambition but changes the object being stitched. Instead of small tape segments, it joins carefully phase-propagated spectra. The audible problem remains recognizable: continuity is everything.

Flanagan’s digital formulation

James L. Flanagan introduced the phase-vocoder idea in the 1960s as part of speech analysis and synthesis research. His formulation described a signal in terms of slowly varying amplitudes and instantaneous phases for bands of frequency. By the mid-1970s, Mark Portnoff showed how the short-time Fourier transform and the fast Fourier transform made the method practical on digital hardware.

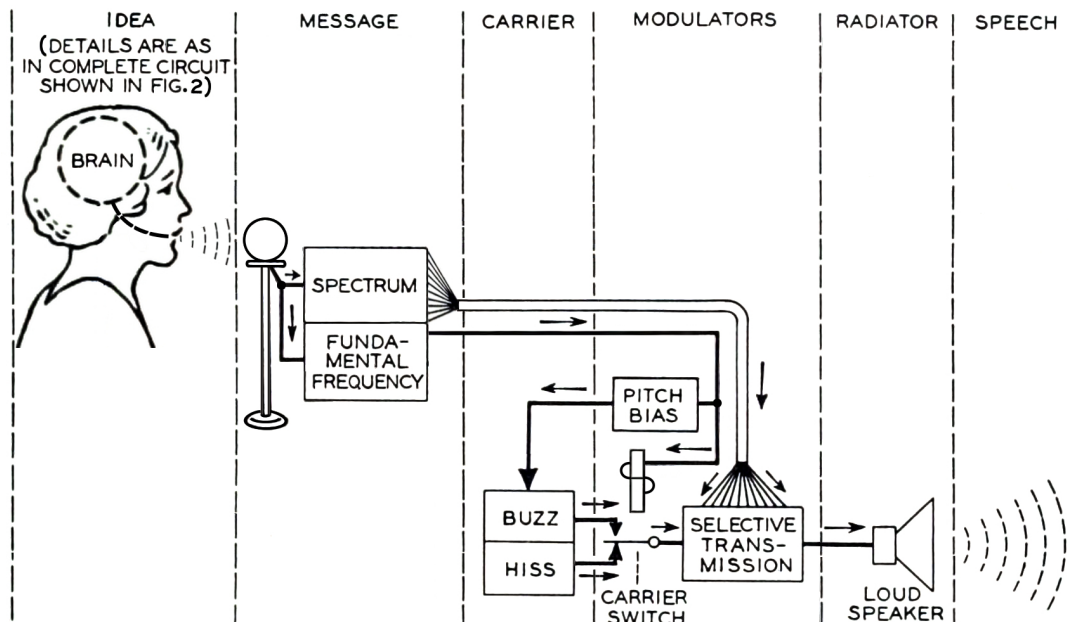


Fig. 7—Schematic circuit of the vocoder.

Figure 1.12: A historical vocoder schematic from Dudley’s 1940 paper. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

The key move is to compare observed phase advance with the advance expected at the center of each FFT bin. Their difference estimates the signal’s local frequency. That estimate is then accumulated on a new synthesis timeline.

$$\Delta\phi_t[k] = \text{princarg}\left(\phi_t[k] - \phi_{t-1}[k] - \frac{2\pi k H_a}{N}\right)$$

where symbols and units follow the definitions established for flanagan’s digital formulation.

$$\hat{\omega}_t[k] = \frac{2\pi k}{N} + \frac{\Delta\phi_t[k]}{H_a}$$

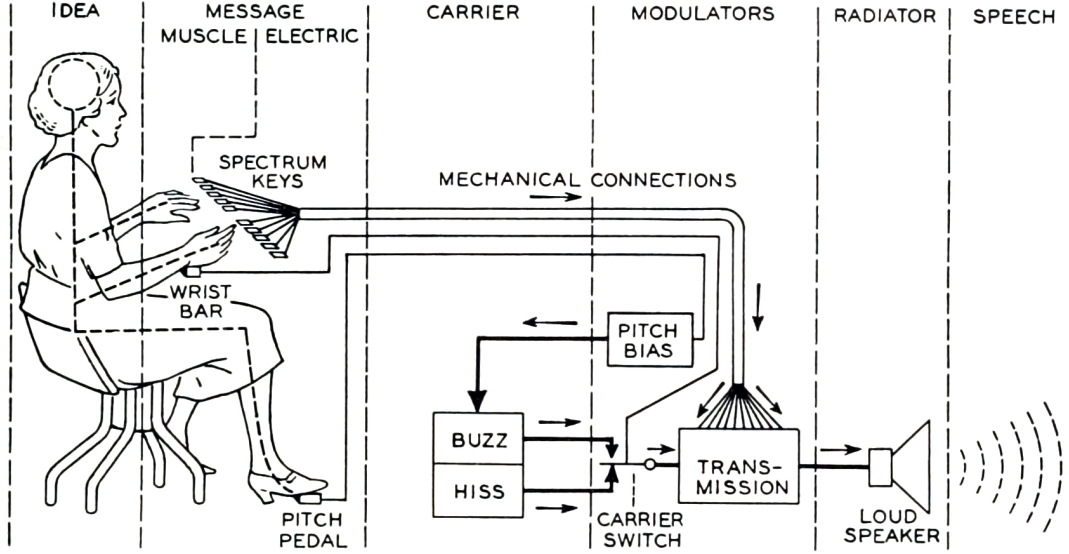


Fig. 8—Schematic circuit of the voder.

Figure 1.13: Dudley’s companion Voder schematic, showing the related synthesis-oriented system. Public-domain or no-known-restrictions scan via Internet Archive Book Images on Wikimedia Commons.

where symbols and units follow the definitions established for flanagan’s digital formulation.

$$\hat{\phi}_t[k] = \hat{\phi}_{t-1}[k] + \hat{\omega}_t[k]H_s$$

where symbols and units follow the definitions established for flanagan’s digital formulation.

Here H_a is the analysis hop, H_s is the synthesis hop, N is the FFT size, and k identifies a spectral bin. Increasing H_s relative to H_a spreads reconstructed frames farther apart, lengthening the signal.

From research technique to musical tool

In the 1980s and 1990s, the phase vocoder became an expressive music-processing technique as personal computers and dedicated DSP systems became capable of enough overlapping FFTs. The method made radical time stretching, pitch transposition, spectral freezing, and formant experiments possible in a way tape systems could not. The tradeoff was clear: a phase vocoder can preserve stable tones remarkably well, but transients and rapidly changing spectra require special care.

Modern systems therefore add phase locking, transient detection, multi-resolution analysis, identity phase handling, and hybrid resynthesis. pvx follows this tradition with controls that make the quality decisions explicit rather than mysterious.

A chronology of ideas, not merely machines

The history of the phase vocoder is easiest to understand as a sequence of changing representations. Dudley’s channel vocoder represented speech by slowly varying band energies and source decisions. Flanagan and Golden represented a signal by short-time amplitude and phase information. Portnoff reorganised the same ideas around the FFT so they could be computed efficiently. Dolson made the digital method legible to musicians and computer-music practitioners. Laroche and Dolson then revisited the phase assumptions that caused loss of presence and gave the field a practical account of vertical coherence.

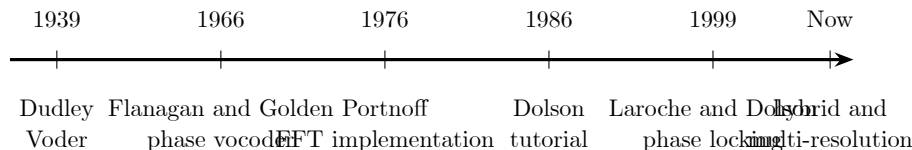


Figure 1.14: A selected chronology of representations and implementation ideas leading to modern phase-vocoder practice.

This chronology should not be read as a single straight line of replacement. Channel vocoders, sinusoidal models, overlap-add systems, time-domain methods, and phase-vocoder techniques continued to influence one another. The important shift is that each generation made a different quantity convenient to manipulate. Once short-time phase derivatives became available, duration could be changed by editing the resynthesis schedule rather than by mechanically changing playback speed.

Flanagan and Golden: phase as transmitted information

James L. Flanagan and Robert M. Golden’s 1966 paper, *Phase Vocoder*, described speech in terms of short-time amplitude and phase spectra and simulated a complete analysis-transmission-synthesis system on a digital computer. Their title joined an older word, vocoder, to the quantity that distinguished the new representation. The method did not merely transmit the output of a bank of envelope followers. It retained information about local phase change, which could be interpreted as instantaneous frequency within each analysis channel.

For a complex channel output $X_k(t) = A_k(t)e^{j\phi_k(t)}$, the local angular frequency can be expressed as:

$$\omega_k(t) = \frac{d\phi_k(t)}{dt}$$

where $\omega_k(t)$ is instantaneous angular frequency for channel k , $\phi_k(t)$ is its unwrapped phase, and t is continuous time.

The equation looks modest, but it changes the meaning of phase from a static angle into a trajectory. If that trajectory is sampled consistently, a sinusoid that falls between channel centres can be represented by the difference between expected and observed phase advance. That principle survives in modern STFT implementations.

Flanagan and Golden also discussed time-scale expansion and compression. The historical significance is not that every later implementation copied their system detail for detail. It is that the paper established a phase-aware analysis-resynthesis vocabulary in which timing could become an independent operation. The paper appeared in the *Bell System Technical Journal*, volume 45, number 9, pages 1493 through 1509, with DOI 10.1002/j.1538-7305.1966.tb01706.x.

Portnoff: the FFT makes the method practical

Mark R. Portnoff’s 1976 paper, *Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform*, moved the method toward the computational form now familiar in software. The FFT performed the bulk of both analysis and synthesis. Windows, frame hops, phase estimates, and overlap-add reconstruction could be described as a repeatable block algorithm rather than as a large bank of individually implemented filters.

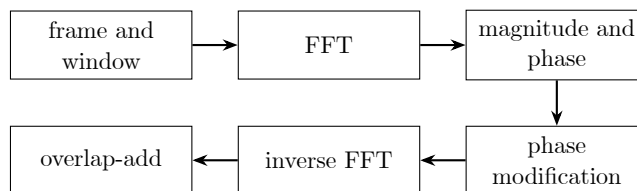


Figure 1.15: The FFT-oriented analysis and synthesis loop associated with the practical digital phase vocoder.

The computational saving matters because a phase vocoder requires many transforms. For a signal of length L , FFT size N , and analysis hop H_a , an approximate transform count is:

$$F \approx 2 \left\lceil \frac{L - N}{H_a} \right\rceil + 2$$

where F is the combined number of forward and inverse transforms, L is the signal length in samples, N is the FFT size, and H_a is the analysis hop in samples.

The exact count depends on padding and boundary policy, but the relationship explains why hop size is both a quality parameter and a cost parameter. Smaller hops increase temporal sampling and overlap while increasing the number of transforms. Portnoff's FFT formulation made that tradeoff manageable on the minicomputers of the period and natural on later workstations.

Portnoff also placed window design close to implementation. A phase estimate is only as useful as the frame that produced it. Leakage, main-lobe width, and overlap behaviour determine how clearly a sinusoidal component appears and how smoothly frames recombine. The complete window atlas later in this book develops that part of the inheritance.

Computer music and the widening of purpose

By the late 1970s and 1980s, the phase vocoder had escaped the narrow category of speech transmission. Computer-music systems treated analysis data as compositional material. Magnitudes could be frozen, interpolated, filtered, or transferred. Phase trajectories could be advanced at a new rate. A sound could be stretched until its internal partials became a landscape.

This change of audience mattered. A telecommunications engineer might measure intelligibility and bandwidth. A composer might value a transformation precisely because it revealed hidden modulation or made a transient dissolve into texture. The same artifact could be an error in one setting and an aesthetic resource in another. Phase-vocoder history is therefore also a history of listening criteria.

The separation of analysis from resynthesis encouraged archives of spectral frames and offline transformations. A program could analyze once and render many times. This pattern remains visible in pvx analysis artifacts, response profiles, checkpointing, and batch workflows. What changed is the speed and scale: operations that once required a research workstation can now be scripted across a corpus.

Mark Dolson and the tutorial synthesis

Mark Dolson's 1986 article, *The Phase Vocoder: A Tutorial*, became a major bridge between specialist signal-processing literature and computer-music practice. Its importance lies partly in synthesis. The phase vocoder had accumulated several equivalent descriptions, implementation conventions, and practical uses. Dolson presented the method as a coherent system and related analysis parameters to transformations that musicians could recognize.

The tutorial emphasized that the analysis produces time-varying amplitude and frequency information. Time scaling can then be understood as resampling those trajectories or changing the relationship between analysis and synthesis time. Pitch modification can be built by combining time scaling with sample-rate conversion, or by altering spectral placement more directly.

Let the time-scale factor be α . A simple hop relationship is:

$$H_s = \alpha H_a$$

where H_s is the synthesis hop, H_a is the analysis hop, and α is the requested output-duration ratio.

This compact equation is not a complete algorithm because phase must follow the new schedule. It is nevertheless the control relationship users encounter most directly. If $\alpha > 1$, output frames are spaced farther apart and the sound becomes longer. If $0 < \alpha < 1$, they are spaced closer together and the sound becomes shorter.

Dolson also helped establish a vocabulary for practical applications: time scaling, pitch transposition, cross-synthesis, spectral modification, and the analysis of musical sounds. The tutorial appeared in *Computer*

Music Journal, volume 10, number 4, pages 14 through 27. Its enduring role is pedagogical. It taught readers to see the phase vocoder as a general musical instrument whose operations emerge from one analysis-resynthesis model.

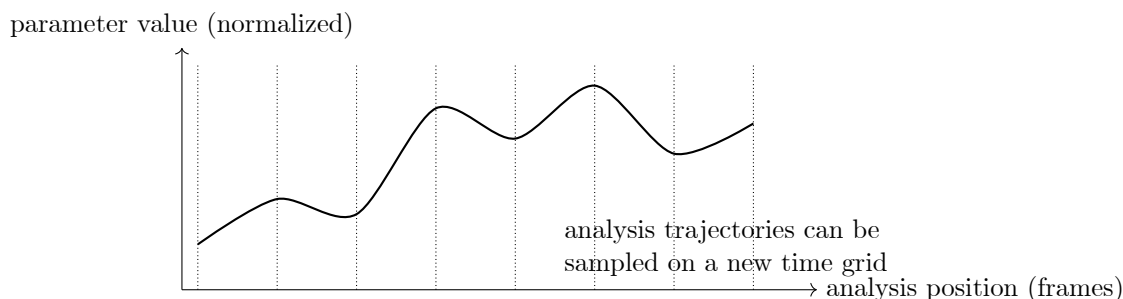


Figure 1.16: A conceptual parameter trajectory, the representation that makes time remapping and spectral editing intelligible.

Horizontal coherence and vertical coherence

For many years the classic phase vocoder’s central weakness was described with perceptual words: phasiness, reverberation, diffuseness, or loss of presence. These terms overlap but are not identical. They point to a sound whose partials no longer seem to belong to one compact source.

Two kinds of relationship help explain the problem. Horizontal coherence concerns a bin across successive frames. Its phase should evolve according to the component’s instantaneous frequency. Vertical coherence concerns neighbouring bins within one frame. Bins belonging to the same spectral peak should retain a meaningful phase relationship.

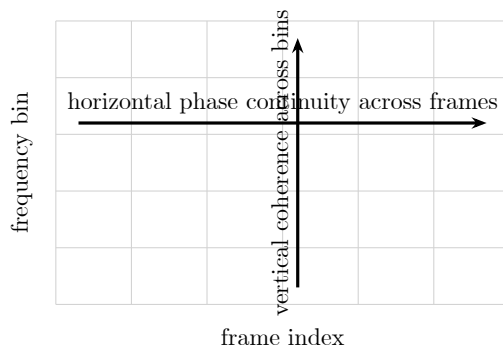


Figure 1.17: Horizontal and vertical phase relationships in the STFT lattice.

Classic propagation can preserve horizontal continuity while allowing bins within a peak to drift independently. The resynthesised peak then behaves less like one sinusoidal component observed through a window and more like a collection of unrelated oscillators. That decorrelation is one source of the diffuse quality associated with phase-vocoder stretching.

Jean Laroche and Mark Dolson: explaining phasiness

Jean Laroche and Mark Dolson’s 1999 paper, *Improved Phase Vocoder Time-Scale Modification of Audio*, directly examined phasiness and proposed new phase-calculation techniques. The paper’s historical importance is that it connected a familiar perceptual complaint to the loss of phase coherence across bins and then offered practical algorithms that improved sound quality.

The authors distinguished phase propagation at spectral peaks from the treatment of bins around those peaks. A peak can be regarded as the centre of a region of influence. Rather than allowing every bin to accumulate phase independently, surrounding bins can inherit a phase relationship from the peak.

For peak bin p and neighbouring bin k , identity phase locking can be written conceptually as:

$$\hat{\phi}_t[k] = \hat{\phi}_t[p] + \phi_t[k] - \phi_t[p]$$

where $\hat{\phi}_t[k]$ is the synthesis phase of neighbour bin k , $\hat{\phi}_t[p]$ is the propagated synthesis phase of peak bin p , $\phi_t[k]$ is the observed analysis phase of bin k , and $\phi_t[p]$ is the observed analysis phase of the peak.

The subtraction retains the analysis-frame phase offset between the neighbour and its peak. The addition places that offset around the peak's coherent synthesis phase. The bins therefore preserve more of the structure imposed by one underlying partial and its window transform.

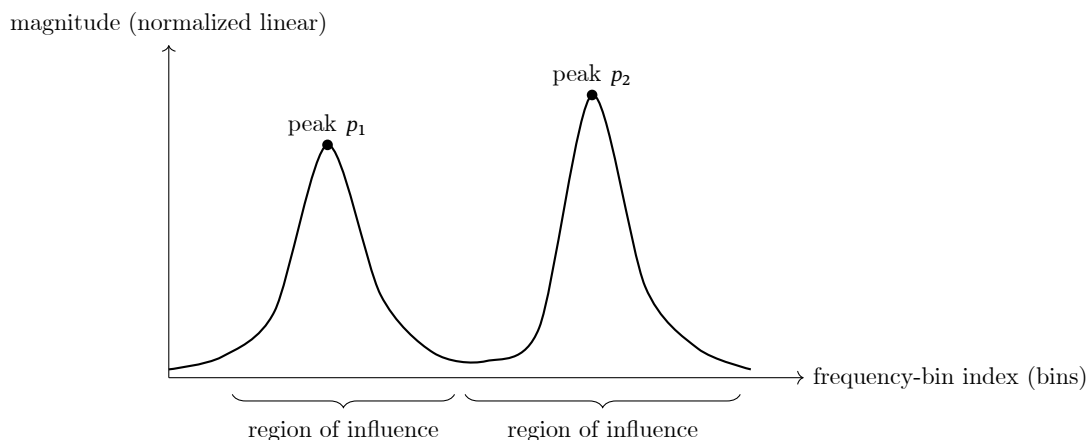


Figure 1.18: Identity phase locking assigns neighbouring bins to spectral peaks and preserves their observed relative phases.

Laroche and Dolson described two extensions and reported significantly improved results. Their work also showed that improved coherence could reduce computational cost in some formulations. The paper appeared in *IEEE Transactions on Speech and Audio Processing*, volume 7, number 3, pages 323 through 332, DOI 10.1109/89.759041.

The contribution should not be reduced to one option labelled phase locking. It changed the field's account of why the classic sound occurred. Once phasiness was described as a coherence problem, later systems could compare strategies for peak selection, region assignment, transient handling, and channel consistency.

Pitch shifting, harmonising, and spectral translation

Laroche and Dolson also described frequency-domain techniques for pitch shifting, harmonising, chorusing, and non-standard frequency modification. Their 1999 workshop paper presented integer-bin and fractional-bin spectral translation approaches with phase adjustment across successive frames. This line of work is important for pvx because pitch processing is not merely a time stretch followed by anonymous resampling. It can be understood as controlled movement of spectral regions.

For a semitone displacement s , the ideal frequency ratio is:

$$r = 2^{s/12}$$

where r is the multiplicative frequency ratio and s is the displacement in equal-tempered semitones.

For a cent displacement c , the corresponding relation is:

$$r = 2^{c/1200}$$

where r is the multiplicative frequency ratio and c is the displacement in cents.

These equations state the musical target, not the full spectral operation. A practical algorithm must decide how magnitudes move between bins, how phases remain continuous, how components crossing one another are treated, and whether formant structure should follow the pitch shift.

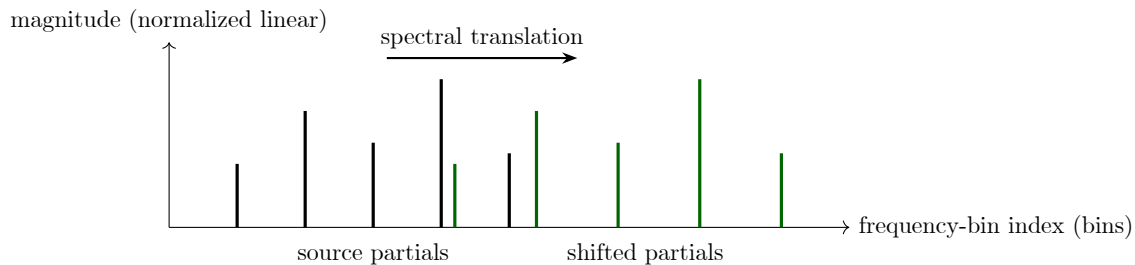


Figure 1.19: A conceptual spectral translation used for pitch shifting and harmonisation.

Transients expose the limits of stationarity

The phase vocoder assumes that a short frame can be treated as a useful local spectral description. A transient challenges that assumption. Its energy changes rapidly, and the onset may occupy only a small fraction of the window. Stretching the surrounding spectral frames can spread that event through time and reduce its impact.

Transient-aware systems detect sudden changes, preserve or reset selected phases, shorten the effective window, or route the event through a time-domain method. Each strategy makes a different compromise. A reset restores alignment at an onset but can interrupt a stable sinusoidal trajectory. A longer window improves low-frequency discrimination but includes more samples from both sides of the attack.

One common spectral-flux measure is:

$$SF_t = \sum_k \max(|X_t[k]| - |X_{t-1}[k]|, 0)$$

where SF_t is positive spectral flux at frame t , $X_t[k]$ is the complex STFT coefficient at bin k , and the maximum retains only increases in magnitude.

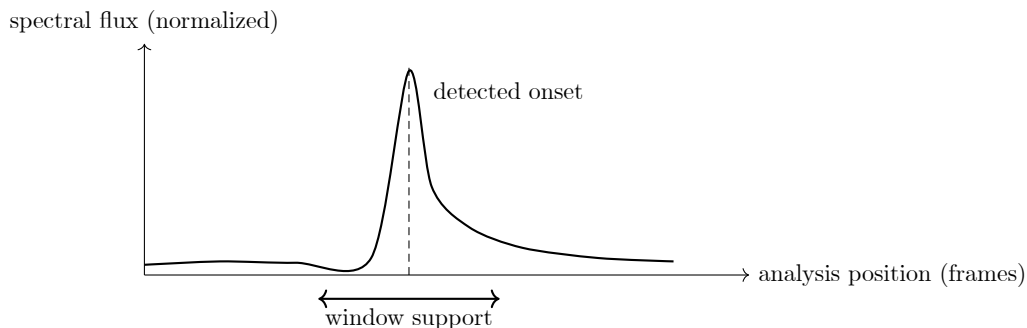


Figure 1.20: A transient can occupy only a small part of a frame, making onset preservation a separate design problem.

This problem led to transient detection, phase resets, hybrid methods, and multi-resolution systems. It also explains why no single window is best for every source. A percussive signal asks the analysis to localise time; a sustained bass note asks it to discriminate nearby low frequencies.

Stereo and multichannel coherence

The historical phase vocoder was often described for one channel. Modern production material is commonly stereo or multichannel, and independent processing can disturb interchannel time and level differences. A

stable image requires the algorithm to consider relationships between channels as well as relationships between bins.

For channels i and j , interchannel phase difference is:

$$\Delta\phi_{ij}(k, t) = \phi_i(k, t) - \phi_j(k, t)$$

where $\Delta\phi_{ij}(k, t)$ is the phase difference at bin k and frame t , while ϕ_i and ϕ_j are the channel phases.

Preserving that difference exactly is not always the only goal. Low-frequency image stability, diffuse ambience, decorrelated noise, and transient direction can require different treatment. `pvx` therefore exposes coherence choices rather than assuming that copying one channel's phase is universally correct.

1.10 A deeper history of spectral time

The phase vocoder did not appear from one isolated invention. It emerged when several older lines of work became computationally compatible: harmonic analysis, telephone speech research, filter-bank engineering, tape manipulation, digital sampling, fast Fourier transforms, and computer music. Each line contributed a different idea about what sound was and what part of it could be edited. The resulting instrument is therefore best understood as a meeting point rather than a single device with one uninterrupted genealogy.

This longer account follows both engineering and musical history. Engineering history explains why amplitude, frequency, and phase became measurable trajectories. Musical history explains why anyone wished to stretch those trajectories, freeze them, exchange them between sources, or make them cease to resemble their origin. The two histories repeatedly altered one another. A representation designed for economical transmission became a means of composition; an artifact identified by musicians became a research problem; a laboratory process became an interactive effect.

Harmonic analysis before electronic sound

The distant mathematical prehistory begins with the idea that a complex periodic motion can be represented as a sum of simpler oscillations. Joseph Fourier's work on heat conduction in the early nineteenth century supplied the formal language later used to describe sound spectra. Fourier was not building an audio processor, and it would be misleading to make him the inventor of every spectral technique. His importance is more specific: Fourier series and transforms made it possible to treat a waveform and its frequency components as two descriptions of related information.

Nineteenth-century acoustics made that equivalence tangible. Resonators, tuning forks, sirens, flame apparatus, and mechanical waveform devices allowed investigators to isolate or display periodic behavior. Hermann von Helmholtz used tuned resonators to study partials and vowel quality. His account of tone sensation connected physical spectra with auditory experience and helped establish the distinction between fundamental pitch and spectral coloration. Later phase-vocoder practice would revisit the same distinction through pitch shifting and formant preservation.

Early acoustical instruments were selective but not instantaneous. A resonator might reveal energy around one frequency, yet it did not produce the dense, uniformly sampled time-frequency grid familiar from a modern spectrogram. The historical achievement was conceptual. Sound could be analyzed into components, components could have unequal strengths, and the pattern of those strengths mattered to timbre. Once those claims became ordinary, the later question was how rapidly and precisely the pattern could be measured as it changed.

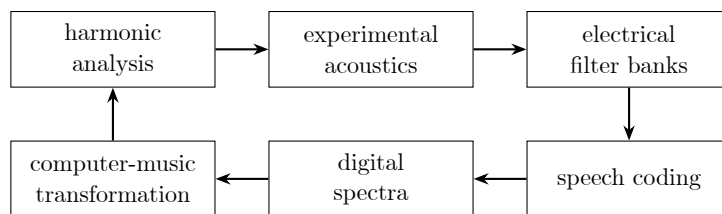


Figure 1.21: The phase vocoder grew from a loop of mathematical, acoustical, communications, and musical practices rather than a single linear ancestry.

The role of phase was less obvious to early listeners than the role of amplitude. A steady sinusoid shifted in absolute phase can sound unchanged when heard alone. This encouraged the belief that phase was perceptually unimportant. The belief is only conditionally useful. Relative phase affects waveform shape, transients, localization, interference, and the ability to join adjacent frames. The phase vocoder’s history can be read as the gradual discovery that phase may be unobtrusive in one static test and indispensable in a changing analysis-synthesis system.

Telegraphy, telephony, and the filter-bank imagination

Electrical communication transformed acoustical analysis into an engineering problem. Telegraph and telephone systems had to transmit useful information through channels with limited bandwidth and noise. Engineers learned to describe circuits by frequency response, to divide spectra into bands, and to measure how speech intelligibility survived filtering. The telephone network created both the institutional scale and the practical urgency for speech analysis.

A filter bank turns one broadband signal into several narrower signals. Each output describes what occurs in one frequency region over time. This architecture encouraged an important abstraction: speech might be transmitted through control functions rather than by reproducing every detail of the waveform. If the slowly varying energy in each band could be measured, perhaps a receiver could reconstruct an intelligible approximation from those measurements and a suitable excitation.

The abstraction was economical and scientific at once. It reduced transmission requirements, but it also offered a model of speech production. A source, periodic for voiced sounds or noisy for unvoiced sounds, passed through a resonant vocal tract. The resonances shaped vowels and speaker identity. Later source-filter models, linear predictive coding, true-envelope methods, and formant-preserving pitch shifts would refine this separation. The historical vocoder was one early operational version of it.

Filter-bank thinking also introduced a tension that remains in spectral software. Narrow bands provide fine frequency selectivity but respond slowly to change. Wide bands respond more quickly but merge nearby components. In digital terminology, this becomes the time-frequency tradeoff of the analysis window. The underlying compromise predates the FFT by decades.

Homer Dudley, the vocoder, and the Voder

At Bell Telephone Laboratories, Homer Dudley developed the channel vocoder during the 1920s and 1930s. The system analyzed speech with bandpass filters, extracted control envelopes, and used those envelopes to shape synthesized excitation at the receiver. A voiced-unvoiced decision selected periodic or noise-like excitation. Dudley’s public Voder demonstration at the 1939 New York World’s Fair reversed the emphasis: a trained operator controlled a keyboard and pedal system to synthesize speech directly.

These systems are ancestors in concept, not phase vocoders in the modern algorithmic sense. They did not use overlapping FFT frames or propagate short-time spectral phase. Their historical contribution was the separation of a sound into excitation and time-varying spectral control. They demonstrated that recognizability could survive a radical change of representation and that intelligible speech could be reconstructed from parameters.

The Voder also revealed the labor hidden by a representation. The machine did not automatically understand language. Skilled operators learned coordinated gestures that produced vowels, consonants, pitch

inflection, and timing. That fact is relevant to current software. A sophisticated transform still requires a user to learn which controls correspond to audible outcomes. Better automation changes the interface to the labor; it does not eliminate the need for judgment.

Dudley’s diagrams remain striking because they place physiology, analysis, and synthesis beside one another. The human vocal mechanism is shown as a source filtered by resonances. The electrical vocoder replaces that mechanism with detectors and controlled oscillators or noise. Modern formant tools still operate inside this conceptual space, even when their analysis uses cepstra, sinusoidal models, or high-resolution envelopes.

Secure speech and large-scale vocoder engineering

During the Second World War, vocoder principles contributed to secure speech systems, most famously SIGSALY. Its engineering history involved quantization, synchronization, encryption, and transmission as well as spectral coding. SIGSALY was not a phase vocoder, but it showed that analysis-synthesis speech systems could operate as large technical infrastructures rather than isolated laboratory demonstrations.

The scale of such systems mattered. They required stable oscillators, coordinated timing, calibrated channels, and disciplined operation. In miniature, the same concerns recur in modern offline rendering. Analysis and synthesis must agree on sample rate, hop, window, phase convention, and channel order. A transform can be mathematically plausible yet fail because its states are not synchronized.

Wartime and postwar communications research also accelerated digital and statistical treatments of signals. Quantization noise, prediction, coding, and transmission became mature fields. The phase vocoder would eventually benefit from this environment even though its distinctive formulation arrived later. It inherited a culture in which speech could be decomposed, represented numerically, evaluated, and reconstructed.

The ethical context should not disappear from the technical story. Communications research was funded by military, governmental, and commercial priorities, while musical applications often arrived as secondary appropriations. Computer music repeatedly transformed tools built for control, transmission, or calculation into means of ambiguity and expression. That redirection is part of the phase vocoder’s cultural history.

Tape music and independent control by segmentation

The tape studio approached time and pitch through physical motion. Ordinary varispeed couples the two quantities because recorded wavelength is fixed on the medium. Faster playback raises pitch and shortens duration. Slower playback lowers pitch and lengthens duration. Composers embraced this coupling as an effect, but broadcasting, film synchronization, language research, and post-production often demanded independent control. A commercial might need to lose three seconds without making its announcer sound excited and small. A film voice might need to descend without making every phrase proportionally slower.

If a signal is recorded at tape speed v_r and reproduced by a stationary head at speed v_p , the elementary varispeed relationships are:

$$\frac{f_{\text{out}}}{f_{\text{in}}} = \frac{v_p}{v_r}, \quad \frac{T_{\text{out}}}{T_{\text{in}}} = \frac{v_r}{v_p}.$$

where f_{in} is the recorded frequency, f_{out} is the reproduced frequency, T_{in} is the recorded duration, T_{out} is the reproduced duration, v_r is recording speed, and v_p is playback speed. The two ratios are reciprocals. A one-octave rise obtained by doubling playback speed necessarily halves the duration.

This coupling was useful rather than merely restrictive. Disc and tape musicians used it to transpose voices, reveal high-frequency detail at slower rates, turn attacks into gestures, and produce instrumental registers unavailable to the original performer. Les Paul’s multitrack practice made speed transposition part of popular studio craft. Pierre Schaeffer’s studio used the phonogène family to play tape loops at selected speeds. The chromatic phonogène offered capstans associated with tempered pitch steps, while the continuously variable version permitted glissandi. These were powerful transposition instruments, but changing capstan speed still changed duration with pitch. They should not be mistaken for independent time-scale processors.

Physical editing as time compression.

The most literal way to shorten recorded speech without raising its pitch is to remove pieces of tape. An editor can cut out pauses, but that changes rhetoric and may soon become audible. A more systematic machine removes many short intervals distributed across the recording. If each omission is shorter than a syllable and the remaining boundaries join tolerably, the message can become faster while local waveform pitch remains approximately unchanged.

Expansion reverses the operation. Short intervals are repeated to create additional duration. Both procedures preserve the local speed at which tape crosses the playback head, so the pitch inside each retained segment stays close to the original. The global schedule changes because segments have been discarded or repeated.

For an input divided into segments of nominal duration L , a simple duration approximation is:

$$T_{\text{out}} \approx N_k L,$$

where T_{out} is output duration, N_k is the number of segments kept or emitted after deletion or repetition, and L is nominal segment duration. This expression hides the difficult part: finding joins that do not click, flutter, duplicate an attack, or interrupt a periodic waveform at an incompatible phase.

The mechanical sampling method anticipated later overlap-add and granular procedures. It did not analyze a spectrum, estimate instantaneous frequency, or propagate phase. Its unit of control was a short physical portion of recorded waveform. The phase relationships that mattered were present implicitly at the joins.

Rotating heads before the Springer machine.

Rotating-head playback has a complicated ancestry. Eduard Schüller patented an early rotating-head principle in 1938, and a wartime AEG Tonschreiber used related mechanics to slow rapid telegraphic or speech material for monitoring. Its purpose and operating conditions differed from later studio time regulators. The historical importance is that head motion could alter effective scanning speed without requiring the entire tape transport to move at that same speed.

Postwar speech researchers pursued related sampling machines. Grant Fairbanks, Wilbur L. Everitt, and Robert P. Jaeger described a rotating-head method for time or frequency compression and expansion of speech in 1954. Research interest was practical and perceptual: faster speech could reduce transmission or listening time, but ordinary acceleration raised pitch and distorted voice quality. The rotating assembly automated the removal or repetition of short intervals that a human tape editor could never cut at sufficient density.

These developments should not be reduced to one clean line of invention. Patents, laboratory prototypes, communications devices, and commercial machines overlapped. Some altered effective playback velocity; some sampled speech in short intervals; some were designed for intelligence or transmission rather than music. Anton Springer's achievement belongs inside this field but became distinctive through a practical regulator suitable for sustained program material.

Anton Springer, the Tempophon, and the Eltro.

Anton Marian Springer developed the machine variously marketed as the *Tempophon*, *Acoustic Time Regulator*, *Information Rate Changer*, and *Springer machine*. Eltro GmbH later marketed versions internationally. Early forms emphasized time compression and expansion, while subsequent machines also provided direct pitch shifting. The multiplicity of names partly explains why the device is often missing from simplified histories.

At the center of the machine was a rotating column carrying four small magnetic playback heads at 90-degree intervals. Tape contacted the head assembly over a limited arc. As one head left the tape, the next entered contact and took over playback. The tape transport controlled how quickly the recording as a whole passed through the machine. Rotation controlled head-to-tape velocity inside each short scan.

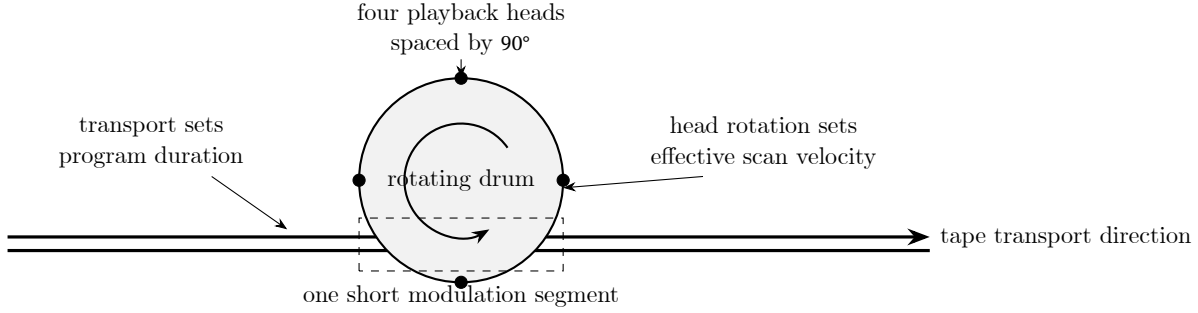


Figure 1.22: Operating principle of a Springer-type rotating-head regulator. The drawing is schematic: transport velocity and head rotation provide two mechanical degrees of freedom that ordinary fixed-head varispeed does not have.

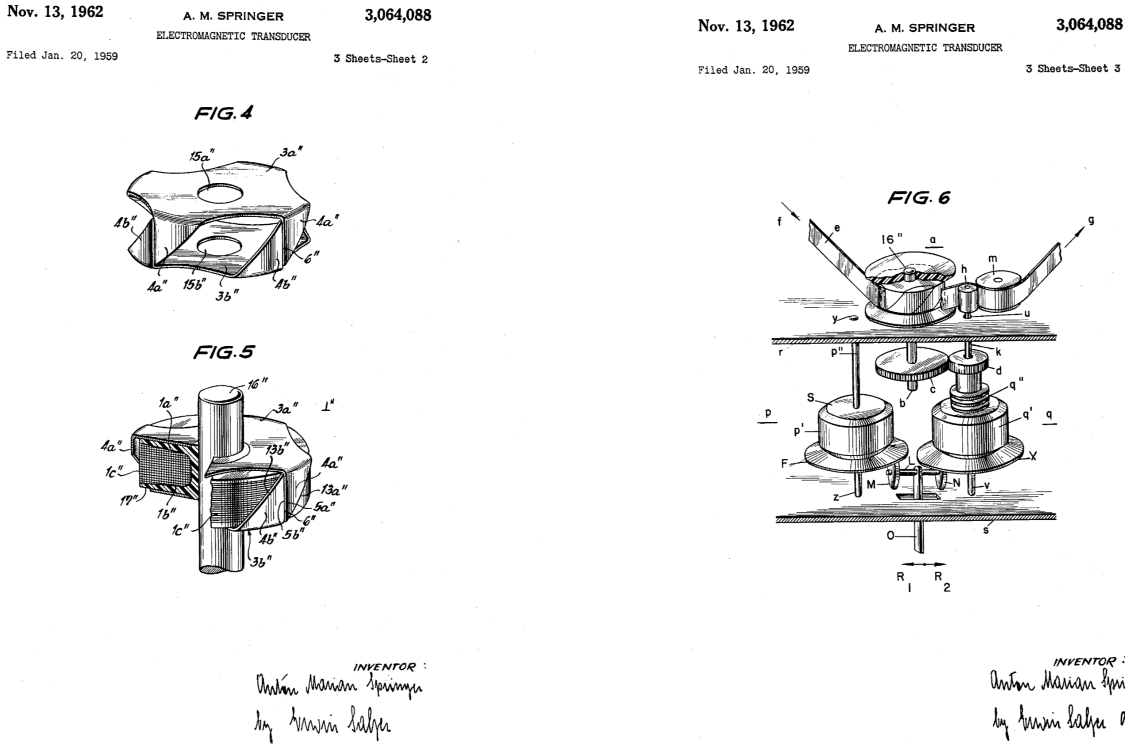


Figure 1.23: Anton Marian Springer's rotating multipolar transducer, left, and its geared tape-playback application, right. Figures 4 through 6 from [United States Patent 3,064,088](#), filed in 1959 and issued in 1962. United States patent drawings, public domain.

The useful separation can be expressed by distinguishing tape velocity from relative scan velocity:

$$\frac{f_{\text{out}}}{f_{\text{in}}} = \frac{v_{\text{rel}}}{v_r}, \quad \frac{T_{\text{out}}}{T_{\text{in}}} = \frac{v_r}{v_t}, \quad v_{\text{rel}} = v_t \pm v_h.$$

where v_{rel} is effective head-to-tape scan velocity, v_t is tape transport velocity, v_h is tangential head velocity, v_r is the original recording speed, and the sign depends on whether the active head moves with or against tape motion. The idealized equations omit head handoff and segment repetition, but they reveal the machine's conceptual breakthrough: pitch and duration depend on separately adjustable motions.

In pitch mode, the host transport could move tape at its normal rate while drum rotation changed relative scan speed. The same recorded length then occupied approximately its normal duration, but its cycles crossed the active head faster or slower and emerged at a new pitch. In tempo mode, differential

gearing coordinated capstan and drum so tape moved faster or slower while relative scan speed stayed near the recorded value. Duration changed while pitch remained approximately stable.

The head handoff was not a negligible detail. Springer’s system divided the recording into modulation segments of roughly 40 milliseconds. Repeated segments filled time during expansion; omitted segments removed time during compression. Contact angle influenced overlap between one head and the next, and engineers adjusted it for speech, music, or effects. Too little overlap exposed a gap or discontinuity. Too much overlap mixed neighboring scans and produced coloration.

The result had a recognizable mechanical signature. Regular head changes could impose flutter or a buzzing periodicity. Sustained tones exposed crossover modulation. Percussive events could be duplicated or truncated. Speech often tolerated these defects because intelligibility survives many local omissions, while dense music could make the splice rhythm more obvious. The artifact depended on source material, setting, tape condition, alignment, and operator judgment.

Studio work and the voice of HAL.

The Eltro’s ordinary work was often utilitarian. Radio and television spots had to fit fixed durations. Educational projects investigated speeded listening. Film dialogue had to follow edits or altered picture rates. The machine processed one mono pass that was commonly recorded onto another tape machine, so a transformation was also a transfer generation.

Wendy Carlos’s account of using an Eltro Mark II at Gotham Recording Studios provides unusually concrete operating testimony. She describes separate pitch and tempo modes, a calibrated control marked in musical intervals and percentage duration, and physical adjustment of the tape-wrap angle around the four-head drum. Her account also identifies the Eltro in Stanley Kubrick’s *2001: A Space Odyssey*. Douglas Rain’s performance as HAL received a subtle global time expansion, and the computer’s final decline combined a more extreme downward pitch pass with a separate duration-expansion pass.

That example demonstrates why independent controls matter artistically. Ordinary tape slowing would force pitch and duration to fall by reciprocal amounts. HAL needed two trajectories with different shapes and degrees. Processing in separate passes let pitch approach collapse while speech duration expanded less drastically. The machine turned a post-production correction technology into a dramatic model of failing cognition.

Gabor, grains, and an optical neighbor.

Dennis Gabor’s 1946 and 1947 work on communication and acoustical quanta provided another route toward local time-frequency thinking. A finite packet of sound has both temporal extent and spectral spread. Gabor’s experimental kinematical frequency-conversion work used short acoustic units in an electromechanical or optical setting rather than an FFT. The historical connection to granular synthesis and later time-frequency processing is conceptual as well as technical: continuous sound can be treated as a succession of bounded events.

The Springer regulator and Gabor’s acoustic quanta are sometimes described together as ancestors of granular processing. That description is useful if kept precise. The Springer machine physically scanned short tape segments and handed playback among rotating heads. Gabor developed a general language of localized quanta and an experimental converter. Neither was a phase vocoder, and neither computed a short-time complex spectrum. Both weakened the assumption that recorded sound must be reproduced as one indivisible, uniformly moving object.

Analog delay lines and Doppler transposition.

Mechanical tape was not the only pre-software route. A continuously changing analog delay produces a Doppler-like pitch change. For a delay trajectory $d(t)$, an ideal variable-delay output is:

$$y(t) = x(t - d(t)), \quad f_{\text{out}}(t) = f_{\text{in}}(t) \left(1 - \frac{d}{dt} d(t) \right).$$

where $x(t)$ is the input, $y(t)$ is the output, $d(t)$ is time-varying delay, $f_{\text{in}}(t)$ is local input frequency, and $f_{\text{out}}(t)$ is the resulting local output frequency under the idealized delay model. Increasing delay makes the read point fall behind and lowers pitch. Decreasing delay makes it catch up and raises pitch.

A finite delay cannot ramp forever. It reaches an endpoint and must reset, which would create a discontinuity. Practical transposers use multiple read paths whose ramps are staggered and crossfaded. While

one path resets, another carries the output. Magnetic delay, rotating storage, and later bucket-brigade or digital delay technologies implemented variations of this strategy. The repeated crossfade has the same family resemblance as rotating-head handoff and waveform overlap-add.

Tape flanging and chorus use related delay modulation but are not constant pitch transposition. Their delay trajectories oscillate, so pitch deviation rises and falls while delayed and direct signals interfere. A Leslie cabinet uses physical motion to create changing Doppler shift and amplitude, again producing modulation rather than a fixed musical interval. These effects belong to the history because they made time variation audible, but they solve a different problem from independent file-length control.

Frequency shifting is not pitch shifting.

Analog frequency shifters form another neighboring lineage. Harald Bode described frequency shifters based on single-sideband techniques, quadrature phase networks, multipliers, and quadrature oscillators. Such a device adds or subtracts a fixed number of hertz from every spectral component:

$$f'_k = f_k \pm f_s.$$

where f_k is input partial frequency k , f'_k is its shifted output frequency, and f_s is the fixed frequency-shift amount. A musical pitch transposer instead approximately multiplies every partial by one ratio, $f'_k = \rho f_k$, where ρ is the transposition ratio. Constant addition destroys ordinary harmonic spacing except in special cases; constant multiplication preserves it.

Bode's historical account explicitly distinguished the Springer apparatus from frequency shifting. The rotating-head machine demonstrated transposition by dividing program material into short splices, compressing or expanding them, and recombining them. The electronic frequency shifter used heterodyning or quadrature cancellation. Both could make a voice or instrument uncanny, but their spectra moved according to different laws.

What analog methods taught digital audio.

The analog and electromechanical era established a vocabulary of problems that later software inherited. Ordinary varispeed demonstrated the coupled baseline. Distributed cutting and repetition demonstrated time modification by local scheduling. Rotating heads supplied separate transport and scan velocities. Variable delays demonstrated pitch change through a moving read point. Crossfades hid finite read-window resets. Frequency shifters clarified the difference between additive and multiplicative spectral motion.

The following comparison keeps these mechanisms separate:

Method	Primary mechanism	Pitch and duration
Fixed-head varispeed	Change tape or disc speed	Necessarily coupled.
Physical splice	Remove short waveform intervals	Duration changes; local pitch is mostly retained.
Springer or Eltro regulator	Coordinate tape transport and rotating heads	Independently adjustable within mechanical limits.
Phonogène	Select or vary capstan speed	Coupled, often calibrated musically.
Variable analog delay	Move a delayed read point and crossfade resets	Pitch changes while the program clock can remain fixed.
Analog frequency shifter	Use single-sideband or quadrature translation	Frequencies move by a fixed hertz offset.
Phase vocoder	Estimate and reschedule short-time spectral trajectories	Independently adjustable in a numerical model.

Table 1.2: Mechanical, analog, and digital approaches to controlling recorded pitch and duration.

The comparison also reveals why no single analog precursor is simply a phase vocoder made of metal. The Springer machine most closely anticipates waveform segmentation and overlap-add. Gabor's work anticipates localized time-frequency representation. Filter-bank vocoders anticipate analysis and resynthesis. Analog frequency shifters anticipate direct spectral transformation. The digital phase vocoder assembled different parts of this inheritance around complex short-time phase.

This family of methods established practical lessons that remain current. Time modification can be accomplished by changing the schedule of local units. Units must overlap or meet at compatible waveform positions to hide joins. A unit length appropriate for voiced speech may be poor for noise or percussion. Periodic switching leaves periodic artifacts. Controls that seem independent mathematically can still interact perceptually.

Tape studios added another crucial practice: repeated audition. A transformation was judged in relation to material, not only by a generic specification. Engineers aligned heads, marked tape, rehearsed edits, and compared transfers. The command-line render inherits this iterative studio method. A phase-vocoder parameter is historically continuous with a physical adjustment in the sense that both become meaningful only in a particular sound.

Sampling, digital audio, and the FFT threshold

Digital signal processing supplied the numerical conditions for a general phase-aware spectral instrument. Sampling represented a waveform as a sequence of numbers. The discrete Fourier transform represented a finite block as complex frequency coefficients. Overlap-add and filter-bank theory explained how blocks could reconstruct a continuous signal. The remaining obstacle was computational cost.

The publication of the Cooley-Tukey fast Fourier transform algorithm in 1965 made efficient Fourier calculation widely available, though related fast methods had earlier precedents. The historical point is not that the FFT instantly created digital audio. Rather, it moved repeated spectral analysis from an extravagant operation toward a practical building block. This mattered enormously for a system that required one transform after another across an entire recording.

Mainframe and minicomputer audio remained expensive. Samples occupied scarce storage, converters were specialized, and a render could take far longer than its program duration. Offline work encouraged a particular software architecture: analyze a sound into an intermediate file, perform transformations on that representation, and resynthesize later. Spectral data became an artifact that could be archived and edited.

This architecture shaped musical thought. A spectrum was no longer merely an explanatory graph. It became material with persistence. A composer could return to analysis data, alter selected bands, interpolate frames, or exchange features between sounds. The file-oriented ancestry remains visible in modern analysis caches, response artifacts, and resumable render plans.

Table 1.3: Representational thresholds in the long history of the phase vocoder.

Period	Convenient representation	New practical possibility
Nineteenth century	harmonic components and resonances	relate waveform complexity to pitch and timbre
Early telephony	filter-bank channel energies	transmit spectral control rather than a complete waveform
Tape era	short physical segments	alter duration through local rearrangement
Early digital era	sampled waveforms and DFT blocks	calculate and store complex spectra
FFT era	repeated short-time transforms	analyze and resynthesize complete sounds efficiently
Workstation era	editable spectral files	compose with trajectories, peaks, envelopes, and frames
Contemporary systems	adaptive and multichannel models	vary resolution, coherence, and processing policy over time

Flanagan and Golden in the Bell Labs setting

James L. Flanagan and Robert M. Golden's 1966 *Phase Vocoder* belongs to Bell Labs speech research but introduced a representation with consequences beyond transmission. The system analyzed short-time amplitude and phase, represented phase change as frequency information, and resynthesized speech after manipulation. Its concern with phase distinguished it from a channel-envelope vocoder.

The word *phase* in the title should be understood dynamically. Absolute phase at one instant is less useful than phase change over time. A channel's phase derivative describes local frequency. This allows a component between nominal channel centers to be represented more accurately than a fixed filter label alone would permit. It also provides the information needed to continue that component on a modified time grid.

The paper included time expansion and compression among its applications. That placed independent temporal control inside an analysis-synthesis framework before digital music studios could use the method routinely. The publication date, one year after the influential Cooley-Tukey FFT paper, marks a threshold at which phase-aware spectral representation and fast computation were becoming mutually relevant.

Flanagan's broader work on speech analysis and synthesis provided context. The phase vocoder was one element in a sustained effort to understand speech production, perception, and coding. Later musical accounts sometimes detach the algorithm from speech history, but doing so hides why formants, voiced-unvoiced distinctions, intelligibility, and channel behavior entered the field so early.

Portnoff and the practical digital formulation

Mark Portnoff's work in the 1970s gave the digital phase vocoder a form recognizable to present-day implementers. His 1976 paper described an FFT implementation, and his 1980 work addressed time-scale modification of speech through short-time Fourier analysis. Windows, overlapping frames, phase differences, and reconstruction were organized into a block-processing method.

Portnoff's contribution was not merely speed. An FFT-oriented description standardized the objects a programmer manipulated. One had arrays of complex bins rather than a diagram of idealized continuous filters. Expected phase advance could be computed from bin index and hop size. Residual phase could be wrapped to a principal interval. Synthesis phase could be accumulated.

That concreteness exposed implementation questions that continue to matter: how frames are centered, how boundaries are padded, whether the window is periodic or symmetric, how overlap is normalized, and what happens when a bin has negligible magnitude. Different programs could implement the same paper and still disagree audibly at transients or file edges.

Portnoff's speech examples also kept evaluation tied to intelligibility and naturalness. Musical uses would add other criteria, but they did not replace these. Dialogue stretching, language learning, accessibility, and synchronization still ask for the transparent operation that motivated early work.

James A. Moorer and the computer-music turn

James A. Moorer's 1978 article on the use of the phase vocoder in computer music helped move the technique into a compositional frame. Computer music had already developed synthesis languages, digital recording systems, and analysis tools. The phase vocoder joined these as a way to derive a manipulable description from recorded sound.

The computer-music turn changed the questions asked of the algorithm. How far could time be extended before a source became texture? Could amplitude trajectories from one sound control another? Could individual spectral regions be shifted independently? Could analysis reveal structures that notation did not capture? Such questions treated the intermediate representation as an instrument.

Moorer's work also belongs to a wider period of laboratory software development in which algorithms traveled through reports, code listings, shared systems, and personal contact. A method was not disseminated only by a journal paper. It moved when a researcher visited another studio, when code was ported to a new machine, or when a composer learned to interpret an analysis file.

This social mode of transmission explains why software genealogies are often difficult to reconstruct. Program names changed, local modifications went unpublished, and an algorithm described in one institution might be rewritten elsewhere under different conventions. A responsible history distinguishes documented lineage from plausible influence.

UC San Diego, CARL, and a software culture

The Computer Audio Research Laboratory at the University of California, San Diego developed an influential environment for computer music and audio processing. CARL software included phase-vocoder tools and encouraged the Unix model of composable programs. Analysis, transformation, and synthesis could be separated into commands connected by files and scripts.

This tool culture matters to pvx directly. A command-line program makes an algorithm repeatable and inspectable. Parameters can be recorded in shell history, placed under version control, swept across a corpus, or reused in a composition. The result is not automatically artistically better, but the procedure becomes easier to reconstruct.

CARL also illustrates the importance of intermediate formats. Spectral analysis data could be passed among operations rather than hidden inside one monolithic application. Such modularity encourages experimentation because a user can insert an unusual step between established ones. It also creates compatibility obligations: frame metadata, bin conventions, and phase interpretation must remain consistent.

The laboratory context brought researchers, composers, and software developers into contact. Phase-vocoder history at UCSD therefore includes both algorithmic refinement and a way of working. Tools were not merely products delivered to passive users. They were objects of study that musicians could modify.

Mark Dolson: research, software, and pedagogy

Mark Dolson's early 1980s research examined a tracking phase vocoder and the analysis of ensemble sounds. Tracking addressed a difficulty hidden by fixed-bin language: a physical partial can move from one bin to another. Treating every bin as a permanent oscillator confuses representation with source. Peak and trajectory models attempt to follow the component instead.

Dolson's work connected several roles. He investigated algorithms, contributed to software practice, explored musical applications, and wrote the 1986 tutorial that became the most widely cited introduction for computer musicians. The tutorial did more than simplify mathematics. It organized a scattered practice into a teachable family of transformations.

Pedagogical texts shape technology because they determine what a generation considers normal. Dolson's account made amplitude and frequency trajectories central. It showed time scaling and pitch shifting as related operations and described cross-synthesis and spectral modification as natural extensions. Readers could imagine the phase vocoder as a general-purpose instrument rather than a specialized speech coder.

The tutorial also preserved the offline, file-oriented imagination of its period. That can seem remote in an age of real-time plug-ins, but it encouraged careful transformations impossible to perform interactively on available hardware. Offline time was not only a limitation. It allowed ambitious calculations, long sources, and compositional planning.

Sinusoidal modeling and a neighboring lineage

The phase vocoder developed beside sinusoidal analysis-synthesis. McAulay and Quatieri modeled speech as tracked sinusoids. Xavier Serra and Julius O. Smith's spectral modeling synthesis separated deterministic sinusoidal components from a stochastic residual. PARSHL and related systems gave musicians explicit control over peaks and tracks.

These methods share short-time spectra with the phase vocoder but interpret them differently. A classic phase vocoder propagates dense bin phases. A sinusoidal model identifies peaks, estimates their parameters, connects them into tracks, and resynthesizes oscillators. The residual accounts for energy not well described by stable sinusoids.

The neighboring lineage influenced phase locking, peak tracking, formant processing, and hybrid transformation. It also clarified that one analysis model need not explain all parts of a sound equally. A violin tone, bow noise, room response, and attack may demand different representations.

Modern systems frequently combine these ideas without announcing a strict category. They may use an STFT frame, identify sinusoidal peaks, classify transients and noise, propagate phases for one region, and resynthesize another region stochastically. The history is therefore braided rather than competitive.

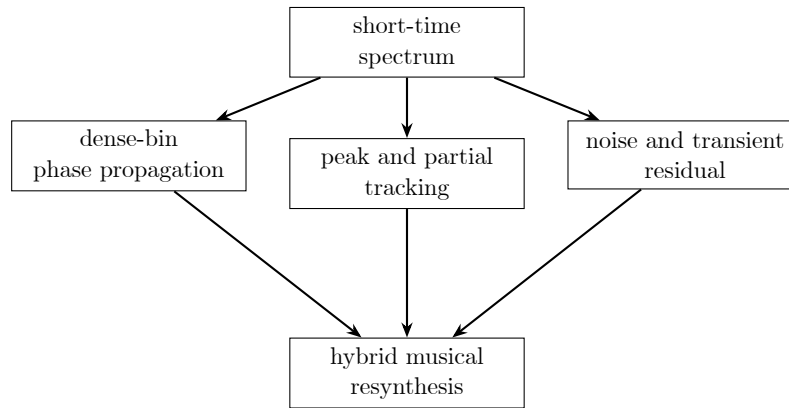


Figure 1.24: Phase-vocoder, sinusoidal-modeling, and residual-processing lineages increasingly converged in hybrid systems.

IRCAM and the institutionalization of spectral transformation

IRCAM provided another major setting in which spectral analysis became a compositional resource. Research, software development, and commissioned music were unusually close. Tools could be tested through demanding artistic projects, while compositional questions could motivate new analysis and transformation methods.

Accounts of the Super Phase Vocoder lineage commonly connect Moorer’s early p-voc, CARL-related work, Dolson’s collaboration at IRCAM, developments by Philippe Depalle, and later work by Axel Roebel. Exact software ancestry should be stated cautiously because systems were rewritten and extended. What is well documented is that SuperVP became an advanced engine for analysis, time scaling, transposition, filtering, cross-synthesis, and source-filter transformations.

AudioSculpt placed such processing behind visual analysis and editing. Spectrograms, markers, and treatment parameters made the intermediate representation available to users who did not write signal-processing code. SuperVP also entered real-time Max environments through modules for playback, scrubbing, transformation, and cross-synthesis.

The IRCAM trajectory shows how a research algorithm becomes infrastructure. A mature engine includes file handling, parameter conventions, real-time state, quality modes, envelope estimation, transient classification, and documentation. The core phase update remains important, but reliability emerges from the surrounding system.

Jonathan Harvey and spectral ambiguity

Jonathan Harvey’s *Mortuos Plango, Vivos Voco* (1980) is a landmark of computer music made at IRCAM. The work draws on recordings of the great tenor bell of Winchester Cathedral and the voice of Harvey’s son, a chorister there. Analysis of the bell supplied partial frequencies that shaped the piece’s pitch organization, while transformations brought voice and bell into ambiguous relation.

The piece should not be described as one continuous phase-vocoder demonstration. Its realization involved FFT analysis, Music V, synthesis, transposition, looping, and other processes. The phase-vocoder context is nevertheless important because spectral analysis and time-pitch transformation made the recorded sources structurally available.

Its historical force lies in the relation between technology and form. The bell is not merely an effect sample. Its inharmonic spectrum becomes harmonic material. The voice is not simply speech laid over electronics. Its vowels and partials become a route between human and metallic identities. Computer analysis mediates a spiritual and acoustical idea.

For a phase-vocoder listener, the work offers a discipline: attend to when source recognition remains stable, when it becomes ambiguous, and when a hybrid identity emerges. Those transitions are more historically revealing than trying to label every sound by one algorithm.

Roger Reynolds and the genealogy of *Transfigured Wind*

Roger Reynolds's *Transfigured Wind* series developed through several versions in the 1980s. The Library of Congress preserves an unusually rich genealogy of the work, including plans, commentary, audio, and discussion of computer components. Reynolds used recorded flute material and processes including phase-vocoder analysis and resynthesis, along with editorial algorithms such as SPLITZ and SPIRLZ.

The project makes transformation audible as formal thought. A flute gesture can be decomposed, prolonged, separated into layers, and placed in relation to live performance. Reynolds emphasized that the computer sounds derive from recorded flute. Transformation enlarges the instrument's world without replacing its identity with an unrelated synthesizer.

The archival record is historically valuable because it shows decisions, revisions, and versions rather than only a finished master. It reveals the labor of choosing source gestures, categorizing them, editing outputs, and coordinating fixed media with performers. This corrects the fantasy that an algorithm produces a composition automatically.

The series also demonstrates how phase-vocoder time operates at multiple scales. A single attack can become a spectral object. A phrase can be expanded. Layers can move at different speeds. Large form can be planned around the return and transformation of recorded identity.

Trevor Wishart and sound morphing

Trevor Wishart's work gives one of the clearest artistic accounts of phase-vocoder data as material. In writings on *Vox 5* and later retrospectives, Wishart described exploring the contents of analysis files and developing tools for sound morphing. The aim was not simply to stretch a voice cleanly. It was to move continuously among vocal, animal, mechanical, and environmental identities.

Morphing depends on selecting what changes and what remains continuous. Spectral envelopes, partial frequencies, noise, duration, and gesture can move at different rates. A convincing transformation may preserve the rhythm of one source while gradually adopting the spectrum of another. The phase-vocoder representation makes several of these dimensions separately addressable.

Vox 5 became a canonical example because the transformation participates in a larger vocal imagination. Extended performance, speech, and computer processing occupy the same world. The technology does not function as an external accompaniment. It expands the possible anatomy of the voice.

Wishart's later account, extending from early experiments to *Supernova*, also reminds historians that techniques mature through sustained personal toolmaking. A published algorithm is only a starting vocabulary. Composers construct families of operations, test perceptual thresholds, and develop conventions specific to their work.

JoAnn Kuchera-Morin and expanded duration

JoAnn Kuchera-Morin's *Dreampaths* (1989) is frequently cited for extensive early phase-vocoder transformation. Accounts describe dramatic expansion of recorded duration, spectral glissandi, and the alteration of instrumental and vocal sounds. Her broader work joins composition, computer systems, and immersive media.

Extreme stretching changes the listener's scale of attention. A thirty-second source expanded to several minutes no longer functions as an ordinary phrase. Vibrato becomes a slow trajectory, breath becomes

texture, and room noise becomes part of the orchestration. The phase vocoder acts less like a correction tool than a microscope for temporal structure.

Such work also reveals the limits of simple quality language. Smearing may be unacceptable in dialogue and productive in a dreamlike electroacoustic texture. Historical interpretation must ask what the composition values. Fidelity can mean fidelity to source identity, to gesture, to spectral detail, or to a planned transformation.

Kuchera-Morin’s practice broadens the institutional map beyond Bell Labs, Stanford, UCSD, and IRCAM. Phase-vocoder history was made by composers and researchers across universities and studios, including figures whose contributions were not always centered in standard textbook narratives.

Puckette, phase locking, and real-time environments

Miller Puckette’s phase-locked vocoder work in the mid-1990s addressed the loss of coherence that made stretched tonal sounds diffuse. It belongs to a broader period in which real-time FFT processing became available in flexible environments such as Max. Spectral transformation could increasingly respond during performance rather than only after an offline render.

Phase locking groups bins around spectral peaks so they do not evolve as unrelated oscillators. The idea drew phase-vocoder practice closer to sinusoidal modeling while retaining an STFT resynthesis framework. It made explicit that neighboring bins often describe one windowed partial.

Real-time operation changed design priorities. Latency became audible to performers. Peak detection and phase updates had to complete before the next block. Parameters needed smoothing because an abrupt control gesture could produce a discontinuity. User interfaces had to expose enough control without requiring interpretation of every bin.

The history of Max, Pure Data, Csound, and related systems is therefore part of the phase vocoder’s history. They made spectral operations available as reusable objects or opcodes. A technique formerly encountered through specialist code could enter an improvisation, installation, or classroom patch.

Laroche and Dolson: from artifact to explanatory model

Jean Laroche and Mark Dolson’s work in the late 1990s gave phasiness a more precise explanation. The classic algorithm could maintain phase continuity for each bin across time while allowing bins within a spectral peak to lose their relative organization. The output retained horizontal coherence but weakened vertical coherence.

This distinction turned a musician’s complaint into an engineering model. The diffuse quality was not merely the inevitable sound of Fourier processing. It followed from a particular independence assumption. Identity and scaled phase-locking methods offered alternative relationships around spectral peaks.

The 1999 work on improved time-scale modification became a reference point for later algorithms and evaluations. It also influenced real-time systems such as PhaVoRIT, which combined scaled phase locking with interactive concerns and perceptual testing.

The historical lesson is methodological. Artifacts can be evidence. A repeated perceptual description may indicate that the representation has broken a relationship listeners rely on. Naming that relationship can lead to a better algorithm and a better listening vocabulary.

Transients become a separate research object

As phase locking improved tonal coherence, transients remained conspicuous. An onset is localized in time and spread in frequency, almost the opposite of a stationary sinusoid. A long analysis window mixes samples before and after the event. When synthesis frames are moved apart, the onset’s energy can be distributed across a longer interval.

Research by Chris Duxbury, Mike Davies, Mark Sandler, Axel Roebel, and others treated transient detection and preservation as explicit problems. Proposed strategies included detecting attacks, resetting

phase, moving transient frames without stretching them, separating transient and steady components, and using multiple resolutions.

Roebel’s work emphasized transient processing within the phase-vocoder framework. Later adaptive systems varied analysis according to transience or corrected magnitudes with multiresolution information while retaining a coherent phase path. The field increasingly accepted that one global resolution was a convenience rather than an acoustical truth.

This research also changed software interfaces. A user might now choose sensitivity, protection duration, reset mode, or hybrid behavior. Such parameters are historical sediment: each exists because a class of sounds exposed a weakness in an earlier default.

Table 1.4: Artifacts as historical research questions.

Perceptual report	Representation problem	Research response
diffuse or reverberant tone	neighboring bins lose relative phase organization	peak-based identity and scaled phase locking
smeared attack	one long frame treats an onset as stationary content	transient detection, phase reset, and hybrid routing
metallic noise	noise is forced into deterministic bin trajectories	stochastic or noise-aware resynthesis
shifted vowel identity	harmonics move with the broad spectral envelope	source-filter separation and envelope preservation
unstable stereo center	channels are transformed independently	shared timing, reference phase, and interchannel constraints

WSOLA, PSOLA, granular methods, and productive rivalry

The phase vocoder has never been the only method for time or pitch modification. Time-domain overlap-add methods align waveform segments. WSOLA selects segments by waveform similarity. PSOLA works near pitch-synchronous positions and became influential in speech synthesis and modification. Granular methods reorganize short grains with flexible timing and envelopes.

Each method makes different assumptions. PSOLA can preserve voiced speech efficiently when pitch marks are reliable, but irregular or polyphonic material complicates the model. WSOLA can retain attacks and local waveform character, but large transformations or sustained complex tones may reveal repetition and alignment choices. Granular methods embrace a textural unit whose size becomes part of the sound.

Rivalry improved phase-vocoder research by clarifying evaluation. A method might excel on sustained harmony and lose on drums. Another might preserve speech attacks and struggle with polyphony. Hybrid systems emerged because source categories did not respect algorithm names.

Commercial time-stretching increasingly concealed these choices behind quality modes or source presets. The user selected solo voice, polyphonic music, rhythmic material, or extreme stretch, and the engine changed its internal strategy. pvx exposes more of the policy because reproducibility and experimentation benefit from explicit controls.

Spectral envelopes, formants, and the source-filter return

Pitch shifting made an old speech problem newly audible. Moving every spectral feature upward can make a voice seem smaller; moving everything downward can make it seem larger or muffled. The harmonic series and the broad vocal-tract envelope do not play the same perceptual role.

Cepstral methods and later true-envelope estimation allowed systems to separate fine harmonic structure from broad resonant shape. Axel Roebel and Xavier Rodet developed efficient spectral-envelope estimation and applied it to pitch shifting with envelope preservation. Shape-invariant voice transformation extended the idea.

This line reconnects the phase vocoder to Dudley’s source-filter model. The technology has changed from filter-bank envelopes to high-resolution short-time spectra, but the conceptual split remains: excitation can move in pitch while resonant identity follows another rule.

Formant preservation is not always transparency. A composer may deliberately move formants against pitch, exchange envelopes between sources, or animate vocal size. Once the envelope is explicit, identity becomes another trajectory available for design.

Noise, residuals, and the limits of sinusoidal phase

Noise exposes another limit. A stable sinusoid benefits from a coherent phase trajectory. Broadband noise does not consist of partials that should all remain phase-locked in the same way. Stretching random phase as though it were deterministic can produce metallic or frozen coloration.

Researchers explored stochastic representations, noise-specific phase treatment, and hybrid decomposition. Gaussian-noise stretching became a specialized topic because ordinary phase propagation changes statistical character. Sinusoidal plus residual models offered one route: track coherent peaks and synthesize the remaining energy as shaped noise.

The problem is musically important. Breath, bow scrape, cymbal wash, reverberation, and environmental sound contain mixtures of stable and stochastic behavior. A processor that preserves only the pitched center may remove the material’s scale or intimacy.

Modern quality often depends on classification across time and frequency rather than one decision for an entire frame. A region can be treated as sinusoidal, transient, or noisy, then crossfaded with neighboring decisions. The phase vocoder becomes a framework for local policies.

Stereo, spatial hearing, and multichannel history

Early formulations commonly assumed a single channel. Stereo and multichannel production added phase relationships that encode direction, width, and ambience. Processing channels independently can move the center image, change apparent width, or weaken mono compatibility.

Spatial coherence research draws on time-delay estimation, array processing, and spatial audio as well as phase-vocoder work. Strategies include using one reference channel for peak and phase decisions, preserving interchannel phase differences, or treating diffuse components separately from localized ones.

There is no universal spatial rule. A centered vocal may require strong locking. Diffuse reverberation may become unnaturally narrow if forced to follow one channel. Low-frequency coherence may matter differently from high-frequency decorrelation. Multichannel transformation therefore made policy and source analysis even more important.

The historical shift from mono speech to immersive media is substantial. The same short-time representation now participates in binaural production, surround installations, microphone arrays, and spatial machine-learning data. Channel count multiplies both creative possibility and failure modes.

Open tools, standards, and software migration

Phase-vocoder techniques spread through Csound, CARL, the Composers’ Desktop Project, IRCAM Forum software, Max, Pure Data, SoundHack, GRM tools, research toolboxes, and code examples. Each environment emphasized different workflows: score-driven synthesis, Unix commands, graphical patching, offline transformation, or plug-in use.

Open and academic tools preserved techniques that might otherwise have disappeared with obsolete hardware. They also exposed implementation differences. One program stored amplitude and frequency, another magnitude and phase. One used oscillator-bank resynthesis, another inverse FFT. File formats and window conventions did not always travel cleanly.

Porting became a form of scholarship. To move a phase vocoder to a new machine, a developer had to interpret old code, documentation, and numerical assumptions. Bugs could become characteristic sounds; fixes could break compatibility with archived analysis files.

Standards such as SDIF attempted to make sound descriptions portable. The broader lesson for pvx is that an analysis artifact needs schema, units, version, and provenance. Spectral data without interpretation is not a durable historical object.

From offline render to interactive instrument

By the 1990s and 2000s, faster processors and dedicated DSP hardware made real-time phase-vocoder use increasingly practical. Interactive time stretching appeared in performance systems, DJ tools, games, installations, and musical interfaces. PhaVoRIT explicitly investigated real-time interactive stretching and user evaluation.

Real time is not simply offline processing performed faster. It changes what can be known. Future transients may not yet be available for lookahead. A performer expects bounded latency. Control values can move abruptly. A dropped block is more damaging than a slow offline job.

Sliding-transform methods explored different latency and update structures. GPU implementations pursued throughput. Real-time SuperVP modules brought advanced transformations to Max. Such systems turned phase-vocoder state into part of an instrument that had to respond continuously.

Interactive control also shifted musical agency. A stretch factor could follow a runner's pace, a gesture sensor, or another performer. Time transformation became relational rather than predetermined. The output clock could negotiate with an external clock.

Adaptive and nonstationary time-frequency analysis

One fixed window embodies one compromise. Adaptive and nonstationary approaches attempt to vary that compromise according to frequency or signal behavior. Constant-Q methods provide frequency-dependent resolution. Nonstationary Gabor frames offer invertible representations with changing windows. Multiresolution phase vocoders combine analyses to protect transients and resolve sustained partials.

These approaches are historically significant because they challenge the idea that the STFT grid is the sound. The grid is a measurement choice. A bass partial, a cymbal attack, and a consonant may each become clearer under a different observation scale.

Adaptive methods also create new coherence problems. Results from different resolutions must be combined without phase contradiction. Some systems retain one phase path and use other resolutions to correct magnitude. Others build mathematically consistent nonstationary frames.

The increasing mathematical sophistication does not remove listening. Adaptation criteria still encode assumptions about transience, harmonicity, and relevance. A detector trained on drums may misread a bowed attack. Historical progress remains a dialogue between formal guarantees and material exceptions.

Machine learning enters the surrounding workflow

Contemporary audio systems often use machine learning around, beside, or in place of classical transforms. Neural source separation can provide stems before time modification. Pitch models can supply more robust trajectories. Learned transient or quality models can guide parameters. Neural vocoders synthesize waveforms from acoustic features, though their use of the word *vocoder* belongs to a different lineage.

The shared name can cause confusion. A neural speech vocoder such as WaveNet, HiFi-GAN, or related systems reconstructs audio from learned acoustic representations. A phase vocoder propagates phase in a short-time spectral transformation. Both are analysis-synthesis ideas, but their models, training requirements, and artifacts differ.

Hybrid workflows are more historically interesting than replacement stories. A learned separator can isolate voice and drums; a phase vocoder can then use different policies for each. A learned pitch tracker can drive formant-aware transformation. Classical transforms provide determinism and inspectable controls where a model may provide classification or estimation.

The phase vocoder remains useful partly because its failures are legible. Window, hop, phase, and transient decisions can be inspected. That transparency matters in research, education, dataset generation, and reproducible production.

A historiography of claims and cautions

Technical histories often become too tidy. They assign one invention to one person, treat later papers as complete replacements, and describe musical works as demonstrations of a named algorithm. The documentary record is messier. Ideas were developed in parallel, software was rewritten locally, and compositions combined many processes.

Several cautions improve accuracy. Dudley’s vocoder is a conceptual ancestor, not an FFT phase vocoder. Cooley and Tukey popularized a fast algorithm but did not originate every FFT factorization. IRCAM’s SuperVP has a lineage of many contributors rather than one author. A work made in a phase-vocoder studio need not use that process in every passage.

The distinction between invention and dissemination also matters. Flanagan and Golden formulated the phase vocoder. Portnoff made an FFT implementation practical and explicit. Moorer and Dolson helped make the technique meaningful to computer musicians. Puckette, Laroche, Dolson, Roebel, and many others improved coherence and transient behavior. Tool builders brought it to wider communities.

Finally, user practice is part of history. Defaults, presets, forum explanations, code examples, and listening habits determine which algorithms survive. A technically superior method can disappear if it is difficult to integrate. A simple implementation can become canonical because generations learn from it.

Reading the foundational papers as historical documents

A list of publication dates cannot show how the phase vocoder changed. The major papers differ not only in method but also in audience, vocabulary, diagrams, examples, and assumptions about available machines. Reading them in sequence reveals a change from communications engineering to an increasingly musical and perceptual discipline. It also reveals continuity. Questions about intelligibility, parameter economy, and reconstruction never vanished when composers entered the story.

Flanagan and Golden’s 1966 paper belongs to an environment in which speech communication was a central technical object. Its diagrams describe analysis and synthesis channels, phase derivatives, and signal transmission. Time expansion appears as an application of a representation designed to preserve more information than a magnitude-only channel vocoder. The paper’s historical originality lies in treating phase change as useful transmitted information and in showing how that information could support a modified reconstruction.

Portnoff’s publications move the account closer to code. The FFT makes a bank of narrow channels computationally regular, and the short-time transform places analysis in a repeated frame structure. A reader can recognize the modern ingredients: windowed blocks, overlaps, complex spectra, expected phase advance, and synthesis. The papers also preserve the intellectual connection between a modulated filter bank and a discrete transform. That connection matters because the phase vocoder is not simply an image editor for spectrograms. It is an analysis-synthesis system whose channels must agree over time.

Allen and Rabiner’s account of short-time Fourier analysis and synthesis provided a broader frame in which these operations could be understood. Their work helped make reconstruction conditions, time-varying spectra, and overlap processing part of a shared signal-processing vocabulary. It supplied intellectual infrastructure even when a later implementation did not cite every detail directly.

Moorer’s computer-music article changes the implied reader. The important question is no longer only whether speech can be transmitted or expanded intelligibly. It is what a composer can do with an analysis. Spectral modification, time scaling, and resynthesis become parts of a studio process. The article appears during a period when computer music systems were becoming extensive enough to support libraries of reusable transformations, yet scarce enough that access to a laboratory still determined who could experiment.

Dolson’s tutorial is a document of consolidation. It translates a technically scattered field into a coherent explanation for musicians and programmers. The tutorial’s enduring value comes from the way it joins

pictures, equations, implementation ideas, and audible consequences. It presents the phase vocoder as a family of operations on trajectories rather than as one fixed effect. This pedagogical act helped establish the conceptual interface later programs inherited.

Griffin and Lim’s work on estimating a signal from a modified short-time magnitude addresses a related but distinct problem. If an operation changes magnitude without retaining a compatible phase, inversion becomes an estimation task. The iterative algorithm alternates between time-domain consistency and a desired magnitude. Its place in history is important because it demonstrates that an arbitrary spectrogram is not necessarily the transform of any ordinary waveform. Spectral editing therefore has geometrical constraints, not merely aesthetic ones.

Puckette’s phase-locked approach and Laroche and Dolson’s later analysis bring the listening complaint called phasiness into the center of technical explanation. These writings reinterpret a spectral peak as a structured neighborhood rather than an accidental collection of bins. Their diagrams and derivations make coherence visible as a relation both across frames and within a frame. The resulting methods belong to a moment when computer-music practice had generated enough shared listening experience for an artifact to become a stable research object.

Roebel’s transient research extends that perceptual turn. The attack is no longer treated as a difficult exception to an otherwise uniform signal. It becomes a component with its own detection, preservation, and reconstruction problem. Later multiresolution research continues this shift by asking whether analysis scale itself should follow the material. The documents thus trace a widening ontology: first channels, then bins, then peaks, then transient and noise classes, and finally adaptive regions with different models.

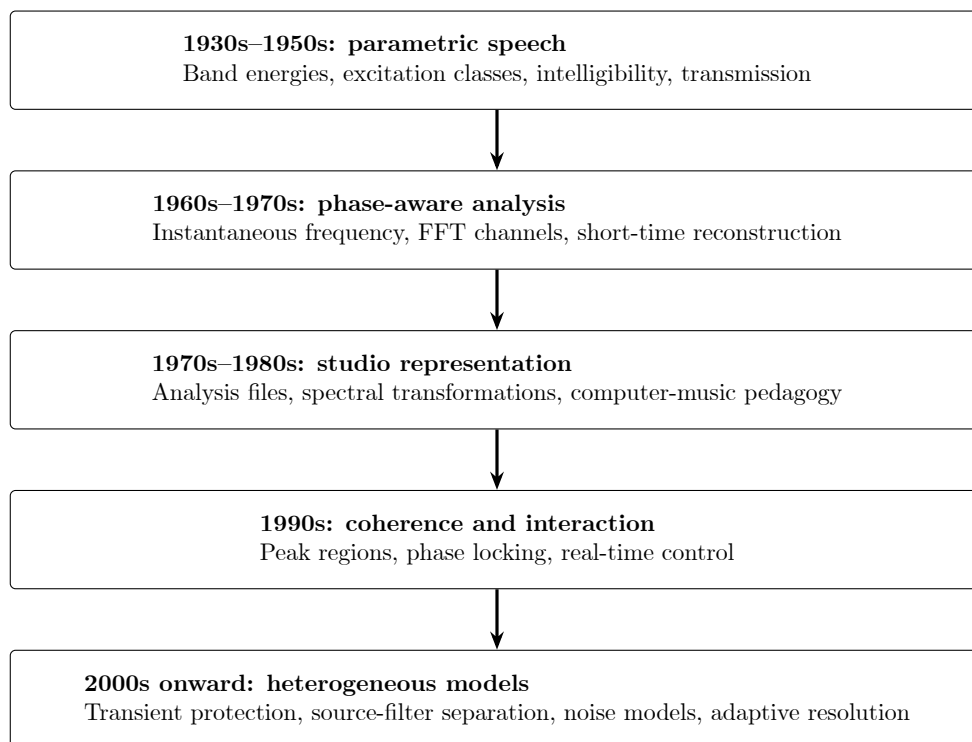


Figure 1.25: Successive research eras enlarged the set of sound properties represented explicitly. Earlier questions survived inside later systems rather than being discarded.

The hidden history of computation time

Algorithm histories can make ideas seem available as soon as they were published. In practice, the cost of conversion, storage, transformation, and audition governed what artists could attempt. A short modern render may conceal a chain that once required a scheduled laboratory session, dedicated converters, a mainframe job, an intermediate tape, and a return visit after computation finished.

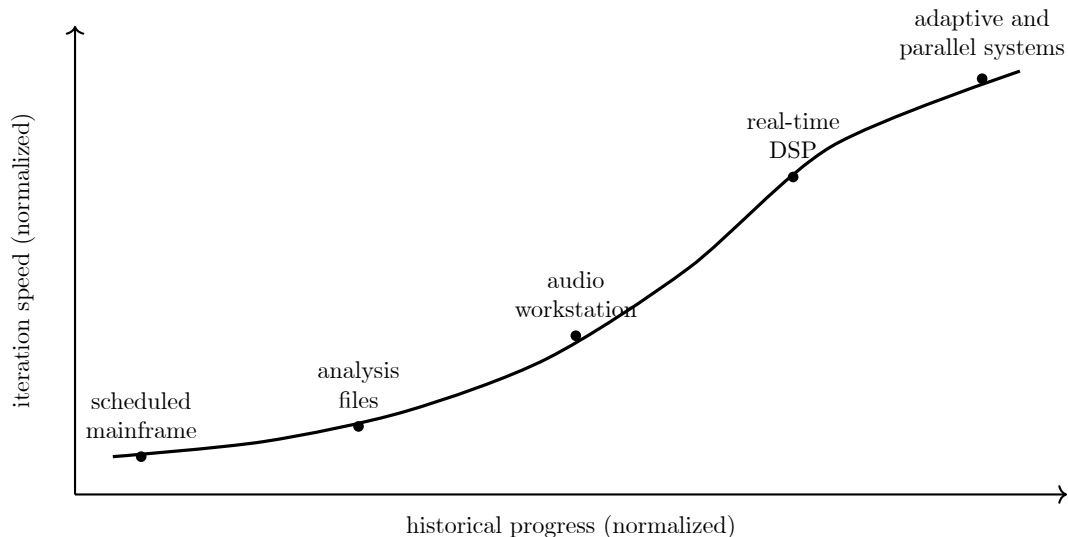
Early digital audio consumed memory at a scale that was disproportionate to ordinary computing resources. A few seconds of sound could be a substantial dataset. Analysis enlarged it because each frame carried many spectral values and because some systems stored amplitude-frequency pairs rather than a compact waveform. Long stretches multiplied output duration again. A musically interesting request could therefore become a storage problem before it became a signal-processing problem.

Processing time affected form. When each trial was expensive, composers had reason to plan transformations in advance, work from short source excerpts, and preserve intermediate results. Parameter sweeps were possible, but they were deliberate batches rather than instant gestures. Listening occurred after a delay. That delay encouraged written plans, catalogs of source sounds, and naming systems for renders.

The workstation changed the rhythm of iteration. A composer could inspect a spectrogram, select a region, launch an operation, and compare alternatives in one working session. Graphical systems made time-frequency selection spatially direct. The mouse did not simplify the mathematics, but it shortened the path between noticing a feature and acting on it.

Real-time processing changed the epistemology again. A performer could learn a transformation through bodily feedback. Controls could be explored continuously rather than as a sequence of files. Yet interactivity introduced a strict deadline. An offline renderer could use future samples, revise an estimate, or spend several seconds on one frame. A live system had to return the next block on time.

Contemporary machines support much larger transforms and more elaborate classifiers, but abundance introduces its own historical condition. A modern user may audition hundreds of settings without recording why one worked. Reproducible command lines, manifests, and checkpoints restore some of the documentary discipline imposed by scarce computing. *pvx* belongs to this later culture: computation is relatively cheap, while preserving intention remains difficult.



The curve is conceptual, not a benchmark. It records a change in the practical interval between proposing a transformation and hearing it.

Figure 1.26: The shrinking audition loop altered compositional method as profoundly as increases in nominal signal quality.

A fuller genealogy of phase-vocoder software

Software lineage is harder to narrate than publication history because code is mutable. A program may be copied, translated, optimized, renamed, or partly rewritten. Documentation may survive while source code disappears, or source may survive without a reliable account of local use. The following genealogy therefore emphasizes working cultures and documented relationships rather than claiming one unbroken family tree.

Early laboratory implementations were inseparable from institutional computing. Bell Labs systems supported speech research. Stanford, UCSD, MIT, and other centers developed computer-music programs around the machines and converters they possessed. A phase vocoder was often not one portable application. It was a pipeline of analysis, transformation, and synthesis programs embedded in local formats and job-control conventions.

At UCSD, CARL software helped establish a Unix-oriented audio culture. Small programs could be composed into larger processes, and files allowed analysis to persist between stages. This architecture encouraged users to treat spectral information as a durable medium. It also made errors inspectable. One could examine headers, compare stages, or rerun only the transformation rather than repeat the analysis.

Paul Koonce's PVC package preserved and extended this command-line tradition in a particularly revealing form. Koonce described PVC 1.0 as a collection of phase-vocoder signal-processing routines and accompanying shell scripts, written in C for Unix. He located the package within a practical lineage that included Eric Lyon, Chris Penrose, F. Richard Moore, and Mark Dolson. The [archived PVC manual](#) is valuable historical evidence because it records not only algorithms, but also installation, file formats, parameter conventions, and the working habits expected of a late twentieth-century computer-music user.

PVC joined a generic phase vocoder, `plainpv`, to a broad family of specialized spectral tools. Its catalog included time warping, noise filtering, spectral companding, band-amplitude selection, harmonization, chord mapping, static and time-varying filtering, convolution, spectral resonance, envelope extraction, and control-function reshaping. The range matters historically. It presents the phase vocoder not as one time-stretching effect, but as an engine from which additive, subtractive, cross-synthetic, and deliberately experimental processes could be assembled.

The package also made time-varying control a first-class part of the Unix workflow. Parameters marked as functions could read headerless streams of 32-bit floating-point values. PVC fitted each stream to the requested duration and interpolated it for continuity, while the bundled CMUSIC generation utilities created the control data. Shell scripts stored large parameter sets and connected preliminary analyses to later synthesis commands. This arrangement anticipated later automation curves, reproducible command manifests, and analysis-driven control routing, even though its interface depended on files and scripts rather than a graphical timeline.

Koonce's documentation distinguished two resynthesis routes. Magnitude-only changes could use the faster overlap-add method, while frequency modification required an oscillator bank because the altered spectrum no longer retained the structure expected by direct inverse transformation. PVC also separated analysis from reuse through `pvanalysis` files, which supplied `twarp`, convolution, and time-varying filtering. These choices make the package an instructive bridge between laboratory phase-vocoder programs and contemporary command-line systems: implementation policy remained visible, and the user was expected to understand how a requested transformation changed the appropriate synthesis method.

PVC's first release deliberately included only routines Koonce considered stable, useful, and moderately transparent, while more speculative routines remained outside the release. That distinction between a dependable surface and an experimental edge is not merely modern release vocabulary. It reflects a long-standing problem in musical signal processing: exploratory algorithms invite discovery, but tools used in sustained compositional work also need repeatable behavior, documented limits, and an interface that can preserve decisions.

The organization of PVC's commands reveals how the package divided spectral work into reusable representations. The following table groups representative routines by the kind of intermediate object they created or transformed. It is not a complete command inventory; its purpose is to show the conceptual architecture that made the collection composable.

Table 1.5: Representative PVC tool families and their working representations.

PVC family	Working representation	Principal operation
<code>plainpv</code> , <code>twarp</code>	spectral frames and time functions	resynthesize on an altered time or frequency schedule
<code>pvanalysis</code>	persistent complex analysis frames	prepare reusable source data for later transformations
<code>freqresponse</code> , <code>chordresponsemaker</code>	static spectral response	derive or synthesize a frequency-domain template
<code>filter</code> , <code>tvfilter</code> , <code>convolver</code>	static or evolving source spectra	filter or cross-synthesize one sound with another representation
<code>harmonizer</code> , <code>chordmapper</code>	spectral replication and mapping rules	construct harmonies or remap harmonic components
<code>ring</code> , <code>ringfilter</code> , <code>ringtvfilter</code>	spectral feedback and response data	create resonances shaped by source or filter spectra
<code>envelope</code> , <code>reshape</code>	scalar control streams	extract, transform, and reuse time-varying parameter functions

Seen this way, PVC treated analysis frames, frequency responses, mapping specifications, and scalar functions as different kinds of musical documents. A response could outlive the sound segment from which it was measured. A time function could be reshaped and assigned to another parameter. An analysis could support several readings through filtering, warping, convolution, or resonance. The package therefore belongs to a history of representation design as much as to a history of individual effects.

Csound offered another path. Its score-and-orchestra model placed analysis-based processing beside synthesis, sampling, and control-rate composition. Phase-vocoder opcodes and analysis formats allowed a sound file to participate in a notated computational instrument. Csound’s portability helped techniques travel beyond the institutions where they were first developed.

The Composers’ Desktop Project emphasized affordable offline tools for composers using personal computers. Its extensive catalog of transformations made spectral operations part of a broader craft of sound editing. CDP’s command-oriented workflow preserved the idea that unusual results often arise by chaining modest processes rather than selecting one comprehensive effect.

SoundHack, developed by Tom Erbe, gave artists direct access to classic and experimental sound transformations on personal computers. Its phase-vocoder and convolution tools became important in studios and classrooms. The program illustrates a recurrent form of dissemination: a researcher-programmer interprets specialist literature, builds an approachable implementation, and thereby changes which ideas become part of ordinary musical practice.

At IRCAM, the SuperVP and AudioSculpt lineage integrated research on high-quality stretching, transposition, source-filter transformation, transients, and spectral selection. AudioSculpt made analysis visually navigable, while SuperVP supplied a deep processing engine. Max integrations then brought parts of that engine into interactive performance. The lineage demonstrates how one institution can sustain an algorithm across offline, graphical, and real-time forms.

Miller Puckette’s Max and Pure Data environments made FFT processing patchable. Educational phase-vocoder examples exposed windowing, real and imaginary channels, phase accumulation, and overlap as signal networks. A patch could be altered while it ran. That visibility made the algorithm teachable not only through prose and equations but also through live signal flow.

Commercial workstations and plug-ins translated research into source categories and quality settings. Names such as monophonic, polyphonic, rhythmic, speech, transient, or formant-preserving hid combinations of algorithms behind production language. This was useful for users, but it made exact genealogy less transparent. Two products could both advertise high-quality stretching while using substantially different mixtures of phase propagation, waveform alignment, transient slicing, and resynthesis.

Open-source libraries later supplied reusable primitives for STFT processing, resampling, onset detection, and pitch analysis. Rubber Band, SoundTouch, librosa, SuperCollider extensions, and many research repositories widened access, though not all are phase vocoders and not all pursue the same quality goals. Their importance lies partly in testability. Implementations can be compared, modified, and incorporated into reproducible systems.

pvx inherits several branches of this history. Its command-line form recalls CARL and CDP. Its explicit analysis and transform stages recall laboratory pipelines. Its concern with stable manifests and resumable work responds to long offline renders. Its phase, transient, formant, and multichannel policies reflect later perceptual research. The result is not a museum reconstruction. It is a contemporary arrangement of ideas whose origins remain legible.

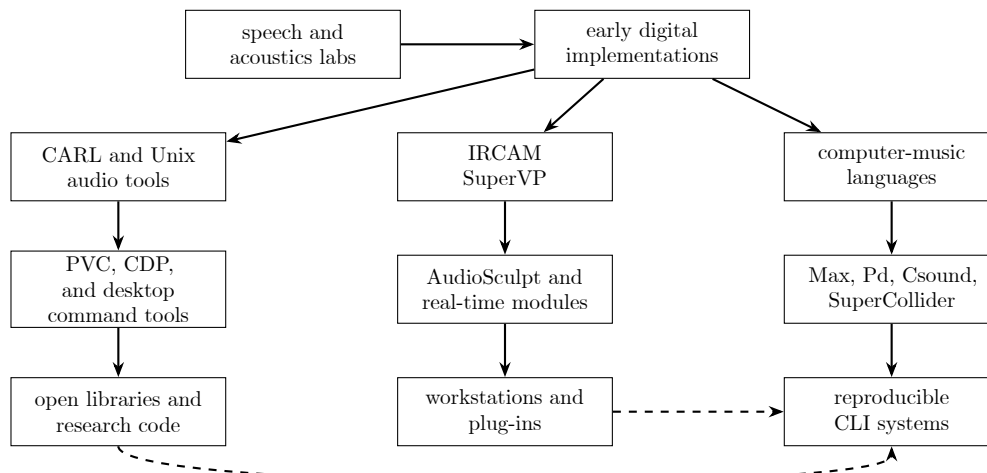


Figure 1.27: A conservative software genealogy. Solid lines indicate broad institutional or interface continuities; dashed lines indicate contemporary exchange rather than direct descent.

The analysis file as a new kind of score

The intermediate analysis file deserves a history of its own. It changed a recording from a fixed waveform into a collection of trajectories that could be revised without repeating the original performance. For composers, this was comparable to gaining a score for properties that conventional notation rarely describes: partial amplitude, local frequency, spectral envelope, noise distribution, and the evolution of a transient.

The analogy must not be taken too literally. An analysis is not neutral transcription. Window length determines what counts as local. Hop size determines the density of observations. Peak thresholds decide which components appear stable. A file format determines whether phase, frequency, or only magnitude survives. Every analysis is already an interpretation.

Still, persistence changes creative work. A user can inspect one frame, draw an envelope, transpose only a region, or use the amplitude pattern of one source to reshape another. Operations can be chained while keeping the original analysis intact. Several resyntheses can be understood as readings of the same document.

Analysis files also create archival problems. A waveform is comparatively self-describing when its sample rate, channel count, and encoding are known. Spectral data requires more context: transform size, window definition, hop, frame centering, normalization, phase convention, units, channel policy, and software version. Without these, a file may remain numerically readable while losing its musical meaning.

The history of formats such as phase-vocoder analysis files and SDIF shows attempts to make representations portable. Portability is partly social. A format succeeds when tools agree on semantics, documentation is maintained, and users trust that old work can be reopened. A formally elegant container without active readers may preserve less history than a plain but widely understood file.

The score analogy also clarifies authorship. An analysis may derive from a performer’s recording, be transformed by a composer, rendered by a program, and revised by later software. The output embodies

decisions at every stage. Good provenance does not settle all artistic questions, but it prevents the machine from appearing as an anonymous source of sound.

Compositional histories beyond the canonical examples

The most responsible musical history distinguishes direct documentation from stylistic proximity. Many late twentieth-century works engage spectral analysis, prolonged sound, resynthesis, or electronic transformation. Not all use a phase vocoder, and similarity of sound is not proof. The wider repertory is nevertheless important because it established the listening habits through which phase-vocoder transformations became musically intelligible.

Jean-Claude Risset’s computer-music research demonstrated that acoustic analysis could inform synthesis and composition. His studies of trumpet tones and his paradoxical pitch and rhythm effects made perceptual organization a compositional parameter. Risset’s work is broader than phase-vocoder history, but it helped create the intellectual world in which an analyzed sound could become a model rather than merely a recording.

Paul Lansky’s tape and computer works explored speech, language, and the boundary between recognition and abstraction. Processes in this repertory include linear predictive coding, granular procedures, and other computer transformations, depending on the work. The relevance is methodological. Speech is heard both as a carrier of meaning and as structured sound, a double attention central to the earlier communications history of the vocoder.

Denis Smalley’s writing on spectromorphology supplied a vocabulary for how spectral content and temporal shape behave in electroacoustic music. It is not an implementation manual and does not prescribe a phase vocoder. Its historical importance lies in giving listeners and composers terms for onset, continuation, termination, motion, texture, and source bonding. These categories help describe why two mathematically similar stretches can function differently.

Horacio Vaggione’s work foregrounded composition across multiple time scales. Microsonic detail, event structure, and larger form were treated as connected levels rather than separate domains. Phase-vocoder analysis likewise makes local spectral evolution available to larger compositional planning. The connection is conceptual, and it should not be mistaken for a claim that one algorithm defines Vaggione’s output.

Kaija Saariaho’s electroacoustic and instrumental practice at IRCAM developed within a culture of spectral analysis, timbral interpolation, and computer-assisted composition. Works such as *Verblendungen*, *Lichtbogen*, and *Nymphaea* occupy a historical environment where spectra could inform harmony, orchestration, and electronics. Documentation for an individual work must determine which processes were actually used. The broader significance is the integration of analysis into a composer’s sustained language.

Tristan Murail and other composers associated with spectral music likewise transformed the cultural meaning of a spectrum. A spectrum could generate harmony, formal direction, orchestration, and processes of fusion or separation. Phase-vocoder technology is not synonymous with spectral composition, but both challenge the idea that timbre is a surface placed on independently conceived notes.

Curtis Roads explored granular and microsound methods that often provide alternatives to phase-vocoder stretching. His work matters here because it made the size and scheduling of sound particles audible as compositional choices. Comparing a granular prolongation with a phase-coherent spectral prolongation teaches more history than declaring one method universally superior.

Barry Truax developed real-time granular synthesis and composed extensively with environmental sound. His practice offers another neighboring history of duration change, source recognition, and texture. The contrast is instructive. Granular methods foreground event particles and density, while a phase vocoder foregrounds evolving spectral channels or peaks. Many modern processors borrow from both.

Natasha Barrett’s electroacoustic and spatial work demonstrates how transformation history extends into multichannel composition. Once processed sound moves through an immersive field, spectral coherence interacts with localization, distance, and room impression. The phase vocoder’s mono origins are no longer sufficient. Spatial behavior becomes part of whether a transformation preserves identity.

These neighboring practices prevent a narrow great-inventor narrative. The algorithm developed because engineers solved technical problems, but its musical meaning developed across studios, concerts, teaching,

criticism, and repeated listening. Composers who selected, rejected, combined, or deliberately exposed its artifacts participated in that history even when they did not publish an equation.

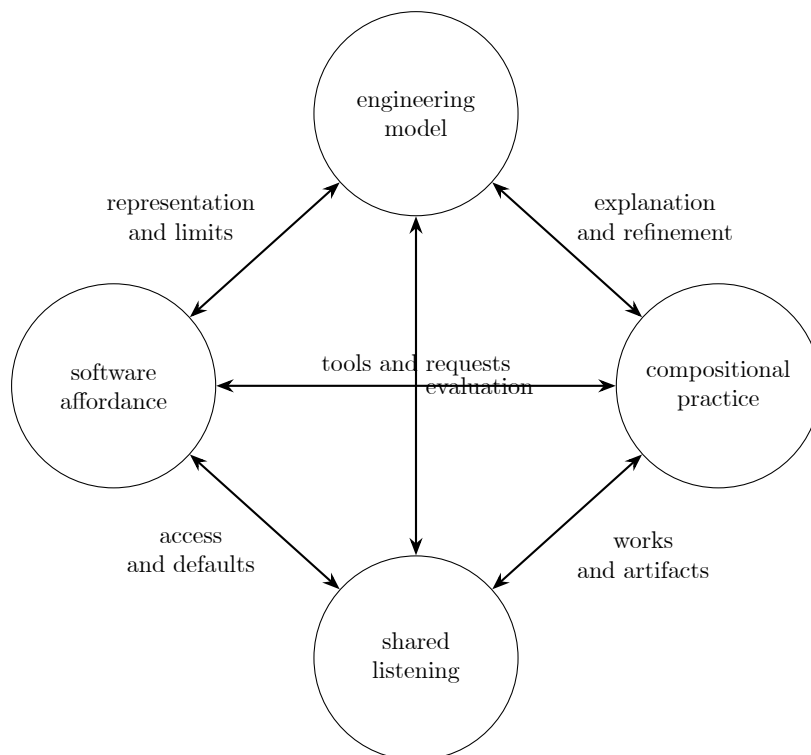


Figure 1.28: Musical technology develops through feedback among models, implementations, artistic practice, and learned perception.

Artifact history as listening history

Every period heard phase-vocoder quality through the expectations of its own media. Early speech researchers prioritized intelligibility and channel economy. A synthetic voice could sound unnatural yet count as a success if words survived transmission. Computer musicians often valued recognizability under transformations far beyond ordinary speech needs. Production users later expected stretched material to sit invisibly inside a polished mix.

The word *phasiness* is itself historical evidence. It gathers several impressions: diffuseness, loss of focus, a reverberant halo, weakened attacks, or chorus-like instability. The term became useful before every mechanism was agreed upon. Researchers then separated horizontal phase continuity from vertical phase relations and proposed peak-based locking. Listening vocabulary helped organize mathematical inquiry.

Transient smear followed a similar path. A stretched attack could sound doubled, softened, or preceded by a faint cloud. Waveform inspection showed why a long analysis window distributed an abrupt event. Detection and phase reset became explicit remedies. Later systems separated transient energy or used alternate resolutions. What began as a complaint became an architectural branch.

Metallic noise revealed that a sinusoidal account had been extended too far. Random components acquired deterministic repetition or frozen phase relations. Hybrid sinusoidal and stochastic models responded by representing the residual differently. The artifact taught developers that coherence can be excessive as well as insufficient.

Stereo image motion exposed another hidden assumption. Two channels that sounded stable before processing could drift when analyzed independently. The historical response included shared peak decisions, reference phases, and constraints on interchannel relations. Once again, a perceptual failure revealed information that the original model had not represented.

Artifacts also became genres of sound. Long-window smearing, frozen spectra, phase randomization, and metallic extension were cultivated in ambient, electroacoustic, experimental, and popular production. A correction in one context could destroy the desired effect in another. Mature tools therefore preserve both transparent and exposed modes rather than assuming that historical progress means eliminating every trace of the process.

This double status complicates evaluation. A test for speech naturalness cannot rank an intentional spectral drone. A preference test may obscure whether listeners value fidelity, novelty, rhythm, source identity, or reduced roughness. Historical listening requires the criterion to be stated before the score is interpreted.

Teaching, diagrams, and the public life of the algorithm

The phase vocoder spread through diagrams as much as through source code. A bank of filters with envelope and phase paths made the communications model visible. A row of overlapping windows made short-time analysis visible. A spiral of accumulated phase made continuity visible. Peak-centered groups made phase locking visible. Each diagram selected what a reader should imagine the algorithm to be.

Textbooks often begin from Fourier analysis and proceed toward implementation. Computer-music tutorials frequently begin from an audible task, such as stretching a voice without changing its pitch. Patch-based lessons begin from signal objects and connections. Command-line manuals begin from files and options. These routes produce different intuitions even when they reach related mathematics.

The most successful explanations move among representations. A waveform shows timing, a spectrogram shows energy distribution, a phase plot shows continuity, and an audio example shows the perceptual result. No single view is sufficient. The history of pedagogy is therefore also a history of media for explanation.

Mailing lists, workshops, course handouts, and online examples carried practical knowledge omitted from papers. Users learned which windows behaved well, how much overlap a particular implementation expected, why boundaries clicked, and when a phase-locking mode harmed drums. Such advice is difficult to archive, yet it often determines whether an algorithm becomes usable.

The current abundance of tutorial videos and code repositories makes access easier while creating a verification problem. A demonstration may call any spectral effect a phase vocoder. A code sample may reconstruct audio while silently violating overlap normalization. Historical literacy gives the user a way to ask what representation, phase rule, and reconstruction policy are actually present.

For that reason, this book treats history as practical equipment. Knowing why a control exists helps predict what it will do. Knowing which problem a method was designed to solve helps identify when not to use it. Knowing that a famous composition combined several processes prevents a listener from mistaking one timbre for a universal algorithmic signature.

Bell Laboratories as a long technical precondition

Bell Laboratories appears repeatedly in this history because the phase vocoder depended on more than one isolated discovery. The laboratory joined telephone engineering, psychoacoustics, speech science, electronics, and later digital computation. Problems could move among departments and persist across decades. A device for compressing speech bandwidth, a study of vocal production, and a mathematical description of a filter bank belonged to a common institutional world.

The telephone system gave research an unusual scale. Speech was not an occasional laboratory signal. It was the content of a national and international infrastructure. Improvements in intelligibility, bandwidth, switching, coding, and noise performance could have immense practical value. This sustained investment in how speech survives technical mediation.

Homer Dudley's work illustrates the resulting breadth. The channel vocoder was at once a communication system and a model of speech. The Voder was at once a demonstration machine and a demanding human interface. Later secure-speech systems turned parametric coding into synchronized infrastructure. These projects established analysis-synthesis as a serious engineering practice long before musicians had routine digital access.

Flanagan and Golden's phase vocoder belongs to that accumulated practice. Their formulation assumes a reader comfortable with speech channels, phase, modulation, and reconstruction. The new representation was radical, but its questions were inherited: what information should pass through the system, how accurately can a source be reconstructed, and which changes remain intelligible?

Bell Labs also influenced computer music through figures whose work crossed research and art. Max Mathews developed foundational computer-music systems there. Jean-Claude Risset conducted important analysis and synthesis research in that environment. The institution did not simply hand telecommunications tools to composers. It helped create a setting in which digital sound itself became a research material.

The lesson is institutional rather than celebratory. Long-lived research environments can support ideas whose applications are not foreseeable at the start. The phase vocoder required mathematics, hardware, speech data, engineering practice, and enough continuity for these to meet. Its history warns against narratives that credit only the final paper while ignoring the infrastructure that made the paper thinkable.

Stanford, CCRMA, and model-based sound

Stanford's computer-music history provides another route from acoustic description to musical transformation. Work associated with the Center for Computer Research in Music and Acoustics developed synthesis, physical modeling, signal processing, and digital musical systems. The phase vocoder was one tool in a wider inquiry into how sound could be represented for both analysis and composition.

Sinusoidal modeling became especially important in this setting. Julius O. Smith, Xavier Serra, and collaborators developed analysis-synthesis methods that tracked deterministic partials and represented remaining energy as a stochastic residual. The resulting systems encouraged a source model richer than a uniform field of bins.

This model changed what a transformation could preserve. A tracked partial had a trajectory and could be continued, modified, or ended as an entity. A residual could retain noise character without pretending to be a collection of long-lived sinusoids. The distinction was particularly useful for musical sounds that mixed stable resonance with breath, scrape, or attack.

The relationship to the phase vocoder is reciprocal. Peak tracking and sinusoidal interpretation informed later phase-locking approaches. Phase-vocoder infrastructure supplied efficient short-time spectra from which peaks could be found. Hybrid systems increasingly treated the argument between dense bins and tracked sinusoids as a design choice within one processor.

Stanford's role also demonstrates the value of public technical writing. Reports, course materials, software descriptions, and later online books made advanced signal-processing ideas available well beyond one laboratory. An institution contributes to history not only by inventing a method but by maintaining a language through which others can implement and challenge it.

UCSD, CARL, and the command-line studio in detail

The CARL environment deserves close attention because its working model resembles contemporary reproducible audio tooling. Rather than hiding every stage in one application, Unix programs could analyze, transform, synthesize, inspect, and convert data. The shell became a compositional surface.

This modularity changed experimentation. A user could retain an expensive analysis, run several transformations from it, and compare outputs. A script could document a sequence more precisely than handwritten recollection. Programs could be chained in combinations their authors had not anticipated. The operating system supplied a general composition mechanism.

Modularity also made formats consequential. A tool had to know how frames, channels, and metadata were organized. If one program interpreted frequency as hertz and another expected phase radians, the pipeline failed semantically even if every file opened. Shared formats were therefore social contracts among developers and users.

Mark Dolson's work belongs to this culture of research through implementation. The tutorial is often encountered as a self-contained text, but its practical intelligence reflects experience with working systems

and musical material. The account of transformations is persuasive because it connects representation to operations a composer might actually perform.

The command-line studio could be austere. Users needed to understand files, names, paths, parameter ranges, and processing order. Yet that effort produced a durable benefit: procedures were externalized. A session could be rerun after the sound had been forgotten, and a surprising result could be traced to its commands.

Modern interfaces often oppose usability to explicitness, but the CARL history suggests a more productive balance. A tool can provide safe defaults and still expose a complete invocation. It can explain a parameter in perceptual language while preserving its exact value. *pvx*'s manifests and stable commands pursue this balance rather than treating the shell as nostalgia.

IRCAM as a meeting of research and commission

IRCAM's importance comes partly from the proximity of long-term technical research to concrete artistic projects. Composers arrived with difficult requests, researchers developed representations and algorithms, and software teams turned those results into systems that could survive beyond one premiere. The phase-vocoder lineage became one strand within a larger ecology of analysis, synthesis, spatialization, and computer-assisted composition.

Commissioned works supplied demanding tests. A process that succeeded on a laboratory speech sample might fail on a bell, a flute multiphonic, or a whispered consonant. A composer might value an artifact that a speech engineer would reject. Artistic requirements widened the domain over which an algorithm had to be understood.

The progression from command-line or batch systems to AudioSculpt made spectral processing visually situated. A user could correlate a heard event with a region in a spectrogram, select it, and apply a treatment. This changed access while preserving the underlying analysis. The display became an interpretive layer between mathematical data and musical gesture.

SuperVP's continuing development shows that a phase-vocoder engine can become a platform for many related models. Time stretching and transposition coexist with source-filter processing, envelope control, transient handling, noise and sinusoidal treatment, cross-synthesis, and real-time operation. The name persists even as the internal system becomes more heterogeneous than the classic algorithm.

IRCAM Forum distribution and Max integrations widened the community around these tools. Workshops and documentation taught not only commands but listening strategies. Users carried techniques into universities, studios, installations, and commercial production. Institutional history thus extends through education and licensing as well as papers.

The archive of works and technical notes also permits unusually careful reconstruction. For Harvey's *Mortuos Plango, Vivos Voco* or Reynolds's *Transfigured Wind*, one can compare compositional statements, source descriptions, algorithm names, and audible results. This evidence helps resist the habit of attributing an entire work to one famous process.

Speech as the first difficult material

Speech drove much of the early research because it combines strict perceptual demands with complex acoustics. A listener notices disruptions in timing, pitch, consonant articulation, vowel identity, rhythm, and speaker character. The source changes quickly, yet it also contains stable harmonic and resonant structures.

Voiced speech invites a sinusoidal account. Vocal-fold pulses create a harmonic series, while the vocal tract shapes broad resonances. A time-scaling system should preserve local pitch while changing the schedule of syllables. A pitch shifter may need to move the harmonic source while retaining the vocal-tract envelope. These requirements made phase continuity and formant treatment practical necessities.

Unvoiced consonants resist the same model. Fricatives contain turbulent noise. Stops contain closures and abrupt releases. Aspirated sounds mix noise with voicing. A long stationary window can blur these events, while aggressive phase locking can make noise unnaturally coherent.

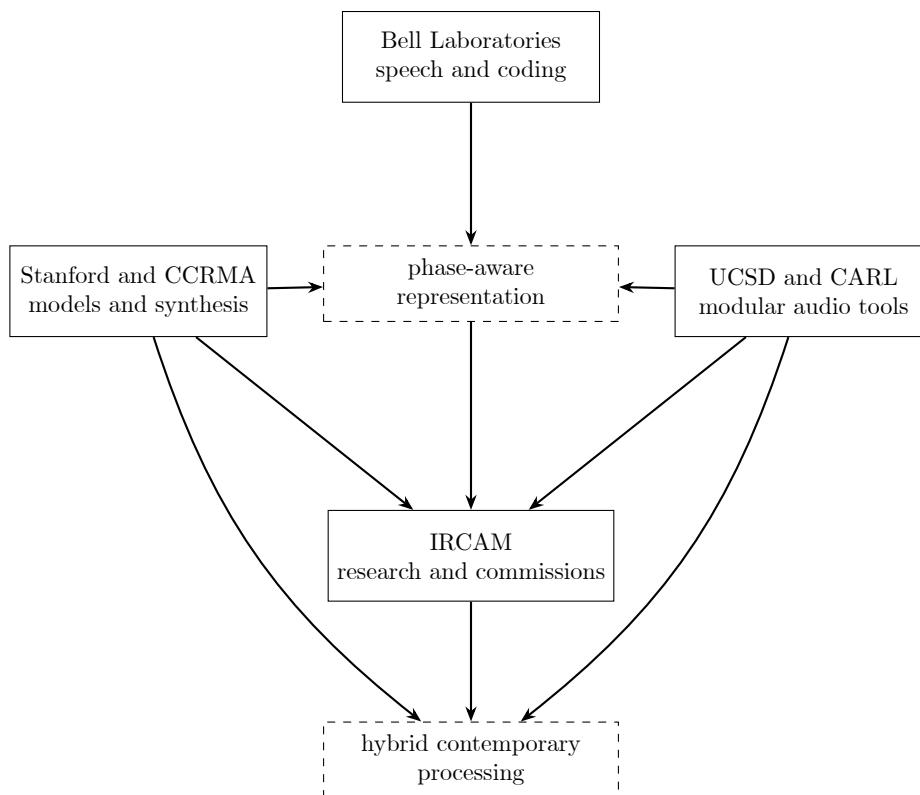


Figure 1.29: Institutions contributed different working emphases. The diagram records exchange and convergence, not exclusive ownership of ideas.

Prosody adds a larger time scale. Speech is not a line of independent frames. Pitch contours, stress, pauses, and coarticulation establish phrases. Expanding every local interval uniformly may be mathematically faithful yet rhetorically strange. Practical speech modification often requires event-aware timing as well as spectral reconstruction.

Early evaluation emphasized intelligibility, but naturalness and identity became increasingly important. A system could preserve every word while making the speaker sound distant, robotic, smaller, or emotionally altered. These outcomes encouraged perceptual tests and source-specific algorithms.

Musicians inherited all of these tensions. A voice may be stretched until words disappear while breath and resonance remain. A formant shift may deliberately change apparent body size. A consonant can become a percussive event. Speech research provided the problems, while composition expanded the acceptable answers.

Sustained instrumental sound and the discovery of coherence

Sustained pitched instruments initially appear ideal for a phase vocoder. Their partials persist across many frames, and large time changes can preserve a recognizable pitch. Yet these sounds made vertical incoherence painfully audible. A flute or string tone could become diffuse even when every bin followed a continuous phase path.

The reason lies in how a window represents one partial. Energy spreads across nearby bins according to the window's frequency response. Those bins are not independent physical oscillators. They are samples of one local spectral structure. If their phases evolve independently, the reconstructed waveform loses the organization associated with the original partial.

Phase locking responds by identifying a spectral peak and coordinating nearby bins. Identity locking preserves the relative relation within the peak region, while scaled approaches adjust those relations under

transformation. The exact strategy differs among implementations, but the historical advance is shared: a peak neighborhood becomes a unit.

Vibrato complicates the picture. A partial moves in frequency, crossing the fixed grid. Peak tracking must decide whether adjacent observations belong to one trajectory. Too little continuity produces jitter; too much can force unrelated components together. Ensemble sounds make the assignment more difficult because several instruments contribute close or crossing partials.

This is why Dolson’s work on tracking and ensemble analysis matters. It directs attention away from a static-bin ontology toward moving components. Later sinusoidal and hybrid models elaborate the same insight.

For composition, improved coherence expanded the range between transparency and abstraction. A sustained note could be prolonged while retaining presence, or its peak groups could be deliberately loosened into a cloud. The artifact became controllable because its cause was better understood.

Percussion and the historical challenge of the onset

Percussive sound reverses many assumptions that favor sustained tones. An attack is brief in time and broad in frequency. Its identity may depend more on the alignment of energy than on stable partial trajectories. A large Fourier window sees spectral detail but spreads the event across its own duration.

Ordinary time scaling moves synthesis frames to new positions. If several adjacent analysis frames contain pieces of one attack, spacing those frames farther apart distributes the attack energy. The result can be a soft lead-in, a doubled strike, or a reverberant tail that was not present in the source.

Transient detection attempts to mark the event before it is stretched. A processor may reset synthesis phase, preserve the local timing of transient frames, route the event through a time-domain method, or use a shorter window. Each remedy protects one property while risking another. A reset can cause discontinuity; a short window reduces frequency resolution; a detector can miss soft or gradual attacks.

Drums also contain resonant tails. The initial strike may need transient handling while the shell or cymbal decay benefits from spectral continuation. This mixed behavior encouraged within-event classification rather than one preset for the whole file.

Rhythmic material adds metrical expectations. A tiny onset shift can weaken groove even if the spectrum sounds clean. Evaluation therefore includes event timing, not just waveform similarity or spectral distance. Music information retrieval tools for onset detection became relevant to audio effects.

Historically, percussion kept the phase vocoder honest. It prevented improvements on speech and steady tones from being mistaken for universal quality. It also accelerated hybrid processing, one of the defining characteristics of modern stretching systems.

Noise, ambience, and environmental recordings

Environmental recordings combine events, textures, reverberation, and spatial cues. Wind may behave as colored noise, birds as moving partials, footsteps as transients, and a room as a decaying multichannel field. No single phase policy describes the whole recording.

Noise has statistical identity. Its exact waveform can change while its distribution remains perceptually similar. A phase vocoder that preserves the wrong details may produce a frozen or metallic texture. Noise-aware synthesis instead attempts to preserve energy, color, modulation, and decorrelation.

Reverberation poses a related problem. A room tail contains dense reflections whose phase relations support spaciousness but are not equivalent to one stable sinusoidal source. Independent channel processing can alter width and localization. Excessive phase locking can collapse diffuse energy toward a point.

Environmental composition often values source recognition. A stretched wave or insect may remain identifiable long after ordinary timing has disappeared. Alternatively, an apparently abstract texture may suddenly reveal its source through one unprocessed transient. The composer manages recognition as a formal parameter.

Field recordings also carry documentary and ethical context. Transformation can detach a sound from place, person, or event. Metadata and source notes become part of responsible practice, especially when a

recording includes voices or culturally specific material. Technical provenance and cultural provenance meet in the archive.

For pvx, such material argues for automation that can vary through a file. A transient region, a stable call, and a diffuse background need not share one strategy. Time-varying control is not an ornamental feature. It is a response to the heterogeneous history of recorded sound.

Extreme duration and the change of listening scale

Moderate time scaling asks whether a performance can remain natural at a new duration. Extreme scaling asks a different question: what hidden structure appears when ordinary time ceases to govern perception? A short gesture can become a landscape. A modulation once heard as timbre can become rhythm.

This change of scale has precedents in tape slowing, granular synthesis, and studio feedback, but the phase vocoder offers distinctive control over pitch and spectral continuity. It can prolong a sound without the obligatory pitch descent of varispeed. The result may retain a spectral identity while abandoning the source's original gesture.

At large ratios, details ignored in ordinary playback become structural. Vibrato becomes a wide oscillation. A room reflection becomes a separate event. Quantization noise, background hum, and microphone motion become audible layers. The distinction between source and recording apparatus weakens.

Extreme stretching also magnifies algorithmic assumptions. Frame repetition becomes rhythm, phase errors become spatial haze, and a transient detector's decisions become formal cuts. A setting that is transparent at a ratio near one may produce strong periodicity at a ratio of hundreds or thousands.

Composers including JoAnn Kuchera-Morin and many later ambient and experimental artists treated this behavior as material rather than defect. The practice widened public awareness of phase-vocoder sound, though popular examples sometimes use other or hybrid stretch algorithms. Historical accuracy again requires listening and documentation rather than inference from duration alone.

A millionfold stretch reaches beyond ordinary rendering into installation, data, and archival questions. Output size and time become dominant. The process may require stages, checkpoints, sparse audition, and alternate representations. At that scale, software architecture is part of the composition.

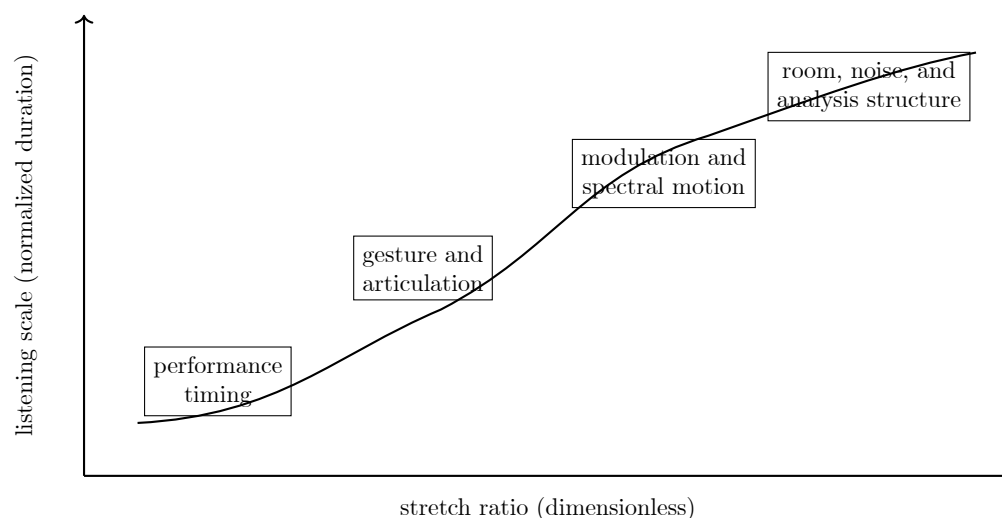


Figure 1.30: Extreme stretching changes which temporal layer behaves as musical form. The boundaries are material-dependent rather than fixed ratio thresholds.

Preservation, reconstruction, and obsolete systems

Historical phase-vocoder work is unusually vulnerable because the meaningful object may be distributed across several media. A source recording, an analysis file, transformation parameters, custom software, and a final tape can each preserve different parts of the process. Losing one may prevent exact reconstruction.

Obsolete analysis formats are a particular risk. Even when bytes survive, the reader may not know the window, hop, phase convention, scaling, or header semantics. Reimplementing an old synthesizer from a paper may produce a plausible sound without reproducing the original program’s rounding, interpolation, or boundary behavior.

Hardware contributes another layer. Converter quality, sample rate, tape transfer, and analog studio stages influenced historical outputs. A mathematically exact modern rerender may sound cleaner while being less faithful to the work as realized. Preservation must distinguish the algorithmic plan from the historical artifact.

Composer archives can resolve some questions through sketches, job logs, source lists, tapes, and correspondence. Reynolds’s preserved materials are valuable for precisely this reason. They show a work as a sequence of choices rather than an unexplained audio file.

Software preservation may use source archives, virtual machines, emulation, test fixtures, and rendered reference examples. No one method is sufficient. Source code without a compiler may be unreadable in practice; a binary without documentation may run but remain uninterpretable; a reference render without parameters preserves sound but not method.

Contemporary projects can learn from these losses. A manifest should record versions and units. Analysis artifacts should identify their schema. Tests should preserve expected behavior for representative sources. Documentation should say which public surface is stable. These habits are not administrative extras. They are the conditions under which future users can understand what present users heard.

A decade-by-decade change in the imagined user

In the 1960s, the implied phase-vocoder user was a communications researcher or engineer. The system was described through channels, modulators, phase derivatives, and speech examples. Access depended on a major laboratory. A successful output demonstrated analysis and resynthesis more than everyday creative convenience.

In the 1970s, the implied user increasingly included a programmer working with digital signal-processing systems. FFT implementations and short-time analysis papers made the method more concrete. The user still needed specialized computing, but the algorithm could be described as repeatable operations on arrays.

In the 1980s, the computer musician became a central figure. Tutorials, institutional software, and landmark compositions made spectral transformation a recognizable studio practice. The user might wait for offline jobs, maintain analysis files, and collaborate closely with researchers. Time expansion became composition, not only signal correction.

In the 1990s, the implied user widened toward the interactive musician and desktop studio. Personal computers, graphical systems, Max, Pure Data, Csound, SoundHack, CDP, and improved workstations reduced institutional barriers. Research focused increasingly on audible artifacts, phase locking, pitch shifting, and practical musical quality.

In the 2000s, the production user expected real-time or near-real-time results, visual editing, source presets, and integration with larger sessions. Transient preservation, formants, and comparative evaluation became prominent. The algorithm often disappeared behind an application mode.

In the 2010s, phase-vocoder ideas circulated through open libraries, mobile processors, web demonstrations, DJ systems, games, and large research ecosystems. Hybrid methods became normal. Users selected outcomes while engines selected among multiple internal policies.

In the 2020s, learned models increasingly surround classical transforms. Separation, pitch estimation, transcription, classification, and neural synthesis can occur before or after a phase-vocoder stage. At the same time, reproducible command-line tools and open research code keep explicit algorithms valuable.

The imagined user is now plural. A dialogue editor wants transparent duration correction. A composer wants unstable spectral matter. A researcher wants inspectable intermediate data. A performer wants low latency. An archivist wants deterministic reconstruction. A durable tool must state which of these promises it supports rather than hiding incompatibilities behind the word *quality*.

Annotated chronology

The following chronology is selective. It emphasizes events that changed representation, implementation, dissemination, or musical use. Dates of compositions indicate completion or first realization where generally documented; software lineages often span several years.

Table 1.6: Selected chronology of phase-vocoder history and its neighboring lineages.

Date	Person, work, or system		Historical significance
1822	Joseph Fourier		Published the analytical theory of heat, helping establish harmonic decomposition as a general mathematical language.
1863	Hermann von Helmholtz		Connected partial structure, resonance, pitch, and tone sensation through acoustical experiments and theory.
1920s-1930s	Homer Dudley		Developed channel-vocoder analysis and synthesis at Bell Telephone Laboratories.
1939	Voder demonstration		Presented manually controlled speech synthesis to a mass public at the New York World's Fair.
1940s	SIGSALY		Demonstrated large-scale secure speech coding, synchronization, and reconstruction.
1950s-1960s	Tape rate changers		Used segmentation and rotating heads to alter speech duration with reduced pitch change.
1965	Cooley and Tukey		Published an influential fast Fourier transform algorithm suited to repeated digital spectral calculation.
1966	Flanagan and Golden		Published *Phase Vocoder*, presenting phase-aware analysis, transmission, resynthesis, and time modification.
1976	Mark Portnoff		Published an FFT implementation of the digital phase vocoder.
1977	Allen and Rabiner		Unified short-time Fourier analysis and synthesis in a widely cited signal-processing account.
1978	James A. Moorer		Described phase-vocoder use in computer-music applications.
1980	Jonathan Harvey		Completed <i>Mortuos Plango, Vivos Voco</i> , linking source spectra, synthesis, and musical form at IRCAM.
1980	Mark Portnoff		Published short-time-Fourier-based time-scale modification of speech.

Date	Person, work, or system		Historical significance
Early 1980s	Mark Dolson		Investigated tracking phase vocoders, ensemble analysis, software, and musical transformations.
1984	Griffin and Lim		Published iterative signal estimation from modified short-time Fourier magnitude.
1984-1985	Roger Reynolds		Developed the <i>Transfigured Wind</i> series with transformed flute materials and computer processes.
1986	Mark Dolson		Published *The Phase Vocoder: A Tutorial*, consolidating theory and musical applications.
1986	McAulay	and	Published influential sinusoidal speech analysis-synthesis based on tracked components.
	Quatieri		
1986	Trevor Wishart		Realized <i>Vox 5</i> , a canonical work of vocal spectral transformation and morphing.
1989	JoAnn Kuchera-Morin		Realized <i>Dreampaths</i> , noted for extensive time and spectral transformation.
1990	Serra and Smith		Presented spectral modeling synthesis using deterministic and stochastic decomposition.
1993	Verhelst and Roelands		Published WSOLA, an influential waveform-similarity method for time-scale modification.
1995	Miller Puckette		Presented a phase-locked vocoder approach for improved spectral coherence.
1997	Laroche and Dolson		Framed phasiness as a vertical-coherence problem in phase-vocoder output.
1999	Laroche and Dolson		Published improved time-scale modification using identity and scaled phase locking.
1999	Laroche and Dolson		Published new phase-vocoder techniques for pitch shifting, harmonizing, and related effects.
2002	Duxbury, Sandler	Davies,	Developed transient-aware phase-locking approaches for musical time scaling.
2003	Axel Roebel		Presented new transient-processing approaches within the phase vocoder.
2005	Roebel and Rodet		Published efficient spectral-envelope estimation for pitch shifting and envelope preservation.
2006	Karrer, Lee, Borchers		Presented PhaVoRIT for real-time interactive time stretching and perceptual comparison.
2007	Bradford, ffitch	Dobson,	Presented a sliding phase-vocoder structure oriented toward continuous updates.

Date	Person, work, or system		Historical significance
2010s	SuperVP extensions		Consolidated high-quality time, pitch, envelope, transient, noise, and cross-synthesis processing in research and professional tools.
2016	Driedger and Muller		Published a broad review of music time-scale-modification methods and evaluation concerns.
2017	Ottosen and Dorfler		Developed a phase vocoder based on nonstationary Gabor frames.
2017	Juillerat and Hirsbrunner		Presented adaptive multiresolution time stretching with magnitude correction.
2023	Akaishi, Oikawa	Yatabe,	Applied time-directional spectrogram squeezing to improve long phase-vocoder stretches of percussive material.

Reading the history through software design

The chronology can be translated into design principles. Filter-bank history explains why band behavior and spectral envelopes matter. Tape history explains why local units, joins, and attacks matter. FFT history explains why frame size and hop are exposed. Computer-music history explains why intermediate representations and automation matter. Coherence research explains phase-locking controls. Transient research explains hybrid modes.

A modern command therefore contains decades of argument. Choosing a Hann window invokes a history of spectral measurement and overlap. Choosing identity phase locking invokes the explanation of phasiness developed by Puckette, Laroche, and Dolson. Choosing transient preservation invokes the recognition that stationary sinusoidal assumptions fail at attacks. Choosing formant preservation invokes the source-filter tradition.

This perspective prevents parameter lists from becoming arbitrary. Options are not decorations around one timeless algorithm. They are answers to historical failures, extensions, and artistic demands. Some answers conflict, which is why a program needs policies rather than one universal maximum-quality switch.

The design of pvx continues that lineage through explicit commands, reproducible settings, checkpoints, and analysis artifacts. Its contribution is not to erase history behind a button. It is to make historical choices available in a coherent working environment.

Source notes for the expanded history

The analog and electromechanical account draws on Fabian Voigtschild, Jonathan Sterne, and Mara Mills’s [history of Anton Springer and the time and pitch regulator](#), Wendy Carlos’s first-person [account of the Eltro Mark II and the voice of HAL](#), Grant Fairbanks, Wilbur L. Everitt, and Robert P. Jaeger’s 1954 paper *Method for Time or Frequency Compression-Expansion of Speech*, Dennis Gabor’s 1946 and 1947 work on communication and acoustical quanta, and Harald Bode’s 1984 *History of Electronic Sound Modification*. These sources distinguish ordinary varispeed, the phonogène, rotating-head regulation, variable delay, and single-sideband frequency shifting rather than treating them as one technique.

The principal technical lineage in this account is documented by Flanagan and Golden’s 1966 paper, Portnoff’s 1976 and 1980 papers, Moorser’s 1978 computer-music article, Dolson’s 1986 tutorial, Puckette’s 1995 phase-locked vocoder, Laroche and Dolson’s 1997 and 1999 work, Roebel’s transient and spectral-envelope research, and later multiresolution studies. Full bibliographic records appear in the Bibliography.

Musical and institutional accounts rely on composer writings and archives where possible. Trevor Wishart’s retrospective describes his long engagement with phase-vocoder data and morphing. The Library of Congress genealogy of Roger Reynolds’s *Transfigured Wind* documents source recordings, algorithms, plans, and versions. IRCAM’s research and software documentation describes SuperVP, AudioSculpt, and later real-time modules. IRCAM’s analytical account of *Mortuos Plango, Vivos Voco* distinguishes FFT analysis, Music V, synthesis, and transformations used in Harvey’s work.

Claims about JoAnn Kuchera-Morin’s *Dreampaths* are supported by composer records and secondary technical literature including Curtis Roads’s discussion of extensive phase-vocoder transformations. Software genealogy is stated conservatively because local versions, ports, and unpublished changes complicate simple lines of descent.

This chapter uses *phase vocoder* narrowly for phase-aware short-time analysis and resynthesis, while acknowledging neighboring channel-vocoder, sinusoidal-modeling, overlap-add, granular, and neural-vocoder histories. That terminological discipline is necessary because the same word *vocoder* now names substantially different systems.

What the historical work leaves us

The historical line from Dudley through Flanagan, Golden, Portnoff, Dolson, Laroche, and later researchers leaves three durable lessons. First, representation controls imagination: once amplitude and phase trajectories are explicit, time and pitch become editable structures. Second, efficiency changes art: the FFT turned a laboratory model into a repeatable process, and modern hardware turns it into a live or corpus-scale operation. Third, perceptual quality depends on relationships. A bin is not heard alone; it belongs to a peak, a frame, a transient, a channel, and a musical event.

pvx inherits all three lessons. It treats the phase vocoder as a practical engine, a controllable source of artifacts, and a framework around which automation, analysis, checkpointing, and reproducibility can be built. The rest of the book moves from this history into implementation detail, but the listening problem remains the same: decide which relationships must survive the transformation.

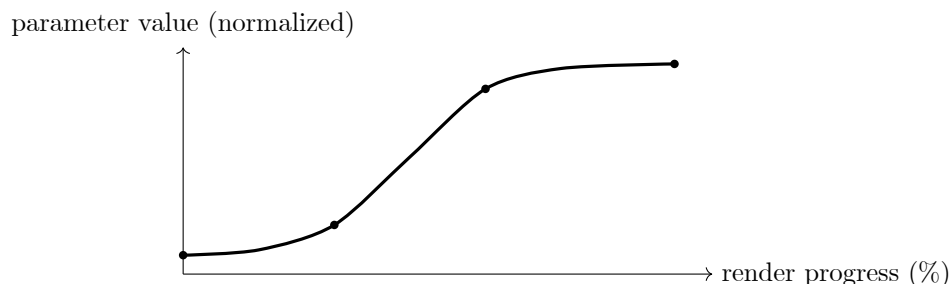


Figure 1.31: Automation treats a parameter as a continuous curve across the render, not as a sequence of unrelated settings.

1.11 What the transform sees

For every analysis frame, pvx creates a complex spectrum $X_t[k]$. Its magnitude describes the energy at bin k ; its phase describes that component’s cycle position. A standard short-time Fourier transform is:

$$X_t[k] = \sum_{n=0}^{N-1} x[n + tH_a]w[n]e^{-j2\pi kn/N}$$

where symbols and units follow the definitions established for what the transform sees.

The window $w[n]$ softens frame boundaries. Overlap between windows lets the reconstructed signal remain smooth. A narrow window follows quick attacks well but provides less precise pitch information. A longer

window gives a clearer view of closely spaced frequencies but can smear a drum hit or consonant across time. There is no universal perfect choice; musical material decides.

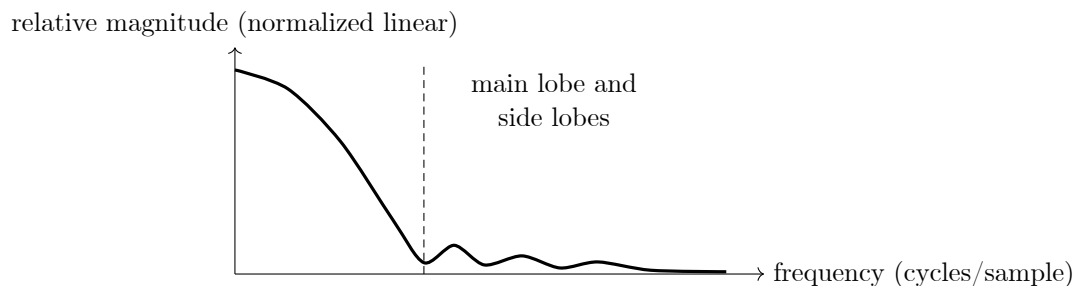


Figure 1.32: A conceptual window response. Window choice balances temporal detail against frequency selectivity.

1.12 A small comparison

The table below gathers the principal details of a small comparison.

Table 1.7: Three ways to think about changing recorded sound.

Method	What it changes directly	Characteristic limitation
Tape varispeed	Playback speed	Pitch and duration remain coupled.
Segmented tape systems	Short physical segments	Joins and repeats can be audible.
Phase vocoder	Spectral frames and phase trajectories	Transients and dense attacks need careful handling.

1.13 How pvx puts the idea to work

In pvx, the phase vocoder is the engine beneath time stretching, pitch shifting, freeze-like spectral holds, formant-aware workflows, and many automated transformations. The stable approach is usually modest: choose a sensible window and hop, preserve transients when the material needs it, use phase locking for harmonic sources, and audition a small section before committing to an extreme render.

```
pvx voc voice.wav --stretch 1.25 --transient-preserve --phase-locking identity --output voice_longer.wav
pvx voc piano.wav --pitch -3 --formant-preserve --output piano_down.wav
pvx freeze cymbal.wav --freeze-time 0.42 --duration 12 --output cymbal_cloud.wav
```

The commands look compact because the conceptual work is inside the analysis loop. The rest of this guide turns those choices into repeatable practice.

Part II

Orientation and Core Workflow

CHAPTER 2

Getting Started with pvx

This guide is for first-time users who want to understand what **pvx** does, why it exists, and how to get useful results without treating digital signal processing (DSP) as magic. It is practical first, mystical later.

2.1 Quick Setup (Install + PATH)

The following session demonstrates the quick setup (install + PATH) in practice.

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e .
pvx --help
```

Same setup with uv:

```
uv venv .venv
source .venv/bin/activate
uv pip install -e .
uv run pvx --help
```

If **pvx** is not found, add the project virtualenv to your path environment variable (PATH) (zsh):

```
printf 'export PATH="%s/.venv/bin:$PATH"\n' "$(pwd)" >> ~/.zshrc
source ~/.zshrc
pvx --help
```

No-PATH fallback:

```
.venv/bin/pvx voc input.wav --stretch 1.2 --output output.wav
# or
\index{or}
python3 -m pvx.cli.pvx voc input.wav --stretch 1.2 --output output.wav
```

No-PATH fallback with uv:

```
uv run pvx voc input.wav --stretch 1.2 --output output.wav
```

Man-page generation:

```
python3 scripts/scripts_install_man_pages.py
MANPATH="$(pwd)/man:$MANPATH" man pvx
```

2.2 Launch-Ready Helper Commands

Before announcing or demoing, run these:

```

pvx doctor
pvx quickstart input.wav --output output.wav
pvx safe input.wav --material mix --output output_safe.wav
pvx transforms
pvx smoke --output smoke_out.wav
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
pvx augment-manifest validate aug_out/augment_manifest.jsonl --strict

```

What each does: - **pvx doctor**: checks environment, path environment variable (PATH), and optional dependencies. - **pvx quickstart**: prints the minimal launch command sequence. - **pvx safe**: runs pvx voc with conservative quality-first defaults. - **pvx transforms**: explains transform options and runtime availability. - **pvx smoke**: executes a fast synthetic end-to-end sanity render. - **pvx augment**: builds deterministic augmentation datasets and writes manifests for machine-learning pipelines. - **pvx augment-manifest**: validates, merges, and summarizes augmentation manifests.

2.3 Running Any Command with uv

uv can run every command in this guide without changing the DSP arguments.

- `pvx ... -> uv run pvx ...`
- `python3 some_script.py ... -> uv run python3 some_script.py ...`
- `python3 -m module.path ... -> uv run python3 -m module.path ...`

Example:

```

pvx voc input.wav --stretch 1.25 --output out.wav
uv run pvx voc input.wav --stretch 1.25 --output out.wav

```

If you are already muttering at your shell, that is normal.

2.4 First 3 Commands to Run

This reference introduces the first 3 commands to run before presenting its individual entries.

```

# Stretch without pitch change
\index{Stretch without pitch change}
pvx voc input.wav --stretch 1.25 --output input_stretch.wav

# Pitch change without duration change
\index{Pitch change without duration change}
pvx voc input.wav --stretch 1.0 --pitch -3 --output input_pitch.wav

# Same pitch shift, but with formant protection
\index{Same pitch shift but with formant protection}
pvx voc input.wav --stretch 1.0 --pitch -3 --pitch-mode formant-preserving --output input_pitch_formant.wav

```

What to listen for: - **input_stretch.wav**: longer timing, same note center - **input_pitch.wav**: lower note center, possible timbral darkening - **input_pitch_formant.wav**: lower note center with more stable vowel/timbre identity - if all three sound identical, double-check that you did not typo the output path at 2 a.m.

2.5 What Problem pvx Solves

Audio workflows often need one or more of the following, with controllable artifacts: - change duration without changing pitch - change pitch without changing duration - apply time-varying pitch/time trajectories from

a map - preserve transients and formants where possible - process many files consistently from the command line

pvx gives you explicit control over those operations using phase-vocoder/short-time Fourier transform (STFT) methods plus specialized companion tools (freeze, harmonize, morph, retune, denoise, dereverb).

Design priority: - quality first: preserve musical intent and minimize artifacts - speed second: optimize runtime after quality targets are achieved - translation: we would rather be right than merely fast

2.6 Analog Tape Methods for Pitch/Time Shifting

Historically, reel-to-reel varispeed coupled time and pitch. Analog systems like the Eltro information rate changer and related Anton Springer regulator concepts were early attempts to separate them using segmented rotating-head playback methods instead of simple fixed-speed tape transport.

Why this matters for **pvx**: - artifact control is largely a continuity problem (then: segment stitching; now: phase/transient/stereo continuity), - high-quality time/pitch work usually benefits from conservative, staged processing.

Sources: - [Eltro information rate changer \(Wikipedia\)](#) - [Vintage Technologies: The Eltro and the Voice of HAL \(Wendy Carlos\)](#) - [Anton Springer and the Time and Pitch Regulator \(Sound and Science\)](#)

2.7 Basic DSP Terms (Plain Language)

Basic DSP Terms (Plain Language) depends on a few concrete choices.

- Sample rate: how many audio samples per second (e.g. 48,000 Hz).
- Frame: a short window of audio processed as one block.
- STFT: repeated short Fourier transforms over overlapping frames.
- Bin: one frequency slot in an STFT frame.
- Window: taper applied to each frame to reduce spectral leakage.
- Hop size: frame advance between adjacent STFT frames.
- Phase coherence: consistency of phase evolution across frames/bins.
- Transient: short attack event (drum hit, consonant burst).
- Formant: resonant spectral envelope that carries vowel/timbre identity.

2.8 Supported File Types

pvx audio I/O is provided by `soundfile/libsndfile`, so exact format support depends on your runtime build.

- Quick summary: `wav`, `flac`, `aiff`, `ogg`, `caf` are supported for stream output and are safe defaults.
- Full current format table: `FILE_TYPES.md`

2.9 Stretch vs Pitch Shift

For stretch vs pitch shift, start with these practical details.

- Stretch controls duration.
 - `stretch = 2.0` means output is twice as long.
 - `stretch = 0.5` means output is half as long.
- Pitch shift controls musical frequency position.
 - `+12 semitones` = one octave up.
 - `-12 semitones` = one octave down.

In **pvx** `voc`, duration and pitch can be controlled independently:

```
pvx voc input.wav --stretch 1.25 --pitch 0 --output stretched.wav
pvx voc input.wav --stretch 1.0 --pitch 3 --output pitched.wav
```

2.10 Denoising with Phase-Consistent STFT Processing

`pvx denoise` uses short-time Fourier transform (STFT) spectral subtraction while preserving phase continuity in overlap-add resynthesis.

Starter profiles:

```
# Speech-safe (conservative)
\index{Speech-safe (conservative)}
pvx denoise speech.wav --noise-seconds 0.4 --reduction-db 5 --floor 0.2 --smooth 9 --output speech_clean.wav

# Music-safe (preserve more ambience/harmonics)
\index{Music-safe (preserve more ambience/harmonics)}
pvx denoise mix.wav --noise-seconds 0.3 --reduction-db 4 --floor 0.25 --smooth 7 --output mix_clean.wav
```

If you hear chirpy/watery artifacts: - reduce `--reduction-db` - increase `--smooth` - raise `--floor` slightly
 - use `--noise-file roomtone.wav` when the beginning of the source is not a clean noise-only segment

2.11 Time-Varying Parameter Control (CSV/JSON)

You can pass a control file directly to many core phase-vocoder numeric flags.

Examples:

```
pvx voc input.wav --stretch controls/stretch.csv --interp linear --output out.wav
pvx voc input.wav --pitch-shift-ratio controls/pitch.json --interp polynomial --order 3 --output out.wav
pvx voc input.wav --n-fft controls/nfft.csv --hop-size controls/hop.csv --output out.wav
```

Common flags that accept scalar values or control files (`.csv` / `.json`): - time/pitch trajectory: `--stretch`, `--time-stretch`, `--pitch-shift-ratio`, `--pitch-shift-semitones`, `--pitch-shift-cents` - frame/spectral resolution: `--n-fft`, `--win-length`, `--hop-size`, `--kaiser-beta` - phase/transient/stereo shaping: `--ambient-phase-mix`, `--transient-threshold`, `--transient-sensitivity`, `--transient-protect-ms`, `--transient-crossfade-ms`, `--coherence-strength` - multistage/Fourier-sync tuning: `--extreme-stretch-threshold`, `--max-stage-stretch`, `--fourier-sync-min-fft`, `--fourier-sync-max-fft`, `--fourier-sync-smooth` - onset/formant shaping: `--onset-credit-pull`, `--onset-credit-max`, `--formant-lifter`, `--formant-strength`, `--formant-max-gain-db`

Control interpolation options: `--interp none` (stairstep / no interpolation) - `--interp linear` (default) - `--interp nearest` - `--interp cubic` - `--interp polynomial --order N` (any integer $N \geq 1$, default $N=3$; effective degree is $\min(N, \text{control_points}-1)$)

Polynomial order examples: `--interp polynomial --order 1` - `--interp polynomial --order 2` - `--interp polynomial --order 3` - `--interp polynomial --order 5`

Point-style CSV:

```
time_sec,value
0.0,1.0
0.5,1.2
1.0,1.8
```

Segment-style CSV:

```
start_sec,end_sec,value
0.0,0.4,1.0
0.4,0.8,1.2
0.8,1.2,1.6
```

JSON points:

```
{
  "interpolation": "linear",
  "points": [
    {"time_sec": 0.0, "value": 1.0},
    {"time_sec": 0.5, "value": 1.2},
    {"time_sec": 1.0, "value": 1.8}
  ]
}
```

JSON schema quick reference:

Key	Required	Type	Meaning
interpolation / interp	no	string	Interpolation override (none, linear, nearest, cubic, exponential, s_curve, smootherstep, polynomial)
order	no	integer	Polynomial order for polynomial interpolation
points	yes (point mode)	array	Time/value points
points[].time_sec	yes	number	Timestamp in seconds
points[].value	yes	number/string	Parameter value
segments	yes (segment mode)	array	Piecewise constant control regions
segments[].start_sec, segments[].end_sec	yes	number	Segment boundaries in seconds
segments[].value	yes	number/string	Segment value
parameters	no	object	Multi-parameter container keyed by parameter name

Notes: - per-parameter dynamic controls cannot be combined with legacy `--pitch-map` / `--pitch-map-stdin` in the same command - dynamic `--time-stretch` cannot be combined with `--target-duration`

Interpolation graph examples:

Mode	Example curve
none (stairstep)	
nearest	
linear	
cubic	
exponential	

Mode	Example curve
s_curve (smoothstep)	
smootherstep	
polynomial order 1	
polynomial order 2	
polynomial order 3	
polynomial order 5	

Core processing function charts:

Function family	Graph
Pitch ratio vs semitones	
Dynamics transfer curves	
Soft clip transfer functions	
Morph blend magnitude curves	
Mask exponent response	

2.12 Visual Mental Model of STFT

The following session demonstrates the visual mental model of STFT in practice.

```

Time-domain signal
x[n] ----->

Frame + windowing
    [w[n] * x[n:n+N]]
      [w[n] * x[n+H:n+H+N]]
        [w[n] * x[n+2H:n+2H+N]]

Per-frame transform
    FFT -> X0[k]
    FFT -> X1[k]
    FFT -> X2[k]

Process in spectral domain
    modify magnitude/phase trajectories

Synthesis
    IFFT + overlap-add -> y[n]

```

Key tradeoff: - larger N (FFT) -> better frequency resolution, worse time localization - smaller N -> better time localization, rougher low-frequency precision - yes, this is the classic engineering compromise you were hoping to avoid

2.13 Example Audio Scenarios

These are the details that matter for example audio scenarios.

- Dialogue cleanup + mild timing correction: `pvx voc + pvx denoise + pvx deverb`.
- Ambient texture from short sample: `pvx voc --preset ambient --target-duration ....`
- Vocal harmonies: `pvx harmonize` with interval and pan controls.
- Timeline-locked effects: `pvx conform / pvx warp` with CSV maps.

Use-Case Matrix (Where to start quickly)

The relevant properties of the use-case matrix (where to start quickly) are compared in the following table.

Use case	First command to try
Speech slowdown for transcription	<code>pvx voc speech.wav --preset vocal_studio --stretch 1.25 --output speech_slow.wav</code>
Podcast timing cleanup	<code>pvx voc voice.wav --stretch 0.95 --output voice_tight.wav</code>
Vocal pitch correction	<code>pvx retune vocal.wav --scale major --root C --output-dir out --suffix _retune</code>
Harmonic backing voices	<code>pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.8,0.65 --output-dir out</code>
Unison widen synth	<code>pvx unison synth.wav --voices 7 --detune-cents 16 --width 1.1 --output-dir out</code>
Long ambient from short source	<code>pvx voc oneshot.wav --preset extreme_ambient --target-duration 600 --output ambient.wav</code>
Drum-safe stretch	<code>pvx voc drums.wav --preset drums_safe --stretch 1.3 --output drums_stretch.wav</code>
Stereo image stability	<code>pvx voc mix.wav --stretch 1.15 --stereo-mode mid_side_lock --coherence-strength 0.9 --output mix_lock.wav</code>
Noise reduction pre-pass	<code>pvx denoise field.wav --reduction-db 8 --output-dir out --suffix _den</code>
Room tail reduction	<code>pvx deverb room.wav --strength 0.5 --output-dir out --suffix _dry</code>
CSV time/pitch choreography	<code>pvx conform source.wav --map map_conform.csv --output-dir out</code>
Automated quality comparison	<code>uv run python3 benchmarks/run_bench.py --quick --out-dir benchmarks/out --gate --baseline benchmarks/baseline_small.json</code>

2.14 Step-by-Step Walkthrough (One Small WAV)

Assume `sample.wav` is about 2-5 seconds long.

Step A: Baseline stretch

The example below puts Step A: baseline stretch into executable form.

```
pvx voc sample.wav --stretch 1.2 --output sample_stretch.wav
```

Expected: - 20% longer duration - same pitch - slight smearing possible on sharp transients

Step B: Add transient protection

The following session demonstrates Step B: add transient protection in practice.

```
pvx voc sample.wav --stretch 1.2 --transient-preserve --phase-locking identity --output
sample_stretch_transient.wav
```

Expected: - attacks should feel tighter than Step A - fewer “swishy” transients

Step C: Pitch shift with formant preservation

The following session demonstrates Step C: pitch shift with formant preservation in practice.

```
pvx voc sample.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output sample_pitch_formant.
wav
```

Expected: - pitch lowered by 4 semitones - vowel/timbre identity more stable than plain shift

Step D: Auto profile planning

A working command makes Step D: auto profile planning concrete.

```
pvx voc sample.wav --auto-profile --auto-transform --explain-plan
```

Expected: - JSON plan with resolved profile, transform, and core processing settings - no audio output (plan only)

2.15 What Outputs Should Sound Like

These are the details that matter for what outputs should sound like.

- **sample_stretch.wav**: same pitch, longer phrase timing.
- **sample_stretch_transient.wav**: similar to above, but cleaner attack edges.
- **sample_pitch_formant.wav**: lower note center with less “character collapse.”

If artifacts are strong: - lower ratio magnitude - increase overlap (**--hop-size** smaller) - try preset **vocal** for speech/singing - test **--multires-fusion** for mixed content

2.16 New Quality Modes (Beginner Version)

The account of the new quality modes (beginner version) proceeds through the topics that follow.

Hybrid transient mode (recommended on speech/drums)

The example below puts the hybrid transient mode (recommended on speech/drums) into executable form.

```
pvx voc sample.wav \  
  --transient-mode hybrid \  
  --transient-sensitivity 0.6 \  
  --transient-protect-ms 30 \  
  --transient-crossfade-ms 10 \  
  --stretch 1.25 \  
  --output sample_hybrid.wav
```

What to expect: - steadier harmonic regions from phase-vocoder processing - cleaner attacks from WSOLA handling around detected onsets

Stereo coherence lock (recommended for wide stereo sources)

The following session demonstrates the stereo coherence lock (recommended for wide stereo sources) in practice.

```
pvx voc stereo_mix.wav \  
  --stereo-mode mid_side_lock \  
  --coherence-strength 0.9 \  
  --stretch 1.2 \  
  --output stereo_locked.wav
```

What to expect: - less left/right image wobble - more stable center image after heavy stretch

2.17 Intent Presets

Use presets when you do not want to tune many flags:

- `--preset vocal_studio`: formant-aware vocal defaults + transient hybrid handling
- `--preset drums_safe`: WSOLA-heavy transient safety for percussive content
- `--preset extreme_ambient`: extreme long-form ambient settings
- `--preset stereo_coherent`: stereo coupling defaults

Legacy presets remain available: `vocal`, `ambient`, `extreme`.

2.18 Simpler One-Line Pipelines

If you do not want long Unix pipe chains, use managed helpers:

```
pvx follow guide.wav target.wav --output target_follow.wav --emit pitch_to_stretch --pitch-conf-min 0.75  
pvx chain sample.wav --pipeline "voc --stretch 1.2 | formant --mode preserve" --output sample_chain.wav  
pvx stream sample.wav --output sample_stream.wav --chunk-seconds 0.2 --time-stretch 2.0  
pvx stream sample.wav --mode wrapper --output sample_stream_wrapper.wav --chunk-seconds 0.2 --time-stretch  
2.0  
pvx stretch-budget sample.wav --disk-budget 20GB --bit-depth 16 --requested-stretch 1000000
```

- `pvx follow` replaces long sidechain pipes for pitch/control-map-driven workflows.
- `pvx chain` runs serial stages with managed intermediate files.
- `pvx stream` defaults to a stateful chunk processor for smoother long-form continuity.
- `pvx stream --mode wrapper` keeps legacy segmented-wrapper behavior.
- `pvx stretch-budget` estimates maximum practical stretch before you commit to a long render.

2.19 Estimate Stretch Budget Before Extreme Jobs

Use this helper before very large ratios (100x, 1000x, 1000000x) so disk limits are explicit.

```
pvx stretch-budget input.wav --disk-budget 20GB --bit-depth 16
pvx stretch-budget input.wav --disk-budget 20GB --requested-stretch 1000000 --fail-if-exceeds --json
```

What it uses: - input shape (frames/channels/sample rate) - output storage assumption (`--output-format`, `--bit-depth` / `--subtype`) - budget (`--disk-budget` or free space at `--budget-path`) - headroom (`--safety-margin`, default 0.90)

Recommendation: - for production, prefer `--target-duration` over arbitrary huge ratios. - if you run extreme jobs, combine `--stretch-mode multistage` with `--auto-segment-seconds`, `--checkpoint-dir`, and `--resume`.

2.20 Feature Tracking for Sidechain Control

`pvx pitch-track` now emits feature vectors (not just pitch map fields), including: - pitch/voicing (`f0_hz`, `pitch_ratio`, `confidence`, `voicing_prob`, `pitch_stability`) - loudness/dynamics (`rms`, `rms_db`, `short_lufs_db`, `crest_factor_db`) - spectral features (`spectral_centroid_hz`, `spectral_flatness`, `spectral_flux`, `rolloff_hz`) - formants and cepstra (`formant_f1_hz`..`formant_f3_hz`, `mfcc_01`..`mfcc_N`) - rhythm markers (`tempo_bpm`, `beat_phase`, `downbeat_phase`, `onset_strength`, `transient_mask`) - MPEG-7-style descriptors (`mpeg7_*` columns, including audio spectrum envelope bands)

Use with control-bus routes:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - \
| pvx voc target.wav --control-stdin \
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
  --route stretch=affine(spectral_flux,0.03,1.0) \
  --route stretch=clip(stretch,0.85,1.5) \
  --output target_feature_follow.wav
```

For a larger gallery (single-feature, multi-feature, MFCC/MPEG-7 vector, and multi-guide workflows), see: - docs/FEATURE_SIDECHAIN_EXAMPLES.md

You can also print built-in command snippets directly from the command-line interface (CLI):

```
pvx follow --example
pvx follow --example all
pvx follow --example formant_onset
```

2.21 Output Policy Controls

All audio-output tools now share deterministic output policy flags:

- `--bit-depth {inherit,16,24,32f}`
- `--dither {none,tpdf}` and `--dither-seed`
- `--true-peak-max-dbtp`
- `--metadata-policy {none,sidecar,copy}`
- `--subtype` for explicit low-level output subtype override

2.22 Next Steps

Before applying next steps, keep these constraints in view.

- Run practical recipes and advanced use cases (72+): docs/EXAMPLES.md
- Learn architecture and DSP diagrams (26+): docs/DIAGRAMS.md
- Study equation-level behavior (31 sections): docs/MATHEMATICAL_FOUNDATIONS.md
- Use Python API directly: docs/API_OVERVIEW.md

- Use benchmark runner: `benchmarks/run_bench.py`

2.23 Extra Beginner Recipes (Quick Wins)

The recipes in this section turn the preceding ideas into repeatable working methods.

```
# Speech slowdown
\index{Speech slowdown}
pvx voc speech.wav --preset vocal_studio --stretch 1.30 --output speech_slow.wav

# Drum-safe stretch
\index{Drum-safe stretch}
pvx voc drums.wav --preset drums_safe --stretch 1.20 --output drums_safe.wav

# Stereo coherence lock
\index{Stereo coherence lock}
pvx voc mix.wav --stretch 1.2 --stereo-mode mid_side_lock --coherence-strength 0.9 --output mix_lock.wav

# Freeze a transient into a pad
\index{Freeze a transient into a pad}
pvx freeze hit.wav --freeze-time 0.22 --duration 12 --output hit_pad.wav

# Morph two sources
\index{Morph two sources}
pvx morph a.wav b.wav --alpha 0.45 --blend-mode carrier_a_envelope_b --output a_b_morph.wav

# True A->B trajectory morph in one command
\index{True A- B trajectory morph in one command}
pvx morph A.wav B.wav --alpha controls/alpha_curve.csv --interp linear --blend-mode linear --output
    A_to_B_morph.wav

# Major-scale retune
\index{Major-scale retune}
pvx retune vocal.wav --root C --scale major --strength 0.8 --output vocal_c_major.wav

# Alternate concert pitch retune (A4 = 432 Hz)
\index{Alternate concert pitch retune (A4 432 Hz)}
pvx retune vocal.wav --root A --scale minor --a4-reference-hz 432 --output vocal_a432.wav

# Explicit root fundamental retune (C4 ~= 261.6256 Hz)
\index{Explicit root fundamental retune (C4 261.6256 Hz)}
pvx retune vocal.wav --root-hz 261.6256 --scale major --output vocal_c4_root.wav

# Auto-recommend root fundamental from the source
\index{Auto-recommend root fundamental from the source}
pvx retune vocal.wav --recommend-root --scale minor --output vocal_auto_root.wav

# Denoise then dereverb
\index{Denoise then dereverb}
pvx denoise noisy.wav --reduction-db 8 --stdout | pvx deverb - --strength 0.3 --output noisy_clean.wav

# Denoise then stretch
\index{Denoise then stretch}
pvx denoise noisy.wav --reduction-db 6 --stdout | pvx voc - --stretch 2.0 --output noisy_clean_stretch.wav

# Generate an LFO control map (triangle) and apply it as time-varying stretch
\index{Generate an LFO control map (triangle) and apply it as time-varying stretch}
pvx lfo --wave triangle --duration 12 --frequency-hz 0.4 --center 1.0 --amplitude 0.2 --key stretch --output
    controls/stretch_tri.csv
pvx voc input.wav --stretch controls/stretch_tri.csv --interp linear --output input_tri_stretch.wav
```

Run them in order, listen after each step, and resist changing ten parameters at once unless chaos is the objective.

CHAPTER 3

pvx Quality Guide

This guide maps audible artifacts to concrete `pvx` `voc` fixes.

3.1 Fast Starting Presets

A tabular view makes the distinctions within the fast starting presets easier to inspect.

Material	Recommended preset	Why
Vocal/speech	<code>--preset vocal_studio</code>	formant-aware pitch + hybrid transient handling
Drums/percussion	<code>--preset drums_safe</code>	WSOLA-focused transient path
Wide stereo mix	<code>--preset stereo_coherent</code>	channel-coupled coherence
Extreme ambient	<code>--preset extreme_ambient</code>	multistage long-stretch strategy

3.2 Artifact -> Fix Table

The entries in this section organize the artifact -> fix table for comparison and practical use.

What you hear	Likely cause	Primary fixes
Attack smear / soft transients	transient regions processed purely in PV mode	<code>--transient-mode hybrid</code> or <code>--transient-mode wsola</code> ; raise <code>--transient-sensitivity</code> ; reduce <code>--transient-crossfade-ms</code>
Phasy tone / chorus blur	weak phase coherence	<code>--phase-locking identity</code> ; consider <code>--stereo-mode ref_channel_lock</code> with <code>--coherence-strength 0.7</code>
Stereo image wobble	per-channel phase drift	<code>--stereo-mode mid_side_lock</code> or <code>--stereo-mode ref_channel_lock --ref-channel 0</code>
Robotic vocal timbre after pitch shift	formant drift	<code>--pitch-mode formant-preserving</code> ; tune <code>--formant-lifter</code> and <code>--formant-strength</code>
Pumping/flat loudness	aggressive mastering chain	reduce compressor/limiter settings; disable unused mastering stages

What you hear	Likely cause	Primary fixes
Grainy extreme stretch	ratio too high for single-stage path	<code>--stretch-mode multistage;</code> lower <code>--max-stage-stretch</code> ; optionally <code>--multires-fusion</code>
Unexpected overs/clipped delivery files	output policy not constrained	set <code>--true-peak-max-dbtp</code> and target <code>--bit-depth</code> ; use <code>--dither tpdf</code> for PCM exports
Chirpy/watery denoise residuals	overly aggressive subtraction or weak profile estimate	lower <code>--reduction-db</code> ; raise <code>--smooth</code> ; raise <code>--floor</code> ; provide <code>--noise-file</code> room tone

3.3 Practical Recipes

The recipes in this section turn the preceding ideas into repeatable working methods.

Speech stretch with transient protection

The following session demonstrates the speech stretch with transient protection in practice.

```
pvx voc speech.wav \
  --preset vocal_studio \
  --transient-mode hybrid \
  --stretch 1.3 \
  --output speech_clean_stretch.wav
```

Drum-safe stretch

The example below puts the drum-safe stretch into executable form.

```
pvx voc drums.wav \
  --preset drums_safe \
  --time-stretch 1.4 \
  --output drums_safe.wav
```

Stereo coherence lock

The example below puts the stereo coherence lock into executable form.

```
pvx voc mix.wav \
  --stereo-mode ref_channel_lock \
  --ref-channel 0 \
  --coherence-strength 0.9 \
  --stretch 1.2 \
  --output mix_locked.wav
```

Denoise without musical-noise artifacts

The following session demonstrates the denoise without musical-noise artifacts in practice.

```
pvx denoise speech.wav \  
  --noise-seconds 0.4 \  
  --reduction-db 5 \  
  --floor 0.2 \  
  --smooth 9 \  
  --output speech_clean.wav
```

3.4 Parameter Ranges That Usually Work

A tabular view makes the distinctions within the parameter ranges that usually work easier to inspect.

Parameter	Typical range	Notes
<code>--transient-sensitivity</code>	0.45 .. 0.75	higher = more transient regions
<code>--transient-protect-ms</code>	20 .. 45	increase for noisy or percussive content
<code>--transient-crossfade-ms</code>	6 .. 14	larger values smooth boundaries
<code>--coherence-strength</code>	0.5 .. 0.95	higher = stronger channel coupling
<code>--max-stage-stretch</code>	1.2 .. 1.8	lower values are safer for extreme ratios

3.5 Debug Workflow

A reliable approach to debug workflow begins with these points.

1. Run `pvx voc --example all` to pick a close recipe.
2. Use `--dry-run --explain-plan` to inspect resolved settings.
3. Adjust one control at a time (`transient-mode`, then `coherence-strength`, then phase/window settings).
4. Compare A/B using short excerpts before full renders.
5. Only after quality is acceptable, optimize runtime (`--n-fft`, staging, segmentation/checkpoint options).

CHAPTER 4

Supported Surface

This page is the short version of what `pvx` expects alpha users to rely on.

If a longer generated reference lists more commands or flags, treat that as inventory, not a broader compatibility promise.

4.1 Stable in 0.1.x

These commands are the intended scripted/public alpha surface:

- `pvx`
- `pvxvoc`
- `pvxfreeze`
- `pvxwarp`
- `pvxformant`
- `pvxfilter`
- `pvxretune`
- `pvxanalysis`

4.2 Beta in 0.1.x

These are usable, but minor releases may still change flags or defaults:

- `pvxharmonize`
- `pvxconform`
- `pvxmorph`
- `pvxtransient`
- `pvxunison`
- `pvxdenoise`
- `pvxdeverb`
- `pvxlayer`
- `pvxresponse`
- `pvxenvelope`
- `pvxreshape`
- `pvxtvfilter`
- `pvxnoisefilter`
- `pvxbandamp`
- `pvxspeccompander`
- `pvxring`
- `pvxringfilter`
- `pvxringtvfilter`
- `pvxharmmap`

4.3 Experimental in 0.1.x

These are available for exploration, not for stable automation contracts:

- `hps-pitch-track`
- `pvxchordmapper`
- `pvxinharmonator`
- `pvxtrajectoryreverb`
- `pvxnoise`
- `pvxrir`
- `pvxcodec`
- `pvxspecaugment`
- `pvxgain`

4.4 Compatibility Shims

The `pvxalgorithms*` import aliases remain available during 0.1.x only to ease migration.

- Canonical imports: `pvx.algorithms*`
- Deprecated aliases: `pvxalgorithms`, `pvxalgorithms.base`, `pvxalgorithms.registry`
- Removal target: first 0.2.0 release unless the release notes say otherwise

4.5 Docs Guide

If you only need the supported user-facing paths, start with:

- README
- Getting Started
- Homebrew Install
- Alpha Release Guide

Use the generated references only when you need exhaustive flag/file inventory:

- CLI Flags Reference
- Python File Help

CHAPTER 5

Supported File Types

`pvx` uses `python-soundfile` (`soundfile`) backed by `libsndfile` for audio input/output (I/O). This means some format support is runtime/platform dependent.

5.1 I/O Behavior by Category

The table below gathers the principal details of the I/O behavior by category.

Category	Used by	File types
Audio input (file path)	<code>pvxvoc</code> , <code>pvxfreeze</code> , <code>pvxharmonize</code> , <code>pvxmorph</code> , <code>pvxretune</code> , <code>pvxdenoise</code> , <code>pvxdeverb</code> , and other audio CLIs	Any audio container supported by the active <code>soundfile</code> / <code>libsndfile</code> build (see full table below)
Audio input (stdin, -)	CLIs that accept - as input	Any container <code>soundfile</code> can decode from byte stream
Audio output (file path)	Audio CLIs with <code>--output-format</code> or inferred output extension	Any format <code>soundfile</code> can write in the active build
Audio output (stdout, --stdout)	Stream/pipeline mode in <code>pvxvoc</code> and common CLI helpers	Explicitly supported: <code>wav</code> , <code>flac</code> , <code>aiff/aif</code> , <code>ogg/oga</code> , <code>caf</code>
Control maps	<code>pvxvoc --pitch-map</code> , <code>pvxwarp -map</code> , <code>pvxconform --map</code>	<code>csv</code>
Time-varying per-parameter control signals	<code>pvxvoc --stretch stretch.csv</code> , <code>pvxvoc --pitch-shift-ratio pitch.json</code> , <code>pvxvoc --n-fft nfft.csv</code> , <code>pvx stream ... --stretch stretch.csv</code>	<code>csv</code> , <code>json</code>
Pitch tracking map output	<code>pvx pitch-track / hps-pitch-track</code>	<code>csv</code>
Render manifest	<code>pvxvoc --manifest-json</code>	<code>json</code>
Documentation output	docs generators	<code>html</code> , <code>pdf</code>

5.2 Full Audio Container List (Current Build)

The following formats are reported by the current local `soundfile`/`libsndfile` runtime:

	<u>Extension</u>	<u>Format</u>
aiff		AIFF (Apple/SGI)
au		AU (Sun/NeXT)
avr		AVR (Audio Visual Research)
caf		CAF (Apple Core Audio File)
flac		FLAC (Free Lossless Audio Codec)
htk		HTK (HMM Tool Kit)
ircam		SF (Berkeley/IRCAM/CARL)
mat4		MAT4 (GNU Octave 2.0 / Matlab 4.2)
mat5		MAT5 (GNU Octave 2.1 / Matlab 5.0)
mp3		MPEG-1/2 Audio
mpc2k		MPC (Akai MPC 2k)
nist		WAV (NIST Sphere)
ogg		OGG (OGG Container format)
paf		PAF (Ensoniq PARIS)
pvf		PVF (Portable Voice Format)
raw		RAW (header-less)
rf64		RF64 (RIFF 64)
sd2		SD2 (Sound Designer II)
sds		SDS (Midi Sample Dump Standard)
svx		IFF (Amiga IFF/SVX8/SV16)
voc		VOC (Creative Labs)
w64		W64 (SoundFoundry WAVE 64)
wav		WAV (Microsoft)
wavex		WAVEX (Microsoft)
wve		WVE (Psion Series 3)
xi		XI (FastTracker 2)

5.3 Verify on Your Machine

Use this command to print the exact formats available in your environment:

```
python3 - <<'PY'
import soundfile as sf
for ext, name in sorted(sf.available_formats().items()):
    print(f"{ext.lower():<8} {name}")
PY
```

Part III

Theory and Internal Model

CHAPTER 6

pvx Mathematical Foundations

Generated from commit **ca6797a** (commit date: 2026-07-30T08:48:43-04:00).

This document explains the core signal-processing equations used by pvx, with plain-English interpretation. All equations are written in GitHub-renderable LaTeX and are intended to render directly in normal GitHub Markdown view.

6.1 Notation

A reliable approach to notation begins with these points.

- Analysis hop: H_a
- Synthesis hop: H_s
- FFT size: N
- Frame index: t
- Frequency-bin index: k
- Frame phase: $\phi_t[k]$
- Bin-center angular frequency: $\omega_k = 2\pi k/N$
- Selected transform backend: $m \in \{\text{fft}, \text{dft}, \text{czt}, \text{dct}, \text{dst}, \text{hartley}\}$

6.2 STFT Analysis and Synthesis

Analysis transform:

$$X_t[k] = \sum_{n=0}^{N-1} x[n + tH_a]w[n]e^{-j2\pi kn/N}$$

where symbols and units follow the definitions established for stft analysis and synthesis.

Plain English: each frame $\mathbf{t}$ is windowed by $\mathbf{w}[n]$ and transformed into complex frequency bins $\mathbf{k}$.

Weighted overlap-add synthesis:

$$\hat{x}[n] = \frac{\sum_t \hat{x}_t[n - tH_s]w[n - tH_s]}{\sum_t w^2[n - tH_s] + \varepsilon}$$

where symbols and units follow the definitions established for stft analysis and synthesis.

Plain English: reconstructed frames are overlap-added, then normalized by accumulated window energy.

6.3 Transform Backend Families and Tradeoffs

pvx allows selecting the per-frame transform with `--transform` in STFT/ISTFT paths. The same overlap-add framework is used, but per-bin meaning and phase behavior differ by transform family.

Complex Fourier family (`fft`, `dft`):

$$X_t[k] = \sum_{n=0}^{N-1} x_t[n]e^{-j2\pi kn/N}, \quad x_t[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_t[k]e^{j2\pi kn/N}$$

where symbols and units follow the definitions established for transform backend families and tradeoffs.

Chirp-Z (**cz****t**, Bluestein-style contour):

$$X_t[k] = \sum_{n=0}^{N-1} x_t[n] A^{-n} W^{nk}$$

where symbols and units follow the definitions established for transform backend families and tradeoffs. With pvx defaults $A = 1$ and $W = e^{-j2\pi/N}$, CZT matches DFT samples but uses a different numerical path.

DCT-II / IDCT-II (**d****ct**):

$$C_t[k] = \alpha_k \sum_{n=0}^{N-1} x_t[n] \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)k\right)$$

where symbols and units follow the definitions established for transform backend families and tradeoffs. DST-II / IDST-II (**d****st**):

$$S_t[k] = \beta_k \sum_{n=0}^{N-1} x_t[n] \sin\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right)(k+1)\right)$$

where symbols and units follow the definitions established for transform backend families and tradeoffs. Discrete Hartley (**h****art****ley**):

$$H_t[k] = \sum_{n=0}^{N-1} x_t[n] \operatorname{cas}\left(\frac{2\pi kn}{N}\right), \quad \operatorname{cas}(\theta) = \cos(\theta) + \sin(\theta)$$

where symbols and units follow the definitions established for transform backend families and tradeoffs.

Transform	Strengths	Tradeoffs	Recommended use cases
fft	Fastest common path, stable, native one-sided complex bins, best CUDA coverage.	Needs careful window/hop tuning for extreme nonstationarity.	Default for most time-stretch, pitch-shift, and batch processing.
dft	Canonical Fourier definition, deterministic parity baseline versus FFT implementations.	Usually slower than fft , little practical quality advantage in normal runs.	Cross-checking, verification, or research comparisons.
cz t	Useful alternative numerical path for awkward frame sizes; can be robust for prime/non-power sizes.	Requires SciPy; typically CPU-only and slower than FFT.	Edge-case frame-size experiments and diagnostic reruns.
d ct	Real-valued compact basis with strong energy compaction for smooth envelopes.	Loses explicit complex phase; less transparent for strict phase-coherent resynthesis.	Spectral shaping, denoise-like creative processing, coefficient-domain experiments.
d st	Real-valued odd-symmetry basis; emphasizes different boundary behavior than DCT.	Same phase limitations as DCT; content-dependent coloration.	Creative timbre variants and odd-symmetry analysis experiments.

Transform	Strengths	Tradeoffs	Recommended use cases
hartley	Real transform using cas , often helpful for CPU-friendly exploratory analysis.	Different bin semantics vs complex STFT; can alter artifact profile.	Alternative real-basis phase-vocoder experiments and pedagogical comparisons.

Plain English: choose **fft** first, then move to other transforms when you specifically need different numerical behavior, basis structure, or artifact character.

Sample Transform Use Cases

A working command makes the sample transform use cases concrete.

```
pvx voc dialog.wav --transform fft --time-stretch 1.08 --transient-preserve --output-dir out
pvx voc test_tones.wav --transform dft --time-stretch 1.00 --output-dir out
pvx voc archival_take.wav --transform czt --n-fft 1536 --win-length 1536 --hop-size 384 --output-dir out
pvx voc strings.wav --transform dct --pitch-shift-cents -17 --soft-clip-level 0.94 --output-dir out
pvx voc percussion.wav --transform dst --time-stretch 0.92 --output-dir out
pvx voc synth.wav --transform hartley --phase-locking off --time-stretch 1.30 --output-dir out
```

6.4 Phase-Vocoder Frequency and Phase Update

The relation below states the phase-vocoder frequency and phase update precisely.

$$\Delta\phi_t[k] = \text{princarg}(\phi_t[k] - \phi_{t-1}[k] - \omega_k H_a)$$

where symbols and units follow the definitions established for phase-vocoder frequency and phase update.

$$\hat{\omega}_t[k] = \omega_k + \frac{\Delta\phi_t[k]}{H_a}$$

where symbols and units follow the definitions established for phase-vocoder frequency and phase update.

$$\hat{\phi}_t[k] = \hat{\phi}_{t-1}[k] + \hat{\omega}_t[k] H_s$$

where symbols and units follow the definitions established for phase-vocoder frequency and phase update.

Plain English: pvx estimates true per-bin frequency from wrapped phase deviation, then re-accumulates phase at the synthesis hop.

6.5 Time Stretch and Pitch Ratio

Pitch shift ratio from semitones/cents:

$$r_{\text{pitch}} = 2^{\Delta s/12} = 2^{\Delta c/1200}$$

where symbols and units follow the definitions established for time stretch and pitch ratio.

Plain English: semitone and cent controls map to the same multiplicative ratio.

6.6 Dynamic Control Interpolation (CSV/JSON)

When a numeric flag receives a CSV/JSON control track, pvx samples a control function $u(t)$ over render time.

Polynomial mode fits a global polynomial of degree d :

$$u(t) = \sum_{i=0}^d a_i t^i, \quad d = \min(\text{order}, M - 1)$$

where M is the number of control points. In command-line interface (CLI) usage, `--order` accepts any integer ≥ 1 . The effective degree is automatically capped to avoid over-specification when there are too few points.

Nearest/linear/cubic/exponential/S-curve are local interpolation modes; `none` is sample-and-hold (stairstep). `exponential` uses piecewise exponential easing and `s_curve/smoootherstep` use Hermite S-shaped easing between adjacent control points. Legend used in each plot: blue solid line = interpolated control curve $u(t)$, red dashed line = piecewise connection of control points, red circles labeled p_i = original control points.

Interpolation mode	CLI form	Example plot
none (stairstep)	<code>--interp none</code>	
nearest	<code>--interp nearest</code>	
linear	<code>--interp linear</code>	
cubic	<code>--interp cubic</code>	
exponential	<code>--interp exponential</code>	
S-curve (smoothstep)	<code>--interp s_curve</code>	
S-curve (smoootherstep)	<code>--interp smoootherstep</code>	
polynomial order 1	<code>--interp polynomial --order 1</code>	
polynomial order 2	<code>--interp polynomial --order 2</code>	
polynomial order 3	<code>--interp polynomial --order 3</code>	
polynomial order 5	<code>--interp polynomial --order 5</code>	

6.7 Microtonal Retune Mapping

For a detected frequency f , pvx scale-aware retuning chooses the nearest permitted scale target f_{scale} and applies:

$$\Delta s = 12 \log_2 \left(\frac{f_{\text{scale}}}{f} \right)$$

where symbols and units follow the definitions established for microtonal retune mapping.

Plain English: each frame is shifted only as much as needed to land on the selected microtonal scale degree.

6.8 Loudness and Dynamics

LUFS target gain:

$$g_{\text{LUFS}} = 10^{(L_{\text{target}} - L_{\text{in}})/20}, \quad y[n] = g_{\text{LUFS}} x[n]$$

where symbols and units follow the definitions established for loudness and dynamics.

Compressor gain law (conceptual):

$$g(x) = \begin{cases} 1, & |x| \leq T \\ \left(\frac{T + (|x| - T)/R}{|x|} \right), & |x| > T \end{cases}$$

where symbols and units follow the definitions established for loudness and dynamics.

Plain English: level is reduced above threshold T by ratio R , then makeup/loudness stages may be applied.

6.9 Spatial Coherence and Channel Alignment

Inter-channel phase difference:

$$\Delta\phi_{ij}(k, t) = \phi_i(k, t) - \phi_j(k, t)$$

where symbols and units follow the definitions established for spatial coherence and channel alignment.
Phase-drift objective:

$$J = \sum_{k,t} |\Delta\phi_{ij}^{\text{out}}(k, t) - \Delta\phi_{ij}^{\text{in}}(k, t)|$$

where symbols and units follow the definitions established for spatial coherence and channel alignment.
Channel delay estimate by phase-weighted cross-correlation:

$$\tau^* = \arg \max_{\tau} \mathcal{F}^{-1} \left\{ \frac{X_i(\omega)X_j^*(\omega)}{|X_i(\omega)X_j^*(\omega)| + \varepsilon} \right\}(\tau)$$

where symbols and units follow the definitions established for spatial coherence and channel alignment.

Plain English: pvx spatial modules prioritize channel coherence, stable image cues, and robust delay alignment for multichannel processing chains.

6.10 Function Graph Gallery

The plots below visualize core transfer functions and parameter curves used across pvx processing modules.

Function family	Plot	Why this matters
Pitch ratio from semitones		<code>r = 2^(semitones/12)</code> conversion used by pitch-shift flags.
Pitch ratio from cents		<code>r = 2^(cents/1200)</code> conversion for microtonal cent controls.
Dynamics transfer functions		Compressor/expander/limiter conceptual transfer curves used in mastering stages.
Soft-clip transfer functions		Shows <code>tanh</code> , <code>arctan</code> , and <code>cubic</code> soft clipping behavior.
Morph blend magnitude curves		Compares <code>linear</code> , <code>geometric</code> , <code>product</code> , <code>max_mag</code> , and <code>min_mag</code> blend behaviors.
Mask exponent response		Shows how <code>--mask-exponent</code> shapes cross-synthesis mask gain.
Phase mix to output angle		Example of complex-vector phase blending used by morph phase mix.

CHAPTER 7

pvx Architecture

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

System architecture for runtime processing, algorithm dispatch, and documentation pipelines.

7.1 Runtime and CLI Flow

The material below develops the runtime and CLI flow in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.2 Algorithm Registry and Dispatch

The next example makes the role of the algorithm registry and dispatch explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.3 Documentation Build Graph

The visual material in this section develops the documentation build graph through annotated examples.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

7.4 CI + Pages

The next example makes the role of the ci + pages explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

CHAPTER 8

pvx Window Reference

This chapter is a mathematical and visual atlas of every analysis window supported by pvx. A window is not a decorative preprocessing choice. It determines which samples contribute most strongly to each frame, how a sinusoid spreads across bins, how much neighbouring partials interfere, and how sharply a transient can be located in time.

8.1 Notation

The notation below is used throughout the atlas. Each symbol is restated in the where clause following every equation so an entry can be read independently.

The principal quantities are N , the window length; n , the sample index in $0, \dots, N-1$; $m = (N-1)/2$, the center index; $w[n]$, the window coefficient; and $W(e^{j\omega})$, the discrete-time Fourier transform of the window.

8.2 Formula Families

The 50 named windows map to 19 formula families. The constants and parameter values in each atlas entry identify the exact member used by pvx.

W0

The relation below states the w0 precisely.

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N-1$.

W1

The mathematical form of the w1 is given in the next equation.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

W2

A compact equation captures the central relation in the w2.

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

W3

The relation below states the w3 precisely.

$$w[n] = 1 - \left| \frac{n-m}{m} \right|$$

where $m = (N-1)/2$ is the center sample, n is the sample index, and N is the window length.

W4

The mathematical form of the w4 is given in the next equation.

$$w[n] = \max\left(1 - \left| \frac{n-m}{(N+1)/2} \right|, 0\right)$$

where $m = (N-1)/2$ is the center sample, n is the sample index, and N is the window length.

W5

A compact equation captures the central relation in the w5.

$$x = \frac{n}{N-1} - \frac{1}{2}, \quad w[n] = 0.62 - 0.48|x| + 0.38 \cos(2\pi x)$$

where x is the centered normalized position, n is the sample index, and N is the window length.

W6

The relation below states the w6 precisely.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos\left(\pi \left(\frac{2x}{\alpha} - 1 \right)\right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos\left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right)\right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N-1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

W7

The mathematical form of the w7 is given in the next equation.

$$w[n] = \begin{cases} 1 - 6u^2 + 6u^3, & 0 \leq u \leq 1/2 \\ 2(1-u)^3, & 1/2 < u \leq 1 \\ 0, & u > 1 \end{cases}$$

where $u = |2n/(N-1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

W8

A compact equation captures the central relation in the w8.

$$w[n] = \text{sinc}\left(\frac{2n}{N-1} - 1\right)$$

where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$, n is the sample index, and N is the window length.

W9

The relation below states the w9 precisely.

$$w[n] = \max(1 - x_n^2, 0)$$

where $x_n = (n - m)/m$ is centered normalized position, $m = (N - 1)/2$, and N is the window length.

W10

The mathematical form of the w10 is given in the next equation.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n - m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N - 1)/2$, and n is the sample index.

W11

A compact equation captures the central relation in the w11.

$$w[n] = \exp\left[-\frac{1}{2}\left|\frac{n - m}{\sigma}\right|^{2p}\right]$$

where p controls shoulder shape, σ controls spread, $m = (N - 1)/2$, and n is the sample index.

W12

The relation below states the w12 precisely.

$$w[n] = \exp\left(-\frac{|n - m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N - 1)/2$, and n is the sample index.

W13

The mathematical form of the w13 is given in the next equation.

$$w[n] = \frac{1}{1 + \left(\frac{n - m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N - 1)/2$, and n is the sample index.

W14

A compact equation captures the central relation in the w14.

$$w[n] = \sin^p\left(\frac{\pi n}{N - 1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

W15

The relation below states the w15 precisely.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n - m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N - 1)/2$.

W16

The mathematical form of the w16 is given in the next equation.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

W17

A compact equation captures the central relation in the w17.

$$w[n] = (1 - x) \cos(\pi x) + \frac{\sin(\pi x)}{\pi}$$

where $x = |2n/(N-1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

W18

The relation below states the w18 precisely.

$$w[n] = \frac{I_0(\beta\sqrt{1-r_n^2})}{I_0(\beta)}, \quad r_n = \frac{n-m}{m}$$

where I_0 is the modified Bessel function, β controls taper strength, $m = (N-1)/2$, and n is the sample index.

8.3 Quantitative Metrics

The comparison uses coherent gain, equivalent noise bandwidth, scalloping loss, main-lobe width, and peak sidelobe level. Together these measurements describe amplitude calibration, noise integration, sensitivity to frequencies between FFT bins, frequency discrimination, and leakage rejection.

$$CG = \frac{1}{N} \sum_{n=0}^{N-1} w[n]$$

where CG is coherent gain, $w[n]$ is the window coefficient, n is the sample index, and N is the window length.

$$ENBW = N \frac{\sum_{n=0}^{N-1} w^2[n]}{\left(\sum_{n=0}^{N-1} w[n]\right)^2}$$

where $ENBW$ is equivalent noise bandwidth measured in FFT bins, $w[n]$ is the window coefficient, n is the sample index, and N is the window length.

$$SL = 20 \log_{10} \left| \frac{W(2\pi/(2N))}{W(0)} \right|$$

where SL is half-bin scalloping loss in decibels, $W(\omega)$ is the window spectrum, and N is the window length.

8.4 Comparative Window Summary

The following landscape table is deliberately limited to the quantities most useful for first selection. The atlas entries that follow carry the parameters, complete formula, strengths, limitations, advice, and full-size graphs.

Table 8.1: Comparison of all pvx analysis windows.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
hann	Cosine series	0.499756	1.500733	4.000	-31.468	Default choice for most pvx time-stretch and pitch-shift workflows.
hamming	Cosine series	0.539775	1.363305	4.000	-42.675	Default choice for most pvx time-stretch and pitch-shift workflows.
blackman	Cosine series	0.419795	1.727601	6.000	-58.109	Use for dense harmonic material when leakage artifacts dominate.
blackmanharris	Cosine series	0.358575	2.005332	8.000	-92.011	Use for dense harmonic material when leakage artifacts dominate.
nutall	Cosine series	0.355594	2.022220	8.000	-93.325	Use for dense harmonic material when leakage artifacts dominate.
flattop	Cosine series	0.999514	3.772117	10.000	-68.311	Use for measurement-grade magnitude tracking, not fine pitch separation.
blackman_nuttall	Cosine series	0.363405	1.977073	8.000	-98.174	Use for dense harmonic material when leakage artifacts dominate.
exact_blackman	Cosine series	0.426386	1.694500	6.000	-68.236	Use for dense harmonic material when leakage artifacts dominate.
sine	Sinusoidal	0.636309	1.234304	3.000	-22.999	Use for stable, low-complexity alternatives to Hann.
bartlett	Triangular	0.499756	1.333985	4.000	-26.523	Use for low-cost processing or quick exploratory runs.
boxcar	Rectangular	1.000000	1.000000	2.000	-13.264	Use only for controlled test tones or when leakage is acceptable.
triangular	Triangular	0.500244	1.332683	4.000	-26.523	Use for low-cost processing or quick exploratory runs.
bartlett_hann	Hybrid taper	0.499756	1.456559	4.000	-35.874	Use as a middle-ground taper when Bartlett feels too sharp.
tukey	Tukey	0.749634	1.222819	2.656	-15.123	Use when you need a controllable flat center with soft edges.
tukey_0p1	Tukey	0.949536	1.039289	2.094	-13.309	Use when you need a controllable flat center with soft edges.
tukey_0p25	Tukey	0.874573	1.102579	2.281	-13.601	Use when you need a controllable flat center with soft edges.
tukey_0p75	Tukey	0.624695	1.360664	3.188	-19.395	Use when you need a controllable flat center with soft edges.
tukey_0p9	Tukey	0.549731	1.446988	3.625	-24.972	Use when you need a controllable flat center with soft edges.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
parzen	Polynomial	0.374817	1.918397	8.000	-53.046	Use for spectral denoising/analysis where sidelobe cleanup is critical.
lanczos	Sinc	0.589202	1.299668	3.281	-26.405	Use for interpolation-adjacent analysis experiments.
welch	Quadratic	0.666341	1.200587	2.875	-21.295	Use when center weighting is desired with minimal complexity.
gaussian_0p25	Gaussian	0.313156	2.258144	11.406	-87.677	Use sigma presets to tune transient locality versus spectral leakage.
gaussian_0p35	Gaussian	0.436580	1.626488	6.719	-52.077	Use sigma presets to tune transient locality versus spectral leakage.
gaussian_0p45	Gaussian	0.548949	1.320556	4.125	-35.366	Use sigma presets to tune transient locality versus spectral leakage.
gaussian_0p55	Gaussian	0.641515	1.171861	3.000	-28.906	Use sigma presets to tune transient locality versus spectral leakage.
gaussian_0p65	Gaussian	0.713490	1.097657	2.625	-22.575	Use sigma presets to tune transient locality versus spectral leakage.
general_gaussian_1p5_0p35	Generalized Gaussian	0.393587	2.016584	5.156	-24.895	Use for research/tuning tasks where shoulder steepness matters.
general_gaussian_2p0_0p35	Generalized Gaussian	0.377081	2.230016	5.281	-19.717	Use for research/tuning tasks where shoulder steepness matters.
general_gaussian_3p0_0p35	Generalized Gaussian	0.364287	2.445593	5.469	-16.299	Use for research/tuning tasks where shoulder steepness matters.
general_gaussian_4p0_0p35	Generalized Gaussian	0.359267	2.552433	5.562	-15.060	Use for research/tuning tasks where shoulder steepness matters.
exponential_0p25	Exponential	0.245310	2.075492	15.562	-31.885	Use for experiments emphasizing center-heavy weighting.
exponential_0p5	Exponential	0.432187	1.313323	3.625	-19.190	Use for experiments emphasizing center-heavy weighting.
exponential_1p0	Exponential	0.631991	1.082055	2.594	-20.141	Use for experiments emphasizing center-heavy weighting.
cauchy_0p5	Cauchy / Lorentzian	0.553402	1.229775	3.562	-22.733	Use when you want softer attenuation of far-window samples.
cauchy_1p0	Cauchy / Lorentzian	0.785259	1.041963	2.375	-19.034	Use when you want softer attenuation of far-window samples.
cauchy_2p0	Cauchy / Lorentzian	0.927233	1.004393	2.094	-14.689	Use when you want softer attenuation of far-window samples.
cosine_power_2	Cosine power	0.499756	1.500733	4.000	-31.468	Use to smoothly increase edge attenuation versus basic sine.

Window	Family	CG	ENBW	Main lobe	Sidelobe	Recommended use
<code>cosine_power_3</code>	Cosine power	0.424206	1.735739	5.000	-39.295	Use to smoothly increase edge attenuation versus basic sine.
<code>cosine_power_4</code>	Cosine power	0.374817	1.945394	6.000	-46.741	Use to smoothly increase edge attenuation versus basic sine.
<code>hann_poisson_0p5</code>	Hann-Poisson	0.432946	1.609944	5.188	-35.245	Use when you need stronger edge decay than Hann without full flattop cost.
<code>hann_poisson_1p0</code>	Hann-Poisson	0.378797	1.734176	103.750	-80.017	Use when you need stronger edge decay than Hann without full flattop cost.
<code>hann_poisson_2p0</code>	Hann-Poisson	0.297878	2.023174	164.719	-80.005	Use when you need stronger edge decay than Hann without full flattop cost.
<code>general_hamming_0p50</code>	General Hamming	0.499756	1.500733	4.000	-31.468	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p60</code>	General Hamming	0.599805	1.222475	3.469	-31.600	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p70</code>	General Hamming	0.699854	1.091920	2.656	-24.078	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>general_hamming_0p80</code>	General Hamming	0.799902	1.031273	2.312	-18.649	Use to sweep sidelobe-vs-width behavior near Hamming family defaults.
<code>bohman</code>	Bohman	0.405087	1.786613	6.000	-45.997	Use for exploratory spectral work needing smooth derivatives.
<code>cosine</code>	Sinusoidal	0.636309	1.234304	3.000	-22.999	Use for stable, low-complexity alternatives to Hann.
<code>kaiser</code>	Kaiser-Bessel	0.331708	2.162181	9.125	-105.921	Use when you need explicit control over leakage versus resolution.
<code>rect</code>	Rectangular	1.000000	1.000000	2.000	-13.264	Use only for controlled test tones or when leakage is acceptable.

8.5 Complete Window Atlas

Each entry gives the formula, measured properties, two plots, and a short listening note. Read the time-domain plot as a map of which samples influence a frame. Read the spectrum as the cost of that choice: narrow main lobes help separate nearby partials, while low sidelobes keep strong components from masking weak ones.

For listening tests, hold the FFT size and hop constant and compare the candidate against Hann. Use a short passage that contains both attacks and sustained tones. Listen for attack shape, tonal focus, high-frequency haze, and stereo stability, then verify overlap-add behavior before adopting the window for a long render.

hann

hann is a cosine series design with `coeffs=(0.5, -0.5)`. Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.499756, and its equivalent noise bandwidth is 1.500733 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.2: Measured properties of the hann window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB

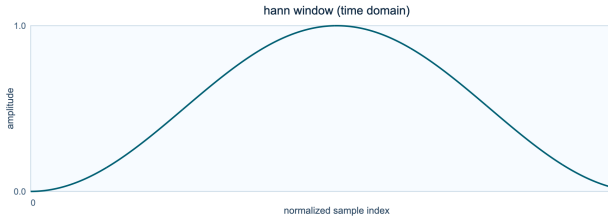


Figure 8.1: Time-domain shape of the hann window.

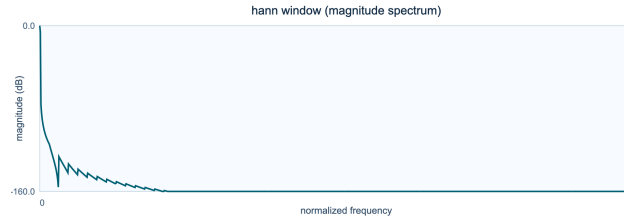


Figure 8.2: Magnitude spectrum of the hann window.

Interpretation and use

Strength. Balanced leakage suppression and frequency resolution.

Limitation. Not optimal for amplitude metering or extreme sidelobe rejection.

Recommendation. Default choice for most pvx time-stretch and pitch-shift workflows.

hamming

hamming is a cosine series design with `coeffs=(0.54, -0.46)`. Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.539775, and its equivalent noise bandwidth is 1.363305 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.3: Measured properties of the hamming window.

Metric	Value
Coherent gain	0.539775
Equivalent noise bandwidth	1.363305 bins
Scalloping loss	-1.750 dB
Main-lobe width	4.000 bins
Peak sidelobe	-42.675 dB

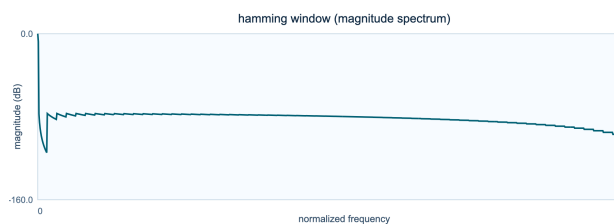
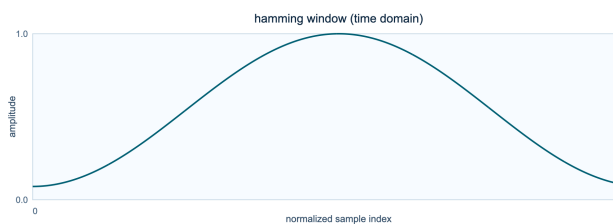


Figure 8.3: Time-domain shape of the hamming window. **Figure 8.4:** Magnitude spectrum of the hamming window.

Interpretation and use

Strength. Balanced leakage suppression and frequency resolution.

Limitation. Not optimal for amplitude metering or extreme sidelobe rejection.

Recommendation. Default choice for most pvx time-stretch and pitch-shift workflows.

blackman

blackman is a cosine series design with `coeffs`=(0.42, -0.5, 0.08). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.419795, and its equivalent noise bandwidth is 1.727601 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.4: Measured properties of the blackman window.

Metric	Value
Coherent gain	0.419795
Equivalent noise bandwidth	1.727601 bins
Scalloping loss	-1.098 dB
Main-lobe width	6.000 bins
Peak sidelobe	-58.109 dB

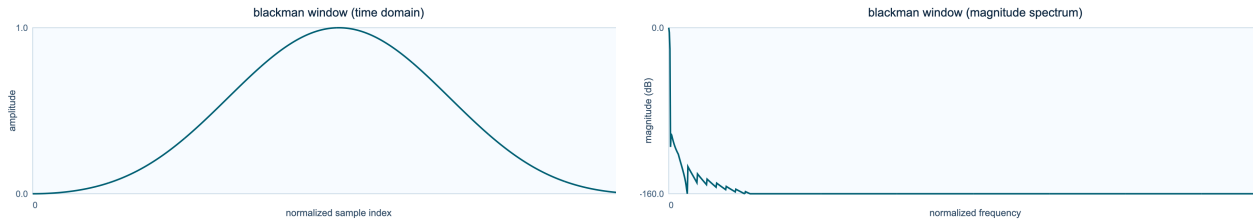


Figure 8.5: Time-domain shape of the blackman window. **Figure 8.6:** Magnitude spectrum of the blackman window.

Interpretation and use

Strength. Strong sidelobe suppression for cleaner spectral separation.

Limitation. Wider main lobe than Hann/Hamming.

Recommendation. Use for dense harmonic material when leakage artifacts dominate.

blackmanharris

`blackmanharris` is a cosine series design with `coeffs`=(0.35875, -0.48829, 0.14128, -0.01168). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.358575, and its equivalent noise bandwidth is 2.005332 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.5: Measured properties of the blackmanharris window.

Metric	Value
Coherent gain	0.358575
Equivalent noise bandwidth	2.005332 bins
Scalloping loss	-0.825 dB
Main-lobe width	8.000 bins
Peak sidelobe	-92.011 dB

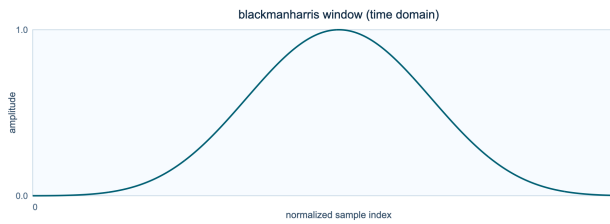


Figure 8.7: Time-domain shape of the blackmanharris window.

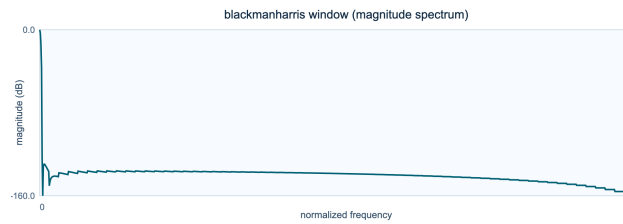


Figure 8.8: Magnitude spectrum of the blackmanharris window.

Interpretation and use

Strength. Strong sidelobe suppression for cleaner spectral separation.

Limitation. Wider main lobe than Hann/Hamming.

Recommendation. Use for dense harmonic material when leakage artifacts dominate.

nutall

`nutall` is a cosine series design with `coeffs`=(0.355768, -0.487396, 0.144232, -0.012604). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.355594, and its equivalent noise bandwidth is 2.022220 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.6: Measured properties of the nuttall window.

Metric	Value
Coherent gain	0.355594
Equivalent noise bandwidth	2.022220 bins
Scalloping loss	-0.811 dB
Main-lobe width	8.000 bins
Peak sidelobe	-93.325 dB

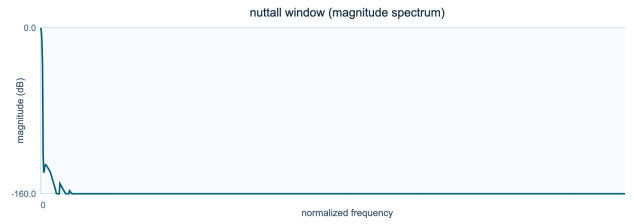
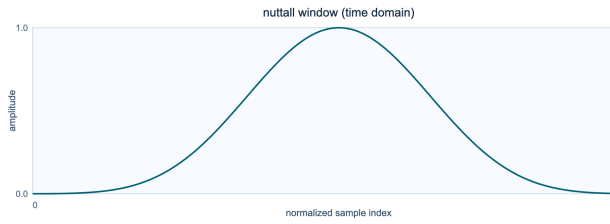


Figure 8.9: Time-domain shape of the nuttall window.

Figure 8.10: Magnitude spectrum of the nuttall window.

Interpretation and use

Strength. Strong sidelobe suppression for cleaner spectral separation.

Limitation. Wider main lobe than Hann/Hamming.

Recommendation. Use for dense harmonic material when leakage artifacts dominate.

flattop

flattop is a cosine series design with `coeffs`=(1, -1.93, 1.29, -0.388, 0.0322). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.999514, and its equivalent noise bandwidth is 3.772117 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.7: Measured properties of the flattop window.

Metric	Value
Coherent gain	0.999514
Equivalent noise bandwidth	3.772117 bins
Scalloping loss	-0.016 dB
Main-lobe width	10.000 bins
Peak sidelobe	-68.311 dB

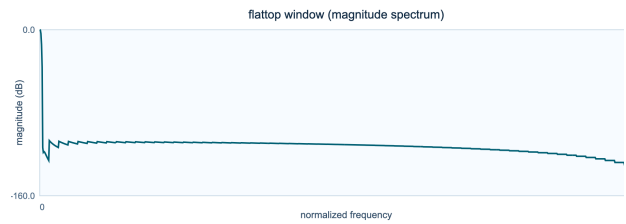
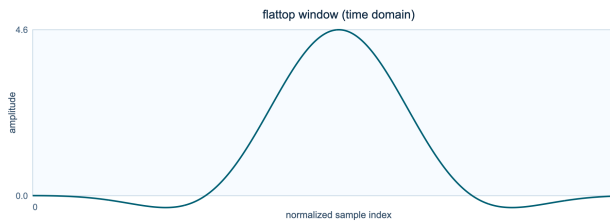


Figure 8.11: Time-domain shape of the flattop window. **Figure 8.12:** Magnitude spectrum of the flattop window.

Interpretation and use

Strength. Very accurate amplitude estimation in FFT bins.

Limitation. Very wide main lobe and reduced frequency discrimination.

Recommendation. Use for measurement-grade magnitude tracking, not fine pitch separation.

blackman nuttall

`blackman_nuttall` is a cosine series design with `coeffs`=(0.363582, -0.489177, 0.1366, -0.0106411). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.363405, and its equivalent noise bandwidth is 1.977073 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.8: Measured properties of the blackman nuttall window.

Metric	Value
Coherent gain	0.363405
Equivalent noise bandwidth	1.977073 bins
Scalloping loss	-0.850 dB
Main-lobe width	8.000 bins
Peak sidelobe	-98.174 dB

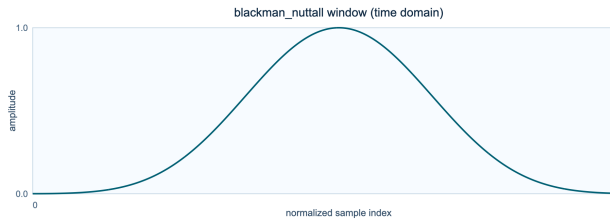


Figure 8.13: Time-domain shape of the blackman nuttall window.

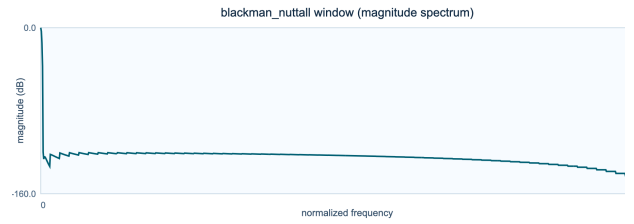


Figure 8.14: Magnitude spectrum of the blackman nuttall window.

Interpretation and use

Strength. Strong sidelobe suppression for cleaner spectral separation.

Limitation. Wider main lobe than Hann/Hamming.

Recommendation. Use for dense harmonic material when leakage artifacts dominate.

exact blackman

`exact_blackman` is a cosine series design with `coeffs`=(0.426591, -0.496561, 0.0768487). Its coefficients trade main-lobe width against sidelobe level in a controlled way. Its coherent gain is 0.426386, and its equivalent noise bandwidth is 1.694500 bins.

$$w[n] = \sum_{k=0}^K a_k \cos\left(\frac{2\pi kn}{N-1}\right)$$

where a_k is cosine coefficient k , K is the highest cosine order, n is the sample index, and N is the window length.

Table 8.9: Measured properties of the exact blackman window.

Metric	Value
Coherent gain	0.426386
Equivalent noise bandwidth	1.694500 bins
Scalloping loss	-1.149 dB
Main-lobe width	6.000 bins
Peak sidelobe	-68.236 dB

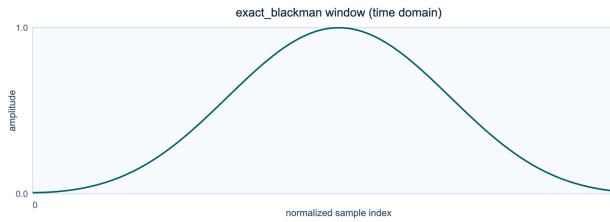


Figure 8.15: Time-domain shape of the exact blackman window.

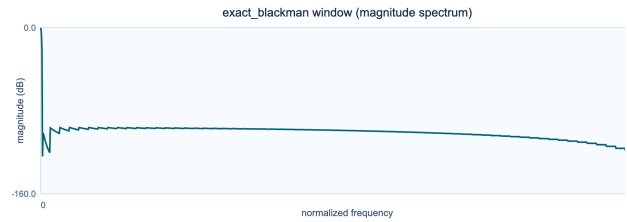


Figure 8.16: Magnitude spectrum of the exact blackman window.

Interpretation and use

Strength. Strong sidelobe suppression for cleaner spectral separation.

Limitation. Wider main lobe than Hann/Hamming.

Recommendation. Use for dense harmonic material when leakage artifacts dominate.

sine

sine is a sinusoidal design with **none**. The half-sine contour is smooth, symmetric, and easy to reason about under overlap. Its coherent gain is 0.636309, and its equivalent noise bandwidth is 1.234304 bins.

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

Table 8.10: Measured properties of the sine window.

Metric	Value
Coherent gain	0.636309
Equivalent noise bandwidth	1.234304 bins
Scalloping loss	-2.096 dB
Main-lobe width	3.000 bins
Peak sidelobe	-22.999 dB

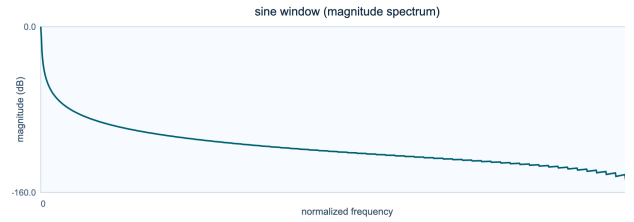
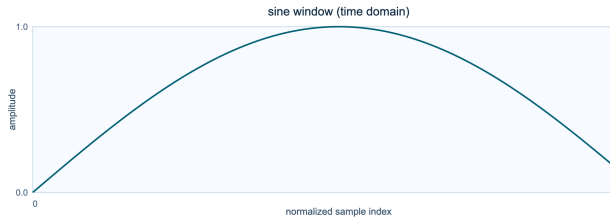


Figure 8.17: Time-domain shape of the sine window.

Figure 8.18: Magnitude spectrum of the sine window.

Interpretation and use

Strength. Smooth endpoint behavior and straightforward implementation.

Limitation. Less configurable than Kaiser/Tukey families.

Recommendation. Use for stable, low-complexity alternatives to Hann.

bartlett

bartlett is a triangular design with **none**. Its linear slopes make the time-domain weighting obvious and the spectral compromise easy to hear. Its coherent gain is 0.499756, and its equivalent noise bandwidth is 1.333985 bins.

$$w[n] = 1 - \left| \frac{n - m}{m} \right|$$

where $m = (N - 1)/2$ is the center sample, n is the sample index, and N is the window length.

Table 8.11: Measured properties of the bartlett window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.333985 bins
Scalloping loss	-1.822 dB
Main-lobe width	4.000 bins
Peak sidelobe	-26.523 dB

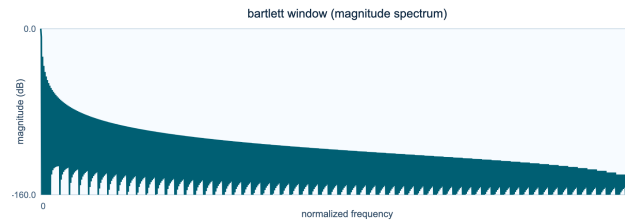
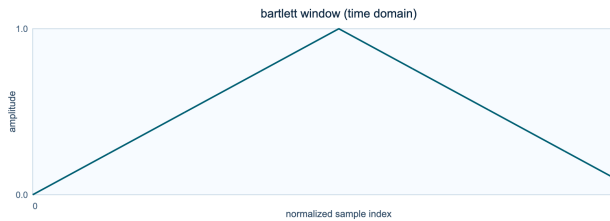


Figure 8.19: Time-domain shape of the bartlett window. **Figure 8.20:** Magnitude spectrum of the bartlett window.

Interpretation and use

Strength. Cheap linear taper with intuitive behavior.

Limitation. Higher sidelobes than stronger cosine-sum windows.

Recommendation. Use for low-cost processing or quick exploratory runs.

boxcar

boxcar is a rectangular design with **none**. With no taper at all, it provides the sharpest baseline for seeing leakage caused by abrupt edges. Its coherent gain is 1.000000, and its equivalent noise bandwidth is 1.000000 bins.

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N - 1$.

Table 8.12: Measured properties of the boxcar window.

Metric	Value
Coherent gain	1.000000
Equivalent noise bandwidth	1.000000 bins
Scalloping loss	-3.922 dB
Main-lobe width	2.000 bins
Peak sidelobe	-13.264 dB

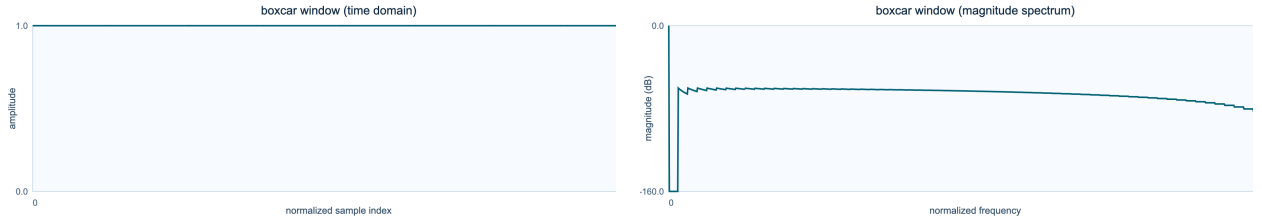


Figure 8.21: Time-domain shape of the boxcar window. **Figure 8.22:** Magnitude spectrum of the boxcar window.

Interpretation and use

Strength. Narrowest main lobe and maximal bin sharpness.

Limitation. Highest sidelobes and strongest leakage/phasing on non-bin-centered content.

Recommendation. Use only for controlled test tones or when leakage is acceptable.

triangular

triangular is a triangular design with **none**. Its linear slopes make the time-domain weighting obvious and the spectral compromise easy to hear. Its coherent gain is 0.500244, and its equivalent noise bandwidth is 1.332683 bins.

$$w[n] = \max\left(1 - \left|\frac{n - m}{(N + 1)/2}\right|, 0\right)$$

where $m = (N - 1)/2$ is the center sample, n is the sample index, and N is the window length.

Table 8.13: Measured properties of the triangular window.

Metric	Value
Coherent gain	0.500244
Equivalent noise bandwidth	1.332683 bins
Scalloping loss	-1.826 dB
Main-lobe width	4.000 bins
Peak sidelobe	-26.523 dB

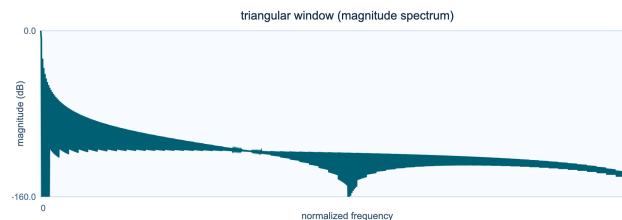
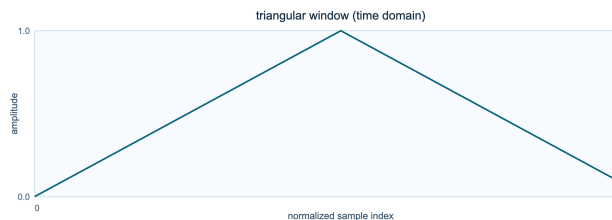


Figure 8.23: Time-domain shape of the triangular window. **Figure 8.24:** Magnitude spectrum of the triangular window.

Interpretation and use

Strength. Cheap linear taper with intuitive behavior.

Limitation. Higher sidelobes than stronger cosine-sum windows.

Recommendation. Use for low-cost processing or quick exploratory runs.

bartlett hann

`bartlett_hann` is a hybrid taper design with **fixed coefficients**. It combines polynomial and cosine behavior rather than committing to either alone. Its coherent gain is 0.499756, and its equivalent noise bandwidth is 1.456559 bins.

$$x = \frac{n}{N-1} - \frac{1}{2}, \quad w[n] = 0.62 - 0.48|x| + 0.38 \cos(2\pi x)$$

where x is the centered normalized position, n is the sample index, and N is the window length.

Table 8.14: Measured properties of the bartlett hann window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.456559 bins
Scalloping loss	-1.517 dB
Main-lobe width	4.000 bins
Peak sidelobe	-35.874 dB

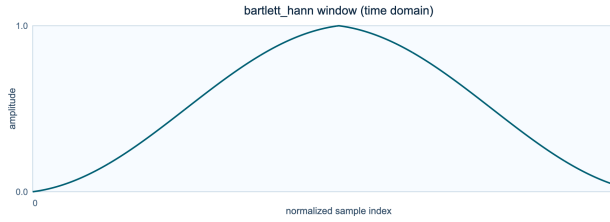


Figure 8.25: Time-domain shape of the bartlett hann window.

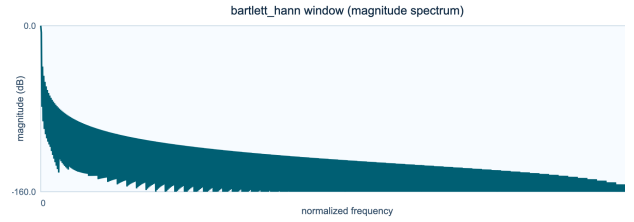


Figure 8.26: Magnitude spectrum of the bartlett hann window.

Interpretation and use

Strength. Moderate leakage suppression with lightweight computation.

Limitation. Less common and less interpretable than standard Hann/Hamming.

Recommendation. Use as a middle-ground taper when Bartlett feels too sharp.

tukey

tukey is a tukey design with **alpha=0.5**. A flat center and adjustable cosine shoulders let it move between rectangular and Hann-like behavior. Its coherent gain is 0.749634, and its equivalent noise bandwidth is 1.222819 bins.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.15: Measured properties of the tukey window.

Metric	Value
Coherent gain	0.749634
Equivalent noise bandwidth	1.222819 bins
Scalloping loss	-2.236 dB
Main-lobe width	2.656 bins
Peak sidelobe	-15.123 dB

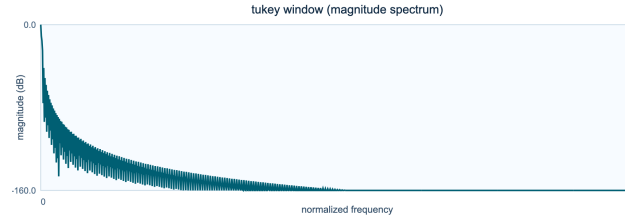
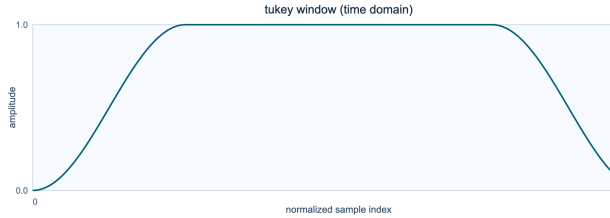


Figure 8.27: Time-domain shape of the tukey window.

Figure 8.28: Magnitude spectrum of the tukey window.

Interpretation and use

Strength. Interpolates between rectangular and Hann behavior.

Limitation. Behavior changes strongly with alpha and can be inconsistent across presets.

Recommendation. Use when you need a controllable flat center with soft edges.

tukey 0p1

tukey_0p1 is a tukey design with **alpha=0.1**. A flat center and adjustable cosine shoulders let it move between rectangular and Hann-like behavior. Its coherent gain is 0.949536, and its equivalent noise bandwidth is 1.039289 bins.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.16: Measured properties of the tukey 0p1 window.

Metric	Value
Coherent gain	0.949536
Equivalent noise bandwidth	1.039289 bins
Scalloping loss	-3.505 dB
Main-lobe width	2.094 bins
Peak sidelobe	-13.309 dB

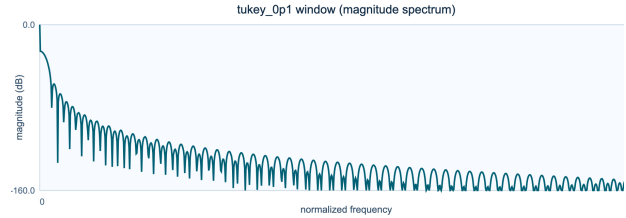
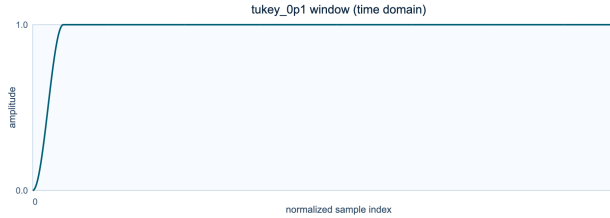


Figure 8.29: Time-domain shape of the tukey 0p1 window.

Figure 8.30: Magnitude spectrum of the tukey 0p1 window.

Interpretation and use

Strength. Interpolates between rectangular and Hann behavior.

Limitation. Behavior changes strongly with alpha and can be inconsistent across presets.

Recommendation. Use when you need a controllable flat center with soft edges.

tukey 0p25

`tukey_0p25` is a tukey design with `alpha=0.25`. A flat center and adjustable cosine shoulders let it move between rectangular and Hann-like behavior. Its coherent gain is 0.874573, and its equivalent noise bandwidth is 1.102579 bins.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.17: Measured properties of the tukey 0p25 window.

Metric	Value
Coherent gain	0.874573
Equivalent noise bandwidth	1.102579 bins
Scalloping loss	-2.960 dB
Main-lobe width	2.281 bins
Peak sidelobe	-13.601 dB

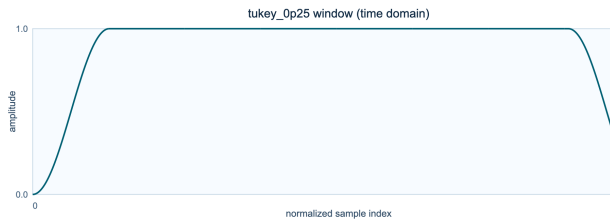


Figure 8.31: Time-domain shape of the tukey 0p25 window.

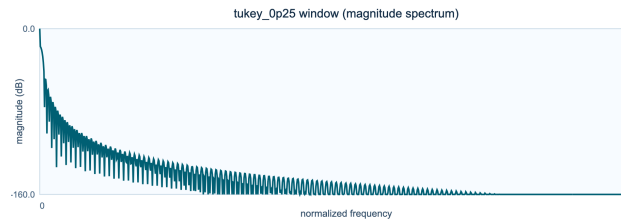


Figure 8.32: Magnitude spectrum of the tukey 0p25 window.

Interpretation and use

Strength. Interpolates between rectangular and Hann behavior.

Limitation. Behavior changes strongly with α and can be inconsistent across presets.

Recommendation. Use when you need a controllable flat center with soft edges.

tukey 0p75

tukey_0p75 is a tukey design with **alpha=0.75**. A flat center and adjustable cosine shoulders let it move between rectangular and Hann-like behavior. Its coherent gain is 0.624695, and its equivalent noise bandwidth is 1.360664 bins.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.18: Measured properties of the tukey 0p75 window.

Metric	Value
Coherent gain	0.624695
Equivalent noise bandwidth	1.360664 bins
Scalloping loss	-1.728 dB
Main-lobe width	3.188 bins
Peak sidelobe	-19.395 dB

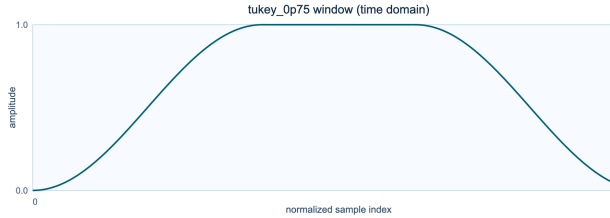


Figure 8.33: Time-domain shape of the tukey 0p75 window.

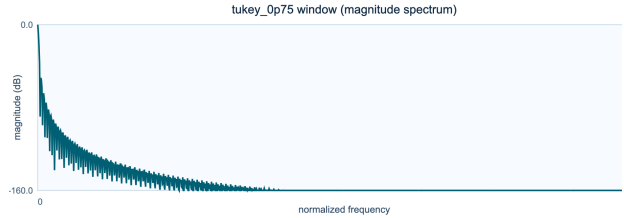


Figure 8.34: Magnitude spectrum of the tukey 0p75 window.

Interpretation and use

Strength. Interpolates between rectangular and Hann behavior.

Limitation. Behavior changes strongly with alpha and can be inconsistent across presets.

Recommendation. Use when you need a controllable flat center with soft edges.

tukey 0p9

tukey_0p9 is a tukey design with **alpha=0.9**. A flat center and adjustable cosine shoulders let it move between rectangular and Hann-like behavior. Its coherent gain is 0.549731, and its equivalent noise bandwidth is 1.446988 bins.

$$w[n] = \begin{cases} \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - 1 \right) \right) \right], & 0 \leq x < \alpha/2 \\ 1, & \alpha/2 \leq x < 1 - \alpha/2 \\ \frac{1}{2} \left[1 + \cos \left(\pi \left(\frac{2x}{\alpha} - \frac{2}{\alpha} + 1 \right) \right) \right], & 1 - \alpha/2 \leq x \leq 1 \end{cases}$$

where $x = n/(N - 1)$ is normalized position, α is the tapered fraction, n is the sample index, and N is the window length.

Table 8.19: Measured properties of the tukey 0p9 window.

Metric	Value
Coherent gain	0.549731
Equivalent noise bandwidth	1.446988 bins
Scalloping loss	-1.521 dB
Main-lobe width	3.625 bins
Peak sidelobe	-24.972 dB

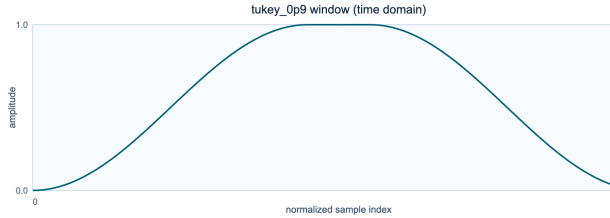


Figure 8.35: Time-domain shape of the tukey 0p9 window.

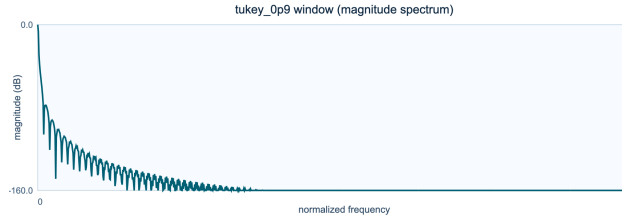


Figure 8.36: Magnitude spectrum of the tukey 0p9 window.

Interpretation and use

Strength. Interpolates between rectangular and Hann behavior.

Limitation. Behavior changes strongly with alpha and can be inconsistent across presets.

Recommendation. Use when you need a controllable flat center with soft edges.

parzen

parzen is a polynomial design with **piecewise cubic**. Its piecewise polynomial contour reaches zero smoothly and suppresses distant frame samples. Its coherent gain is 0.374817, and its equivalent noise bandwidth is 1.918397 bins.

$$w[n] = \begin{cases} 1 - 6u^2 + 6u^3, & 0 \leq u \leq 1/2 \\ 2(1 - u)^3, & 1/2 < u \leq 1 \\ 0, & u > 1 \end{cases}$$

where $u = |2n/(N - 1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

Table 8.20: Measured properties of the parzen window.

Metric	Value
Coherent gain	0.374817
Equivalent noise bandwidth	1.918397 bins
Scalloping loss	-0.897 dB
Main-lobe width	8.000 bins
Peak sidelobe	-53.046 dB

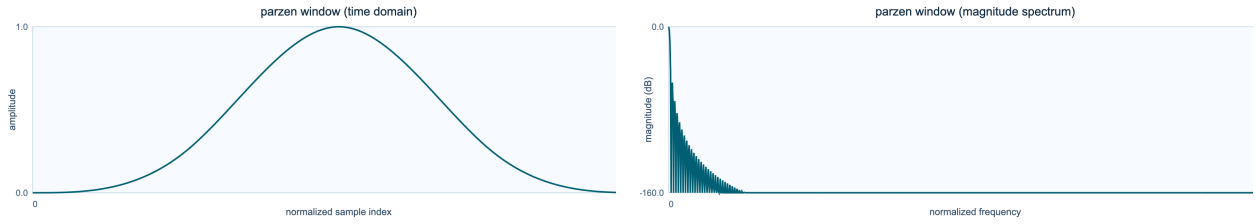


Figure 8.37: Time-domain shape of the parzen window. **Figure 8.38:** Magnitude spectrum of the parzen window.

Interpretation and use

Strength. Smooth high-order taper with good sidelobe control.

Limitation. Broader main lobe than lightweight windows.

Recommendation. Use for spectral denoising/analysis where sidelobe cleanup is critical.

lanczos

lanczos is a sinc design with **none**. Its sampled sinc contour connects window design to interpolation and band-limited reconstruction ideas. Its coherent gain is 0.589202, and its equivalent noise bandwidth is 1.299668 bins.

$$w[n] = \text{sinc}\left(\frac{2n}{N-1} - 1\right)$$

where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$, n is the sample index, and N is the window length.

Table 8.21: Measured properties of the lanczos window.

Metric	Value
Coherent gain	0.589202
Equivalent noise bandwidth	1.299668 bins
Scalloping loss	-1.889 dB
Main-lobe width	3.281 bins
Peak sidelobe	-26.405 dB

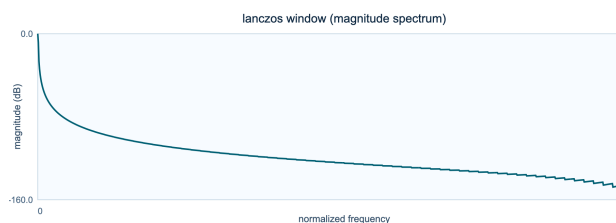
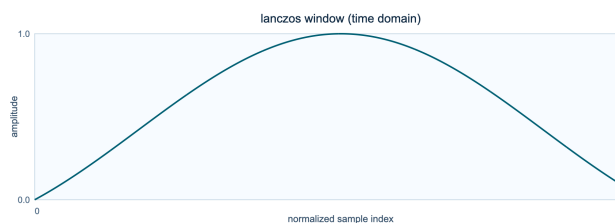


Figure 8.39: Time-domain shape of the lanczos window. **Figure 8.40:** Magnitude spectrum of the lanczos window.

Interpretation and use

Strength. Sinc-derived shape with useful compromise behavior.

Limitation. Can exhibit oscillatory spectral behavior versus cosine-sum defaults.

Recommendation. Use for interpolation-adjacent analysis experiments.

welch

welch is a quadratic design with **none**. Its simple parabolic profile emphasizes the middle of the frame with modest computational cost. Its coherent gain is 0.666341, and its equivalent noise bandwidth is 1.200587 bins.

$$w[n] = \max(1 - x_n^2, 0)$$

where $x_n = (n - m)/m$ is centered normalized position, $m = (N - 1)/2$, and N is the window length.

Table 8.22: Measured properties of the welch window.

Metric	Value
Coherent gain	0.666341
Equivalent noise bandwidth	1.200587 bins
Scalloping loss	-2.223 dB
Main-lobe width	2.875 bins
Peak sidelobe	-21.295 dB

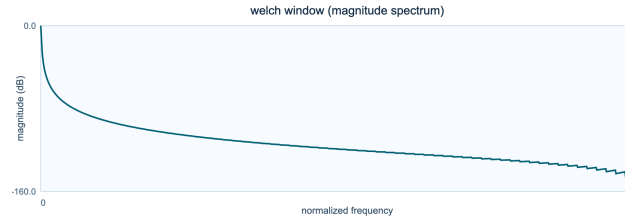
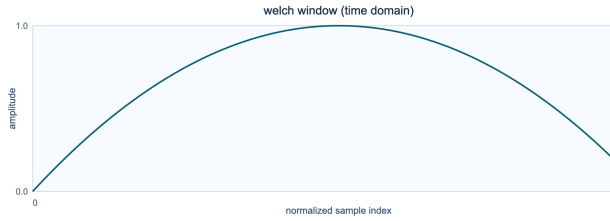


Figure 8.41: Time-domain shape of the welch window.

Figure 8.42: Magnitude spectrum of the welch window.

Interpretation and use

Strength. Center-emphasizing parabola with simple form.

Limitation. Not as strong at sidelobe suppression as Blackman-family windows.

Recommendation. Use when center weighting is desired with minimal complexity.

gaussian 0p25

`gaussian_0p25` is a gaussian design with `sigma_ratio=0.25`. The spread parameter directly controls how tightly the analysis is concentrated around frame center. Its coherent gain is 0.313156, and its equivalent noise bandwidth is 2.258144 bins.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.23: Measured properties of the gaussian 0p25 window.

Metric	Value
Coherent gain	0.313156
Equivalent noise bandwidth	2.258144 bins
Scalloping loss	-0.668 dB
Main-lobe width	11.406 bins
Peak sidelobe	-87.677 dB

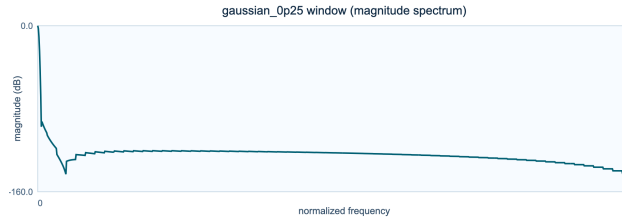
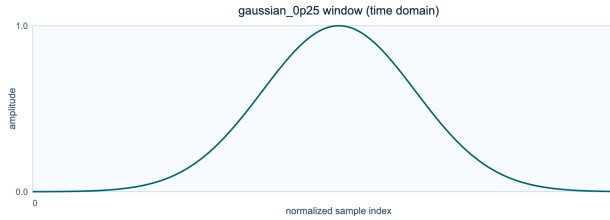


Figure 8.43: Time-domain shape of the gaussian 0p25 window. **Figure 8.44:** Magnitude spectrum of the gaussian 0p25 window.

Interpretation and use

Strength. Smooth bell taper with low ringing and predictable decay.

Limitation. Frequency resolution changes noticeably with sigma.

Recommendation. Use sigma presets to tune transient locality versus spectral leakage.

gaussian 0p35

`gaussian_0p35` is a gaussian design with `sigma_ratio=0.35`. The spread parameter directly controls how tightly the analysis is concentrated around frame center. Its coherent gain is 0.436580, and its equivalent noise bandwidth is 1.626488 bins.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.24: Measured properties of the gaussian 0p35 window.

Metric	Value
Coherent gain	0.436580
Equivalent noise bandwidth	1.626488 bins
Scalloping loss	-1.268 dB
Main-lobe width	6.719 bins
Peak sidelobe	-52.077 dB

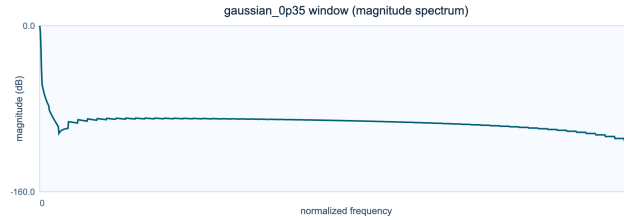
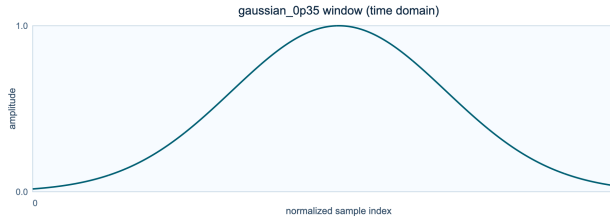


Figure 8.45: Time-domain shape of the gaussian 0p35 window. **Figure 8.46:** Magnitude spectrum of the gaussian 0p35 window.

Interpretation and use

Strength. Smooth bell taper with low ringing and predictable decay.

Limitation. Frequency resolution changes noticeably with sigma.

Recommendation. Use sigma presets to tune transient locality versus spectral leakage.

gaussian 0p45

`gaussian_0p45` is a gaussian design with `sigma_ratio=0.45`. The spread parameter directly controls how tightly the analysis is concentrated around frame center. Its coherent gain is 0.548949, and its equivalent noise bandwidth is 1.320556 bins.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.25: Measured properties of the gaussian 0p45 window.

Metric	Value
Coherent gain	0.548949
Equivalent noise bandwidth	1.320556 bins
Scalloping loss	-1.869 dB
Main-lobe width	4.125 bins
Peak sidelobe	-35.366 dB

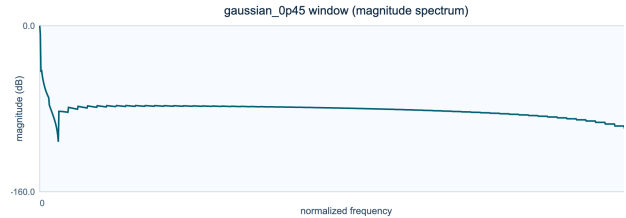
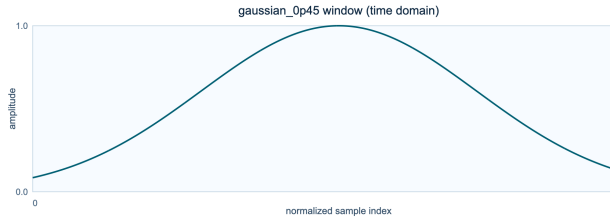


Figure 8.47: Time-domain shape of the gaussian 0p45 window.

Figure 8.48: Magnitude spectrum of the gaussian 0p45 window.

Interpretation and use

Strength. Smooth bell taper with low ringing and predictable decay.

Limitation. Frequency resolution changes noticeably with sigma.

Recommendation. Use sigma presets to tune transient locality versus spectral leakage.

gaussian 0p55

`gaussian_0p55` is a gaussian design with `sigma_ratio=0.55`. The spread parameter directly controls how tightly the analysis is concentrated around frame center. Its coherent gain is 0.641515, and its equivalent noise bandwidth is 1.171861 bins.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.26: Measured properties of the gaussian 0p55 window.

Metric	Value
Coherent gain	0.641515
Equivalent noise bandwidth	1.171861 bins
Scalloping loss	-2.350 dB
Main-lobe width	3.000 bins
Peak sidelobe	-28.906 dB

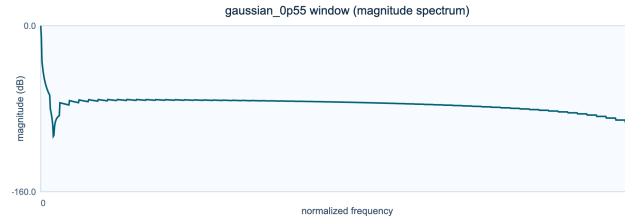
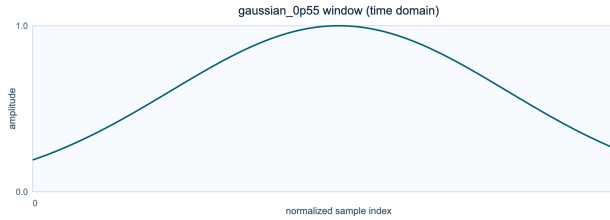


Figure 8.49: Time-domain shape of the gaussian 0p55 window. **Figure 8.50:** Magnitude spectrum of the gaussian 0p55 window.

Interpretation and use

Strength. Smooth bell taper with low ringing and predictable decay.

Limitation. Frequency resolution changes noticeably with sigma.

Recommendation. Use sigma presets to tune transient locality versus spectral leakage.

gaussian 0p65

`gaussian_0p65` is a gaussian design with `sigma_ratio=0.65`. The spread parameter directly controls how tightly the analysis is concentrated around frame center. Its coherent gain is 0.713490, and its equivalent noise bandwidth is 1.097657 bins.

$$w[n] = \exp\left[-\frac{1}{2}\left(\frac{n-m}{\sigma}\right)^2\right], \quad \sigma = r_\sigma m$$

where σ is spread, r_σ is the configured spread ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.27: Measured properties of the gaussian 0p65 window.

Metric	Value
Coherent gain	0.713490
Equivalent noise bandwidth	1.097657 bins
Scalloping loss	-2.705 dB
Main-lobe width	2.625 bins
Peak sidelobe	-22.575 dB

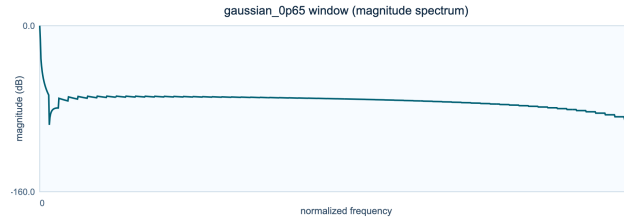
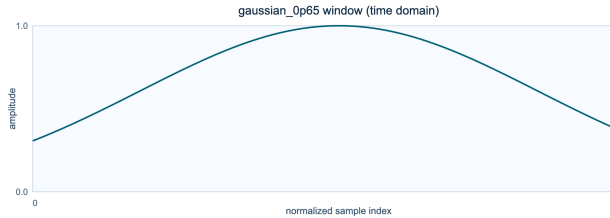


Figure 8.51: Time-domain shape of the gaussian 0p65 window. **Figure 8.52:** Magnitude spectrum of the gaussian 0p65 window.

Interpretation and use

Strength. Smooth bell taper with low ringing and predictable decay.

Limitation. Frequency resolution changes noticeably with sigma.

Recommendation. Use sigma presets to tune transient locality versus spectral leakage.

general gaussian 1p5 0p35

`general_gaussian_1p5_0p35` is a generalized gaussian design with `power=1.5`, `sigma_ratio=0.35`. Separate spread and shape controls make the shoulders independently adjustable. Its coherent gain is 0.393587, and its equivalent noise bandwidth is 2.016584 bins.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.28: Measured properties of the general gaussian 1p5 0p35 window.

Metric	Value
Coherent gain	0.393587
Equivalent noise bandwidth	2.016584 bins
Scalloping loss	-0.784 dB
Main-lobe width	5.156 bins
Peak sidelobe	-24.895 dB

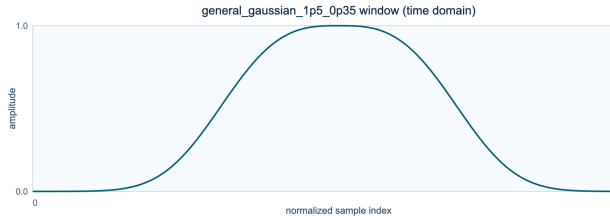


Figure 8.53: Time-domain shape of the general gaussian 1p5 0p35 window.

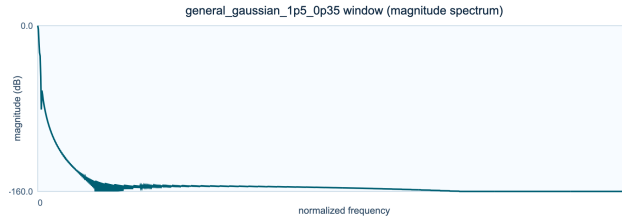


Figure 8.54: Magnitude spectrum of the general gaussian 1p5 0p35 window.

Interpretation and use

Strength. Additional shape control beyond standard Gaussian.

Limitation. Extra parameterization increases tuning complexity.

Recommendation. Use for research/tuning tasks where shoulder steepness matters.

general gaussian 2p0 0p35

`general_gaussian_2p0_0p35` is a generalized gaussian design with `power=2`, `sigma_ratio=0.35`. Separate spread and shape controls make the shoulders independently adjustable. Its coherent gain is 0.377081, and its equivalent noise bandwidth is 2.230016 bins.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.29: Measured properties of the general gaussian 2p0 0p35 window.

Metric	Value
Coherent gain	0.377081
Equivalent noise bandwidth	2.230016 bins
Scalloping loss	-0.633 dB
Main-lobe width	5.281 bins
Peak sidelobe	-19.717 dB

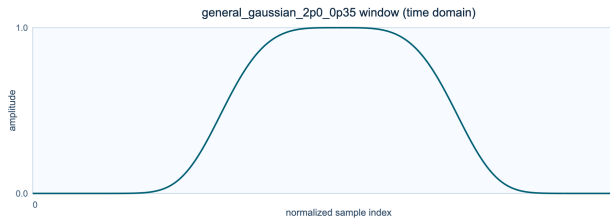


Figure 8.55: Time-domain shape of the general gaussian 2p0 0p35 window.

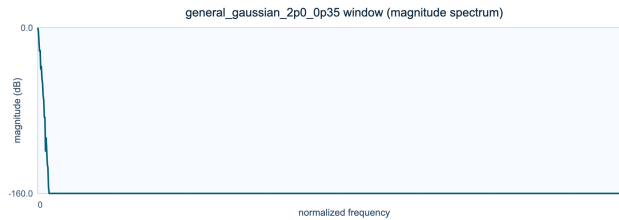


Figure 8.56: Magnitude spectrum of the general gaussian 2p0 0p35 window.

Interpretation and use

Strength. Additional shape control beyond standard Gaussian.

Limitation. Extra parameterization increases tuning complexity.

Recommendation. Use for research/tuning tasks where shoulder steepness matters.

general gaussian 3p0 0p35

`general_gaussian_3p0_0p35` is a generalized gaussian design with `power=3`, `sigma_ratio=0.35`. Separate spread and shape controls make the shoulders independently adjustable. Its coherent gain is 0.364287, and its equivalent noise bandwidth is 2.445593 bins.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.30: Measured properties of the general gaussian 3p0 0p35 window.

Metric	Value
Coherent gain	0.364287
Equivalent noise bandwidth	2.445593 bins
Scalloping loss	-0.532 dB
Main-lobe width	5.469 bins
Peak sidelobe	-16.299 dB

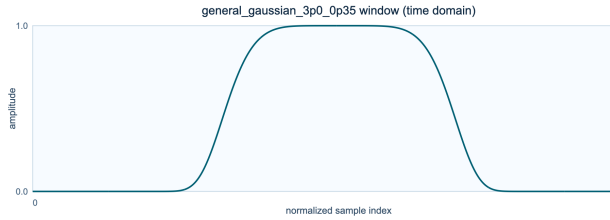


Figure 8.57: Time-domain shape of the general gaussian 3p0 0p35 window.

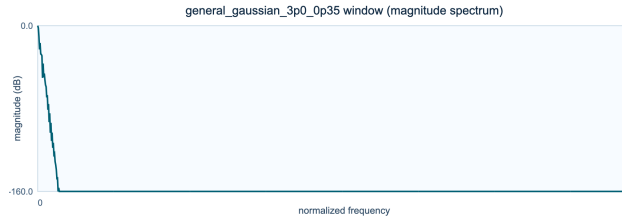


Figure 8.58: Magnitude spectrum of the general gaussian 3p0 0p35 window.

Interpretation and use

Strength. Additional shape control beyond standard Gaussian.

Limitation. Extra parameterization increases tuning complexity.

Recommendation. Use for research/tuning tasks where shoulder steepness matters.

general gaussian 4p0 0p35

`general_gaussian_4p0_0p35` is a generalized gaussian design with `power=4`, `sigma_ratio=0.35`. Separate spread and shape controls make the shoulders independently adjustable. Its coherent gain is 0.359267, and its equivalent noise bandwidth is 2.552433 bins.

$$w[n] = \exp \left[-\frac{1}{2} \left| \frac{n-m}{\sigma} \right|^{2p} \right]$$

where p controls shoulder shape, σ controls spread, $m = (N-1)/2$, and n is the sample index.

Table 8.31: Measured properties of the general gaussian 4p0 0p35 window.

Metric	Value
Coherent gain	0.359267
Equivalent noise bandwidth	2.552433 bins
Scalloping loss	-0.496 dB
Main-lobe width	5.562 bins
Peak sidelobe	-15.060 dB

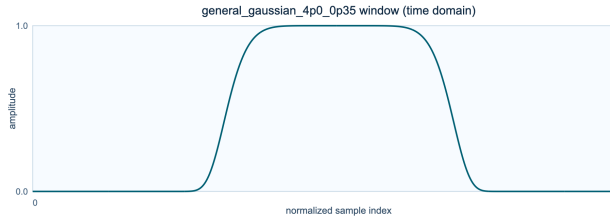


Figure 8.59: Time-domain shape of the general gaussian 4p0 0p35 window.

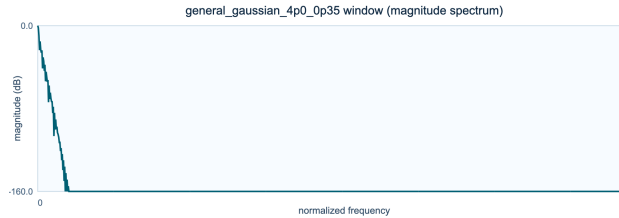


Figure 8.60: Magnitude spectrum of the general gaussian 4p0 0p35 window.

Interpretation and use

Strength. Additional shape control beyond standard Gaussian.

Limitation. Extra parameterization increases tuning complexity.

Recommendation. Use for research/tuning tasks where shoulder steepness matters.

exponential 0p25

`exponential_0p25` is an exponential design with `tau_ratio=0.25`. Its cusp-like center and rapid decay make it unlike the smoother cosine tapers. Its coherent gain is 0.245310, and its equivalent noise bandwidth is 2.075492 bins.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.32: Measured properties of the exponential 0p25 window.

Metric	Value
Coherent gain	0.245310
Equivalent noise bandwidth	2.075492 bins
Scalloping loss	-1.022 dB
Main-lobe width	15.562 bins
Peak sidelobe	-31.885 dB

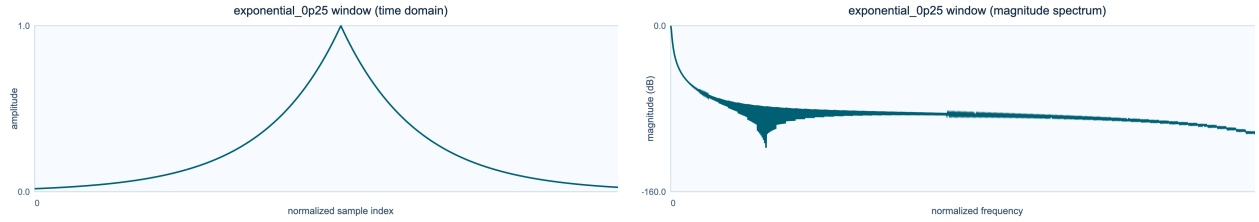


Figure 8.61: Time-domain shape of the exponential 0p25 window. **Figure 8.62:** Magnitude spectrum of the exponential 0p25 window.

Interpretation and use

Strength. Simple edge decay with fast computation.

Limitation. Can produce less-uniform center weighting than cosine families.

Recommendation. Use for experiments emphasizing center-heavy weighting.

exponential 0p5

`exponential_0p5` is an exponential design with `tau_ratio=0.5`. Its cusp-like center and rapid decay make it unlike the smoother cosine tapers. Its coherent gain is 0.432187, and its equivalent noise bandwidth is 1.313323 bins.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.33: Measured properties of the exponential 0p5 window.

Metric	Value
Coherent gain	0.432187
Equivalent noise bandwidth	1.313323 bins
Scalloping loss	-2.032 dB
Main-lobe width	3.625 bins
Peak sidelobe	-19.190 dB

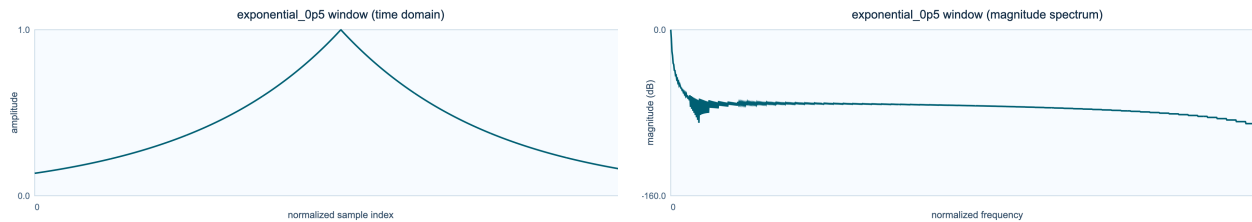


Figure 8.63: Time-domain shape of the exponential 0p5 window. **Figure 8.64:** Magnitude spectrum of the exponential 0p5 window.

Interpretation and use

Strength. Simple edge decay with fast computation.

Limitation. Can produce less-uniform center weighting than cosine families.

Recommendation. Use for experiments emphasizing center-heavy weighting.

exponential 1p0

`exponential_1p0` is a exponential design with `tau_ratio=1`. Its cusp-like center and rapid decay make it unlike the smoother cosine tapers. Its coherent gain is 0.631991, and its equivalent noise bandwidth is 1.082055 bins.

$$w[n] = \exp\left(-\frac{|n-m|}{\tau}\right), \quad \tau = r_\tau m$$

where τ is the decay constant, r_τ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.34: Measured properties of the exponential 1p0 window.

Metric	Value
Coherent gain	0.631991
Equivalent noise bandwidth	1.082055 bins
Scalloping loss	-2.854 dB
Main-lobe width	2.594 bins
Peak sidelobe	-20.141 dB

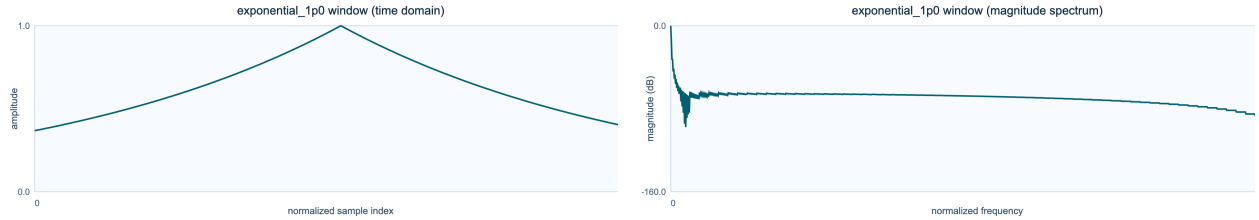


Figure 8.65: Time-domain shape of the exponential 1p0 window. **Figure 8.66:** Magnitude spectrum of the exponential 1p0 window.

Interpretation and use

Strength. Simple edge decay with fast computation.

Limitation. Can produce less-uniform center weighting than cosine families.

Recommendation. Use for experiments emphasizing center-heavy weighting.

cauchy_0p5

`cauchy_0p5` is a cauchy / lorentzian design with `gamma_ratio=0.5`. Its long tails retain more distant samples than a Gaussian of similar center width. Its coherent gain is 0.553402, and its equivalent noise bandwidth is 1.229775 bins.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.35: Measured properties of the cauchy 0p5 window.

Metric	Value
Coherent gain	0.553402
Equivalent noise bandwidth	1.229775 bins
Scalloping loss	-2.209 dB
Main-lobe width	3.562 bins
Peak sidelobe	-22.733 dB

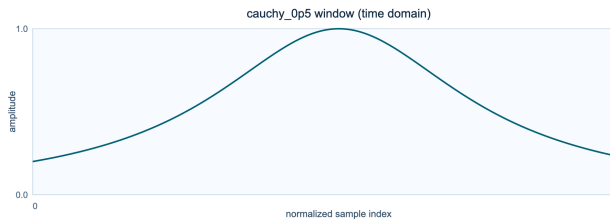


Figure 8.67: Time-domain shape of the cauchy 0p5 window.

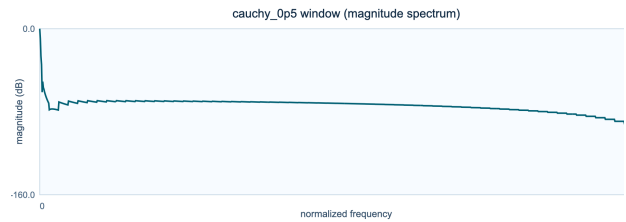


Figure 8.68: Magnitude spectrum of the cauchy 0p5 window.

Interpretation and use

Strength. Heavy tails preserve more peripheral samples than Gaussian.

Limitation. Tail energy can increase leakage relative to steeper tapers.

Recommendation. Use when you want softer attenuation of far-window samples.

cauchy_1p0

cauchy_1p0 is a cauchy / lorentzian design with `gamma_ratio=1`. Its long tails retain more distant samples than a Gaussian of similar center width. Its coherent gain is 0.785259, and its equivalent noise bandwidth is 1.041963 bins.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.36: Measured properties of the cauchy 1p0 window.

Metric	Value
Coherent gain	0.785259
Equivalent noise bandwidth	1.041963 bins
Scalloping loss	-3.108 dB
Main-lobe width	2.375 bins
Peak sidelobe	-19.034 dB

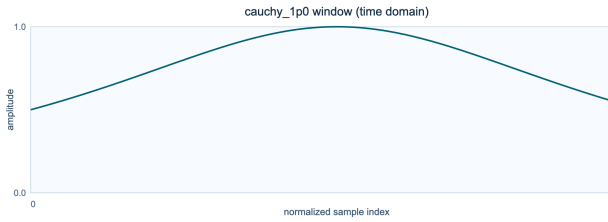


Figure 8.69: Time-domain shape of the cauchy 1p0 window.

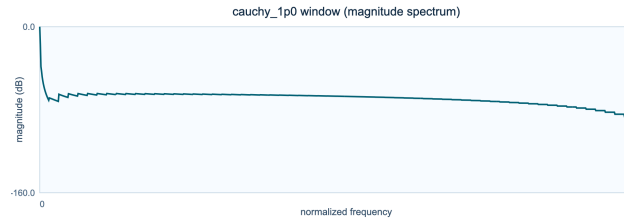


Figure 8.70: Magnitude spectrum of the cauchy 1p0 window.

Interpretation and use

Strength. Heavy tails preserve more peripheral samples than Gaussian.

Limitation. Tail energy can increase leakage relative to steeper tapers.

Recommendation. Use when you want softer attenuation of far-window samples.

cauchy 2p0

`cauchy_2p0` is a cauchy / lorentzian design with `gamma_ratio=2`. Its long tails retain more distant samples than a Gaussian of similar center width. Its coherent gain is 0.927233, and its equivalent noise bandwidth is 1.004393 bins.

$$w[n] = \frac{1}{1 + \left(\frac{n-m}{\gamma}\right)^2}, \quad \gamma = r_\gamma m$$

where γ is the Lorentzian scale, r_γ is its configured ratio, $m = (N-1)/2$, and n is the sample index.

Table 8.37: Measured properties of the cauchy 2p0 window.

Metric	Value
Coherent gain	0.927233
Equivalent noise bandwidth	1.004393 bins
Scalloping loss	-3.648 dB
Main-lobe width	2.094 bins
Peak sidelobe	-14.689 dB

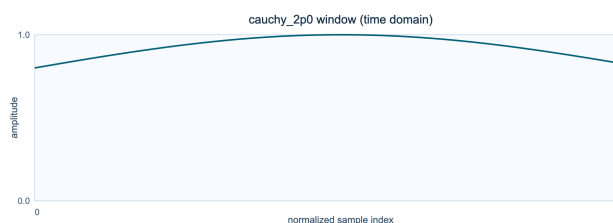


Figure 8.71: Time-domain shape of the cauchy 2p0 window.

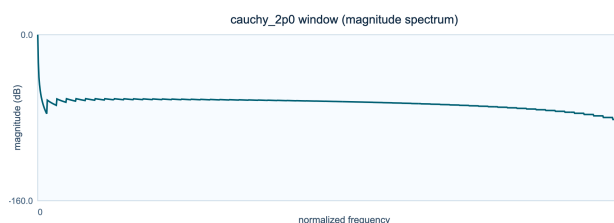


Figure 8.72: Magnitude spectrum of the cauchy 2p0 window.

Interpretation and use

Strength. Heavy tails preserve more peripheral samples than Gaussian.

Limitation. Tail energy can increase leakage relative to steeper tapers.

Recommendation. Use when you want softer attenuation of far-window samples.

cosine power 2

`cosine_power_2` is a cosine power design with `power=2`. Raising the sine profile changes edge attenuation without introducing a new functional family. Its coherent gain is 0.499756, and its equivalent noise bandwidth is 1.500733 bins.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.38: Measured properties of the cosine power 2 window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB

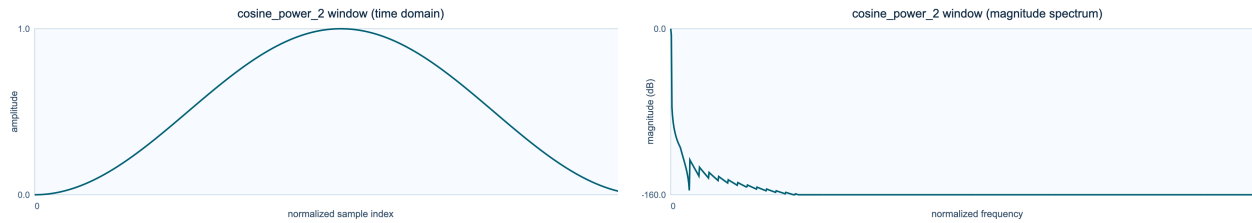


Figure 8.73: Time-domain shape of the cosine power 2 window.

Figure 8.74: Magnitude spectrum of the cosine power 2 window.

Interpretation and use

Strength. Simple power parameter controls edge steepness.

Limitation. Higher powers can overly narrow effective support.

Recommendation. Use to smoothly increase edge attenuation versus basic sine.

cosine power 3

`cosine_power_3` is a cosine power design with `power=3`. Raising the sine profile changes edge attenuation without introducing a new functional family. Its coherent gain is 0.424206, and its equivalent noise bandwidth is 1.735739 bins.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.39: Measured properties of the cosine power 3 window.

Metric	Value
Coherent gain	0.424206
Equivalent noise bandwidth	1.735739 bins
Scalloping loss	-1.074 dB
Main-lobe width	5.000 bins
Peak sidelobe	-39.295 dB

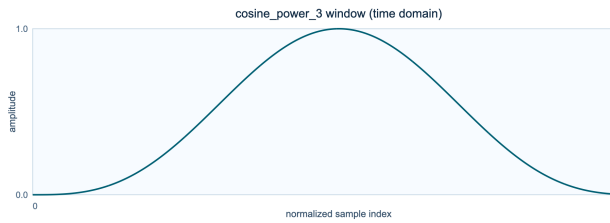


Figure 8.75: Time-domain shape of the cosine power 3 window.

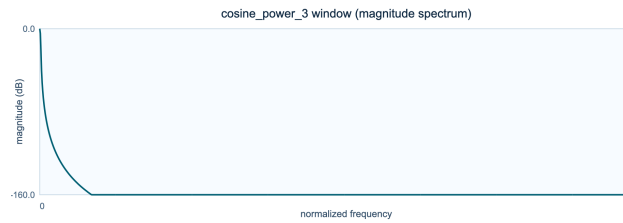


Figure 8.76: Magnitude spectrum of the cosine power 3 window.

Interpretation and use

Strength. Simple power parameter controls edge steepness.

Limitation. Higher powers can overly narrow effective support.

Recommendation. Use to smoothly increase edge attenuation versus basic sine.

cosine power 4

`cosine_power_4` is a cosine power design with `power=4`. Raising the sine profile changes edge attenuation without introducing a new functional family. Its coherent gain is 0.374817, and its equivalent noise bandwidth is 1.945394 bins.

$$w[n] = \sin^p\left(\frac{\pi n}{N-1}\right)$$

where p is the cosine-power exponent, n is the sample index, and N is the window length.

Table 8.40: Measured properties of the cosine power 4 window.

Metric	Value
Coherent gain	0.374817
Equivalent noise bandwidth	1.945394 bins
Scalloping loss	-0.862 dB
Main-lobe width	6.000 bins
Peak sidelobe	-46.741 dB

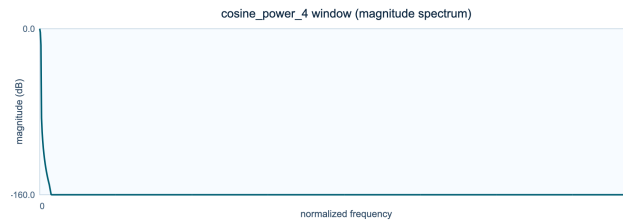
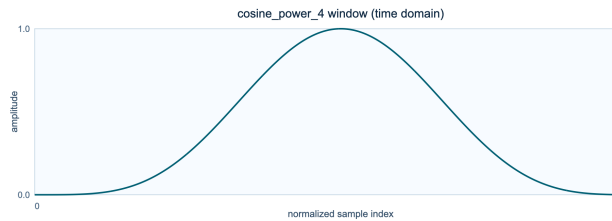


Figure 8.77: Time-domain shape of the cosine power 4 window.

Figure 8.78: Magnitude spectrum of the cosine power 4 window.

Interpretation and use

Strength. Simple power parameter controls edge steepness.

Limitation. Higher powers can overly narrow effective support.

Recommendation. Use to smoothly increase edge attenuation versus basic sine.

hann poisson 0p5

`hann_poisson_0p5` is a hann-poisson design with `alpha=0.5`. Multiplying Hann by an exponential envelope strengthens edge decay while preserving a familiar center. Its coherent gain is 0.432946, and its equivalent noise bandwidth is 1.609944 bins.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.41: Measured properties of the hann poisson 0p5 window.

Metric	Value
Coherent gain	0.432946
Equivalent noise bandwidth	1.609944 bins
Scalloping loss	-1.258 dB
Main-lobe width	5.188 bins
Peak sidelobe	-35.245 dB

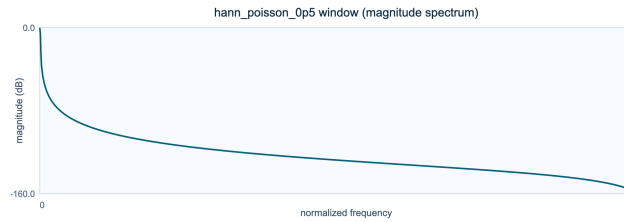
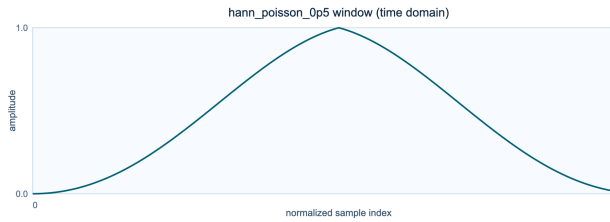


Figure 8.79: Time-domain shape of the hann poisson 0p5 window. **Figure 8.80:** Magnitude spectrum of the hann poisson 0p5 window.

Interpretation and use

Strength. Combines smooth Hann center with exponential edge suppression.

Limitation. Can over-attenuate edges for high alpha values.

Recommendation. Use when you need stronger edge decay than Hann without full flattop cost.

hann poisson 1p0

`hann_poisson_1p0` is a hann-poisson design with `alpha=1`. Multiplying Hann by an exponential envelope strengthens edge decay while preserving a familiar center. Its coherent gain is 0.378797, and its equivalent noise bandwidth is 1.734176 bins.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.42: Measured properties of the hann poisson 1p0 window.

Metric	Value
Coherent gain	0.378797
Equivalent noise bandwidth	1.734176 bins
Scalloping loss	-1.112 dB
Main-lobe width	103.750 bins
Peak sidelobe	-80.017 dB

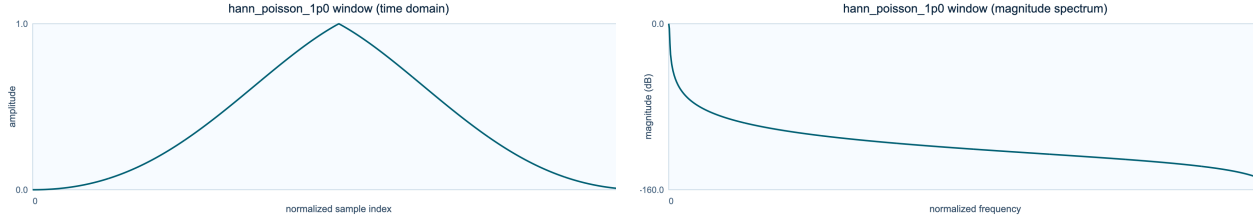


Figure 8.81: Time-domain shape of the hann poisson 1p0 window. **Figure 8.82:** Magnitude spectrum of the hann poisson 1p0 window.

Interpretation and use

Strength. Combines smooth Hann center with exponential edge suppression.

Limitation. Can over-attenuate edges for high alpha values.

Recommendation. Use when you need stronger edge decay than Hann without full flattop cost.

hann poisson 2p0

`hann_poisson_2p0` is a hann-poisson design with `alpha=2`. Multiplying Hann by an exponential envelope strengthens edge decay while preserving a familiar center. Its coherent gain is 0.297878, and its equivalent noise bandwidth is 2.023174 bins.

$$w[n] = w_{\text{Hann}}[n] \exp\left(-\alpha \frac{|n-m|}{m}\right)$$

where $w_{\text{Hann}}[n]$ is the Hann window, α controls exponential decay, and $m = (N-1)/2$.

Table 8.43: Measured properties of the hann poisson 2p0 window.

Metric	Value
Coherent gain	0.297878
Equivalent noise bandwidth	2.023174 bins
Scalloping loss	-0.870 dB
Main-lobe width	164.719 bins
Peak sidelobe	-80.005 dB

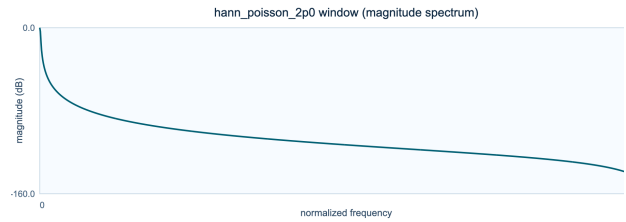
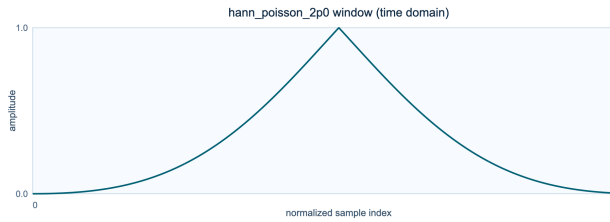


Figure 8.83: Time-domain shape of the hann poisson 2p0 window.

Figure 8.84: Magnitude spectrum of the hann poisson 2p0 window.

Interpretation and use

Strength. Combines smooth Hann center with exponential edge suppression.

Limitation. Can over-attenuate edges for high alpha values.

Recommendation. Use when you need stronger edge decay than Hann without full flattop cost.

general hamming 0p50

`general_hamming_0p50` is a general hamming design with `alpha=0.50`. Changing the constant term moves continuously through several familiar cosine tapers. Its coherent gain is 0.499756, and its equivalent noise bandwidth is 1.500733 bins.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.44: Measured properties of the general hamming 0p50 window.

Metric	Value
Coherent gain	0.499756
Equivalent noise bandwidth	1.500733 bins
Scalloping loss	-1.422 dB
Main-lobe width	4.000 bins
Peak sidelobe	-31.468 dB

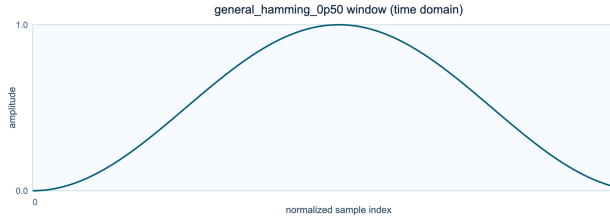


Figure 8.85: Time-domain shape of the general hamming 0p50 window.

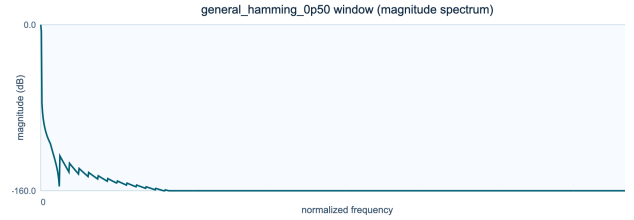


Figure 8.86: Magnitude spectrum of the general hamming 0p50 window.

Interpretation and use

Strength. Tunable Hamming-style cosine weighting.

Limitation. Less standardized than classic Hann/Hamming choices.

Recommendation. Use to sweep sidelobe-vs-width behavior near Hamming family defaults.

general hamming 0p60

`general_hamming_0p60` is a general hamming design with `alpha=0.60`. Changing the constant term moves continuously through several familiar cosine tapers. Its coherent gain is 0.599805, and its equivalent noise bandwidth is 1.222475 bins.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.45: Measured properties of the general hamming 0p60 window.

Metric	Value
Coherent gain	0.599805
Equivalent noise bandwidth	1.222475 bins
Scalloping loss	-2.179 dB
Main-lobe width	3.469 bins
Peak sidelobe	-31.600 dB

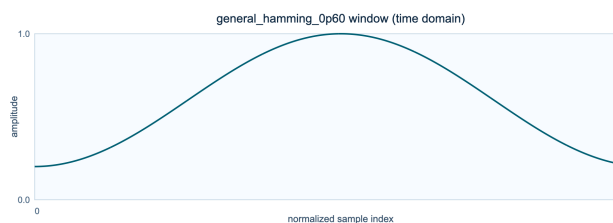


Figure 8.87: Time-domain shape of the general hamming 0p60 window.

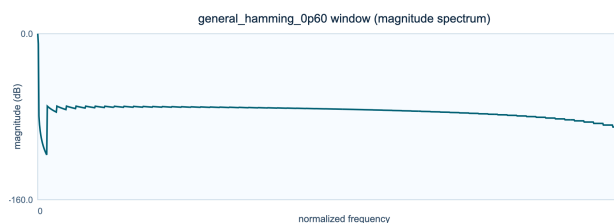


Figure 8.88: Magnitude spectrum of the general hamming 0p60 window.

Interpretation and use

Strength. Tunable Hamming-style cosine weighting.

Limitation. Less standardized than classic Hann/Hamming choices.

Recommendation. Use to sweep sidelobe-vs-width behavior near Hamming family defaults.

general hamming 0p70

`general_hamming_0p70` is a general hamming design with `alpha=0.70`. Changing the constant term moves continuously through several familiar cosine tapers. Its coherent gain is 0.699854, and its equivalent noise bandwidth is 1.091920 bins.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.46: Measured properties of the general hamming 0p70 window.

Metric	Value
Coherent gain	0.699854
Equivalent noise bandwidth	1.091920 bins
Scalloping loss	-2.762 dB
Main-lobe width	2.656 bins
Peak sidelobe	-24.078 dB

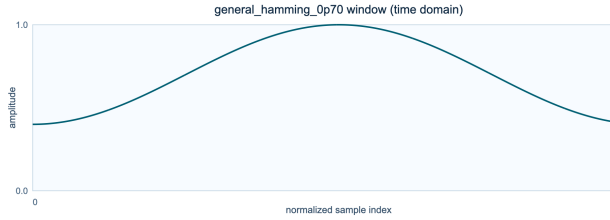


Figure 8.89: Time-domain shape of the general hamming 0p70 window.

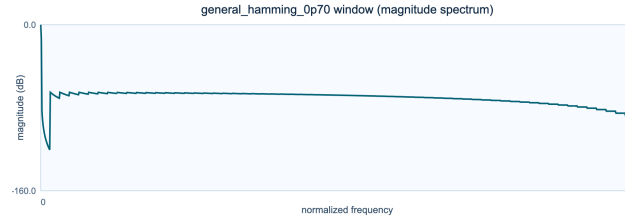


Figure 8.90: Magnitude spectrum of the general hamming 0p70 window.

Interpretation and use

Strength. Tunable Hamming-style cosine weighting.

Limitation. Less standardized than classic Hann/Hamming choices.

Recommendation. Use to sweep sidelobe-vs-width behavior near Hamming family defaults.

general hamming 0p80

`general_hamming_0p80` is a general hamming design with `alpha=0.80`. Changing the constant term moves continuously through several familiar cosine tapers. Its coherent gain is 0.799902, and its equivalent noise bandwidth is 1.031273 bins.

$$w[n] = \alpha - (1 - \alpha) \cos\left(\frac{2\pi n}{N-1}\right)$$

where α sets the constant-to-cosine balance, n is the sample index, and N is the window length.

Table 8.47: Measured properties of the general hamming 0p80 window.

Metric	Value
Coherent gain	0.799902
Equivalent noise bandwidth	1.031273 bins
Scalloping loss	-3.227 dB
Main-lobe width	2.312 bins
Peak sidelobe	-18.649 dB

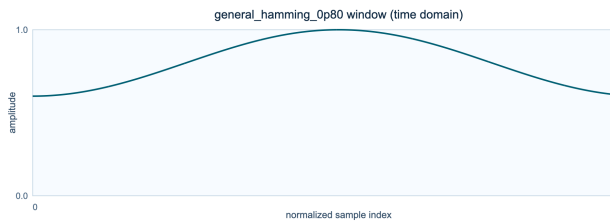


Figure 8.91: Time-domain shape of the general hamming 0p80 window.

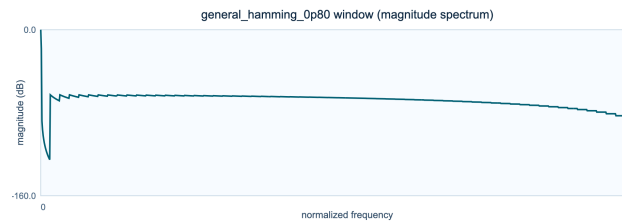


Figure 8.92: Magnitude spectrum of the general hamming 0p80 window.

Interpretation and use

Strength. Tunable Hamming-style cosine weighting.

Limitation. Less standardized than classic Hann/Hamming choices.

Recommendation. Use to sweep sidelobe-vs-width behavior near Hamming family defaults.

bohman

bohman is a bohman design with **none**. Its smooth edge behavior favors low leakage over a compact main lobe. Its coherent gain is 0.405087, and its equivalent noise bandwidth is 1.786613 bins.

$$w[n] = (1 - x) \cos(\pi x) + \frac{\sin(\pi x)}{\pi}$$

where $x = |2n/(N - 1) - 1|$ is absolute centered position, n is the sample index, and N is the window length.

Table 8.48: Measured properties of the bohman window.

Metric	Value
Coherent gain	0.405087
Equivalent noise bandwidth	1.786613 bins
Scalloping loss	-1.022 dB
Main-lobe width	6.000 bins
Peak sidelobe	-45.997 dB

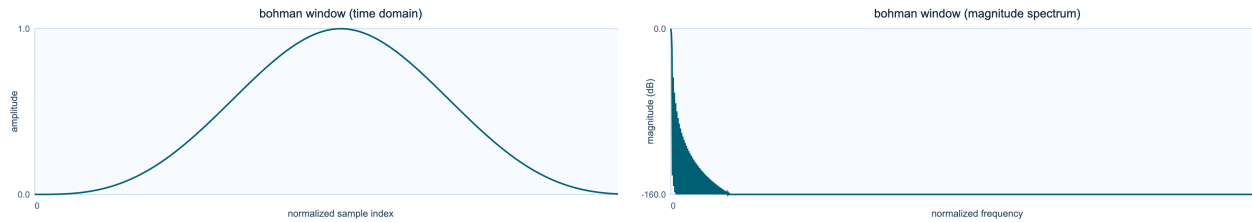


Figure 8.93: Time-domain shape of the bohman window. **Figure 8.94:** Magnitude spectrum of the bohman window.

Interpretation and use

Strength. Continuous-slope taper with good qualitative leakage control.

Limitation. Less common in audio tooling and harder to tune by intuition.

Recommendation. Use for exploratory spectral work needing smooth derivatives.

cosine

`cosine` is a sinusoidal design with `none`. The half-sine contour is smooth, symmetric, and easy to reason about under overlap. Its coherent gain is 0.636309, and its equivalent noise bandwidth is 1.234304 bins.

$$w[n] = \sin\left(\frac{\pi n}{N-1}\right)$$

where n is the sample index and N is the window length.

Table 8.49: Measured properties of the cosine window.

Metric	Value
Coherent gain	0.636309
Equivalent noise bandwidth	1.234304 bins
Scalloping loss	-2.096 dB
Main-lobe width	3.000 bins
Peak sidelobe	-22.999 dB

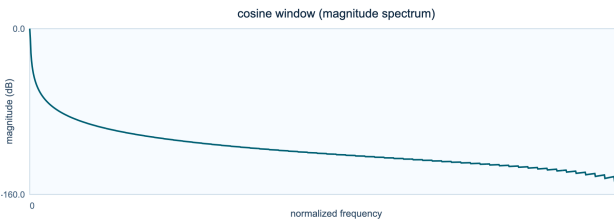
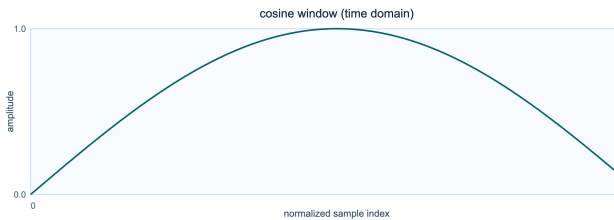


Figure 8.95: Time-domain shape of the cosine window. **Figure 8.96:** Magnitude spectrum of the cosine window.

Interpretation and use

Strength. Smooth endpoint behavior and straightforward implementation.

Limitation. Less configurable than Kaiser/Tukey families.

Recommendation. Use for stable, low-complexity alternatives to Hann.

kaiser

kaiser is a kaiser-bessel design with **beta** from `--kaiser-beta`. The **beta** parameter offers a direct and widely understood control over spectral concentration. Its coherent gain is 0.331708, and its equivalent noise bandwidth is 2.162181 bins.

$$w[n] = \frac{I_0(\beta\sqrt{1-r_n^2})}{I_0(\beta)}, \quad r_n = \frac{n-m}{m}$$

where I_0 is the modified Bessel function, β controls taper strength, $m = (N-1)/2$, and n is the sample index.

Table 8.50: Measured properties of the kaiser window.

Metric	Value
Coherent gain	0.331708
Equivalent noise bandwidth	2.162181 bins
Scalloping loss	-0.712 dB
Main-lobe width	9.125 bins
Peak sidelobe	-105.921 dB

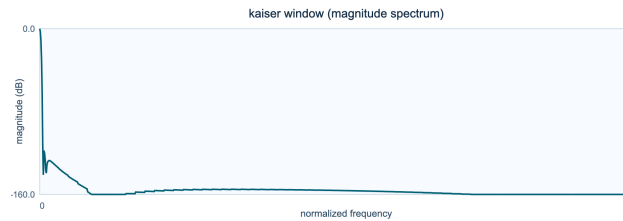
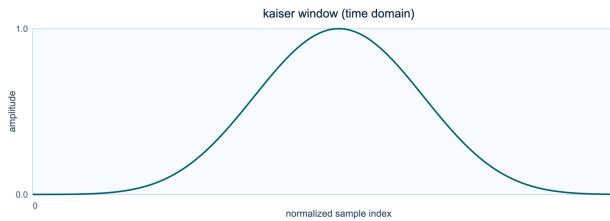


Figure 8.97: Time-domain shape of the kaiser window.

Figure 8.98: Magnitude spectrum of the kaiser window.

Interpretation and use

Strength. Continuously tunable width/sidelobe tradeoff via **beta**.

Limitation. Needs **beta** tuning; poor **beta** choices can over-blur or under-suppress sidelobes.

Recommendation. Use when you need explicit control over leakage versus resolution.

rect

rect is a rectangular design with **none**. With no taper at all, it provides the sharpest baseline for seeing leakage caused by abrupt edges. Its coherent gain is 1.000000, and its equivalent noise bandwidth is 1.000000 bins.

$$w[n] = 1$$

where $w[n]$ is the window coefficient and n is any sample index from 0 through $N - 1$.

Table 8.51: Measured properties of the rect window.

Metric	Value
Coherent gain	1.000000
Equivalent noise bandwidth	1.000000 bins
Scalloping loss	-3.922 dB
Main-lobe width	2.000 bins
Peak sidelobe	-13.264 dB

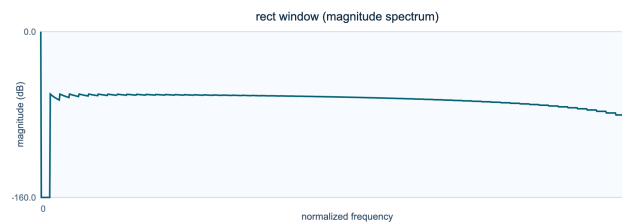
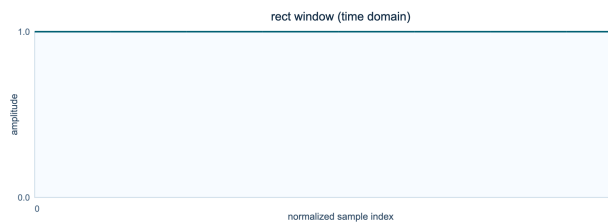


Figure 8.99: Time-domain shape of the rect window.

Figure 8.100: Magnitude spectrum of the rect window.

Interpretation and use

Strength. Narrowest main lobe and maximal bin sharpness.

Limitation. Highest sidelobes and strongest leakage/phasing on non-bin-centered content.

Recommendation. Use only for controlled test tones or when leakage is acceptable.

CHAPTER 9

Lessons from Paul Koonce's PVC for pvx

Paul Koonce's PVC is useful to `pvx` for reasons deeper than a shared interest in phase-vocoder processing. PVC treated spectral transformation as a collection of focused Unix tools, persistent analysis files, reusable response data, and time-varying control functions. Its manual documented not only what each routine did, but how a musician was expected to assemble routines into a repeatable working method.

This chapter studies that method rather than proposing literal command compatibility. Historical PVC and modern `pvx` differ in file formats, performance expectations, interface conventions, and implementation architecture. The aim is to preserve the durable ideas: make representations explicit, make automation ordinary, keep commands composable, expose consequential processing choices, and distinguish stable tools from experiments.

The principal historical source is Koonce's archived PVC manual. A secondary catalog record helps establish the package's circulation in the Linux-audio community.

- [Paul Koonce, PVC manual, Princeton archive](#)
- [PVC package entry, Linux Audio applications index](#)

9.1 PVC as a working environment

Koonce described PVC 1.0 as a collection of phase-vocoder signal-processing routines and shell scripts written in C for Unix. The package was built in the spirit of work by Eric Lyon and Chris Penrose and pointed readers toward F. Richard Moore and Mark Dolson for the phase-vocoder foundations. This genealogy matters because PVC was not presented as an isolated invention. It was a working layer built from inherited code, published explanations, local research, and the practical needs of a composer-programmer.

The collection ranged from a generic phase vocoder to specialized tools for time warping, filtering, companding, harmonic remapping, convolution, resonance, envelope extraction, and function reshaping. Several commands first produced an analysis or response artifact that another command consumed. Others read parameter functions from external files. Shell scripts held the many values needed for a processing pass and connected preparatory analysis to resynthesis.

This design made the command line a compositional environment. A user did not merely choose an effect and move a few controls. The user selected a representation, prepared data, transformed it, chose a resynthesis path, and preserved the decisions in a script. That sequence remains a strong model for serious offline audio work.

PVC family	Persistent or controlling object	Musical purpose
<code>plainpv</code> , <code>twarp</code>	spectral frames and time functions	change time, pitch, spectral position, and amplitude
<code>pvanalysis</code>	reusable complex analysis data	prepare a source for several later readings
<code>freqresponse</code> , <code>chordresponsemaker</code>	measured or synthesized response	construct spectral templates
<code>filter</code> , <code>tvfilter</code> , <code>convolver</code>	static or evolving spectra	filter and cross-synthesize sounds
<code>harmonizer</code> , <code>chordmapper</code>	frequency replication and mapping rules	construct harmonies and spectral chords

PVC family	Persistent or controlling object	Musical purpose
<code>ring</code> , <code>ringfilter</code> , <code>ringtvfilter</code>	spectral feedback and response data	create source-shaped resonances
<code>envelope</code> , <code>reshape</code>	scalar function streams	extract and transform automation data

The table shows that PVC's real unit of design was not the effect name. It was the relationship among a sound, an intermediate representation, a control stream, and a resynthesis method. That distinction guides every lesson that follows.

9.2 Lesson one: small commands can form a broad language

PVC exposed focused routines such as `plainpv`, `twarp`, `filter`, `harmonizer`, and `ringfilter`. Each routine had a recognizable purpose even when it offered many parameters. The collection became broad through composition rather than through one universal command with every possible flag.

Small commands improve more than discoverability. They establish boundaries for testing, documentation, failure reporting, and saved examples. A user can learn what kind of artifact enters a command and what kind leaves it. A developer can change one operator while preserving shared conventions for input, output, automation, and diagnostics.

`pvx` carries this idea forward through subcommands including `voc`, `freeze`, `formant`, `filter`, `retune`, `analysis`, `response`, `tvfilter`, `ringfilter`, `chordmapper`, `envelope`, and `reshape`. The `chain` command provides managed serial composition when a workflow should remain one reproducible invocation.

```
pvx chain vocal.wav \
  --pipeline "denoise --reduction-db 6 | voc --stretch 1.3 | formant --mode preserve" \
  --output vocal_prepared.wav
```

The lesson has a limit. A project can fragment itself into dozens of commands whose conventions disagree. Focused tools remain useful only when they share naming, map formats, channel behavior, output rules, error messages, and help structure. The command boundary should clarify an operation, not merely relocate complexity.

9.3 Lesson two: time-varying control belongs in the foundation

PVC allowed parameters marked as functions to read headerless streams of 32-bit floating-point values. The package fitted a function stream to the requested duration and interpolated between values. CMUSIC generation tools supplied control data, while `envelope` and `reshape` could derive or transform functions. This made automation part of ordinary command-line practice rather than an optional graphical layer.

A simple linear interpretation of adjacent function samples is:

$$u(t) = (1 - \lambda)u_i + \lambda u_{i+1}$$

where $u(t)$ is the parameter value at time t , u_i and u_{i+1} are adjacent control values, and λ is the normalized position between their timestamps in the interval from zero to one.

The equation is modest, but its architectural consequence is large. A parameter should not need a separate implementation for scalar and time-varying use. The renderer should be able to ask for the value at the current time while the control layer decides whether that value came from a constant, a CSV column, a JSON object, a generated envelope, or a routed analysis feature.

Modern `pvx` uses named, inspectable control maps rather than PVC's anonymous binary float streams. A control trajectory can be generated, reshaped, and then applied to a render:


```

pvx envelope \
  --mode adsr --duration 8 --rate 20 \
  --attack-sec 0.2 --decay-sec 0.6 --sustain 1.1 --release-sec 1.0 \
  --key stretch --output controls/stretch_env.csv

pvx reshape controls/stretch_env.csv \
  --key stretch --operation resample --rate 50 \
  --interp polynomial --order 5 \
  --output controls/stretch_dense.csv

pvx voc input.wav \
  --stretch controls/stretch_dense.csv --interp linear \
  --output input_envdriven.wav

```

The durable lesson is not that every parameter should move constantly. It is that constant values and trajectories should participate in one coherent control system. A scalar is simply the least complicated trajectory.

9.4 Lesson three: intermediate artifacts support musical memory

PVC's `pvanalysis` files separated analysis from transformation. One analysis could supply time warping, convolution, or time-varying filtering without requiring the source to be analyzed again. Static response files similarly represented measured or synthesized spectral shapes for filtering, companding, and resonance.

Persistent artifacts change the creative process. They make analysis reusable, but they also make it comparable. A user can inspect metadata, derive several responses from one analysis, archive a useful filter profile, or rerun only the stage that changed. A long render becomes a sequence of named decisions rather than one opaque calculation.

The modern equivalent must carry more context than PVC's raw working files could reliably preserve. Transform size, window, hop, sample rate, channel policy, normalization, units, software version, and integrity information determine whether stored spectral data remains meaningful. `pvx` therefore uses structured PVXAN analysis artifacts and PVXRF response artifacts.

```

pvx analysis create source.wav \
  --output source.pvxan.npz \
  --n-fft 4096 --win-length 4096 --hop-size 256 --window hann

pvx analysis inspect source.pvxan.npz

pvx response create source.pvxan.npz \
  --output source.pvxrf.npz \
  --method median --phase-mode mean --normalize peak --smoothing-bins 3

pvx response inspect source.pvxrf.npz

```

This is not persistence for its own sake. An artifact earns its place when it can be inspected, validated, reused, and connected to the source and parameters that produced it. Otherwise, caching merely creates another opaque file.

9.5 Lesson four: expose the processing choice that changes the sound

Koonce's manual distinguished overlap-add and oscillator-bank resynthesis. Magnitude-only changes could preserve enough spectral structure for the faster overlap-add route. Frequency changes disturbed that structure and required sinusoidal oscillator-bank resynthesis. A threshold could suppress low-amplitude oscillators to reduce cost.

The exact division belongs to PVC's implementation, but the interface lesson remains current. When an algorithmic choice changes transients, phase coherence, noise texture, or computational cost, it should not disappear behind a vague quality label. Users need an intelligible model of why two modes sound different and when one is appropriate.

Modern processors can choose among direct inverse transforms, phase propagation, peak locking, sinusoidal models, transient replacement, waveform-aligned segments, and hybrid methods. Not every internal switch deserves a public flag. The ones that alter the audible contract should appear as stable concepts with meaningful defaults and listening guidance.

This principle also discourages false equivalence. Two modes are not necessarily points on a single better-to-worse scale. One may preserve harmonic continuity while another protects attacks. A third may embrace diffusion as a useful texture. Documentation should describe the trade rather than hide it behind words such as high quality.

9.6 Lesson five: scripts are part of the instrument

PVC shipped shell scripts that Koonce described as a practical substitute for a graphical interface. The scripts stored numerous parameters, ran preliminary analyses, and invoked the main command. Their value was not visual polish. They turned a successful experiment into a procedure that could be read, revised, and repeated.

This remains one of the command line's strongest musical properties. A command can function as a score for a transformation. Version control can show how that score changed. Batch execution can apply it to a corpus. A collaborator can inspect the exact operation rather than infer it from a filename such as `final_really_final.wav`.

Modern workflow support should improve on raw shell history without making the process less transparent. Manifests, checkpoints, deterministic modes, explicit seeds, managed chains, and machine-readable reports can preserve intention around a command. The script remains valuable because it is both executable and legible.

9.7 Lesson six: stable releases need an experimental edge

Koonce wrote that PVC 1.0 included routines he considered stable, useful, and moderately transparent, while other experimental routines remained outside the release. That statement captures a mature release discipline. Musical software needs room for speculative processing, but a user also needs to know which surfaces can support a composition, a class, or a production pipeline.

For `pvx`, the appropriate response is not to stop experimentation. It is to mark status honestly. Supported commands need tests, stable help, documented formats, and migration care. Experimental commands need clear warnings, disposable outputs, and freedom to change. The distinction protects both reliability and invention.

The lesson is especially important for a large command collection. Adding a name to top-level help creates an expectation. A narrow alpha surface can be more useful than a broad inventory whose behaviors and file contracts cannot yet be maintained.

9.8 What pvx should not copy

Historical respect does not require historical reenactment. Several PVC constraints arose from its time and should not define a modern implementation.

PVC accepted NeXT/Sun 16-bit sound files and used conventions suited to the Unix systems on which it ran. Modern software should accept well-defined contemporary formats, preserve channel and sample-rate metadata, and avoid making one workstation's native representation the user's problem.

PVC's binary function streams were efficient and simple, but a headerless vector does not identify its rate, units, key, interpolation, duration policy, or provenance. Human-readable CSV and structured JSON are better interchange formats for control data, while compact binary storage can remain an internal optimization.

The package's shell scripts managed commands with very large flag sets. Scripts remain valuable, but the modern answer to parameter growth also includes presets, schemas, grouped help, validation, manifests, and focused subcommands. A script should preserve a decision, not compensate indefinitely for an interface that no one can understand.

Finally, parity should not mean reproducing every artifact or implementation limitation. **pvx** should preserve the musical operation and the inspectable workflow while using current knowledge about phase coherence, transient handling, multichannel processing, numerical precision, and testing.

9.9 Three PVC-inspired laboratories

The following laboratories turn the historical lessons into small, reproducible **pvx** exercises. Each begins with a representation and ends with a listening question. Use short, legally obtained source files and keep the unprocessed source for comparison.

Laboratory one: a reusable spectral imprint

This laboratory follows the PVC pattern of analysis, response construction, and time-varying filtering. The source supplies a durable spectral profile, while a separate control map determines how strongly that profile enters the target.

```
pvx analysis create source.wav --output source.pvxan.npz
pvx response create source.pvxan.npz \
  --output source.pvxrf.npz --method median --normalize peak
pvx envelope \
  --mode ramp --duration 12 --rate 30 \
  --start 0.0 --end 1.0 --key response_mix \
  --output controls/response_mix.csv
pvx tvfilter target.wav \
  --response source.pvxrf.npz \
  --tv-map controls/response_mix.csv --tv-key response_mix \
  --tv-interp s_curve --output target_imprinted.wav
```

Listen for the point at which the source profile becomes identifiable. Compare that moment with the control-map value. Then change only the response method or smoothing and ask whether identity arrives through broad envelope shape, narrow resonances, or noise coloration.

Laboratory two: control data as compositional material

This laboratory treats an envelope as an independent object. Generate it once, create several transformed versions, and apply each version to the same source. The comparison isolates the musical effect of control shaping from the underlying phase-vocoder settings.

```

pvx envelope \
  --mode exp --duration 10 --rate 30 \
  --start 0.7 --end 1.8 --exp-curve 6 \
  --key stretch --output controls/stretch_exp.csv
pvx reshape controls/stretch_exp.csv \
  --key stretch --operation smooth --window 11 \
  --output controls/stretch_smooth.csv
pvx voc input.wav \
  --stretch controls/stretch_exp.csv \
  --output input_exp.wav
pvx voc input.wav \
  --stretch controls/stretch_smooth.csv \
  --output input_smooth.wav

```

Listen at moments where the stretch changes most rapidly. The question is not simply which render is cleaner. Ask whether smoothing improves the intended gesture or removes an articulation that made the trajectory musically clear.

Laboratory three: spectral resonance and harmonic mapping

PVC used analyzed and synthetic responses to shape resonance, while its harmonic tools replicated or remapped spectral components. This laboratory compares those two ways of imposing pitch organization on a sound.

```

pvx ringfilter input.wav \
  --frequency-hz 60 --depth 0.8 --mix 0.9 \
  --resonance-hz 1200 --resonance-q 9 \
  --resonance-mix 0.4 --resonance-decay 0.2 \
  --output input_ringfilter.wav

pvx chordmapper input.wav \
  --root-hz 220 --chord min7 \
  --strength 0.75 --tolerance-cents 35 \
  --boost-db 6 --attenuation 0.45 \
  --output input_chordmapped.wav

```

The resonator emphasizes persistence near selected frequencies; the chord mapper reorganizes energy according to a harmonic model. Compare source recognition, attack clarity, tonal center, and the behavior of noise. The two renders may suggest a useful chain, but first learn what each operation contributes alone.

9.10 A continuing design checklist

PVC's example suggests a compact checklist for future `pvx` work. The checklist should be applied before an advanced operator is considered complete.

1. Give the operator one clear musical purpose and shared input-output conventions.
2. Document scalar and time-varying control separately, including interpolation and units.
3. Make persistent artifacts self-describing, inspectable, and versioned.
4. Explain any resynthesis choice that materially changes sound or cost.
5. Provide one conservative example, one expressive example, and one warning about unsuitable material.
6. Add focused tests for the operator and an end-to-end fixture for its normal workflow.
7. Record whether the command is supported, provisional, or experimental.
8. Preserve commands, manifests, seeds, and relevant metadata for reproducible renders.

PVC's enduring lesson is that algorithms, representations, controls, and working procedures should be designed together. This chapter paraphrases Koonce's *PVC* manual in Princeton University's spring 1999 CS325 archive, with the Linux Audio index as a secondary record. No direct source-code descent from PVC to `pvx` is claimed.

CHAPTER 10

PVC-to-pvx Parity Matrix

This document tracks functional parity work between Paul Koonce’s historical PVC package and modern `pvx`.

Primary references: - Princeton archive page (Paul Koonce): <https://www.cs.princeton.edu/courses/archive/spr99/cs325/koonce.html> - Linux Audio package index entry: <https://wiki.linuxaudio.org/apps/all/pvc>

10.1 Scope

Before applying scope, keep these constraints in view.

- Goal: preserve useful PVC workflow primitives while keeping `pvx` quality-first and automation-friendly.
- Out of scope: byte-for-byte command-line compatibility with historical PVC binaries.

10.2 Capability Matrix

The relevant properties of the capability matrix are compared in the following table.

PVC capability	Why it matters	Current <code>pvx</code> status	Gap level
Persistent phase-vocoder analysis files (<code>pvanalysis</code>)	Reuse analysis across multiple processing passes	Implemented in phase 1/2 via <code>PVXAN</code> schema and <code>pvx analysis</code>	Closed (foundation)
Response-file workflows (<code>freqresponse</code> , <code>chordresponsemaker</code>)	Reusable filter/target curves	Implemented in phase 1/2 via <code>PVXRF</code> schema and <code>pvx response</code>	Closed (foundation)
Time-varying spectral filtering from response files (<code>tvfilter</code>)	Dynamic timbral trajectories	Implemented in phase 3 via <code>pvx tvfilter</code> + scalar map interpolation	Closed (phase 3)
Ring/resonator family (<code>ring</code> , <code>ringfilter</code> , <code>ringtvfilter</code>)	Controlled resonant coloration	Implemented in phase 4 via dedicated ring CLI family	Closed (phase 4)
Response-driven compander/noise/band emphasis (<code>compander</code> , <code>noisefilter</code> , <code>bandamp</code>)	Precision spectral dynamics	Implemented in phase 3 (<code>pvx spec-compander</code> , <code>pvx noisefilter</code> , <code>pvx bandamp</code>)	Closed (phase 3)
Harmonic/chord remapping (<code>harmonizer</code> , <code>chordmapper</code> , <code>inharmonator</code>)	Musically constrained spectral remap	Implemented in phase 5 via <code>pvx chordmapper</code> and <code>pvx inharmonator</code>	Closed (phase 5)
Spectral convolver with response semantics (<code>convolver</code>)	Controlled blend of source/target spectra	Partial overlap via morph/cross-synthesis	Medium
Generic function stream utilities (<code>envelope</code> , <code>reshape</code>)	Text-based control-signal authoring	Implemented in phase 6 via <code>pvx envelope</code> and <code>pvx reshape</code>	Closed (phase 6)

PVC capability	Why it matters	Current pvx status	Gap level
Parity benchmark scenarios and regression gates	Reproducible quality checks for PVC-inspired operators	Implemented in phase 7 via <code>benchmarks/run_pvc_parity.py</code> + CI workflow gate	Closed (phase 7)
Explicit resynthesis families and thresholds	Operator transparency and reproducibility	Partially available but not as PVC-style dedicated modes	Medium

10.3 Implemented in Phase 1 and Phase 2

Implemented in Phase 1 and Phase 2 depends on a few concrete choices.

1. Data-model and schema foundations:
 - PVXAN analysis artifact schema in `/Users/cleider/dev/pvx/src/pvx/core/analysis_store.py`
 - PVXRF response artifact schema in `/Users/cleider/dev/pvx/src/pvx/core/response_store.py`
2. User-facing command-line interface (CLI) foundations:
 - `pvx analysis create|inspect` in `/Users/cleider/dev/pvx/src/pvx/cli/pvxanalysis.py`
 - `pvx response create|inspect` in `/Users/cleider/dev/pvx/src/pvx/cli/pvxresponse.py`
 - Unified CLI registration in `/Users/cleider/dev/pvx/src/pvx/cli/pvx.py`

10.4 Schema Notes

The discussion of the schema notes begins by separating its main parts.

PVXAN

PVXAN depends on a few concrete choices.

- Container: compressed NumPy `.npz`
- Required members:
 - `meta_json`
 - `spectrum_real (channels x frames x bins)`
 - `spectrum_imag (channels x frames x bins)`
- Includes deterministic `sha256` digest function for reproducibility checks.

PVXRF

A reliable approach to `pvxrf` begins with these points.

- Container: compressed NumPy `.npz`
- Required members:
 - `meta_json`
 - `frequencies_hz (bins)`
 - `magnitude (channels x bins)`
 - `phase (channels x bins)`
- Includes deterministic `sha256` digest function for reproducibility checks.

10.5 Implemented in Phase 3, Phase 4, and Phase 5

These are the details that matter for implemented in phase 3, phase 4, and phase 5.

1. Response-driven spectral operators (`filter`, `tvfilter`, `noisefilter`, `bandamp`, `spec-compander`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_ops.py`
 - command-line interface (CLI): `/Users/cleider/dev/pvx/src/pvx/cli/pvxfilter.py` + wrappers
2. Ring/resonator family (`ring`, `ringfilter`, `ringtvfilter`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_resonators.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxring.py` + wrappers
3. Harmonic/chord mapping family (`chordmapper`, `inharmonator`):
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_harmony.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxharmpmap.py` + wrappers

10.6 Implemented in Phase 6 and Phase 7

These are the details that matter for implemented in phase 6 and phase 7.

1. Function-stream generators/transforms (`envelope`, `reshape`) interoperable with control-bus workflows:
 - core: `/Users/cleider/dev/pvx/src/pvx/core/pvc_functions.py`
 - CLI: `/Users/cleider/dev/pvx/src/pvx/cli/pvxenvelope.py`, `/Users/cleider/dev/pvx/src/pvx/cli/pvxreshape.py`
 - installed entry points: `pvx envelope`, `pvx reshape`
2. Parity benchmarks and regression gates:
 - benchmark runner: `/Users/cleider/dev/pvx/benchmarks/run_pvc_parity.py`
 - baseline: `/Users/cleider/dev/pvx/benchmarks/baseline_pvc_parity.json`
 - CI workflow: `/Users/cleider/dev/pvx/.github/workflows/pvc-parity-regression.yml`
 - expanded pipeline scenario: `analysis_response_function_chain` (analysis-derived response + function-stream modulation)

10.7 Immediate Next Steps

The working assumptions for immediate next steps are:

1. Add explicit PVC-style resynthesis family toggles and thresholds where they improve auditability without fragmenting the core `pvx voc` interface.
2. Add additional parity scenarios that stress the same pipeline family with persisted artifact round-trips (`pvx analysis create` + `pvx response create`) and stricter drift gates.
3. Add the sorted top-level command roadmap below to stage user-facing workflow expansion in quality-first order.

Sorted Top-Level Command Roadmap

This reference introduces the sorted top-level command roadmap before presenting its individual entries.

Phase	Priority	Proposed commands	Parity and quality rationale
Phase 1	Highest	<code>doctor</code> , <code>inspect</code> , <code>validate</code> , <code>schema</code> , <code>preset</code> , <code>config</code>	Improves reproducibility and inspectability before introducing more operator complexity.
Phase 2	High	<code>render</code> , <code>graph</code> , <code>queue</code> , <code>watch</code> , <code>cache</code>	Supports longer research- and production-oriented runs with less shell friction.

10. PVC-to-pvx Parity Matrix

Phase	Priority	Proposed commands	Parity and quality rationale
Phase 3	Medium-High	<code>mod</code> , <code>derive</code> , <code>route</code> , <code>quantize</code> , <code>smooth</code>	Brings function-stream/control-signal workflows closer to classic PVC flexibility with modern parameter routing.
Phase 4	Medium	<code>bench</code> , <code>compare</code> , <code>regress</code> , <code>abx</code> , <code>report</code>	Establishes objective parity and regression tracking as first-class command surfaces.
Phase 5	Medium-Low	<code>align</code> , <code>match</code> , <code>stem</code> , <code>spatialize</code> , <code>live</code> , <code>serve</code>	Expands into advanced alignment, immersive, and deployment workflows after core parity and QA gates are stable.

Part IV

Workflow Cookbook

CHAPTER 11

pvx Example Cookbook

All commands are designed to be copy-paste runnable from the repository root. If one fails, the shell will usually inform you with all the warmth of a tax letter.

11.1 Starter Path (Run These First)

If you are new to **pvx**, run these in order:

```
pvx voc input.wav --stretch 1.2 --output step1_stretch.wav
pvx voc input.wav --stretch 1.0 --pitch -3 --output step2_pitch.wav
pvx voc input.wav --stretch 1.0 --pitch -3 --pitch-mode formant-preserving --output step3_pitch_formant.wav
pvx voc drums.wav --preset drums_safe --stretch 1.25 --output step4_drums.wav
pvx voc mix.wav --stretch 1.2 --stereo-mode mid_side_lock --coherence-strength 0.9 --output step5_stereo.wav
```

Expected audible progression: - **step1**: timing changes only - **step2**: pitch changes only - **step3**: pitch change with improved vocal timbre stability - **step4**: better attack retention on percussive material - **step5**: more stable stereo image during stretch - if all five sound identical, something is broken and tea alone will not save it

Preflight for extreme ratios:

```
pvx stretch-budget one_shot.wav --disk-budget 20GB --bit-depth 16 --requested-stretch 1000000
pvx stretch-budget one_shot.wav --disk-budget 20GB --requested-stretch 1000000 --fail-if-exceeds --json
```

Use this before launching very large renders so budget limits are explicit.

11.2 Use-Case Index (By Theme)

A tabular view makes the distinctions within the use-case index (by theme) easier to inspect.

Theme	Start here	Typical tools
Speech intelligibility and timing	1, 2, 25, 61	pvx voc
Musical pitch and formants	3, 4, 5, 29, 37, 48	pvx voc , pvx retune
Extreme ambient and drones	6, 28, 45, 50, 65	pvx voc , pvx freeze
Transient-safe processing	26, 30, 32, 33, 52	pvx voc
Stereo/multichannel coherence	27, 34, 35, 57, 58	pvx voc
Morphing and hybrid sound design	9, 23, 49, 55	pvx morph , pvx layer , pvx harmonize
Cleanup/restoration	11, 12, 50, 53, 64	pvx denoise , pvx deverb

Theme	Start here	Typical tools
Automation and reproducibility	13, 14, 24, 31, 44, 47, 63	<code>pvx conform</code> , <code>pvx voc</code> , benchmarks
Transform and backend research	40, 41, 62	<code>pvx voc</code>
Live-style piping workflows	17, 46, 59	<code>pvx voc</code> + downstream tools

11.3 Slow Down Speech

The next example makes the role of the slow down speech explicit.

Command

```
pvx voc speech.wav --preset vocal --stretch 1.35 --output speech_slow.wav
```

Explanation - Uses vocal preset and moderate stretch to increase intelligibility review time.

Before/After - Before: normal speech rate. - After: slower phrase timing, mostly same pitch.

Parameters that matter most - `--stretch` - `--preset vocal`

Artifacts to listen for - consonant smearing - subtle flutter in sustained vowels

11.4 Time-Compress Speech (Faster Playback)

The next example makes the role of the time-compress speech (faster playback) explicit.

Command

```
pvx voc speech.wav --preset vocal --stretch 0.85 --output speech_fast.wav
```

Explanation - Compresses duration while trying to preserve speech quality.

Before/After - Before: original pace. - After: shorter/faster delivery.

Parameters that matter most - `--stretch < 1` - `--phase-locking identity`

Artifacts to listen for - transient roughness on plosives - “grainy” tails on fricatives

11.5 Raise Pitch Without Changing Speed

A closer look at the raise pitch without changing speed clarifies the choices involved.

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav
```

Explanation - Raises pitch by 3 semitones with duration unchanged.

Before/After - Before: original key center. - After: same timing, higher pitch.

Parameters that matter most - `--pitch` - `--stretch 1.0`

Artifacts to listen for - timbral thinning - phasey upper harmonics at larger shifts

11.6 Lower Pitch With Formant Preservation

A closer look at the lower pitch with formant preservation clarifies the choices involved.

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output vocal_down4_formant.wav
```

Explanation - Applies downward pitch shift while compensating formant envelope drift.

Before/After - Before: natural vocal size. - After: lower notes with less “giant voice” coloration.

Parameters that matter most - --pitch - --pitch-mode formant-preserving - --formant-lifter

Artifacts to listen for - residual “hollow” vowels - over-correction if formant settings are aggressive

11.7 Stretch + Preserve Formants

The next example makes the role of the stretch + preserve formants explicit.

Command

```
pvx voc vocal.wav --stretch 1.2 --pitch -2 --pitch-mode formant-preserving --output vocal_stretch_formant.wav
```

Explanation - Combined timing and pitch modification tuned for voice.

Before/After - Before: original timing and key. - After: slower and lower while retaining more vowel identity.

Parameters that matter most - --stretch - --pitch - --pitch-mode

Artifacts to listen for - transient blur at phrase starts - comb-like coloration on loud sustained notes

11.8 Extreme Time Stretch Ambient Pad

A closer look at the extreme time stretch ambient pad clarifies the choices involved.

Command

```
pvx voc one_shot.wav --preset ambient --target-duration 600 --output one_shot_10min.wav
```

Explanation - Uses ambient preset for very large stretch ratios.

Before/After - Before: short transient-rich source. - After: long evolving drone texture.

Parameters that matter most - --preset ambient - --target-duration - --n-fft, --hop-size

Artifacts to listen for - low-level flutter on bright components - transient residue if source is very percussive

11.9 Freeze a Harmonic Moment

The next example makes the role of the freeze a harmonic moment explicit.

Command

```
pvx freeze guitar_chord.wav --freeze-time 0.45 --duration 12 --output-dir out --suffix _freeze
```

Explanation - Captures one spectral slice and resynthesizes sustained texture.

Before/After - Before: normal decay envelope. - After: held spectral “snapshot.”

Parameters that matter most - --freeze-time - --duration

Artifacts to listen for - static texture if freeze point is too narrow-band - periodic shimmer if random phase is enabled heavily

11.10 Retune Vocal to A440

The material below develops the retune vocal to a440 in practical terms.

Command

```
pvx voc vocal_note.wav --target-f0 440 --pitch-mode formant-preserving --output vocal_A440.wav
```

Explanation - Estimates source F0 and applies a global ratio so the tracked center lands at A4 = 440 Hz.

Before/After - Before: note center may sit sharp/flat vs A440 reference. - After: center frequency aligns to 440 Hz.

Parameters that matter most - --target-f0 - --pitch-mode

Artifacts to listen for - metallic edges on strong consonants - formant drift if --pitch-mode standard is used

11.11 Morph Two Sounds

The next example makes the role of the morph two sounds explicit.

Command

```
pvx morph source_a.wav source_b.wav --alpha 0.35 --blend-mode linear --output morph_35.wav --overwrite
```

Explanation - Interpolates spectral content between A and B (linear mode).

Before/After - Before: two distinct sources. - After: hybrid timbre weighted toward A at alpha=0.35.

Parameters that matter most - --alpha - --blend-mode - --phase-mix (optional phase control)

Artifacts to listen for - phasing if sources differ strongly in timing - hollow midrange from spectral mismatch

Cross-synthesis variants

```
# Envelope transfer: carrier A with modulator B spectral envelope
\index{Envelope transfer carrier A with modulator B spectral envelope}
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_envelope_b --alpha 0.8 --envelope-lifter 36 --
  output morph_env_ab.wav --overwrite

# Spectral-mask transfer: carrier A masked by modulator B
\index{Spectral-mask transfer carrier A masked by modulator B}
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_mask_b --alpha 0.7 --mask-exponent 1.2 --output
  morph_mask_ab.wav --overwrite

# Magnitude/phase exchange style
\index{Magnitude/phase exchange style}
pvx morph source_a.wav source_b.wav --blend-mode magnitude_b_phase_a --alpha 0.65 --phase-mix 0.1 --output
  morph_magB_phaseA.wav --overwrite
```

True A->B trajectory morph (time-varying alpha in one command)

```
pvx morph source_a.wav source_b.wav --alpha controls/alpha_curve.csv --interp linear --blend-mode linear --
output morph_a_to_b.wav --overwrite
```

Example controls/alpha_curve.csv:

```
time_sec,value
0.0,0.0
1.5,0.35
3.0,0.70
4.5,1.0
```

Notes: - --alpha accepts scalar or control file (.csv/.json). - --interp and --order control interpolation for trajectory files. - This produces a genuine frame-wise morph progression from A toward B over output time.

11.12 Create Chorus/Unison Thickness

A closer look at the create chorus/unison thickness clarifies the choices involved.

Command

```
pvx unison synth.wav --voices 7 --detune-cents 18 --width 1.2 --dry-mix 0.15 --output-dir out --suffix _uni7
```

Explanation - Generates detuned multi-voice spread.

Before/After - Before: narrow mono/stereo lead. - After: wider, denser ensemble-like body.

Parameters that matter most - --voices - --detune-cents - --width

Artifacts to listen for - excessive beating at large detune - low-end smearing if source is bass-heavy

11.13 De-Noise Field Recording

The entries in this section organize the de-noise field recording for comparison and practical use.

Command

```
pvx denoise field.wav --noise-seconds 0.5 --reduction-db 10 --smooth 7 --output-dir out --suffix _den
```

Explanation - Builds a noise estimate from the opening region and subtracts adaptively.

Before/After - Before: broadband hiss/air/noise floor. - After: lower noise floor with possible detail loss.

Parameters that matter most - --noise-seconds - --reduction-db - --smooth

Artifacts to listen for - musical noise - dullness in high frequencies

Useful variants

```
# Speech-safe denoise (preserve intelligibility and avoid over-subtraction)
\index{Speech-safe denoise (preserve intelligibility and avoid over-subtraction)}
pvx denoise speech.wav --noise-seconds 0.4 --reduction-db 5 --floor 0.2 --smooth 9 --output speech_clean.wav

# Music-safe denoise (retain more ambience/harmonic detail)
\index{Music-safe denoise (retain more ambience/harmonic detail)}
pvx denoise mix.wav --noise-seconds 0.3 --reduction-db 4 --floor 0.25 --smooth 7 --output mix_clean.wav

# Denoise before stretching
\index{Denoise before stretching}
pvx denoise noisy.wav --reduction-db 6 --stdout | pvx voc - --stretch 2.0 --output noisy_clean_stretch.wav
```

Variant notes - Lower `--reduction-db` and higher `--smooth` generally reduce metallic residual artifacts.
 - `--noise-file` is recommended when the file start is not representative noise.

11.14 De-Reverb a Room Recording

The entries in this section organize the de-reverb a room recording for comparison and practical use.

Command

```
pvx deverb room_voice.wav --strength 0.55 --decay 0.90 --floor 0.12 --output-dir out --suffix _dry
```

Explanation - Suppresses late reverberant tails in spectral domain.

Before/After - Before: long room decay and reduced articulation. - After: drier presentation and clearer direct sound.

Parameters that matter most - `--strength` - `--decay` - `--floor`

Artifacts to listen for - pumping on sustained vowels - “underwater” tails if too aggressive

11.15 Segment-Based Dynamic Stretch via CSV

A closer look at the segment-based dynamic stretch via CSV clarifies the choices involved.

Command

```
pvx voc phrase.wav --pitch-map map_warp.csv --output phrase_map.wav
```

Example map_warp.csv

```
start_sec,end_sec,stretch
0.0,0.8,1.00
0.8,1.6,1.25
1.6,2.2,0.90
2.2,3.0,1.10
```

Explanation - Applies piecewise timing control across segments.

Before/After - Before: fixed timing. - After: phrase-level temporal reshaping.

Parameters that matter most - CSV `stretch` - segment boundaries - `--pitch-map-crossfade-ms`

Artifacts to listen for - boundary clicks if crossfade is too short - unnatural tempo discontinuities

11.16 Sidechain Pitch Trajectory (A Controls B)

The next example makes the role of the sidechain pitch trajectory (a controls b) explicit.

Command

```
pvx follow A.wav B.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output B_follow.wav
```

Explanation - Runs the full sidechain flow in one command: track A, build control map, apply to B.

Before/After - Before: B has its own pitch motion. - After: B follows A’s timing contour where confidence is sufficient.

Parameters that matter most - `--backend` and tracker frame settings - `--pitch-conf-min` - `--pitch-lowconf-mode`

Artifacts to listen for - warbling from noisy tracker segments - abrupt ratio jumps at low-confidence transitions

Advanced variant (feature vector sidechain with MFCC + MPEG-7-style flux)

```
pvx pitch-track A.wav --feature-set all --mfcc-count 13 --output - \  
| pvx voc B.wav --control-stdin \  
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \  
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \  
  --route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \  
  --route stretch=clip(stretch,0.85,1.6) \  
  --output B_feature_follow.wav
```

The feature sidechain recipes provide more routing examples and complex control combinations.
Built-in pvx follow recipe printer:

```
pvx follow --example  
pvx follow --example all  
pvx follow --example noise_aware
```

11.17 GPU Acceleration

The material below develops the GPU acceleration in practical terms.

Command

```
pvx voc mix.wav --gpu --stretch 1.12 --output mix_gpu.wav
```

Explanation - Uses CUDA path if available (--gpu alias for --device cuda).

Before/After - Before: CPU render time baseline. - After: same algorithmic output path with potentially lower render time.

Parameters that matter most - --device / --gpu - FFT size and hop

Artifacts to listen for - normally identical class of artifacts to CPU; compare for parity in edge cases

11.18 Batch Processing Folder

A closer look at the batch processing folder clarifies the choices involved.

Command

```
pvx voc stems/*.wav --preset vocal --stretch 1.05 --output-dir out/stems --overwrite
```

Explanation - Applies consistent processing across many files.

Before/After - Before: original folder material. - After: each file rendered with _pv suffix in target folder.

Parameters that matter most - wildcard expansion - --output-dir - --overwrite

Artifacts to listen for - per-file content mismatch under one global setting

11.19 Pipe Multiple Tools in One Line

The next example makes the role of the pipe multiple tools in one line explicit.

Command


```
pvx voc input.wav --stretch 1.15 --stdout \
| pvx denoise - --reduction-db 8 --stdout \
| pvx deverb - --strength 0.4 --stdout > cleaned.wav
```

Explanation - Builds a linear chain without intermediate files.

Before/After - Before: raw source. - After: stretched + denoised + dereverberated result.

Parameters that matter most - each stage intensity - pipe ordering

Artifacts to listen for - cumulative over-processing - clipped peaks if mastering limits are omitted

11.20 Auto Profile + Auto Transform (Planning)

The next example makes the role of the auto profile + auto transform (planning) explicit.

Command

```
pvx voc input.wav --auto-profile --auto-transform --explain-plan
```

Explanation - Prints selected processing plan JSON without rendering audio.

Before/After - Before: no run metadata. - After: explicit resolved settings.

Parameters that matter most - --auto-profile-lookahead-seconds

Artifacts to listen for - n/a (plan-only mode)

11.21 Multi-Resolution Fusion Render

A closer look at the multi-resolution fusion render clarifies the choices involved.

Command

```
pvx voc input.wav --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --
stretch 1.25 --output input_multires.wav
```

Explanation - Blends multiple FFT scales to reduce single-resolution bias.

Before/After - Before: single-resolution render. - After: potentially smoother balance between transient sharpness and tonal stability.

Parameters that matter most - --multires-ffts - --multires-weights

Artifacts to listen for - slight chorus/phasing if fusion weights over-emphasize conflicting scales

11.22 Checkpoint + Resume Long Render

A closer look at the checkpoint + resume long render clarifies the choices involved.

Command (first pass)

```
pvx voc long_source.wav --preset extreme --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --manifest-
json reports/run_manifest.json --output long_pass1.wav
```

Command (resume)

```
pvx voc long_source.wav --preset extreme --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --resume --manifest-json reports/run_manifest.json --manifest-append --output long_pass2.wav
```

Explanation - Stores per-segment chunks and reuses completed ones.

Before/After - Before: interrupted run loses progress. - After: restart reuses cached segments.

Parameters that matter most - --auto-segment-seconds - --checkpoint-dir - --resume

Artifacts to listen for - segment boundary smoothness (tune crossfade)

11.23 Harmonize Triad Stack

The material below develops the harmonize triad stack in practical terms.

Command

```
pvx harmonize lead.wav --intervals 0,4,7 --intervals-cents 0,4,-3 --gains 1.0,0.85,0.78 --pans -0.35,0.0,0.35 --force-stereo --output-dir out --suffix _triad
```

Explanation - Builds triad with subtle micro-detune offsets.

Before/After - Before: single voice. - After: harmonized stack with stereo spread.

Parameters that matter most - --intervals - --intervals-cents - --pans

Artifacts to listen for - beating and clutter if gains are too high

11.24 Timeline Warp with pvxwarp

The material below develops the timeline warp with pvxwarp in practical terms.

Command

```
pvx warp drums.wav --map map_warp.csv --crossfade-ms 10 --output-dir out --suffix _warp
```

Explanation - Applies time-only map to drive phrase-level groove edits.

Before/After - Before: fixed groove. - After: section-dependent push/pull timing.

Parameters that matter most - map values - --crossfade-ms

Artifacts to listen for - timing discontinuities at map boundaries

11.25 Layer Harmonic and Percussive Paths

The next example makes the role of the layer harmonic and percussive paths explicit.

Command

```
pvx layer full_mix.wav \  
  --harmonic-stretch 1.15 --harmonic-pitch-semitones 2 --harmonic-gain 0.95 \  
  --percussive-stretch 1.00 --percussive-pitch-semitones 0 --percussive-gain 1.05 \  
  --output-dir out --suffix _layered
```

Explanation - Separates harmonic/percussive content and processes each path differently.

Before/After - Before: one global process for all content. - After: tonal and transient content treated independently.

Parameters that matter most - harmonic/percussive stretch and gain - HPSS kernels

Artifacts to listen for - separation bleed - phase cancellation between paths if gains are extreme

11.26 JSON Manifest + Automation-Friendly Logging

The next example makes the role of the JSON manifest + automation-friendly logging explicit.

Command

```
pvx voc input.wav --stretch 1.08 --pitch -1 \
  --manifest-json reports/pvx_runs.json --manifest-append \
  --output out/input_take1.wav
```

Explanation - Writes machine-readable run metadata for CI jobs, dataset pipelines, or batch monitoring.

Before/After - Before: render exists but settings/provenance are not captured. - After: output audio plus JSON metadata with resolved parameters and timing.

Parameters that matter most - `--manifest-json` - `--manifest-append` - stable output naming convention

Artifacts to listen for - not audio-specific; validate manifest integrity and path consistency

11.27 Speech Natural Stretch (Hybrid Transients)

A closer look at the speech natural stretch (hybrid transients) clarifies the choices involved.

Command

```
pvx voc speech.wav --preset vocal_studio --transient-mode hybrid --stretch 1.25 --output speech_natural.wav
```

Explanation - Uses speech-focused preset plus hybrid transient handling to reduce consonant smear.

Before/After - Before: original speech rate. - After: slower pacing with cleaner consonants vs pure PV.

Parameters that matter most - `--preset vocal_studio` - `--transient-mode hybrid` - `--transient-sensitivity`

Artifacts to listen for - residual metallic tone on strong fricatives

11.28 Drums Safe Stretch

The next example makes the role of the drums safe stretch explicit.

Command

```
pvx voc drums.wav --preset drums_safe --time-stretch 1.4 --output drums_safe.wav
```

Explanation - Uses WSOLA-forward transient handling for percussive attacks.

Before/After - Before: tight attack transients. - After: longer groove with fewer softened hits than basic PV.

Parameters that matter most - `--preset drums_safe` - `--transient-mode wsola`

Artifacts to listen for - repetitive texture on sustained cymbal tails

11.29 Stereo Coherent Stretch

The material below develops the stereo coherent stretch in practical terms.

Command

```
pvx voc wide_mix.wav --preset stereo_coherent --stretch 1.2 --output wide_mix_coherent.wav
```

Explanation - Uses mid/side coherence coupling to reduce image wobble.

Before/After - Before: original stereo image. - After: stretched image with more stable center and side phase relation.

Parameters that matter most - `--stereo-mode` - `--coherence-strength`

Artifacts to listen for - slight narrowing if coherence is set too high

11.30 Extreme Ambient Stretch (10-minute target)

A closer look at the extreme ambient stretch (10-minute target) clarifies the choices involved.

Command

```
pvx voc one_shot.wav --preset extreme_ambient --target-duration 600 --output one_shot_ambient_10min.wav
```

Explanation - Long-form multistage stretch preset with transient-aware blending.

Before/After - Before: short source event. - After: long ambient texture.

Parameters that matter most - `--preset extreme_ambient` - `--target-duration` - `--max-stage-stretch`

Artifacts to listen for - low-level swirl if source is very broadband

11.31 Pitch Shift with Formant Preservation

A closer look at the pitch shift with formant preservation clarifies the choices involved.

Command

```
pvx voc vocal.wav --stretch 1.0 --pitch -4 --pitch-mode formant-preserving --output vocal_down4_formant.wav
```

Explanation - Separates pitch movement from formant envelope correction.

Before/After - Before: original pitch center. - After: lower notes with better vowel identity retention.

Parameters that matter most - `--pitch` - `--pitch-mode formant-preserving` - `--formant-strength`

Artifacts to listen for - hollow timbre if over-corrected

11.32 Hybrid Transient Mode (Manual Tuning)

The next example makes the role of the hybrid transient mode (manual tuning) explicit.

Command

```
pvx voc source.wav \  
  --transient-mode hybrid \  
  --transient-sensitivity 0.65 \  
  --transient-protect-ms 26 \  
  --transient-crossfade-ms 8 \  
  --time-stretch 1.3 \  
  --output source_hybrid_tuned.wav
```

Explanation - Explicitly tunes detector and stitch behavior instead of relying on preset defaults.

Before/After - Before: single global stretch strategy. - After: transient windows handled differently from steady-state regions.

Parameters that matter most - `--transient-sensitivity` - `--transient-protect-ms` - `--transient-crossfade-ms`

Artifacts to listen for - boundary roughness if crossfade is too short

11.33 Benchmark Command (pvx vs Rubber Band vs librosa)

This reference introduces the benchmark command (pvx vs rubber band vs librosa) before presenting its individual entries.

Command

```
python3 benchmarks/run_bench.py --quick --out-dir benchmarks/out --baseline benchmarks/baseline_small.json --gate
```

Explanation - Runs cycle-consistency benchmark and fails if pvx regresses against baseline.

Before/After - Before: no objective quality snapshot. - After: JSON/Markdown benchmark report plus regression verdict.

Parameters that matter most - `--quick` - `--baseline` - `--gate`

Artifacts to listen for - n/a (objective benchmark command)

11.34 Transient Reset Mode for Cleaner Attacks

The material below develops the transient reset mode for cleaner attacks in practical terms.

Command

```
pvx voc drums.wav --stretch 1.22 --transient-mode reset --transient-sensitivity 0.62 --output drums_reset.wav
```

Explanation - Applies phase resets around detected onsets while leaving steady regions in PV flow.

Before/After - Before: transient-rich grooves may soften at larger stretches. - After: snare/kick attack definition is usually sharper.

Parameters that matter most - `--transient-mode reset` - `--transient-sensitivity`

Artifacts to listen for - onset clicks if sensitivity is too high

11.35 WSOLA-Only Transient Handling

A closer look at the wsola-only transient handling clarifies the choices involved.

Command

```
pvx voc percussion.wav --stretch 1.35 --transient-mode wsola --transient-protect-ms 35 --output percussion_wsola.wav
```

Explanation - Prioritizes time-domain overlap/similarity handling for transient regions.

Before/After - Before: attack smearing in strictly spectral paths. - After: stronger transient solidity with different texture in sustains.

Parameters that matter most - `--transient-mode wsola` - `--transient-protect-ms`

Artifacts to listen for - grain boundaries if crossfade/protect windows are mismatched

11.36 Mid/Side Stereo Coherence Stretch

The material below develops the mid/side stereo coherence stretch in practical terms.

Command

```
pvx voc stereo_mix.wav --stretch 1.18 --stereo-mode mid_side_lock --coherence-strength 0.9 --output stereo_ms_lock.wav
```

Explanation - Preserves stereo image geometry by constraining M/S phase behavior.

Before/After - Before: image may wobble on sustained sections. - After: center and width usually feel more stable.

Parameters that matter most - `--stereo-mode mid_side_lock` - `--coherence-strength`

Artifacts to listen for - over-constrained width if coherence is set too high

11.37 Reference-Channel Lock for Multichannel Content

This reference introduces the reference-channel lock for multichannel content before presenting its individual entries.

Command

```
pvx voc multichannel.wav --stretch 1.12 --stereo-mode ref_channel_lock --ref-channel 0 --coherence-strength 0.85 --output multichannel_ref_lock.wav
```

Explanation - Uses one channel as phase anchor to reduce inter-channel decorrelation.

Before/After - Before: surround image drift on strong harmonics. - After: tighter channel relationship across processing.

Parameters that matter most - `--stereo-mode ref_channel_lock` - `--ref-channel` - `--coherence-strength`

Artifacts to listen for - channel dominance bias if reference channel is atypical

11.38 Irrational Pitch Ratio Input

The next example makes the role of the irrational pitch ratio input explicit.

Command

```
pvx voc tone.wav --stretch 1.0 --pitch-ratio "2^(1/12)" --output tone_up_1semitone_expr.wav
```

Explanation - Uses expression parsing for irrational ratio input directly on CLI.

Before/After - Before: original fundamental. - After: +1 equal-tempered semitone.

Parameters that matter most - `--pitch-ratio` expression - `--stretch 1.0`

Artifacts to listen for - none specific for small shifts; monitor formant drift on voice

11.39 Just-Ratio Pitch Input

A closer look at the just-ratio pitch input clarifies the choices involved.

Command

```
pvx voc tone.wav --stretch 1.0 --pitch-ratio 3/2 --output tone_perfect_fifth.wav
```

Explanation - Uses just-intonation ratio syntax for exact rational interval control.

Before/After - Before: root pitch. - After: perfect fifth above (3:2).

Parameters that matter most - --pitch-ratio 3/2

Artifacts to listen for - beating vs reference if mixed with equal-tempered material

11.40 N-TET CSV Map (19-TET) with pvxconform

A closer look at the n-tet CSV map (19-tet) with pvxconform clarifies the choices involved.

Command

```
pvx conform lead.wav --map maps/map_19tet.csv --output-dir out --suffix _19tet
```

Explanation - Applies segmentwise pitch/time map from CSV using 19-tone equal temperament ratios.

Before/After - Before: 12-TET phrases. - After: 19-TET trajectory according to map.

Parameters that matter most - --map - CSV pitch_ratio values

Artifacts to listen for - boundary roughness if map segments are too short

11.41 Just-Intonation CSV Conform Map

The material below develops the just-intonation CSV conform map in practical terms.

Command

```
pvx conform choir.wav --map maps/map_just_intonation.csv --output-dir out --suffix _ji
```

Explanation - Retunes with segment ratios such as 5/4, 6/5, 7/4, 9/8.

Before/After - Before: equal-tempered intonation center. - After: just-intonation harmonic color.

Parameters that matter most - map segment durations - map pitch_ratio

Artifacts to listen for - abrupt color shifts between unrelated just ratios

11.42 Transform A/B: FFT vs DCT

A closer look at the transform a/b: FFT vs dct clarifies the choices involved.

Command

```
pvx voc source.wav --stretch 1.08 --transform fft --output fft_out.wav
pvx voc source.wav --stretch 1.08 --transform dct --output dct_out.wav
```

Explanation - Compares complex-phase Fourier path against real cosine basis path.

Before/After - Before: one baseline source. - After: two different transform-character outputs.

Parameters that matter most - --transform - identical stretch/pitch settings for fair comparison

Artifacts to listen for - phase character and transient envelope differences

11.43 CZT for Non-Power-of-Two Frame Setup

A closer look at the czt for non-power-of-two frame setup clarifies the choices involved.

Command

```
pvx voc source.wav --transform czt --n-fft 1536 --win-length 1536 --hop-size 384 --stretch 1.1 --output czt_1536.wav
```

Explanation - Uses chirp-z backend for a non-power-of-two frame strategy.

Before/After - Before: default FFT path. - After: alternate numerical behavior for same macro task.

Parameters that matter most - --transform czt - --n-fft, --win-length, --hop-size

Artifacts to listen for - subtle high-frequency texture differences vs FFT

11.44 Full Mastering Chain in One Render

A closer look at the full mastering chain in one render clarifies the choices involved.

Command

```
pvx voc mix.wav --stretch 1.03 --target-lufs -14 --compressor-threshold-db -18 --compressor-ratio 2.2 --limiter-threshold -0.9 --soft-clip-level 0.96 --output mix_mastered.wav
```

Explanation - Demonstrates integrated DSP + mastering path for one-step render.

Before/After - Before: un-leveled output with higher crest variance. - After: controlled loudness with safety limiting/clipping.

Parameters that matter most - --target-lufs - compressor and limiter thresholds - --soft-clip-level

Artifacts to listen for - pumping, flattened transients, clip harshness

11.45 Safety Limiter + Hard Clip Guard

The material below develops the safety limiter + hard clip guard in practical terms.

Command

```
pvx voc loud_source.wav --stretch 1.0 --limiter-threshold -1.0 --hard-clip-level 0.98 --output loud_safe.wav
```

Explanation - Adds a limiter stage followed by a hard guard ceiling.

Before/After - Before: possible overs and intersample spikes. - After: bounded peaks with predictable ceiling.

Parameters that matter most - --limiter-threshold - --hard-clip-level

Artifacts to listen for - transient flattening if limiter is overworked

11.46 Plan-Only Diagnostic (No Render)

The next example makes the role of the plan-only diagnostic (no render) explicit.

Command


```
pvx voc source.wav --auto-profile --auto-transform --manifest-json plan.json --dry-run --explain-plan
```

Explanation - Produces resolved strategy without writing output audio.

Before/After - Before: no visibility into automatic decision path. - After: JSON and console explanation of chosen settings.

Parameters that matter most - --auto-profile - --auto-transform - --dry-run

Artifacts to listen for - n/a (planning command)

11.47 Checkpointed Long Render Workflow

A closer look at the checkpointed long render workflow clarifies the choices involved.

Command

```
pvx voc long_source.wav --target-duration 1200 --auto-segment-seconds 0.5 --checkpoint-dir .pvx_ckpt --output long_out.wav
pvx voc long_source.wav --target-duration 1200 --auto-segment-seconds 0.5 --checkpoint-dir .pvx_ckpt --resume --output long_out.wav
```

Explanation - First command starts segmented checkpointed render; second resumes if interrupted.

Before/After - Before: full rerender after interruption. - After: resumed render from saved chunk state.

Parameters that matter most - --checkpoint-dir - --auto-segment-seconds - --resume

Artifacts to listen for - chunk-boundary discontinuities if segmenting is too coarse

11.48 Multi-Tool Pipe with Streaming I/O

The next example makes the role of the multi-tool pipe with streaming I/O explicit.

Command

```
pvx voc input.wav --stretch 1.1 --stdout \
| pvx denoise - --reduction-db 8 --stdout \
| pvx deverb - --strength 0.35 --output cleaned_chain.wav
```

Explanation - Streams PCM data through multiple tools in one line.

Before/After - Before: separate intermediate files. - After: one-pass chained processing.

Parameters that matter most - upstream --stdout - downstream - stdin input

Artifacts to listen for - cumulative artifact stacking from multiple stages

11.49 Batch Tree Processing with find + xargs

A closer look at the batch tree processing with find + xargs clarifies the choices involved.

Command

```
find sessions -name "*.wav" -print0 | xargs -0 -I{} pvx voc "{}" --stretch 1.05 --output-dir renders --suffix _x105 --overwrite
```

Explanation - Applies a consistent transform to an entire tree of WAV files.

Before/After - Before: manual per-file invocation. - After: reproducible batch render set.

Parameters that matter most - stable suffix conventions - overwrite policy

Artifacts to listen for - content classes that need per-file presets instead of one global recipe

11.50 Vocal Retune by Scale/Root (pvxretune)

A closer look at the vocal retune by scale/root (pvxretune) clarifies the choices involved.

Command

```
pvx retune vocal.wav --scale minor --root A --a4-reference-hz 432 --f0-min 70 --f0-max 1000 --output-dir out
--suffix _retune_amin

# Optional: anchor to an explicit root fundamental instead of note class
\index{Optional anchor to an explicit root fundamental instead of note class}
pvx retune vocal.wav --scale major --root-hz 261.6256 --output vocal_root_c4.wav

# Optional: ask pvx to estimate and use a root fundamental from the input
\index{Optional ask pvx to estimate and use a root fundamental from the input}
pvx retune vocal.wav --scale minor --recommend-root --output vocal_auto_root.wav
```

Explanation - Tracks monophonic F0 and snaps segments to requested tonal set.

Before/After - Before: natural pitch drift. - After: scale-constrained melody line.

Parameters that matter most - --scale, --root - --root-hz, --recommend-root - --a4-reference-hz - --f0-min, --f0-max

Artifacts to listen for - octave errors on noisy consonants

11.51 Harmonize Then Widen in One Pipeline

The material below develops the harmonize then widen in one pipeline in practical terms.

Command

```
pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.75,0.65 --stdout \
| pvx unison - --voices 5 --detune-cents 10 --width 1.0 --output harmonized_wide.wav
```

Explanation - Builds harmony stack first, then adds unison detuned width.

Before/After - Before: single melodic line. - After: harmonized and spatially widened texture.

Parameters that matter most - harmonic intervals/gains - unison voice count and detune

Artifacts to listen for - dense masking in low-mid region

11.52 Ambient Macro-Chain (Stretch + Cleanup + Loudness)

The next example makes the role of the ambient macro-chain (stretch + cleanup + loudness) explicit.

Command

```
pvx voc seed.wav --preset extreme_ambient --target-duration 600 --stdout \
| pvx denoise - --reduction-db 4 --smooth 9 --stdout \
| pvx deverb - --strength 0.25 --stdout \
| pvx voc - --stretch 1.0 --target-lufs -18 --output ambient_final.wav
```

Explanation - Demonstrates long-form ambient generation plus light restoration and final loudness targeting.

Before/After - Before: short seed sample. - After: long-form ambient render with controlled output level.

Parameters that matter most - ambient preset stretch controls - cleanup strength values - final --target-lufs

Artifacts to listen for - accumulated haze from over-processing

11.53 Broadcast-Style Speech Finishing (Stretch + Loudness + Safety)

A closer look at the broadcast-style speech finishing (stretch + loudness + safety) clarifies the choices involved.

Command

```
pvx voc narration.wav --preset vocal_studio --stretch 1.08 --target-lufs -16 --limiter-threshold -1.0 --soft
  -clip-level 0.98 --hard-clip-level 0.995 --output narration_finished.wav
```

Explanation - Applies subtle timing expansion and a conservative mastering finish in one pass.

Before/After - Before: untreated narration. - After: slightly slower pacing with controlled final loudness and peaks.

Parameters that matter most - --stretch - --target-lufs - limiter/clip safety thresholds

Artifacts to listen for - pumping from aggressive limiting - clipped consonant edges if clip levels are too low

11.54 Extreme Stretch with Checkpointed Recovery

The next example makes the role of the extreme stretch with checkpointed recovery explicit.

Command

```
pvx voc short_source.wav --preset extreme_ambient --target-duration 900 --auto-segment-seconds 0.25 --
  checkpoint-dir checkpoints/short_source --output short_source_15min.wav
```

Explanation - Runs a very large render with small recoverable segments and persistent checkpoints.

Before/After - Before: short source gesture. - After: long ambient evolution with resumable rendering.

Parameters that matter most - --target-duration - --auto-segment-seconds - --checkpoint-dir

Artifacts to listen for - timbral drift from repeated stage boundaries

11.55 Room Recording Rehab (De-Reverb Then Stretch)

The entries in this section organize the room recording rehab (de-reverb then stretch) for comparison and practical use.

Command

```
pvx deverb room_take.wav --strength 0.45 --stdout \
| pvx voc --preset vocal_studio --stretch 1.15 --output room_take_rehab.wav
```

Explanation - Reduces reverberant smear first, then performs moderate timing stretch on cleaner material.

Before/After - Before: dense room reflections. - After: clearer direct signal with slower timing.

Parameters that matter most - pvx deverb --strength - pvx voc --stretch

Artifacts to listen for - metallic tails from over-dereverb - new blur if stretch is too large

11.56 Scale-Quantized Retune Followed by Unison Widening

The material below develops the scale-quantized retune followed by unison widening in practical terms.

Command

```
pvx retune lead.wav --scale major --root D --output-dir out --suffix _retuned \  
&& pvx unison out/lead_retuned.wav --voices 5 --detune-cents 9 --width 0.9 --output-dir out --suffix  
_retuned_unison
```

Explanation - First constrains melody to key, then adds controlled chorus/unison width.

Before/After - Before: dry monophonic lead. - After: tuned and widened lead texture.

Parameters that matter most - retune scale/root - unison voices and detune

Artifacts to listen for - tuning jumps on weak F0 regions - phasey image when width/detune are too high

11.57 Morph Then Freeze for Hybrid Pads

The material below develops the morph then freeze for hybrid pads in practical terms.

Command

```
pvx morph bells.wav choir.wav --alpha 0.5 --output bells_choir_morph.wav \  
&& pvx freeze bells_choir_morph.wav --freeze-time 0.8 --duration 20 --output-dir out --suffix _frozen
```

Explanation - Builds a hybrid timbre first, then captures a stable harmonic moment as a sustained pad.

Before/After - Before: two independent sources. - After: one blended frozen harmonic texture.

Parameters that matter most - morph --alpha - freeze capture time and duration

Artifacts to listen for - static texture if freeze time is poorly chosen

11.58 CUDA Attempt with Deterministic CPU Fallback Workflow

The next example makes the role of the cuda attempt with deterministic CPU fallback workflow explicit.

Command

```
pvx voc mix.wav --device cuda --stretch 1.12 --output mix_cuda.wav || pvx voc mix.wav --device cpu --stretch  
1.12 --output mix_cpu.wav
```

Explanation - Tries GPU first, then falls back to CPU using the same core settings.

Before/After - Before: original mix timing. - After: consistent stretched output regardless of device path.

Parameters that matter most - --device - stretch/quality controls shared between both paths

Artifacts to listen for - no expected quality change between CPU and CUDA modes

11.59 5.1/Multichannel Reference-Lock Stretch

This reference introduces the 5.1/multichannel reference-lock stretch before presenting its individual entries.

Command

```
pvx voc surround.wav --stretch 1.1 --stereo-mode ref_channel_lock --ref-channel 2 --coherence-strength 0.95
--output surround_lock.wav
```

Explanation - Anchors phase evolution to a chosen reference channel to reduce inter-channel drift.

Before/After - Before: original multichannel phase relationships. - After: stretched output with better preserved channel coupling.

Parameters that matter most - --stereo-mode ref_channel_lock - --ref-channel - --coherence-strength

Artifacts to listen for - narrowed width if coherence is overly strong for decorrelated ambience

11.60 Mid/Side Lock with Formant-Preserving Pitch Shift

The next example makes the role of the mid/side lock with formant-preserving pitch shift explicit.

Command

```
pvx voc stereo_vocal.wav --stretch 1.0 --pitch 2 --pitch-mode formant-preserving --stereo-mode mid_side_lock
--coherence-strength 0.9 --output stereo_vocal_up2_lock.wav
```

Explanation - Combines vocal formant handling with stereo image preservation.

Before/After - Before: natural stereo vocal. - After: upshifted vocal with tighter image stability.

Parameters that matter most - --pitch, --pitch-mode - --stereo-mode, --coherence-strength

Artifacts to listen for - center blur if coherence strength is too low

11.61 Confidence-Gated External Pitch Control via Pipe

The next example makes the role of the confidence-gated external pitch control via pipe explicit.

Command

```
pvx pitch-track A.wav --output - | pvx voc B.wav --control-stdin --pitch-conf-min 0.75 --pitch-lowconf-mode
hold --pitch-map-crossfade-ms 20 --output B_pitch_follow.wav
```

Explanation - Tracks pitch trajectory from A.wav, then applies only high-confidence pitch control to B.wav.

Before/After - Before: independent B.wav pitch contour. - After: B.wav follows trusted sections of A.wav pitch shape.

Parameters that matter most - --pitch-conf-min - --pitch-lowconf-mode - --pitch-map-crossfade-ms

Artifacts to listen for - contour lag if crossfade is too long - pitch jitter if confidence threshold is too low

11.62 Microtonal Just-Ratio Map Playback

The next example makes the role of the microtonal just-ratio map playback explicit.

Command

```
pvx voc mono_line.wav --pitch-map maps/just_ratios.csv --pitch-map-crossfade-ms 15 --output mono_line_ji.wav
```

Explanation - Applies time-varying just-ratio pitch targets from CSV.

Before/After - Before: fixed tuning. - After: trajectory-constrained microtonal tuning.

Parameters that matter most - --pitch-map - --pitch-map-crossfade-ms

Artifacts to listen for - step audibility if map density is too sparse

11.63 Speech Clarity Pass (Identity Lock + Conservative Window/Hop)

The material below develops the speech clarity pass (identity lock + conservative window/hop) in practical terms.

Command

```
pvx voc lecture.wav --stretch 1.18 --phase-locking identity --n-fft 2048 --hop-size 256 --output lecture_clear_stretch.wav
```

Explanation - Uses a quality-focused speech profile manually tuned for lower phase blur.

Before/After - Before: original lecture pace. - After: slower and clearer for note-taking.

Parameters that matter most - --phase-locking identity - --n-fft, --hop-size

Artifacts to listen for - mild transient smear if hop becomes too large

11.64 Transform Research A/B (FFT vs Hartley)

The next example makes the role of the transform research a/b (FFT vs hartley) explicit.

Command

```
pvx voc source.wav --stretch 1.1 --transform fft --output source_fft.wav \  
&& pvx voc source.wav --stretch 1.1 --transform hartley --output source_hartley.wav
```

Explanation - Produces matched renders to compare transform-domain behavior.

Before/After - Before: one reference source. - After: two algorithmic variants for blind listening tests.

Parameters that matter most - --transform - all other parameters held constant

Artifacts to listen for - fine-grain differences in high-frequency texture

11.65 Multi-Resolution Fusion With Explicit Weights

The material below develops the multi-resolution fusion with explicit weights in practical terms.

Command

```
pvx voc texture.wav --stretch 1.4 --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --output texture_multires_weighted.wav
```

Explanation - Fuses multiple FFT scales with explicit weighting rather than implicit defaults.

Before/After - Before: single-scale spectral processing. - After: blended resolution emphasizing low-frequency stability.

Parameters that matter most - --multires-ffts - --multires-weights

Artifacts to listen for - diffuse attacks if large-FFT weight is too dominant

11.66 Conservative Field-Restoration Chain

The next example makes the role of the conservative field-restoration chain explicit.

Command

```
pvx denoise field.wav --reduction-db 5 --smooth 7 --stdout \
| pvx deverb - --strength 0.3 --stdout \
| pvx voc - --stretch 1.0 --target-lufs -20 --limiter-threshold -1.5 --output field_restored.wav
```

Explanation - Applies light denoise + dereverb and ends with conservative loudness normalization.

Before/After - Before: noisy reverberant field capture. - After: cleaner and level-aligned output.

Parameters that matter most - denoise reduction amount - dereverb strength - final loudness/limiter settings

Artifacts to listen for - chirpy residuals from over-denoise

11.67 Reproducible Ambient Session With Manifest Logging

A closer look at the reproducible ambient session with manifest logging clarifies the choices involved.

Command

```
pvx voc seed.wav --preset extreme_ambient --target-duration 600 --manifest-json sessions/ambient_manifest.json --manifest-append --output ambient_take.wav
```

Explanation - Writes run metadata into a JSON manifest for repeatable, auditable experiments.

Before/After - Before: short seed sample. - After: long ambient render plus machine-readable run record.

Parameters that matter most - --manifest-json - --manifest-append - chosen preset/stretch strategy

Artifacts to listen for - broad smearing from overly aggressive stretch factors

11.68 Managed One-Line Chain (No Manual Pipe Wiring)

A closer look at the managed one-line chain (no manual pipe wiring) clarifies the choices involved.

Command

```
pvx chain source.wav --pipeline "voc --stretch 1.2 | formant --mode preserve --formant-shift-ratio 1.0" --output source_chain.wav
```

Explanation - Runs serial stages with managed intermediate files. - Useful when you want multi-stage processing without writing long shell pipelines.

Before/After - Before: single source file. - After: time-stretched + formant-preserved render in one managed command.

Parameters that matter most - --pipeline stage order and settings - per-stage quality controls (inside the pipeline string)

Artifacts to listen for - cumulative coloration when too many strong stages are chained

11.69 Chunked Stream Wrapper + Output Policy Controls

The material below develops the chunked stream wrapper + output policy controls in practical terms.

Command

```
pvx stream source.wav --output source_stream.wav --chunk-seconds 0.2 --time-stretch 2.0 --bit-depth 24 --dither tpdf --dither-seed 7 --metadata-policy sidecar
```

Explanation - Uses `pvx stream` stateful mode (default) for chunked long renders with continuity context.
- Adds deterministic output policy controls and metadata sidecar emission.

Before/After - Before: original source. - After: chunked long-form render with explicit output-depth/dither policy and sidecar metadata.

Parameters that matter most - `--chunk-seconds` - `--bit-depth` - `--dither`, `--dither-seed` - `--metadata-policy`

Artifacts to listen for - over-short chunks may increase boundary coloration

Legacy compatibility mode:

```
pvx stream source.wav --mode wrapper --output source_stream_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0
```

11.70 Dynamic Stretch via Direct CSV Flag Input

This reference introduces the dynamic stretch via direct CSV flag input before presenting its individual entries.

Command

```
pvx voc source.wav --stretch controls/stretch.csv --interp linear --output source_dyn_stretch.wav
```

Example controls/stretch.csv

```
time_sec,value
0.0,1.0
1.0,1.2
2.0,1.6
3.0,2.0
```

Explanation - Uses a time-varying control-rate signal directly on `--stretch` (no legacy `--pitch-map` wrapper needed). - Interpolates control points with `--interp linear` (default mode).

Before/After - Before: fixed global stretch ratio. - After: continuously changing stretch trajectory over time.

Parameters that matter most - `--stretch <csv/json>` - `--interp` - `--order` (if polynomial)

Artifacts to listen for - excessive local tempo curvature from over-aggressive control points - boundary coloration if your control points force rapid ratio changes

11.71 Dynamic Pitch Ratio via JSON + Polynomial Interpolation

The next example makes the role of the dynamic pitch ratio via JSON + polynomial interpolation explicit.

Command


```
pvx voc vocal.wav --pitch-shift-ratio controls/pitch_curve.json --interp polynomial --order 3 --output
vocal_dyn_pitch.wav
```

Example controls/pitch_curve.json

```
{
  "points": [
    {"time_sec": 0.0, "value": 1.0},
    {"time_sec": 0.8, "value": 1.122462048},
    {"time_sec": 1.6, "value": 1.334839854},
    {"time_sec": 2.4, "value": 1.0}
  ]
}
```

Explanation - Drives pitch from a JSON control-rate signal attached directly to `--pitch-shift-ratio`.
`--interp polynomial --order 3` fits a smooth polynomial trajectory through points. `--order` supports any integer ≥ 1 ; the effective polynomial degree is capped to `min(order, control_points-1)`.

Before/After - Before: static transposition. - After: continuous pitch contour over time.

Parameters that matter most - `--pitch-shift-ratio <csv/json>` - `--interp` - `--order`

Artifacts to listen for - over/undershoot from high-order polynomial fitting - formant drift if large excursions are used without formant-preserving mode

Interpolation order quick visual:

Mode/order	Graph
none	
linear	
cubic	
exponential	
s_curve	
smootherstep	
polynomial --order 1	
polynomial --order 2	
polynomial --order 3	
polynomial --order 5	

Related function charts for morphing and mastering:

Function family	Graph
Morph blend magnitude curves	
Mask exponent curves (<code>--mask-exponent</code>)	
Phase mix curve (<code>--phase-mix</code>)	
Dynamics transfer curves	
Soft clip transfer functions	

11.72 Stairstep (Sample-and-Hold) Control Mode

The material below develops the stairstep (sample-and-hold) control mode in practical terms.

Command

```
pvx voc phrase.wav --stretch controls/step_stretch.csv --interp none --output phrase_step_control.wav
```

Example controls/step_stretch.csv

```
start_sec,end_sec,value
0.0,0.5,1.0
0.5,1.0,1.25
1.0,1.5,0.9
1.5,2.0,1.1
```

Explanation - Uses staircase control (`--interp none`) with piecewise constant parameter regions. - Best when you explicitly want quantized timeline regions.

Before/After - Before: smooth or fixed trajectory. - After: discrete region-based time changes.

Parameters that matter most - `--interp none` - segment boundaries - control **value**

Artifacts to listen for - audible discontinuities if regions are too short or jumps are too large

11.73 Dynamic Analysis Resolution (N-FFT + Hop Size Maps)

A closer look at the dynamic analysis resolution (n-FFT + hop size maps) clarifies the choices involved.

Command

```
pvx voc source.wav --n-fft controls/nfft.csv --hop-size controls/hop.csv --stretch 1.2 --output
source_dyn_resolution.wav
```

Example controls/nfft.csv

```
time_sec,value
0.0,1024
2.0,2048
4.0,4096
```

Example controls/hop.csv

```
time_sec,value
0.0,128
2.0,256
4.0,512
```

Explanation - Time-varying spectral resolution can trade transient precision and tonal stability across song sections. - Useful for research workflows where you want section-dependent analysis behavior.

Before/After - Before: one fixed analysis resolution for the whole render. - After: section-adaptive STFT resolution.

Parameters that matter most - `--n-fft <csv/json>` - `--hop-size <csv/json>` - `--win-length` (static or dynamic)

Artifacts to listen for - timbral shifts at parameter-transition boundaries - mismatched hop/window trajectories causing roughness

11.74 Stateful Stream Mode with Dynamic Stretch CSV

The next example makes the role of the stateful stream mode with dynamic stretch CSV explicit.

Command

```
pvx stream input.wav --output output_stream.wav --chunk-seconds 0.2 --stretch controls/stretch.csv --interp
linear
```

Example controls/stretch.csv

```
time_sec,value
0.0,1.0
3.0,1.4
6.0,2.0
```

Explanation - Uses chunked stateful stream processing with the same control-file interface as `pvx voc`.
 - The stretch value is sampled from the interpolated trajectory for each processed chunk.

Before/After - Before: one global stretch factor. - After: evolving stretch profile over time, while staying in stream mode.

Parameters that matter most - `--chunk-seconds` - `--stretch <csv/json>` - `--interp`

Artifacts to listen for - chunk-boundary texture changes if `--chunk-seconds` is too short - abrupt pacing changes from steep control curves

11.75 Generate a Stretch Envelope (Function Stream)

The material below develops the generate a stretch envelope (function stream) in practical terms.

Command

```
pvx envelope --mode adsr --duration 8 --rate 20 --attack-sec 0.2 --decay-sec 0.6 --sustain 1.1 --release-sec
1.0 --key stretch --output controls/stretch_env.csv
```

Periodic LFO variants (same tool, `pvx lfo` alias)

```
pvx lfo --wave sine --duration 12 --cycles 6 --center 1.0 --amplitude 0.25 --key stretch --output controls/
stretch_sine.csv
pvx lfo --wave triangle --duration 8 --frequency-hz 0.5 --center 1.0 --amplitude 0.2 --key stretch --output
controls/stretch_triangle.csv
pvx lfo --wave square --duration 8 --frequency-hz 2.0 --center 1.0 --amplitude 0.3 --duty-cycle 0.35 --key
pitch_ratio --output controls/pitch_square.csv
pvx lfo --wave saw_up --duration 8 --frequency-hz 0.5 --center 1.0 --amplitude 0.2 --key stretch --output
controls/stretch_saw_up.csv
```

Explanation - Generates a control-rate trajectory directly from the command line for later reuse in `pvx voc`, `pvx tvfilter`, and other map-driven tools. - Keeps control authoring reproducible and scriptable in shell pipelines.

Before/After - Before: manual spreadsheet editing for every map revision. - After: deterministic, one-command control-map generation.

Parameters that matter most - `--mode` - `--wave` (alias for `--mode`) - `--duration` - `--rate` - `--frequency-hz` or `--cycles` for periodic shapes - `--center` and `--amplitude` for periodic offsets/depth - `--key`

Artifacts to listen for - abrupt transitions if envelope segment times are too short - over-aggressive modulation ranges for stretch/pitch controls

11.76 Reshape and Densify a Control Map

The material below develops the reshape and densify a control map in practical terms.

Command

```
pvx reshape controls/stretch_env.csv --key stretch --operation resample --rate 50 --interp polynomial --
order 5 --output controls/stretch_env_dense.csv
```

Explanation - Resamples sparse control points into a denser trajectory for smoother frame-level sampling in downstream tools. - Supports additional transform operations (**scale**, **offset**, **clip**, **normalize**, **smooth**, **time-scale**, and others).

Before/After - Before: coarse control points with larger step size. - After: denser curve with smoother interpolation behavior.

Parameters that matter most - **--operation** - **--rate** - **--interp** - **--order**

Artifacts to listen for - overshoot if high-order polynomial interpolation is used on sparse/irregular points - excessive smoothing reducing intended modulation detail

11.77 Run PVC Parity Benchmark Gate

The material below develops the run pvc parity benchmark gate in practical terms.

Command

```
python3 benchmarks/run_pvc_parity.py --quick --out-dir benchmarks/out_pvc_parity --baseline benchmarks/
baseline_pvc_parity.json --gate --gate-tolerance 0.20
```

Explanation - Runs deterministic parity scenarios for PVC-inspired phase 3-7 operators, then compares metrics to a committed baseline. - Useful for regression gating in local workflows and continuous integration (CI).

Before/After - Before: no focused parity regression coverage for the new operator family. - After: repeatable identity/effect scenario checks with a hard pass/fail gate.

Parameters that matter most - **--baseline** - **--gate** - **--gate-tolerance** - **--quick**

Artifacts to listen for - identity-case drift (unexpected coloration when effect depth should be near-neutral) - unstable runtime or metric jumps between revisions

11.78 Persist STFT Analysis Artifact (PVXAN)

The next example makes the role of the persist STFT analysis artifact (pvxan) explicit.

Command

```
pvx analysis create input.wav --output input.pvxan.npz --n-fft 4096 --win-length 4096 --hop-size 256 --
window hann
pvx analysis inspect input.pvxan.npz
```

Explanation - Saves a reusable short-time Fourier transform (STFT) analysis artifact so you can inspect or re-use spectral analysis across multiple downstream workflows. - Useful when iterating on response design, parity checks, and research traces without re-running the same front-end analysis assumptions manually each time.

Before/After - Before: analysis lives only in-memory during a single command invocation. - After: analysis is persisted as a versionable artifact (**.pvxan.npz**) you can inspect and pass to other tools.

Parameters that matter most - **--n-fft** - **--win-length** - **--hop-size** - **--window**

Artifacts to listen for - not an audio render; inspect analysis metadata consistency (sample rate, frame count, transform/window choices) - mismatch between analysis resolution and intended downstream response usage

11.79 Reusable Response Artifact (PVXRF)

The material below develops the reusable response artifact (pvxrf) in practical terms.

Command

```
pvx response create input.pvxn.npz --output input.pvxrf.npz --method median --phase-mode mean --normalize
peak --smoothing-bins 3
pvx response inspect input.pvxrf.npz
```

Explanation - Builds a reusable frequency-response profile from a persisted analysis artifact. - This response can drive `pvx filter`, `pvx tvfilter`, `pvx noisefilter`, `pvx bandamp`, and `pvx spec-compander`.

Before/After - Before: no explicit reusable spectral profile for cross-session reuse. - After: deterministic response artifact (`.pvxrf.npz`) with inspectable metadata.

Parameters that matter most - `--method` - `--phase-mode` - `--normalize` - `--smoothing-bins`

Artifacts to listen for - not an audio render; watch for overly jagged responses if smoothing is too low - flattened response detail if smoothing is too high

11.80 Response-Profile Denoising with pvx noisefilter

A closer look at the response-profile denoising with `pvx noisefilter` clarifies the choices involved.

Command

```
pvx analysis create roomtone.wav --output roomtone.pvxn.npz
pvx response create roomtone.pvxn.npz --output roomtone.pvxrf.npz --method median --normalize peak
pvx noisefilter dialog.wav --response roomtone.pvxrf.npz --noise-floor 1.2 --response-mix 1.0 --dry-mix 0.05
--output dialog_noisefilter.wav
```

Explanation - Uses external room-tone analysis to build a targeted noise profile, then applies response-referenced suppression to the noisy program material. - This is often more controlled than blind denoising when consistent environmental noise is present.

Before/After - Before: broadband hiss/hum/air conditioning texture over dialogue. - After: reduced steady-state noise with dialogue content better preserved.

Parameters that matter most - `--noise-floor` - `--response-mix` - `--dry-mix` - quality of roomtone. wav

Artifacts to listen for - speech dulling from over-suppression - tonal holes if the room-tone profile does not match the program noise

11.81 Time-Varying Spectral Filtering with pvx tvfilter

The material below develops the time-varying spectral filtering with `pvx tvfilter` in practical terms.

Command

```
pvx tvfilter pad.wav --response color_profile.pvxrf.npz --tv-map controls/response_mix.csv --tv-key
response_mix --tv-interp s_curve --tv-order 3 --output pad_tvfilter.wav
```

Example `controls/response_mix.csv`

```
time_sec,response_mix
0.0,0.10
4.0,0.65
8.0,1.00
12.0,0.35
```

Explanation - Morphs response influence over time so timbre evolves dynamically instead of staying static for the full render. - Useful for long-form pads, scenes, and texture design where one fixed filter shape is too rigid.

Before/After - Before: static timbre profile. - After: time-varying spectral color trajectory controlled by a map.

Parameters that matter most - --tv-map - --tv-key - --tv-interp - --response-mix

Artifacts to listen for - abrupt tonal jumps if control map points are sparse and interpolation is too sharp - over-filtered sections when response mix is pinned near 1.0 for too long

11.82 Spectral Peak Emphasis with pvx bandamp

A closer look at the spectral peak emphasis with pvx bandamp clarifies the choices involved.

Command

```
pvx bandamp source.wav --response profile.pvxrf.npz --band-gain-db 8 --peak-count 10 --band-width-bins 6 --
output source_bandamp.wav
```

Explanation - Detects high-energy response regions and boosts corresponding bands to emphasize characteristic partial groups. - Helpful for presence enhancement and stylized spectral contour exaggeration.

Before/After - Before: flatter spectral emphasis. - After: more pronounced response-defined band structure.

Parameters that matter most - --band-gain-db - --peak-count - --band-width-bins - response profile content

Artifacts to listen for - harshness if gain and peak count are too high - ringing coloration when boosted bands are too narrow

11.83 Response-Shaped Spectral Dynamics with pvx spec-compander

The next example makes the role of the response-shaped spectral dynamics with pvx spec-compander explicit.

Command

```
pvx spec-compander mix.wav --response profile.pvxrf.npz --comp-threshold-db -18 --comp-ratio 2.5 --expand-
ratio 1.3 --output mix_speccomp.wav
```

Explanation - Applies response-informed compression and expansion behavior in the spectral domain. - Useful for reshaping dynamic contrast in specific timbral regions rather than broad full-band dynamics only.

Before/After - Before: less controlled spectral dynamic contour. - After: response-informed dynamic shaping that can clarify or stylize spectral balance.

Parameters that matter most - --comp-threshold-db - --comp-ratio - --expand-ratio - --response

Artifacts to listen for - pumping/breathing if ratios are aggressive - unstable ambience tails if threshold is too low

11.84 Ring Modulation Fundamentals with `pvx ring`

The next example makes the role of the ring modulation fundamentals with `pvx ring` explicit.

Command

```
pvx ring synth.wav --frequency-hz 43 --depth 0.9 --mix 0.8 --feedback 0.15 --output synth_ring.wav
```

Explanation - Applies controllable ring modulation with optional feedback memory for sideband-rich timbre design. - Works well for metallic, bell-like, and animated tremolo-adjacent transformations.

Before/After - Before: original oscillator or instrument tone. - After: sideband-rich ring-modulated texture with controllable wet/dry blend.

Parameters that matter most - `--frequency-hz` - `--depth` - `--mix` - `--feedback`

Artifacts to listen for - harsh alias-like texture when carrier frequency/depth are extreme - runaway coloration if feedback is too high

11.85 Resonant Ring Filtering with `pvx ringfilter`

The material below develops the resonant ring filtering with `pvx ringfilter` in practical terms.

Command

```
pvx ringfilter input.wav --frequency-hz 60 --depth 0.8 --mix 0.9 --resonance-hz 1200 --resonance-q 9 --resonance-mix 0.4 --resonance-decay 0.2 --output input_ringfilter.wav
```

Explanation - Chains ring modulation with a resonant peak stage for tuned, animated spectral coloration. - Useful for vocal/synth character design where simple ring modulation alone is too blunt.

Before/After - Before: dry source or plain ring modulation. - After: ringed tone plus focused resonant emphasis and decay behavior.

Parameters that matter most - `--resonance-hz` - `--resonance-q` - `--resonance-mix` - `--resonance-decay`

Artifacts to listen for - whistling resonances if `--resonance-q` is too high - tonal masking if resonance center sits on dominant formants/partials unintentionally

11.86 Harmonic Chord Mapping with `pvx chordmapper`

A closer look at the harmonic chord mapping with `pvx chordmapper` clarifies the choices involved.

Command

```
pvx chordmapper vocal.wav --root-hz 220 --chord min7 --strength 0.75 --tolerance-cents 35 --boost-db 6 --attenuation 0.45 --output vocal_chordmapped.wav
```

Explanation - Pulls spectral emphasis toward chord-constrained pitch-class regions relative to a chosen root. - Good for harmonic re-coloring and pseudo-harmonization textures without explicit multi-voice synthesis.

Before/After - Before: unconstrained harmonic emphasis. - After: chord-guided spectral emphasis with out-of-chord attenuation.

Parameters that matter most - `--root-hz` - `--chord` - `--strength` - `--tolerance-cents`

Artifacts to listen for - unnatural pitch-class bias if tolerance is too narrow - muffling when attenuation is too high for non-chord content

11.87 Inharmonic Spectral Warping with pvx inharmonator

The material below develops the inharmonic spectral warping with pvx inharmonator in practical terms.

Command

```
pvx inharmonator bell.wav --inharmonic-f0-hz 220 --inharmoniccity 0.0002 --inharmonic-mix 1.0 --dry-mix 0.1
--output bell_inharm.wav
```

Explanation - Applies controlled inharmonic spectral remapping inspired by stiff-string/inharmonic partial spacing models. - Useful for bell, plate, metallic, and abstract hybrid timbres.

Before/After - Before: harmonic or near-harmonic partial spacing. - After: increasingly inharmonic overtone structure with controllable blend.

Parameters that matter most - --inharmonic-f0-hz - --inharmoniccity - --inharmonic-mix - --dry-mix

Artifacts to listen for - excessive dissonance if inharmoniccity is too high - loss of note center perception at high wet mix on dense sources

11.88 Diagnose Environment and Install Issues (pvx doctor)

A closer look at the diagnose environment and install issues (pvx doctor) clarifies the choices involved.

Command

```
pvx doctor
```

Explanation - Runs launch-readiness diagnostics for virtual environment, path environment variable (PATH), and optional dependency visibility. - Prints concrete remediation commands when warnings are found.

Before/After - Before: trial-and-error debugging when pvx commands fail or optional features are missing. - After: one-command diagnosis with direct fix suggestions.

Parameters that matter most - --strict - --json

Artifacts to listen for - not an audio render; inspect warnings list for missing PATH entries and dependency gaps

11.89 Minimal Public-Demo Sequence (pvx quickstart)

The next example makes the role of the minimal public-demo sequence (pvx quickstart) explicit.

Command

```
pvx quickstart input.wav --output output.wav
```

Explanation - Prints a concise launch script that chains diagnostics, safe first render, transform guidance, examples, and smoke test. - Useful when preparing demos and avoiding ad-hoc command selection.

Before/After - Before: inconsistent launch flow and missed preflight checks. - After: repeatable launch sequence suitable for public demos.

Parameters that matter most - positional input path - --output - --material

Artifacts to listen for - not an audio render; this is command-sequence generation

11.90 Quality-First First Pass (`pvx safe`)

A closer look at the quality-first first pass (`pvx safe`) clarifies the choices involved.

Command

```
pvx safe input.wav --material mix --output input_safe.wav
```

Explanation - Wraps `pvx voc` with conservative defaults (`identity` phase locking, hybrid transient handling, stereo coherence defaults) to reduce first-pass artifact risk. - Intended for “first render should not embarrass us” scenarios.

Before/After - Before: first render quality depends heavily on manual flag selection. - After: consistent conservative baseline output with one command.

Parameters that matter most - `--material` - passthrough `pvx voc` arguments (for example `--stretch`, `--pitch`) - `--overwrite`

Artifacts to listen for - reduced, but not eliminated, smear/phase artifacts on extreme settings - material-specific tuning may still be required for final delivery

11.91 Transform Availability and Selection (`pvx transforms`)

A closer look at the transform availability and selection (`pvx transforms`) clarifies the choices involved.

Command

```
pvx transforms
```

Explanation - Prints transform backend options (`fft`, `dft`, `czt`, `dct`, `dst`, `hartley`) with runtime availability and practical recommendations. - Clarifies when optional SciPy-backed transforms are unavailable.

Before/After - Before: uncertainty about whether non-FFT transforms are supported in the active runtime. - After: explicit transform availability and usage guidance in one place.

Parameters that matter most - `--json`

Artifacts to listen for - not an audio render; review backend availability and recommendation text

11.92 Fast End-to-End Health Check (`pvx smoke`)

The material below develops the fast end-to-end health check (`pvx smoke`) in practical terms.

Command

```
pvx smoke --output smoke_out.wav
```

Explanation - Generates a short synthetic input, runs a conservative `pvx voc` render, verifies output creation, and reports output metadata. - Useful for pre-demo confidence checks and basic continuous integration (CI) sanity gates.

Before/After - Before: no quick integrated health check for “does this build actually render audio.” - After: one-command smoke render with pass/fail signal.

Parameters that matter most - `--output` - `--duration` - `--sample-rate` - `--stretch` - `--pitch`

Artifacts to listen for - simple synthetic tone render; severe distortion/silence indicates environment/runtime problems

11.93 Deterministic AI Dataset Augmentation (pvx augment)

A closer look at the deterministic AI dataset augmentation (pvx augment) clarifies the choices involved.

Command

```
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --split 0.8,0.1,0.1
--seed 1337
```

Explanation - Generates reproducible augmentation variants per source file and writes machine-readable manifests for training pipelines. - Intended for automatic speech recognition (ASR), music information retrieval (MIR), and self-supervised learning (SSL) experiments.

Before/After - Before: ad-hoc augmentation scripts with weak reproducibility and incomplete provenance. - After: deterministic rendered variants plus JSON Lines (JSONL) and comma-separated values (CSV) manifests including split assignment and parameters.

Parameters that matter most - `--variants-per-input` - `--intent` - `--seed` - `--split` - `--grouping` / `--group-separator` - `--dry-run`

Artifacts to listen for - augmentation policy mismatch to task (too mild for contrastive learning, too aggressive for label-sensitive training) - split leakage if downstream pipeline ignores source-family grouping

11.94 Speaker-Balanced Split Assignment for Augmentation

The next example makes the role of the speaker-balanced split assignment for augmentation explicit.

Command

```
pvx augment data/*.wav --output-dir aug_speaker --variants-per-input 3 --intent asr_robust --split-mode
speaker_balanced --labels-csv labels.csv --seed 1337
```

Explanation - Uses speaker metadata to assign train/validation/test splits with better speaker balance. - Reduces cross-speaker skew and helps avoid accidental split leakage.

Before/After - Before: random split may under-represent some speakers in validation/test. - After: split assignment uses speaker-aware balancing constraints.

Parameters that matter most - `--split-mode speaker_balanced` - `--labels-csv` - `--split` - `--grouping`

Artifacts to listen for - metadata mismatch (wrong speaker IDs) causing unhelpful balancing - tiny corpora where strict balancing is statistically limited

11.95 Contrastive Paired Views for Self-Supervised Learning

A closer look at the contrastive paired views for self-supervised learning clarifies the choices involved.

Command

```
pvx augment data/*.wav --output-dir aug_ssl --variants-per-input 2 --intent ssl_contrastive --pair-mode
contrastive2 --seed 2026
```

Explanation - Generates paired views (`view a` / `view b`) per augmentation variant for contrastive learning pipelines. - Pair relationships are tracked in manifest fields (`pair_id`, `view_id`).

Before/After - Before: only independent single-view augment outputs. - After: structured paired views suitable for positive-pair training setups.

Parameters that matter most - `--pair-mode contrastive2` - `--intent ssl_contrastive` - `--seed`

Artifacts to listen for - over-aggressive pair divergence reducing semantic overlap - too-similar pairs limiting contrastive learning value

11.96 Resume + Append for Long Augmentation Runs

A closer look at the resume + append for long augmentation runs clarifies the choices involved.

Command

```
pvx augment data/*.wav --output-dir aug_resume --variants-per-input 6 --intent mir_music --resume --append-manifest --workers 4 --seed 9001
```

Explanation - Resumes from existing outputs/manifests and appends new records without losing previous lineage metadata. - Useful for interrupted jobs and incremental dataset growth.

Before/After - Before: restarts can duplicate work or overwrite provenance. - After: incremental, resumable augmentation workflow.

Parameters that matter most - `--resume` - `--append-manifest` - `--workers` - `--seed`

Artifacts to listen for - stale manifest rows if files are manually moved/deleted outside tooling - mixed policy runs when appending across dissimilar settings

11.97 Manifest Validation and Merge

The material below develops the manifest validation and merge in practical terms.

Command

```
pvx augment-manifest validate aug_run_a/augment_manifest.jsonl --strict
pvx augment-manifest merge aug_run_a/augment_manifest.jsonl aug_run_b/augment_manifest.jsonl --output-jsonl
aug_merged/manifest.jsonl --output-csv aug_merged/manifest.csv
pvx augment-manifest stats aug_merged/manifest.jsonl
```

Explanation - Validates required manifest fields, merges runs with de-duplication, and prints quick stats for auditability. - Supports production data curation workflows where multiple augmentation passes are combined.

Before/After - Before: ad-hoc manual joins and uncertain manifest integrity. - After: deterministic validation + merge + reporting in one command family.

Parameters that matter most - `validate` - `--strict` - `merge` - `--dedupe-by` - `stats`

Artifacts to listen for - duplicate lineage entries if dedupe key does not match your data model - invalid rows from older manifests lacking required fields

11.98 Augmentation Benchmark Gate

The material below develops the augmentation benchmark gate in practical terms.

Command

```
python benchmarks/run_augment_bench.py --quick --out-dir benchmarks/out_augment --baseline benchmarks/  
baseline_augment_small.json --gate --gate-tolerance 0.30
```

Explanation - Runs deterministic augmentation benchmark checks and compares summary metrics against a baseline. - Intended for release and continuous integration (CI) regression gating.

Before/After - Before: no objective gate for augmentation drift. - After: reproducible benchmark reports with pass/fail criteria.

Parameters that matter most - --baseline - --gate - --gate-tolerance - --quick

Artifacts to listen for - split-balance drift - reduced augmentation diversity (low stretch/pitch standard deviation) - clipping/level anomalies in audit metrics

CHAPTER 12

Advanced Session Recipes

This chapter collects complete `pvx` sessions rather than isolated commands. Each recipe begins with a production or compositional goal, creates any required control or analysis artifacts, performs the render, and names the main failure modes to audition. The examples assume the current directory contains `controls`, `maps`, `artifacts`, `renders`, `checkpoints`, and `reports` directories.

```
mkdir -p controls maps artifacts renders checkpoints reports
```

The recipes deliberately overlap in technique. Repetition at this level is useful because a command acquires a different meaning when it appears before restoration, after harmonic mapping, inside a sidechain system, or at the end of a checkpointed long render. Treat filenames as a session plan and preserve the intermediate artifacts until the result has been approved.

12.1 Restoration and delivery

The first group treats phase-vocoder work as part of a larger production chain. Conservative values are intentional. Restoration artifacts accumulate quickly when denoising, dereverberation, stretching, and loudness control are all applied to the same signal.

Recipe 1: dialogue reconstruction from matched room tone

Build a spectral noise profile from isolated room tone, apply moderate profile-driven suppression, reduce the remaining room tail, then slow the dialogue slightly and finish it at a controlled delivery level.

```
pvx analysis create roomtone.wav --output artifacts/roomtone.pvxn.npz
pvx response create artifacts/roomtone.pvxn.npz \
  --output artifacts/roomtone.pvxrf.npz --method median --normalize peak
pvx noisefilter dialog.wav \
  --response artifacts/roomtone.pvxrf.npz \
  --noise-floor 1.2 --response-mix 1.0 --dry-mix 0.05 \
  --output renders/dialog_profiled.wav
pvx deverb renders/dialog_profiled.wav --strength 0.30 --stdout \
| pvx voc - --preset vocal_studio --stretch 1.06 \
  --target-lufs -16 --limiter-threshold -1.0 \
  --output renders/dialog_delivery.wav
```

Audition sibilants, breaths, and word endings before judging the noise floor. If the voice becomes hollow, reduce profile influence before reducing more reverberation.

Recipe 2: archival speech with two-stage timing repair

Use a gentle first pass to improve intelligibility, then apply a small pacing correction in a separate render. Separate stages make it easier to determine whether an artifact came from restoration or time modification.

```
pvx denoise archive.wav --reduction-db 5 --smooth 7 --stdout \
| pvx deverb - --strength 0.25 --output renders/archive_clean.wav
pvx voc renders/archive_clean.wav \
--stretch 1.12 --phase-locking identity \
--n-fft 2048 --hop-size 256 \
--target-lufs -18 --limiter-threshold -1.5 \
--manifest-json reports/archive.json --manifest-append \
--output renders/archive_retimed.wav
```

Compare consonant edges against `archive_clean.wav`. The second pass should alter pace without making the first pass sound retrospectively overprocessed.

Recipe 3: field recording preservation with multiresolution stretch

Clean a field recording lightly, then combine three analysis scales so low-frequency ambience and short foreground events do not depend on one FFT size.

```
pvx denoise field.wav --reduction-db 4 --smooth 9 --stdout \
| pvx deverb - --strength 0.20 --output renders/field_clean.wav
pvx voc renders/field_clean.wav \
--stretch 1.35 --multires-fusion \
--multires-ffts 1024,2048,4096 \
--multires-weights 0.25,0.35,0.40 \
--target-lufs -20 --limiter-threshold -1.5 \
--output renders/field_multires.wav
```

Listen separately to insects, wind, distant traffic, and nearby impacts. A weight set that improves one layer can blur another, so retain the cleaned intermediate for comparison.

Recipe 4: coherent multichannel restoration

For a multichannel recording, remove modest noise first and anchor phase evolution to a reference channel during the time adjustment. The reference should contain a stable version of the events shared across channels.

```
pvx denoise surround.wav --reduction-db 4 --smooth 7 \
--output renders/surround_clean.wav
pvx voc renders/surround_clean.wav \
--stretch 1.10 \
--stereo-mode ref_channel_lock --ref-channel 2 \
--coherence-strength 0.95 \
--bit-depth 24 --dither tpdf --dither-seed 41 \
--metadata-policy sidecar \
--output renders/surround_coherent.wav
```

Check channel pairs, mono compatibility, and diffuse ambience. Excessive coherence can pull legitimately decorrelated material toward the reference channel.

12.2 Automation as composition

The next recipes treat control files as reusable musical objects. Generate them deterministically, inspect them as data, and reshape them independently of the sound render.

Recipe 5: exponential expansion with a smoothed alternate take

Create one strongly curved stretch trajectory and a smoothed derivative, then render both against identical source and phase settings.

```

pvx envelope --mode exp --duration 10 --rate 30 \
  --start 0.7 --end 1.8 --exp-curve 6 \
  --key stretch --output controls/stretch_exp.csv
pvx reshape controls/stretch_exp.csv \
  --key stretch --operation smooth --window 11 \
  --output controls/stretch_exp_smooth.csv
pvx voc input.wav --stretch controls/stretch_exp.csv \
  --interp linear --output renders/stretch_exp.wav
pvx voc input.wav --stretch controls/stretch_exp_smooth.csv \
  --interp linear --output renders/stretch_exp_smooth.wav

```

The useful comparison is not simply smooth versus rough. Decide whether the sharper trajectory contributes phrasing or merely exposes control-rate discontinuity.

Recipe 6: polynomial pitch arc with formant protection

Drive pitch from JSON control points, use cubic-order polynomial interpolation, and preserve vocal identity during the excursion.

```

{
  "points": [
    {"time_sec": 0.0, "value": 1.0},
    {"time_sec": 1.5, "value": 1.122462048},
    {"time_sec": 3.0, "value": 1.334839854},
    {"time_sec": 4.5, "value": 0.890898718},
    {"time_sec": 6.0, "value": 1.0}
  ]
}

```

Save the object above as `controls/pitch_arc.json`, then render it:

```

pvx voc vocal.wav \
  --pitch-shift-ratio controls/pitch_arc.json \
  --interp polynomial --order 3 \
  --pitch-mode formant-preserving \
  --stereo-mode mid_side_lock --coherence-strength 0.90 \
  --output renders/vocal_pitch_arc.wav

```

Listen for polynomial overshoot between points, vowel-size drift, and center-image movement. Add points or reduce interpolation order before adding more smoothing.

Recipe 7: rhythmic sample-and-hold time folding

Use explicit time regions and sample-and-hold interpolation to alternate compression and expansion. This is intentionally discontinuous and works best when boundaries align with musical events.

```

start_sec,end_sec,value
0.0,0.5,1.0
0.5,1.0,1.5
1.0,1.5,0.75
1.5,2.0,2.0
2.0,3.0,1.0

```

Save the map as `controls/fold.csv`, then render it:

```
pvx voc loop.wav \  
  --stretch controls/fold.csv --interp none \  
  --transient-mode hybrid \  
  --output renders/loop_time_fold.wav
```

Move one boundary by a few milliseconds and compare. The experiment reveals whether the gesture depends on event alignment or on the ratio sequence alone.

Recipe 8: section-adaptive analysis resolution

Change FFT and hop sizes over time so transient sections use shorter analysis while sustained sections receive greater frequency resolution.

```
time_sec,value  
0.0,1024  
4.0,1024  
4.1,4096  
12.0,4096  
12.1,2048
```

Save that map as `controls/nfft.csv`. Create a matching hop map named `controls/hop.csv` with values 128, 128, 512, 512, and 256 at the same timestamps.

```
pvx voc arrangement.wav \  
  --n-fft controls/nfft.csv --hop-size controls/hop.csv \  
  --stretch 1.25 --interp linear \  
  --output renders/arrangement_adaptive.wav
```

Audition the two transition regions in isolation. Resolution changes can become timbral edits even when the requested stretch remains constant.

12.3 Sidechain and feature routing

Feature routing allows one file to shape another without replacing its spectrum directly. Always clamp derived values. A guide feature can contain outliers that are harmless as measurements but destructive as stretch or pitch ratios.

Recipe 9: inverse melodic breathing

Use the guide's pitch contour to compress target time at higher notes and expand it at lower notes while leaving target pitch unchanged.

```
pvx follow melody.wav drone.wav \  
  --emit pitch_map --stretch 1.0 --pitch-conf-min 0.75 \  
  --route 'stretch=inv(pitch_ratio)' \  
  --route 'stretch=clip(stretch,0.70,1.80)' \  
  --route 'pitch_ratio=const(1.0)' \  
  --output renders/drone_inverse_melody.wav
```

The result follows relative pitch movement, not note duration or score position. Listen for unstable regions where the guide becomes unvoiced.

Recipe 10: MFCC pitch with flux-driven time

Route one timbral coefficient to target pitch and MPEG-7 spectral flux to target stretch. This produces coupled motion from two different aspects of the guide.


```

pvx follow guide.wav target.wav \
  --feature-set all --mfcc-count 13 \
  --emit pitch_map --stretch 1.0 \
  --route 'pitch_ratio=affine(mfcc_01,0.002,1.0)' \
  --route 'pitch_ratio=clip(pitch_ratio,0.5,2.0)' \
  --route 'stretch=affine(mpeg7_spectral_flux,0.05,1.0)' \
  --route 'stretch=clip(stretch,0.85,1.6)' \
  --output renders/target_mfcc_flux.wav

```

MFCC values are source-dependent. Treat the coefficients as control signals whose scaling must be calibrated, not as universal perceptual units.

Recipe 11: transient-protected feature following

Use loudness to create broad timing motion while the renderer's hybrid transient mode protects attacks. Track first so the feature map can be inspected before rendering.

```

pvx pitch-track percussion_guide.wav \
  --feature-set all --output maps/percussion_features.csv
pvx voc pad.wav --pitch-map maps/percussion_features.csv \
  --route 'stretch=affine(rms_norm,1.0,0.7)' \
  --route 'stretch=clip(stretch,0.80,1.60)' \
  --route 'pitch_ratio=const(1.0)' \
  --transient-mode hybrid \
  --output renders/pad_percussion_motion.wav

```

Inspect the routed map if attacks still bloom. Feature-derived control and the renderer's transient mode solve related but different problems.

Recipe 12: two guides with separate musical roles

Let one guide supply pitch and another guide supply spectral-flux timing. The short Python stage merges two inspected feature files into one explicit control artifact.

```

pvx pitch-track pitch_guide.wav --feature-set all --output maps/pitch_guide.csv
pvx pitch-track rhythm_guide.wav --feature-set all --output maps/rhythm_guide.csv
python3 - <<'PY'
import csv
from pathlib import Path

a = Path("maps/pitch_guide.csv")
b = Path("maps/rhythm_guide.csv")
out = Path("maps/two_guide.csv")
with a.open() as fa, b.open() as fb, out.open("w", newline="") as fo:
    pitch_rows = list(csv.DictReader(fa))
    rhythm_rows = list(csv.DictReader(fb))
    fields = list(dict.fromkeys(list(pitch_rows[0]) + ["stretch"]))
    writer = csv.DictWriter(fo, fieldnames=fields)
    writer.writeheader()
    for index, row in enumerate(pitch_rows):
        rhythm = rhythm_rows[min(index, len(rhythm_rows) - 1)]
        flux = float(rhythm.get("spectral_flux", 0.0) or 0.0)
        row["stretch"] = max(0.8, min(1.5, 1.0 + 0.03 * flux))
        writer.writerow(row)
PY
pvx voc target.wav --pitch-map maps/two_guide.csv \
  --route 'pitch_ratio=clip(pitch_ratio,0.75,1.35)' \
  --route 'stretch=clip(stretch,0.80,1.50)' \
  --output renders/target_two_guides.wav

```

This merge aligns rows rather than musical events. For guides with different durations or analysis rates, resample them to a shared timeline before combining features.

12.4 Persistent spectral artifacts

These recipes use PVXAN and PVXRF files as session assets. Keep their metadata with the project and name them after both source and derivation method.

Recipe 13: reusable timbral imprint with an automated entrance

Derive a median response from one source, generate a slow response-mix curve, and introduce that spectral identity into another source over twelve seconds.

```
pvx analysis create color.wav --output artifacts/color.pvxan.npz \
  --n-fft 4096 --win-length 4096 --hop-size 256 --window hann
pvx response create artifacts/color.pvxan.npz \
  --output artifacts/color_median.pvxrf.npz \
  --method median --phase-mode mean --normalize peak --smoothing-bins 3
pvx envelope --mode ramp --duration 12 --rate 30 \
  --start 0.0 --end 1.0 --key response_mix \
  --output controls/color_mix.csv
pvx tvfilter carrier.wav \
  --response artifacts/color_median.pvxrf.npz \
  --tv-map controls/color_mix.csv --tv-key response_mix \
  --tv-interp s_curve --tv-order 3 \
  --output renders/carrier_color_entrance.wav
```

Compare median response against an RMS-derived response. The change can be more consequential than the automation curve.

Recipe 14: response-informed noise removal before extreme stretch

Use a noise-only excerpt to build a profile, clean the source, then perform a resumable fifteen-minute stretch. Cleaning first prevents stationary noise from becoming the dominant long-form texture.

```
pvx analysis create noise_only.wav --output artifacts/noise.pvxan.npz
pvx response create artifacts/noise.pvxan.npz \
  --output artifacts/noise.pvxrf.npz --method median --normalize peak
pvx noisefilter seed.wav --response artifacts/noise.pvxrf.npz \
  --noise-floor 1.1 --response-mix 0.8 --dry-mix 0.10 \
  --output renders/seed_clean.wav
pvx voc renders/seed_clean.wav \
  --preset extreme_ambient --target-duration 900 \
  --auto-segment-seconds 0.25 \
  --checkpoint-dir checkpoints/seed_15min \
  --manifest-json reports/seed_15min.json --manifest-append \
  --output renders/seed_15min.wav
```

Also render ten seconds from the untreated seed. Extreme stretching makes profile mistakes easy to hear and difficult to diagnose after a long render.

Recipe 15: morph, freeze, and finish a hybrid pad

Create a balanced spectral morph, freeze a stable point, remove a small amount of accumulated haze, and set a restrained final loudness.

```

pvx morph bells.wav choir.wav --alpha 0.5 \
  --output renders/bells_choir.wav
pvx freeze renders/bells_choir.wav \
  --freeze-time 0.8 --duration 40 \
  --output renders/bells_choir_frozen.wav
pvx denoise renders/bells_choir_frozen.wav \
  --reduction-db 3 --smooth 9 --stdout \
| pvx voc - --stretch 1.0 --target-lufs -20 \
  --limiter-threshold -1.5 \
  --output renders/bells_choir_pad.wav

```

Choose the freeze point by listening to the morph first. A technically stable frame can still have an unhelpful vowel, beating pattern, or stereo balance.

Recipe 16: response-shaped dynamics and resonant afterimage

Use one response artifact for spectral companding, then add a restrained tuned resonance as a separate, inspectable stage.

```

pvx analysis create reference.wav --output artifacts/reference.pvxn.npz
pvx response create artifacts/reference.pvxn.npz \
  --output artifacts/reference.pvxrf.npz --method rms --normalize rms
pvx spec-compander source.wav \
  --response artifacts/reference.pvxrf.npz \
  --comp-threshold-db -18 --comp-ratio 2.5 --expand-ratio 1.3 \
  --output renders/source_speccomp.wav
pvx ringfilter renders/source_speccomp.wav \
  --frequency-hz 43 --depth 0.35 --mix 0.35 \
  --resonance-hz 860 --resonance-q 7 \
  --resonance-mix 0.25 --resonance-decay 0.18 \
  --output renders/source_afterimage.wav

```

The first stage changes dynamic relationships among frequency regions; the second adds persistence. If the result masks attacks, reduce resonance before weakening the compander.

12.5 Harmonic, inharmonic, and spatial design

The following recipes deliberately stack pitch-class operations, spectral remapping, and stereo processing. Render intermediate stages because harmonic mistakes become difficult to attribute after widening and resonance.

Recipe 17: retuned harmony choir with controlled width

Retune the lead, construct a triadic stack, then add modest unison width. Each operation receives its own file so tuning, voicing, and spatial density can be approved separately.

```

pvx retune lead.wav --scale major --root D \
  --output renders/lead_retuned.wav
pvx harmonize renders/lead_retuned.wav \
  --intervals 0,4,7 --gains 1,0.75,0.65 \
  --output renders/lead_harmony.wav
pvx unison renders/lead_harmony.wav \
  --voices 5 --detune-cents 9 --width 0.9 \
  --output renders/lead_choir.wav

```

Approve octave tracking in the retuned file before judging the choir. Unison thickening can conceal tuning errors without correcting them.

Recipe 18: chord-constrained resonant cloud

Map an ambience recording toward a minor-seventh collection, stretch it, then add a high-Q resonant layer at low wet level.

```
pvx chordmapper ambience.wav \  
  --root-hz 110 --chord min7 --strength 0.80 \  
  --tolerance-cents 40 --boost-db 6 --attenuation 0.40 \  
  --output renders/ambience_min7.wav  
pvx voc renders/ambience_min7.wav \  
  --stretch 3.0 --multires-fusion \  
  --multires-ffts 1024,2048,4096 \  
  --multires-weights 0.20,0.35,0.45 \  
  --output renders/ambience_min7_stretched.wav  
pvx ringfilter renders/ambience_min7_stretched.wav \  
  --frequency-hz 27.5 --depth 0.25 --mix 0.25 \  
  --resonance-hz 1320 --resonance-q 12 \  
  --resonance-mix 0.18 --resonance-decay 0.30 \  
  --output renders/ambience_min7_cloud.wav
```

Narrow chord tolerance and high resonance Q can fight each other. Widen the mapping before increasing resonant energy.

Recipe 19: inharmonic bell transformed into a frozen bed

Warp a bell toward stiff-string partial spacing, stretch the result, and freeze a moment after the initial strike.

```
pvx inharmonator bell.wav \  
  --inharmonic-f0-hz 220 --inharmonic-ity 0.0002 \  
  --inharmonic-mix 1.0 --dry-mix 0.1 \  
  --output renders/bell_inharmonic.wav  
pvx voc renders/bell_inharmonic.wav \  
  --stretch 4.0 --transient-mode hybrid \  
  --output renders/bell_inharmonic_stretched.wav  
pvx freeze renders/bell_inharmonic_stretched.wav \  
  --freeze-time 1.2 --duration 60 \  
  --output renders/bell_inharmonic_bed.wav
```

The freeze point should occur after the protected strike but before the evolving partials lose the interval structure that motivated the recipe.

Recipe 20: stereo vocal transposition and harmony with image control

Shift the stereo vocal with formant preservation and mid-side locking, then harmonize and apply restrained unison width.

```
pvx voc stereo_vocal.wav \  
  --stretch 1.0 --pitch 2 \  
  --pitch-mode formant-preserving \  
  --stereo-mode mid_side_lock --coherence-strength 0.90 \  
  --output renders/stereo_vocal_up2.wav  
pvx harmonize renders/stereo_vocal_up2.wav \  
  --intervals 0,3,7 --gains 1,0.60,0.50 \  
  --output renders/stereo_vocal_harmony.wav  
pvx unison renders/stereo_vocal_harmony.wav \  
  --voices 3 --detune-cents 6 --width 0.65 \  
  --output renders/stereo_vocal_harmony_wide.wav
```

Check the center after each stage. Width should be the final decoration, not a remedy for phase instability introduced earlier.

12.6 Long renders, comparison, and reproducibility

Complex sessions need failure recovery and evidence. The final group emphasizes checkpoints, manifests, deterministic exports, and matched alternatives.

Recipe 21: four-hour resumable installation render

Render an extreme duration through small checkpointable segments and preserve the resolved session in a manifest. The same command resumes after interruption.

```
pvx voc installation_seed.wav \
  --target-duration 14400 \
  --preset extreme_ambient \
  --checkpoint-dir checkpoints/installation \
  --auto-segment-seconds 0.25 --resume \
  --manifest-json reports/installation.json --manifest-append \
  --output renders/installation_4h.wav
```

Test the first minute before committing hours of computation. Recovery protects time, but it does not protect against an unattractive parameter choice.

Recipe 22: deterministic delivery and null-test pair

Render the same transform twice with an explicit dither seed. Matching outputs provide a quick reproducibility check for the supported deterministic path.

```
pvx voc master.wav --stretch 1.08 \
  --bit-depth 24 --dither tpdf --dither-seed 123 \
  --manifest-json reports/deterministic.json --manifest-append \
  --output renders/master_seed123_a.wav
pvx voc master.wav --stretch 1.08 \
  --bit-depth 24 --dither tpdf --dither-seed 123 \
  --manifest-json reports/deterministic.json --manifest-append \
  --output renders/master_seed123_b.wav
```

Compare hashes and perform a null test in an audio editor. A mismatch is evidence to investigate, not automatically proof of an audible regression.

Recipe 23: matched transform and resolution suite

Create four controlled alternatives while changing only one family of analysis choices. Consistent naming turns the output directory into a compact listening experiment.

```
pvx voc source.wav --stretch 2.0 --transform fft \
  --n-fft 1024 --output renders/source_fft_1024.wav
pvx voc source.wav --stretch 2.0 --transform fft \
  --n-fft 4096 --output renders/source_fft_4096.wav
pvx voc source.wav --stretch 2.0 --transform hartley \
  --n-fft 1024 --output renders/source_hartley_1024.wav
pvx voc source.wav --stretch 2.0 --transform hartley \
  --n-fft 4096 --output renders/source_hartley_4096.wav
```

Level-match and randomize playback order. Without that discipline, file naming and render time can bias the comparison before listening begins.

Recipe 24: complex session with plan, render, and regression gate

Resolve an automatic plan without rendering, perform a manifested checkpointed render, then run the focused PVC-inspired parity benchmark. This closes the loop from intention to artifact to repository health.

```
pvx voc source.wav \  
  --auto-profile --auto-transform \  
  --manifest-json reports/complex_plan.json \  
  --dry-run --explain-plan  
pvx voc source.wav \  
  --auto-profile --auto-transform \  
  --target-duration 1200 \  
  --checkpoint-dir checkpoints/complex_session \  
  --auto-segment-seconds 0.5 --resume \  
  --manifest-json reports/complex_render.json --manifest-append \  
  --output renders/complex_session.wav  
python3 benchmarks/run_pvc_parity.py \  
  --quick --out-dir benchmarks/out_pvc_parity \  
  --baseline benchmarks/baseline_pvc_parity.json \  
  --gate --gate-tolerance 0.20
```

The benchmark does not certify the artistic render. It verifies that the implementation still satisfies a focused set of deterministic regression expectations after the session's tools have been exercised.

12.7 Adapting the recipes

Complexity should be added one auditable stage at a time. Keep every intermediate while developing a recipe, compare at matched loudness, and write down the source region used for judgment. Once the chain is understood, remove intermediates only as an operational optimization.

A reliable adaptation changes one class of decision at a time. First establish the source and analysis settings. Next establish automation. Then add restoration, harmonic processing, or spatial policy. Finish with loudness, bit depth, dither, metadata, and manifests. This order is not mandatory, but it makes failures easier to locate and successful gestures easier to preserve.

CHAPTER 13

Feature-Driven Sidechain Examples

This guide shows copy-paste command-line interface (CLI) recipes where one file (the guide) controls another file (the target) using tracked audio features.

- Guide file: **guide.wav** (feature source).
- Target file: **target.wav** (sound being transformed).
- Output folder in examples: **out/**.
- Most recipes use **pvx follow** (shortest path) and **--route** formulas.

Create an output folder once:

```
mkdir -p out maps
```

Quick built-in recipe discovery:

```
pvx follow --example  
pvx follow --example all  
pvx follow --example mfcc_flux
```

13.1 Quick Start

Basic pitch-to-stretch sidechain:

```
pvx follow guide.wav target.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output out/follow_basic.wav
```

Direct pitch-follow (guide pitch contour drives target pitch contour):

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 --output out/follow_pitch.wav
```

Track first, render later (decoupled analysis/render):

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output maps/guide_features.csv  
pvx voc target.wav --pitch-map maps/guide_features.csv \  
  --route stretch=pitch_ratio \  
  --route pitch_ratio=const(1.0) \  
  --output out/track_then_render.wav
```

13.2 Inspect Feature Columns Before Routing

Emit all tracked columns:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 20 --output maps/guide_all.csv
```

Print the CSV header as one column per line:

```
head -n 1 maps/guide_all.csv | tr ',' '\n'
```

Useful columns include: - pitch/voicing: f0_hz, pitch_ratio, confidence, voicing_prob, pitch_stability - spectral: spectral_centroid_hz, spectral_flux, spectral_flatness, rolloff_hz - dynamics: rms, rms_db, short_lufs_db, transientness, crest_factor_db - timbre vectors: mfcc_01..mfcc_N, mpeg7_*, mpeg7_audio_spectrum_envelope_01..10

13.3 Single-Feature Recipes

The recipes in this section turn the preceding ideas into repeatable working methods.

f0_hz -> pitch_ratio

The following session demonstrates the f0hz -> pitch ratio in practice.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(f0_hz,0.002272727,0.0) \  
--route pitch_ratio=clip(pitch_ratio,0.5,2.0) \  
--output out/f0_to_pitch.wav
```

pitch_ratio -> stretch (inverse motion)

A working command makes the pitchratio -> stretch (inverse motion) concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=inv(pitch_ratio) \  
--route stretch=clip(stretch,0.7,1.8) \  
--route pitch_ratio=const(1.0) \  
--output out/pitch_to_stretch_inverse.wav
```

confidence -> subtle pitch bend depth

The example below puts the confidence -> subtle pitch bend depth into executable form.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(confidence,0.25,0.9) \  
--route pitch_ratio=clip(pitch_ratio,0.85,1.15) \  
--output out/confidence_to_pitch.wav
```

voicing_prob -> stretch

A working command makes the voicingprob -> stretch concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
--route stretch=affine(voicing_prob,0.6,0.8) \  
--route stretch=clip(stretch,0.8,1.4) \  
--route pitch_ratio=const(1.0) \  
--output out/voicing_to_stretch.wav
```

rms_norm -> stretch

The following session demonstrates the rmsnorm -> stretch in practice.


```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(rms_norm,1.2,0.6) \
--route stretch=clip(stretch,0.8,1.8) \
--route pitch_ratio=const(1.0) \
--output out/rms_to_stretch.wav
```

spectral_flux -> stretch

A working command makes the spectralflux -> stretch concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(spectral_flux,0.04,1.0) \
--route stretch=clip(stretch,0.85,1.6) \
--route pitch_ratio=const(1.0) \
--output out/flux_to_stretch.wav
```

onset_strength -> attack-reactive stretch

A working command makes the onsetstrength -> attack-reactive stretch concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(onset_strength,0.05,0.95) \
--route stretch=clip(stretch,0.75,1.7) \
--route pitch_ratio=const(1.0) \
--output out/onset_to_stretch.wav
```

transientness -> transient-safe stretch variation

The example below puts the transientness -> transient-safe stretch variation into executable form.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(transientness,-0.4,1.2) \
--route stretch=clip(stretch,0.8,1.2) \
--route pitch_ratio=const(1.0) \
--output out/transientness_to_stretch.wav
```

spectral_centroid_hz -> brightness-linked pitch

The example below puts the spectralcentroidhz -> brightness-linked pitch into executable form.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(spectral_centroid_hz,0.00025,0.7) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.6) \
--output out/centroid_to_pitch.wav
```

spectral_flatness -> noisy/tonal pitch response

The example below puts the spectralflatness -> noisy/tonal pitch response into executable form.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(spectral_flatness,-0.5,1.25) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \
--output out/flatness_to_pitch.wav
```

harmonic_ratio -> harmonicity-linked pitch

The following session demonstrates the harmonicratio -> harmonicity-linked pitch in practice.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
  --route pitch_ratio=affine(harmonic_ratio,0.3,0.85) \  
  --route pitch_ratio=clip(pitch_ratio,0.85,1.2) \  
  --output out/harmonic_ratio_to_pitch.wav
```

inharmonicicity -> anti-harmonic pitch detune

A working command makes the inharmonicicity -> anti-harmonic pitch detune concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
  --route pitch_ratio=affine(inharmonicicity,-0.8,1.2) \  
  --route pitch_ratio=clip(pitch_ratio,0.75,1.2) \  
  --output out/inharmonicicity_to_pitch.wav
```

rolloff_norm -> pitch

The following session demonstrates the rolloffnorm -> pitch in practice.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
  --route pitch_ratio=affine(rolloff_norm,0.45,0.8) \  
  --route pitch_ratio=clip(pitch_ratio,0.8,1.3) \  
  --output out/rolloff_to_pitch.wav
```

pitch_stability -> micro-variation amount

The example below puts the pitchstability -> micro-variation amount into executable form.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \  
  --route pitch_ratio=affine(pitch_stability,0.2,0.9) \  
  --route pitch_ratio=clip(pitch_ratio,0.9,1.12) \  
  --output out/pitch_stability_to_pitch.wav
```

13.4 Multi-Feature Recipes (Different Features -> Different Targets)

The recipes in this section turn the preceding ideas into repeatable working methods.

MFCC + MPEG-7 spectral flux

A working command makes the mfcc + mpeg-7 spectral flux concrete.

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \  
  --route pitch_ratio=affine(mfcc_01,0.002,1.0) \  
  --route pitch_ratio=clip(pitch_ratio,0.5,2.0) \  
  --route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \  
  --route stretch=clip(stretch,0.85,1.6) \  
  --output out/mfcc_flux_dual.wav
```

Formant + onset

A working command makes the formant + onset concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(formant_f1_hz,0.0016,0.2) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \
--route stretch=affine(onset_norm,-0.35,1.2) \
--route stretch=clip(stretch,0.8,1.3) \
--output out/formant_onset_dual.wav
```

Beat phase + downbeat phase

A working command makes the beat phase + downbeat phase concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(beat_phase,0.12,0.94) \
--route pitch_ratio=clip(pitch_ratio,0.88,1.12) \
--route stretch=affine(downbeat_phase,0.3,0.85) \
--route stretch=clip(stretch,0.85,1.25) \
--output out/beat_downbeat_dual.wav
```

Tempo + centroid

A working command makes the tempo + centroid concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(tempo_bpm,0.003,0.6) \
--route stretch=clip(stretch,0.8,1.4) \
--route pitch_ratio=affine(centroid_norm,0.4,0.8) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.3) \
--output out/tempo_centroid_dual.wav
```

Stereo cues: interaural level difference + interaural time difference

A working command makes the stereo cues: interaural level difference + interaural time difference concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(ild_db,0.02,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.1) \
--route stretch=affine(itd_ms,0.12,1.0) \
--route stretch=clip(stretch,0.9,1.15) \
--output out/ild_itd_dual.wav
```

Noise-aware: hiss + hum

The example below puts the noise-aware: hiss + hum into executable form.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(hiss_ratio,-0.6,1.2) \
--route stretch=clip(stretch,0.8,1.2) \
--route pitch_ratio=affine(hum_60_ratio,-0.4,1.15) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.2) \
--output out/noise_aware_dual.wav
```

Speech vs music probabilities

The example below puts the speech vs music probabilities into executable form.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(speech_prob,0.5,0.8) \
--route stretch=clip(stretch,0.8,1.35) \
--route pitch_ratio=affine(music_prob,0.2,0.9) \
--route pitch_ratio=clip(pitch_ratio,0.9,1.2) \
--output out/speech_music_dual.wav
```

Formant 2 + spectral spread

A working command makes the formant 2 + spectral spread concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(formant_f2_hz,0.0008,0.5) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.5) \
--route stretch=affine(spectral_spread_hz,0.00008,0.85) \
--route stretch=clip(stretch,0.8,1.4) \
--output out/formant2_spread_dual.wav
```

Loudness proxy + transient mask

The example below puts the loudness proxy + transient mask into executable form.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(short_lufs_db,0.02,1.4) \
--route stretch=clip(stretch,0.75,1.4) \
--route pitch_ratio=affine(transient_mask,0.1,0.95) \
--route pitch_ratio=clip(pitch_ratio,0.95,1.08) \
--output out/lufs_transientmask_dual.wav
```

MPEG-7 centroid + MPEG-7 spread

The example below puts the mpeg-7 centroid + mpeg-7 spread into executable form.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mpeg7_spectral_centroid_hz,0.00022,0.75) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \
--route stretch=affine(mpeg7_spectral_spread_hz,0.00008,0.9) \
--route stretch=clip(stretch,0.8,1.5) \
--output out/mpeg7_centroid_spread.wav
```

MPEG-7 attack-time + MFCC driver

The following session demonstrates the mpeg-7 attack-time + mfcc driver in practice.

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \
--route stretch=affine(mpeg7_log_attack_time_s,0.4,1.0) \
--route stretch=clip(stretch,0.85,1.3) \
--route pitch_ratio=affine(mfcc_03,0.0015,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \
--output out/mpeg7_attack_mfcc.wav
```

13.5 Route Operator Patterns (const, inv, pow, mul, add, affine, clip)

The discussion of the route operator patterns (const, inv, pow, mul, add, affine, clip) begins by separating its main parts.

Start from a feature, then scale and bias

A working command makes the start from a feature, then scale and bias concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route pitch_ratio=pitch_ratio \
--route pitch_ratio=mul(pitch_ratio,0.85) \
--route pitch_ratio=add(pitch_ratio,0.15) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.4) \
--output out/route_mul_add.wav
```

Exaggerate movement with power law

A working command makes the exaggerate movement with power law concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=affine(flux_norm,1.0,0.0) \
--route stretch=pow(stretch,1.8) \
--route stretch=affine(stretch,0.9,0.7) \
--route stretch=clip(stretch,0.8,1.8) \
--route pitch_ratio=const(1.0) \
--output out/route_pow_flux.wav
```

Invert and clamp

A working command makes the invert and clamp concrete.

```
pvx follow guide.wav target.wav --emit pitch_map --stretch 1.0 \
--route stretch=inv(rms_norm) \
--route stretch=clip(stretch,0.8,1.5) \
--route pitch_ratio=const(1.0) \
--output out/route_inv_rms.wav
```

13.6 Feature-Vector Recipes (MFCC + MPEG-7)

The recipes in this section turn the preceding ideas into repeatable working methods.

MFCC second coefficient driver

The following session demonstrates the mfcc second coefficient driver in practice.

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \
--route pitch_ratio=affine(mfcc_02,0.0018,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.7,1.4) \
--output out/mfcc02_pitch.wav
```

MFCC + MPEG-7 audio spectrum envelope band

The following session demonstrates the mfcc + mpeg-7 audio spectrum envelope band in practice.

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 13 --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(mfcc_04,0.0015,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.75,1.35) \  
--route stretch=affine(mpeg7_audio_spectrum_envelope_03,0.08,1.0) \  
--route stretch=clip(stretch,0.85,1.4) \  
--output out/mfcc_envband_dual.wav
```

MPEG-7 spectral crest and decrease

A working command makes the mpeg-7 spectral crest and decrease concrete.

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(mpeg7_spectral_crest,0.03,0.9) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \  
--route stretch=affine(mpeg7_spectral_decrease,-3.0,1.0) \  
--route stretch=clip(stretch,0.85,1.2) \  
--output out/mpeg7_crest_decrease.wav
```

Increase MFCC dimensionality to 20

The example below puts the increase mfcc dimensionality to 20 into executable form.

```
pvx follow guide.wav target.wav --feature-set all --mfcc-count 20 --emit pitch_map --stretch 1.0 \  
--route pitch_ratio=affine(mfcc_10,0.0012,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.8,1.25) \  
--route stretch=affine(mfcc_12,0.03,1.0) \  
--route stretch=clip(stretch,0.9,1.2) \  
--output out/mfcc10_mfcc12_dual.wav
```

Build custom vector features, then route them

The example below puts the build custom vector features, then route them into executable form.

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 20 --output maps/guide_vec.csv  
python3 - <<'PY'  
import csv  
from pathlib import Path  
src = Path('maps/guide_vec.csv')  
dst = Path('maps/guide_vec_aug.csv')  
with src.open() as f_in, dst.open('w', newline='') as f_out:  
    r = csv.DictReader(f_in)  
    fn = list(r.fieldnames or []) + ['mfcc_energy_1_4', 'mfcc_tilt_1_8']  
    w = csv.DictWriter(f_out, fieldnames=fn)  
    w.writeheader()  
    for row in r:  
        m1 = float(row.get('mfcc_01', 0.0) or 0.0)  
        m2 = float(row.get('mfcc_02', 0.0) or 0.0)  
        m3 = float(row.get('mfcc_03', 0.0) or 0.0)  
        m4 = float(row.get('mfcc_04', 0.0) or 0.0)  
        m8 = float(row.get('mfcc_08', 0.0) or 0.0)  
        row['mfcc_energy_1_4'] = 0.25 * (m1 + m2 + m3 + m4)  
        row['mfcc_tilt_1_8'] = m1 - m8  
    w.writerow(row)  
PY  
pvx voc target.wav --pitch-map maps/guide_vec_aug.csv \  
--route pitch_ratio=affine(mfcc_tilt_1_8,0.0018,1.0) \  
--route pitch_ratio=clip(pitch_ratio,0.75,1.4) \  
--route stretch=affine(mfcc_energy_1_4,0.05,1.0) \  
--route stretch=clip(stretch,0.85,1.4) \  
--output out/vector_augmented.wav
```

13.7 Multiple Guide Files (A and B both influence target)

The discussion of the multiple guide files (a and b both influence target) begins by separating its main parts.

Blend pitch behavior from two guides

The following session demonstrates the blend pitch behavior from two guides in practice.

```
pvx pitch-track guide_A.wav --feature-set all --output maps/guide_A.csv
pvx pitch-track guide_B.wav --feature-set all --output maps/guide_B.csv
python3 - <<'PY'
import csv
from pathlib import Path

a = Path('maps/guide_A.csv')
b = Path('maps/guide_B.csv')
out = Path('maps/guide_blend.csv')
with a.open() as fa, b.open() as fb, out.open('w', newline='') as fo:
    ra = csv.DictReader(fa)
    rb = csv.DictReader(fb)
    rows_b = list(rb)
    fields = list(ra.fieldnames or [])
    if 'pitch_ratio' not in fields:
        fields.append('pitch_ratio')
    w = csv.DictWriter(fo, fieldnames=fields)
    w.writeheader()
    rows_b_len = max(1, len(rows_b))
    for i, row_a in enumerate(ra):
        row_b = rows_b[min(i, rows_b_len - 1)]
        pa = float(row_a.get('pitch_ratio', 1.0) or 1.0)
        pb = float(row_b.get('pitch_ratio', 1.0) or 1.0)
        row_a['pitch_ratio'] = 0.6 * pa + 0.4 * pb
        w.writerow(row_a)

PY
pvx voc target.wav --pitch-map maps/guide_blend.csv \
--route pitch_ratio=clip(pitch_ratio,0.7,1.5) \
--route stretch=const(1.0) \
--output out/two_guide_pitch_blend.wav
```

Guide A controls pitch, guide B controls stretch

The example below puts the guide a controls pitch, guide b controls stretch into executable form.

```
pvx pitch-track guide_A.wav --feature-set all --output maps/guide_A.csv
pvx pitch-track guide_B.wav --feature-set all --output maps/guide_B.csv
python3 - <<'PY'
import csv
from pathlib import Path

a = Path('maps/guide_A.csv')
b = Path('maps/guide_B.csv')
out = Path('maps/guide_Apitch_Bstretch.csv')
with a.open() as fa, b.open() as fb, out.open('w', newline='') as fo:
    ra = csv.DictReader(fa)
    rb = csv.DictReader(fb)
    rows_b = list(rb)
    fields = list(dict.fromkeys((ra.fieldnames or []) + ['stretch']))
    w = csv.DictWriter(fo, fieldnames=fields)
    w.writeheader()
    rows_b_len = max(1, len(rows_b))
```

```

    for i, row_a in enumerate(ra):
        row_b = rows_b[min(i, rows_b_len - 1)]
        flux_b = float(row_b.get('spectral_flux', 0.0) or 0.0)
        row_a['stretch'] = max(0.8, min(1.5, 1.0 + 0.03 * flux_b))
    w.writerow(row_a)
PY
pvx voc target.wav --pitch-map maps/guide_Apitch_Bstretch.csv \
--route pitch_ratio=clip(pitch_ratio,0.75,1.35) \
--route stretch=clip(stretch,0.8,1.5) \
--output out/two_guide_split_roles.wav

```

13.8 Manual Pipe Variants

Pitch-track to pvx voc in one pipe:

```

pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - \
| pvx voc target.wav --control-stdin \
--route pitch_ratio=affine(mfcc_01,0.002,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
--route stretch=affine(mpeg7_spectral_flux,0.05,1.0) \
--route stretch=clip(stretch,0.85,1.6) \
--output out/manual_pipe_feature_follow.wav

```

Time-varying control map followed by denoise/deverb in one chain:

```

pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
--route stretch=affine(spectral_flux,0.04,1.0) \
--route stretch=clip(stretch,0.9,1.4) \
--route pitch_ratio=affine(mfcc_01,0.0015,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.85,1.2) \
--output out/follow_stage.wav
pvx denoise out/follow_stage.wav --reduction-db 8 --output out/follow_stage_dn.wav
pvx deverb out/follow_stage_dn.wav --strength 0.35 --output out/follow_stage_dn_dr.wav

```

13.9 Reuse a Saved Feature Map for Fast Iteration

A working command makes the reuse a saved feature map for fast iteration concrete.

```

pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output maps/guide_full.csv
pvx voc target.wav --pitch-map maps/guide_full.csv \
--route pitch_ratio=affine(mfcc_01,0.002,1.0) \
--route pitch_ratio=clip(pitch_ratio,0.5,2.0) \
--route stretch=affine(spectral_flux,0.04,1.0) \
--route stretch=clip(stretch,0.85,1.6) \
--output out/reused_map_follow.wav

pvx voc target.wav --pitch-map maps/guide_full.csv \
--route pitch_ratio=affine(formant_f1_hz,0.0012,0.6) \
--route pitch_ratio=clip(pitch_ratio,0.8,1.4) \
--route stretch=affine(onset_norm,-0.3,1.2) \
--route stretch=clip(stretch,0.8,1.3) \
--output out/reused_map_alt_formula.wav

```

13.10 Compact Feature Set (Smaller CSV)

The example below puts the compact feature set (smaller CSV) into executable form.


```
pvx pitch-track guide.wav --feature-set basic --output maps/guide_basic.csv
pvx voc target.wav --pitch-map maps/guide_basic.csv \
  --route pitch_ratio=affine(spectral_centroid_hz,0.0002,0.8) \
  --route pitch_ratio=clip(pitch_ratio,0.75,1.4) \
  --output out/basic_features_follow.wav
```

13.11 Safe Starting Ranges

When designing formulas, start conservative and widen only if needed:

- pitch_ratio: 0.85 .. 1.15
- stretch: 0.8 .. 1.4
- confidence floor: 0.65 .. 0.85
- map smoothing: --pitch-map-smooth-ms 10..40

Example conservative profile:

```
pvx follow guide.wav target.wav --feature-set all --emit pitch_map --stretch 1.0 \
  --pitch-conf-min 0.8 --pitch-lowconf-mode hold --pitch-map-smooth-ms 20 \
  --route pitch_ratio=affine(mfcc_01,0.001,1.0) \
  --route pitch_ratio=clip(pitch_ratio,0.9,1.1) \
  --route stretch=affine(spectral_flux,0.02,1.0) \
  --route stretch=clip(stretch,0.9,1.2) \
  --output out/conservative_sidechain.wav
```

CHAPTER 14

Follow Workflow Migration

This note describes how to migrate from long pitch-follow shell pipelines to the unified `pvx follow` helper.

14.1 Why This Change

A reliable approach to why this change begins with these points.

- shorter commands for common sidechain workflows
- fewer shell-specific quoting and pipe errors
- same control-map architecture under the hood (`pitch-track + voc --control-stdin`)
- backward compatibility for existing pipelines

14.2 Old vs New

Old (manual pipe):

```
pvx pitch-track A.wav --emit pitch_to_stretch --output - \  
| pvx voc B.wav --control-stdin --pitch-conf-min 0.75 --output B_follow.wav
```

New (one command):

```
pvx follow A.wav B.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output B_follow.wav
```

14.3 When to Keep Manual Pipes

Use explicit pipes when you need:

- custom route rewrites (`--route stretch=pitch_ratio, --route pitch_ratio=const(1.0)`)
- external non-pvx processing between producer and consumer
- shell orchestration with additional non-audio transforms

Equivalent advanced manual path:

```
pvx pitch-track A.wav --output - \  
| pvx voc B.wav --control-stdin \  
  --route stretch=pitch_ratio \  
  --route pitch_ratio=const(1.0) \  
  --output B_manual.wav
```

14.4 Rollout Guidance

Rollout Guidance depends on a few concrete choices.

1. For new scripts, default to `pvx follow`.
2. Keep existing manual pipelines unchanged until they are touched for other reasons.
3. Add regression tests for representative follow jobs (already covered in `tests/test_cli_regression.py`).
4. Keep manual route examples in documentation for advanced control-bus users.

CHAPTER 15

100 Crazy Things To Try With pvx

These are intentionally bold, sometimes impractical ideas for sound-design exploration. Use `pvx stretch-budget` and sensible disk limits before launching extreme renders.

15.1 Extreme Time-Scale Madness

These are the details that matter for extreme time-scale madness.

1. Stretch a one-shot to absurd length:

```
pvx stretch-budget one_shot.wav --disk-budget 50GB --requested-stretch 1000000 --fail-if-exceeds --json &&
pvx voc one_shot.wav --stretch 1000000 --stretch-mode multistage --auto-segment-seconds 0.25 --
checkpoint-dir ckpt --resume --output one_shot_blackhole.wav
```

2. Make a 10-minute ambient tail from a single hit:

```
pvx voc hit.wav --target-duration 600 --preset extreme_ambient --output hit_10min.wav
```

3. Stretch then shrink in two stages for texture hysteresis:

```
pvx chain input.wav --pipeline "voc --stretch 12 | voc --stretch 0.08" --output hysteresis.wav
```

4. Warp tempo with aggressive map swings:

```
pvx warp loop.wav --map maps/warp_chaos.csv --crossfade-ms 10 --output loop_chaos_warp.wav
```

5. Build a staircase of tempo regimes:

```
pvx conform input.wav --map maps/tempo_stairs.csv --output input_tempo_stairs.wav
```

6. Force tiny windows for intentionally crunchy time stretch:

```
pvx voc source.wav --stretch 8 --n-fft 512 --hop-size 64 --output source_gritstretch.wav
```

7. Force oversized windows for smeared macro motion:

```
pvx voc source.wav --stretch 8 --n-fft 8192 --hop-size 256 --output source_glassstretch.wav
```

8. Alternate compress/expand blocks in one chain:

```
pvx chain groove.wav --pipeline "voc --stretch 0.5 | voc --stretch 2.0 | voc --stretch 0.5 | voc --stretch
2.0" --output groove_pumpfold.wav
```

9. Create a “time collapse” impulse sculpture:

```
pvx voc ambience.wav --stretch 0.03 --output ambience_collapse.wav
```

10. Use stateful stream for ultra-long continuity tests:

```
pvx stream long_take.wav --output long_take_stateful_20x.wav --chunk-seconds 0.2 --time-stretch 20
```

15.2 Freeze + Morph Experiments

Before applying freeze + morph experiments, keep these constraints in view.

11. Freeze the loudest drum transient into a pad:

```
pvx freeze drums.wav --freeze-time 0.42 --duration 90 --output drums_freeze_pad.wav
```

12. Freeze a vocal consonant into a synthetic whisper bed:

```
pvx freeze vocal.wav --freeze-time 1.03 --duration 40 --output vocal_consonant_pad.wav
```

13. Morph two unrelated worlds with static alpha:

```
pvx morph piano.wav subway.wav --alpha 0.5 --output piano_subway_50.wav
```

14. Morph by trajectory map from A to B to A:

```
pvx morph a.wav b.wav --alpha controls/alpha_aba.csv --interp linear --output a_b_a_morph.wav
```

15. Use envelope cross-synthesis mode:

```
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_envelope_b --alpha 0.8 --envelope-lifter 36 --output env_cross.wav
```

16. Use mask-driven morph with hard spectral selection:

```
pvx morph source_a.wav source_b.wav --blend-mode carrier_a_mask_b --alpha 0.7 --mask-exponent 1.5 --output mask_cross.wav
```

17. Keep A magnitude but steal B phase attitude:

```
pvx morph source_a.wav source_b.wav --blend-mode magnitude_b_phase_a --alpha 0.65 --phase-mix 0.1 --output phase_cross.wav
```

18. Morph after a freeze for evolving drone color:

```
pvx freeze hit.wav --freeze-time 0.2 --duration 60 --output hit_frozen.wav && pvx morph hit_frozen.wav choir.wav --alpha 0.35 --output frozen_morph.wav
```

19. Morph speech into noise and retune it:

```
pvx morph speech.wav noise.wav --alpha 0.6 --output speech_noise_morph.wav && pvx retune speech_noise_morph.wav --root C --scale minor --strength 1.0 --output speech_noise_retune.wav
```

20. Build a morph ladder at fixed alpha steps for curation:

```
pvx morph a.wav b.wav --alpha controls/alpha_ladder.csv --interp none --output morph_ladder.wav
```

15.3 Pitch, Harmony, and Tuning Extremes

Pitch, Harmony, and Tuning Extremes depends on a few concrete choices.

21. Build a choir from one voice in one command:

```
pvx harmonize lead.wav --intervals 0,4,7,12 --gains 1,0.8,0.7,0.5 --pans -0.4,-0.1,0.1,0.4 --force-stereo --output lead_choir.wav
```

22. Create detuned micro-harmony clusters:

```
pvx harmonize lead.wav --intervals 0,3,7 --intervals-cents 0,7,-11 --gains 1,0.8,0.8 --output lead_microcluster.wav
```

23. Hard retune spoken voice to a scale:

```
pvx retune speech.wav --root D --scale major --strength 1.0 --output speech_tuned.wav
```

24. Gentle retune with minimal intervention:

```
pvx retune vocal.wav --root A --scale minor --strength 0.25 --output vocal_subtle_retune.wav
```

25. Force ratio-based pitch with no stretch change:

```
pvx voc input.wav --stretch 1.0 --ratio 7/4 --output input_ratio_7_4.wav
```

26. Extreme semitone dive with formant-preserving mode:

```
pvx voc vocal.wav --stretch 1.0 --pitch -19 --pitch-mode formant-preserving --output vocal_deep_formant.wav
```

27. Conform to 19-TET map:

```
pvx conform lead.wav --map maps/map_19tet.csv --output lead_19tet.wav
```

28. Conform to just intonation map:

```
pvx conform choir.wav --map maps/map_just_intonation.csv --output choir_ji.wav
```

29. Layer two opposite pitch moves:

```
pvx chain vocal.wav --pipeline "voc --pitch 7 | voc --pitch -12" --output vocal_pitch_fold.wav
```

30. Pitch-shift then freeze at peak weirdness:

```
pvx chain vocal.wav --pipeline "voc --pitch 12 | freeze --freeze-time 0.5 --duration 45" --output vocal_octave_freeze.wav
```

15.4 Transients, Stereo, and Layer Split

The working assumptions for transients, stereo, and layer split are:

31. Run hybrid transient preservation on speech at high stretch:

```
pvx voc speech.wav --transient-mode hybrid --transient-sensitivity 0.65 --stretch 2.5 --output  
speech_hybrid_2_5.wav
```

32. Stretch drums with transient-first preset:

```
pvx voc drums.wav --preset drums_safe --stretch 1.8 --output drums_safe_1_8.wav
```

33. Mid/side lock for stable wide mixes:

```
pvx voc mix.wav --stereo-mode mid_side_lock --coherence-strength 0.95 --stretch 1.4 --output mix_ms_lock.wav
```

34. Intentionally lower coherence for drifting sides:

```
pvx voc stereo.wav --stereo-mode mid_side_lock --coherence-strength 0.2 --stretch 2 --output  
stereo_side_drift.wav
```

35. Split HPSS layers and stretch only harmonics:

```
pvx layer full_mix.wav --harmonic-stretch 2.0 --percussive-stretch 1.0 --output full_mix_harm_stretch.wav
```

36. Split HPSS layers and tighten percussion only:

```
pvx layer beat.wav --harmonic-stretch 1.0 --percussive-stretch 0.7 --output beat_tight_perc.wav
```

37. Pitch harmonics up while lowering percussion gain:

```
pvx layer groove.wav --harmonic-pitch-semitones 5 --percussive-gain 0.7 --output groove_harm_up.wav
```

38. Push transient mode with tiny protect windows:

```
pvx voc drumloop.wav --transient-mode hybrid --transient-protect-ms 8 --transient-crossfade-ms 2 --stretch  
1.5 --output drumloop_snappy.wav
```

39. Push transient mode with huge protect windows:

```
pvx voc drumloop.wav --transient-mode hybrid --transient-protect-ms 80 --transient-crossfade-ms 20 --stretch  
1.5 --output drumloop_smearshield.wav
```

40. Compare wrapper vs stateful stream seam behavior:

```
pvx stream source.wav --mode wrapper --output source_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0 &&  
pvx stream source.wav --mode stateful --output source_stateful.wav --chunk-seconds 0.2 --time-stretch  
2.0
```

15.5 Sidechain Follow and Feature-Driven Control

For sidechain follow and feature-driven control, start with these practical details.

41. Basic pitch-follow stretch transfer:

```
pvx follow guide.wav target.wav --emit pitch_to_stretch --pitch-conf-min 0.75 --output target_follow.wav
```

42. Build pitch map and route manually through voc:

```
pvx pitch-track guide.wav --emit pitch_map --output - | pvx voc target.wav --control-stdin --route  
pitch_ratio=pitch_ratio --output target_manual_follow.wav
```

43. Modulate stretch from spectral flux:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=  
affine(spectral_flux,0.04,1.0) --route stretch=clip(stretch,0.8,1.6) --output target_flux_stretch.wav
```

44. Modulate pitch from MFCC component:

```
pvx pitch-track guide.wav --feature-set all --mfcc-count 13 --output - | pvx voc target.wav --control-stdin  
--route pitch_ratio=affine(mfcc_01,0.002,1.0) --route pitch_ratio=clip(pitch_ratio,0.5,2.0) --output  
target_mfcc_pitch.wav
```

45. Gate effect strength by voicing probability:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=  
affine(voicing_prob,0.4,0.8) --output target_voicing_gate.wav
```

46. Route onset strength into dynamic stretch pull:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=  
affine(onset_strength,0.2,0.9) --route stretch=clip(stretch,0.9,1.3) --output target_onset_push.wav
```

47. Use MPEG-7 spectral descriptors for wild macro motion:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=  
affine(mpeg7_spectral_flux,0.05,1.0) --output target_mpeg_flux.wav
```

48. Build a “rhythm-following pad” from spoken guide:

```
pvx follow speech.wav pad.wav --feature-set all --emit pitch_map --route stretch=affine(transientness  
,0.2,0.9) --output pad_rhythm_follow.wav
```

49. Use low-confidence pitch regions as chaos zones:

```
pvx pitch-track guide.wav --feature-set all --output - | pvx voc target.wav --control-stdin --route stretch=  
affine(confidence,-0.5,1.5) --route stretch=clip(stretch,0.6,2.0) --output target_conf_chaos.wav
```

50. Create sidechain-driven timing from pitch map with manual routing:

```
pvx pitch-track guide.wav --emit pitch_map --output - | pvx voc target.wav --control-stdin --route stretch=  
pitch_ratio --route pitch_ratio=const(1.0) --output target_pitchtime_follow.wav
```

15.6 Control-Map Authoring With Envelope + Reshape

For control-map authoring with envelope + reshape, start with these practical details.

51. Generate ADSR stretch macro map:

```
pvx envelope --mode adsr --duration 12 --rate 20 --attack-sec 0.3 --decay-sec 1.0 --sustain 1.4 --release-sec 2.0 --key stretch --output controls/stretch_adsr.csv
```

52. Generate sine pitch-ratio wobble map:

```
pvx envelope --mode sine --duration 10 --rate 40 --start 1.0 --peak 0.08 --sine-cycles 12 --key pitch_ratio --output controls/pitch_wobble.csv
```

53. Generate exponential ramp map for acceleration:

```
pvx envelope --mode exp --duration 8 --rate 30 --start 0.7 --end 1.8 --exp-curve 6 --key stretch --output controls/stretch_exp.csv
```

54. Normalize a control map to safe range:

```
pvx reshape controls/stretch_raw.csv --key stretch --operation normalize --target-min 0.8 --target-max 1.5 --output controls/stretch_norm.csv
```

55. Resample map to dense control rate:

```
pvx reshape controls/stretch_env.csv --key stretch --operation resample --rate 80 --interp polynomial --order 5 --output controls/stretch_dense.csv
```

56. Smooth noisy map before applying:

```
pvx reshape controls/stretch_noisy.csv --key stretch --operation smooth --window 11 --output controls/stretch_smooth.csv
```

57. Time-shift map late by half second:

```
pvx reshape controls/stretch_env.csv --key stretch --operation time-shift --offset 0.5 --output controls/stretch_shifted.csv
```

58. Time-scale map to half speed modulation:

```
pvx reshape controls/stretch_env.csv --key stretch --operation time-scale --factor 2.0 --output controls/stretch_slowmod.csv
```

59. Invert a modulation map for opposite behavior:

```
pvx reshape controls/mix.csv --key response_mix --operation invert --output controls/mix_inverted.csv
```

60. Build map then apply it in two deterministic steps:

```
pvx envelope --mode adsr --duration 8 --rate 20 --key stretch --output controls/stretch_env.csv && pvx reshape controls/stretch_env.csv --key stretch --operation resample --rate 50 --interp polynomial --order 5 --output controls/stretch_env_dense.csv && pvx voc input.wav --stretch controls/stretch_env_dense.csv --interp linear --output input_envdriven.wav
```

15.7 Analysis/Response Artifact and Spectral Operator Ideas

These are the details that matter for analysis/response artifact and spectral operator ideas.

61. Cache PV analysis for repeated experiments:


```
pvx analysis create source.wav --output source.pvxan.npz --n-fft 4096 --hop-size 256
```

62. Inspect artifact stats for pipeline sanity:

```
pvx analysis inspect source.pvxan.npz --json
```

63. Build median response profile:

```
pvx response create source.pvxan.npz --method median --normalize peak --output source.pvxrf.npz
```

64. Build RMS response profile for denser weighting:

```
pvx response create source.pvxan.npz --method rms --normalize rms --output source_rms.pvxrf.npz
```

65. Apply static spectral imprint:

```
pvx filter target.wav --response source.pvxrf.npz --response-mix 1.0 --output target_imprint.wav
```

66. Animate response imprint over time:

```
pvx tvfilter target.wav --response source.pvxrf.npz --tv-map controls/response_mix.csv --tv-interp linear --output target_tv_imprint.wav
```

67. Use response-referenced noise filtering:

```
pvx noisefilter noisy.wav --response noise_profile.pvxrf.npz --noise-floor 1.2 --output noisy_cleaner.wav
```

68. Exaggerate dominant response peaks:

```
pvx bandamp source.wav --response source.pvxrf.npz --band-gain-db 10 --peak-count 12 --output source_peakpump.wav
```

69. Compress/expand in response-relative spectral space:

```
pvx spec-compander source.wav --response source.pvxrf.npz --comp-threshold-db -22 --comp-ratio 3 --expand-ratio 1.4 --output source_specdyn.wav
```

70. Pitch-shift response curve instead of source pitch:

```
pvx filter source.wav --response source.pvxrf.npz --transpose-semitones 7 --shift-bins 24 --output source_resp_shifted.wav
```

15.8 Ring, Resonator, Chordmapper, Inharmonator

Before applying ring, resonator, chordmapper, inharmonator, keep these constraints in view.

71. Slow ring tremolo with deep mix:

```
pvx ring input.wav --frequency-hz 3.5 --depth 1.0 --mix 1.0 --output input_ring_trem.wav
```

72. Audio-rate ring metallicizer:

```
pvx ring input.wav --frequency-hz 97 --depth 0.95 --mix 0.9 --output input_ring_metal.wav
```

73. Ring with near-chaotic feedback memory:

```
pvx ring input.wav --frequency-hz 43 --depth 1.0 --mix 1.0 --feedback 0.92 --output input_ring_fb.wav
```

74. Ring + resonant peak filter:

```
pvx ringfilter input.wav --frequency-hz 60 --resonance-hz 1600 --resonance-q 12 --resonance-mix 0.6 --output input_ringfilter.wav
```

75. Time-varying ring map performance:

```
pvx ringtvfilter input.wav --tv-map controls/ring_map.csv --tv-interp cubic --output input_ringtv.wav
```

76. Chord-lock noisy ambience to minor harmony:

```
pvx chordmapper ambience.wav --root-hz 110 --chord minor --strength 1.0 --boost-db 8 --attenuation 0.3 --output ambience_minorlock.wav
```

77. Tight chord attraction with low tolerance:

```
pvx chordmapper pad.wav --root-hz 220 --chord diminished --tolerance-cents 12 --output pad_tight_chord.wav
```

78. Soft chord attraction with high tolerance:

```
pvx chordmapper pad.wav --root-hz 220 --chord major7 --tolerance-cents 80 --output pad_soft_chord.wav
```

79. Inharmonic stiff-string emulation:

```
pvx inharmonator bells.wav --inharmonic-f0-hz 220 --inharmonicity 0.0002 --inharmonic-mix 1.0 --output bells_stiff.wav
```

80. Subtle inharmonic shimmer blend:

```
pvx inharmonator pad.wav --inharmonic-f0-hz 110 --inharmonicity 0.00005 --inharmonic-mix 0.25 --dry-mix 0.8 --output pad_shimmer.wav
```

15.9 Multi-Stage Pipelines, Streaming, and Batch Tricks

Before applying multi-stage pipelines, streaming, and batch tricks, keep these constraints in view.

81. One-line cleanup + stretch + formant chain:

```
pvx chain vocal.wav --pipeline "denoise --reduction-db 6 | deverb --strength 0.35 | voc --stretch 1.3 | formant --mode preserve" --output vocal_full_chain.wav
```

82. Stream with explicit output policy controls:

```
pvx stream source.wav --output source_stream.wav --chunk-seconds 0.2 --time-stretch 2.0 --bit-depth 24 --dither tpdf --dither-seed 7 --metadata-policy sidecar
```

83. Stream wrapper compatibility baseline:

```
pvx stream source.wav --mode wrapper --output source_wrapper.wav --chunk-seconds 0.2 --time-stretch 2.0
```

84. Stateful stream driven by stretch map:

```
pvx stream input.wav --output output_stream_map.wav --chunk-seconds 0.2 --stretch controls/stretch.csv --  
interp linear
```

85. Pipe denoise straight into deverb with no temp file:

```
pvx denoise field.wav --reduction-db 5 --smooth 7 --stdout | pvx deverb - --strength 0.35 --output  
field_clean.wav
```

86. Pipe harmonize into voc for post-stack stretch:

```
pvx harmonize lead.wav --intervals 0,4,7 --gains 1,0.75,0.65 --stdout | pvx voc - --stretch 1.5 --output  
lead_stack_stretched.wav
```

87. Chain with alternating transient modes for A/B blend:

```
pvx chain speech.wav --pipeline "voc --transient-mode hybrid --stretch 1.5 | voc --transient-mode off --  
stretch 1.0" --output speech_transient_combo.wav
```

88. Run two-pass pitch: retune then ratio-jump:

```
pvx chain vocal.wav --pipeline "retune --root E --scale minor --strength 0.7 | voc --ratio 9/8" --output  
vocal_two_pass_pitch.wav
```

89. Long-run resume-safe rendering script target:

```
pvx voc long.wav --target-duration 14400 --checkpoint-dir ckpt_long --auto-segment-seconds 0.25 --resume --  
output long_4h.wav
```

90. Dynamic spectral map workflow with reusable artifacts:

```
pvx analysis create source.wav --output source.pvxn.npz && pvx response create source.pvxn.npz --output  
source.pvxrf.npz && pvx tvfilter source.wav --response source.pvxrf.npz --tv-map controls/mix.csv --  
output source_dynspec.wav
```

15.10 QA, Stress, and Reproducibility Challenges

The working assumptions for qa, stress, and reproducibility challenges are:

91. Budget-gate every render before execution:

```
pvx stretch-budget input.wav --disk-budget 10GB --requested-stretch 5000 --fail-if-exceeds
```

92. Compare quality at multiple FFT sizes quickly:

```
pvx chain source.wav --pipeline "voc --stretch 2 --n-fft 1024 | voc --stretch 1 --n-fft 4096" --output  
source_fft_compare.wav
```

93. Build deterministic dithered exports for null tests:

```
pvx voc input.wav --stretch 1.1 --bit-depth 24 --dither tpdf --dither-seed 123 --output out_seeded.wav
```

94. Generate manifest metadata for every run:

```
pvx voc input.wav --stretch 1.08 --manifest-json reports/pvx_runs.json --manifest-append --output out_manifest.wav
```

95. Stress dynamic controls with dense maps:

```
pvx voc input.wav --stretch controls/stretch_dense.csv --interp polynomial --order 5 --output out_densectrl.wav
```

96. Test phase behavior by toggling phase engines:

```
pvx chain source.wav --pipeline "voc --stretch 2 --phase-engine propagate | voc --stretch 1 --phase-engine random" --output source_phase_duel.wav
```

97. Validate sidechain routing with built-in examples:

```
pvx follow --example all
```

98. Run parity benchmark regression module directly:

```
python3 -m unittest tests.test_pvc_parity_benchmark
```

99. Run full regression tests after extreme experiments:

```
python3 -m unittest discover -s tests -p "test_*.py"
```

100. Build a curated “crazy pack” by rendering 100 variants and scoring them with metrics:

```
pvx voc input.wav --stretch 1.2 --output candidate.wav && python3 -m unittest tests.test_cli_regression
```

Tips: - Keep experimental commands in scripts with fixed seeds/options for repeatability. - For very long jobs, always use `--checkpoint-dir` and `--resume`. - Validate impossible requests with `pvx stretch-budget` before running expensive renders.

Part V

Python, ML, and Dataset Work

CHAPTER 16

pvx Application Programming Interface (API) Overview

This guide shows how to use `pvx` as a Python library instead of shell commands.

16.1 Import Paths

If installed with `pip install -e .`:

```
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d
```

From repository source without install:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd() / "src"))
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d
```

16.2 Minimal Time-Stretch Example

A working command makes the minimal time-stretch example concrete.

```
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)

cfg = VocoderConfig(
    n_fft=2048,
    win_length=2048,
    hop_size=512,
    window="hann",
    center=True,
    phase_locking="identity",
    transient_preserve=True,
    transient_threshold=2.0,
)

y = phase_vocoder_time_stretch(mono, 1.25, cfg)
sf.write("output_stretch.wav", y, sr)
```

16.3 Minimal Pitch-Shift (Duration-Preserving)

A working command makes the minimal pitch-shift (duration-preserving) concrete.

```

import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)

semitones = 3.0
ratio = 2 ** (semitones / 12.0)
internal_stretch = ratio

cfg = VocoderConfig(
    n_fft=2048,
    win_length=2048,
    hop_size=512,
    window="hann",
    center=True,
    phase_locking="identity",
    transient_preserve=True,
    transient_threshold=2.0,
)

stretched = phase_vocoder_time_stretch(mono, internal_stretch, cfg)
y = resample_1d(stretched, int(round(stretched.size / ratio)), mode="linear")
sf.write("output_pitch.wav", y, sr)

```

16.4 Jupyter Notebook Snippets

The sources gathered here provide the historical and technical basis for further study.

Spectrogram visualization snippet

The example below puts the spectrogram visualization snippet into executable form.

```

import numpy as np
import matplotlib.pyplot as plt

def db_mag(sig, sr):
    n_fft = 2048
    hop = 512
    win = np.hanning(n_fft)
    frames = []
    for i in range(0, max(1, sig.size - n_fft), hop):
        chunk = sig[i:i+n_fft]
        if chunk.size < n_fft:
            break
        mag = np.abs(np.fft.rfft(chunk * win))
        frames.append(20 * np.log10(mag + 1e-9))
    return np.array(frames).T

S = db_mag(y, sr)
plt.figure(figsize=(10, 4))
plt.imshow(S, origin="lower", aspect="auto", cmap="magma")
plt.colorbar(label="dB")
plt.title("Output spectrogram")
plt.xlabel("Frame")
plt.ylabel("Bin")
plt.tight_layout()

```

Compare phase locking modes

The following session demonstrates the compare phase locking modes in practice.

```
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

cfg_free = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512, window="hann",
    center=True, phase_locking="off", transient_preserve=True, transient_threshold=2.0
)
cfg_lock = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512, window="hann",
    center=True, phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

y_free = phase_vocoder_time_stretch(mono, 1.25, cfg_free)
y_lock = phase_vocoder_time_stretch(mono, 1.25, cfg_lock)
```

16.5 Batch Processing Example (Python)

A working command makes the batch processing example (python) concrete.

```
from pathlib import Path
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch

cfg = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512,
    window="hann", center=True,
    phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

inp = Path("stems")
out = Path("out_stems")
out.mkdir(exist_ok=True)

for path in sorted(inp.glob("*.wav")):
    x, sr = sf.read(path, always_2d=True)
    y_channels = []
    for ch in range(x.shape[1]):
        y_channels.append(phase_vocoder_time_stretch(x[:, ch], 1.08, cfg))
    n = max(len(ch) for ch in y_channels)
    y = np.zeros((n, len(y_channels)), dtype=np.float64)
    for idx, ch in enumerate(y_channels):
        y[:len(ch), idx] = ch
    sf.write(out / f"{path.stem}_pv.wav", y, sr)
```

16.6 Segment Processing Example (Python)

The following session demonstrates the segment processing example (python) in practice.


```

from dataclasses import dataclass
import numpy as np
import soundfile as sf
from pvx.core.voc import VocoderConfig, phase_vocoder_time_stretch, resample_1d

@dataclass
class Segment:
    start_sec: float
    end_sec: float
    stretch: float
    pitch_ratio: float

segments = [
    Segment(0.0, 1.0, 1.00, 1.00),
    Segment(1.0, 2.0, 1.20, 1.00),
    Segment(2.0, 3.0, 0.95, 2 ** (2/12)),
]

x, sr = sf.read("input.wav", always_2d=True)
mono = np.mean(x, axis=1)
cfg = VocoderConfig(
    n_fft=2048, win_length=2048, hop_size=512,
    window="hann", center=True,
    phase_locking="identity", transient_preserve=True, transient_threshold=2.0
)

parts = []
for seg in segments:
    s = int(round(seg.start_sec * sr))
    e = int(round(seg.end_sec * sr))
    if e <= s:
        continue
    block = mono[s:e]
    internal = seg.stretch * seg.pitch_ratio
    z = phase_vocoder_time_stretch(block, internal, cfg)
    if abs(seg.pitch_ratio - 1.0) > 1e-9:
        z = resample_1d(z, int(round(z.size / seg.pitch_ratio)), mode="linear")
    parts.append(z)

y = np.concatenate(parts) if parts else np.zeros(0, dtype=np.float64)
sf.write("segment_output.wav", y, sr)

```

16.7 Related CLI Equivalents

For related cli equivalents, start with these practical details.

- `pvx voc ...` for direct production workflows.
- `pvx voc --explain-plan` to inspect resolved processing settings.
- `pvx voc --manifest-json ...` to log run metadata.

CHAPTER 17

ML/DL Framework Integration Guide

Alpha release (0.1.0a1) , `pvx.augment` is under active development. Public interfaces may change between minor versions until 1.0. Pin your dependency to an exact version (`pvx==0.1.0a1`) if you need stability, and please report issues on GitHub.

`pvx` ships a first-class Python API for audio data augmentation (`pvx.augment`) and thin, zero-boilerplate adapters for the three most popular deep-learning ecosystems.

17.1 Contents

For contents, start with these practical details.

1. Installation
 2. Quick Start , Pure Python
 3. PyTorch Integration
 4. HuggingFace Datasets Integration
 5. TensorFlow / tf.data Integration
 6. Determinism and Reproducibility
 7. Performance Tips
 8. GPU-Accelerated Transforms (PyTorch)
 9. Available Transforms
-

17.2 Installation

The example below puts the installation into executable form.

```
# Core DSP + augmentation (no ML framework required)
\index{Core DSP + augmentation (no ML framework required)}
pip install pvx

# With PyTorch
\index{With PyTorch}
pip install "pvx[torch]"

# With HuggingFace
\index{With HuggingFace}
pip install "pvx[huggingface]"

# With TensorFlow
\index{With TensorFlow}
pip install "pvx[tensorflow]"

# All ML extras
\index{All ML extras}
pip install "pvx[ml]"
```

17.3 Quick Start , Pure Python

The following session demonstrates the quick start , pure python in practice.

```
import soundfile as sf
from pvx.augment import (
    Pipeline,
    GainPerturber,
    RoomSimulator,
    AddNoise,
    SpecAugment,
    CodecDegradation,
)

# Load audio (any format supported by soundfile)
\index{Load audio (any format supported by soundfile)}
audio, sr = sf.read("speech.wav", dtype="float32", always_2d=False)

# Build an augmentation pipeline
\index{Build an augmentation pipeline}
pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    RoomSimulator(rt60_range=(0.05, 0.8), wet_range=(0.1, 0.5), p=0.4),
    AddNoise(snr_db=(15, 35), noise_type="pink", p=0.5),
    CodecDegradation(codec="random", p=0.2),
    SpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

# Apply with a reproducible seed
\index{Apply with a reproducible seed}
audio_aug, sr_out = pipeline(audio, sr, seed=0)

# Save result
\index{Save result}
sf.write("speech_aug.wav", audio_aug, sr_out)
```

17.4 Intent Preset Pipelines

Ready-made pipelines for common research use cases:

```
from pvx.augment import asr_pipeline, music_pipeline, speech_enhancement_pipeline, contrastive_pipeline

# ASR robustness training
\index{ASR robustness training}
pipeline = asr_pipeline(seed=42)

# Music information retrieval
\index{Music information retrieval}
pipeline = music_pipeline(seed=42)

# Speech enhancement (noisy/reverberant input, clean target)
\index{Speech enhancement (noisy/reverberant input clean target)}
noisy_pipeline = speech_enhancement_pipeline(seed=42)

# Contrastive self-supervised learning (two correlated views)
\index{Contrastive self-supervised learning (two correlated views)}
pipeline_a, pipeline_b = contrastive_pipeline(seed=42)
clean_audio, sr = sf.read("speech.wav", dtype="float32")
view_a, _ = pipeline_a(clean_audio, sr, seed=idx)
```

```
view_b, _ = pipeline_b(clean_audio, sr, seed=idx)
```

17.5 PyTorch Integration

The account of the pytorch integration proceeds through the topics that follow.

PvxAugmentDataset

The example below puts the pvxaugmentdataset into executable form.

```
import torch
from torch.utils.data import DataLoader
from pvx.augment import Pipeline, AddNoise, RoomSimulator, SpecAugment
from pvx.integrations.pytorch import PvxAugmentDataset

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
    SpecAugment(freq_mask_param=30, time_mask_param=50, p=0.5),
], seed=42)

file_list = ["data/train/speech_001.wav", "data/train/speech_002.wav", ...]
labels    = [0, 1, ...] # optional

dataset = PvxAugmentDataset(
    file_list,
    pipeline=pipeline,
    sample_rate=16000,
    labels=labels,
    duration_s=3.0,      # crop/pad all clips to 3 seconds
    mono=True,
)

loader = DataLoader(
    dataset.as_torch_dataset(),
    batch_size=32,
    num_workers=4,
    collate_fn=PvxAugmentDataset.collate_fn,
    pin_memory=True,
)

for batch in loader:
    audio = batch["audio"] # (B, T) or (B, C, T)
    label = batch["label"] # (B,)
    lengths = batch["lengths"] # (B,) , true clip lengths before padding
    # ... training step ...
    \index{training step}
```

PvxAugmentTransform (torchvision-style)

Use as a drop-in transform in any torchaudio-compatible dataset:

```

from pvx.integrations.pytorch import PvxAugmentTransform

transform = PvxAugmentTransform(pipeline, sample_rate=16000)

# Use with torchaudio
\index{Use with torchaudio}
import torchaudio
ds = torchaudio.datasets.LIBRISPEECH(
    root="data/", url="train-clean-100",
    download=True,
)
# Apply transform manually:
\index{Apply transform manually}
audio, sr, transcript, *_ = ds[0]
audio_aug = transform(audio)

```

AudioCollator for variable-length batches

The example below puts the audiocollator for variable-length batches into executable form.

```

from pvx.integrations.pytorch import AudioCollator

loader = DataLoader(
    dataset.as_torch_dataset(),
    batch_size=16,
    collate_fn=AudioCollator(return_lengths=True),
)

for audio_batch, lengths in loader:
    # audio_batch: (B, C, T_max) padded to longest clip
    \index{audio batch (B C T max) padded to longest clip}
    # lengths: (B,) true lengths
    \index{lengths (B ) true lengths}
    pass

```

17.6 HuggingFace Datasets Integration

The account of the huggingface datasets integration proceeds through the topics that follow.

Basic map() usage

The following session demonstrates the basic map() usage in practice.

```

from datasets import load_dataset
from pvx.augment import Pipeline, AddNoise, RoomSimulator
from pvx.integrations.huggingface import make_augment_map_fn

ds = load_dataset("mozilla-foundation/common_voice_13_0", "en", split="train")

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
], seed=42)

# HuggingFace Audio feature rows have format: {"array": np.ndarray, "sampling_rate": int}
\index{HuggingFace Audio feature rows have format array np.ndarray sampling rate int}
augment_fn = make_augment_map_fn(
    pipeline,

```

```
    audio_column="audio",
    output_column="audio_aug", # write to new column (keeps original)
    base_seed=42,
)

ds_aug = ds.map(
    augment_fn,
    batched=False,
    with_indices=True, # required for per-row seed
    num_proc=8,
)
```

Contrastive view generation (SSL)

A working command makes the contrastive view generation (ssl) concrete.

```
from pvx.integrations.huggingface import HFAugmentMapper
from pvx.augment import contrastive_pipeline

pipeline_a, pipeline_b = contrastive_pipeline(seed=42)

# Generate both views in a single map pass
\index{Generate both views in a single map pass}
mapper_a = HFAugmentMapper(pipeline_a, audio_column="audio", output_prefix="view_a")
mapper_b = HFAugmentMapper(pipeline_b, audio_column="audio", output_prefix="view_b")

# Or use built-in pair mode
\index{Or use built-in pair mode}
from pvx.augment import music_pipeline
mapper = HFAugmentMapper(
    music_pipeline(seed=42),
    audio_column="audio",
    generate_pair=True,
    output_prefix="view",
)
ds_ssl = ds.map(mapper, batched=False, with_indices=True)
# ds_ssl["view_a"]["array"], ds_ssl["view_b"]["array"]
\index{ds ssl view a array ds ssl view b array}
```

One-liner convenience

The example below puts the one-liner convenience into executable form.

```
from pvx.integrations.huggingface import augment_dataset

ds_aug = augment_dataset(ds, pipeline, audio_column="audio", num_proc=8)
```

Saving augmented datasets

A working command makes the saving augmented datasets concrete.

```
# Save to disk for fast reloading
\index{Save to disk for fast reloading}
ds_aug.save_to_disk("data/augmented_librispeech")

# Or push to HuggingFace Hub
\index{Or push to HuggingFace Hub}
ds_aug.push_to_hub("my-org/librispeech-augmented")
```

17.7 TensorFlow / tf.data Integration

Several distinct concerns meet in the tensorflow / tf.data integration; they are taken in turn below.

Tensor map function

The example below puts the tensor map function into executable form.

```
import tensorflow as tf
from pvx.augment import Pipeline, AddNoise, RoomSimulator
from pvx.integrations.tensorflow import make_tf_augment_fn, pvx_augment_tf_dataset

pipeline = Pipeline([
    AddNoise(snr_db=(10, 30), p=0.5),
    RoomSimulator(rt60_range=(0.1, 0.8), p=0.4),
], seed=42)

# From raw tensor dataset
\index{From raw tensor dataset}
ds = tf.data.Dataset.from_tensor_slices(audio_array) # shape: (N, T)
augment_fn = make_tf_augment_fn(pipeline, sample_rate=16000)
ds_aug = ds.map(augment_fn, num_parallel_calls=tf.data.AUTOTUNE)

# From dict dataset (e.g. TF-Datasets)
\index{From dict dataset (e.g. TF-Datasets)}
import tensorflow_datasets as tfds
tfds_raw = tfds.load("speech_commands", split="train")
ds_aug = pvx_augment_tf_dataset(tfds_raw, pipeline, audio_key="audio")
```

Dict-based map for keyed datasets

The following session demonstrates the dict-based map for keyed datasets in practice.

```
from pvx.integrations.tensorflow import make_tf_augment_map_fn

fn = make_tf_augment_map_fn(
    pipeline,
    audio_key="audio",
    output_key="audio_aug",
    default_sr=16000,
)
ds_aug = ds.map(fn, num_parallel_calls=tf.data.AUTOTUNE)
```

17.8 Determinism and Reproducibility

All pvx transforms accept a **seed** parameter that fully determines their output. The seed hierarchy is:

```
global_seed + item_index -> per-item seed
per-item seed + transform_index -> per-transform seed
```

This means: - Same **seed** + same input -> identical output (byte-for-byte reproducible) - Different item indices produce different augmentations even with the same global seed - Validation sets can be deterministically generated by fixing the seed

```
# Training: random seed per epoch for variety
\index{Training random seed per epoch for variety}
audio_aug, _ = pipeline(audio, sr, seed=epoch * n_items + item_idx)

# Validation: fixed seed for reproducible evaluation
\index{Validation fixed seed for reproducible evaluation}
audio_aug, _ = pipeline(audio, sr, seed=99999 + item_idx)
```

17.9 Performance Tips

The account of the performance tips proceeds through the topics that follow.

Offline pre-computation (recommended for large datasets)

The example below puts the offline pre-computation (recommended for large datasets) into executable form.

```
# Generate the augmented dataset once and save it
\index{Generate the augmented dataset once and save it}
pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --variants-per-input 4 \
  --intent asr_robust \
  --seed 1337 \
  --workers 16
```

Then load pre-augmented files directly , eliminates augmentation overhead at training time.

Online augmentation with worker processes

A working command makes the online augmentation with worker processes concrete.

```
# PyTorch: set num_workers >= 4 , each worker gets a separate process
\index{PyTorch set num workers 4 each worker gets a separate process}
loader = DataLoader(dataset.as_torch_dataset(), num_workers=8, ...)

# HuggingFace: use num_proc
\index{HuggingFace use num proc}
ds_aug = ds.map(augment_fn, num_proc=8)

# TensorFlow: use AUTOTUNE
\index{TensorFlow use AUTOTUNE}
ds_aug = ds.map(fn, num_parallel_calls=tf.data.AUTOTUNE)
```

Caching expensive transforms

For transforms like RoomSimulator that are computationally intensive, cache intermediate results:

```
# HuggingFace , cache augmented dataset after first run
\index{HuggingFace cache augmented dataset after first run}
ds_aug = ds.map(augment_fn, num_proc=8, cache_file_name="data/cache/aug.arrow")
```

TimeStretch and PitchShift , Engine Selection

TimeStretch and PitchShift support five processing engines via the `engine` parameter:

Engine	Requires	Notes
"auto" (default)	,	Prefers torchaudio > pytorch > pvx-cli
"torchaudio"	<code>pip install "pvx[torch]"</code>	Optimized C++ phase-vocoder kernel via torchaudio
"pytorch"	<code>pip install "pvx[torch]"</code>	Vectorized phase-vocoder in pure PyTorch
"wavelet"	built-in Haar; optional <code>pip install PyWavelets</code>	Experimental multiband wavelet backend
"pvx-cli"	<code>pvx CLI</code> on PATH	Full pvxvoc DSP stack via subprocess

```

from pvx.augment import TimeStretch, PitchShift

# Auto-select best available engine (recommended)
\index{Auto-select best available engine (recommended)}
ts = TimeStretch(rate=(0.8, 1.2), engine="auto")

# Force torchaudio engine (fastest, requires torchaudio)
\index{Force torchaudio engine (fastest requires torchaudio)}
ts = TimeStretch(rate=(0.8, 1.2), engine="torchaudio")

# Force PyTorch engine (no torchaudio needed, just torch)
\index{Force PyTorch engine (no torchaudio needed just torch)}
ts = TimeStretch(rate=(0.8, 1.2), engine="pytorch")

# Force experimental wavelet engine.
\index{Force experimental wavelet engine}
# wavelet="auto" uses PyWavelets db4 when available, otherwise built-in Haar.
\index{wavelet auto uses PyWavelets db4 when available otherwise built-in Haar}
ts = TimeStretch(rate=(0.8, 1.2), engine="wavelet", wavelet="auto", wavelet_levels=3)
ps = PitchShift(semitones=(-2, 2), engine="wavelet", wavelet="haar", wavelet_levels=4)

# Force pvx CLI subprocess (full DSP stack: transients, formants, stereo)
\index{Force pvx CLI subprocess (full DSP stack transients formants stereo)}
ts = TimeStretch(rate=(0.8, 1.2), engine="pvx-cli")

```

The CLI `pvx augment` command also accepts `--engine`:

```

pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --engine torchaudio \
  --intent asr_robust

```

To compare backend speed and simple pitch/duration smoke metrics:

```
python benchmarks/run_backend_compare.py --engines pytorch,pvx-cli,wavelet --wavelet auto
```

If no engine is available, `TimeStretch` and `PitchShift` will raise a `RuntimeError` with a clear message, they will **not** silently return unmodified audio.

For a lightweight, pure-Python pitch shift that requires no external engine, use `PitchShiftSimple` (spectral bin interpolation, ± 1 -3 semitone approximation).

Streaming Augmentation for Long-Form Audio

For podcasts, audiobooks, and other long-form content, use the streaming API to process files in chunks with bounded memory:

```
from pvx.augment import stream_augment_file, Pipeline, AddNoise, GainPerturber

pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    AddNoise(snr_db=(15, 35), noise_type="pink", p=0.5),
], seed=42)

# Process a 2-hour podcast in 30-second chunks
\index{Process a 2-hour podcast in 30-second chunks}
stream_augment_file(
    "podcast_2h.wav",
    "podcast_2h_aug.wav",
    pipeline=pipeline,
    chunk_duration_s=30.0,
    seed=42,
)
```

Impulse Response Database

For physics-based room acoustics, use real impulse responses instead of synthetic approximations:

```
from pvx.augment import IRDatabase, ImpulseResponseConvolver

db = IRDatabase()
db.download("echothief") # 115 real-world IRs, cached locally

# Use all IRs
\index{Use all IRs}
aug = ImpulseResponseConvolver(db.ir_dir("echothief"), wet_range=(0.4, 1.0))

# Or filter by category
\index{Or filter by category}
halls = db.filter("echothief", category="hall")
aug = ImpulseResponseConvolver(halls, wet_range=(0.5, 0.9))
```

Mix online and offline augmentation

Use the CLI for heavy transforms (time-stretch, pitch-shift) and the Python API for lightweight ones (gain, noise, SpecAugment) at runtime:

```
# Heavy transforms pre-computed offline
\index{Heavy transforms pre-computed offline}
ds_heavy = load_from_disk("data/train_heavy_aug")

# Lightweight transforms applied online
\index{Lightweight transforms applied online}
light_pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.8),
    AddNoise(snr_db=(15, 35), noise_type="white", p=0.4),
    SpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

fn = make_augment_map_fn(light_pipeline, audio_column="audio")
ds_final = ds_heavy.map(fn, batched=False, with_indices=True, num_proc=4)
```

17.10 GPU-Accelerated Transforms (PyTorch)

For maximum throughput during GPU training, `pvx.augment.gpu` provides native PyTorch transforms that operate directly on `torch.Tensor` objects, no NumPy round-tripping required. These support batched operation and run on CUDA, MPS, or CPU.

```
import torch
from pvx.augment.gpu import (
    TorchPipeline,
    TorchGainPerturber,
    TorchAddNoise,
    TorchEQPerturber,
    TorchSpecAugment,
    TorchNormalizer,
    TorchClippingSimulator,
    TorchTimeStretch,
    TorchPitchShift,
    TorchRoomSimulator,
    TorchMixup,
    NumpyTransformAdapter, # wrap any NumPy transform for GPU use
)

# Build a GPU-native pipeline
\index{Build a GPU-native pipeline}
gpu_pipeline = TorchPipeline([
    TorchGainPerturber(gain_db=(-6, 6), p=0.8),
    TorchAddNoise(snr_db=(10, 30), noise_type="pink", p=0.5),
    TorchEQPerturber(n_bands=4, gain_db_range=(-6, 6), p=0.4),
    TorchSpecAugment(freq_mask_param=27, time_mask_param=100, p=0.5),
], seed=42)

# Apply to a batch of tensors (B, C, T)
\index{Apply to a batch of tensors (B C T)}
audio_batch = torch.randn(32, 1, 48000).cuda()
audio_aug = gpu_pipeline(audio_batch, sr=16000)
```

Mixing GPU and NumPy transforms

Use `NumpyTransformAdapter` to wrap any NumPy-based `pvx` transform for use in a `TorchPipeline`. The adapter moves data to CPU per-sample, so native `Torch*` transforms are preferred for the hot path:

```
from pvx.augment import CodecDegradation

gpu_pipeline = TorchPipeline([
    TorchGainPerturber(gain_db=(-6, 6)),
    NumpyTransformAdapter(CodecDegradation(codec="random", p=0.2)),
    TorchSpecAugment(freq_mask_param=27),
])
```

Available GPU transforms

A tabular view makes the distinctions within the available GPU transforms easier to inspect.

Transform	Equivalent NumPy transform	Notes
<code>TorchGainPerturber</code>	<code>GainPerturber</code>	Batched random gain
<code>TorchAddNoise</code>	<code>AddNoise</code>	white/pink/brown via FFT on GPU

Transform	Equivalent NumPy transform	Notes
<code>TorchEQPerturber</code>	<code>EQPerturber</code>	STFT-domain parametric EQ
<code>TorchSpecAugment</code>	<code>SpecAugment</code>	Frequency + time masking
<code>TorchNormalizer</code>	<code>Normalizer</code>	Peak/RMS normalization
<code>TorchClippingSimulator</code>	<code>ClippingSimulator</code>	Hard/soft clipping
<code>TorchTimeStretch</code>	<code>TimeStretch</code>	Phase-vocoder time stretch on GPU
<code>TorchPitchShift</code>	<code>PitchShift</code>	Pitch shift via stretch + resample
<code>TorchRoomSimulator</code>	<code>RoomSimulator</code>	Synthetic reverb approximation
<code>TorchMixup</code>	,	Zhang et al. 2018 batch mixing
<code>NumpyTransformAdapter</code>	any Transform	CPU fallback wrapper

17.11 Available Transforms

The available transforms records summarize what each transform controls and where it is useful.

Transform: `AddNoise`. **Module:** `noise`. **Key Parameters:** `snr_db`, `noise_type`. **Use Case:** ASR, VAD, denoising.

Transform: `BackgroundMixer`. **Module:** `noise`. **Key Parameters:** `background_sources`, `snr_db`. **Use Case:** Real-world noise robustness.

Transform: `ImpulseNoise`. **Module:** `noise`. **Key Parameters:** `rate`, `amplitude_range`. **Use Case:** Transmission artifact simulation.

Transform: `RoomSimulator`. **Module:** `room`. **Key Parameters:** `rt60_range`, `wet_range`. **Use Case:** Synthetic reverb approximation for augmentation.

Transform: `ImpulseResponseConvolver`. **Module:** `room`. **Key Parameters:** `ir_sources`, `wet_range`. **Use Case:** Real IR convolution.

Transform: `CodecDegradation`. **Module:** `codec`. **Key Parameters:** `codec`. **Use Case:** Codec-robust ASR, telephony.

Transform: `BitCrusher`. **Module:** `codec`. **Key Parameters:** `bits`. **Use Case:** Lo-fi, retro effects.

Transform: `BandwidthLimiter`. **Module:** `codec`. **Key Parameters:** `cutoff_hz`. **Use Case:** Telephone/radio simulation.

Transform: `SpecAugment`. **Module:** `spectral`. **Key Parameters:** `freq_mask_param`, `time_mask_param`. **Use Case:** ASR, keyword spotting.

Transform: `EQPerturber`. **Module:** `spectral`. **Key Parameters:** `n_bands`, `gain_db_range`. **Use Case:** Microphone/room coloration.

Transform: `SpectralNoise`. **Module:** `spectral`. **Key Parameters:** `noise_std_range`. **Use Case:** Feature extractor robustness.

Transform: `GainPerturber`. **Module:** `time_domain`. **Key Parameters:** `gain_db`. **Use Case:** Loudness invariance.

Transform: `Normalizer`. **Module:** `time_domain`. **Key Parameters:** `mode`, `target_db`. **Use Case:** Pre-processing.

Transform: ClippingSimulator. **Module:** time_domain. **Key Parameters:** percentile, mode. **Use Case:** ADC artifact simulation.

Transform: TimeShift. **Module:** time_domain. **Key Parameters:** shift. **Use Case:** Temporal invariance.

Transform: Reverse. **Module:** time_domain. **Key Parameters:** , . **Use Case:** Data doubling.

Transform: Fade. **Module:** time_domain. **Key Parameters:** fade_in, fade_out. **Use Case:** Boundary artifact reduction.

Transform: TrimSilence. **Module:** time_domain. **Key Parameters:** threshold_db. **Use Case:** Pre-processing.

Transform: FixedLengthCrop. **Module:** time_domain. **Key Parameters:** duration_s. **Use Case:** Batching fixed-size inputs.

Transform: TimeStretch. **Module:** time_domain. **Key Parameters:** rate, preset, engine, wavelet, wavelet_levels. **Use Case:** Tempo invariance (auto/torchaudio/pytorch/wavelet/pvx-cli).

Transform: PitchShift. **Module:** time_domain. **Key Parameters:** semitones, formant_mode, engine, wavelet, wavelet_levels. **Use Case:** Pitch invariance (auto/torchaudio/pytorch/wavelet/pvx-cli).

Transform: stream_augment_file. **Module:** streaming. **Key Parameters:** chunk_duration_s, overlap_s. **Use Case:** Memory-bounded long-form processing.

Transform: IRDatabase. **Module:** ir_database. **Key Parameters:** cache_dir. **Use Case:** Real IR collection manager.

Transform: PitchShiftSimple. **Module:** spectral. **Key Parameters:** semitones. **Use Case:** Lightweight pitch approximation.

Transform: Pipeline. **Module:** core. **Key Parameters:** transforms, seed. **Use Case:** Sequential composition.

Transform: OneOf. **Module:** core. **Key Parameters:** transforms, weights. **Use Case:** Random selection.

Transform: SomeOf. **Module:** core. **Key Parameters:** transforms, k. **Use Case:** Random subset.

Transform: RandomApply. **Module:** core. **Key Parameters:** transform, p. **Use Case:** Stochastic application.

CHAPTER 18

AI Research and Data Augmentation with pvx

`pvx augment` provides deterministic, manifest-driven audio augmentation for machine-learning workflows.

Primary use cases: - automatic speech recognition (ASR) robustness experiments - music information retrieval (MIR) augmentation studies - self-supervised learning (SSL) contrastive view generation - ablation studies where exact augmentation parameters must be replayed

18.1 Core Command

This reference introduces the core command before presenting its individual entries.

```
pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
```

What this does: - resolves input files/globs/directories - renders N deterministic variants per input (`--variants-per-input`) - writes per-variant metadata to: - `augment_manifest.jsonl` - `augment_manifest.csv`

Manifest tool companion:

```
pvx augment-manifest validate aug_out/augment_manifest.jsonl --strict
```

18.2 Intent Profiles

The table below gathers the principal details of the intent profiles.

Intent	Typical target	Behavior
<code>asr_robust</code>	Speech pipelines	Mild time/pitch perturbation with conservative vocal/formant defaults
<code>mir_music</code>	Music analytics	Moderate timing/pitch/spectral variation suited for musical content
<code>ssl_contrastive</code>	Contrastive representation learning	Wider perturbation envelope for view diversity while remaining reproducible

18.3 Reproducibility Controls

The table below gathers the principal details of the reproducibility controls.

Option	Meaning
<code>--seed</code>	Global deterministic seed for all sampled parameters
<code>--split train,val,test</code>	Deterministic split assignment ratios written into manifest
<code>--split-mode</code>	Split strategy: <code>random</code> , <code>label_balanced</code> , <code>speaker_balanced</code>
<code>--labels-csv</code>	Metadata table with <code>path/label/speaker</code> fields for balanced split modes
<code>--grouping</code>	Split assignment grouping strategy (<code>stem-prefix</code> or <code>none</code>)
<code>--group-separator</code>	Prefix separator for grouping (default: <code>__</code>)
<code>--pair-mode</code>	Pair-view mode: <code>off</code> or <code>contrastive2</code>
<code>--label-policy</code>	Perturbation policy: <code>allow_alter</code> or <code>preserve</code>
<code>--policy</code>	JSON policy file with reproducible defaults and parameter bounds
<code>--workers</code>	Parallel render workers with deterministic seed behavior
<code>--resume</code>	Skip previously rendered outputs using existing manifests/output files
<code>--append-manifest</code>	Merge with existing manifests rather than replace
<code>--dry-run</code>	Plan outputs and write manifests without rendering audio
<code>--manifest-jsonl / --manifest-csv</code>	Explicit manifest output paths

18.4 Manifest Fields

Each JSON Lines (JSONL) row contains:

- `source_path`
- `output_path`
- `intent`
- `seed`
- `split` (`train`, `val`, `test`)
- `group_key` (split-group identifier)
- `status` (`planned`, `rendered`, or `error:<code>`)
- `source_sha256`, `output_sha256`
- `params` object, including:
 - `stretch`
 - `pitch`
 - `preset`
 - `window`
 - `transform`
 - `formant_strength`
 - `transient_sensitivity`
 - `target_lufs`
- optional `audit` object:

```
- duration_sec
- peak_dbfs
- rms_dbfs
- clip_pct
- zcr
```

CSV manifest includes the same essential fields in tabular form.

18.5 Example Workflows

The discussion of the example workflows begins by separating its main parts.

Speech Robustness Set

A working command makes the speech robustness set concrete.

```
pvx augment corpus/speech/*.wav \
  --output-dir data_aug/speech \
  --variants-per-input 6 \
  --intent asr_robust \
  --grouping stem-prefix \
  --group-separator "__" \
  --split 0.8,0.1,0.1 \
  --seed 1337
```

Music Information Retrieval Set

The example below puts the music information retrieval set into executable form.

```
pvx augment corpus/music/**/*.wav \
  --output-dir data_aug/music \
  --variants-per-input 4 \
  --intent mir_music \
  --split-mode label_balanced \
  --labels-csv labels.csv \
  --split 0.7,0.2,0.1 \
  --seed 2026
```

Contrastive Planning-Only Pass

The following session demonstrates the contrastive planning-only pass in practice.

```
pvx augment corpus/*.wav \
  --output-dir data_aug/plan_only \
  --variants-per-input 3 \
  --intent ssl_contrastive \
  --pair-mode contrastive2 \
  --dry-run \
  --seed 42
```

With explicit manifest paths

A working command makes the with explicit manifest paths concrete.


```
pvx augment corpus/*.wav \
  --output-dir data_aug/run_a \
  --variants-per-input 5 \
  --intent asr_robust \
  --label-policy preserve \
  --policy policies/asr_policy.json \
  --manifest-jsonl reports/run_a_manifest.jsonl \
  --manifest-csv reports/run_a_manifest.csv \
  --seed 9001
```

Resume and append

The following session demonstrates the resume and append in practice.

```
pvx augment corpus/*.wav \
  --output-dir data_aug/run_resume \
  --variants-per-input 5 \
  --intent mir_music \
  --resume \
  --append-manifest \
  --seed 9001
```

Manifest merge + stats

The example below puts the manifest merge + stats into executable form.

```
pvx augment-manifest merge \
  reports/run_a_manifest.jsonl reports/run_b_manifest.jsonl \
  --output-jsonl reports/merged_manifest.jsonl \
  --output-csv reports/merged_manifest.csv

pvx augment-manifest stats reports/merged_manifest.jsonl
```

18.6 Policy File Template

A working command makes the policy file template concrete.

```
{
  "augment": {
    "intent": "asr_robust",
    "variants_per_input": 4,
    "split": "0.8,0.1,0.1",
    "split_mode": "speaker_balanced",
    "grouping": "stem-prefix",
    "group_separator": "_-",
    "pair_mode": "off",
    "label_policy": "preserve",
    "workers": 4,
    "device": "cpu",
    "output_format": "wav",
    "bounds": {
      "stretch": [0.94, 1.08],
      "pitch": [-1.0, 1.0],
      "formant_strength": [0.60, 0.92],
      "transient_sensitivity": [0.50, 0.75],
      "target_lufs": [-24.0, -16.0]
    },
  },
  "choices": {
    "window": ["hann", "hamming"],
```

```

    "transform": ["fft", "dft"],
    "preset": ["vocal_studio"]
  }
}
}

```

18.7 Why Augmentation Works

Audio augmentation changes the training distribution without requiring a new recording session. Its purpose is not to make a corpus look large on disk. Its purpose is to expose the model to variations that should not change the desired decision, while retaining the acoustic evidence that should change that decision. This distinction separates principled augmentation from arbitrary signal damage.

For supervised learning, a useful starting objective is:

$$\mathcal{R}_{\text{aug}}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{\tau \sim q(\tau)} [\ell(f_{\theta}(\tau(x)), g_{\tau}(y))].$$

where $\mathcal{D}$ is the empirical training distribution, x is an audio example, y is its target, τ is a sampled transformation, $q(\tau)$ is the augmentation policy, g_{τ} is the corresponding target transformation, f_{θ} is the model with parameters θ , and ℓ is the training loss. For a label-preserving transform, $g_{\tau}(y) = y$. For an equivariant task, such as pitch estimation under transposition, g_{τ} must alter the target.

The two expectations express two different sources of variation. Sampling examples from $\mathcal{D}$ exposes the model to the recorded corpus. Sampling transforms from q exposes it to a designed neighborhood around each recording. If that neighborhood resembles plausible deployment conditions, augmentation can improve robustness. If it crosses semantic boundaries or exaggerates irrelevant artifacts, the model learns the wrong invariances.

18.8 Invariance, Equivariance, and Invalid Transformations

Every augmentation policy should begin with a statement about labels. An invariant label stays unchanged after transformation. An equivariant label changes in a known way. An invalid transformation destroys or obscures the evidence needed to define the target. Treating all three cases as label preserving is one of the most common causes of disappointing augmentation experiments.

The following examples show how the same operation can have different meanings in different tasks.

Transformation	Label-preserving example	Label-changing or invalid example
Time stretch	Speaker identity under mild stretching	Beat timestamps and event boundaries must move
Pitch shift	Many environmental sound classes	Fundamental-frequency and key labels must transpose
Gain change	ASR transcript or genre label	Absolute loudness prediction target must change
Polarity reversal	Most monaural classification	Some waveform-generation and spatial tasks may be sensitive
Channel swap	Stereo event classification when orientation is irrelevant	Left-right localization labels must swap
Room simulation	Speech transcript	Dry-target enhancement pairs require careful target design
Additive noise	Transcript at intelligible SNR	Clean-signal reconstruction target remains unmodified
Cropping	Clip-level class present throughout	Event labels outside the crop must be removed

A policy review should classify every transform before training begins. For a scalar target y and transform parameter a , the transformed target can be written as:

$$y' = g_a(y).$$

where y' is the target attached to the transformed audio, g_a is the task-specific label mapping, and a is the sampled transform parameter. For pitch in hertz shifted by s semitones, $g_s(y) = y2^{s/12}$. For an event time stretched by factor r , $g_r(y) = ry$ when the implementation defines r as output duration divided by input duration. Verify the rate convention before transforming annotations.

18.9 Designing the Augmentation Distribution

Choosing a transform class is only the beginning. The probability of applying it, the parameter distribution, its order relative to other transforms, and correlations among parameters determine the distribution the model actually sees. A policy that says only “add noise and reverb” is underspecified.

For a transform with application probability p , the effective training distribution is a mixture:

$$p_{\text{train}}(z) = (1 - p)p_{\text{clean}}(z) + p \int p(z | x, \tau)q(\tau) d\tau.$$

where z is the example presented to the model, p_{clean} is the clean-example distribution, $p(z | x, \tau)$ describes the result of applying τ to source x , and $q(\tau)$ is the parameter distribution. Keeping $p < 1$ retains clean anchors. Setting every transform to $p = 1$ can unintentionally remove the original domain from training.

Uniform sampling is convenient but not automatically perceptually uniform. A uniform distribution in amplitude does not produce a uniform distribution in decibels. A uniform distribution in frequency ratio does not produce a uniform distribution in semitones. Define ranges in the coordinate system that corresponds to the intended perceptual or operational variation.

The signal-to-noise ratio used by additive-noise transforms is:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{P_n} \right),$$

where P_s is the average signal power and P_n is the average added-noise power over the measured region. Silence handling matters because estimating P_s over long silent margins can produce an augmentation that is much louder than expected during active speech. Compare active-region and whole-file measurements when validating an SNR policy.

The following policy questions are worth answering explicitly before a large render:

- What deployment variation is each transform intended to approximate?
- Which labels are invariant, which are transformed, and which examples become invalid?
- What percentage of training examples remains clean?
- Are transform parameters sampled in perceptually meaningful coordinates?
- Are parameter combinations independent, correlated, or mutually exclusive?
- Does transform order reproduce a plausible acoustic process?
- Which held-out conditions will test the policy’s intended benefit?

18.10 Transform Order Is Part of the Model

Audio operations generally do not commute. Adding noise and then applying room simulation reverberates both signal and noise. Applying room simulation and then adding noise models a reverberant source recorded with noise introduced closer to the microphone or electronics. Both can be useful, but they represent different worlds.

Let A and B be two transforms. In general:

$$A(B(x)) \neq B(A(x)).$$

where x is the source waveform, A is one augmentation operation, and B is another. The inequality can arise from nonlinear processing, time variation, resampling, clipping, or simply from a different physical interpretation.

A defensible speech pipeline often follows an approximate causal sequence: source variation, room propagation, microphone or channel coloration, transmission codec, and sensor noise. Creative MIR and self-supervised policies may deliberately violate that order, but the violation should be intentional rather than accidental.

```
from pvx.augment import (
    AddNoise,
    CodecDegradation,
    EQPerturber,
    GainPerturber,
    Pipeline,
    RoomSimulator,
)

causal_recording_model = Pipeline(
    [
        GainPerturber(gain_db=(-4, 4), p=0.8),
        RoomSimulator(rt60_range=(0.15, 0.9), wet_range=(0.1, 0.55), p=0.5),
        EQPerturber(n_bands=4, gain_db_range=(-5, 5), p=0.4),
        CodecDegradation(codec="random", p=0.2),
        AddNoise(snr_db=(8, 35), noise_type="pink", p=0.6),
    ],
    seed=1337,
)
```

18.11 Phase-Vocoder Augmentation as a Special Case

Time stretching and pitch shifting are unusually powerful because they alter musical or linguistic structure while leaving many other properties recognizable. They are also unusually easy to misuse. A phase vocoder can introduce transient blur, vertical phase incoherence, stereo-image changes, and formant displacement. These artifacts may become shortcuts that a model associates with a class or data source.

The synthesis hop in a basic time-stretch schedule is:

$$H_s = rH_a,$$

where H_s is the synthesis hop, H_a is the analysis hop, and r is the requested output-duration ratio. The ratio changes frame placement, while phase propagation attempts to preserve sinusoidal continuity. At large r , attacks occupy more output time unless transient handling resets or reroutes them.

Pitch shifting by s semitones uses the frequency ratio:

$$\rho = 2^{s/12},$$

where ρ is the resampling or spectral frequency ratio and s is the signed semitone shift. A pitch-label target must be multiplied by ρ . A key label must be transposed modulo twelve. A speaker-identity label may remain unchanged only for a narrow shift range that does not undermine identity.

Phase-vocoder augmentation deserves its own audit. Listen for artifacts at policy extrema, not only at median settings. Compare several source classes, including isolated attacks, voiced speech, dense mixtures, bass instruments, stereo ambience, and quiet tails. If a model can detect the processing engine more easily than the intended acoustic variation, the policy may improve training loss while reducing real-world validity.

The following settings are conservative starting points rather than universal truths:

The records for phase-vocoder augmentation as a special case keep the relevant measurements and notes together.

Task: ASR. **Time range:** 0.94 to 1.08. **Pitch range:** -1 to +1 semitone. **Principal caution:** Preserve intelligibility and speaker cues.

Task: Speaker recognition. **Time range:** 0.97 to 1.03. **Pitch range:** 0 or very narrow. **Principal caution:** Avoid changing perceived identity.

Task: Music tagging. **Time range:** 0.85 to 1.18. **Pitch range:** -2 to +2 semitones. **Principal caution:** Check tag invariance and transient quality.

Task: Beat tracking. **Time range:** 0.75 to 1.30. **Pitch range:** Usually 0. **Principal caution:** Transform beat timestamps with duration.

Task: Pitch tracking. **Time range:** Mild or none. **Pitch range:** Task-dependent. **Principal caution:** Transform frequency labels exactly.

Task: Contrastive SSL. **Time range:** Wider, paired by policy. **Pitch range:** Wider, paired by policy. **Principal caution:** Avoid trivially identifiable processing signatures.

18.12 Severity, Coverage, and Curriculum

More severe augmentation is not necessarily better. Mild policies may fail to cover deployment conditions, while extreme policies can increase label noise and optimization difficulty. Validation error often follows a shallow basin rather than a monotonic curve. A curriculum can begin near the clean distribution and gradually widen the parameter ranges.

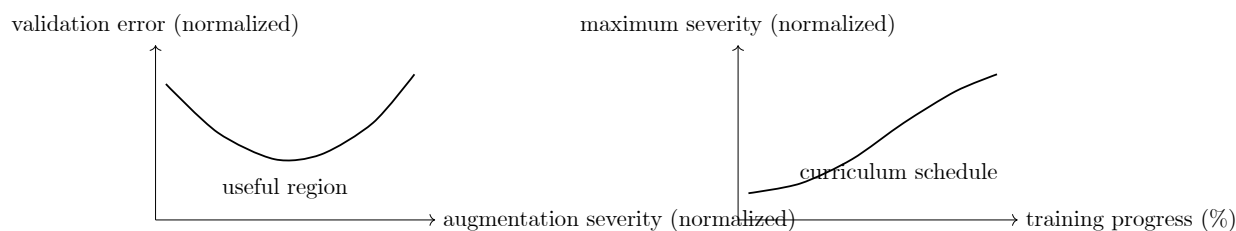


Figure 18.1: Two policy-design concepts. The left graph motivates a measured severity search. The right graph shows one possible curriculum that widens the allowed range during training.

A linear severity curriculum can be written as:

$$a(e) = a_0 + \frac{\min(e, E)}{E}(a_1 - a_0),$$

where $a(e)$ is the maximum augmentation severity at epoch e , a_0 is the initial severity, a_1 is the final severity, and E is the number of epochs over which the range expands. The parameter a may control one range or a coordinated family of ranges. Measure whether curriculum scheduling helps rather than assuming it will.

Coverage should be checked empirically from manifests. If a transform is configured with probability p but conditional failures, clipping rejection, or incompatible inputs reduce its realized use, the actual rate can differ from p . Count rendered parameter combinations and compare them with the intended policy.

18.13 Determinism and Seed Architecture

Reproducibility requires more than recording one global seed. Parallel workers, shuffled data loaders, resumed jobs, and changes in corpus ordering can all perturb a naive random-number stream. A stable design derives each example seed from immutable identifiers.

A conceptual seed derivation is:

$$s_i = \text{Hash}(s_0, u_i, v_i, k) \bmod 2^{31},$$

where s_i is the child seed for one transform, s_0 is the experiment seed, u_i is the stable source identifier, v_i is the variant number, and k is the transform position or name. A cryptographic hash is useful for stable distribution, but the exact hash and serialization must be recorded if independent implementations need bitwise agreement.

The pvx manifest should be treated as an experimental artifact. Preserve it beside model checkpoints and evaluation reports. Hashes establish which waveforms were used, while parameters explain how they were made. A seed without transform versions, source hashes, and policy bounds is not enough to recreate a dataset years later.

The following provenance fields are especially valuable:

- pvx version and source commit
- operating system, architecture, and relevant dependency versions
- source and output hashes
- policy-file hash
- global seed and derived per-example seed
- transform order and sampled parameters
- engine, preset, window, and phase-handling settings
- render status and error code
- label transformation version
- corpus license and source identifier

18.14 Split Hygiene and Family Leakage

Augmented siblings must not cross data splits. If one source recording appears in training while a stretched or noisy sibling appears in validation, the validation score measures memorization of source-specific details as well as generalization. File-name differences do not make examples independent.

Assign splits before augmentation and group all descendants by a stable source identity. For speech, the group may need to be the speaker rather than the utterance. For music, it may need to be the composition, performance, session, album, or artist depending on the scientific claim. For bioacoustics, recording site and date can matter as much as species.

The leakage unit should match the strongest nuisance correlation that the model could exploit. A speaker-independent ASR experiment requires speaker-disjoint evaluation. A cover-song retrieval experiment requires composition-aware splits. A room-robustness study may require room-disjoint evaluation even when speakers overlap.

```
pvx augment corpus/train/*.wav \  
  --output-dir derived/train \  
  --variants-per-input 5 \  
  --split-mode speaker_balanced \  
  --labels-csv metadata/train_labels.csv \  
  --grouping stem-prefix \  
  --group-separator "__" \  
  --seed 1337
```

After rendering, inspect the manifest for group overlap rather than trusting naming conventions. A strict validation step should fail the pipeline when one `group_key` occurs in more than one split.

```
pvx augment-manifest validate derived/train/augment_manifest.jsonl --strict
pvx augment-manifest stats derived/train/augment_manifest.jsonl
```

18.15 Online, Offline, and Hybrid Augmentation

Offline augmentation renders finite variants before training. It provides auditability, stable throughput, and easy listening inspection at the cost of storage and limited diversity. Online augmentation samples transformations during training. It provides effectively unbounded variation but consumes training compute and makes exact replay more demanding. A hybrid system precomputes expensive deterministic transforms and applies inexpensive random transforms in the data loader.

The approximate offline storage requirement is:

$$B \approx MVSRC,$$

where B is the number of stored bytes, M is the number of source clips, V is the number of variants per source, S is mean duration in seconds, R is sample rate, and C is bytes per multichannel sample frame. Container overhead is omitted. Lossless compression changes the realized total but not the basic scaling.

The approximate online compute fraction is:

$$\eta_{\text{aug}} = \frac{T_{\text{aug}}}{T_{\text{load}} + T_{\text{aug}} + T_{\text{model}}},$$

where η_{aug} is the fraction of step time consumed by augmentation, T_{load} is data-loading time, T_{aug} is transform time, and T_{model} is forward and backward model time. Measure this fraction with realistic worker counts and storage, because a policy that starves the accelerator may cost more than an extra offline corpus copy.

The following division is often practical:

Precompute offline	Sample online
High-quality phase-vocoder stretching	Gain perturbation
Expensive impulse-response convolution	Cropping and time shift
Codec round trips	Lightweight additive noise
Source separation or stem processing	Spectral masks
Audited label-equivariant renders	Mixup when targets support it

18.16 Contrastive and Self-Supervised Views

Contrastive learning needs two views that preserve the identity or content relation the objective is meant to capture. If the views are nearly identical, the task can be trivial. If they no longer share the intended content, the positive pair becomes false. The policy must sit between those failures.

For two views $x_a = \tau_a(x)$ and $x_b = \tau_b(x)$, a common normalized temperature-scaled loss is:

$$\ell_i = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/T)}{\sum_{j \neq i} \exp(\text{sim}(z_i, z_j)/T)},$$

where z_i is the embedding of one view, z_i^+ is the embedding of its paired positive view, z_j are candidate embeddings in the denominator, sim is a similarity function, and T is the temperature. Augmentation defines which information the loss encourages the representation to discard.

Independent view sampling is not always appropriate. Two aggressive crops may contain no common event. Two large pitch shifts may alter identity for a speaker objective. Coordinated policies can enforce minimum temporal overlap, shared source regions, bounded relative pitch, or one weak and one strong view.

```
pvx augment corpus/unlabeled/*.wav \  
  --output-dir derived/ssl_views \  
  --variants-per-input 2 \  
  --intent ssl_contrastive \  
  --pair-mode contrastive2 \  
  --manifest-jsonl reports/ssl_views.jsonl \  
  --seed 4242
```

Audit positive pairs by listening to random, median-severity, and maximum-severity examples. Also train a small classifier to predict augmentation type from embeddings. Very high transform predictability can reveal that the representation is organizing itself around processing signatures rather than useful content.

18.17 Paired and Structured Targets

Enhancement, separation, source localization, beat tracking, alignment, and sequence labeling require coordinated changes to audio and targets. The safest implementation represents an example as a structured record rather than applying waveform transforms in isolation.

For a waveform $x(t)$ with event intervals $[a_j, b_j]$, a constant time stretch r gives:

$$x'(t) = x(t/r), \quad [a'_j, b'_j] = [ra_j, rb_j].$$

where $x'(t)$ is the stretched waveform, r is output duration divided by input duration, and $[a'_j, b'_j]$ is the transformed interval for event j . Cropping by an offset c then maps retained boundaries to $a''_j = a'_j - c$ and $b''_j = b'_j - c$, with clipping to the crop interval.

For enhancement training, the noisy input and clean target should usually remain sample aligned. If room simulation or resampling changes latency, record and compensate for that latency before calculating a sample-domain loss. A transform applied only to the input represents corruption. A geometric transform applied to both input and target represents a new aligned pair. Confusing these cases teaches delay, filtering, or pitch errors as if they were desired output.

18.18 Task-Specific Policy Reasoning

Different tasks reward different invariances. A single global “audio augmentation” recipe cannot be optimal across ASR, music transcription, speaker recognition, enhancement, and generative modeling. The policy should follow the task’s target semantics and deployment environment.

Automatic speech recognition

ASR usually treats moderate room, noise, channel, gain, and speaking-rate variation as label preserving. Large pitch changes, severe time compression, aggressive cropping, or low SNR can make the transcript ambiguous. Measure word error rate separately on clean, noisy, reverberant, accented, and channel-degraded subsets so one aggregate number does not hide regressions.

Speaker and paralinguistic modeling

Speaker identity, age, emotion, pathology, and prosody depend on cues that ordinary ASR may discard. Formant shifts and pitch transforms can directly alter those cues. Prefer environmental and channel augmentation first, then introduce voice transformations through controlled ablations. Report performance by demographic and recording-condition strata when metadata and consent permit.

Music information retrieval

Music tags differ in transposition and tempo invariance. Instrument labels may survive moderate pitch shifts, while key labels must transpose and tuning labels may not survive. Genre labels may tolerate modest tempo changes, while beat and downbeat timestamps must move. Chord roots transpose but chord qualities usually remain. Use task-specific target adapters rather than one file-level label policy.

Pitch and melody estimation

Pitch shifting is valuable precisely because it requires exact label equivariance. Keep a high-precision mapping between sample time and target frames, account for resampling conventions, and verify frequencies after rendering. Formant-preserving shifts may be preferable for voice realism, but the model should not learn that one formant algorithm uniquely identifies shifted examples.

Enhancement and source separation

The clean target defines what counts as distortion. Additive noise, reverberation, clipping, and codecs may be applied to the input alone when removal is desired. Gain and timing changes may need to affect both input and target. For source separation, transforms applied independently to stems increase mixture diversity, but the stems must be remixed exactly and their sum checked against the mixture.

Generative and neural-vocoder training

Waveform and spectrogram generators are sensitive to phase, sample alignment, loudness, and bandwidth. An augmentation that is harmless for classification may create inconsistent conditioning-target pairs for generation. Decide whether transforms belong before feature extraction, after feature extraction, on the waveform target, or on both sides of the pair.

18.19 Class Imbalance and Conditional Policies

Augmentation can increase the number of minority-class examples, but replication alone does not create new semantic coverage. If every rare example produces many near-identical children, the model may memorize the source family. Vary nuisance conditions while preserving group-aware splits, and monitor performance per original source rather than per rendered file.

A class-dependent sampling weight can be written as:

$$w_c = \frac{(n_c + \epsilon)^{-\alpha}}{\sum_j (n_j + \epsilon)^{-\alpha}},$$

where w_c is the probability of selecting class c , n_c is its source-example count, ϵ prevents division by zero, and α controls the strength of balancing. Setting $\alpha = 0$ gives uniform class-independent sampling over the existing selection process, while larger α increasingly favors rare classes. Keep source selection and augmentation severity as separate decisions.

Conditional policies can also reflect known deployment structure. A telephone codec belongs on telephone-like speech, not necessarily on every music clip. Very long reverberation may be plausible for distant speech but implausible for close-miked studio labels. Document these conditions so the policy does not become an opaque set of class-specific interventions.

18.20 Ablation Studies

An augmentation result is difficult to interpret when ten transforms are introduced at once. Begin with a clean baseline, evaluate individual transform families, then test combinations and severity ranges. The

purpose of an ablation is not only to find the best score. It reveals which variation the model lacked and whether transforms interact.

A minimum experiment matrix should include the following runs:

Run	Policy	Question answered
A	No augmentation	What is the clean baseline?
B	Gain and crop only	Does cheap geometric variation help?
C	Noise only	Is additive robustness missing?
D	Room only	Is reverberation the limiting condition?
E	Time and pitch only	Does phase-vocoder variation help?
F	Full policy	Do the families combine constructively?
G	Full policy without one family	Which family contributes marginal value?
H	Full policy at half and double severity	Is the chosen range near a useful operating point?

Run multiple training seeds when the expected improvement is close to ordinary training variance. Record mean, dispersion, and the individual runs. A single favorable seed is not evidence that the policy is robust.

For metric m measured on an augmented system and baseline, report:

$$\Delta m = m_{\text{aug}} - m_{\text{base}},$$

where Δm is the absolute metric change, m_{aug} is the score after training with augmentation, and m_{base} is the baseline score under the same evaluation protocol. Also report relative change when it is meaningful, but do not replace the absolute result with a percentage that hides scale.

18.21 Evaluation Beyond One Aggregate Score

An augmentation policy should be evaluated on clean data and on targeted stress conditions. Clean performance reveals whether invariance came at the cost of useful information. Stress subsets reveal whether the intended robustness actually improved. A synthetic stress test is useful for diagnosis, but it should not be the only evidence because matching training and test transformations can reward engine-specific artifacts.

The evaluation suite should include these perspectives:

- clean in-domain performance
- natural out-of-domain recordings
- controlled synthetic severity sweeps
- task-specific subgroup metrics
- calibration and confidence under corruption
- worst-group or low-percentile performance
- compute, latency, and storage cost
- random listening inspection and artifact annotation

Plot metrics against severity rather than reporting only one endpoint. Every graph should label the transform parameter and unit on the horizontal axis and the metric and direction on the vertical axis. Include confidence intervals or run dispersion when multiple seeds are available.

18.22 Detecting Shortcuts and Augmentation Artifacts

Models exploit stable details that researchers may not notice. A resampler’s passband, a codec delay, deterministic file padding, a phase-vocoder texture, or a naming-dependent split can become predictive. Shortcut checks should therefore be part of policy validation.

Useful diagnostics include the following experiments:

- Train a classifier to predict whether and how an example was augmented.
- Compare multiple DSP engines for the same nominal transform.
- Randomize output encoding and metadata independently of class.
- Inspect spectrograms at policy boundaries and around transients.
- Evaluate on naturally occurring versions of the targeted condition.
- Shuffle augmentation parameters across labels and confirm performance collapses when it should.
- Check whether source duration, padding, or output length leaks a target.
- Verify that failed renders are not concentrated in particular classes.

A high augmentation-detector score is not automatically fatal because transformations necessarily leave evidence. It is a warning to test whether the downstream model relies on that evidence. Engine diversity and natural evaluation recordings are stronger safeguards than attempting to make every synthetic transform undetectable.

18.23 Quality Control Before Training

Large augmentation jobs should have a release gate just like software. Validate manifests, inspect distributions, listen to stratified samples, and reject outputs that violate technical limits. The gate should be deterministic and should produce a report that can travel with the dataset.

The following checks catch many failures cheaply:

1. Confirm that output count matches the dry-run plan.
2. Validate that every output opens and has the expected channel count and sample rate.
3. Check peak level, clipping percentage, duration, silence fraction, and nonfinite samples.
4. Compare realized parameter histograms with policy bounds.
5. Confirm that source groups occur in one split only.
6. Verify label transforms on hand-calculated examples.
7. Listen to random samples and all extrema.
8. Compare source and output hashes for accidental duplicates.
9. Record render failures and investigate class-dependent failure rates.
10. Freeze manifests and policy files before model training begins.

```
pvx augment corpus/*.wav \
  --output-dir candidate_aug \
  --variants-per-input 4 \
  --intent mir_music \
  --dry-run \
  --manifest-jsonl reports/candidate_plan.jsonl \
  --seed 2026

pvx augment-manifest validate reports/candidate_plan.jsonl --strict
pvx augment-manifest stats reports/candidate_plan.jsonl
```

18.24 A Reproducible Experiment Layout

A predictable directory structure reduces the chance that policies, manifests, and model results become separated. Treat derived audio as replaceable, but treat its recipe and provenance as durable research records.

```
experiment/  
  README.md  
  environment.txt  
  policies/  
    baseline.json  
    full.json  
    no_room.json  
  manifests/  
    sources.jsonl  
    full_train.jsonl  
    no_room_train.jsonl  
  reports/  
    augmentation_qc.json  
    evaluation_by_condition.csv  
  checkpoints/  
  logs/  
  derived_audio/
```

The experiment README should define the task, corpus version, split unit, label semantics, primary metric, expected deployment conditions, and stopping rule. Write these choices before reading test results. This makes later policy changes easier to recognize as exploratory rather than confirmatory.

18.25 Complex Policy Example: Robust Speech with Auditable Views

The following example combines deterministic offline phase-vocoder variants with online environmental corruption. The expensive structural variation is frozen in a manifest. Noise and gain remain dynamic during training so each epoch sees fresh conditions.

```
pvx augment speech/train/*.wav \  
  --output-dir derived_audio/speech_structure \  
  --variants-per-input 3 \  
  --intent asr_robust \  
  --label-policy preserve \  
  --split-mode speaker_balanced \  
  --labels-csv metadata/speakers.csv \  
  --grouping stem-prefix \  
  --group-separator "_" \  
  --manifest-jsonl manifests/speech_structure.jsonl \  
  --manifest-csv manifests/speech_structure.csv \  
  --workers 8 \  
  --seed 314159
```

The online stage can then add recording-condition variation without changing transcripts.

```
from pvx.augment import AddNoise, GainPerturber, Pipeline, RoomSimulator  
  
online_environment = Pipeline(  
  [  
    GainPerturber(gain_db=(-5.0, 5.0), p=0.8),  
    RoomSimulator(  
      rt60_range=(0.08, 0.85),  
      wet_range=(0.08, 0.50),  
      p=0.45,  
    ),  
    AddNoise(snr_db=(8.0, 35.0), noise_type="pink", p=0.60),  
  ],  
  seed=271828,  
)
```

```
def augment_training_item(audio, sample_rate, stable_item_seed):
    return online_environment(audio, sample_rate, seed=stable_item_seed)
```

Evaluate this system against clean speech, naturally noisy speech, naturally reverberant speech, and synthetic sweeps made with a different noise and room collection. The different evaluation generator reduces the chance that success reflects exact matching to the training engine.

18.26 Complex Policy Example: Equivariant Music Labels

A music pipeline often needs to transform several labels together. Suppose each example contains audio, beat times, a key class, and framewise fundamental frequency. A stretch factor r multiplies beat times, while a pitch shift s multiplies frequency by $2^{s/12}$ and transposes key by s semitones when s is integral.

The combined mapping is:

$$g_{r,s}(b_j, k, f_t) = (rb_j, (k + s) \bmod 12, 2^{s/12} f_t),$$

where b_j is beat time j , k is the pitch-class key label, f_t is fundamental frequency at frame t , r is the output-duration ratio, and s is the semitone shift. Frame timestamps must also be stretched before the transformed f_t sequence is aligned to output audio.

This mapping illustrates why a generic `label_policy=preserve` switch cannot express every research task. Render parameters in the `pvx` manifest, then run a task-specific annotation transformer that consumes those exact parameters. Validate the transformed annotation against a small set of synthetic tones and click tracks before applying it to a corpus.

18.27 Fairness, Consent, and Dataset Governance

Augmentation does not repair missing populations. Pitch shifting one voice does not create a new speaker, accent, age group, language, room, microphone practice, or cultural context. It can reduce sensitivity to selected nuisance variables, but it cannot replace representative collection and careful governance.

Policies may also affect groups differently. A denoising or time-scaling transform can alter high-frequency consonants, breathiness, vocal tremor, or low-energy speech in ways that are uneven across speakers. Evaluate subgroup performance where ethically collected metadata permits it. When metadata is unavailable or inappropriate, state the limitation rather than treating the aggregate score as universal.

Respect source licenses, consent terms, and restrictions on derivative data. A manifest should retain lineage from output to source without exposing private paths or identities in public artifacts. Hashes can support integrity checks, but hashes do not anonymize a small or identifiable corpus.

18.28 Publication and Handoff Checklist

A published augmentation study should provide enough information for another researcher to reconstruct both the policy and the evaluation. The following checklist is a useful minimum:

- corpus versions, licenses, and inclusion criteria
- split assignment unit and leakage checks
- `pvx` version or commit and installation method
- complete policy files with parameter distributions and transform order
- seeds and seed-derivation method
- variants per source and realized transform frequencies
- label-invariance or label-equivariance rules
- clean and stress-test metrics with multiple training seeds
- compute, storage, and rendering-failure summaries

- manifest schema and hashes for released artifacts
- listening or quality-control procedure
- known artifacts, failed conditions, and excluded examples

18.29 Research Questions Worth Testing

The chapter’s methods support several deeper experiments. These questions are more informative than asking whether augmentation helps in the abstract:

- Does phase-vocoder engine diversity improve transfer to natural tempo variation?
- At what severity does a label-preserving pitch shift begin to alter perceived speaker identity?
- Do curriculum policies outperform a fixed mixture when total transform exposure is held constant?
- Does preserving a clean anchor fraction improve calibration under in-domain conditions?
- Which transform interactions are constructive, redundant, or destructive?
- Can a model predict the processing engine from learned embeddings?
- Does group-balanced splitting change conclusions drawn from ordinary random splitting?
- How much apparent robustness survives evaluation with independently generated corruptions?
- Are minority-class gains due to nuisance diversity or simple oversampling?
- Which manifest statistics best predict a policy’s downstream benefit?

18.30 Benchmarking Augmentation Pipelines

Use the profile suite benchmark (speech/music/noisy/stereo):

```
python benchmarks/run_augment_profile_suite.py \  
--quick \  
--gate \  
--out-dir benchmarks/out_augment_profiles
```

Refresh all per-profile baselines after intentional benchmark changes:

```
python benchmarks/run_augment_profile_suite.py \  
--quick \  
--refresh-baselines \  
--out-dir benchmarks/out_augment_profiles_refresh
```

Outputs: - Per-profile reports: - benchmarks/out_augment_profiles/<profile>/report.json - benchmarks/out_augment_profiles/<profile>/report.md - Suite report: - benchmarks/out_augment_profiles/suite_report.json - benchmarks/out_augment_profiles/suite_report.md

18.31 Running with uv

A working command makes the running with uv concrete.

```
uv run pvx augment data/*.wav --output-dir aug_out --variants-per-input 4 --intent asr_robust --seed 1337
```

18.32 Research Notes

A reliable approach to research notes begins with these points.

- Keep original clean file IDs in your training metadata so you can group augmented siblings.
- Avoid split leakage: use `--grouping stem-prefix` with a stable naming convention (for example `speaker42__take3.wav`).

- Prefer fixed seeds for published experiments; change seeds only for explicit variance studies.
- Use `--dry-run` before large jobs to validate output counts and manifest content.

CHAPTER 19

Audio Augmentation Cookbook

Alpha release (0.1.0a1) , These recipes use `pvx.augment`, which is under active development. APIs may change before 1.0.

Real-world augmentation recipes for common deep-learning audio tasks. Each recipe includes the rationale, a pipeline configuration, and CLI equivalents.

19.1 Contents

For contents, start with these practical details.

1. ASR , Automatic Speech Recognition
 2. Keyword Spotting / Wake-Word Detection
 3. Speaker Verification / Identification
 4. Speech Enhancement and Denoising
 5. Music Classification and Tagging
 6. Beat Tracking and Tempo Estimation
 7. Pitch Estimation and Melody Extraction
 8. Audio Event Detection / Sound Classification
 9. Self-Supervised / Contrastive Learning (SimCLR, MoCo, BYOL)
 10. Music Source Separation
-

19.2 ASR , Automatic Speech Recognition

Goal: Train models robust to microphone quality, background noise, room acoustics, and channel distortion.

Augmentation rationale: - Room simulation: phones and meetings happen in reverberant spaces - Background noise: cafes, offices, street noise are the real world - Codec degradation: telephone and VoIP channels are common deployment targets - Gain perturbation: microphone levels vary across recordings - SpecAugment: proven to reduce WER on LibriSpeech, Common Voice

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, CodecDegradation, SpecAugment

asr_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    RoomSimulator(rt60_range=(0.05, 1.0), wet_range=(0.1, 0.7), p=0.5),
    AddNoise(snr_db=(5, 35), noise_type="pink", p=0.6),
    CodecDegradation(codec="random", p=0.2),
    SpecAugment(
        freq_mask_param=27,
        time_mask_param=100,
        num_freq_masks=2,
        num_time_masks=2,
        p=0.8,
    ),
], seed=42)
```

CLI:


```
# Step 1: Heavy offline augmentation (time-stretch + pitch)
\index{Step 1 Heavy offline augmentation (time-stretch + pitch)}
pvx augment data/train/*.wav \
  --output-dir data/train_aug \
  --variants-per-input 3 \
  --intent asr_robust \
  --engine auto \
  --seed 1337 \
  --workers 16

# Step 2: Add noise per file
\index{Step 2 Add noise per file}
for f in data/train_aug/*.wav; do
  pvxnoise "$f" --snr 5,30 --noise-type pink --output "$f"
done

# Step 3: Simulate codec degradation (optional)
\index{Step 3 Simulate codec degradation (optional)}
for f in data/train_aug/*.wav; do
  pvxcodec "$f" --codec random --seed "$RANDOM" --output "$f"
done
```

Note: SpecAugment has been shown to improve WER in the original [Park et al. 2019](#) paper. The impact of the full pipeline above will depend on your model, data, and evaluation conditions, we recommend benchmarking on your own held-out test set.

19.3 Keyword Spotting / Wake-Word Detection

Goal: Detect specific words under highly diverse acoustic conditions.

Key considerations: - Label-preserving augmentation only, semantic content must be unchanged - Aggressive time-stretch causes word timing drift; keep it mild - Heavy SpecAugment helps greatly since models often rely on a few spectral cues

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, CodecDegradation, SpecAugment, TimeShift, Fade

kws_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    Fade(fade_in=(0.001, 0.02), fade_out=(0.001, 0.02), p=0.4),
    TimeShift(shift=(-0.05, 0.05), p=0.3),
    AddNoise(snr_db=(5, 30), noise_type="pink", p=0.7),
    CodecDegradation(codec="random", p=0.3),
    SpecAugment(
        freq_mask_param=30,
        time_mask_param=25, # smaller T, keyword clips are short
        num_freq_masks=2,
        num_time_masks=2,
        p=0.7,
    ),
], seed=42)
```

CLI:

```
pvx augment keywords/*.wav \  
  --output-dir keywords_aug \  
  --variants-per-input 5 \  
  --intent asr_robust \  
  --label-policy preserve \ # keeps pitch/time perturbation mild  
  --seed 7
```

19.4 Speaker Verification / Identification

Goal: Train embeddings invariant to microphone, room, and channel effects but discriminative across speakers.

Important: Pitch perturbation should be minimal (large shifts can change perceived speaker identity and confuse the model during training).

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, CodecDegradation, EQPerturber  
  
sv_pipeline = Pipeline([  
    GainPerturber(gain_db=(-6, 6), p=0.9),  
    EQPerturber(n_bands=4, gain_db_range=(-8, 8), p=0.6), # microphone colouration  
    RoomSimulator(rt60_range=(0.05, 1.5), wet_range=(0.1, 0.8), p=0.6),  
    AddNoise(snr_db=(10, 40), noise_type="pink", p=0.5),  
    CodecDegradation(codec="random", p=0.25),  
], seed=42)
```

Contrastive learning setup (ECAPA-TDNN / ResNet style):

```
from pvx.augment import contrastive_pipeline  
  
pipeline_a, pipeline_b = contrastive_pipeline(seed=42)  
  
for audio, speaker_id in dataloader:  
    view_a, _ = pipeline_a(audio, sr, seed=item_idx)  
    view_b, _ = pipeline_b(audio, sr, seed=item_idx)  
    loss = nt_xent_loss(embed(view_a), embed(view_b), labels=speaker_id)
```

19.5 Speech Enhancement and Denoising

Goal: Train models to map noisy/reverberant speech -> clean speech. Augmentation creates the *noisy input*; the clean original is the *target*.

```
from pvx.augment import Pipeline, GainPerturber, RoomSimulator, AddNoise, ImpulseNoise, CodecDegradation,  
    OneOf, Identity  
  
# This pipeline degrades clean speech to create training pairs  
\index{This pipeline degrades clean speech to create training pairs}  
degradation_pipeline = Pipeline([  
    GainPerturber(gain_db=(-6, 6), p=1.0),  
    RoomSimulator(rt60_range=(0.1, 2.5), wet_range=(0.2, 0.95), p=0.7),  
    AddNoise(snr_db=(0, 20), noise_type="pink", p=0.8),  
    OneOf([  
        CodecDegradation(codec="voip_narrow"),  
        CodecDegradation(codec="telephone"),  
        Identity(),  
    ], p=0.4),  
])
```

```

    ImpulseNoise(rate=1.0, amplitude_range=(0.01, 0.1), p=0.1),
], seed=42)

# During training:
\index{During training}
for clean_audio, sr in clean_dataset:
    noisy_audio, _ = degradation_pipeline(clean_audio, sr, seed=item_idx)
    predicted_clean = model(noisy_audio)
    loss = si_snr_loss(predicted_clean, clean_audio)

```

Data generation CLI:

```

# Parallel degradation of a clean corpus
\index{Parallel degradation of a clean corpus}
pvxrir clean/*.wav --rt60 0.1,2.0 --wet 0.3,0.9 --output-dir reverberant/
pvxnoise reverberant/*.wav --snr 0,20 --noise-type pink --output-dir noisy/
pvxcodec noisy/*.wav --codec random --output-dir degraded/

```

19.6 Music Classification and Tagging

Goal: Train genre, mood, instrument, and tag classifiers robust to diverse recording conditions and production styles.

```

from pvx.augment import Pipeline, GainPerturber, EQPerturber, RoomSimulator, AddNoise, SpecAugment, OneOf,
    Identity, ImpulseNoise

music_cls_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    EQPerturber(n_bands=5, gain_db_range=(-8, 8), p=0.6),
    RoomSimulator(rt60_range=(0.1, 1.0), wet_range=(0.05, 0.3), p=0.3),
    OneOf([
        AddNoise(snr_db=(25, 45), noise_type="white"),
        ImpulseNoise(rate=0.5),
        Identity(),
    ], p=0.3),
    SpecAugment(
        freq_mask_param=30,
        time_mask_param=80,
        num_freq_masks=2,
        num_time_masks=2,
        p=0.5,
    ),
], seed=42)

```

Offline augmentation with pvx augment:

```

pvx augment music/*.wav \
  --output-dir music_aug \
  --variants-per-input 4 \
  --intent mir_music \
  --seed 42 \
  --workers 12

```

19.7 Beat Tracking and Tempo Estimation

Goal: Train tempo-invariant beat trackers. Time-stretch is the most critical augmentation.

Important: Time-stretch must preserve beat structure. Use pvx's drum-safe preset which uses hybrid transient mode to keep kick/snare clicks intact.

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, SpecAugment, TimeStretch

beat_pipeline = Pipeline([
    TimeStretch(
        rate=(0.8, 1.25),
        preserve_pitch=True,
        preset="drums_safe",
        engine="auto",  # uses PyTorch when available, else pvx CLI
        p=0.8,
    ),
    GainPerturber(gain_db=(-4, 4), p=0.8),
    AddNoise(snr_db=(25, 45), noise_type="white", p=0.3),
], seed=42)
```

CLI , offline generation across a tempo range:

```
for rate in 0.80 0.90 1.00 1.10 1.20; do
    pvxvoc drums.wav \
        --stretch $rate \
        --preset drums_safe \
        --transient-mode wsola \
        --output "drums_${rate}.wav"
done
```

19.8 Pitch Estimation and Melody Extraction

Goal: Train f0 estimators robust to background noise, vibrato, and pitch transpositions.

```
from pvx.augment import Pipeline, GainPerturber, AddNoise, EQPerturber

pitch_pipeline = Pipeline([
    GainPerturber(gain_db=(-4, 4), p=0.8),
    EQPerturber(n_bands=4, gain_db_range=(-6, 6), p=0.5),
    AddNoise(snr_db=(15, 40), noise_type="pink", p=0.6),
], seed=42)

# For pitch shift augmentation use pvx voc (label must shift with pitch)
\index{For pitch shift augmentation use pvx voc (label must shift with pitch)}
# pvxvoc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav
\index{pvxvoc vocal.wav --stretch 1.0 --pitch 3 --output vocal_up3.wav}
```

Offline pitch-shift augmentation with label tracking:

```
for semitones in -4 -2 0 2 4; do
    pvxvoc vocal.wav \
        --stretch 1.0 \
        --pitch $semitones \
        --preset vocal_studio \
        --output "vocal_pitch${semitones}.wav"
    echo "$semitones" > "vocal_pitch${semitones}_label.txt"
done
```

19.9 Audio Event Detection / Sound Classification

Goal: Classify acoustic events (dogs barking, glass breaking, gunshots, etc.) in diverse background conditions.

```
from pvx.augment import (
    Pipeline, GainPerturber, AddNoise, BackgroundMixer,
    RoomSimulator, SpecAugment, Fade, TimeShift
)

sed_pipeline = Pipeline([
    GainPerturber(gain_db=(-6, 6), p=0.9),
    Fade(fade_in=(0.0, 0.1), fade_out=(0.0, 0.1), p=0.3),
    TimeShift(shift=(-0.2, 0.2), p=0.4),
    BackgroundMixer("backgrounds/", snr_db=(5, 20), p=0.6),
    RoomSimulator(rt60_range=(0.1, 1.5), wet_range=(0.1, 0.6), p=0.4),
    SpecAugment(freq_mask_param=30, time_mask_param=60, p=0.5),
], seed=42)
```

19.10 Self-Supervised / Contrastive Learning

Goal: Learn audio representations without labels using contrastive objectives (SimCLR, MoCo, BYOL, VICReg, data2vec).

Two independently augmented views of the same clip form a positive pair.

```
from pvx.augment import contrastive_pipeline
from pvx.integrations.huggingface import HFAugmentMapper

# Two strongly-correlated but statistically independent views
\index{Two strongly-correlated but statistically independent views}
pipeline_a, pipeline_b = contrastive_pipeline(seed=42)

# HuggingFace
\index{HuggingFace}
mapper = HFAugmentMapper(
    pipeline_a, # pipeline_b handled internally via generate_pair=True
    generate_pair=True,
    output_prefix="view",
)

ds_ssl = ds.map(mapper, batched=False, with_indices=True, num_proc=8)
# ds_ssl columns: audio, view_a, view_b
\index{ds ssl columns audio view a view b}

# PyTorch training loop
\index{PyTorch training loop}
for batch in loader:
    view_a = batch["view_a"] # (B, T)
    view_b = batch["view_b"] # (B, T)
    z_a = encoder(view_a)
    z_b = encoder(view_b)
    loss = nt_xent(z_a, z_b)
```

Recommended SSL augmentation strength: - p values between 0.4-0.7 per transform - Include at least: gain, room simulation, noise, SpecAugment - Avoid transforms that destroy too much semantic content (extreme bit-crush, strong clipping) - Two views should be recognizably related but perceptually distinct

19.11 Music Source Separation

Goal: Separate music into stems (vocals, drums, bass, other). Augmentation creates harder mixtures and prevents over-fitting to recording conditions.

```
from pvx.augment import Pipeline, GainPerturber, EQPerturber, AddNoise

# Per-stem augmentation (applied before mixing)
\index{Per-stem augmentation (applied before mixing)}
stem_pipeline = Pipeline([
    GainPerturber(gain_db=(-3, 3), p=0.9),
    EQPerturber(n_bands=3, gain_db_range=(-4, 4), p=0.5),
], seed=42)

# During training , augment each stem independently then remix
\index{During training augment each stem independently then remix}
for vocal, drums, bass, other in stems:
    vocal_aug, _ = stem_pipeline(vocal, sr, seed=item_idx * 4 + 0)
    drums_aug, _ = stem_pipeline(drums, sr, seed=item_idx * 4 + 1)
    bass_aug, _ = stem_pipeline(bass, sr, seed=item_idx * 4 + 2)
    other_aug, _ = stem_pipeline(other, sr, seed=item_idx * 4 + 3)
    mixture = vocal_aug + drums_aug + bass_aug + other_aug
    predicted_stems = separator(mixture)
    loss = si_snr_loss(predicted_stems, [vocal_aug, drums_aug, bass_aug, other_aug])
```

19.12 Augmentation Intensity Guide

The records for augmentation intensity guide keep the relevant measurements and notes together.

Strength: Very mild (preserve label). **Stretch:** 0.97-1.03. **Pitch:** ± 0.5 st. **SNR:** > 30 dB. **RT60:** < 0.3 s. **Notes:** KWS preserve mode.

Strength: Mild. **Stretch:** 0.92-1.12. **Pitch:** ± 1.5 st. **SNR:** 20-35 dB. **RT60:** 0.1-0.6 s. **Notes:** ASR, SV.

Strength: Moderate. **Stretch:** 0.85-1.25. **Pitch:** ± 3.0 st. **SNR:** 10-30 dB. **RT60:** 0.2-1.5 s. **Notes:** Music tagging, classification.

Strength: Strong. **Stretch:** 0.75-1.40. **Pitch:** ± 5.0 st. **SNR:** 5-25 dB. **RT60:** 0.3-2.5 s. **Notes:** SSL, enhancement.

Strength: Extreme. **Stretch:** 0.50-2.00. **Pitch:** ± 7.0 st. **SNR:** 0-15 dB. **RT60:** 0.5-4.0 s. **Notes:** Robustness stress tests.

CHAPTER 20

pvx Pipeline Cookbook

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

Curated one-line workflows for practical chaining, mastering, microtonal processing, and batch operation.

20.1 Analysis/QA

The discussion of the analysis/qa begins by separating its main parts.

Quality metrics on processed speech

The example below puts the quality metrics on processed speech into executable form.

```
python3 -m pvx.algorithms.analysis_qa_and_automation.pesq_stoi_visqol_quality_metrics clean.wav noisy.wav --output out/qa.json
```

Why: Collects objective quality indicators for regression tracking.

20.2 Automation

Several distinct concerns meet in the automation; they are taken in turn below.

A/B report generation

A working command makes the a/b report generation concrete.

```
python3 scripts/scripts_ab_compare.py --input mix.wav --a-args "--time-stretch 1.1 --transform fft" --b-args "--time-stretch 1.1 --transform dct" --out-dir reports/ab --name fft_vs_dct
```

Why: Creates JSON/Markdown objective reports for fast algorithm and parameter comparisons.

Benchmark matrix sweep

The following session demonstrates the benchmark matrix sweep in practice.

```
python3 scripts/scripts_benchmark_matrix.py --input mix.wav --transforms fft,dft,dct --windows hann,kaiser --n-ffts 1024,2048 --devices cpu --out-dir reports/bench
```

Why: Produces reproducible CSV/JSON runtime matrices across transform/window/FFT combinations.

Quality regression check

The following session demonstrates the quality regression check in practice.

```
python3 scripts/scripts_quality_regression.py --input mix.wav --output out/reg.wav --render-args "--time-stretch 1.2 --transform fft" --baseline-json reports/baseline.json --report-json reports/regression.json
```

Why: Compares current renders against baseline objective metrics with configurable tolerances.

20.3 Batch

The account of the batch proceeds through the topics that follow.

Batch stretch over folder

The following session demonstrates the batch stretch over folder in practice.

```
pvx voc stems/*.wav --time-stretch 1.08 --output-dir out/stems --overwrite
```

Why: Applies consistent transform to many files with one command.

Dry-run output validation

A working command makes the dry-run output validation concrete.

```
pvx denoise takes/*.wav --reduction-db 8 --dry-run --output-dir out/preview
```

Why: Checks filename resolution and collisions without writing audio.

20.4 Mastering

The account of the mastering proceeds through the topics that follow.

Integrated loudness targeting with limiter

The example below puts the integrated loudness targeting with limiter into executable form.

```
pvx voc mix.wav --time-stretch 1.0 --target-lufs -14 --compressor-threshold-db -20 --compressor-ratio 3 --limiter-threshold 0.98 --output-dir out --suffix _master
```

Why: Combines dynamics and loudness controls in shared mastering chain.

Soft clip and hard safety ceiling

The following session demonstrates the soft clip and hard safety ceiling in practice.

```
pvx harmonize bus.wav --intervals 0,7,12 --mix 0.35 --soft-clip-level 0.92 --soft-clip-type tanh --hard-clip-level 0.99 --output-dir out
```

Why: Adds saturation while enforcing a strict final peak ceiling.

20.5 Microtonal

Several distinct concerns meet in the microtonal; they are taken in turn below.

Custom cents map retune

The following session demonstrates the custom cents map retune in practice.

```
pvx retune vox.wav --root 60 --scale-cents 0,90,204,294,408,498,612,702,816,906,1020,1110 --strength 0.8 --output-dir out
```

Why: Maps incoming notes to a custom 12-degree microtonal scale.

Conform CSV with per-segment ratios

The following session demonstrates the conform CSV with per-segment ratios in practice.

```
pvx conform solo.wav map_conform.csv --pitch-mode ratio --output-dir out --suffix _conform
```

Why: Applies timeline-specific time and pitch trajectories from CSV.

20.6 Phase-vocoder core

Several distinct concerns meet in the phase-vocoder core; they are taken in turn below.

Moderate vocal stretch with formant preservation

A working command makes the moderate vocal stretch with formant preservation concrete.

```
pvx voc vocal.wav --time-stretch 1.15 --pitch-mode formant-preserving --output-dir out --suffix _pv
```

Why: Retains speech-like vowel envelope while stretching timing.

Independent cents retune

A working command makes the independent cents retune concrete.

```
pvx voc lead.wav --pitch-shift-cents -23 --time-stretch 1.0 --output-dir out --suffix _cents
```

Why: Applies precise microtonal offset without tempo change.

Extreme stretch with multistage strategy

The following session demonstrates the extreme stretch with multistage strategy in practice.

```
pvx voc ambience.wav --target-duration 600 --ambient-preset --n-fft 16384 --win-length 16384 --hop-size 2048 --window kaiser --kaiser-beta 18 --output-dir out --suffix _ambient600x
```

Why: PaulStretch-style ambient profile for very large ratios using stochastic phase and onset time-credit controls.

Ultra-smooth speech stretch (600x)

The example below puts the ultra-smooth speech stretch (600x) into executable form.

```
pvx voc speech.wav --target-duration 600 --stretch-mode standard --phase-engine propagate --phase-locking identity --n-fft 8192 --win-length 8192 --hop-size 256 --window hann --normalize peak --peak-dbfs -1 --compressor-threshold-db -30 --compressor-ratio 2.0 --compressor-attack-ms 25 --compressor-release-ms 250 --compressor-makeup-db 4 --limiter-threshold 0.95 --output-dir out --suffix _speech600x
```

Why: Prefers continuity and intelligibility over texture animation; avoids choppy stochastic artifacts on speech sources.

Auto-profile plan preview

The following session demonstrates the auto-profile plan preview in practice.

```
pvx voc input.wav --auto-profile --auto-transform --explain-plan
```

Why: Prints the resolved profile/config plan before long renders.

Multi-resolution fusion stretch

The example below puts the multi-resolution fusion stretch into executable form.

```
pvx voc input.wav --multires-fusion --multires-ffts 1024,2048,4096 --multires-weights 0.2,0.35,0.45 --time-stretch 1.4 --output-dir out --suffix _multires
```

Why: Blends several FFT scales to reduce single-resolution bias on complex program material.

Checkpointed long render with manifest

The example below puts the checkpointed long render with manifest into executable form.

```
pvx voc long.wav --time-stretch 12 --auto-segment-seconds 0.5 --checkpoint-dir checkpoints --manifest-json reports/run_manifest.json --output-dir out --suffix _long
```

Why: Caches segment renders for resume workflows and writes run metadata for reproducibility.

20.7 Pipelines

Several distinct concerns meet in the pipelines; they are taken in turn below.

Time-stretch -> denoise -> dereverb in one pipe

A working command makes the time-stretch -> denoise -> dereverb in one pipe concrete.

```
pvx voc input.wav --time-stretch 1.25 --stdout | pvx denoise - --reduction-db 10 --stdout | pvx dereverb - --strength 0.45 --output-dir out --suffix _clean
```

Why: Single-pass CLI chain for serial DSP in Unix pipes.

Morph -> formant -> unison

A working command makes the morph -> formant -> unison concrete.

```
pvx morph a.wav b.wav -o - | pvx formant - --mode preserve --stdout | pvx unison - --voices 5 --detune-cents 8 --output-dir out --suffix _morph_stack
```

Why: Builds a richer timbre chain with no intermediate files.

Pitch-follow sidechain map (A controls B)

A working command makes the pitch-follow sidechain map (a controls b) concrete.

```
pvx pitch-track A.wav | pvx voc B.wav --pitch-follow-stdin --pitch-conf-min 0.75 --pitch-lowconf-mode hold --time-stretch-factor 1.0 --output output.wav
```

Why: Tracks F0 contour from source A and applies it as a dynamic pitch-ratio control map on source B.

20.8 Spatial

Several distinct concerns meet in the spatial; they are taken in turn below.

VBAP adaptive panning via algorithm dispatcher

The following session demonstrates the vbap adaptive panning via algorithm dispatcher in practice.

```
python3 -m pvx.algorithms.spatial_and_multichannel.imaging_and_panning.vbap_adaptive_panning input.wav --output-channels 6 --azimuth-deg 35 --width 0.8 --output out/vbap.wav
```

Why: Demonstrates algorithm-level spatial module invocation.

20.9 Transform selection

Several distinct concerns meet in the transform selection; they are taken in turn below.

Default production backend (FFT + transient protection)

The following session demonstrates the default production backend (FFT + transient protection) in practice.

```
pvx voc mix.wav --transform fft --time-stretch 1.07 --transient-preserve --phase-locking identity --output-dir out --suffix _fft
```

Why: Use when you need the fastest and most stable general-purpose phase-vocoder path.

Reference Fourier baseline using explicit DFT mode

This reference introduces the reference fourier baseline using explicit dft mode before presenting its individual entries.

```
pvx voc tone_sweep.wav --transform dft --time-stretch 1.00 --pitch-shift-semitones 0 --output-dir out --suffix _dft_ref
```

Why: Useful for parity checks and controlled transform-comparison experiments.

Prime-size frame experiment with CZT backend

The following session demonstrates the prime-size frame experiment with czf backend in practice.

```
pvx voc archival_take.wav --transform czt --n-fft 1531 --win-length 1531 --hop-size 382 --time-stretch 1.03
--output-dir out --suffix _czt
```

Why: Alternative numerical path for awkward/prime frame sizes when validating edge cases.

DCT timbral compaction for smooth harmonic material

The following session demonstrates the dct timbral compaction for smooth harmonic material in practice.

```
pvx voc strings.wav --transform dct --pitch-shift-cents -18 --soft-clip-level 0.95 --output-dir out --suffix
_dct
```

Why: Real-basis coefficients can emphasize envelope-like structure for creative reshaping.

DST odd-symmetry color pass

A working command makes the dst odd-symmetry color pass concrete.

```
pvx voc snare_loop.wav --transform dst --time-stretch 0.92 --phase-locking off --output-dir out --suffix
_dst
```

Why: Provides an alternate real-basis artifact profile useful for creative percussive processing.

Hartley real-basis exploratory render

A working command makes the hartley real-basis exploratory render concrete.

```
pvx voc synth_pad.wav --transform hartley --time-stretch 1.30 --pitch-shift-semitones 3 --output-dir out --
suffix _hartley
```

Why: Compares Hartley-domain behavior against complex FFT phase-vocoder output.

A/B sweep of transform backends from shell loop

The example below puts the a/b sweep of transform backends from shell loop into executable form.

```
for t in fft dft czt dct dst hartley; do pvx voc voice.wav --transform "$t" --time-stretch 1.1 --output-dir
out --suffix "_$t"; done
```

Why: Fast listening workflow for selecting the least-artifact transform on your source.

20.10 Notes

These are the details that matter for notes.

- Use `--stdout/-` to chain tools without intermediate files.
- Add `--quiet` for script-driven runs; use default verbosity for live progress bars.
- For production mastering, validate true peaks and loudness after all nonlinear stages.

Part VI

Operations and Delivery

CHAPTER 21

pvx Benchmarks

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

Reproducible benchmark summary for core short-time Fourier transform/inverse short-time Fourier transform (STFT/ISTFT) path across central processing unit/Compute Unified Device Architecture/Apple-Silicon-native contexts.

21.1 Reproduce

A working command makes the reproduce concrete.

```
python3 scripts/scripts_generate_docs_extras.py --run-benchmarks
```

21.2 Benchmark Spec

Before applying benchmark spec, keep these constraints in view.

- Sample rate: 48000 Hz
- Duration: 4.0 s
- Signal suite size: 4
- Signal recipe: deterministic synthetic suite: tonal, speech-like, transient stereo, chirp/texture stereo
- STFT config: `n_fft=2048`, `win_length=2048`, `hop_size=512`, `window=hann`, `center=True`

The benchmark spec records identify each signal and the role it plays in the test set.

Signal: `tonal_ramp_mono`. **Channels:** 1. **Duration (s):** 4.000. **Description:** Deterministic harmonic blend with slow ramp.

Signal: `speech_like_mono`. **Channels:** 1. **Duration (s):** 4.000. **Description:** Speech-like harmonic formants with sparse glottal-like impulses.

Signal: `transient_drums_stereo`. **Channels:** 2. **Duration (s):** 4.000. **Description:** Percussive burst train for onset retention and stereo timing drift.

Signal: `chirp_texture_stereo`. **Channels:** 2. **Duration (s):** 4.000. **Description:** Wideband chirp-plus-texture to stress spectral and phase stability.

21.3 Host

For host, start with these practical details.

- Platform: `macOS-15.1.1-arm64-arm-64bit-Mach-0`
- Machine: `arm64`
- Python: `3.14.3`

21.4 Backend Summary

The backend summary results pair each backend with its timing, quality, and resource measurements.

Backend: cpu. **Status:** ok. **Cases (ok/total):** 4/4. **Mean xRT:** 146.39. **Mean elapsed (ms):** 31.02. **Peak host memory (MB):** 17.70. **Mean quality score (/100):** 97.50. **Mean SNR in (dB):** 100.980. **Mean SI-SDR (dB):** 153.813. **Mean LSD (dB):** 0.0000. **Mean ModSpec:** 0.0000. **Mean Smear:** 0.0000. **Mean EnvCorr:** 1.0000. **Mean Coherence Drift:** 0.0000. **Mean Phasiness:** 0.0511. **Mean Musical Noise:** 0.0000. **Mean SNR vs CPU (dB):** n/a. **Mean LSD vs CPU (dB):** n/a.

Backend: cuda. **Status:** unavailable. **Cases (ok/total):** 0/4. **Mean xRT:** n/a. **Mean elapsed (ms):** n/a. **Peak host memory (MB):** n/a. **Mean quality score (/100):** n/a. **Mean SNR in (dB):** n/a. **Mean SI-SDR (dB):** n/a. **Mean LSD (dB):** n/a. **Mean ModSpec:** n/a. **Mean Smear:** n/a. **Mean EnvCorr:** n/a. **Mean Coherence Drift:** n/a. **Mean Phasiness:** n/a. **Mean Musical Noise:** n/a. **Mean SNR vs CPU (dB):** n/a. **Mean LSD vs CPU (dB):** n/a. **Notes:** CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.

Backend: apple_silicon_native_cpu. **Status:** ok. **Cases (ok/total):** 4/4. **Mean xRT:** 147.28. **Mean elapsed (ms):** 30.58. **Peak host memory (MB):** 17.70. **Mean quality score (/100):** 97.50. **Mean SNR in (dB):** 100.980. **Mean SI-SDR (dB):** 153.813. **Mean LSD (dB):** 0.0000. **Mean ModSpec:** 0.0000. **Mean Smear:** 0.0000. **Mean EnvCorr:** 1.0000. **Mean Coherence Drift:** 0.0000. **Mean Phasiness:** 0.0511. **Mean Musical Noise:** 0.0000. **Mean SNR vs CPU (dB):** 100.980. **Mean LSD vs CPU (dB):** 0.0000.

21.5 Per-Signal Results

The per-signal results results pair each backend with its timing, quality, and resource measurements.

Backend: apple_silicon_native_cpu. **Signal:** chirp_texture_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** ok. **xRT:** 96.99. **Elapsed (ms):** 41.24. **Quality (/100):** 100.00. **SNR in (dB):** 102.306. **SI-SDR (dB):** 155.139. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** 0.0000. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** 102.306. **LSD vs CPU (dB):** 0.0000.

Backend: apple_silicon_native_cpu. **Signal:** speech_like_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** ok. **xRT:** 197.66. **Elapsed (ms):** 20.24. **Quality (/100):** 100.00. **SNR in (dB):** 99.925. **SI-SDR (dB):** 152.758. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** n/a. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** 99.925. **LSD vs CPU (dB):** 0.0000.

Backend: apple_silicon_native_cpu. **Signal:** tonal_ramp_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** ok. **xRT:** 195.35. **Elapsed (ms):** 20.48. **Quality (/100):** 100.00. **SNR in (dB):** 107.760. **SI-SDR (dB):** 160.593. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** n/a. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** 107.760. **LSD vs CPU (dB):** 0.0000.

Backend: apple_silicon_native_cpu. **Signal:** transient_drums_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** ok. **xRT:** 99.12. **Elapsed (ms):** 40.36. **Quality (/100):** 90.00. **SNR in (dB):** 93.930. **SI-SDR (dB):** 146.763. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** 0.0000. **Phasiness:** 0.2042. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** 93.930. **LSD vs CPU (dB):** 0.0000.

Backend: cpu. **Signal:** chirp_texture_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** ok. **xRT:** 93.13. **Elapsed (ms):** 42.95. **Quality (/100):** 100.00. **SNR in (dB):** 102.306. **SI-SDR (dB):** 155.139. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** 0.0000. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a.

Backend: cpu. **Signal:** speech_like_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** ok. **xRT:** 198.94. **Elapsed (ms):** 20.11. **Quality (/100):** 100.00. **SNR in (dB):** 99.925. **SI-SDR (dB):** 152.758. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** n/a. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a.

Backend: cpu. **Signal:** tonal_ramp_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** ok. **xRT:** 194.70. **Elapsed (ms):** 20.54. **Quality (/100):** 100.00. **SNR in (dB):** 107.760. **SI-SDR (dB):** 160.593. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** n/a. **Phasiness:** 0.0000. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a.

Backend: cpu. **Signal:** transient_drums_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** ok. **xRT:** 98.77. **Elapsed (ms):** 40.50. **Quality (/100):** 90.00. **SNR in (dB):** 93.930. **SI-SDR (dB):** 146.763. **LSD (dB):** 0.0000. **ModSpec:** 0.0000. **Smear:** 0.0000. **EnvCorr:** 1.0000. **Coherence Drift:** 0.0000. **Phasiness:** 0.2042. **Musical Noise:** 0.0000. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a.

Backend: cuda. **Signal:** chirp_texture_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** unavailable. **xRT:** n/a. **Elapsed (ms):** n/a. **Quality (/100):** n/a. **SNR in (dB):** n/a. **SI-SDR (dB):** n/a. **LSD (dB):** n/a. **ModSpec:** n/a. **Smear:** n/a. **EnvCorr:** n/a. **Coherence Drift:** n/a. **Phasiness:** n/a. **Musical Noise:** n/a. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a. **Notes:** CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.

Backend: cuda. **Signal:** speech_like_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** unavailable. **xRT:** n/a. **Elapsed (ms):** n/a. **Quality (/100):** n/a. **SNR in (dB):** n/a. **SI-SDR (dB):** n/a. **LSD (dB):** n/a. **ModSpec:** n/a. **Smear:** n/a. **EnvCorr:** n/a. **Coherence Drift:** n/a. **Phasiness:** n/a. **Musical Noise:** n/a. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a. **Notes:** CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.

Backend: cuda. **Signal:** tonal_ramp_mono. **Channels:** 1. **Duration (s):** 4.000. **Status:** unavailable. **xRT:** n/a. **Elapsed (ms):** n/a. **Quality (/100):** n/a. **SNR in (dB):** n/a. **SI-SDR (dB):** n/a. **LSD (dB):** n/a. **ModSpec:** n/a. **Smear:** n/a. **EnvCorr:** n/a. **Coherence Drift:** n/a. **Phasiness:** n/a. **Musical Noise:** n/a. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a. **Notes:** CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.

Backend: cuda. **Signal:** transient_drums_stereo. **Channels:** 2. **Duration (s):** 4.000. **Status:** unavailable. **xRT:** n/a. **Elapsed (ms):** n/a. **Quality (/100):** n/a. **SNR in (dB):** n/a. **SI-SDR (dB):** n/a. **LSD (dB):** n/a. **ModSpec:** n/a. **Smear:** n/a. **EnvCorr:** n/a. **Coherence Drift:** n/a. **Phasiness:** n/a. **Musical Noise:** n/a. **SNR vs CPU (dB):** n/a. **LSD vs CPU (dB):** n/a. **Notes:** CUDA mode requires CuPy. Install a matching cupy-cudaXXx package.

Interpretation notes: - **xRT** (times real-time): higher is faster. - **Quality (/100)** is a composite from core artifact metrics; use it as a quick ranking aid, not as a single acceptance criterion. - Lower is better for LSD, ModSpec, Smear, Coherence Drift, Phasiness, and Musical Noise. - Higher is better for SNR, SI-SDR, and Envelope Correlation.

Raw machine-readable benchmark output: [docs/benchmarks/latest.json](https://docs.benchmarks/latest.json).

CHAPTER 22

Installing and Managing pvx with Homebrew

Homebrew is the supported installation path for macOS users who want the `pvx` command suite without managing a project checkout or Python virtual environment. The formula is published through the `TheColby/pvx` tap because Homebrew requires third-party formulae to live in a tap. Installing `Formula/pvx.rb` directly from a local path is not supported by current Homebrew releases.

22.1 Stable Installation

The stable formula installs the tagged alpha release. Homebrew fetches the release archive, verifies its SHA-256 checksum, installs `libsndfile`, creates an isolated Python environment, and links the command-line programs into the Homebrew prefix.

```
brew tap TheColby/pvx
brew install TheColby/pvx/pvx
```

The tap command only needs to be run once. After installation, verify both Homebrew's record and the primary command:

```
brew list --versions pvx
pvx --help
pvxvoc --help
```

Homebrew 6 requires trust for third-party tap content. Homebrew can trust the selected formula during installation; when it instead reports that the tap is not trusted, grant trust only to the `pvx` formula and retry:

```
brew trust --formula TheColby/pvx/pvx
brew install TheColby/pvx/pvx
```

Formula-level trust is narrower than trusting the entire tap. Inspect the formula in the tap repository before granting trust, particularly on a shared or security-sensitive machine.

The formula installs every command entry point shipped by the Python package. The alpha support promise remains deliberately narrower: `pvx`, `pvxvoc`, `pvxfreeze`, `pvxwarp`, `pvxformant`, `pvxfilter`, `pvxre-tune`, and `pvxanalysis` are the stable command-line surface documented for production scripts.

22.2 Development Installation

The `HEAD` formula follows the latest commit on the repository's `main` branch. It is useful for testing an unreleased fix, but its behavior can move ahead of the guide and tagged release.

```
brew install --HEAD TheColby/pvx/pvx
```

Homebrew does not automatically replace a stable installation with HEAD. Reinstall explicitly when switching tracks:

```
brew uninstall pvx
brew install --HEAD TheColby/pvx/pvx
```

Return to the stable release in the same way:

```
brew uninstall pvx
brew install TheColby/pvx/pvx
```

22.3 Upgrading

Homebrew updates the tap metadata before upgrading the package. These commands show the available formula and install a newer tagged version when one has been published:

```
brew update
brew info TheColby/pvx/pvx
brew upgrade pvx
```

A HEAD installation is rebuilt from the latest repository state with:

```
brew reinstall --HEAD TheColby/pvx/pvx
```

Existing audio files, render outputs, presets, and manifests are not stored inside the Homebrew cellar and are not removed by an upgrade.

22.4 Uninstalling

Uninstalling removes the Homebrew-managed virtual environment and linked commands. It does not delete projects or rendered audio in user directories.

```
brew uninstall pvx
```

Remove the tap as well when no formula from it is needed:

```
brew untap TheColby/pvx
```

22.5 What the Formula Installs

The formula uses Homebrew's `Language::Python::Virtualenv` support. `pvx` and its Python runtime dependencies live under the formula's private `libexec` directory, while small launcher scripts are linked into `$(brew --prefix)/bin`. This prevents `pvx` packages from modifying the system Python or an activated project environment.

`libsndfile` is a Homebrew dependency because `pvx` uses it for audio file input and output through the Python `soundfile` package. The stable formula currently uses Homebrew's `python@3.12`. Optional machine-learning integrations such as PyTorch and TensorFlow are intentionally excluded from the base formula because of their size and platform-specific installation requirements.

The formula also installs available manual pages from `man/man1`. Homebrew exposes them through its normal manual-page path, so a supported command can be inspected with a command such as:

```
man pvx
```

22.6 Shell Path

Homebrew normally configures its binary directory during Homebrew installation. If the `brew` command works but `pvx` is not found after a successful install, inspect the active prefix and shell path:

```
brew --prefix
command -v brew
command -v pvx
printf '%s\n' "$PATH"
```

Apple Silicon Homebrew normally uses `/opt/homebrew`; Intel Homebrew normally uses `/usr/local`. Homebrew's recommended shell environment can be loaded on Apple Silicon with:

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Place the corresponding `brew shellenv` command in the shell startup file only when Homebrew itself is not already configuring the path.

22.7 Troubleshooting

A failed install is easiest to diagnose by separating tap state, formula state, dependencies, and command linking. Begin with the following inspection commands:

```
brew update
brew tap
brew info TheColby/pvx/pvx
brew doctor
```

If Homebrew reports that a local `Formula/pvx.rb` is rejected, install through the tap instead:

```
brew tap TheColby/pvx
brew install TheColby/pvx/pvx
```

If an interrupted build left partial state, remove and reinstall the formula:

```
brew uninstall --force pvx
brew cleanup pvx
brew install TheColby/pvx/pvx
```

If Homebrew reports that the Xcode Command Line Tools are outdated, update them through System Settings under General and Software Update. Homebrew may require a newer Command Line Tools release even when the `clang` compiler already works for ordinary local builds. Complete that system update before retrying the `pvx` formula.

If audio files fail to open after installation, confirm that Homebrew installed and linked `libsndfile`, then run the `pvx` smoke test:

```
brew list --versions libsndfile
pvx smoke
```

The full Homebrew build log is available through Homebrew’s usual diagnostic output. Include `brew config`, `brew info TheColby/pvx/pvx`, the failing command, and the relevant log tail in a bug report.

22.8 Maintainer Release Flow

The repository keeps the source formula at `Formula/pvx.rb`. A tagged release must have a stable archive URL, its exact SHA-256 checksum, and an explicit version before the formula is copied to the tap. The refresh helper is repeatable, so it can update both a previously stamped formula and a newly prepared release.

```
./scripts/refresh_homebrew_formula.sh v0.1.0a1
ruby -c Formula/pvx.rb
python3 scripts/scripts_sync_homebrew_tap_formula.py ../homebrew-pvx
```

The tap checkout should then be tested as Homebrew sees it:

```
brew style TheColby/pvx/pvx
brew audit --strict TheColby/pvx/pvx
brew install --build-from-source TheColby/pvx/pvx
brew test TheColby/pvx/pvx
```

After a successful test, commit and push `Formula/pvx.rb` in `TheColby/homebrew-pvx`. The tag-driven release workflow also validates Ruby syntax and uploads the stamped formula beside the Python distributions, providing one release artifact from which the tap can be synchronized.

22.9 Formula Version Policy

The stable formula follows tagged pvx releases and never points at a moving branch. `HEAD` is the only branch-following installation. This separation keeps normal `brew upgrade` behavior reproducible while retaining an explicit route for development testing.

The Homebrew formula is a distribution mechanism, not a wider stability promise. Alpha, beta, and experimental command tiers remain governed by the supported-surface chapter and release notes. A successful installation confirms packaging and executable discovery; it does not promote experimental commands into the stable interface.

CHAPTER 23

Alpha Release Guide

This page is the release-facing contract for `pvx` alpha builds.

23.1 Supported Surface

Stable entry points for 0.1.0a1: - `pvx` - `pvxvoc` - `pvxfreeze` - `pvxwarp` - `pvxformant` - `pvxfilter` - `pvxtune` - `pvxanalysis`

Beta entry points: - functional and documented, but flags may still move in minor releases

Experimental entry points: - useful for exploration - not promised stable for scripting

Compatibility shims: - `pvxalgorithms`, `pvxalgorithms.base`, and `pvxalgorithms.registry` remain available only as deprecated migration shims

23.2 Release Checklist

Before tagging an alpha build:

1. Run `uv run python scripts/scripts_alpha_check.py` Optional convenience wrapper: `make alpha-check` when local `make` is available
2. Run `uv run pytest -q`
3. Run `uv run python benchmarks/run_augment_profile_suite.py --quick --gate --out-dir benchmarks/out_augment_profiles_release`
4. Confirm `README.md`, `RELEASE.md`, and `pyproject.toml` agree on the stable/beta/experimental command surface
5. If publishing a tagged formula, run `./scripts/refresh_homebrew_formula.sh v0.1.0a1`, sync it with `python3 scripts/scripts_sync_homebrew_tap_formula.py ../homebrew-pvx`, publish the refreshed formula to `TheColby/homebrew-pvx`, and verify the tap instructions still match `docs/HOMEBREW.md`
6. Regenerate source docs with `make docs`
7. Regenerate HTML/man outputs only when user-facing rendered docs changed or when preparing the tag with `make docs-generated`
8. Make sure deprecated compatibility layers still warn cleanly and still route to the canonical modules

23.3 Alpha Principles

Alpha Principles depends on a few concrete choices.

- prefer a smaller honest surface over a wider unstable one
- keep compatibility shims explicit and noisy rather than magical
- publish only the supported surface as the alpha contract; everything else is docs/reference, not a promise
- keep the supported-slice coverage gate at 100%
- treat docs source files as the review artifact; generated outputs are release artifacts
- add direct seam tests when extracting modules from `pvx.py` or `pvx.core.voc`

APPENDIX A

pvx Command-Line Interface (CLI) Flags Reference

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

This file enumerates long-form CLI flags discovered from argparse declarations in canonical pvx CLI sources.

Total tool+flag entries: **476** Total unique long flags: **338**

A.1 Unique Long Flags

--a4-reference-hz, --alpha, --ambient-phase-mix, --ambient-preset, --amplitude, --analysis-channel, --append-manifest, --attack-sec, --attenuation, --audit-metrics, --auto-profile, --auto-profile-lookahead-seconds, --auto-segment-seconds, --auto-transform, --backend, --background-dir, --band, --band-gain-db, --band-width-bins, --batch-size, --bit-depth, --bits, --blend-mode, --boost-db, --budget-path, --cache-dir, --category, --center, --cents, --checkpoint-dir, --checkpoint-id, --chord, --chunk-ms, --chunk-seconds, --clip, --codec, --coherence-strength, --comp-ratio, --comp-threshold-db, --companioner-attack-ms, --companioner-compress-ratio, --companioner-expand-ratio, --companioner-makeup-db, --companioner-release-ms, --companioner-threshold-db, --compressor-attack-ms, --compressor-makeup-db, --compressor-ratio, --compressor-release-ms, --compressor-threshold-db, --confidence-floor, --context-ms, --control-stdin, --coord-system, --cpu, --crossfade-ms, --cuda-device, --cycles, --decay, --decay-sec, --dedupe-by, --depth, --detune-cents, --device, --disk-budget, --distance-law, --dither, --dither-seed, --drr, --dry, --dry-mix, --dry-run, --duration, --duty-cycle, --emit, --end, --engine, --envelope-lifter, --example, --exp-curve, --expand-ratio, --expander-attack-ms, --expander-ratio, --expander-release-ms, --expander-threshold-db, --explain-plan, --exponent, --extreme-stretch-threshold, --extreme-time-stretch, --f0-max, --f0-min, --factor, --fail-if-exceeds, --feature-set, --feedback, --fill, --floor, --fmax, --fmin, --force, --force-stereo, --formant-lifter, --formant-max-gain-db, --formant-shift-ratio, --formant-strength, --format, --fourier-sync, --fourier-sync-max-fft, --fourier-sync-min-fft, --fourier-sync-smooth, --frame-length, --freeze-time, --freq, --freq-mask, --frequency-hz, --gain, --gains, --gpu, --group-separator, --grouping, --guided, --hard-clip-level, --harmonic-gain, --harmonic-kernel, --harmonic-pitch-cents, --harmonic-pitch-semitones, --harmonic-stretch, --hop-length, --hop-size, --inharmonic-f0-hz, --inharmonic-mix, --inharmonic-pitch, --input-format, --intent, --interp, --intervals, --intervals-cents, --ir, --ir-database, --ir-dir, --ir-file, --json, --kaiser-beta, --keep-intermediate, --key, --label-policy, --labels-csv, --limiter-threshold, --list-databases, --manifest-append, --manifest-csv, --manifest-json, --manifest-jsonl, --map, --mask-exponent, --material, --max, --max-length-s, --max-stage-stretch, --metadata-policy, --method, --mfcc-count, --min, --mix, --mode, --multires-ffts, --multires-fusion, --multires-weights, --n-fft, --no-audit-metrics, --no-center, --no-normalize-gains, --no-onset-realign, --no-progress, --no-trim, --noise-file, --noise-floor, --noise-seconds, --noise-type, --normalize, --normalize-energy, --normalize-gains, --num-freq-masks, --num-time-masks, --offset, --onset-credit-max, --onset-credit-pull, --onset-time-credit, --operation, --operator, --order, --out, --output, --output-csv, --output-dir, --output-format, --output-jsonl, --output-key, --output-subtype, --output-suffix, --overlap-ms, --overwrite, --pair-mode, --pans, --peak, --peak-count,

--peak-dbfs, --percussive-gain, --percussive-kernel, --percussive-pitch-cents, --percussive-pitch-semitones, --percussive-stretch, --phase, --phase-engine, --phase-locking, --phase-mix, --phase-mode, --phase-rad, --phase-random-seed, --pipeline, --pipeline-config, --pitch, --pitch-conf-min, --pitch-follow-stdin, --pitch-lowconf-mode, --pitch-map, --pitch-map-crossfade-ms, --pitch-map-smooth-ms, --pitch-map-stdin, --pitch-mode, --pitch-shift-cents, --pitch-shift-ratio, --pitch-shift-semitones, --policy, --pre-delay-ms, --preset, --quality-profile, --quiet, --random-phase, --rate, --ratio, --ratio-max, --ratio-min, --ratio-reference, --recommend-root, --reduction-db, --ref-channel, --reference-hz, --release-sec, --requested-stretch, --resample-mode, --resonance-decay, --resonance-hz, --resonance-mix, --resonance-q, --response, --response-gain-db, --response-mix, --resume, --rms-dbfs, --root, --root-hz, --route, --rt60, --safety-margin, --sample-rate, --scale, --scale-cents, --seed, --semitones, --shift-bins, --silent, --sine-cycles, --sine-phase-rad, --smooth, --smooth-frames, --smoothing-bins, --snr, --soft-clip-drive, --soft-clip-level, --soft-clip-type, --speaker-angles, --split, --split-mode, --sr, --start, --stdout, --stereo-mode, --strength, --stretch, --stretch-from, --stretch-max, --stretch-min, --stretch-mode, --stretch-scale, --strict, --strict-manifest, --subtype, --suffix, --summary-json, --sustain, --target, --target-duration, --target-f0, --target-lufs, --target-max, --target-min, --target-pitch-shift-semitones, --target-sample-rate, --time-mask, --time-stretch, --time-stretch-factor, --tolerance-cents, --trajectory-shape, --transform, --transient-crossfade-ms, --transient-mode, --transient-preserve, --transient-protect-ms, --transient-sensitivity, --transient-threshold, --transpose-semitones, --true-peak-max-dbtp, --tv-interp, --tv-key, --tv-map, --tv-order, --variants-per-input, --verbose, --verbosity, --voices, --wave, --wet, --width, --win-length, --window, --work-dir, --workers

A.2 hps-pitch-track

hps-pitch-track accepts the options below; defaults and parser sources are included for reference.

Flag: --backend. **Required:** False. **Default:** auto. **Choices:** auto, pyin, acf. **Action:** (none). **Description:** Pitch backend (default: auto -> pyin if available, else acf). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --confidence-floor. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Set confidence below this floor to 0.0 (default: 0.0). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --emit. **Required:** False. **Default:** pitch_map. **Choices:** (none). **Action:** (none). **Description:** Output mode: pitch_map (default), stretch_map, or pitch_to_stretch. **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --feature-set. **Required:** False. **Default:** all. **Choices:** none, basic, advanced, all. **Action:** (none). **Description:** Feature tracking preset emitted as extra CSV columns. none/basic/advanced/all (default: all). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --fmax. **Required:** False. **Default:** 1200.0. **Choices:** (none). **Action:** (none). **Description:** Maximum F0 in Hz (default: 1200). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --fmin. **Required:** False. **Default:** 50.0. **Choices:** (none). **Action:** (none). **Description:** Minimum F0 in Hz (default: 50). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --frame-length. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** Frame length in samples (default: 2048). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --hop-size. **Required:** False. **Default:** 256. **Choices:** (none). **Action:** (none). **Description:** Hop size in samples (default: 256). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: --mfcc-count. **Required:** False. **Default:** 13. **Choices:** (none). **Action:** (none). **Description:** Number of MFCC columns (mfcc_01..mfcc_N) when feature-set is advanced/all (default: 13). **Source:** src/pvx/cli/hps_pitch_track.py.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output CSV path (default: '-' for stdout). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--ratio-max`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none). **Description:** Upper clamp for emitted `pitch_ratio` (default: 4.0). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--ratio-min`. **Required:** False. **Default:** 0.25. **Choices:** (none). **Action:** (none). **Description:** Lower clamp for emitted `pitch_ratio` (default: 0.25). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--ratio-reference`. **Required:** False. **Default:** median. **Choices:** median, mean, first, hz. **Action:** (none). **Description:** Reference for emitted `pitch_ratio` values (default: median voiced f0). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--reference-hz`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Reference frequency in Hz when `-ratio-reference` hz. **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--smooth-frames`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Smoothing window for `pitch_ratio` frames (default: 5). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--stretch`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Emit constant stretch column value for `-emit pitch_map` (default: 1.0). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--stretch-from`. **Required:** False. **Default:** `pitch_ratio`. **Choices:** (none). **Action:** (none). **Description:** Source signal used to derive stretch in stretch-oriented emit modes (default: `pitch_ratio`). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--stretch-max`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none). **Description:** Upper clamp for emitted stretch in stretch-oriented modes (default: 4.0). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--stretch-min`. **Required:** False. **Default:** 0.25. **Choices:** (none). **Action:** (none). **Description:** Lower clamp for emitted stretch in stretch-oriented modes (default: 0.25). **Source:** `src/pvx/cli/hps_pitch_track.py`.

Flag: `--stretch-scale`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Scale factor for derived stretch tracks (default: 1.0). **Source:** `src/pvx/cli/hps_pitch_track.py`.

A.3 pvx

pvx accepts the options below; defaults and parser sources are included for reference.

Flag: `--append-manifest`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Append/merge with existing manifest instead of replacing it. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--audit-metrics`. **Required:** False. **Default:** True. **Choices:** (none). **Action:** `store_true`. **Description:** Compute per-output audit metrics (default: on). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--backend`. **Required:** False. **Default:** auto. **Choices:** auto, pyin, acf. **Action:** (none). **Description:** Pitch tracker backend. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--batch-size`. **Required:** False. **Default:** 16. **Choices:** (none). **Action:** (none). **Description:** Number of files processed simultaneously on GPU (default: 16). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--bit-depth`. **Required:** False. **Default:** inherit. **Choices:** inherit, 16, 24, 32f. **Action:** (none). **Description:** Bit-depth assumption when `-subtype` is not set (default: inherit from input subtype). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--budget-path`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Path used to query free space when `--disk-budget` is omitted (default: current directory). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--chunk-seconds`. **Required:** False. **Default:** 0.25. **Choices:** (none). **Action:** (none). **Description:** Chunk/segment duration for `--auto-segment-seconds` (default: 0.25). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--confidence-floor`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Minimum tracker confidence. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--context-ms`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional stateful context window in milliseconds (default: auto from window/hop). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--crossfade-ms`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Crossfade used for segment assembly in milliseconds (default: 0.0). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--dedupe-by`. **Required:** False. **Default:** `output_path`. **Choices:** (none). **Action:** (none). **Description:** Deduplication key (default: `output_path`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--device`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `cuda`, `mps`, `cpu`. **Action:** (none). **Description:** Compute device (default: `auto`, `CUDA` > `MPS` > `CPU`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--disk-budget`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Total budget size (e.g., 500MB, 20GB, 2TiB). If omitted, use free space at `--budget-path`. **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--dry-run`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Plan manifest and filenames without rendering audio. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--duration`. **Required:** False. **Default:** 0.3. **Choices:** (none). **Action:** (none). **Description:** Synthetic input duration seconds (default: 0.30). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--emit`. **Required:** False. **Default:** `pitch_to_stretch`. **Choices:** `pitch_map`, `stretch_map`, `pitch_to_stretch`. **Action:** (none). **Description:** Control map emit mode for the guide track (default: `pitch_to_stretch`). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--engine`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `pytorch`, `torchaudio`, `wavelet`, `pvx-cli`. **Action:** (none). **Description:** DSP engine for time-stretch and pitch-shift transforms. ‘auto’ prefers `torchaudio` > `pytorch` > `pvx-cli`. ‘torchaudio’ uses `torchaudio.functional.phase_vocoder`. ‘pytorch’ uses native PyTorch phase vocoder. ‘wavelet’ uses the experimental wavelet backend. ‘pvx-cli’ always uses subprocess. (default: `auto`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--example`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Print follow example command(s) and exit. Use `--example` for basic or `--example all` for the full set. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--fail-if-exceeds`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Return non-zero when `--requested-stretch` does not fit the usable budget. **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--feature-set`. **Required:** False. **Default:** `all`. **Choices:** `none`, `basic`, `advanced`, `all`. **Action:** (none). **Description:** Feature columns emitted by pitch tracker (default: `all`). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--fmax`. **Required:** False. **Default:** 1200.0. **Choices:** (none). **Action:** (none). **Description:** Maximum tracked f0 in Hz. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--fmin`. **Required:** False. **Default:** 50.0. **Choices:** (none). **Action:** (none). **Description:** Minimum tracked f0 in Hz. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--frame-length`. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** Tracker frame length in samples. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--group-separator`. **Required:** False. **Default:** `__`. **Choices:** (none). **Action:** (none). **Description:** Separator used by stem-prefix grouping (default: `'__'`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--grouping`. **Required:** False. **Default:** `stem-prefix`. **Choices:** `none`, `stem-prefix`. **Action:** (none). **Description:** Split-grouping strategy (default: `stem-prefix`). `stem-prefix` keeps variants from similarly named sources in one split. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--hop-size`. **Required:** False. **Default:** 256. **Choices:** (none). **Action:** (none). **Description:** Tracker hop size in samples. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--intent`. **Required:** False. **Default:** `asr_robust`. **Choices:** `asr_robust`, `mir_music`, `ssl_contrastive`. **Action:** (none). **Description:** Built-in augmentation intent profile when `-pipeline-config` is not given (default: `asr_robust`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Emit JSON output. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--keep-intermediate`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Keep intermediate stage files after successful completion. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--label-policy`. **Required:** False. **Default:** `allow_alter`. **Choices:** `allow_alter`, `preserve`. **Action:** (none). **Description:** Label perturbation policy (default: `allow_alter`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--labels-csv`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional metadata CSV with path/label/speaker columns for balanced split modes. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--manifest-csv`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional CSV manifest path (default: `/augment_manifest.csv`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--manifest-jsonl`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional JSONL manifest path (default: `/augment_manifest.jsonl`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--material`. **Required:** False. **Default:** `mix`. **Choices:** `mix`, `speech`, `vocal`, `drums`, `ambient`. **Action:** (none). **Description:** Material profile for `pvx safe` command generation. **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--max-length-s`. **Required:** False. **Default:** 30.0. **Choices:** (none). **Action:** (none). **Description:** Maximum file duration in seconds (default: 30.0). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--mfcc-count`. **Required:** False. **Default:** 13. **Choices:** (none). **Action:** (none). **Description:** MFCC column count emitted by pitch tracker (default: 13). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--mode`. **Required:** False. **Default:** `stateful`. **Choices:** `stateful`, `wrapper`. **Action:** (none). **Description:** Stream engine: `stateful` chunk processor (default) or `wrapper` compatibility mode. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--no-audit-metrics`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_false`. **Description:** Disable audit metric computation. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--no-progress`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Suppress progress messages. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--out`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio path. **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--output`. **Required:** False. **Default:** `output.wav`. **Choices:** (none). **Action:** (none). **Description:** Output audio path (default: `output.wav`). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--output-csv`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional merged CSV output path. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--output-dir`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output directory for augmented files. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--output-format`. **Required:** False. **Default:** wav. **Choices:** wav, flac, aiff, ogg, caf. **Action:** (none). **Description:** Output format container (default: wav). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--output-jsonl`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Merged JSONL output path. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--output-subtype`. **Required:** False. **Default:** PCM_16. **Choices:** PCM_16, PCM_24, PCM_32, FLOAT. **Action:** (none). **Description:** Output sample format (default: PCM_16). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--output-suffix`. **Required:** False. **Default:** `_aug`. **Choices:** (none). **Action:** (none). **Description:** Suffix appended before extension (default: `_aug`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--overwrite`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Overwrite existing outputs. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--pair-mode`. **Required:** False. **Default:** off. **Choices:** off, contrastive2. **Action:** (none). **Description:** Pair-view mode: off or two-view contrastive output (default: off). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--pipeline`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pipeline string with stages separated by `' | '`. Example: `"voc -stretch 1.2 | formant -mode preserve"`. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--pipeline-config`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional YAML/JSON pipeline manifest (see `pvx.augment.config.load_pipeline`). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--pitch`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Smoke render pitch semitones (default: 0.0). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--pitch-conf-min`. **Required:** False. **Default:** 0.75. **Choices:** (none). **Action:** (none). **Description:** Minimum accepted map confidence for pvx voc (default: 0.75). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--pitch-lowconf-mode`. **Required:** False. **Default:** hold. **Choices:** hold, unity, interp. **Action:** (none). **Description:** Low-confidence handling mode in pvx voc (default: hold). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--pitch-map-crossfade-ms`. **Required:** False. **Default:** 20.0. **Choices:** (none). **Action:** (none). **Description:** Map segment crossfade in pvx voc (milliseconds, default: 20). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--pitch-map-smooth-ms`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Additional map smoothing in pvx voc (milliseconds). **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--policy`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional JSON policy file with augmentation defaults and bounds/choices overrides. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--quiet`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Reduce logs. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--ratio-max`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none). **Description:** Maximum pitch_ratio clamp. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--ratio-min`. **Required:** False. **Default:** 0.25. **Choices:** (none). **Action:** (none). **Description:** Minimum pitch_ratio clamp. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--ratio-reference`. **Required:** False. **Default:** median. **Choices:** median, mean, first, hz. **Action:** (none). **Description:** Reference mode for pitch_ratio derivation in tracking. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--reference-hz`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Reference Hz when `--ratio-reference` hz. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--requested-stretch`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional stretch ratio to evaluate against the computed budget. **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--resume`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Skip already-rendered outputs found in existing manifest/output-dir. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--route`. **Required:** False. **Default:** []. **Choices:** (none). **Action:** `append`. **Description:** Optional pvx voc control route expression. Repeat to chain. Example: `--route stretch=pitch_ratio --route pitch_ratio=const(1.0)`. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--safety-margin`. **Required:** False. **Default:** 0.9. **Choices:** (none). **Action:** (none). **Description:** Usable fraction of budget in (0, 1]; default: 0.90 (10%% headroom). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--sample-rate`. **Required:** False. **Default:** 24000. **Choices:** (none). **Action:** (none). **Description:** Synthetic input sample rate (default: 24000). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--seed`. **Required:** False. **Default:** 1337. **Choices:** (none). **Action:** (none). **Description:** Deterministic base seed (default: 1337). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--silent`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Suppress logs. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--smooth-frames`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Smoothing window in frames. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--split`. **Required:** False. **Default:** 0.8,0.1,0.1. **Choices:** (none). **Action:** (none). **Description:** train, val, test split ratios (default: 0.8, 0.1, 0.1). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--split-mode`. **Required:** False. **Default:** random. **Choices:** random, label_balanced, speaker_balanced. **Action:** (none). **Description:** Split assignment mode (default: random). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--sr`. **Required:** False. **Default:** 16000. **Choices:** (none). **Action:** (none). **Description:** Target sample rate (default: 16000). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--stretch`. **Required:** False. **Default:** 1.25. **Choices:** (none). **Action:** (none). **Description:** Smoke render stretch factor (default: 1.25). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--stretch-from`. **Required:** False. **Default:** pitch_ratio. **Choices:** pitch_ratio, inv_pitch_ratio, f0_hz. **Action:** (none). **Description:** Source for deriving stretch in stretch-oriented emit modes. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--stretch-max`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none). **Description:** Upper clamp for derived stretch. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--stretch-min`. **Required:** False. **Default:** 0.25. **Choices:** (none). **Action:** (none). **Description:** Lower clamp for derived stretch. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--stretch-scale`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Scale factor for derived stretch track. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--strict`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Return non-zero on validation errors. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--strict-manifest`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Return non-zero if manifest validation errors are found. **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--subtype`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Explicit libsndfile subtype assumption (e.g., PCM_16, PCM_24, FLOAT). **Source:** `src/pvx/cli/pvx_helpers.py`.

Flag: `--variants-per-input`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Number of augmented outputs per input file (default: 3). **Source:** `src/pvx/cli/pvx_augment.py`.

Flag: `--work-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional directory for intermediate stage files. **Source:** `src/pvx/cli/pvx_pipeline.py`.

Flag: `--workers`. **Required:** False. **Default:** 1. **Choices:** (none). **Action:** (none). **Description:** Parallel worker count for rendering (default: 1). **Source:** `src/pvx/cli/pvx_augment.py`.

A.4 pvxanalysis

pvxanalysis accepts the options below; defaults and parser sources are included for reference.

Flag: `--analysis-channel`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Channel index to analyze; -1 means all channels (default: -1). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--cuda-device`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** CUDA device index for `-device cuda/auto` (default: 0). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--device`. **Required:** False. **Default:** auto. **Choices:** auto, cpu, cuda. **Action:** (none). **Description:** Compute device selection (default: auto). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--hop-size`. **Required:** False. **Default:** 512. **Choices:** (none). **Action:** (none). **Description:** Hop size (default: 512). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Print summary as JSON. **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--kaiser-beta`. **Required:** False. **Default:** 14.0. **Choices:** (none). **Action:** (none). **Description:** Kaiser beta when `-window kaiser` (default: 14.0). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--n-fft`. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** STFT FFT size (default: 2048). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--no-center`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Disable centered framing. **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output PVXAN path (default: `.pvxan.npz`). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--summary-json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional path to write summary JSON. **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--transform`. **Required:** False. **Default:** fft. **Choices:** (none). **Action:** (none). **Description:** Transform backend (default: fft). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--win-length`. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** Window length (default: 2048). **Source:** `src/pvx/cli/pvxanalysis.py`.

Flag: `--window`. **Required:** False. **Default:** hann. **Choices:** (none). **Action:** (none). **Description:** Window type. **Source:** `src/pvx/cli/pvxanalysis.py`.

A.5 pvxcodec

pvxcodec accepts the options below; defaults and parser sources are included for reference.

Flag: `--bit-depth`. **Required:** False. **Default:** 24. **Choices:** 16, 24, 32, float. **Action:** (none). **Description:** Output file bit depth (default: 24). **Source:** `src/pvx/cli/pvxcodec.py`.

Flag: `--bits`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Bit depth for raw bit-crushing (4-16). Applied after codec if both specified. **Source:** `src/pvx/cli/pvxcodec.py`.

Flag: `--codec`. **Required:** False. **Default:** (none). **Choices:** `mp3_low`, `mp3_medium`, `telephone`, `voip_narrow`, `voip_wide`, `am_radio`, `lo-fi`, `random`. **Action:** (none). **Description:** Codec preset to simulate (default: none, use `--bits` for raw bit-crush). **Source:** `src/pvx/cli/pvxcodec.py`.

Flag: `--output`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio file. **Source:** `src/pvx/cli/pvxcodec.py`.

Flag: `--seed`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Random seed (default: 0). **Source:** `src/pvx/cli/pvxcodec.py`.

A.6 pvxconform

`pvxconform` accepts the options below; defaults and parser sources are included for reference.

Flag: `--crossfade-ms`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none). **Description:** Segment crossfade in milliseconds. **Source:** `src/pvx/cli/pvxconform.py`.

Flag: `--map`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** CSV map path. **Source:** `src/pvx/cli/pvxconform.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `fft`, `linear`. **Action:** (none). **Source:** `src/pvx/cli/pvxconform.py`.

A.7 pvxdenoise

`pvxdenoise` accepts the options below; defaults and parser sources are included for reference.

Flag: `--floor`. **Required:** False. **Default:** 0.1. **Choices:** (none). **Action:** (none). **Description:** Noise floor multiplier. **Source:** `src/pvx/cli/pvxdenoise.py`.

Flag: `--noise-file`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional external noise reference. **Source:** `src/pvx/cli/pvxdenoise.py`.

Flag: `--noise-seconds`. **Required:** False. **Default:** 0.35. **Choices:** (none). **Action:** (none). **Description:** Noise profile duration from start. **Source:** `src/pvx/cli/pvxdenoise.py`.

Flag: `--reduction-db`. **Required:** False. **Default:** 12.0. **Choices:** (none). **Action:** (none). **Description:** Reduction strength in dB. **Source:** `src/pvx/cli/pvxdenoise.py`.

Flag: `--smooth`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Temporal smoothing frames. **Source:** `src/pvx/cli/pvxdenoise.py`.

A.8 pvxdeverb

`pvxdeverb` accepts the options below; defaults and parser sources are included for reference.

Flag: `--decay`. **Required:** False. **Default:** 0.92. **Choices:** (none). **Action:** (none). **Description:** Tail memory decay 0..1. **Source:** `src/pvx/cli/pvxdeverb.py`.

Flag: `--floor`. **Required:** False. **Default:** 0.12. **Choices:** (none). **Action:** (none). **Description:** Per-bin floor multiplier. **Source:** `src/pvx/cli/pvxdeverb.py`.

Flag: `--strength`. **Required:** False. **Default:** 0.45. **Choices:** (none). **Action:** (none). **Description:** Tail suppression strength 0..1. **Source:** `src/pvx/cli/pvxdeverb.py`.

A.9 pvxenvelope

`pvxenvelope` accepts the options below; defaults and parser sources are included for reference.

Flag: `--amplitude`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Peak/depth/amplitude value (mode-dependent; periodic modes use this as waveform amplitude). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--attack-sec`. **Required:** False. **Default:** 0.1. **Choices:** (none). **Action:** (none). **Description:** ADSR attack in seconds. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--center`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Start/center value (mode-dependent; for periodic modes this is the center/DC offset). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--cycles`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Waveform cycles across full duration (periodic modes). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--decay-sec`. **Required:** False. **Default:** 0.2. **Choices:** (none). **Action:** (none). **Description:** ADSR decay in seconds. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--duration`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Envelope duration in seconds. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--duty-cycle`. **Required:** False. **Default:** 0.5. **Choices:** (none). **Action:** (none). **Description:** Duty cycle in (0, 1) for square mode (default: 0.5). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--end`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** End value for ADSR/ramp/exp modes. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--exp-curve`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none). **Description:** Exponential curve steepness for exp mode. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--format`. **Required:** False. **Default:** auto. **Choices:** csv, json, auto. **Action:** (none). **Description:** Output format (default: auto from extension or csv for stdout). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--freq`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Waveform frequency in Hz for periodic modes (converted to cycles = frequency * duration). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--frequency-hz`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Waveform frequency in Hz for periodic modes (converted to cycles = frequency * duration). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--key`. **Required:** False. **Default:** value. **Choices:** (none). **Action:** (none). **Description:** Output control column/key name (default: value). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--max`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional value clamp maximum. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--min`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional value clamp minimum. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--mode`. **Required:** False. **Default:** adsr. **Choices:** (none). **Action:** (none). **Description:** Envelope/LFO waveform mode. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--out`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output path. Default: stdout. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output path. Default: stdout. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--peak`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Peak/depth/amplitude value (mode-dependent; periodic modes use this as waveform amplitude). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--phase`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Initial waveform phase in radians for periodic modes. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--phase-rad`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Initial waveform phase in radians for periodic modes. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--rate`. **Required:** False. **Default:** 20.0. **Choices:** (none). **Action:** (none). **Description:** Control points per second (default: 20). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--release-sec`. **Required:** False. **Default:** 0.2. **Choices:** (none). **Action:** (none). **Description:** ADSR release in seconds. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--sine-cycles`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Waveform cycles across full duration (periodic modes). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--sine-phase-rad`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Initial waveform phase in radians for periodic modes. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--start`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Start/center value (mode-dependent; for periodic modes this is the center/DC offset). **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--stdout`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Write map to stdout. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--sustain`. **Required:** False. **Default:** 0.7. **Choices:** (none). **Action:** (none). **Description:** Sustain value for ADSR mode. **Source:** `src/pvx/cli/pvxenvelope.py`.

Flag: `--wave`. **Required:** False. **Default:** `adsr`. **Choices:** (none). **Action:** (none). **Description:** Envelope/LFO waveform mode. **Source:** `src/pvx/cli/pvxenvelope.py`.

A.10 pvxfilter

`pvxfilter` accepts the options below; defaults and parser sources are included for reference.

Flag: `--band-gain-db`. **Required:** False. **Default:** 6.0. **Choices:** (none). **Action:** (none). **Description:** Band boost amount for bandamp (dB). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--band-width-bins`. **Required:** False. **Default:** 6. **Choices:** (none). **Action:** (none). **Description:** Band width in bins for bandamp. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--comp-ratio`. **Required:** False. **Default:** 2.0. **Choices:** (none). **Action:** (none). **Description:** Compression ratio for spec-compander. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--comp-threshold-db`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Relative threshold for spec-compander. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--dry-mix`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Dry signal mix amount in [0, 1] (default: 0.0). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--expand-ratio`. **Required:** False. **Default:** 1.2. **Choices:** (none). **Action:** (none). **Description:** Expansion ratio for spec-compander. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--noise-floor`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Threshold scale for noisefilter. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--operator`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--peak-count`. **Required:** False. **Default:** 8. **Choices:** (none). **Action:** (none). **Description:** Number of response peaks for bandamp. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--response`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Input PVXRF response artifact. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--response-gain-db`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Global response gain in dB (default: 0). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--response-mix`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Wet response mix amount (default: 1.0). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--shift-bins`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Shift response curve by FFT bins. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--transpose-semitones`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Transpose response curve in semitones. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--tv-interp`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** TV interpolation mode. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--tv-key`. **Required:** False. **Default:** response_mix. **Choices:** (none). **Action:** (none). **Description:** Column/key in `--tv-map` (default: response_mix). **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--tv-map`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional CSV/JSON map for time-varying response_mix. **Source:** `src/pvx/cli/pvxfilter.py`.

Flag: `--tv-order`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Polynomial order for `--tv-interp` polynomial. **Source:** `src/pvx/cli/pvxfilter.py`.

A.11 pvxformant

pvxformant accepts the options below; defaults and parser sources are included for reference.

Flag: `--formant-lifter`. **Required:** False. **Default:** 32. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--formant-max-gain-db`. **Required:** False. **Default:** 12.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--formant-shift-ratio`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Formant ratio (>1 up, <1 down). **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--mode`. **Required:** False. **Default:** shift. **Choices:** shift, preserve. **Action:** (none). **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--pitch-shift-cents`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Additional microtonal pitch shift in cents before formant stage. **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--pitch-shift-semitones`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Optional pitch shift before formant stage. **Source:** `src/pvx/cli/pvxformant.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** auto. **Choices:** auto, fft, linear. **Action:** (none). **Source:** `src/pvx/cli/pvxformant.py`.

A.12 pvxfreeze

pvxfreeze accepts the options below; defaults and parser sources are included for reference.

Flag: `--duration`. **Required:** False. **Default:** 3.0. **Choices:** (none). **Action:** (none). **Description:** Output freeze duration in seconds. **Source:** `src/pvx/cli/pvxfreeze.py`.

Flag: `--freeze-time`. **Required:** False. **Default:** 0.2. **Choices:** (none). **Action:** (none). **Description:** Freeze anchor time in seconds. **Source:** `src/pvx/cli/pvxfreeze.py`.

Flag: `--phase-mode`. **Required:** False. **Default:** instantaneous. **Choices:** instantaneous, bin, hold. **Action:** (none). **Description:** Phase progression mode inside the frozen segment: instantaneous (default, least flutter), bin (bin-center), hold (no advance). **Source:** `src/pvx/cli/pvxfreeze.py`.

Flag: `--random-phase`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Add subtle phase randomization per frame. **Source:** `src/pvx/cli/pvxfreeze.py`.

A.13 pvxgain

pvxgain accepts the options below; defaults and parser sources are included for reference.

Flag: `--bit-depth`. **Required:** False. **Default:** 24. **Choices:** 16, 24, 32, float. **Action:** (none). **Description:** Output bit depth (default: 24). **Source:** `src/pvx/cli/pvxgain.py`.

Flag: `--gain`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Gain in dB. Single value (fixed) or 'min, max' (random range). Mutually exclusive with `--normalize`. **Source:** `src/pvx/cli/pvxgain.py`.

Flag: `--normalize`. **Required:** False. **Default:** (none). **Choices:** `peak`, `rms`. **Action:** (none). **Description:** Normalization mode. Mutually exclusive with `--gain`. **Source:** `src/pvx/cli/pvxgain.py`.

Flag: `--output`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio file. **Source:** `src/pvx/cli/pvxgain.py`.

Flag: `--seed`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Random seed (default: 0). **Source:** `src/pvx/cli/pvxgain.py`.

Flag: `--target`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Target level in dBFS for normalization (default: -1.0). **Source:** `src/pvx/cli/pvxgain.py`.

A.14 pvxharmmap

`pvxharmmap` accepts the options below; defaults and parser sources are included for reference.

Flag: `--attenuation`. **Required:** False. **Default:** 0.45. **Choices:** (none). **Action:** (none). **Description:** Chordmapper out-of-chord attenuation in [0, 1]. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--boost-db`. **Required:** False. **Default:** 6.0. **Choices:** (none). **Action:** (none). **Description:** Chordmapper in-chord boost in dB. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--chord`. **Required:** False. **Default:** `major`. **Choices:** (none). **Action:** (none). **Description:** Chord quality for chordmapper. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--dry-mix`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Dry signal mix in [0, 1]. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--inharmonic-f0-hz`. **Required:** False. **Default:** 220.0. **Choices:** (none). **Action:** (none). **Description:** Reference f0 for inharmonator. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--inharmonic-mix`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Inharmonator mix in [0, 1]. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--inharmonicity`. **Required:** False. **Default:** 0.0001. **Choices:** (none). **Action:** (none). **Description:** Inharmonic coefficient B. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--operator`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--root-hz`. **Required:** False. **Default:** 220.0. **Choices:** (none). **Action:** (none). **Description:** Chord root frequency in Hz. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--strength`. **Required:** False. **Default:** 0.75. **Choices:** (none). **Action:** (none). **Description:** Chordmapper emphasis strength in [0, 1]. **Source:** `src/pvx/cli/pvxharmmap.py`.

Flag: `--tolerance-cents`. **Required:** False. **Default:** 35.0. **Choices:** (none). **Action:** (none). **Description:** Chordmapper tolerance in cents. **Source:** `src/pvx/cli/pvxharmmap.py`.

A.15 pvxharmonize

`pvxharmonize` accepts the options below; defaults and parser sources are included for reference.

Flag: `--force-stereo`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Mix result as stereo with panning. **Source:** `src/pvx/cli/pvxharmonize.py`.

Flag: `--gains`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional comma-separated linear gain per voice. **Source:** `src/pvx/cli/pvxharmonize.py`.

Flag: `--intervals`. **Required:** False. **Default:** 0,4,7. **Choices:** (none). **Action:** (none). **Description:** Comma-separated semitone intervals per voice (supports fractional values). **Source:** `src/pvx/cli/pvxharmonize.py`.

Flag: `--intervals-cents`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional cents offsets per voice, added to `-intervals` (e.g. 0, 14, -12). **Source:** `src/pvx/cli/pvxharmonize.py`.

Flag: `--pans`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional comma-separated pan per voice [-1..1]. **Source:** `src/pvx/cli/pvxharmonize.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** auto. **Choices:** auto, fft, linear. **Action:** (none). **Source:** `src/pvx/cli/pvxharmonize.py`.

A.16 pvxlayer

pvxlayer accepts the options below; defaults and parser sources are included for reference.

Flag: `--harmonic-gain`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--harmonic-kernel`. **Required:** False. **Default:** 31. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--harmonic-pitch-cents`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--harmonic-pitch-semitones`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--harmonic-stretch`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--percussive-gain`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--percussive-kernel`. **Required:** False. **Default:** 31. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--percussive-pitch-cents`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--percussive-pitch-semitones`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--percussive-stretch`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** auto. **Choices:** auto, fft, linear. **Action:** (none). **Source:** `src/pvx/cli/pvxlayer.py`.

A.17 pvxmorph

pvxmorph accepts the options below; defaults and parser sources are included for reference.

Flag: `--alpha`. **Required:** False. **Default:** 0.5. **Choices:** (none). **Action:** (none). **Description:** Morph amount 0..1 (0=A, 1=B). Accepts scalar or control file (.csv/.json) for time-varying A->B trajectory morphing. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--blend-mode`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** Cross-synthesis blend style. linear/geometric are symmetric blends; carrier_* modes transfer envelope/mask from modulator to carrier. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--envelope-lifter`. **Required:** False. **Default:** 32. **Choices:** (none). **Action:** (none). **Description:** Cepstral lifter cutoff for carrier_envelope modes (default: 32). **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--interp`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** Interpolation mode for `-alpha/-phase-mix` control files (default: linear). **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--mask-exponent`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Exponent used by `carrier_mask` modes (default: 1.0). **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--normalize-energy`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Normalize each output channel RMS toward alpha-blended input RMS. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--order`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Polynomial order for `-interp` polynomial (default: 3). Accepts any integer ≥ 1 ; effective fit degree is $\min(\text{order}, \text{control_points}-1)$. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output file path. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--output-format`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output extension/format; for `-stdout` defaults to `wav`. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--overwrite`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--phase-mix`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Phase blend in $[0, 1]$. If omitted, mode-specific defaults apply (A-phase for `phase_a/carrier_a`, B-phase for `phase_b/carrier_b`, alpha for symmetric modes). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/cli/pvxmorph.py`.

Flag: `--stdout`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Write processed audio to stdout stream (for piping); equivalent to `-o -`. **Source:** `src/pvx/cli/pvxmorph.py`.

A.18 pvxnoise

`pvxnoise` accepts the options below; defaults and parser sources are included for reference.

Flag: `--background-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Directory of background audio files to mix instead of synthetic noise. **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--band`. **Required:** False. **Default:** 300,4000. **Choices:** (none). **Action:** (none). **Description:** Low, high frequency band in Hz for bandlimited noise (default: 300, 4000). **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--bit-depth`. **Required:** False. **Default:** 24. **Choices:** 16, 24, 32, float. **Action:** (none). **Description:** Output bit depth (default: 24). **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--noise-type`. **Required:** False. **Default:** white. **Choices:** white, pink, brown, gaussian, bandlimited. **Action:** (none). **Description:** Noise type (default: white). **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--output`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio file. **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--seed`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Random seed (default: 0). **Source:** `src/pvx/cli/pvxnoise.py`.

Flag: `--snr`. **Required:** False. **Default:** 20. **Choices:** (none). **Action:** (none). **Description:** Signal-to-noise ratio in dB. Single value for fixed SNR or 'min, max' for uniform sampling. (default: 20). **Source:** `src/pvx/cli/pvxnoise.py`.

A.19 pvxreshape

`pvxreshape` accepts the options below; defaults and parser sources are included for reference.

Flag: `--exponent`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Exponent for pow operation. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--factor`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Scale factor for scale/time-scale ops. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--format`. **Required:** False. **Default:** auto. **Choices:** auto, csv, json. **Action:** (none). **Description:** Output format (default: auto from `-output` suffix or csv for stdout). **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--input-format`. **Required:** False. **Default:** auto. **Choices:** auto, csv, json. **Action:** (none). **Description:** Input format override (default: auto by suffix or csv for stdin). **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--interp`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** Interpolation mode. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--key`. **Required:** False. **Default:** value. **Choices:** (none). **Action:** (none). **Description:** Control column/key to read (default: value). **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--max`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Maximum clamp value. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--min`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Minimum clamp value. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--offset`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Offset for offset/time-shift ops. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--operation`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Reshape operation. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--order`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Polynomial order for `-interp` polynomial. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--out`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output file path. Default: stdout. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output file path. Default: stdout. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--output-key`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output control key. Default: same as `-key`. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--rate`. **Required:** False. **Default:** 20.0. **Choices:** (none). **Action:** (none). **Description:** Resample control rate (Hz) for resample operation. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--stdout`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Write reshaped map to stdout. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--target-max`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Target maximum for normalize operation. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--target-min`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Target minimum for normalize operation. **Source:** `src/pvx/cli/pvxreshape.py`.

Flag: `--window`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Window size for smooth operation. **Source:** `src/pvx/cli/pvxreshape.py`.

A.20 pvxresponse

`pvxresponse` accepts the options below; defaults and parser sources are included for reference.

Flag: `--json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Print summary as JSON. **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--method`. **Required:** False. **Default:** median. **Choices:** (none). **Action:** (none). **Description:** Magnitude aggregation across frames (default: median). **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--normalize`. **Required:** False. **Default:** `peak`. **Choices:** (none). **Action:** (none). **Description:** Magnitude normalization mode (default: `peak`). **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output PVXRF path (default: `.pvxf.npz`). **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--phase-mode`. **Required:** False. **Default:** `mean`. **Choices:** (none). **Action:** (none). **Description:** Phase aggregation strategy (default: `mean`). **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--smoothing-bins`. **Required:** False. **Default:** `1`. **Choices:** (none). **Action:** (none). **Description:** Moving-average smoothing width in bins (default: `1`). **Source:** `src/pvx/cli/pvxresponse.py`.

Flag: `--summary-json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional path to write summary JSON. **Source:** `src/pvx/cli/pvxresponse.py`.

A.21 pvxretune

`pvxretune` accepts the options below; defaults and parser sources are included for reference.

Flag: `--a4-reference-hz`. **Required:** False. **Default:** `440.0`. **Choices:** (none). **Action:** (none). **Description:** Concert-pitch reference for A4 in Hz (default: `440.0`). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--chunk-ms`. **Required:** False. **Default:** `80.0`. **Choices:** (none). **Action:** (none). **Description:** Analysis/process chunk duration in ms. **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--f0-max`. **Required:** False. **Default:** `1200.0`. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--f0-min`. **Required:** False. **Default:** `60.0`. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--overlap-ms`. **Required:** False. **Default:** `20.0`. **Choices:** (none). **Action:** (none). **Description:** Chunk overlap in ms. **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--recommend-root`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Estimate and use a per-file root fundamental from tracked F0. **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `fft`, `linear`. **Action:** (none). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--root`. **Required:** False. **Default:** `C`. **Choices:** (none). **Action:** (none). **Description:** Scale root note (C, C#, D, ..., B). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--root-hz`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional root fundamental in Hz (overrides `--root` when provided). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--scale`. **Required:** False. **Default:** `chromatic`. **Choices:** (none). **Action:** (none). **Description:** Named scale for 12-TET quantization. **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--scale-cents`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional comma-separated microtonal scale degrees in cents within one octave, relative to `--root` (example: `0, 90, 204, 294, 408, 498, 612, 702, 816, 906, 1020, 1110`). **Source:** `src/pvx/cli/pvxretune.py`.

Flag: `--strength`. **Required:** False. **Default:** `0.85`. **Choices:** (none). **Action:** (none). **Description:** Correction strength 0..1. **Source:** `src/pvx/cli/pvxretune.py`.

A.22 pvxring

`pvxring` accepts the options below; defaults and parser sources are included for reference.

Flag: `--depth`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Ring modulation depth in [0, 1]. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--feedback`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Feedback amount in [0, 1). **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--frequency-hz`. **Required:** False. **Default:** 40.0. **Choices:** (none). **Action:** (none). **Description:** Carrier frequency in Hz. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--mix`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Ring wet mix in [0, 1]. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--operator`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--resonance-decay`. **Required:** False. **Default:** 0.2. **Choices:** (none). **Action:** (none). **Description:** Resonator feedback/decay in [0, 1). **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--resonance-hz`. **Required:** False. **Default:** 1200.0. **Choices:** (none). **Action:** (none). **Description:** Resonance center frequency (Hz). **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--resonance-mix`. **Required:** False. **Default:** 0.35. **Choices:** (none). **Action:** (none). **Description:** Resonator wet mix in [0, 1]. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--resonance-q`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none). **Description:** Resonance quality factor Q. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--tv-interp`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** TV interpolation mode. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--tv-map`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** CSV/JSON time-varying map for ringtvfilter. **Source:** `src/pvx/cli/pvxring.py`.

Flag: `--tv-order`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Polynomial order for `--tv-interp` polynomial. **Source:** `src/pvx/cli/pvxring.py`.

A.23 pvxrir

pvxrir accepts the options below; defaults and parser sources are included for reference.

Flag: `--bit-depth`. **Required:** False. **Default:** 24. **Choices:** 16, 24, 32, float. **Action:** (none). **Description:** Output bit depth (default: 24). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--cache-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Cache directory for downloaded IR databases (default: `~/.pvx/ir_cache`). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--category`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Filter IRs by category when using `--ir-database` (e.g., hall, church, outdoor, room, large). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--drr`. **Required:** False. **Default:** 3,12. **Choices:** (none). **Action:** (none). **Description:** Direct-to-reverb ratio in dB. Single value or 'min, max' range (default: 3, 12). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--ir-database`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Use IRs from a curated database (auto-downloads on first use). Available: `echothief`, `mit_kemar`. **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--ir-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Directory of impulse response files (one is chosen randomly). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--ir-file`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Path to an impulse response WAV file (overrides synthetic generation). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--list-databases`. **Required:** False. **Default:** False. **Choices:** (none). **Action:** `store_true`. **Description:** List available IR databases and exit. **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--no-trim`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Do not trim output to input length (include reverb tail). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--output`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio file. **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--pre-delay-ms`. **Required:** False. **Default:** 5.0. **Choices:** (none). **Action:** (none). **Description:** Pre-delay in ms before the reverberant tail (default: 5.0). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--rt60`. **Required:** False. **Default:** 0.2,1.5. **Choices:** (none). **Action:** (none). **Description:** Reverberation time in seconds. Single value or 'min, max' range (default: 0.2, 1.5). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--seed`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Random seed (default: 0). **Source:** `src/pvx/cli/pvxrir.py`.

Flag: `--wet`. **Required:** False. **Default:** 0.3,0.7. **Choices:** (none). **Action:** (none). **Description:** Wet/dry mix 0-1. Single value or 'min, max' range (default: 0.3, 0.7). **Source:** `src/pvx/cli/pvxrir.py`.

A.24 pvxspecaugment

`pvxspecaugment` accepts the options below; defaults and parser sources are included for reference.

Flag: `--bit-depth`. **Required:** False. **Default:** 24. **Choices:** 16, 24, 32, float. **Action:** (none). **Description:** Output bit depth (default: 24). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--fill`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Fill value for masked regions: 0, mean, or a float (default: 0). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--freq-mask`. **Required:** False. **Default:** 27. **Choices:** (none). **Action:** (none). **Description:** Maximum frequency mask width F in bins (default: 27). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--hop-length`. **Required:** False. **Default:** 128. **Choices:** (none). **Action:** (none). **Description:** STFT hop length in samples (default: 128). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--n-fft`. **Required:** False. **Default:** 512. **Choices:** (none). **Action:** (none). **Description:** FFT size (default: 512). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--num-freq-masks`. **Required:** False. **Default:** 2. **Choices:** (none). **Action:** (none). **Description:** Number of frequency masks (default: 2). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--num-time-masks`. **Required:** False. **Default:** 2. **Choices:** (none). **Action:** (none). **Description:** Number of time masks (default: 2). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--output`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output audio file. **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--seed`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Random seed (default: 0). **Source:** `src/pvx/cli/pvxspecaugment.py`.

Flag: `--time-mask`. **Required:** False. **Default:** 100. **Choices:** (none). **Action:** (none). **Description:** Maximum time mask width T in frames (default: 100). **Source:** `src/pvx/cli/pvxspecaugment.py`.

A.25 pvxtrajectoryreverb

`pvxtrajectoryreverb` accepts the options below; defaults and parser sources are included for reference.

Flag: `--coord-system`. **Required:** False. **Default:** cartesian. **Choices:** cartesian, spherical. **Action:** (none). **Description:** Coordinate format for `--start/--end` (default: cartesian). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--distance-law`. **Required:** False. **Default:** inverse. **Choices:** none, inverse, inverse-square. **Action:** (none). **Description:** Distance gain law applied across trajectory (default: inverse). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--dry`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Dry direct mix scalar (default: 0.0). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--end`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** End position. Cartesian: x, y, z. Spherical: az_deg, el_deg, r. **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--ir`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Multichannel impulse response path (for example 4-channel room capture). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--no-normalize-gains`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_false`. **Description:** Disable per-sample gain normalization. **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--normalize-gains`. **Required:** False. **Default:** True. **Choices:** (none). **Action:** `store_true`. **Description:** Normalize per-sample channel gains before distance weighting (default: on). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--speaker-angles`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional speaker layout azimuth/elevation list in degrees as “az, el;az, el;...”. Count must match impulse-response channels. **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--start`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Start position. Cartesian: x, y, z. Spherical: az_deg, el_deg, r. **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--trajectory-shape`. **Required:** False. **Default:** linear. **Choices:** linear, ease-in, ease-out, ease-in-out. **Action:** (none). **Description:** Interpolation shape from start to end (default: linear). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

Flag: `--wet`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Wet reverb mix scalar (default: 1.0). **Source:** `src/pvx/cli/pvxtrajectoryreverb.py`.

A.26 pvxtransient

`pvxtransient` accepts the options below; defaults and parser sources are included for reference.

Flag: `--pitch-shift-cents`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional microtonal pitch shift in cents (added to `--pitch-shift-semitones`). **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--pitch-shift-ratio`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch ratio override. Accepts decimals (1.5), integer ratios (3/2), and expressions ($2^{(1/12)}$). **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--pitch-shift-semitones`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** auto. **Choices:** auto, fft, linear. **Action:** (none). **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--target-duration`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Target duration in seconds. **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--time-stretch`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxtransient.py`.

Flag: `--transient-threshold`. **Required:** False. **Default:** 1.6. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxtransient.py`.

A.27 pvxunison

`pvxunison` accepts the options below; defaults and parser sources are included for reference.

Flag: `--detune-cents`. **Required:** False. **Default:** 14.0. **Choices:** (none). **Action:** (none). **Description:** Total detune span in cents. **Source:** `src/pvx/cli/pvxunison.py`.

Flag: `--dry-mix`. **Required:** False. **Default:** 0.2. **Choices:** (none). **Action:** (none). **Description:** Dry signal mix amount. **Source:** `src/pvx/cli/pvxunison.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** auto. **Choices:** auto, fft, linear. **Action:** (none). **Source:** `src/pvx/cli/pvxunison.py`.

Flag: `--voices`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Number of unison voices. **Source:** `src/pvx/cli/pvxunison.py`.

Flag: `--width`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Stereo width multiplier 0..2. **Source:** `src/pvx/cli/pvxunison.py`.

A.28 pvxvoc

pvxvoc accepts the options below; defaults and parser sources are included for reference.

Flag: `--ambient-phase-mix`. **Required:** False. **Default:** 0.5. **Choices:** (none). **Action:** (none). **Description:** Random-phase blend when `--phase-engine` hybrid (0.0=propagated only, 1.0=random only; default: 0.5). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--ambient-preset`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Convenience preset for ambient extreme stretch (random phase engine, onset-time-credit, transient preserve, conservative staging). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--analysis-channel`. **Required:** False. **Default:** mix. **Choices:** first, mix. **Action:** (none). **Description:** Channel strategy for F0 estimation with `--target-f0` (default: mix). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--auto-profile`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Analyze input and choose a profile automatically (speech/music/percussion/ambient/extreme). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--auto-profile-lookahead-seconds`. **Required:** False. **Default:** 6.0. **Choices:** (none). **Action:** (none). **Description:** Seconds of audio used when estimating `--auto-profile` (default: 6.0). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--auto-segment-seconds`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Optional segment size in seconds for long jobs. When >0, processing runs per segment with crossfade assembly. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--auto-transform`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Allow automatic transform selection when `--transform` is not explicitly set. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--bit-depth`. **Required:** False. **Default:** inherit. **Choices:** (none). **Action:** (none). **Description:** Output bit-depth policy (default: inherit). Ignored when `--subtype` is set. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--cents`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in cents (+1200 is one octave up). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--checkpoint-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Directory used to cache per-segment checkpoint chunks for resume workflows. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--checkpoint-id`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional checkpoint run identifier (default: hash of input/settings). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--clip`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** store_true. **Description:** Legacy alias: hard clip at +/-1.0 when set. **Source:** `src/pvx/core/voc.py`.

Flag: `--coherence-strength`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none).
Description: Coherence lock strength in [0, 1] (0=off, 1=full lock). Accepts scalar or control file (.csv/.json).
Source: `src/pvx/core/voc_parser.py`.

Flag: `--comander-attack-ms`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none).
Description: Comander attack time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--comander-compress-ratio`. **Required:** False. **Default:** 3.0. **Choices:** (none). **Action:** (none).
Description: Comander compression ratio (≥ 1). **Source:** `src/pvx/core/voc.py`.

Flag: `--comander-expand-ratio`. **Required:** False. **Default:** 1.8. **Choices:** (none). **Action:** (none).
Description: Comander expansion ratio (≥ 1). **Source:** `src/pvx/core/voc.py`.

Flag: `--comander-makeup-db`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none).
Description: Comander makeup gain in dB. **Source:** `src/pvx/core/voc.py`.

Flag: `--comander-release-ms`. **Required:** False. **Default:** 120.0. **Choices:** (none). **Action:** (none).
Description: Comander release time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--comander-threshold-db`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none).
Description: Enable comander threshold in dBFS. **Source:** `src/pvx/core/voc.py`.

Flag: `--compressor-attack-ms`. **Required:** False. **Default:** 10.0. **Choices:** (none). **Action:** (none).
Description: Compressor attack time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--compressor-makeup-db`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none).
Description: Compressor makeup gain in dB. **Source:** `src/pvx/core/voc.py`.

Flag: `--compressor-ratio`. **Required:** False. **Default:** 4.0. **Choices:** (none). **Action:** (none).
Description: Compressor ratio (≥ 1). **Source:** `src/pvx/core/voc.py`.

Flag: `--compressor-release-ms`. **Required:** False. **Default:** 120.0. **Choices:** (none). **Action:** (none).
Description: Compressor release time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--compressor-threshold-db`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none).
Description: Enable compressor above threshold dBFS. **Source:** `src/pvx/core/voc.py`.

Flag: `--control-stdin`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`.
Description: Alias for `--pitch-map-stdin` (canonical control-bus CSV stdin path). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--cpu`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`.
Description: Alias for `--device cpu`. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--cuda-device`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none).
Description: CUDA device index used when `--device` is `auto/cuda` (default: 0). **Source:** `src/pvx/core/voc.py`.

Flag: `--device`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `cpu`, `cuda`. **Action:** (none).
Description: Compute device: `auto` (prefer CUDA), `cpu`, or `cuda`. **Source:** `src/pvx/core/voc.py`.

Flag: `--dither`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none).
Description: Dither policy before quantized writes (default: none). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--dither-seed`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none).
Description: Deterministic RNG seed for dithering (default: random seed). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--dry-run`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`.
Description: Resolve settings without writing files. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--example`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none).
Description: Print copy-paste example command(s) and exit. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--expander-attack-ms`. **Required:** False. **Default:** 5.0. **Choices:** (none). **Action:** (none).
Description: Expander attack time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--expander-ratio`. **Required:** False. **Default:** 2.0. **Choices:** (none). **Action:** (none).
Description: Expander ratio (≥ 1). **Source:** `src/pvx/core/voc.py`.

Flag: `--expander-release-ms`. **Required:** False. **Default:** 120.0. **Choices:** (none). **Action:** (none). **Description:** Expander release time in ms. **Source:** `src/pvx/core/voc.py`.

Flag: `--expander-threshold-db`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Enable downward expander below threshold dBFS. **Source:** `src/pvx/core/voc.py`.

Flag: `--explain-plan`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Print resolved processing plan JSON and exit without rendering audio. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--extreme-stretch-threshold`. **Required:** False. **Default:** 2.0. **Choices:** (none). **Action:** (none). **Description:** Auto-mode threshold for multistage activation (default: 2.0). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--extreme-time-stretch`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Force multistage strategy even when ratio is moderate. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--f0-max`. **Required:** False. **Default:** 1000.0. **Choices:** (none). **Action:** (none). **Description:** Maximum F0 search bound in Hz (default: 1000). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--f0-min`. **Required:** False. **Default:** 50.0. **Choices:** (none). **Action:** (none). **Description:** Minimum F0 search bound in Hz (default: 50). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--force`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Overwrite existing outputs and other write targets. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--formant-lifter`. **Required:** False. **Default:** 32. **Choices:** (none). **Action:** (none). **Description:** Cepstral lifter cutoff for formant envelope extraction (default: 32). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--formant-max-gain-db`. **Required:** False. **Default:** 12.0. **Choices:** (none). **Action:** (none). **Description:** Max per-bin formant correction gain in dB (default: 12). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--formant-strength`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Formant correction blend 0..1 when pitch mode is formant-preserving (default: 1.0). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--fourier-sync`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Enable fundamental frame locking. Uses generic short-time Fourier transforms with per-frame FFT sizes locked to detected F0. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--fourier-sync-max-fft`. **Required:** False. **Default:** 8192. **Choices:** (none). **Action:** (none). **Description:** Maximum frame FFT size for `--fourier-sync` (default: 8192). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--fourier-sync-min-fft`. **Required:** False. **Default:** 256. **Choices:** (none). **Action:** (none). **Description:** Minimum frame FFT size for `--fourier-sync` (default: 256). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--fourier-sync-smooth`. **Required:** False. **Default:** 5. **Choices:** (none). **Action:** (none). **Description:** Smoothing span (frames) for prescanned F0 track in `--fourier-sync` (default: 5). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--gpu`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Alias for `--device cuda`. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--guided`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Interactive guided mode for first-time users. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--hard-clip-level`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Hard clip level in linear full-scale. **Source:** `src/pvx/core/voc.py`.

Flag: `--hop-size`. **Required:** False. **Default:** 512. **Choices:** (none). **Action:** (none). **Description:** Hop size in samples (default: 512). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--interp`. **Required:** False. **Default:** linear. **Choices:** (none). **Action:** (none). **Description:** Interpolation mode for time-varying control signals loaded from CSV/JSON (default: linear). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--kaiser-beta`. **Required:** False. **Default:** 14.0. **Choices:** (none). **Action:** (none). **Description:** Kaiser window beta parameter used when `--window kaiser` (default: 14.0). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--limiter-threshold`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Peak limiter threshold in linear full-scale. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--manifest-append`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Append entries to an existing `--manifest-json` file instead of replacing it. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--manifest-json`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Write processing manifest JSON with per-file settings and outcomes. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--max-stage-stretch`. **Required:** False. **Default:** 1.8. **Choices:** (none). **Action:** (none). **Description:** Maximum per-stage ratio used in multistage mode (default: 1.8). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--metadata-policy`. **Required:** False. **Default:** none. **Choices:** (none). **Action:** (none). **Description:** Output metadata policy: none, sidecar, or copy (sidecar implementation). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--multires-ffts`. **Required:** False. **Default:** 1024,2048,4096. **Choices:** (none). **Action:** (none). **Description:** Comma-separated FFT sizes for `--multires-fusion` (default: 1024, 2048, 4096). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--multires-fusion`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Blend multiple FFT resolutions for each channel before pitch resampling. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--multires-weights`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Comma-separated fusion weights for `--multires-fusion` (defaults to equal weights). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--n-fft`. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** FFT size (default: 2048). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--no-center`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Disable center padding in STFT/ISTFT. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--no-onset-realign`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Disable fractional read-position realignment on onsets when `--onset-time-credit` is enabled. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--no-progress`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Source:** `src/pvx/core/voc_console.py`.

Flag: `--normalize`. **Required:** False. **Default:** none. **Choices:** none, peak, rms. **Action:** (none). **Description:** Output normalization mode. **Source:** `src/pvx/core/voc.py`.

Flag: `--onset-credit-max`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none). **Description:** Maximum accumulated onset time credit in analysis-frame units (default: 8.0). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--onset-credit-pull`. **Required:** False. **Default:** 0.5. **Choices:** (none). **Action:** (none). **Description:** Fraction of per-frame read advance removable while onset credit exists (0.0..1.0, default: 0.5). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--onset-time-credit`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Enable onset-triggered time-credit scheduling to reduce transient smear during extreme stretching. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--order`. **Required:** False. **Default:** 3. **Choices:** (none). **Action:** (none). **Description:** Polynomial order for `-interp` polynomial (default: 3). Accepts any integer ≥ 1 ; effective fit degree is $\min(\text{order}, \text{control_points}-1)$. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--out`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Explicit output file path (single-input mode only). Alias: `-out`. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--output`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Explicit output file path (single-input mode only). Alias: `-out`. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--output-dir`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Directory for output files (default: same directory as each input). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--output-format`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output format/extension (e.g. wav, flac, aiff). Default: keep input extension. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--overwrite`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Overwrite existing outputs and other write targets. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--peak-dbfs`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Target peak dBFS when `-normalize peak`. **Source:** `src/pvx/core/voc.py`.

Flag: `--phase-engine`. **Required:** False. **Default:** `propagate`. **Choices:** (none). **Action:** (none). **Description:** Phase synthesis engine: `propagate` (classic phase vocoder), `hybrid` (propagated + stochastic blend), `random` (ambient stochastic phase). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--phase-locking`. **Required:** False. **Default:** `identity`. **Choices:** `off`, `identity`. **Action:** (none). **Description:** Inter-bin phase locking mode for transient fidelity (default: `identity`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--phase-random-seed`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Optional deterministic seed for random/hybrid phase generation. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-conf-min`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Minimum accepted map confidence (default: 0 disables gating). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-follow-stdin`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Shortcut for `-pitch-map-stdin` (sidechain pitch-follow workflows). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-lowconf-mode`. **Required:** False. **Default:** `hold`. **Choices:** `hold`, `unity`, `interp`. **Action:** (none). **Description:** Low-confidence map handling mode (default: `hold`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-map`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** CSV control map for time-varying stretch/pitch. Columns: `start_sec`, `end_sec` plus optional `stretch`, `pitch_ratio`/`pitch_cents`/`pitch_semitones`, `confidence`. Use `'-'` to read from stdin. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-map-crossfade-ms`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none). **Description:** Crossfade between processed map segments in milliseconds (default: 8.0). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-map-smooth-ms`. **Required:** False. **Default:** 0.0. **Choices:** (none). **Action:** (none). **Description:** Moving-average smoothing over map pitch ratios in milliseconds. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-map-stdin`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Read control-map CSV from stdin. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-mode`. **Required:** False. **Default:** `standard`. **Choices:** `standard`, `formant-preserving`. **Action:** (none). **Description:** Pitch mode: standard shift or formant-preserving correction (default: standard). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-shift-cents`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in cents (+1200 is one octave up). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-shift-ratio`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch ratio (>1 up, <1 down). Accepts decimals (1.5), integer ratios (3/2), expressions ($2^{(1/12)}$), or a control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--pitch-shift-semitones`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--preset`. **Required:** False. **Default:** `none`. **Choices:** (none). **Action:** (none). **Description:** High-level intent preset. Legacy: `none/vocal/ambient/extreme`. New: `default/vocal_studio/drums_safe/extreme_ambient/stereo_coherent`. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--quality-profile`. **Required:** False. **Default:** `neutral`. **Choices:** (none). **Action:** (none). **Description:** Named tuning profile for vocoder defaults (default: neutral). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--quiet`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Reduce output and hide status bars. **Source:** `src/pvx/core/voc_console.py`.

Flag: `--ratio`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch ratio (>1 up, <1 down). Accepts decimals (1.5), integer ratios (3/2), expressions ($2^{(1/12)}$), or a control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--ref-channel`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** (none). **Description:** Reference channel index used by `-stereo-mode ref_channel_lock` (default: 0). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `fft`, `linear`. **Action:** (none). **Description:** Resampling engine (auto=fft if scipy available, else linear). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--resume`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Reuse existing checkpoint chunks from `-checkpoint-dir` when available. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--rms-dbfs`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Target RMS dBFS when `-normalize rms`. **Source:** `src/pvx/core/voc.py`.

Flag: `--route`. **Required:** False. **Default:** []. **Choices:** (none). **Action:** `append`. **Description:** Control-bus routing expression for map rows. Repeat flag to chain routes. Syntax: `target=source`, `target=const(v)`, `target=inv(source)`, `target=pow(source, exp)`, `target=mul(source, factor)`, `target=add(source, offset)`, `target=affine(source, scale, bias)`, `target=clip(source, lo, hi)`. Targets: `stretch`, `pitch_ratio`. Sources: any numeric column present in the control-map CSV. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--semitones`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (.csv/.json). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--silent`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Suppress all console output. **Source:** `src/pvx/core/voc_console.py`.

Flag: `--soft-clip-drive`. **Required:** False. **Default:** 1.0. **Choices:** (none). **Action:** (none). **Description:** Soft clip drive amount (>0). **Source:** `src/pvx/core/voc.py`.

Flag: `--soft-clip-level`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Soft clip output ceiling in linear full-scale. **Source:** `src/pvx/core/voc.py`.

Flag: `--soft-clip-type`. **Required:** False. **Default:** `tanh`. **Choices:** `tanh`, `arctan`, `cubic`. **Action:** (none). **Description:** Soft clip transfer type. **Source:** `src/pvx/core/voc.py`.

Flag: `--stdout`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Write processed audio to stdout stream (for piping); requires exactly one input. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--stereo-mode`. **Required:** False. **Default:** `independent`. **Choices:** `independent`, `mid_side_lock`, `ref_channel_lock`. **Action:** (none). **Description:** Channel coherence strategy: `independent` (legacy), `mid_side_lock` (M/S-coupled), `ref_channel_lock` (phase-lock to reference channel). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--stretch`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Alias for `-time-stretch`. Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--stretch-mode`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `standard`, `multistage`. **Action:** (none). **Description:** Stretch strategy: `standard` (single pass), `multistage` (chained moderate passes), or `auto` (multistage only for extreme ratios; default: `auto`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--subtype`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Explicit libsndfile output subtype override (e.g., `PCM_16`, `PCM_24`, `FLOAT`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--suffix`. **Required:** False. **Default:** `_pv`. **Choices:** (none). **Action:** (none). **Description:** Suffix appended to output filename stem (default: `_pv`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--target-duration`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Absolute target duration in seconds (overrides `-time-stretch`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--target-f0`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Target fundamental frequency in Hz. Auto-estimates source F0 per file. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--target-lufs`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Integrated loudness target in LUFS. **Source:** `src/pvx/core/voc.py`.

Flag: `--target-pitch-shift-semitones`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Pitch shift in semitones (+12 is one octave up). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--target-sample-rate`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Output sample rate in Hz (default: keep input rate). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--time-stretch`. **Required:** False. **Default:** `1.0`. **Choices:** (none). **Action:** (none). **Description:** Final duration multiplier (1.0=unchanged, 2.0=2x longer). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--time-stretch-factor`. **Required:** False. **Default:** `1.0`. **Choices:** (none). **Action:** (none). **Description:** Final duration multiplier (1.0=unchanged, 2.0=2x longer). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transform`. **Required:** False. **Default:** `fft`. **Choices:** (none). **Action:** (none). **Description:** Per-frame transform backend for STFT/ISTFT paths (default: `fft`; options: `fft`, `dft`, `czt`, `dct`, `dst`, `hartley`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-crossfade-ms`. **Required:** False. **Default:** `10.0`. **Choices:** (none). **Action:** (none). **Description:** Crossfade duration for transient/steady stitching (default: 10 ms). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-mode`. **Required:** False. **Default:** `off`. **Choices:** `off`, `reset`, `hybrid`, `wsola`. **Action:** (none). **Description:** Transient handling mode: `off` (none), `reset` (phase reset), `hybrid` (PV steady + WSOLA transients), or `wsola` (time-domain transient-safe path). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-preserve`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** `store_true`. **Description:** Enable transient phase resets based on spectral flux. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-protect-ms`. **Required:** False. **Default:** 30.0. **Choices:** (none). **Action:** (none). **Description:** Transient protection width in milliseconds (default: 30). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-sensitivity`. **Required:** False. **Default:** 0.5. **Choices:** (none). **Action:** (none). **Description:** Transient detector sensitivity in [0, 1] (higher catches more onsets). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--transient-threshold`. **Required:** False. **Default:** 2.0. **Choices:** (none). **Action:** (none). **Description:** Spectral-flux multiplier for transient detection (default: 2.0). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--true-peak-max-dbtp`. **Required:** False. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** Apply output gain trim to enforce max true-peak in dBTP. **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--verbose`. **Required:** False. **Default:** 0. **Choices:** (none). **Action:** `count`. **Description:** Increase verbosity (repeat for extra detail). **Source:** `src/pvx/core/voc_console.py`.

Flag: `--verbosity`. **Required:** False. **Default:** `normal`. **Choices:** (none). **Action:** (none). **Description:** Console verbosity level. **Source:** `src/pvx/core/voc_console.py`.

Flag: `--win-length`. **Required:** False. **Default:** 2048. **Choices:** (none). **Action:** (none). **Description:** Window length in samples (default: 2048). Accepts scalar or control file (`.csv/.json`). **Source:** `src/pvx/core/voc_parser.py`.

Flag: `--window`. **Required:** False. **Default:** `hann`. **Choices:** (none). **Action:** (none). **Description:** Window type (default: `hann`). **Source:** `src/pvx/core/voc_parser.py`.

A.29 pvxwarp

`pvxwarp` accepts the options below; defaults and parser sources are included for reference.

Flag: `--crossfade-ms`. **Required:** False. **Default:** 8.0. **Choices:** (none). **Action:** (none). **Source:** `src/pvx/cli/pvxwarp.py`.

Flag: `--map`. **Required:** True. **Default:** (none). **Choices:** (none). **Action:** (none). **Description:** CSV map with `start_sec`, `end_sec`, `stretch`. **Source:** `src/pvx/cli/pvxwarp.py`.

Flag: `--resample-mode`. **Required:** False. **Default:** `auto`. **Choices:** `auto`, `fft`, `linear`. **Action:** (none). **Source:** `src/pvx/cli/pvxwarp.py`.

APPENDIX B

pvx Algorithm Limitations and Applicability

Generated from commit 35e9761 (commit date: 2026-05-25T08:14:42-04:00).

This document summarizes assumptions, likely failure modes, and practical exclusion cases for each algorithm group and algorithm module.

B.1 Group-Level Summary

A tabular view makes the distinctions within the group-level summary easier to inspect.

Group	Assumptions	Failure Modes	When Not To Use
analysis_qa_and_automation	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
creative_spectral_effects	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
denoise_and_restoration	Noise/artifacts are distinguishable from desired signal statistics.	Over-reduction can remove detail and create modulation artifacts.	Avoid high reduction settings on sparse acoustic sources without auditioning.
dereverb_and_room_correction	Late reverberation is separable from direct content under chosen model.	Speech/music clarity can drop if early reflections are over-suppressed.	Avoid strong dereverb when room character is part of artistic intent.
dynamics_and_loudness	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
granular_and_modulation	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
pitch_detection_and_tracking	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
retune_and_intonation	Detected notes map cleanly to intended tonal center/scale.	Over-correction can flatten expressive vibrato or slides.	Avoid aggressive correction when preserving natural micro-intonation is required.

Group	Assumptions	Failure Modes	When Not To Use
<code>separation_and_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>spatial_and_multichannel</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spectral_time_frequency_transforms</code>	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
<code>time_scale_and_pitch_core</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.

B.2 analysis_qa_and_automation

A tabular view makes the distinctions within the `analysisqaandautomation` easier to inspect.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>analysis_qa_and_automation.auto_parameter_tuning_bayesian_optimization</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation.batch_preset_recommendation_based_on_source_features</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation.clip_hum_buzz_artifact_detection</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation.key_chord_detection</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation.onset_beat_downbeat_tracking</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>analysis_qa_and_automation_pesq_stoi_visqol_quality_metrics</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_silence_speech_music_classifiers</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.
<code>analysis_qa_and_automation_structure_segmentation_verse_chorus_sections</code>	Feature extraction settings align with domain (speech vs music etc.).	False positives/negatives under domain shift.	Avoid treating single metrics as absolute quality verdicts.

B.3 creative_spectral_effects

The relevant properties of the creativespectraeffects are compared in the following table.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>creative_spectral_effects_cross_synthesis_vocoder</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_formant_painting_warping</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_phase_randomization_textures</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_resonator_filterbank_morphing</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_spectral_blur_smea</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_spectral_contrast_exaggeration</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.
<code>creative_spectral_effects_spectral_convolution_effects</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>creative_spectral_effects.spectral_freeze_banks</code>	Spectral manipulations are desired even with timbral coloration.	Can introduce intentional but strong coloration or temporal artifacts.	Avoid for transparent restoration/mastering paths.

B.4 denoise_and_restoration

The relevant properties of the `denoiseandrestoration` are compared in the following table.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>denoise_and_restoration.declick_decrackle_median_wavelet_interpolation</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.declip_via_sparse_reconstruction</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.diffusion_based_speech_audio_denoise</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.log_mmse</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.minimum_statistics_noise_tracking</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>denoise_and_restoration.mmse_stsa</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.rnnoise_style_denoiser</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.
<code>denoise_and_restoration.wiener_denoising</code>	Noise/artifacts are distinguishable from desired signal statistics. Noise model should be representative of observed noise floor.	Over-reduction can remove detail and create modulation artifacts. Mismatched noise estimate leaves residue or damages detail.	Avoid high reduction settings on sparse acoustic sources without auditioning. Avoid static settings on rapidly varying nonstationary noise.

B.5 dereverb_and_room_correction

The table below gathers the principal details of the dereverb_and_room_correction.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>dereverb_and_room_correction.blind_deconvolution_dereverb</code>	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
<code>dereverb_and_room_correction.drr_guided_dereverb</code>	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
<code>dereverb_and_room_correction.late_reverb_suppression_via_coherence</code>	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dereverb_and_room_correction.multi_band_adaptive_deverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction.neural_dereverb_module	Late reverberation is separable from direct content under chosen model. Model priors assume training-like signal statistics. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Generalization gaps can produce unstable artifacts. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid fully unattended use on out-of-domain material. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction.room_impulse_inverse_filtering	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction.spectral_decay_subtraction	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.
dereverb_and_room_correction.wpe_dereverb	Late reverberation is separable from direct content under chosen model. Reverberation tail is assumed more diffuse than direct content.	Speech/music clarity can drop if early reflections are over-suppressed. Over-suppression can thin tonal body and ambience.	Avoid strong dereverb when room character is part of artistic intent. Avoid for intentionally wet effects unless mix preservation is planned.

B.6 dynamics_and_loudness

The table below gathers the principal details of the dynamicsandloudness.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
dynamics_and_loudness.ebu_r128_normalization	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>dynamics_and_loudness.itu_bs_1770_loudness_measurement_gating</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.lufs_target_mastering_chain</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.multi_band_compression</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.spectral_dynamics_bin_wise_compressor_expander</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.transient_shaping</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.true_peak_limiting</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.
<code>dynamics_and_loudness.upward_compression</code>	Program dynamics fit compressor/limiter time constants and thresholds.	Pumping, breathing, or overs if thresholds and release are mis-set.	Avoid applying multiple strong dynamics stages without gain staging checks.

B.7 granular_and_modulation

The relevant properties of the `granularandmodulation` are compared in the following table.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>granular_and_modulation.am_fm_ring_modulation_blocks</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.envelope_followed_modulation_routing</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>granular_and_modulation.formant_lfo_modulation</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.freeze_grain_morphing</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.grain_cloud_pitch_textures</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.granular_time_stretch_engine</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.rhythmic_gate_stutter_quantizer</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.
<code>granular_and_modulation.spectral_tremolo</code>	Grain and modulation rates are musically matched to source texture.	Incoherent grain scheduling can produce choppiness or blur.	Avoid dense granular settings on speech intelligibility-critical content.

B.8 pitch_detection_and_tracking

The relevant properties of the `pitchdetectionandtracking` are compared in the following table.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>pitch_detection_and_tracking.crepe_style_neural_f0</code>	F0 evidence is strong in the selected analysis band and frame size. Model priors assume training-like signal statistics.	Octave errors and voicing flips under heavy noise/polyphony. Generalization gaps can produce unstable artifacts.	Avoid as the sole control signal for dense polyphonic mixtures. Avoid fully unattended use on out-of-domain material.
<code>pitch_detection_and_tracking.harmonic_product_spectrum_hps</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking.pyin</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking.rapt</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>pitch_detection_and_tracking.subharmonic_summation</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking.swipe</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking.viterbi_smoothed_pitch_contour_tracking</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.
<code>pitch_detection_and_tracking.yin</code>	F0 evidence is strong in the selected analysis band and frame size.	Octave errors and voicing flips under heavy noise/polyphony.	Avoid as the sole control signal for dense polyphonic mixtures.

B.9 retune_and_intonation

A tabular view makes the distinctions within the retuneandintonation easier to inspect.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>retune_and_intonation.adaptive_intonation_context_sensitive_intervals</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.chord_aware_retuning</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.just_intonation_mapping_per_key_center</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.key_aware_retuning_with_confidence_weighting</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>retune_and_intonation.portamento_aware_retune_curves</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.scala_mts_scale_import_and_quantization</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.time_varying_cents_maps</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.
<code>retune_and_intonation.vibrato_preserving_correction</code>	Detected notes map cleanly to intended tonal center/scale. Pitch trajectory estimates should be continuous enough for retuning.	Over-correction can flatten expressive vibrato or slides. Fast F0 jumps can cause audible stepping.	Avoid aggressive correction when preserving natural micro-intonation is required. Avoid high-strength retune on breath/noise segments.

B.10 separation_and_decomposition

The table below gathers the principal details of the separationanddecomposition.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>separation_and_decomposition.demucs_style_stem_separation_backend</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.ica_bss_for_multichannel_stems</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.nmf_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>separation_and_decomposition_probabilistic_latent_component_separation</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.rpca_hpss</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.sinusoidal_residual_transient_decomposition</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.tensor_decomposition_cp_tucker</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.
<code>separation_and_decomposition.u_net_vocal_accompaniment_split</code>	Sources have partially separable spectral or statistical structure.	Component bleeding and musical noise under overlap or model mismatch.	Avoid expecting perfect stems from strongly correlated or co-modulated sources.

B.11 `spatial_and_multichannel`

The relevant properties of the `spatialandmultichannel` are compared in the following table.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spatial_and_multichannel.binaural_itd_ild_synthesis</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.binaural_motion_trajectory_designer</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.coherence_based_dereverb_multichannel</code>	Channel geometry/order and timing metadata are correct. Reverberation tail is assumed more diffuse than direct content.	Spatial collapse, combing, or localization bias from misalignment. Over-suppression can thin tonal body and ambience.	Avoid blind spatial processing when channel order/calibration is unknown. Avoid for intentionally wet effects unless mix preservation is planned.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spatial_and_multichannel.cross_channel_click_pop_repair</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.dbap_distance_based_amplitude_panning</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.decorrelated_reverb_upmix</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.microphone_array_calibration_tones</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.multichannel_noise_psd_tracking</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.multichannel_wiener_postfilter</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.phase_aligned_mid_side_field_rotation</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.phase_consistent_multichannel_denoise</code>	Channel geometry/order and timing metadata are correct. Noise model should be representative of observed noise floor.	Spatial collapse, combing, or localization bias from misalignment. Mismatched noise estimate leaves residue or damages detail.	Avoid blind spatial processing when channel order/calibration is unknown. Avoid static settings on rapidly varying nonstationary noise.
<code>spatial_and_multichannel.pvx_directional_spectral_warp</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.pvx_interaural_coherence_shaping</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>spatial_and_multichannel.pvx_interchannel_phase_locking</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.pvx_multichannel_time_alignment</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.pvx_spatial_freeze_and_trajectory</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.pvx_spatial_transient_preservation</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.rotating_speaker_doppler_field</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.spatial_freeze_resynthesis</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.spectral_spatial_granulator</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.stereo_width_frequency_dependent_control</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.stochastic_spatial_diffusion_cloud</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
<code>spatial_and_multichannel.transaural_crosstalk_cancellation</code>	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.

spatial_and_multichannel.vbap_adaptive_panning	Channel geometry/order and timing metadata are correct.	Spatial collapse, combing, or localization bias from misalignment.	Avoid blind spatial processing when channel order/calibration is unknown.
--	---	--	---

B.12 spectral_time_frequency_transforms

A tabular view makes the distinctions within the spectraltimefrequencytransforms easier to inspect.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
spectral_time_frequency_transforms.chirplet_transform_analysis	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.constant_q_transform_cqt_processing	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.multi_window_stft_fusion	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.nsgt_based_processing	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.reassigned_spectrogram_methods	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.synchrosqueezed_stft	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.variable_q_transform_vqt	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.
spectral_time_frequency_transforms.wavelet_packet_processing	Transform parameterization matches target time-frequency structure.	Incorrect parameterization can smear events or over-fragment spectra.	Avoid default settings for highly nonstationary signals without tuning.

B.13 time_scale_and_pitch_core

A tabular view makes the distinctions within the timescaleandpitchcore easier to inspect.

Algorithm ID	Assumptions	Failure Modes	When Not To Use
<code>time_scale_and_pitch_core.beat_synchronous_time_warping</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core.harmonic_percussive_split_tsm</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core.lp_psola</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core.multi_resolution_phase_vocoder</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth. Phase continuity assumptions hold best for moderate stretch ratios.	High-ratio stretch can introduce phasiness and blurred transients. Extreme settings increase phasiness/transient blur risk.	Avoid for extreme percussive-only material when attack realism is critical. Avoid very large stretch+pitch shifts without transient controls.
<code>time_scale_and_pitch_core.nonlinear_time_maps</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core.td_psola</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.
<code>time_scale_and_pitch_core.wsola_waveform_similarity_overlap_add</code>	Frames are locally quasi-stationary and harmonic evolution is reasonably smooth.	High-ratio stretch can introduce phasiness and blurred transients.	Avoid for extreme percussive-only material when attack realism is critical.

APPENDIX C

pvx Diagram Atlas

This document provides expanded architecture and digital signal processing (DSP) flow diagrams for pvx. Diagrams are a mix of Mermaid (GitHub-rendered) and ASCII (terminal-friendly).

C.1 End-to-End pvxvoc Flow

The material below develops the end-to-end pvxvoc flow in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.2 STFT Analysis / Synthesis Timeline

The following session demonstrates the STFT analysis / synthesis timeline in practice.

```
signal: x[n] ----->

windows:
    |---N---|
      |---N---|
        |---N---|
hop:      <---Ha--->

analysis:
    frame t0 -> FFT -> X0[k]
    frame t1 -> FFT -> X1[k]
    frame t2 -> FFT -> X2[k]

processing:
    magnitude trajectory + phase trajectory updates

synthesis:
    IFFT(frame0), IFFT(frame1), IFFT(frame2) + overlap-add with Hs
```

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.3 Phase Propagation and Locking

A closer look at the phase propagation and locking clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
Frame index:   t0      t1      t2      t3
Peak bin:      0-----0-----0-----0
Neighbor free:  o--o-----o---o-----o----- (drift can increase blur)
Neighbor lock:  o-----o-----o-----o----- (relative peak shape preserved)
```

C.4 Hybrid Transient Engine

A closer look at the hybrid transient engine clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
time ----->
mask 00000011110000001111000000
    steady^^trans^^steady^^trans^^steady
```

C.5 Transient Feature Stack

The next example makes the role of the transient feature stack explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.6 WSOLA Similarity Search

The example below puts the wsola similarity search into executable form.

```
reference analysis frame:    [=====R=====]

candidate search window:
                        [c0] [c1] [c2] [c3] [c4] [c5] [c6]
                        \   \   \   \   \   \
similarity score:          s0  s1  s2  s3  s4  s5

choose argmax(similarity) -> best offset -> overlap-add with crossfade
```

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.7 Stereo / Multichannel Coherence Modes

The material below develops the stereo / multichannel coherence modes in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
independent:    L and R evolve freely
mid_side_lock:  preserve center/side coherence envelope
ref_channel_lock: keep phase relation anchored to one channel
```

C.8 Microtonal Map Processing

The material below develops the microtonal map processing in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.9 Control-Rate Interpolation Graph Examples

The same control points produce different trajectories depending on `--interp` and `--order`.

Mode/order	Graph
none	
nearest	
linear	
cubic	
exponential	
s_curve	

Mode/order	Graph
smootherstep	
polynomial --order 1	
polynomial --order 2	
polynomial --order 3	
polynomial --order 5	

C.10 Function Graph Gallery

The visual material in this section develops the function graph gallery through annotated examples.

Function family	Graph
Pitch ratio vs semitones	
Pitch ratio vs cents	
Dynamics transfer curves	
Soft clip transfer functions	
Morph blend magnitude curves	
Mask exponent response	
Phase mix curve	

C.11 Extreme Stretch: Multistage Strategy

A closer look at the extreme stretch: multistage strategy clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
example total stretch = 600x
600 = 5 * 5 * 4.8 * 5
render sequentially with safer stage factors
```

C.12 Transform Backend Decision Path

The material below develops the transform backend decision PATH in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
preferred default: fft
research/verification: dft/czt/dct/dst/hartley
```

C.13 Mastering Chain Block Diagram

A closer look at the mastering chain block diagram clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.14 Metrics Printing Path

The material below develops the metrics printing PATH in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.15 Benchmark Runner + CI Gate

The next example makes the role of the benchmark runner + ci gate explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.16 Pipe Chaining Pattern

The following session demonstrates the pipe chaining pattern in practice.

```
pvx voc input.wav --stdout \  
| pvx denoise - --stdout \  
| pvx deverb - --output final.wav  
  
rule:  
- producer uses --stdout  
- consumer reads '-' as stdin
```

C.17 Checkpoint / Resume State Machine

The next example makes the role of the checkpoint / resume state machine explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.18 Troubleshooting Decision Tree

The material below develops the troubleshooting decision tree in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.19 Data Product Map

A working command makes the data product map concrete.

```
markdown docs -> docs/*.md  
html docs -> docs/html/*.html  
pdf bundle -> docs/pvx_documentation.pdf  
benchmark json -> benchmarks/out/report.json  
benchmark md -> benchmarks/out/report.md
```

C.20 Window/Overlap Tradeoff Map

A closer look at the window/overlap tradeoff map clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

```
larger window + higher overlap -> smoother/cleaner sustained content  
smaller window + lower overlap -> sharper time localization, rougher bass resolution
```

C.21 Phase Reset vs Hybrid Boundary Behavior

A working command makes the phase reset vs hybrid boundary behavior concrete.

```

time ----->
transient mask:    000011100000001110000

mode=reset:
phase tracks  ---->|reset|----->|reset|----->

mode=hybrid:
pv steady     ----[PV]-----    ----[PV]-----
wsola transient  [WSOLA]         [WSOLA]
stitch         <xfade>           <xfade>

```

C.22 Multi-Resolution Fusion Fan-In

The material below develops the multi-resolution fusion fan-in in practical terms.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.23 Pitch-Map Confidence Gate

A closer look at the pitch-map confidence gate clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.24 Microtonal CSV Interpolation Flow

A closer look at the microtonal CSV interpolation flow clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.25 Audio Metrics Table Generation

The entries in this section organize the audio metrics table generation for comparison and practical use.

```

input.wav  -> basic metrics (sr,ch,duration,peak,rms,crest,dc,zcr,bw95,clip%)
output.wav -> basic metrics (same fields)
paired cmp -> compare metrics (snr,si-sdr,lsd,mod-dist,envlope corr,...)

console:
table A: input summary
table B: output summary
table C: input vs output (input / output / delta)

```

C.26 Quality-First Optimization Loop

The next example makes the role of the quality-first optimization loop explicit.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.27 Benchmark Metric Taxonomy

A closer look at the benchmark metric taxonomy clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

C.28 Documentation Build Pipeline

A closer look at the documentation build pipeline clarifies the choices involved.

Diagram source omitted in the PDF build. Use the HTML docs for the rendered version.

Use this atlas together with: - docs/MATHEMATICAL_FOUNDATIONS.md for equation-level details
- docs/EXAMPLES.md for copy-paste command recipes

APPENDIX D

pvx Phasiness Implementation Plan

D.1 Why this paper matters

About this Phasiness Business isolates why classic phase-vocoder output sounds “phasier” under stretch: weak cross-bin phase coherence, unstable peak-region phase behavior, and frame-to-frame phase inconsistency in dense harmonic material.

For **pvx**, this gives a quality-first roadmap:

- strengthen local phase locking around spectral peaks
- preserve vertical coherence (within a frame) and horizontal coherence (across frames)
- reduce phase random walk in low-energy bins
- make coherence strategy content-aware (speech/tonal/percussive)

D.2 Benefits we can extract for pvx

Before applying benefits we can extract for **pvx**, keep these constraints in view.

1. Better harmonic solidity at moderate and high stretch ratios.
2. Lower “swirl/smear” in sustained vocals and pads.
3. More stable stereo image when coherence is enforced consistently across channels.
4. Cleaner transient neighborhoods when hybrid transient handling and phase resets are coordinated.
5. Stronger objective benchmark results on modulation- and coherence-sensitive metrics.

D.3 Phased implementation

The discussion of the phased implementation begins by separating its main parts.

Phase 1: Coherence-aware phase propagation in core

Files: - `src/pvx/core/voc.py` - `src/pvx/core/stereo.py`

Work: - Add explicit “peak-region phase-locking radius” control. - Lock neighboring bins to anchor-bin phase increments near detected peaks. - Add low-energy-bin damping to avoid unstable phase diffusion. - Keep deterministic branch for central processing unit (CPU) mode.

CLI additions: - `--phase-lock-radius <bins>` - `--phase-lock-strength <0..1>` - `--phase-noise-floor-db <dB>`

Phase 2: Adaptive coherence policy

Files: - `src/pvx/core/presets.py` - `src/pvx/core/transients.py` - `src/pvx/core/voc.py`

Work: - Auto-select coherence mode by frame class: - tonal frames: stronger identity/peak lock - noisy/percussive frames: relaxed lock + transient protection - Fuse with transient-mode handling (`reset`, `hybrid`, `wsola`).

CLI additions: - `--phase-policy {static,adaptive}` - `--phase-tonal-lock <0..1>` - `--phase-noise-lock <0..1>`

Phase 3: Stereo/multichannel coherence extension

Files: - `src/pvx/core/stereo.py` - `src/pvx/metrics/coherence.py`

Work: - Apply lock decisions in mid/side (M/S) domain for stereo-preserving stretch. - Improve reference-channel lock to retain inter-channel phase deltas near peaks. - Add diagnostics for per-band coherence drift.

CLI additions: - `--coherence-report-json <path>`

Phase 4: Benchmark and regression gates

Files: - `benchmarks/metrics.py` - `benchmarks/run_bench.py` - `.github/workflows/bench-regression.yml`

Work: - Add phasiness-focused metrics: - inter-frame phase-deviation variance - peak-region phase-consistency score - modulation-spectrum stability score - Add strict pass/fail thresholds for tonal and speech subsets.

Phase 5: UX and docs

Files: - `README.md` - `docs/QUALITY_GUIDE.md` - `docs/EXAMPLES.md`

Work: - Add “phasiness reduction” intent preset. - Add examples: - vocal stretch quality profile - ambient pad high-ratio profile - stereo-coherent mix-bus profile - Add troubleshooting matrix: artifact -> parameter moves.

D.4 Validation targets

These are the details that matter for validation targets.

- Lower modulation-spectrum distance on sustained harmonic tests.
- Lower stereo coherence drift on stereo test corpus.
- Audible A/B improvement on speech and vocal sustained vowels.
- No regression in `stretch=1.0` identity reconstruction tolerance.

D.5 Source paper and cited references to track

The working assumptions for source paper and cited references to track are:

- J. Laroche; M. Dolson (1997), *About this Phasiness Business*.
- M. Dolson (1986), *The phase vocoder: A tutorial*.
- D. W. Griffin; J. S. Lim (1984), *Signal estimation from modified short-time Fourier transform*.
- J. Laroche; M. Dolson (1997), *Phase-vocoder: About this phasiness business* (submitted manuscript citation in paper).
- E. Moulines; J. Laroche (1995), *Non-parametric techniques for pitch-scale and time-scale modification of speech*.
- M. S. Puckette (1995), *Phase-locked vocoder*.

APPENDIX E

pvx Citation Quality Report

Generated from commit `35e9761` (commit date: `2026-05-25T08:14:42-04:00`).

This report classifies bibliography URLs by citation quality and highlights entries still using search-index links.

Total references analyzed: **248**

E.1 Link-Type Summary

A tabular view makes the distinctions within the link-type summary easier to inspect.

	Link type	Count
	arxiv	8
	doi	90
	publisher_or_standard	10
	scholar	88
	web	52

E.2 Entries Still Using Scholar Links

These are prime targets for future DOI/publisher URL upgrades.

Year	Authors	Title
2021	S. Wisdom et al.	Differentiable Consistency Constraints for Improved Deep Speech Enhancement
2020	J. Kong et al.	HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis
2020	S. Stables et al.	A Study of Loudness Normalization in Streaming Services
2019	R. Prenger et al.	WaveGlow: A Flow-based Generative Network for Speech Synthesis
2019	M. H. C. de Gesmundo et al.	ViSQOL v3: An Open Source Production Ready Objective Speech and Audio Metric
2019	J. Le Roux; S. Wisdom; H. Erdogan; J. R. Hershey	SDR half-baked or well done?
2019	M. Doring; M. M. Kokuer	Practical NSGT audio applications

Year	Authors	Title
2019	F.-R. Stoter; S. Uhlich; A. Liutkus; Y. Mitsufuji	Open-Unmix
2019	A. Defossez et al.	Music Source Separation in the Waveform Domain
2019	K. Kumar et al.	MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis
2019	A. Pandey; D. Wang	A New Framework for CNN-Based Speech Enhancement in the Time Domain
2018	S. Bock; M. Korzeniowski	piano_transcription - F0 and onset tracking
2018	N. W. D. Evans et al.	The voicehome and CHiME challenges
2018	M. Fenton	Practical Dynamics Processing for Modern Music Production
2018	N. Takahashi et al.	MMDenseLSTM for source separation
2018	N. Kalchbrenner et al.	Efficient Neural Audio Synthesis
2018	C. Donahue; J. McAuley; M. Puckette	Adversarial Audio Synthesis
2018	J.-M. Valin	A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement
2017	Y. Wang et al.	Tacotron: Towards End-to-End Speech Synthesis
2017	A. Gibiansky et al.	Deep Voice 2: Multi-Speaker Neural Text-to-Speech
2016	K. Kinoshita et al.	A summary of the REVERB challenge
2015	M. Riess; A. R. Chhetri	Transient Preservation in Time-Scale Modification
2015	B. W. Schuller et al.	Paralinguistics and robust pitch tracking methods
2015	M. Ballou	Handbook for Sound Engineers (Loudness and Dynamics chapters)
2014	J. Driedger; T. Pratzlich; M. Muller	Let It Bee - Towards NMF-Inspired Audio Mosaicing
2013	P. C. Loizou	Speech Enhancement: Theory and Practice
2013	M. Dolson	History of the Phase Vocoder
2012	A. Hines et al.	VISQOL: An Objective Speech Quality Model
2012	B. Zavalishin	The Art of VA Filter Design (sections on phase and spectral processing)
2012	N. D. Gaubitch; P. A. Naylor	Speech Dereverberation

Year	Authors	Title
2012	K. Dressler	Sinusoidal Extraction using Frequency-Domain Matching Pursuit
2012	S. Essid; G. Richard	Musical signal analysis with reassignment methods
2012	Y. Yoshioka; T. Nakatani	Generalization of Multi-Channel Linear Prediction Methods for Blind MIMO Impulse Response Shortening
2012	T. Yoshioka; T. Nakatani	Generalization of Multi-Channel Linear Prediction Methods for Blind MIMO Impulse Response Shortening
2012	D. Giannoulis; M. Massberg; J. Reiss	Digital Dynamic Range Compressor Design
2011	J. Reiss	Under the Hood of a Dynamic Range Compressor
2011	N. Ono	Stable and fast update rules for independent low-rank matrix analysis based on auxiliary function technique
2011	E. J. Candes; X. Li; Y. Ma; J. Wright	Robust Principal Component Analysis?
2011	K. K. Paliwal	Importance of phase in speech enhancement
2011	M. Le Roux; E. Vincent	Consistent Wiener Filtering for Audio Source Separation and Time-Frequency Processing
2011	C. Taal et al.	An Algorithm for Intelligibility Prediction of Time-Frequency Weighted Noisy Speech
2010	D. S. Hamon; A. Lazarides	Improved WSOLA for Real-Time Time-Scale Modification
2010	A. V. Oppenheim; R. W. Schafer	Discrete-Time Signal Processing (STFT and filter-bank chapters)
2010	C. Schörkhuber; A. Klapuri	Constant-Q Transform Toolbox for Music Processing
2010	A. Roebel	A Shape-Invariant Phase Vocoder for Speech Transformation
2009	N. Moreau	Toolbox for Time-Scale Modification and Pitch-Shifting of Audio Signals
2009	C. Fevotte; N. Bertin; J.-L. Durrieu	Nonnegative Matrix Factorization with the Itakura-Saito Divergence
2008	J. Bonada	Wide-Band Harmonic Sinusoidal Modeling for Time-Scale and Pitch-Scale Modification

Year	Authors	Title
2007	T. Virtanen	Monaural Sound Source Separation by Nonnegative Matrix Factorization with Temporal Continuity and Sparseness Criteria
2007	T. Lund	Loudness and True-Peak in Digital Audio
2007	P. Smaragdis	Convolutional Speech Bases and Their Application to Supervised Speech Separation
2006	A. Klapuri	Multiple Fundamental Frequency Estimation by Harmonicity and Spectral Smoothness
2006	A. C. Gilbert; M. J. Strauss	Approximation of signals in sparse Fourier dictionaries
2005	M. McLeod; G. Wyvill	A Smarter Way to Find Pitch
2004	A. de Cheveigne	Pitch and Time Manipulation of Speech
2002	P. Flandrin; F. Auger; E. Chassande-Mottin	Time-Frequency Reassignment: From Principles to Algorithms
2002	C. Duxbury; M. Davies; M. Sandler	Improved Time-Scaling of Musical Audio Using Phase Locking at Transients
2002	S. Arfib; D. Keiler; U. Zölzer	DAFX: Digital Audio Effects (chapter references on pitch/time processing)
2001	I. Cohen; B. Berdugo	Speech Enhancement for Non-Stationary Noise Environments
2001	H. Kawahara; A. de Cheveigne	STRAIGHT, Exploitation of the Other Aspects of Vocal Source Information
2001	A. Rix et al.	Perceptual Evaluation of Speech Quality (PESQ)
2001	K. Grochenig	Foundations of Time-Frequency Analysis
2000	S. Kum; C. L. Nikias	Robust F0 Estimation in Noise
2000	A. Hyvarinen; E. Oja	Independent Component Analysis: Algorithms and Applications
2000	J. Bonada	Automatic Technique in Frequency Domain for Near-Lossless Time-Scale Modification of Audio
1999	S. Ahmadi; H. Spanias	Cepstrum-Based Pitch Detection Using FFT
1998	P. Flandrin	Time-Frequency/Time-Scale Analysis
1998	S. Disch	A New Phase Vocoder Technique for Time-Scale Modification of Audio Signals

Year	Authors	Title
1997	S. Pulkki	Virtual Sound Source Positioning Using Vector Base Amplitude Panning
1996	P. Scalart; J. V. Filho	Speech Enhancement Based on a Priori Signal to Noise Estimation
1995	M. Puckette	Phase-Locked Vocoder
1995	D. L. Donoho	De-Noising by Soft-Thresholding
1995	A. J. Bell; T. J. Sejnowski	An Information-Maximization Approach to Blind Separation and Blind Deconvolution
1995	D. Talkin	A Robust Algorithm for Pitch Tracking (RAPT)
1993	S. Mallat; Z. Zhang	Matching Pursuits with Time-Frequency Dictionaries
1993	W. Verhelst; M. Roelands	An Overlap-Add Technique Based on Waveform Similarity (WSOLA) for High Quality Time-Scale Modification of Speech
1993	P. Boersma	Accurate Short-Term Analysis of the Fundamental Frequency and the Harmonics-to-Noise Ratio of a Sampled Sound
1991	S. Mann; S. Haykin	The Chirplet Transform: Physical Considerations
1989	L. Cohen	Time-Frequency Distributions - A Review
1989	R. Bristow-Johnson; M. Bogdanowicz	Phase Vocoder Done Right
1986	R. J. McAulay; T. F. Quatieri	Speech Analysis/Synthesis Based on a Sinusoidal Representation
1985	N. Roucos; A. Wilgus	High Quality Time-Scale Modification for Speech
1980	M. R. Portnoff	Time-Scale Modification of Speech Based on Short-Time Fourier Analysis
1979	S. Boll	Suppression of Acoustic Noise in Speech Using Spectral Subtraction
1976	M. R. Portnoff	Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform
1976	L. R. Rabiner; M. J. Cheng; A. E. Rosenberg; C. A. McGonegal	A Comparative Performance Study of Several Pitch Detection Algorithms
1967	A. M. Noll	Cepstrum Pitch Determination
1949	N. Wiener	Extrapolation, Interpolation, and Smoothing of Stationary Time Series

APPENDIX F

PVC Phase 3-5 Architecture

This document describes the architecture used to implement the next three PVC-inspired phases:

1. Phase 3: response-driven spectral operators
2. Phase 4: ring/resonator family
3. Phase 5: harmonic/chord mapping family

F.1 Module Layout

Core signal processing modules:

- `/Users/cleider/dev/pvx/src/pvx/core/pvc_ops.py`
- `/Users/cleider/dev/pvx/src/pvx/core/pvc_resonators.py`
- `/Users/cleider/dev/pvx/src/pvx/core/pvc_harmony.py`

Command-line interface (CLI) entry points:

- Response operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxtvfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxnoisefilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxbandamp.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxspeccompannder.py`
- Ring/resonator operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxring.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxringfilter.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxringtvfilter.py`
- Harmonic/chord operators:
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxharmmmap.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxchordmapper.py`
 - `/Users/cleider/dev/pvx/src/pvx/cli/pvxinharmonator.py`

Unified dispatcher registration:

- `/Users/cleider/dev/pvx/src/pvx/cli/pvx.py`

F.2 Processing Contracts

All new operators follow the same signal contract:

1. Read audio (mono or multichannel) as floating point.
2. Process each channel deterministically.
3. Preserve sample rate and target output length.
4. Pass output through shared mastering/output policy.
5. Emit non-silent console metrics tables and status bars.

F.3 Phase 3 Design

`pvc_ops.py` provides five operators:

- **filter**: static response-curve spectral shaping

- **tvfilter**: time-varying response mix from CSV/JSON
- **noisefilter**: response-floor spectral attenuation
- **bandamp**: response-peak selective amplification
- **spec-compander**: response-referenced spectral companding

Key design choices:

- Response artifacts are loaded from PVXRF.
- Response curves are resized/shifted to match active STFT bins.
- Time-varying controls are scalar maps with interpolation modes:
 - `none`, `stairstep`, `nearest`, `linear`, `cubic`, `exponential`, `s_curve`, `smootherstep`, `polynomial`

F.4 Phase 4 Design

`pvc_resonators.py` provides:

- **ring**: ring modulation
- **ringfilter**: ring modulation + resonant peak filter
- **ringtvfilter**: time-varying ring controls + resonant filter

Key design choices:

- Sample-rate-domain modulation for phase continuity.
- Bounded feedback and bounded resonance decay for stability.
- Optional CSV/JSON control maps for `frequency_hz`, `depth`, and `mix`.

F.5 Phase 5 Design

`pvc_harmony.py` provides:

- **chordmapper**: chord-class spectral weighting (root + chord quality)
- **inharmonator**: inharmonic spectral frequency warp

Key design choices:

- Chord mapping uses circular pitch-class distance in cents.
- Inharmonator uses an analytic inverse warp from output-bin frequency to input-bin sampling frequency.
- Both operators are implemented in the short-time Fourier transform (STFT) domain with overlap-add resynthesis.

F.6 Backward Compatibility and UX

The working assumptions for backward compatibility and ux are:

- Existing tools are untouched.
- New tools are additive and available both:
 - via unified command-line interface (CLI): `pvx <tool>`
 - via root wrappers: `python3 pvx<tool>.py ...`
- Shared conventions are reused: progress/status, verbosity, metrics table, mastering chain.

APPENDIX G

A Phase-Vocoder Listening Library: 100 Musical Works

This appendix is a guided listening and processing curriculum. It combines a small set of historically grounded phase-vocoder precedents with a much larger laboratory of famous works chosen because their voices, attacks, resonances, textures, or large-scale timing make particular transform behaviors easy to hear. The label attached to each entry matters. **Documented** identifies direct phase-vocoder practice supported by the historical literature. **Historical context** identifies related spectral and computer-music practice without claiming that every sound in the named work was made by a phase vocoder. **Listening laboratory** means that the work is recommended as source material or as a perceptual reference; it is not a claim about the production of the original recording.

YouTube changes individual upload URLs, regional availability, editions, and rights status frequently. For that reason, each musical title links to a search for complete performances or recordings rather than endorsing one upload. Readers should choose lawful sources, respect recording and composition rights, and use public-domain or properly licensed audio for exported experiments.

The historically grounded group is supported by Trevor Wishart’s retrospective on twenty-five years of phase-vocoder practice, the Library of Congress genealogy of Roger Reynolds’s *Transfigured Wind*, IRCAM’s analysis of Jonathan Harvey’s *Mortuos Plango, Vivos Voco*, and the broader phase-vocoder literature cited in Chapter 1. These sources can be consulted through [Wishart’s retrospective](#), the [Library of Congress essay](#), and the [IRCAM analysis collection](#).

G.1 How to use the library

Each entry proposes one focused experiment. Begin with a short, legally obtained excerpt and keep an unprocessed reference at matched loudness. Change one variable at a time, render enough duration to expose the behavior, and record the command, window, hop, phase mode, transient policy, and random seed. The listening notes are prompts rather than expected answers.

G.2 Documented phase-vocoder practice

The works in this group emphasize documented phase-vocoder practice. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

1. Trevor Wishart: *Vox 5*

Documented phase-vocoder practice. Trevor Wishart: *Vox 5* (1986). Listen for the voice crossing into animal, mechanical, and environmental identities without a clean boundary.

Stretch a spoken vowel eight times, preserve formants, then morph its magnitude envelope toward a noisy recording.

2. Roger Reynolds: *Transfigured Wind I*

Documented phase-vocoder practice. Roger Reynolds: *Transfigured Wind I* (1984). Follow the flute as a performed gesture becomes prolonged spectral architecture.

Freeze a flute tone after its attack, then automate stretch and spectral focus while retaining the original onset.

3. JoAnn Kuchera-Morin: *Dreampaths*

Documented phase-vocoder practice. JoAnn Kuchera-Morin: *Dreampaths* (1989). Attend to long transformations whose internal spectral motion remains perceptible at expanded scale.

Expand a thirty-second source to three minutes and compare independent, locked, and transient-aware phase modes.

4. Jonathan Harvey: *Mortuos Plango, Vivos Voco*

Documented phase-vocoder practice. Jonathan Harvey: *Mortuos Plango, Vivos Voco* (1980). Compare the partial structures of a boy's voice and a cathedral bell as they approach and depart from one another.

Analyze voice and bell recordings, transpose each by measured partial ratios, and cross-synthesize their envelopes.

5. Trevor Wishart: *Supernova*

Documented phase-vocoder practice. Trevor Wishart: *Supernova* (2012). Listen for spectral data treated as compositional material across a large spatial and temporal field.

Create several extreme stretches from one source, filter different spectral regions, and distribute the layers in space.

6. Trevor Wishart: *Vox Cycle*

Documented phase-vocoder practice. Trevor Wishart: *Vox Cycle* (1979-1988). Hear vocal identity as a continuum extending from speech and song to noise and imagined creatures.

Build a chain of short vocal transformations in which each output becomes the source for the next operation.

G.3 Spectral-composition context

The works in this group emphasize spectral-composition context. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

7. Trevor Wishart: *Red Bird*

Related historical and spectral context. Trevor Wishart: *Red Bird* (1977). Study the dramatic continuity between recognizable recordings and transformed sonic images.

Use spectral morphing to connect speech, birdsong, machinery, and crowd noise while preserving selected gestures.

8. Trevor Wishart: *Tongues of Fire*

Related historical and spectral context. Trevor Wishart: *Tongues of Fire* (1994). Listen for the multiplication of a single vocal world into a dense ecology of related timbres.

Separate voiced and noisy regions of speech, stretch them by different amounts, and recombine them with automation.

9. Tristan Murail: *Desintegrations*

Related historical and spectral context. Tristan Murail: *Desintegrations* (1982-1983). Trace how instrumental harmony and computer-generated spectra share one harmonic vocabulary.

Extract prominent partials from a resonant source and retune an instrumental passage toward those frequencies.

10. Wendy Carlos: *Digital Moonscapes*

Related historical and spectral context. Wendy Carlos: *Digital Moonscapes* (1984). Compare sharply articulated synthetic events with broad, evolving digital sonorities.

Render one synthesizer phrase at several stretch factors and compare transient preservation against spectral smoothness.

11. Jean-Claude Risset: *Inharmonique*

Related historical and spectral context. Jean-Claude Risset: *Inharmonique* (1977). Listen for the changing boundary between soprano resonance and computer-generated spectral space.

Freeze several sung vowels, shift their formants independently of pitch, and layer the results under the original voice.

12. Jean-Claude Risset: *Songes*

Related historical and spectral context. Jean-Claude Risset: *Songes* (1979). Attend to illusions of source, scale, and motion in a continuously changing electroacoustic field.

Automate spectral transposition and filtering on resonant recordings so that familiar sources become ambiguous.

13. Kaija Saariaho: *Lichtbogen*

Related historical and spectral context. Kaija Saariaho: *Lichtbogen* (1986). Follow the exchange of brightness, noise, and harmonic focus between ensemble and electronics.

Track spectral centroid from a bowed tone and use it to control filtering and stretch in a second layer.

14. Kaija Saariaho: *NoaNoa*

Related historical and spectral context. Kaija Saariaho: *NoaNoa* (1992). Listen to how breath, flute tone, speech, and electronics form a single timbral sentence.

Split flute attacks, breath noise, and sustained tone into separate paths, then transform each with a different window.

15. Paul Lansky: *Six Fantasies on a Poem by Thomas Campion*

Related historical and spectral context. Paul Lansky: *Six Fantasies on a Poem by Thomas Campion* (1978-1979). Study the musical organization of speech resonance, contour, and repetition.

Retune and time-stretch spoken phrases while comparing preserved and shifted formant envelopes.

G.4 Voice, text, and formants

The works in this group emphasize voice, text, and formants. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

16. Hildegard von Bingen: *O vis aeternitatis*

Comparative listening laboratory. Hildegard von Bingen: *O vis aeternitatis* (12th century). The unaccompanied line exposes pitch continuity, breath, room decay, and vowel formants.

Stretch a sustained phrase four times with formant preservation, then repeat without it and compare vowel identity.

17. Anonymous: *Dies irae*

Comparative listening laboratory. Anonymous: *Dies irae* (13th century). Monophonic chant makes phase continuity and consonant treatment unusually easy to hear.

Use transient protection on consonants while applying a slowly changing pitch map to the sustained syllables.

18. Josquin des Prez: *Ave Maria virgo serena*

Comparative listening laboratory. Josquin des Prez: *Ave Maria virgo serena* (c. 1485). Imitative entries reveal whether independent vocal transformations retain contrapuntal clarity.

Stretch each voice separately with shared timing but independent formant settings, then compare against a linked render.

19. Thomas Tallis: *Spem in alium*

Comparative listening laboratory. Thomas Tallis: *Spem in alium* (c. 1570). Forty vocal parts provide an exacting test of phase density, localization, and spectral accumulation.

Apply a modest global stretch, then compare channel-independent and reference-locked processing for image stability.

20. Claudio Monteverdi: *Lamento d'Arianna*

Comparative listening laboratory. Claudio Monteverdi: *Lamento d'Arianna* (1608). The expressive line combines long vowels with speech-like attacks and rapid ornaments.

Protect attacks, stretch vowels selectively, and automate formant shift only during the longest notes.

21. Henry Purcell: *Dido's Lament*

Comparative listening laboratory. Henry Purcell: *Dido's Lament* (1689). A descending ground and sustained vocal line make pitch drift and vibrato handling conspicuous.

Stretch the voice independently of the ground bass and test identity phase locking on the sustained notes.

22. Johann Sebastian Bach: *Erbarme dich*

Comparative listening laboratory. Johann Sebastian Bach: *Erbarme dich* (1727). Voice and obbligato violin offer two related but distinct sources of vibrato and legato continuity.

Match their stretch maps, preserve each source's attacks, and compare linked versus independent phase treatment.

23. George Frideric Handel: *Lascia ch'io pianga*

Comparative listening laboratory. George Frideric Handel: *Lascia ch'io pianga* (1711). Regular phrases and exposed vowels make formant displacement and modulation artifacts easy to identify.

Create three pitch-shifted versions at minus five, zero, and plus seven semitones with a fixed formant envelope.

24. Wolfgang Amadeus Mozart: *Der Holle Rache*

Comparative listening laboratory. Wolfgang Amadeus Mozart: *Der Holle Rache* (1791). Coloratura attacks stress transient detection, peak tracking, and rapid pitch movement.

Compare short and long windows on a fast passage, keeping output duration and pitch unchanged.

25. Franz Schubert: *Erlkonig*

Comparative listening laboratory. Franz Schubert: *Erlkonig* (1815). Rapid text, register changes, and repeated piano figures reveal temporal blur immediately.

Stretch voice and piano separately, using stronger transient protection for the accompaniment.

26. Richard Wagner: *Mild und leise*

Comparative listening laboratory. Richard Wagner: *Mild und leise* (1859). The long final ascent tests spectral accumulation, vibrato continuity, and orchestral masking.

Freeze a climactic vowel and slowly expand its spectral envelope while the orchestral source remains time-aligned.

27. Gustav Mahler: *Ich bin der Welt abhanden gekommen*

Comparative listening laboratory. Gustav Mahler: *Ich bin der Welt abhanden gekommen* (1901). Quiet sustained singing against transparent orchestration exposes noise, breath, and low-level phase modulation.

Render a gentle 1.5-times stretch with three windows and compare breath texture and instrumental tails.

28. Arnold Schoenberg: *Mondestrunken*

Comparative listening laboratory. Arnold Schoenberg: *Mondestrunken* (1912). Sprechstimme sits between speech and song, making pitch and formant policy an artistic decision.

Extract the pitch contour, smooth it at several rates, and use it to retune a duplicated vocal layer.

29. Luciano Berio: *Sequenza III*

Comparative listening laboratory. Luciano Berio: *Sequenza III* (1965). Whispers, laughter, phonemes, and sung tones demand separate treatment of voiced and unvoiced material.

Route voiced and noisy frames to different transformations, then crossfade the outputs using the original voicing measure.

30. Steve Reich: *Different Trains*

Comparative listening laboratory. Steve Reich: *Different Trains* (1988). Speech melody, instrumental doubling, and repeated pulse invite close comparison of contour extraction and resynthesis.

Track a spoken phrase, retune a string sample to the contour, and preserve consonants in a parallel dry path.

G.5 Sustained tone and resonance

The works in this group emphasize sustained tone and resonance. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

31. Antonio Vivaldi: *Winter, Largo*

Comparative listening laboratory. Antonio Vivaldi: *Winter, Largo* (1725). The violin melody and pizzicato accompaniment separate sustained tone from short attacks.

Stretch the solo line strongly while leaving the accompaniment nearly unchanged, then recombine them.

32. Johann Sebastian Bach: *Air on the G String*

Comparative listening laboratory. Johann Sebastian Bach: *Air on the G String* (c. 1730). Long string lines and stepwise bass motion reveal modulation and ensemble coherence.

Compare a single stereo render with stem-based processing under a shared stretch trajectory.

33. Ludwig van Beethoven: *Symphony No. 7, Allegretto*

Comparative listening laboratory. Ludwig van Beethoven: *Symphony No. 7, Allegretto* (1812). Repeated rhythm under expanding orchestral layers tests sustained resonance without losing pulse.

Freeze selected chord tails while preserving the dry rhythmic attacks in a parallel path.

34. Franz Schubert: *String Quintet in C, Adagio*

Comparative listening laboratory. Franz Schubert: *String Quintet in C, Adagio* (1828). Wide string registers and very long phrases expose beating, detuning, and vertical phase coherence.

Apply a two-times stretch with peak locking, then compare it with independent-bin propagation.

35. Frederic Chopin: *Nocturne in E-flat major, Op. 9 No. 2*

Comparative listening laboratory. Frederic Chopin: *Nocturne in E-flat major, Op. 9 No. 2* (1832). Piano resonance, ornament, and rubato provide a compact test of tail preservation.

Protect note onsets while extending only the decays, using a time-varying stretch map.

36. Richard Wagner: *Prelude to Das Rheingold*

Comparative listening laboratory. Richard Wagner: *Prelude to Das Rheingold* (1869). A single harmonic field grows from quiet fundamentals into dense orchestral resonance.

Extend the opening four times and automate window length as the orchestration becomes denser.

37. Anton Bruckner: *Symphony No. 7, Adagio*

Comparative listening laboratory. Anton Bruckner: *Symphony No. 7, Adagio* (1883). Brass chorales and string sustain test phase stability at high spectral density.

Freeze a brass chord, retune selected partial regions, and crossfade into a stretched continuation.

38. Claude Debussy: *Prelude a l'apres-midi d'un faune*

Comparative listening laboratory. Claude Debussy: *Prelude a l'apres-midi d'un faune* (1894). Solo winds and blended orchestration make timbral identity more important than strict note attacks.

Morph the flute magnitude envelope toward a string harmonic while preserving the flute phase trajectory.

39. Maurice Ravel: *Daphnis et Chloe, Lever du jour*

Comparative listening laboratory. Maurice Ravel: *Daphnis et Chloe, Lever du jour* (1912). The gradual orchestral sunrise tests long dynamic trajectories and accumulating spectral brightness.

Use centroid-following automation to open a spectral filter over an extended stretch.

40. Ralph Vaughan Williams: *Fantasia on a Theme by Thomas Tallis*

Comparative listening laboratory. Ralph Vaughan Williams: *Fantasia on a Theme by Thomas Tallis* (1910). Divided strings and antiphonal groups expose spatial coherence and slowly evolving resonance.

Process ensemble groups separately with one reference phase track and compare the stereo image.

41. Gustav Holst: *Neptune, the Mystic*

Comparative listening laboratory. Gustav Holst: *Neptune, the Mystic* (1915). Static harmony, celesta, and distant chorus make an ideal extreme-stretch texture.

Render an eight-times stretch with slow spectral filtering and a long wet-dry automation curve.

42. Olivier Messiaen: *Louange a l'Eternite de Jesus*

Comparative listening laboratory. Olivier Messiaen: *Louange a l'Eternite de Jesus* (1941). Extremely slow cello melody over repeated piano chords separates continuity from attack.

Stretch the cello with phase locking while processing piano attacks and decays on separate paths.

43. Gyorgy Ligeti: *Lux Aeterna*

Comparative listening laboratory. Gyorgy Ligeti: *Lux Aeterna* (1966). Micropolyphonic voices form a dense, slowly moving spectral cloud.

Apply a subtle spectral tilt and two-times stretch, then inspect whether internal beating remains alive.

44. Arvo Part: *Spiegel im Spiegel*

Comparative listening laboratory. Arvo Part: *Spiegel im Spiegel* (1978). Sparse piano attacks and a sustained melodic line reveal every artifact.

Compare transient reset, identity locking, and independent phase propagation at identical settings.

45. John Tavener: *The Protecting Veil*

Comparative listening laboratory. John Tavener: *The Protecting Veil* (1988). Extended cello phrases and luminous string harmony reward gradual formant and spectral-envelope motion.

Automate a modest downward formant shift across one phrase without changing its fundamental pitch.

G.6 Transient and rhythmic stress tests

The works in this group emphasize transient and rhythmic stress tests. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

46. Johann Sebastian Bach: *Prelude in C major, BWV 846*

Comparative listening laboratory. Johann Sebastian Bach: *Prelude in C major, BWV 846* (1722). Evenly spaced piano attacks make timing blur and pre-echo easy to hear.

Stretch two times using several transient thresholds and compare note-front sharpness.

47. Ludwig van Beethoven: *Piano Sonata No. 14, third movement*

Comparative listening laboratory. Ludwig van Beethoven: *Piano Sonata No. 14, third movement* (1801). Fast repeated attacks stress onset detection and frame resetting.

Slow the passage to half tempo while preserving pitch, then audit flam, smearing, and high-frequency loss.

48. Niccolò Paganini: *Caprice No. 24*

Comparative listening laboratory. Niccolò Paganini: *Caprice No. 24* (1817). Rapid articulation and register leaps reveal peak-tracking failures.

Compare short-window and multiresolution processing on the fastest variation.

49. Frederic Chopin: *Etude Op. 10 No. 4*

Comparative listening laboratory. Frederic Chopin: *Etude Op. 10 No. 4* (1830). Continuous sixteenth notes expose accumulated timing and phase errors.

Render 0.75-times and 1.5-times versions with identical loudness and compare attack definition.

50. Nikolai Rimsky-Korsakov: *Flight of the Bumblebee*

Comparative listening laboratory. Nikolai Rimsky-Korsakov: *Flight of the Bumblebee* (1900). Near-continuous fast notes are a severe test of local stationarity.

Sweep FFT size over the passage and graph transient flux against perceived clarity.

51. Igor Stravinsky: *The Rite of Spring, Augurs of Spring*

Comparative listening laboratory. Igor Stravinsky: *The Rite of Spring, Augurs of Spring* (1913). Accented orchestral chords test synchronized transient handling across many channels.

Use a shared onset map for all stems and compare it with independent per-stem detection.

52. Edgard Varese: *Ionisation*

Comparative listening laboratory. Edgard Varese: *Ionisation* (1931). Percussion timbres expose both attack smear and spectral-noise coloration.

Stretch metal, skin, and noise instruments separately using windows matched to their decay behavior.

53. John Cage: *First Construction in Metal*

Comparative listening laboratory. John Cage: *First Construction in Metal* (1939). Metallic attacks and long resonances invite independent control of onset and tail.

Split at detected transients, preserve each attack, and stretch the following decay by a larger factor.

54. Dave Brubeck Quartet: *Take Five*

Comparative listening laboratory. Dave Brubeck Quartet: *Take Five* (1959). The clear quintuple meter reveals timing drift while cymbals challenge phase coherence.

Slow the track while preserving pitch and compare cymbal texture under several transient modes.

55. James Brown: *Funky Drummer*

Comparative listening laboratory. James Brown: *Funky Drummer* (1970). The famous drum break provides tightly related kick, snare, and hi-hat transients.

Create a half-time render, then compare broadband and band-dependent transient protection.

56. Kraftwerk: *Numbers*

Comparative listening laboratory. Kraftwerk: *Numbers* (1981). Sparse electronic percussion and speech make small timing errors obvious.

Stretch the rhythm independently from the voice, then align both with checkpointed rendering.

57. Public Enemy: *Bring the Noise*

Comparative listening laboratory. Public Enemy: *Bring the Noise* (1987). Dense samples, voice, and percussion test a phase vocoder under deliberate spectral congestion.

Compare full-mix stretching against source-separated processing at the same target duration.

58. Aphex Twin: *Vordhosbn*

Comparative listening laboratory. Aphex Twin: *Vordhosbn* (2001). Rapid programmed attacks and elastic timing push conventional onset models hard.

Apply a gentle stretch, inspect the spectrogram, and tune transient sensitivity to avoid doubled attacks.

59. Squarepusher: *My Red Hot Car*

Comparative listening laboratory. Squarepusher: *My Red Hot Car* (2001). Breakbeat detail and pitched bass expose different optimal frame scales.

Use multiband processing with shorter frames above the crossover and longer frames below it.

60. Autechre: *Gantz Graf*

Comparative listening laboratory. Autechre: *Gantz Graf* (2002). Abrupt synthetic gestures reveal frame boundaries, ringing, and interpolation behavior.

Render a continuously varying stretch curve and compare linear, cubic, and smoothstep automation.

G.7 Polyphony and orchestral density

The works in this group emphasize polyphony and orchestral density. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

61. Giovanni Pierluigi da Palestrina: *Missa Papae Marcelli, Kyrie*

Comparative listening laboratory. Giovanni Pierluigi da Palestrina: *Missa Papae Marcelli, Kyrie* (1562). Closely spaced voices test vertical phase coherence without instrumental attacks.

Stretch a choral excerpt with identity locking and compare the intelligibility of inner voices.

62. Johann Sebastian Bach: *The Art of Fugue, Contrapunctus I*

Comparative listening laboratory. Johann Sebastian Bach: *The Art of Fugue, Contrapunctus I* (1740s). Independent lines provide a controlled test of polyphonic clarity.

Process each contrapuntal voice as a stem under one shared timing map, then compare a full-mix render.

63. Wolfgang Amadeus Mozart: *Symphony No. 40, first movement*

Comparative listening laboratory. Wolfgang Amadeus Mozart: *Symphony No. 40, first movement* (1788). Repeated figures and orchestral doublings reveal loss of articulation and image focus.

Apply a modest tempo change and compare peak locking with transient-reset synthesis.

64. Ludwig van Beethoven: *Symphony No. 9, finale*

Comparative listening laboratory. Ludwig van Beethoven: *Symphony No. 9, finale* (1824). Soloists, chorus, and orchestra create extreme density across the full spectrum.

Render selected sections with stem-aware settings and test loudness normalization only after recombination.

65. Hector Berlioz: *Symphonie fantastique, Marche au supplice*

Comparative listening laboratory. Hector Berlioz: *Symphonie fantastique, Marche au supplice* (1830). Orchestral attacks, rolls, and brass resonance demand mixed transient policies.

Drive transient sensitivity from spectral flux and protect low-frequency drum attacks separately.

66. Johannes Brahms: *Symphony No. 4, finale*

Comparative listening laboratory. Johannes Brahms: *Symphony No. 4, finale* (1885). Variation form supplies repeated material under changing orchestration.

Use matched excerpts to compare how one parameter set behaves as texture density changes.

67. Gustav Mahler: *Symphony No. 2, finale*

Comparative listening laboratory. Gustav Mahler: *Symphony No. 2, finale* (1894). The progression from near silence to full chorus and orchestra tests dynamic and spectral range.

Automate FFT size, wet-dry balance, and spectral smoothing according to measured density.

68. Claude Debussy: *La mer, Dialogue du vent et de la mer*

Comparative listening laboratory. Claude Debussy: *La mer, Dialogue du vent et de la mer* (1905). Layered orchestral motion tests phase coherence without a single dominant pulse.

Stretch by 1.25 times and compare frequency-dependent phase locking across low, middle, and high bands.

69. Charles Ives: *The Unanswered Question*

Comparative listening laboratory. Charles Ives: *The Unanswered Question* (1908). Three independent musical layers invite distinct temporal transformations.

Stretch the string background, leave the trumpet near original time, and compress the woodwind interruptions.

70. Igor Stravinsky: *The Firebird, Infernal Dance*

Comparative listening laboratory. Igor Stravinsky: *The Firebird, Infernal Dance* (1910). Sharp attacks and bright orchestration reveal pre-echo and high-frequency smearing.

Compare overlap factors while holding window length fixed and log transient metrics.

71. Bela Bartok: *Music for Strings, Percussion and Celesta, third movement*

Comparative listening laboratory. Bela Bartok: *Music for Strings, Percussion and Celesta, third movement* (1936). Glissandi, tremolo, percussion, and spatial string groups cover many phase-vocoder edge cases.

Assign different analysis presets to tremolo strings, pitched percussion, and noise-like gestures.

72. Benjamin Britten: *The Young Person's Guide to the Orchestra*

Comparative listening laboratory. Benjamin Britten: *The Young Person's Guide to the Orchestra* (1945). Successive instrumental families make timbre-dependent artifacts easy to compare.

Run one preset across each variation and assemble a window-by-instrument evaluation table.

73. Iannis Xenakis: *Metastaseis*

Comparative listening laboratory. Iannis Xenakis: *Metastaseis* (1954). Massed glissandi challenge peak identity and frequency-trajectory tracking.

Slow the central texture and compare independent bins with peak-tracked partial trajectories.

74. Gyorgy Ligeti: *Atmospheres*

Comparative listening laboratory. Gyorgy Ligeti: *Atmospheres* (1961). Dense micropolyphony tests whether spectral processing preserves internal motion rather than flattening it.

Freeze successive frames and crossfade them to study the boundary between texture and stasis.

75. John Adams: *Short Ride in a Fast Machine*

Comparative listening laboratory. John Adams: *Short Ride in a Fast Machine* (1986). A steady pulse under bright orchestration exposes timing drift immediately.

Apply a curved tempo map while using a shared transient clock for the entire ensemble.

G.8 Electronic, studio, and ambient listening

The works in this group emphasize electronic, studio, and ambient listening. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

76. Karlheinz Stockhausen: *Gesang der Junglinge*

Comparative listening laboratory. Karlheinz Stockhausen: *Gesang der Junglinge* (1956). Electronic tones and a boy's voice establish a foundational comparison between synthesis and recorded sound.

Morph magnitude envelopes between a sung vowel and a synthetic oscillator bank.

77. Karlheinz Stockhausen: *Kontakte*

Comparative listening laboratory. Karlheinz Stockhausen: *Kontakte* (1958-1960). Continuities between pitch, pulse, and timbre suggest phase-vocoder transformations across temporal scale.

Stretch short impulses until their spectra become sustained tones, then reverse the process.

78. Delia Derbyshire: *Doctor Who Theme*

Comparative listening laboratory. Delia Derbyshire: *Doctor Who Theme* (1963). Layered electronic lines and tape-shaped attacks offer clear material for pitch and duration experiments.

Shift one melodic layer by a fifth without changing duration and compare formant-preserving and unconstrained modes.

79. The Beatles: *Tomorrow Never Knows*

Comparative listening laboratory. The Beatles: *Tomorrow Never Knows* (1966). Tape loops, drone, and processed voice provide contrasting repetitive sources.

Create phase-aligned loop layers at several stretch ratios and automate their spectral masks.

80. Pink Floyd: *On the Run*

Comparative listening laboratory. Pink Floyd: *On the Run* (1973). Sequenced motion and noise effects reveal how stretching changes perceived speed and space.

Apply a nonlinear time map that slows the middle while preserving the beginning and ending durations.

81. Brian Eno: *An Ending (Ascent)*

Comparative listening laboratory. Brian Eno: *An Ending (Ascent)* (1983). Slow harmonic motion and diffuse attacks make spectral breathing easy to study.

Extend the piece-like source four times and automate room size and wet-dry mix over the render.

82. Laurie Anderson: *O Superman*

Comparative listening laboratory. Laurie Anderson: *O Superman* (1981). Processed voice and a repeated pulse separate formant character from rhythmic continuity.

Stretch the voice while keeping the pulse fixed, then retune the vocal layer with preserved formants.

83. Kate Bush: *Running Up That Hill*

Comparative listening laboratory. Kate Bush: *Running Up That Hill* (1985). Layered drums, synthesizers, and expressive voice expose compromises in full-mix processing.

Compare a full-mix stretch with a stem-separated render and document vocal consonants and drum transients.

84. Cocteau Twins: *Lorelei*

Comparative listening laboratory. Cocteau Twins: *Lorelei* (1984). Dense reverberant voice and guitar create a complex field of overlapping partials.

Freeze and retune a reverberant tail while preserving a parallel dry attack layer.

85. My Bloody Valentine: *Only Shallow*

Comparative listening laboratory. My Bloody Valentine: *Only Shallow* (1991). Distorted guitars and diffuse vocals test processing on intentionally blurred spectra.

Use spectral filtering and small pitch shifts to separate layers without reducing their textural density.

86. The Orb: *Little Fluffy Clouds*

Comparative listening laboratory. The Orb: *Little Fluffy Clouds* (1990). Speech, ambience, and repeating electronic material invite independent time-scale treatment.

Stretch the environmental bed, keep speech near original timing, and interpolate between the two clocks.

87. Boards of Canada: *Roygbiv*

Comparative listening laboratory. Boards of Canada: *Roygbiv* (1998). Soft attacks and unstable analog pitch make excessive phase correction easy to hear.

Apply gentle time stretching while preserving low-frequency modulation and avoiding aggressive pitch stabilization.

88. Bjork: *All Is Full of Love*

Comparative listening laboratory. Bjork: *All Is Full of Love* (1997). Layered voices and electronic resonance form a rich laboratory for formant-aware processing.

Create a three-voice spectral chorus from one line using independent pitch and formant trajectories.

89. Radiohead: *Everything in Its Right Place*

Comparative listening laboratory. Radiohead: *Everything in Its Right Place* (2000). Looped keyboard harmony and fragmented voice demonstrate repetition at multiple scales.

Freeze the keyboard bed, granularly reshape vocal timing, and join both with a common automation curve.

90. Oneohtrix Point Never: *Replica*

Comparative listening laboratory. Oneohtrix Point Never: *Replica* (2011). Short sampled fragments become harmonically and emotionally legible through repetition and transformation.

Stretch one fragment to a sustained pad, retain another as a transient marker, and morph between them.

G.9 Extreme duration and temporal form

The works in this group emphasize extreme duration and temporal form. Read each note before listening, then use the proposed experiment as a controlled comparison rather than a recipe that must produce one correct sound.

91. Johann Pachelbel: *Canon in D*

Comparative listening laboratory. Johann Pachelbel: *Canon in D* (c. 1680). The repeating bass and accumulating canonic voices make long-form timing relationships transparent.

Assign progressively larger stretch factors to successive entries while preserving one common harmonic grid.

92. Ludwig van Beethoven: *Symphony No. 5, opening*

Comparative listening laboratory. Ludwig van Beethoven: *Symphony No. 5, opening* (1808). The universally recognizable motif makes even radical temporal transformation perceptually traceable.

Expand the four-note gesture one hundred times, protect the initial attacks, and study the resulting internal spectra.

93. Richard Wagner: *Prelude to Tristan und Isolde*

Comparative listening laboratory. Richard Wagner: *Prelude to Tristan und Isolde* (1859). Suspended harmonic resolution and long crescendos suit continuous nonlinear stretching.

Draw a stretch map that slows harmonic tension points and accelerates connective material.

94. Maurice Ravel: *Bolero*

Comparative listening laboratory. Maurice Ravel: *Bolero* (1928). A nearly fixed tempo and melody under continuous orchestral growth isolate changes in timbre and scale.

Hold duration constant while morphing spectral envelopes among successive instrumental statements.

95. Samuel Barber: *Adagio for Strings*

Comparative listening laboratory. Samuel Barber: *Adagio for Strings* (1936). Long arches and gradual registral expansion reveal drift over extended processing.

Apply a stretch factor that rises toward the climax and returns afterward, with checkpointed rendering.

96. John Cage: *Organ2/ASLSP*

Comparative listening laboratory. John Cage: *Organ2/ASLSP* (1987). The premise of performing as slowly as possible invites a direct investigation of computationally extreme duration.

Render a short organ chord at escalating factors, then compare memory, checkpoint, and streaming strategies.

97. Steve Reich: *Piano Phase*

Comparative listening laboratory. Steve Reich: *Piano Phase* (1967). Gradual process depends on tiny timing differences that phase-vocoder automation can imitate or destroy.

Create two copies with slightly different time maps and measure their evolving alignment.

98. Alvin Lucier: *I Am Sitting in a Room*

Comparative listening laboratory. Alvin Lucier: *I Am Sitting in a Room* (1969). Iterated room coloration demonstrates how a transfer function becomes musical form.

Estimate the response of one generation and apply it iteratively while controlling normalization and decay.

99. William Basinski: *The Disintegration Loops 1.1*

Comparative listening laboratory. William Basinski: *The Disintegration Loops 1.1* (2002). Slow material change across repetition suggests automation beyond a fixed stretch factor.

Build a loop whose bandwidth, noise floor, and stretch factor evolve slightly on every pass.

100. Max Richter: *Sleep*

Comparative listening laboratory. Max Richter: *Sleep* (2015). An eight-hour form supplies a practical model for streaming, checkpoints, and imperceptibly slow parameter motion.

Render a long source in resumable chunks while automating room size, wet-dry mix, and spectral brightness.

G.10 Comparative listening worksheet

The catalog becomes more useful when impressions are recorded consistently. For each experiment, note the source and edition, time range, rights status, pvx command, window and hop, stretch or pitch trajectory, phase mode, transient policy, normalization method, and monitoring level. Then rate attack definition, tonal focus, formant credibility, ambience continuity, stereo stability, noise coloration, and overall musical usefulness. A failed transformation is still useful evidence when its settings and source are documented.

The most revealing comparisons use matched loudness and short randomized presentation. Keep the dry source, a conservative transform, and an intentionally extreme transform. Describe artifacts in operational terms such as doubled attack, pre-echo, diffuse partial, unstable center image, shifted vowel, or metallic tail. Those descriptions lead back to parameters more reliably than a single global quality score.

APPENDIX H

Transfer-Function and Automation Graph Atlas

This appendix restores the project graphs at a scale suitable for print. Each graph is introduced by a short statement of the mathematical or control relationship it illustrates. The figures are original pvx project assets.

H.1 Transfer functions

The transfer-function group covers pitch ratios, dynamics, soft clipping, spectral morphing, masking, and phase mixing. These graphs connect command-line values to the curves applied inside a render.

$$r_s = 2^{s/12}, \quad r_c = 2^{c/1200}$$

where r_s is the pitch ratio for s semitones and r_c is the pitch ratio for c cents.

pitch ratio from semitones

Semitones map exponentially to ratio: twelve semitones doubles frequency, while minus twelve halves it.

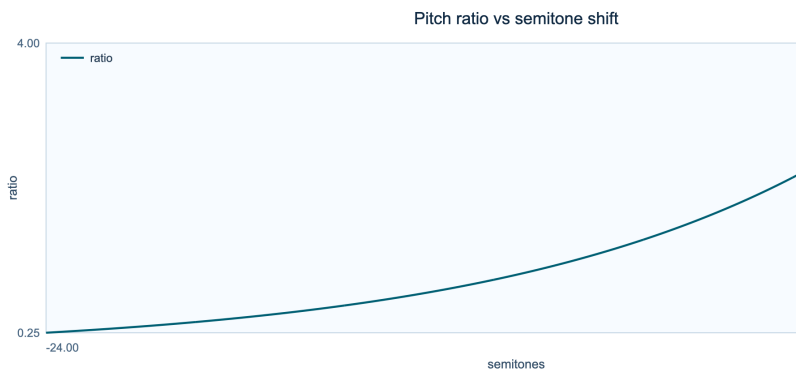


Figure H.1: Pitch ratio from semitones.

pitch ratio from cents

The cents scale uses the same exponential law at finer resolution; 1200 cents is one octave.

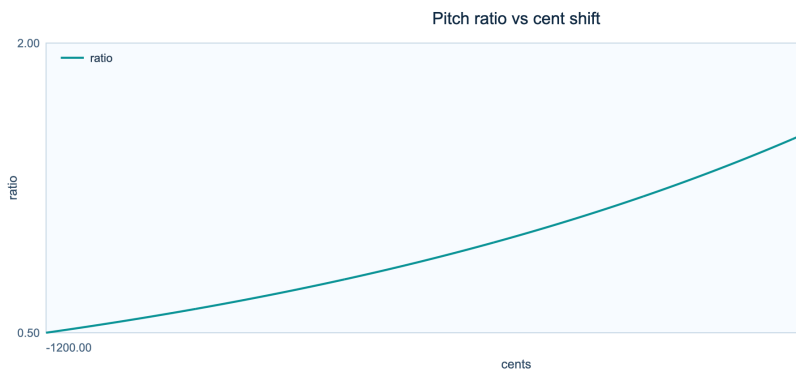


Figure H.2: Pitch ratio from cents.

dynamics transfer curves

The identity, compressor, expander, and limiter curves reveal where gain departs from a one-to-one response.

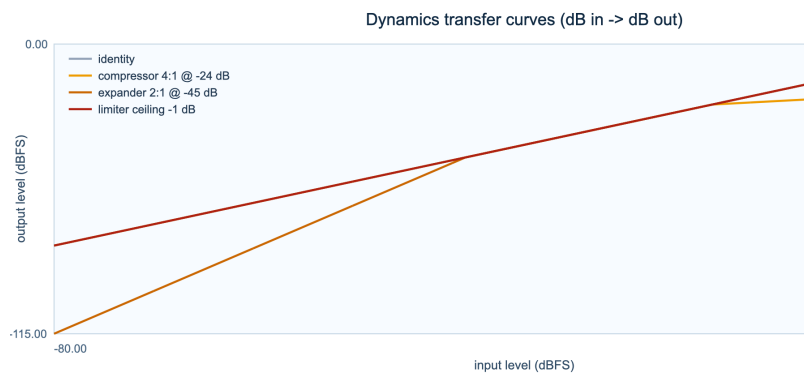


Figure H.3: Dynamics transfer curves.

soft-clip transfer functions

These curves compare how the available soft clippers bend peaks before they reach a hard limit.

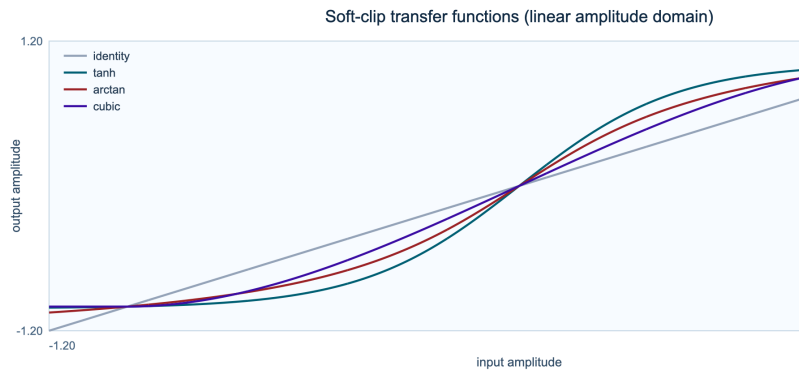


Figure H.4: Soft-clip transfer functions.

morph blend magnitude curves

The blend rules agree at the endpoints but take distinct paths through intermediate magnitudes.

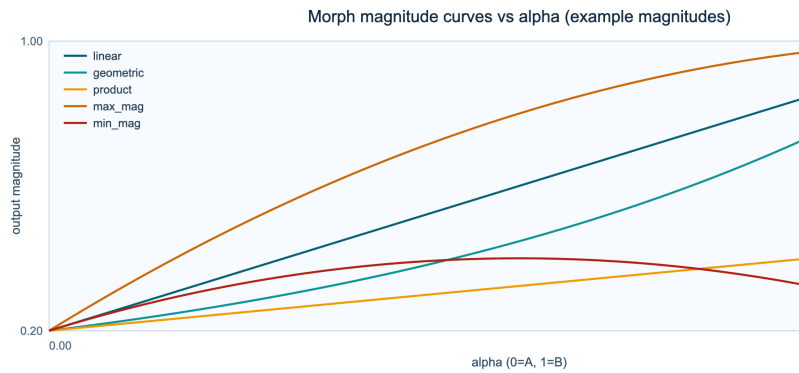


Figure H.5: Morph blend magnitude curves.

mask exponent response

The exponent changes how sharply a normalized mask favors strong bins over weak ones.

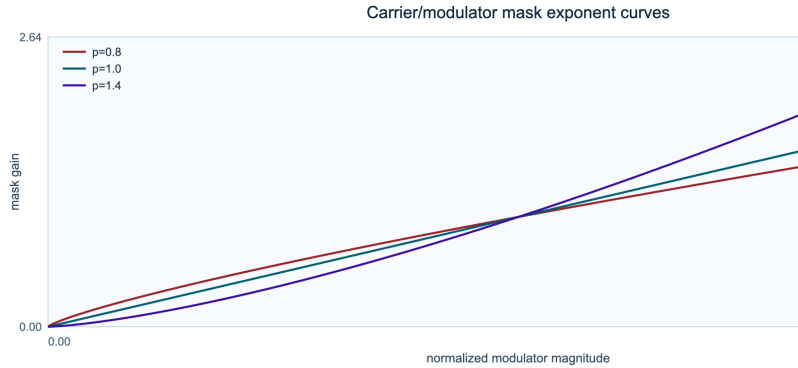


Figure H.6: Mask exponent response.

phase-mix angle response

Complex-vector interpolation does not produce a linear angle sweep; the curve shows the resulting phase path.

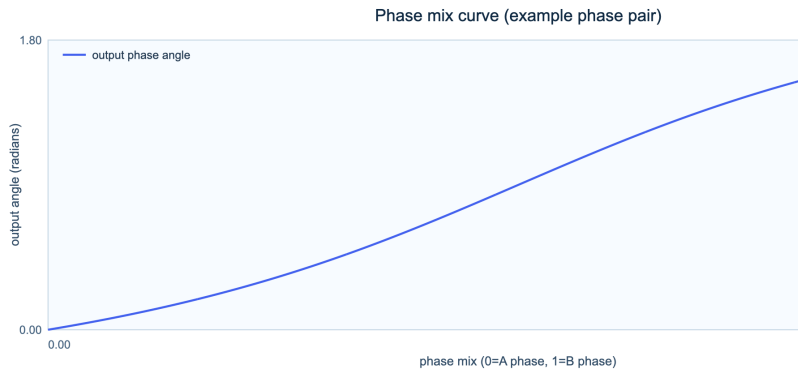


Figure H.7: Phase-mix angle response.

H.2 Automation and interpolation

The interpolation group shows how pvx connects control points across render time. An interpolation rule is not merely a visual convenience. Its slope and curvature determine how quickly an audible parameter moves and whether the motion has discontinuous derivatives.

$$u_{\text{linear}}(t) = (1 - \lambda)u_i + \lambda u_{i+1}$$

where u_i and u_{i+1} are adjacent control values, t lies between their timestamps, and λ is normalized segment position in $[0, 1]$.

$$u_{\text{smooth}}(t) = (1 - h(\lambda))u_i + h(\lambda)u_{i+1}, \quad h(\lambda) = 3\lambda^2 - 2\lambda^3$$

where $h(\lambda)$ is the smoothstep easing function, λ is normalized segment position, and u_i, u_{i+1} are adjacent control values.

sample and hold

Sample-and-hold keeps each value unchanged until the next control point, where it jumps immediately.

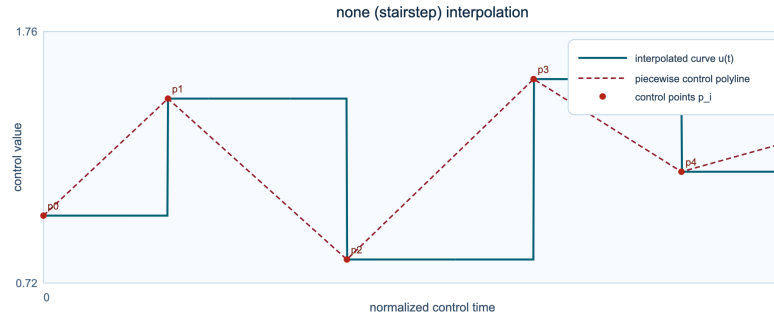


Figure H.8: Sample and hold interpolation.

nearest neighbour

Nearest-neighbour interpolation switches at each segment midpoint rather than at the authored point.

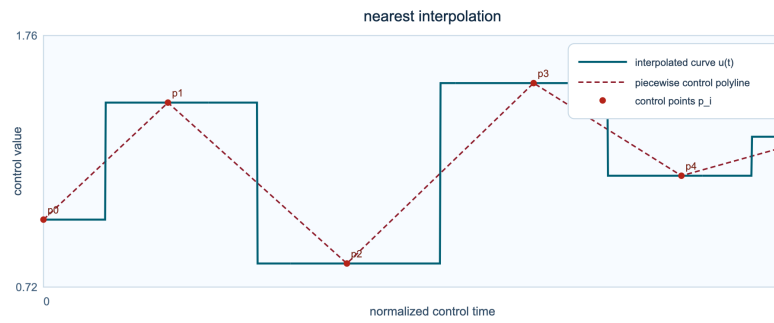


Figure H.9: Nearest neighbour interpolation.

linear

Linear interpolation moves at constant speed and introduces a corner wherever the segment slope changes.

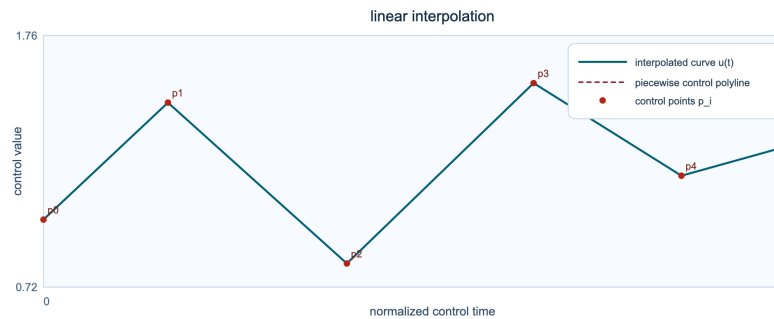


Figure H.10: Linear interpolation.

cubic

The cubic curve smooths the path through the control points, with curvature determined by neighboring values.

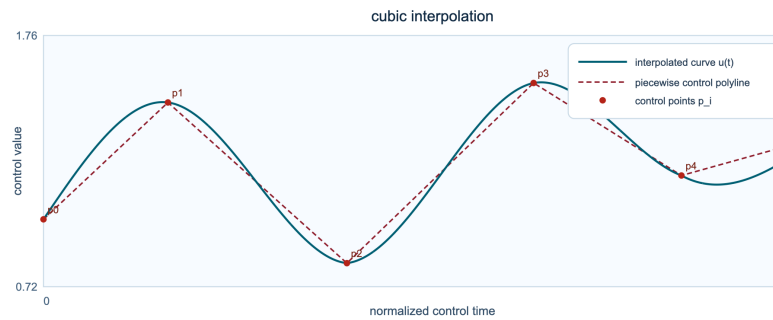


Figure H.11: Cubic interpolation.

exponential

Exponential interpolation changes by proportion rather than by a fixed increment, which suits ratio-like controls.

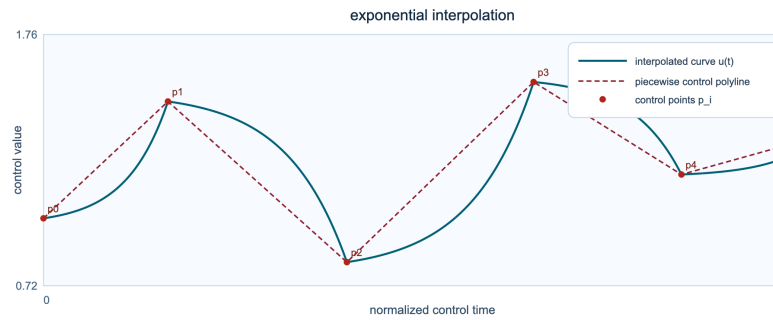


Figure H.12: Exponential interpolation.

smoothstep S-curve

Smoothstep reaches each endpoint with zero slope, softening the joins between segments.

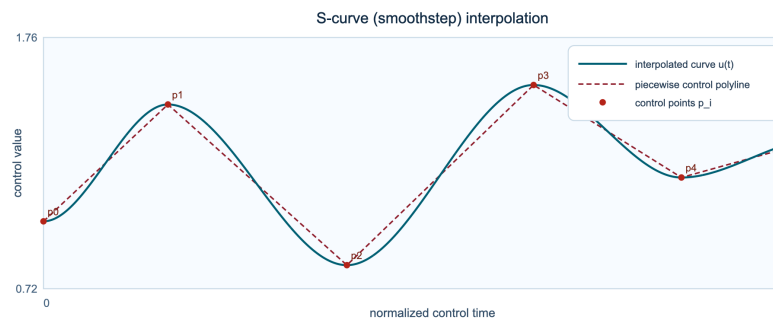


Figure H.13: Smoothstep s-curve interpolation.

smootherstep

Smootherstep also flattens the second derivative at the endpoints, producing gentler acceleration.

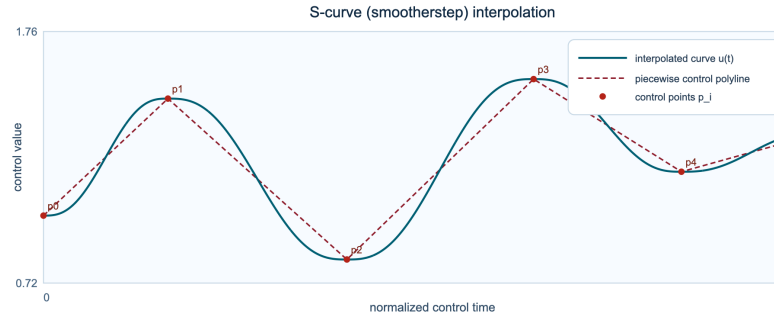


Figure H.14: Smootherstep interpolation.

polynomial order 1

First-order polynomial interpolation is the linear reference case.

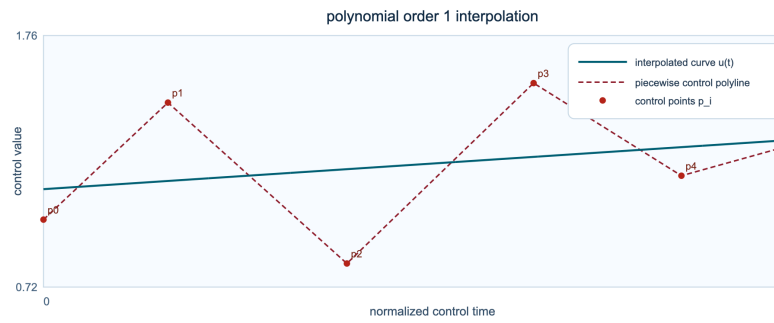


Figure H.15: Polynomial order 1 interpolation.

polynomial order 2

The quadratic case biases motion toward one side of the segment and introduces visible curvature.

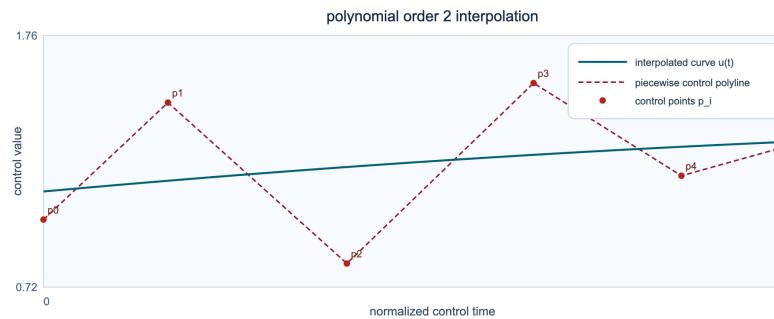


Figure H.16: Polynomial order 2 interpolation.

polynomial order 3

The cubic polynomial provides a stronger ease while remaining comparatively compact.

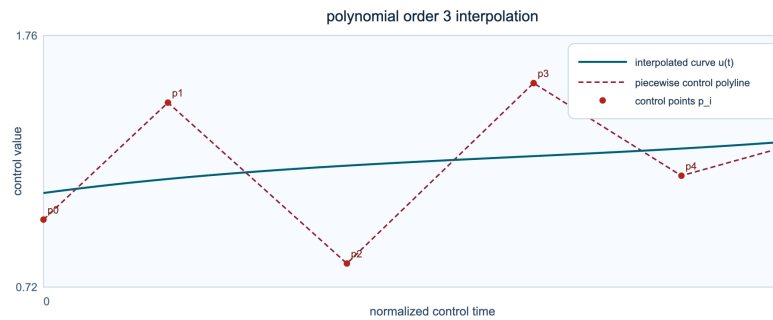


Figure H.17: Polynomial order 3 interpolation.

polynomial order 5

The fifth-order curve spends more time near the endpoints and moves fastest through the middle.

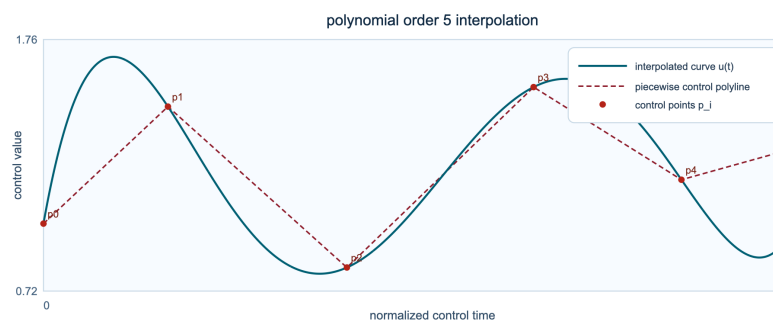


Figure H.18: Polynomial order 5 interpolation.

Glossary

Aliasing. The appearance of false frequencies when a signal contains energy above the Nyquist frequency or when spectral modification moves energy beyond the representable band. Anti-alias filtering and an adequate sample rate reduce it.

Amplitude. A measure of signal strength. In a waveform it describes sample displacement; in a spectrum it describes the strength of a bin or partial. Amplitude may be expressed linearly or in decibels.

Analysis frame. A short, usually overlapping segment selected from the input signal for windowing and transformation. Its length limits the balance between temporal detail and frequency discrimination.

Analysis hop. The number of input samples between the starts of consecutive analysis frames. A smaller hop produces greater overlap and finer control sampling at the cost of more computation.

Analysis-synthesis system. A process that decomposes sound into a representation, modifies or stores that representation, and reconstructs audio. A phase vocoder uses short-time complex spectra as its central representation.

Artifact. An audible feature introduced by processing rather than present in the source. Typical phase-vocoder artifacts include attack smear, pre-echo, phasiness, metallic decay, unstable pitch, and stereo-image drift.

Attack. The beginning of a sound event, often marked by a rapid rise in energy or spectral flux. Attacks require special treatment because the stationary-sinusoid assumption is weakest there.

Automation curve. A time-indexed sequence controlling a parameter such as stretch, pitch, mix, filter gain, or room size. Interpolation determines the values between authored control points.

Bandlimit. To restrict a signal to frequencies below a chosen upper boundary. Bandlimiting before down-sampling or large upward spectral shifts helps prevent aliasing.

Bandwidth. The frequency range occupied or passed by a signal, filter, spectral peak, or analysis band. Bandwidth can be stated in hertz, bins, octaves, or perceptual units.

Bin. One discrete frequency sample of a Fourier spectrum. Bin index is not always identical to sinusoidal frequency because a partial may lie between bin centers and spread into neighboring bins.

Bin-center frequency. The nominal frequency represented by a DFT bin. For bin index k , sample rate F_s , and transform length N , the center frequency is k times F_s divided by N .

Bit depth. The number of bits used to represent each encoded sample in a fixed-point audio format. Greater bit depth lowers the quantization-noise floor but does not repair clipping or processing artifacts.

Block processing. Audio computation performed on groups of samples rather than one sample at a time. Spectral algorithms are naturally block based because every transform consumes a complete frame.

Broadband. Containing energy across a wide frequency range. Impulses, consonants, cymbals, and noise are broadband sources and often require shorter windows or transient-aware processing.

Buffer. A region of memory holding samples, frames, or encoded bytes while they are read, processed, or written. Buffer size affects throughput, latency, and streaming behavior.

Cepstrum. A representation obtained by transforming a log-magnitude spectrum. Slow cepstral components describe the spectral envelope, while faster components can describe harmonic spacing and pitch-related structure.

Channel. One component of a multichannel audio signal, such as left or right in stereo. Independent processing can disturb interchannel phase relationships, so reference locking may be preferable.

Checkpoint. A saved rendering state that permits a long process to resume without starting from the beginning. A useful checkpoint records timing, phase state, automation position, configuration, and output progress.

Chroma. A twelve-class pitch representation that folds pitches separated by octaves into the same class. Chroma is useful for harmony analysis but discards register and much timbral detail.

Clipping. Nonlinear truncation when signal magnitude exceeds an allowed range. Hard clipping creates strong high-frequency products; soft clipping rounds the transition but still changes timbre.

Coherent gain. The average value of a window, interpreted as the gain applied to a sinusoid exactly centered on a transform bin. It is used when comparing or calibrating windowed spectral amplitudes.

COLA. Constant overlap-add. A window and hop combination satisfies COLA when shifted window copies sum to a constant, permitting level-stable reconstruction under the corresponding overlap-add scheme.

Complex number. A value with real and imaginary components. Fourier bins are complex because they jointly encode magnitude and phase, or equivalently the cosine and sine contributions at a frequency.

Complex spectrum. The set of complex Fourier coefficients for a frame. Retaining both magnitude and phase is necessary for direct inverse transformation and coherent time evolution.

Conjugate symmetry. The mirrored relationship in the DFT of a real signal. Positive- and negative-frequency bins are complex conjugates, allowing a real-input transform to store only the nonnegative half.

Convolution. An operation that combines a signal with an impulse response. In audio it models filtering and reverberation; in the frequency domain it corresponds to multiplication of spectra.

Cross-synthesis. A transformation that combines properties of two sounds, commonly the magnitude or envelope of one with the phase, excitation, or timing of another. The result can interpolate between recognizable identities.

Crossfade. A transition formed by decreasing one signal while increasing another. Equal-power shapes are often used when the sources are weakly correlated; complementary linear fades can suit matched signals.

dBFS. Decibels relative to digital full scale. Zero dBFS is the maximum representable peak in a normalized digital system; ordinary signals use negative values below that ceiling.

Decay. The reduction of energy after an attack or excitation. Preserving the texture and modulation of a decay is a different problem from preserving its initial transient.

Deterministic render. A process that produces identical output for identical input, options, software version, and seed. Determinism is essential for regression tests, resumable jobs, and reproducible datasets.

DFT. Discrete Fourier transform. It maps a finite sequence of N samples to N complex frequency coefficients. The FFT is an efficient family of algorithms for computing it.

Dry signal. The unprocessed source in a wet-dry mixture. A dry path can restore attacks, intelligibility, or localization that would otherwise be softened by an extreme spectral effect.

Dynamic range. The span between low and high signal levels. It may describe a recording, a processing stage, or a format, and should not be confused with average loudness.

Energy normalization. Gain compensation intended to keep a chosen energy measure consistent after transformation. Peak, RMS, and perceptual-loudness normalization solve different problems and can produce different balances.

Envelope. A slowly varying contour describing amplitude, spectral shape, or another feature. Envelopes suppress fine oscillation so larger gestures can be measured or transferred.

Envelope follower. A detector that tracks changing signal level using attack and release smoothing. Its time constants determine how strongly it reacts to transients versus sustained energy.

Equalization. Frequency-dependent gain adjustment. In a spectral processor, equalization may be fixed, automated, measured from a reference, or applied separately to time-frequency regions.

ERB. Equivalent rectangular bandwidth. A psychoacoustic scale approximating the frequency selectivity of human hearing. ERB-spaced bands are often more perceptually meaningful than equal-width hertz bands.

Exponential interpolation. Interpolation whose values change by a constant ratio rather than a constant difference. It suits positive quantities such as frequency, pitch ratio, decay time, and some gain controls.

FFT. Fast Fourier transform. A computational method that evaluates the DFT efficiently. FFT size affects frequency sampling, frame duration, memory use, and computational cost.

FFT size. The transform length N . It may equal the window length or exceed it through zero padding. A larger size samples the spectrum more densely but does not automatically add true resolution.

File subtype. The encoded sample representation inside an audio container, such as PCM 16-bit, PCM 24-bit, or floating point. Container extension alone may not identify the subtype.

Filter. A process that changes a signal according to frequency. Filters may be described by magnitude, phase, impulse response, poles and zeros, or a direct time-frequency mask.

Floor. A lower limit imposed on a value. Magnitude floors prevent logarithms and divisions from becoming singular, but a floor set too high can raise noise or flatten weak detail.

Formant. A broad resonance region that shapes the identity of a voice or instrument. Formants are related to the spectral envelope rather than to the fundamental pitch alone.

Formant preservation. A pitch-shifting strategy that holds or separately transforms the spectral envelope while moving harmonic content. It helps a shifted voice retain vowel identity and apparent vocal size.

Frame. A finite slice of samples or spectral data processed as one unit. Frames overlap so that local analysis can be joined into a continuous output.

Frame rate. The number of analysis or control frames per second. It is the sample rate divided by hop size for a uniformly sampled frame sequence.

Frequency resolution. The ability to distinguish nearby frequencies. It depends primarily on effective window duration and shape, not merely on the number of zero-padded FFT samples.

Fundamental frequency. The repetition frequency associated with perceived pitch in a periodic or nearly periodic sound. It is often written f_0 and may be absent as a literal spectral peak.

Gain. A multiplier applied to amplitude. Linear gain 1 leaves level unchanged, while decibel gain expresses the same scaling logarithmically.

Griffin-Lim algorithm. An iterative method for finding a signal whose STFT magnitude approximates a target magnitude when phase is unavailable or inconsistent. It is reconstruction, not ordinary phase propagation.

Group delay. The frequency-dependent delay derived from phase slope. Irregular group delay can smear transient structure even when a magnitude response appears unchanged.

Harmonic. A sinusoidal component near an integer multiple of a fundamental frequency. Real instruments depart from exact harmonicity through stiffness, modulation, noise, and resonant filtering.

Harmonic-percussive separation. A decomposition that distinguishes spectrally persistent components from temporally impulsive components. Separate paths can use long windows for harmonic material and short windows for attacks.

Hertz. Cycles per second, the physical unit of frequency. Hertz spacing is linear, while musical pitch perception is approximately logarithmic.

Hop ratio. A hop expressed relative to window or FFT length. It provides a resolution-independent way to describe overlap, such as one quarter of a window.

Hop size. The sample distance between adjacent frame starts. Analysis and synthesis hops may differ; their ratio is central to basic phase-vocoder time scaling.

Identity phase locking. A method associated with Laroche and Dolson that propagates the phase of prominent peaks and preserves nearby bins' relative phase relationships. It improves vertical coherence for tonal material.

Instantaneous frequency. A local frequency estimate derived from phase change over time after removing the phase advance expected at a bin center. It refines partial trajectories beyond discrete bin locations.

Interpolation. The estimation of values between known control points or samples. Linear, cubic, exponential, and eased methods differ in continuity, overshoot, and perceived motion.

Inverse FFT. The transform that converts complex frequency coefficients back into a time-domain frame. A complete synthesis also requires windowing, overlap, and normalization.

Kaiser beta. The shape parameter of a Kaiser window. Increasing beta generally lowers sidelobes while widening the main lobe, allowing a continuous tradeoff between leakage and frequency separation.

Keyframe. An authored control point assigning a parameter value at a specified time. The interpolation rule and time coordinate system determine the path between keyframes.

Latency. The delay between input and corresponding output. Frame accumulation, lookahead, buffering, transforms, and device I/O all contribute to end-to-end latency.

Leakage. The spreading of a component's energy into neighboring spectral bins because a finite frame does not contain an integer number of cycles or uses a nonideal window.

LFO. Low-frequency oscillator. A slowly varying periodic control source used to modulate pitch, gain, filtering, position, or other parameters.

Linear phase. A phase response proportional to frequency, corresponding to a constant delay. Symmetric finite impulse response filters can provide linear phase but may introduce pre-ringing.

Loudness. The perceptual strength of sound, influenced by level, spectrum, duration, and context. Loudness normalization is more perceptually oriented than peak matching.

Magnitude. The nonnegative length of a complex spectral coefficient. Magnitude describes component strength but omits the phase information needed for exact waveform reconstruction.

Magnitude spectrum. The set of magnitudes across Fourier bins. It reveals spectral distribution while discarding the time alignment encoded by phase.

Main lobe. The central lobe of a window's frequency response. Its width controls how closely spaced sinusoids can be distinguished in a windowed analysis.

Mask. A time-frequency gain field applied to spectral data. Masks can isolate, attenuate, blend, denoise, or route components and may be binary or continuously valued.

Masking. A perceptual interaction in which one sound reduces the audibility of another nearby in frequency or time. It also informally describes multiplication by a spectral mask.

Metadata. Information associated with audio but not itself waveform samples, such as sample rate, channel layout, markers, tags, provenance, processing settings, and checksums.

Mid-side. A stereo representation using sum and difference channels. Mid carries shared center information; side carries channel difference and controls apparent width.

Mono. Audio containing one channel. Mono processing avoids interchannel coherence problems but does not remove phase-coherence issues within the spectrum.

Multichannel. Audio containing more than one channel, including stereo, surround, and microphone arrays. Transform consistency across channels is important for stable localization.

Multiresolution analysis. An approach using different time-frequency resolutions for different bands or signal conditions. Low frequencies often benefit from longer windows while transients benefit from shorter ones.

Noise floor. The background level below which desired details become masked or numerically unreliable. Processing can raise, color, modulate, or expose the noise floor.

Nonstationary. Changing in statistical or spectral character over the observation interval. Speech, music, and transients are nonstationary, which is why frame duration must remain locally appropriate.

Normalized frequency. Frequency expressed relative to sample rate, Nyquist, bin count, or another reference. The convention must be stated because the same numerical value can otherwise be ambiguous.

Nyquist frequency. Half the sample rate and the highest frequency representable without aliasing in a conventionally sampled real signal. Energy crossing it must be filtered or otherwise managed.

Onset. The perceptual beginning of a note or event. Onset detectors often combine energy change, spectral flux, phase behavior, and frequency-band evidence.

Overlap. The shared samples between adjacent frames. Greater overlap increases frame rate and can improve reconstruction or automation smoothness, while increasing computation.

Overlap-add. A synthesis procedure that places reconstructed frames at successive hop positions and sums their overlaps. Window and normalization choices determine whether level remains stable.

Oversampling. Processing at a sample rate higher than the final delivery rate. Oversampling creates transition space for nonlinear or frequency-shifting operations before filtered downsampling.

Partial. One locally coherent sinusoidal component of a sound. Partial can be harmonic, inharmonic, stable, gliding, or amplitude modulated.

Peak bin. A bin whose magnitude is locally greater than its neighbors. Spectral-peak interpolation estimates a more accurate frequency and amplitude between discrete bins.

Peak locking. A family of methods that groups bins around spectral peaks and coordinates their phase evolution. It reduces diffuse, chorus-like coloration in tonal signals.

Phase. The angular position of a periodic component. In a complex spectrum, phase determines the alignment of each sinusoidal contribution within and across frames.

Phase coherence. Consistency among phase relationships. Horizontal coherence concerns evolution across time; vertical coherence concerns relationships among bins within a frame; spatial coherence concerns channels.

Phase increment. The phase advance between frames. Comparing the measured increment with the expected bin-center advance yields an instantaneous-frequency estimate.

Phase propagation. The process of advancing synthesis phase from one frame to the next according to estimated frequencies and the synthesis hop.

Phase reset. The replacement or reinitialization of propagated phase, often near an onset. It can sharpen attacks but may cause discontinuity if applied too broadly.

Phase unwrapping. The selection of equivalent phase values differing by integer multiples of two π so that a phase trajectory changes continuously enough to interpret.

Phase vocoder. An STFT analysis-synthesis method that estimates and propagates spectral phase so duration, pitch, and spectral structure can be modified with greater continuity than frame repetition alone.

Phasiness. A diffuse, hollow, chorus-like artifact commonly associated with weak vertical phase coherence. It is especially noticeable on voices, solo instruments, and reverberant tonal material.

Pitch ratio. The multiplicative relationship between output and input pitch. One octave upward is ratio 2, one octave downward is ratio one half, and twelve-tone semitones use powers of two.

Pitch shifting. Changing perceived pitch while attempting to preserve duration. Phase-vocoder methods often combine spectral modification or time scaling with resampling and optional formant correction.

Pre-echo. Energy appearing before a transient because a frame or zero-phase process spreads future event energy backward in time. Long symmetric windows can make it conspicuous.

PVXRF. The pvx reusable response artifact format. It stores a measured or authored response together with schema, provenance, and interpretation needed for later spectral processing.

Quantization. Mapping a continuous or high-precision value to one of finitely many levels. Audio quantization produces an error whose audibility depends on bit depth, signal level, and dither.

Real-time factor. Processing time divided by audio duration. A factor below one is faster than real time; a factor above one takes longer than the rendered program.

Reconstruction. The conversion of an analysis representation back into audio. Perfect reconstruction is a mathematical property under stated conditions, not a guarantee that a modified representation sounds natural.

Reference locking. Using one channel, stem, or feature trajectory as the timing or phase reference for related signals. It can preserve stereo location and multichannel coherence.

Resampling. Changing sample timing through interpolation and filtering. Resampling changes duration and pitch together unless paired with a separate time-scale operation.

Resonance. A frequency region that responds more strongly or decays more slowly because of an acoustic, mechanical, or filter mode. Resonances shape formants, rooms, instruments, and effects.

RMS. Root mean square, an energy-related level measure computed from squared samples. It is more representative of sustained power than a peak measurement but less perceptual than integrated loudness.

Room size. An acoustic or model parameter influencing reflection density, modal spacing, and decay behavior. Automating room size may require smoothing to avoid pitch-like discontinuities in the reverberant field.

Sample. One discrete amplitude measurement in a digital waveform. A sample has no duration by itself; its time spacing is set by the sample rate.

Sample rate. The number of samples per second. It determines the Nyquist frequency and converts sample counts, frame lengths, and hops into physical time.

Scalloping loss. The reduction in measured magnitude when a sinusoid falls between bin centers rather than on one. Window shape determines the worst-case loss.

Side lobe. A secondary lobe in a window's frequency response. High sidelobes allow strong components to leak into distant bins; suppressing them generally widens the main lobe.

Sinusoid. A periodic waveform defined by amplitude, frequency, and phase. Phase-vocoder analysis treats many locally stable sounds as sums of slowly changing sinusoids plus residual components.

Smoothing. Reduction of rapid variation across time, frequency, or both. Smoothing can stabilize controls and masks but can also erase attacks, modulation, or narrow spectral detail.

Spectral centroid. The magnitude-weighted center of a spectrum. It often correlates with brightness but can move because of filtering, noise, orchestration, or pitch.

Spectral envelope. A smooth curve following broad spectral shape while ignoring individual harmonic peaks. It describes resonant coloration and is central to formant-aware transformation.

Spectral flux. A frame-to-frame measure of spectral change, often computed from positive magnitude differences. It is widely used for onset and transient detection.

Spectral freeze. A process that holds or evolves spectral data from a selected time while synthesis continues. Static magnitudes with coherent moving phase can create sustained, pitched textures.

Spectral leakage. Energy from one component appearing in adjacent or distant bins because of finite framing and window response. Leakage is controlled rather than completely eliminated.

Spectrogram. A visual time-frequency representation built from successive spectra. Color or intensity usually shows magnitude, while phase is commonly omitted.

Stereo. Two-channel audio intended to create a spatial image. Independent spectral modification can disturb center stability, width, and localization cues.

STFT. Short-time Fourier transform. It applies a windowed Fourier transform at successive times, producing a complex matrix indexed by frame and frequency bin.

Streaming. Processing incrementally without loading the entire source or output into memory. Streaming requires explicit state at chunk boundaries so results match uninterrupted processing.

Synthesis frame. A time-domain frame reconstructed from modified spectral data before windowing and overlap-add. Its placement is controlled by the synthesis hop.

Synthesis hop. The sample distance between successive output frames. Relative to the analysis hop, it controls the basic duration change in a phase-vocoder synthesis.

Temporal resolution. The ability to locate changes in time. Shorter windows improve temporal localization but provide less separation between nearby low frequencies.

Time map. A function relating output time to input time or the reverse. Nonlinear maps permit *accelerando*, *ritardando*, holds, jumps, and local alignment.

Time-frequency tradeoff. The constraint that analysis cannot make time and frequency localization arbitrarily precise at once. Window length and shape choose a compromise appropriate to the material.

Time-scale modification. Changing duration while attempting to preserve pitch. Success depends on phase continuity, transient treatment, source complexity, and the requested scale factor.

Transient. A short, rapidly changing event such as a drum hit, consonant, bow change, or pick attack. Transients violate assumptions of slow frame-to-frame spectral evolution.

Transient preservation. A strategy that detects attacks and protects their timing or waveform while transforming surrounding sustained material. Methods include phase reset, dry-path mixing, segmentation, and multiresolution processing.

True envelope. A high-resolution spectral-envelope estimate developed for robust separation of broad resonant shape from harmonic sampling. It is useful in high-quality formant-preserving transformations.

Unvoiced. Describing speech or sound without stable periodic vocal-fold excitation, such as many fricatives and breaths. Unvoiced regions often benefit from noise-aware rather than harmonic phase treatment.

Unwrap. To remove artificial two-pi discontinuities from wrapped phase values. Unwrapping selects a locally consistent trajectory but does not by itself guarantee correct component tracking.

Vibrato. A periodic or near-periodic modulation of pitch, often accompanied by amplitude and timbral modulation. Excessive smoothing can flatten it, while poor tracking can exaggerate or destabilize it.

Vocoder. A broad term for systems that encode or transform voice through an intermediate representation. Channel vocoders, phase vocoders, LPC vocoders, and neural vocoders use substantially different models.

Voiced. Describing sound with a stable or approximately periodic excitation and measurable fundamental frequency. Voiced frames support harmonic tracking and formant-preserving pitch operations.

Wet signal. The processed component of a wet-dry mixture. Its level should be interpreted relative to the dry path, correlation, latency, and any normalization applied inside the effect.

Wet-dry mix. A control blending processed and unprocessed signals. Time-varying mix can introduce comb filtering if paths are misaligned or strongly correlated with different phase responses.

Window. A weighting curve applied to each frame before transformation. It controls endpoint behavior, leakage, main-lobe width, sidelobes, coherent gain, and overlap-add behavior.

Window length. The number of samples receiving nonzero or defined window weights. It determines frame duration and is a principal control over time-frequency resolution.

WOLA. Weighted overlap-add. An analysis-synthesis framework in which analysis and synthesis windows jointly determine reconstruction. Their product and hop must satisfy the relevant overlap condition.

Zero padding. Appending zeros before a transform to sample the spectrum more densely. Zero padding improves visual and interpolation granularity but does not provide the resolving power of a longer observed signal.

Bibliography

This bibliography gathers one hundred twenty-five books and papers that support the theory, engineering practice, listening methods, and historical discussion in this guide. The papers emphasize phase-vocoder development, short-time analysis and reconstruction, pitch and formant processing, transient handling, time-scale modification, source separation, and objective evaluation. The books supply broader foundations in digital signal processing, speech, psychoacoustics, computer music, and sound engineering.

Papers and articles

The one hundred entries in this section are alphabetized by the first named author. DOI links are preferred when the repository record supplies one; otherwise, a stable publisher or catalog record is used where available.

- S. Ahmadi; H. Spanias. “Cepstrum-Based Pitch Detection Using FFT.” *Conference* (1999).
- Natsuki Akaishi; Kohei Yatabe; Yasuhiro Oikawa. “Improving Phase-Vocoder-Based Time Stretching by Time-Directional Spectrogram Squeezing.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2023). [DOI](#).
- J. Engel et al.. “DDSP: Differentiable Digital Signal Processing.” *ICLR* (2020). [catalog record](#).
- S. Wisdom et al.. “Differentiable Consistency Constraints for Improved Deep Speech Enhancement.” *ICASSP* (2021).
- J. B. Allen; L. R. Rabiner. “A Unified Approach to Short-Time Fourier Analysis and Synthesis.” *Proceedings of the IEEE* (1977). [DOI](#).
- F. Auger; P. Flandrin. “Improving the Readability of Time-Frequency and Time-Scale Representations by the Reassignment Method.” *IEEE Trans. SP* (1995). [DOI](#).
- A. J. Bell; T. J. Sejnowski. “An Information-Maximization Approach to Blind Separation and Blind Deconvolution.” *Neural Computation* (1995).
- Michaël Betser; Patrice Collen; Gaël Richard; Bertrand David. “Estimation of Frequency for AM/FM Models Using the Phase Vocoder Framework.” *IEEE Transactions on Signal Processing* (2008). [DOI](#).
- P. Boersma. “Accurate Short-Term Analysis of the Fundamental Frequency and the Harmonics-to-Noise Ratio of a Sampled Sound.” *IFA Proceedings* (1993).
- S. F. Boll. “Suppression of Acoustic Noise in Speech Using Spectral Subtraction.” *IEEE Trans. ASSP* (1979). [DOI](#).
- Ian Bowler; Alan Purvis; Peter Manning; Nick Bailey. “New Techniques for a Real-time Phase Vocoder.” *The Journal of the Abraham Lincoln Association* (1990). [catalog record](#).
- R. Bristow-Johnson; M. Bogdanowicz. “Phase Vocoder Done Right.” *AES Preprint* (1989).
- Robert Bristow-Johnson; K. Bogdanowicz. “Intraframe time-scaling of nonstationary sinusoids within the phase vocoder.” *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics* (2002). [DOI](#).
- J. C. Brown. “Calculation of a Constant Q Spectral Transform.” *JASA* (1991). [DOI](#).
- J. C. Brown; M. S. Puckette. “An Efficient Algorithm for the Calculation of a Constant Q Transform.” *JASA* (1992). [DOI](#).
- A. Camacho; J. G. Harris. “A Sawtooth Waveform Inspired Pitch Estimator for Speech and Music (SWIPE).” *JASA* (2008). [DOI](#).
- E. J. Candes; X. Li; Y. Ma; J. Wright. “Robust Principal Component Analysis?” *Journal of ACM* (2011).
- J. Capon. “High-Resolution Frequency-Wavenumber Spectrum Analysis.” *Proceedings of the IEEE* (1969). [DOI](#).

- A. de Cheveigné; H. Kawahara. “YIN, a Fundamental Frequency Estimator for Speech and Music.” *JASA* (2002). [DOI](#).
- I. Cohen; B. Berdugo. “Speech Enhancement for Non-Stationary Noise Environments.” *Signal Processing* (2001).
- L. Cohen. “Time-Frequency Distributions - A Review.” *Proceedings of the IEEE* (1989).
- R. R. Coifman; M. V. Wickerhauser. “Entropy-Based Algorithms for Best Basis Selection.” *IEEE Trans. IT* (1992). [DOI](#).
- P. Comon. “Independent Component Analysis, A New Concept?” *Signal Processing* (1994). [DOI](#).
- I. Daubechies; J. Lu; H.-T. Wu. “Synchrosqueezed Wavelet Transforms.” *Applied and Computational Harmonic Analysis* (2011). [DOI](#).
- S. Disch. “A New Phase Vocoder Technique for Time-Scale Modification of Audio Signals.” *DAFx* (1998).
- M. Dolson. “The Phase Vocoder: A Tutorial.” *Computer Music Journal* (1986). [catalog record](#).
- Mark Dolson. “A Tracking Phase Vocoder and its Use in the Analysis of Ensemble Sounds.” *Dissertation* (1983). [DOI](#).
- D. L. Donoho. “De-Noising by Soft-Thresholding.” *IEEE Trans. Information Theory* (1995).
- K. Dressler. “Sinusoidal Extraction using Frequency-Domain Matching Pursuit.” *DAFx* (2012).
- J. Driedger; M. Müller. “A Review of Time-Scale Modification of Music Signals.” *Applied Sciences* (2016). [DOI](#).
- C. Duxbury; M. Davies; M. Sandler. “Improved Time-Scaling of Musical Audio Using Phase Locking at Transients.” *AES Convention* (2002).
- Daniel P. W. Ellis; Ron J. Weiss. “Model-Based Monaural Source Separation Using a Vector-Quantized Phase-Vocoder Representation.” *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2006). [DOI](#).
- Y. Ephraim; D. Malah. “Speech Enhancement Using a Minimum Mean-Square Error Short-Time Spectral Amplitude Estimator.” *IEEE Trans. ASSP* (1984). [DOI](#).
- Y. Ephraim; D. Malah. “Speech Enhancement Using a Minimum Mean-Square Error Log-Spectral Amplitude Estimator.” *IEEE Trans. ASSP* (1985). [DOI](#).
- S. Essid; G. Richard. “Musical signal analysis with reassignment methods.” *Book chapter* (2012).
- Gianpaolo Evangelista. “Modified phase vocoder scheme for dynamic frequency warping.” *Proceedings of the IEEE International Symposium on Communications, Control and Signal Processing* (2008). [DOI](#).
- Gianpaolo Evangelista; Monika Dörfler; Ewa Matusiak. “Phase Vocoder With Arbitrary Frequency Band Selection.” *Zenodo (CERN European Organization for Nuclear Research)* (2012). [DOI](#).
- Gianpaolo Evangelista; Monika Dörfler; Ewa Matusiak. “Arbitrary Phase Vocoder by means of Warping.” *SHILAP Revista de lepidopterología* (2013). [DOI](#).
- C. Fevotte; N. Bertin; J.-L. Durrieu. “Nonnegative Matrix Factorization with the Itakura-Saito Divergence.” *Neural Computation* (2009).
- D. Fitzgerald. “Harmonic/Percussive Separation Using Median Filtering.” *DAFx* (2010). [catalog record](#).
- J. L. Flanagan; R. M. Golden. “Phase Vocoder.” *Bell System Technical Journal* (1966). [DOI](#).
- P. Flandrin; F. Auger; E. Chassande-Mottin. “Time-Frequency Reassignment: From Principles to Algorithms.” *Applications in Time-Frequency Signal Processing* (2002).
- John Garas; P.C.W. Sommen. “Time/pitch scaling using the constant-Q phase vocoder.” *Proceedings of ProRISC/IEEE CSSP98* (1998). [catalog record](#).
- D. Giannoulis; M. Massberg; J. Reiss. “Digital Dynamic Range Compressor Design.” *JAES* (2012).
- D. W. Griffin; J. S. Lim. “Signal Estimation from Modified Short-Time Fourier Transform.” *IEEE Trans. ASSP* (1984). [DOI](#).
- Amalia de Götzen; Nicola Bernardini; Daniel Arfib. “TRADITIONAL (?) IMPLEMENTATIONS OF A PHASE-VOCODER: THE TRICKS OF THE TRADE.” *VBN Forskningsportal (Aalborg Universitet)* (2000). [catalog record](#).
- A. Hyvarinen; E. Oja. “Independent Component Analysis: Algorithms and Applications.” *Neural Networks* (2000).

- Nicolas Juillerat; B  at Hirsbrunner. "Audio time stretching with an adaptive multiresolution phase vocoder." *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing* (2017). [DOI](#).
- Thorsten Karrer; Eric Lee; Jan Borchers. "PhaVoRIT: A Phase Vocoder for Real-Time Interactive Time-Stretching." *RWTH Publications (RWTH Aachen)* (2006). [catalog record](#).
- H. Kawahara; A. de Cheveigne. "STRAIGHT, Exploitation of the Other Aspects of Vocal Source Information." *ICASSP* (2001).
- J. W. Kim; J. Salamon; P. Li; J. P. Bello. "CREPE: A Convolutional Representation for Pitch Estimation." *ICASSP* (2018). [catalog record](#).
- A. Klapuri. "Multiple Fundamental Frequency Estimation by Harmonicity and Spectral Smoothness." *IEEE TASLP* (2006).
- C. Knapp; G. Carter. "The Generalized Correlation Method for Estimation of Time Delay." *IEEE Trans. ASSP* (1976). [DOI](#).
- J. Laroche; M. Dolson. "About this Phasiness Business." *IEEE ASSP Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* (1997). [catalog record](#).
- J. Laroche; M. Dolson. "Improved Phase Vocoder Time-Scale Modification of Audio." *IEEE Trans. Speech and Audio Processing* (1999). [DOI](#).
- J. Laroche; M. Dolson. "New Phase-Vocoder Techniques for Pitch-Shifting, Harmonizing and Other Exotic Effects." *WASPAA* (1999). [DOI](#).
- D. D. Lee; H. S. Seung. "Learning the Parts of Objects by Non-negative Matrix Factorization." *Nature* (1999). [DOI](#).
- D. D. Lee; H. S. Seung. "Algorithms for Non-negative Matrix Factorization." *NIPS* (2001). [DOI](#).
- Eric Lee; Thorsten Karrer; Jan Borchers. "An Analysis of Startup and Dynamic Latency in phase vocoder-Based Time-stretching Algorithms." *RWTH Publications (RWTH Aachen)* (2007). [catalog record](#).
- S. Mallat. "A Theory for Multiresolution Signal Decomposition: The Wavelet Representation." *IEEE Trans. PAMI* (1989). [DOI](#).
- S. Mallat; Z. Zhang. "Matching Pursuits with Time-Frequency Dictionaries." *IEEE Trans. Signal Processing* (1993).
- Sylvain Marchand. "Improving Spectral Analysis Precision With an Enhanced Phase Vocoder Using Signal Derivatives." *Proceedings of the International Computer Music Conference* (1998). [catalog record](#).
- R. Martin. "Noise Power Spectral Density Estimation Based on Optimal Smoothing and Minimum Statistics." *IEEE Trans. Speech and Audio Processing* (2001). [DOI](#).
- M. Mauch; S. Dixon. "pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions." *ICASSP* (2014). [DOI](#).
- R. J. McAulay; T. F. Quatieri. "Speech Analysis/Synthesis Based on a Sinusoidal Representation." *IEEE Trans. ASSP* (1986).
- M. McLeod; G. Wyvill. "A Smarter Way to Find Pitch." *ICMC* (2005).
- James A. Moorer. "The Use of the Phase Vocoder in Computer Music Applications." *Journal of the Audio Engineering Society* (1978). [catalog record](#).
- E. Moulines; F. Charpentier. "Pitch-Synchronous Waveform Processing Techniques for Text-to-Speech Synthesis Using Diphones." *Speech Communication* (1990). [DOI](#).
- E. Moulines; J. Laroche. "Non-parametric Techniques for Pitch-scale and Time-scale Modification of Speech." *Speech Communication* (1995). [DOI](#).
- Frederik Nagel; Sascha Disch; Nikolaus Rettelbach. "A Phase Vocoder Driven Bandwidth Extension Method with Novel Transient Handling for Audio Codecs." *Publikationsdatenbank der Fraunhofer-Gesellschaft (Fraunhofer-Gesellschaft)* (2009). [catalog record](#).
- T. Nakatani; M. Miyoshi; K. Kinoshita. "Speech Dereverberation Based on Variance-Normalized Delayed Linear Prediction." *IEEE Trans. Audio, Speech, and Language Processing* (2010). [DOI](#).
- A. M. Noll. "Cepstrum Pitch Determination." *JASA* (1967).

- Emil Solsbaek Ottosen; Monika Dorfler. "A Phase Vocoder Based on Nonstationary Gabor Frames." *IEEE/ACM Transactions on Audio Speech and Language Processing* (2017). [DOI](#).
- K. K. Paliwal. "Importance of phase in speech enhancement." *Speech Communication* (2011).
- M. R. Portnoff. "Implementation of the Digital Phase Vocoder Using the Fast Fourier Transform." *IEEE Trans. ASSP* (1976).
- M. R. Portnoff. "Time-Scale Modification of Speech Based on Short-Time Fourier Analysis." *IEEE Trans. ASSP* (1980).
- M. Puckette. "Phase-Locked Vocoder." *ICMC* (1995).
- Miller Puckette; Judith C. Brown. "Accuracy of frequency estimates using the phase vocoder." *IEEE Transactions on Speech and Audio Processing* (1998). [DOI](#).
- S. Pulkki. "Virtual Sound Source Positioning Using Vector Base Amplitude Panning." *JAES* (1997).
- L. R. Rabiner; M. J. Cheng; A. E. Rosenberg; C. A. McGonegal. "A Comparative Performance Study of Several Pitch Detection Algorithms." *IEEE Trans. ASSP* (1976).
- J. Reiss. "Under the Hood of a Dynamic Range Compressor." *AES* (2011).
- A. W. Rix; J. G. Beerends; M. P. Hollier; A. P. Hekstra. "Perceptual Evaluation of Speech Quality (PESQ)." *ICASSP* (2001). [DOI](#).
- A. Roebel. "A Shape-Invariant Phase Vocoder for Speech Transformation." *Interspeech* (2010).
- N. Roucos; A. Wilgus. "High Quality Time-Scale Modification for Speech." *ICASSP* (1985).
- J. Le Roux; S. Wisdom; H. Erdogan; J. R. Hershey. "SDR half-baked or well done?" *ICASSP* (2019).
- M. Le Roux; E. Vincent. "Consistent Wiener Filtering for Audio Source Separation and Time-Frequency Processing." *IEEE Signal Processing Letters* (2011).
- A. Röbel. "A New Approach to Transient Processing in the Phase Vocoder." *DAFx* (2003). [catalog record](#).
- A. Röbel; X. Rodet. "Efficient Spectral Envelope Estimation and Its Application to Pitch Shifting and Envelope Preservation." *DAFx* (2005). [catalog record](#).
- Axel Röbel; Xavier Rodet. "REAL TIME SIGNAL TRANSPOSITION WITH ENVELOPE PRESERVATION IN THE PHASE VOCODER." *HAL (Le Centre pour la Communication Scientifique Directe)* (2005). [catalog record](#).
- P. Scalart; J. V. Filho. "Speech Enhancement Based on a Priori Signal to Noise Estimation." *ICASSP* (1996).
- X. Serra; J. O. Smith. "Spectral Modeling Synthesis: A Sound Analysis/Synthesis System Based on a Deterministic Plus Stochastic Decomposition." *Computer Music Journal* (1990). [catalog record](#).
- P. Smaragdis. "Convolutional Speech Bases and Their Application to Supervised Speech Separation." *IEEE TASLP* (2007).
- F.-R. Stöter; S. Uhlich; A. Liutkus; Y. Mitsufuji. "Open-Unmix - A Reference Implementation for Music Source Separation." *Journal of Open Source Software* (2019). [DOI](#).
- C. H. Taal; R. C. Hendriks; R. Heusdens; J. Jensen. "An Algorithm for Intelligibility Prediction of Time-Frequency Weighted Noisy Speech (STOI)." *IEEE TASLP* (2011). [DOI](#).
- D. Talkin. "A Robust Algorithm for Pitch Tracking (RAPT)." *In Speech Coding and Synthesis* (1995).
- J.-M. Valin. "A Hybrid DSP/Deep Learning Approach to Real-Time Full-Band Speech Enhancement (RN-Noise)." *MMSP / arXiv* (2018). [catalog record](#).
- G. Velasco; N. Holighaus; M. Dörfler; T. Grill. "Constructing an Invertible Constant-Q Transform with Nonstationary Gabor Frames." *DAFx* (2011). [catalog record](#).
- W. Verhelst; M. Roelands. "An Overlap-Add Technique Based on Waveform Similarity (WSOLA) for High Quality Time-Scale Modification of Speech." *ICASSP* (1993).
- T. Virtanen. "Monaural Sound Source Separation by Nonnegative Matrix Factorization with Temporal Continuity and Sparseness Criteria." *IEEE Trans. Audio, Speech, and Language Processing* (2007).
- Trevor Wishart. "From Sound Morphing to the Synthesis of Starlight. Musical experiences with the Phase Vocoder over 25 years." *SHILAP Revista de lepidopterología* (2013). [DOI](#).

Books

The twenty-five books in this section provide durable background and extended treatments. Editions are stated when the catalog distinguishes them.

Ballou, Glen M., editor. *Handbook for Sound Engineers*. Focal Press, 2015. 5th edition.

Benesty, Jacob; Sondhi, M. Mohan; Huang, Yiteng, editors. *Springer Handbook of Speech Processing*. Springer, 2008.

Bregman, Albert S.. *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, 1990.

Cohen, Leon. *Time-Frequency Analysis*. Prentice Hall, 1995.

Cook, Perry R.. *Real Sound Synthesis for Interactive Applications*. A K Peters, 2002.

Crochiere, Ronald E.; Rabiner, Lawrence R.. *Multirate Digital Signal Processing*. Prentice Hall, 1983.

Dodge, Charles; Jerse, Thomas A.. *Computer Music: Synthesis, Composition, and Performance*. Schirmer Books, 1997. 2nd edition.

Flanagan, James L.. *Speech Analysis, Synthesis and Perception*. Springer, 1972. 2nd edition.

Flandrin, Patrick. *Time-Frequency/Time-Scale Analysis*. Academic Press, 1999.

Grochenig, Karlheinz. *Foundations of Time-Frequency Analysis*. Birkhauser, 2001.

Klapuri, Anssi; Davy, Manuel, editors. *Signal Processing Methods for Music Transcription*. Springer, 2006.

Loizou, Philipos C.. *Speech Enhancement: Theory and Practice*. CRC Press, 2013. 2nd edition.

Muller, Meinard. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.

Oppenheim, Alan V.; Schafer, Ronald W.. *Discrete-Time Signal Processing*. Prentice Hall, 2010. 3rd edition.

Pohlmann, Ken C.. *Principles of Digital Audio*. McGraw-Hill, 2010. 6th edition.

Rabiner, Lawrence R.; Schafer, Ronald W.. *Digital Processing of Speech Signals*. Prentice Hall, 1978.

Roads, Curtis. *The Computer Music Tutorial*. MIT Press, 1996.

Roads, Curtis. *Microsound*. MIT Press, 2001.

Smith, Julius O.. *Mathematics of the Discrete Fourier Transform with Audio Applications*. W3K Publishing, 2007. 2nd edition.

Smith, Julius O.. *Spectral Audio Signal Processing*. W3K Publishing, 2011.

Wiener, Norbert. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. MIT Press, 1949.

Wishart, Trevor. *Audible Design: A Plain and Easy Introduction to Practical Sound Composition*. Orpheus the Pantomime, 1994.

Wishart, Trevor. *On Sonic Art*. Harwood Academic Publishers, 1996. new and revised edition.

Zolzer, Udo, editor. *DAFX: Digital Audio Effects*. Wiley, 2011. 2nd edition.

Zwicker, Eberhard; Fastl, Hugo. *Psychoacoustics: Facts and Models*. Springer, 1999. 2nd edition.

Open Media Credits

The historical illustrations in Chapter 1 are reproduced from Homer Dudley's 1940 paper, *The Carrier Nature of Speech*, via Internet Archive Book Images and Wikimedia Commons. Wikimedia Commons records them as public-domain or no-known-copyright-restrictions scans. Source records: Voder demonstration, voice-mechanism block diagram, and vocoder schematic: <https://commons.wikimedia.org/wiki/Category:Vocoder>. The analytical plots and window diagrams are original pvx project assets and are included with the project documentation.

Index

- 1/Multichannel Reference-Lock Stretch, 166
- 2001: A Space Odyssey, 22
- A chronology of ideas not merely machines, 11
- A continuing design checklist, 142
- A decade-by-decade change in the imagined user, 47
- A deeper history of spectral time, 17
- A fuller genealogy of phase-vocoder software, 36
- A historiography of claims and cautions, 33
- A Phase-Vocoder Listening Library 100 Musical Works, 330
- A quick mental model, 3
- A Reproducible Experiment Layout, 245
- A small comparison, 52
- A/B report generation, 257
- A/B sweep of transform backends from shell loop, 262
- Ablation Studies, 243
- acoustical quanta, 22
- Adagio for Strings*, 343
- Adams, John, 341
- Adapting the recipes, 192
- Adaptive and nonstationary time-frequency analysis, 32
- Air on the G String*, 335
- Algorithm Registry and Dispatch, 79
- Aliasing, 353
- All Is Full of Love*, 342
- Alpha Principles, 271
- Ambient Macro-Chain (Stretch + Cleanup + Loudness), 164
- Amplitude, 353
- An Ending (Ascent)*, 341
- analog delay lines, 22
- analog pitch shifting, 23
- Analog Tape Methods for Pitch/Time Shifting, 56
- analog time stretching, 23
- Analysis frame, 353
- Analysis hop, 353
- analysis hop, 11
- analysis qa and automation, 301
- Analysis-synthesis system, 353
- Analysis/QA, 257
- Analysis/Response Artifact and Spectral Operator Ideas, 210
- Anderson, Laurie, 342
- Annotated chronology, 48
- Anonymous, 333
- Aphex Twin, 338
- Artifact, 353
- Artifact - Fix Table, 66
- Artifact history as listening history, 40
- ASR Automatic Speech Recognition, 250
- Atmospheres*, 340
- Attack, 353
- Audio Event Detection / Sound Classification, 255
- Audio Metrics Table Generation, 319
- AudioCollator for variable-length batches, 223
- Augmentation Benchmark Gate, 181
- Augmentation Intensity Guide, 256
- Autechre, 339
- Auto Profile + Auto Transform (Planning), 155
- Auto-profile plan preview, 260
- Automatic speech recognition, 242
- Automation, 257
- Automation and interpolation, 348
- Automation as composition, 184
- Automation curve, 353
- Available GPU transforms, 229
- Available Transforms, 230
- Ave Maria virgo serena*, 333
- Bach, Johann Sebastian, 334, 335, 337, 339
- Backend Summary, 264
- Backward Compatibility and UX, 329
- Bandlimit, 353
- Bandwidth, 353
- Barber, Samuel, 343
- bartlett, 96
- bartlett hann, 99
- Bartok, Bela, 340
- Basic DSP Terms (Plain Language), 56
- Basic map() usage, 223
- Basinski, William, 344
- Batch, 258
- Batch Processing Example (Python), 218
- Batch Processing Folder, 154
- Batch stretch over folder, 258
- Batch Tree Processing with find + xargs, 163
- Beat phase + downbeat phase, 197
- Beat Tracking and Tempo Estimation, 253
- Beethoven, Ludwig van, 335, 337, 339, 343
- Bell Laboratories as a long technical precondition, 41

-
- Benchmark Command (pvx vs Rubber Band vs librosa), 159
 - Benchmark matrix sweep, 257
 - Benchmark Metric Taxonomy, 319
 - Benchmark Runner + CI Gate, 318
 - Benchmark Spec, 264
 - Benchmarking Augmentation Pipelines, 248
 - Benefits we can extract for pvx, 321
 - Berio, Luciano, 334
 - Berlioz, Hector, 339
 - Beta in 0.1.x, 69
 - bibliography, 362
 - books, 366
 - papers and articles, 362
 - Bin, 353
 - Bin-center frequency, 353
 - Bingen, Hildegard von, 333
 - Bit depth, 353
 - Bjork, 342
 - blackman, 89
 - blackman nuttall, 93
 - blackmanharris, 90
 - Blend pitch behavior from two guides, 201
 - Block processing, 353
 - Boards of Canada, 342
 - Bode, Harald, 23
 - bohman, 133
 - Bolero*, 343
 - boxcar, 97
 - Brahms, Johannes, 339
 - Bring the Noise*, 338
 - Britten, Benjamin, 340
 - Broadband, 353
 - Broadcast-Style Speech Finishing (Stretch + Loudness + Safety), 165
 - Brown, James, 338
 - Bruckner, Anton, 336
 - Buffer, 353
 - Build custom vector features then route them, 200
 - Bush, Kate, 342

 - Caching expensive transforms, 226
 - Cage, John, 338, 343
 - Canon in D*, 343
 - Capability Matrix, 143
 - Caprice No. 24*, 337
 - Carlos, Wendy, 22, 332
 - carrier signal, 2
 - cauchy 0p5, 120
 - cauchy 1p0, 121
 - cauchy 2p0, 122
 - Cepstrum, 354

 - Channel, 354
 - Checkpoint, 354
 - Checkpoint + Resume Long Render, 155
 - Checkpoint / Resume State Machine, 318
 - Checkpointed long render with manifest, 260
 - Checkpointed Long Render Workflow, 163
 - Chopin, Frederic, 335, 337
 - Chroma, 354
 - Chunked Stream Wrapper + Output Policy Controls, 169
 - CI + Pages, 79
 - Class Imbalance and Conditional Policies, 243
 - CLI options
 - 1950s, 52
 - 1970s, 52
 - 1980s, 52
 - a-args, 262
 - a4-reference-hz, 65, 182, 299
 - pvxretune, 352
 - alpha, 65, 182, 192, 214, 299
 - pvxmorph, 352
 - ambient-phase-mix, 65, 299
 - pvxvoc, 352
 - ambient-preset, 262, 299
 - pvxvoc, 352
 - amplitude, 65, 182, 299
 - pvxenvelope, 352
 - analysis-channel, 299
 - pvxanalysis, 352
 - pvxvoc, 352
 - append-manifest, 182, 249, 299
 - pvx, 352
 - attack-sec, 142, 182, 214, 299
 - pvxenvelope, 352
 - attenuation, 142, 182, 192, 214, 299
 - pvxharmmap, 352
 - audit-metrics, 299
 - pvx, 352
 - auto-profile, 65, 182, 192, 262, 299
 - pvxvoc, 352
 - auto-profile-lookahead-seconds, 182, 299
 - pvxvoc, 352
 - auto-segment-seconds, 65, 182, 192, 214, 262, 299
 - pvxvoc, 352
 - auto-transform, 65, 182, 192, 262, 299
 - pvxvoc, 352
 - azimuth-deg, 262
 - b-args, 262
 - backend, 182, 299
 - hps-pitch-track, 352

- pvx, 352
- background-dir, 299
 - pvxnoise, 352
- band, 299
 - pvxnoise, 352
- band-gain-db, 182, 214, 299
 - pvxfilter, 352
- band-width-bins, 182, 299
 - pvxfilter, 352
- baseline, 65, 182, 192
- baseline-json, 262
- batch-size, 299
 - pvx, 352
- bit-depth, 65, 68, 182, 192, 214, 299
 - pvx, 352
 - pvxcodec, 352
 - pvxgain, 352
 - pvxnoise, 352
 - pvxrir, 352
 - pvxspecaugment, 352
 - pvxvoc, 352
- bits, 299
 - pvxcodec, 352
- blend-mode, 65, 182, 214, 299
 - pvxmorph, 352
- boost-db, 142, 182, 192, 214, 299
 - pvxharmmmap, 352
- budget-path, 65, 299
 - pvx, 352
- build-from-source, 270
- cache-dir, 299
 - pvxrir, 352
- category, 299
 - pvxrir, 352
- center, 65, 182, 299
 - pvxenvelope, 352
- cents, 299
 - pvxvoc, 352
- checkpoint-dir, 65, 182, 192, 214, 262, 299
 - pvxvoc, 352
- checkpoint-id, 299
 - pvxvoc, 352
- chord, 142, 182, 192, 214, 299
 - pvxharmmmap, 352
- chunk-ms, 299
 - pvxretune, 352
- chunk-seconds, 65, 182, 214, 299
 - pvx, 352
- clip, 299
 - pvxvoc, 352
- codec, 256, 299
 - pvxcodec, 352
- coherence-report-json, 322
- coherence-strength, 65, 68, 182, 192, 214, 299
 - pvxvoc, 352
- comp-ratio, 182, 192, 214, 299
 - pvxfilter, 352
- comp-threshold-db, 182, 192, 214, 299
 - pvxfilter, 352
- compander-attack-ms, 299
 - pvxvoc, 352
- compander-compress-ratio, 299
 - pvxvoc, 352
- compander-expand-ratio, 299
 - pvxvoc, 352
- compander-makeup-db, 299
 - pvxvoc, 352
- compander-release-ms, 299
 - pvxvoc, 352
- compander-threshold-db, 299
 - pvxvoc, 352
- compressor-attack-ms, 262, 299
 - pvxvoc, 352
- compressor-makeup-db, 262, 299
 - pvxvoc, 352
- compressor-ratio, 182, 262, 299
 - pvxvoc, 352
- compressor-release-ms, 262, 299
 - pvxvoc, 352
- compressor-threshold-db, 182, 262, 299
 - pvxvoc, 352
- confidence-floor, 299
 - hps-pitch-track, 352
 - pvx, 352
- context-ms, 299
 - pvx, 352
- control-stdin, 65, 182, 203, 204, 214, 299
 - pvxvoc, 352
- coord-system, 299
 - pvxtrajectoryreverb, 352
- cpu, 299
 - pvxvoc, 352
- crossfade-ms, 182, 214, 299
 - pvx, 352
 - pvxconform, 352
 - pvxwarp, 352
- cuda-device, 299
 - pvxanalysis, 352
 - pvxvoc, 352
- cycles, 182, 299
 - pvxenvelope, 352

- decay, 182, 299
 - pvxdeverb, 352
- decay-sec, 142, 182, 214, 299
 - pvxenvelope, 352
- dedupe-by, 182, 299
 - pvx, 352
- depth, 142, 182, 192, 214, 299
 - pvxring, 352
- detune-cents, 65, 182, 192, 262, 299
 - pvxunison, 352
- device, 182, 299
 - pvx, 352
 - pvxanalysis, 352
 - pvxvoc, 352
- devices, 262
- disk-budget, 65, 182, 214, 299
 - pvx, 352
- distance-law, 299
 - pvxtrajectoryreverb, 352
- dither, 65, 68, 182, 192, 214, 299
 - pvxvoc, 352
- dither-seed, 65, 182, 192, 214, 299
 - pvxvoc, 352
- drr, 299
 - pvxrir, 352
- dry, 299
 - pvxtrajectoryreverb, 352
- dry-mix, 182, 192, 214, 299
 - pvxfilter, 352
 - pvxharmmap, 352
 - pvxunison, 352
- dry-run, 68, 182, 192, 249, 262, 299
 - pvx, 352
 - pvxvoc, 352
- duration, 52, 65, 142, 182, 192, 214, 299
 - pvx, 352
 - pvxenvelope, 352
 - pvxfreeze, 352
- duty-cycle, 182, 299
 - pvxenvelope, 352
- emit, 65, 182, 192, 203, 204, 214, 299
 - hps-pitch-track, 352
 - pvx, 352
- end, 142, 192, 214, 299
 - pvxenvelope, 352
 - pvxtrajectoryreverb, 352
- engine, 231, 256, 299
 - pvx, 352
- engines, 231
- envelope-lifter, 182, 214, 299
 - pvxmorph, 352
- example, 65, 68, 182, 203, 214, 299
 - pvx, 352
 - pvxvoc, 352
- exp-curve, 142, 192, 214, 299
 - pvxenvelope, 352
- expand-ratio, 182, 192, 214, 299
 - pvxfilter, 352
- expander-attack-ms, 299
 - pvxvoc, 352
- expander-ratio, 299
 - pvxvoc, 352
- expander-release-ms, 299
 - pvxvoc, 352
- expander-threshold-db, 299
 - pvxvoc, 352
- explain-plan, 65, 68, 182, 192, 219, 262, 299
 - pvxvoc, 352
- exponent, 299
 - pvxreshape, 352
- extreme-stretch-threshold, 65, 299
 - pvxvoc, 352
- extreme-time-stretch, 299
 - pvxvoc, 352
- f0-max, 182, 299
 - pvxretune, 352
 - pvxvoc, 352
- f0-min, 182, 299
 - pvxretune, 352
 - pvxvoc, 352
- factor, 214, 299
 - pvxreshape, 352
- fail-if-exceeds, 65, 182, 214, 299
 - pvx, 352
- feature-set, 65, 182, 192, 203, 214, 299
 - hps-pitch-track, 352
 - pvx, 352
- feedback, 182, 214, 299
 - pvxring, 352
- fill, 299
 - pvxspecaugment, 352
- floor, 65, 68, 182, 299
 - pvxdenoise, 352
 - pvxdeverb, 352
- fmax, 299
 - hps-pitch-track, 352
 - pvx, 352
- fmin, 299
 - hps-pitch-track, 352
 - pvx, 352
- force, 270, 299
 - pvxvoc, 352

- force-stereo, 182, 214, 299
 - pvxharmonize, 352
- formant-lifter, 65, 68, 182, 299
 - pvxformant, 352
 - pvxvoc, 352
- formant-max-gain-db, 65, 299
 - pvxformant, 352
 - pvxvoc, 352
- formant-preserve, 52
- formant-shift-ratio, 182, 299
 - pvxformant, 352
- formant-strength, 65, 68, 182, 299
 - pvxvoc, 352
- format, 299
 - pvxenvelope, 352
 - pvxreshape, 352
- formula, 270
- fourier-sync, 299
 - pvxvoc, 352
- fourier-sync-max-fft, 65, 299
 - pvxvoc, 352
- fourier-sync-min-fft, 65, 299
 - pvxvoc, 352
- fourier-sync-smooth, 65, 299
 - pvxvoc, 352
- frame-length, 299
 - hps-pitch-track, 352
 - pvx, 352
- freeze-time, 52, 65, 182, 192, 214, 299
 - pvxfreeze, 352
- freq, 299
 - pvxenvelope, 352
- freq-mask, 299
 - pvxspecaugment, 352
- frequency-hz, 65, 142, 182, 192, 214, 299
 - pvxenvelope, 352
 - pvxring, 352
- gain, 299
 - pvxgain, 352
- gains, 65, 182, 192, 214, 299
 - pvxharmonize, 352
- gate, 65, 182, 192, 249, 271
- gate-tolerance, 182, 192
- gpu, 182, 299
 - pvxvoc, 352
- group-separator, 182, 249, 299
 - pvx, 352
- grouping, 182, 249, 299
 - pvx, 352
- guided, 299
 - pvxvoc, 352
- hard-clip-level, 182, 262, 299
 - pvxvoc, 352
- harmonic-gain, 182, 299
 - pvxlayer, 352
- harmonic-kernel, 299
 - pvxlayer, 352
- harmonic-pitch-cents, 299
 - pvxlayer, 352
- harmonic-pitch-semitones, 182, 214, 299
 - pvxlayer, 352
- harmonic-stretch, 182, 214, 299
 - pvxlayer, 352
- help, 65, 270
- hop-length, 299
 - pvxspecaugment, 352
- hop-size, 65, 78, 142, 182, 192, 214, 262, 299
 - hps-pitch-track, 352
 - pvx, 352
 - pvxanalysis, 352
 - pvxvoc, 352
- inharmonic-f0-hz, 182, 192, 214, 299
 - pvxharmmap, 352
- inharmonic-mix, 182, 192, 214, 299
 - pvxharmmap, 352
- inharmonic-ity, 182, 192, 214, 299
 - pvxharmmap, 352
- input, 262
- input-format, 299
 - pvxreshape, 352
- intent, 65, 182, 231, 249, 256, 299
 - pvx, 352
- interp, 65, 78, 142, 182, 192, 214, 299, 320
 - pvxmorph, 352
 - pvxreshape, 352
 - pvxvoc, 352
- intervals, 65, 182, 192, 214, 262, 299
 - pvxharmonize, 352
- intervals-cents, 182, 214, 299
 - pvxharmonize, 352
- ir, 299
 - pvxtrajectoryreverb, 352
- ir-database, 299
 - pvxrir, 352
- ir-dir, 299
 - pvxrir, 352
- ir-file, 299
 - pvxrir, 352
- json, 65, 182, 214, 299
 - pvx, 352
 - pvxanalysis, 352
 - pvxresponse, 352

- kaiser-beta, 65, 136, 262, 299
 - pvxanalysis, 352
 - pvxvoc, 352
- keep-intermediate, 299
 - pvx, 352
- key, 65, 142, 182, 192, 214, 299
 - pvxenvelope, 352
 - pvxreshape, 352
- label-policy, 249, 256, 299
 - pvx, 352
- labels-csv, 182, 249, 299
 - pvx, 352
- limiter-threshold, 182, 192, 262, 299
 - pvxvoc, 352
- list-databases, 299
 - pvxrir, 352
- manifest-append, 182, 192, 214, 299
 - pvxvoc, 352
- manifest-csv, 249, 299
 - pvx, 352
- manifest-json, 72, 182, 192, 214, 219, 262, 299
 - pvxvoc, 352
- manifest-jsonl, 249, 299
 - pvx, 352
- map, 65, 72, 182, 214, 299
 - pvxconform, 352
 - pvxwarp, 352
- mask-exponent, 78, 182, 214, 299
 - pvxmorph, 352
- material, 65, 182, 299
 - pvx, 352
- max, 299
 - pvxenvelope, 352
 - pvxreshape, 352
- max-length-s, 299
 - pvx, 352
- max-stage-stretch, 65, 68, 182, 299
 - pvxvoc, 352
- metadata-policy, 65, 182, 192, 214, 299
 - pvxvoc, 352
- method, 142, 182, 192, 214, 299
 - pvxresponse, 352
- mfcc-count, 65, 182, 192, 203, 214, 299
 - hps-pitch-track, 352
 - pvx, 352
- min, 299
 - pvxenvelope, 352
 - pvxreshape, 352
- mix, 142, 182, 192, 214, 262, 299
 - pvxring, 352
- mode, 65, 142, 182, 192, 214, 262, 299
 - pvx, 352
 - pvxenvelope, 352
 - pvxformant, 352
- multires-ffts, 182, 192, 262, 299
 - pvxvoc, 352
- multires-fusion, 65, 68, 182, 192, 262, 299
 - pvxvoc, 352
- multires-weights, 182, 192, 262, 299
 - pvxvoc, 352
- n-fft, 65, 68, 72, 78, 142, 182, 192, 214, 262, 299
 - pvxanalysis, 352
 - pvxspecaugment, 352
 - pvxvoc, 352
- n-ffts, 262
- name, 262
- no-audit-metrics, 299
 - pvx, 352
- no-center, 299
 - pvxanalysis, 352
 - pvxvoc, 352
- no-normalize-gains, 299
 - pvxtrajectoryreverb, 352
- no-onset-realign, 299
 - pvxvoc, 352
- no-progress, 299
 - pvx, 352
 - pvxvoc, 352
- no-trim, 299
 - pvxrir, 352
- noise-file, 65, 68, 182, 299
 - pvxdenoise, 352
- noise-floor, 182, 192, 214, 299
 - pvxfilter, 352
- noise-seconds, 65, 68, 182, 299
 - pvxdenoise, 352
- noise-type, 256, 299
 - pvxnoise, 352
- normalize, 142, 182, 192, 214, 262, 299
 - pvxgain, 352
 - pvxresponse, 352
 - pvxvoc, 352
- normalize-energy, 299
 - pvxmorph, 352
- normalize-gains, 299
 - pvxtrajectoryreverb, 352
- num-freq-masks, 299
 - pvxspecaugment, 352
- num-time-masks, 299
 - pvxspecaugment, 352

- o—o—o, 320
- o—o—o—o—, 320
- offset, 214, 299
 - pvxreshape, 352
- onset-credit-max, 65, 299
 - pvxvoc, 352
- onset-credit-pull, 65, 299
 - pvxvoc, 352
- onset-time-credit, 299
 - pvxvoc, 352
- operation, 142, 182, 192, 214, 299
 - pvxreshape, 352
- operator, 299
 - pvxfilter, 352
 - pvxharmpmap, 352
 - pvxring, 352
- order, 65, 78, 142, 182, 192, 214, 299, 320
 - pvxmorph, 352
 - pvxreshape, 352
 - pvxvoc, 352
- out, 299
 - pvx, 352
 - pvxenvelope, 352
 - pvxreshape, 352
 - pvxvoc, 352
- out-dir, 65, 182, 192, 249, 262, 271
- output, 52, 65, 68, 142, 182, 192, 203, 204, 214, 256, 262, 299, 320
 - hps-pitch-track, 352
 - pvx, 352
 - pvxanalysis, 352
 - pvxcodec, 352
 - pvxenvelope, 352
 - pvxgain, 352
 - pvxmorph, 352
 - pvxnoise, 352
 - pvxreshape, 352
 - pvxresponse, 352
 - pvxrir, 352
 - pvxspecaugment, 352
 - pvxvoc, 352
- output-channels, 262
- output-csv, 182, 249, 299
 - pvx, 352
- output-dir, 65, 78, 182, 231, 249, 256, 262, 299
 - pvx, 352
 - pvxvoc, 352
- output-format, 65, 72, 299
 - pvx, 352
 - pvxmorph, 352
 - pvxvoc, 352
- output-jsonl, 182, 249, 299
 - pvx, 352
- output-key, 299
 - pvxreshape, 352
- output-subtype, 299
 - pvx, 352
- output-suffix, 299
 - pvx, 352
- overlap-ms, 299
 - pvxretune, 352
- overwrite, 182, 262, 299
 - pvx, 352
 - pvxmorph, 352
 - pvxvoc, 352
- pair-mode, 182, 249, 299
 - pvx, 352
- pans, 182, 214, 299
 - pvxharmonize, 352
- peak, 214, 299
 - pvxenvelope, 352
- peak-count, 182, 214, 299
 - pvxfilter, 352
- peak-dbf, 262, 299
 - pvxvoc, 352
- percussive-gain, 182, 214, 299
 - pvxlayer, 352
- percussive-kernel, 299
 - pvxlayer, 352
- percussive-pitch-cents, 299
 - pvxlayer, 352
- percussive-pitch-semitones, 182, 299
 - pvxlayer, 352
- percussive-stretch, 182, 214, 299
 - pvxlayer, 352
- phase, 299
 - pvxenvelope, 352
- phase-engine, 214, 262, 299
 - pvxvoc, 352
- phase-lock-radius, 322
- phase-lock-strength, 322
- phase-locking, 52, 65, 68, 78, 182, 192, 262, 299
 - pvxvoc, 352
- phase-mix, 182, 214, 299
 - pvxmorph, 352
- phase-mode, 142, 182, 192, 299
 - pvxfreeze, 352
 - pvxresponse, 352
- phase-noise-floor-db, 322
- phase-noise-lock, 322

-
- phase-policy, 322
 - phase-rad, 299
 - pvxenvelope, 352
 - phase-random-seed, 299
 - pvxvoc, 352
 - phase-tonal-lock, 322
 - pipeline, 65, 142, 182, 214, 299
 - pvx, 352
 - pipeline-config, 299
 - pvx, 352
 - pitch, 52, 65, 182, 192, 214, 256, 299
 - pvx, 352
 - pvxvoc, 352
 - pitch-conf-min, 65, 182, 192, 203, 204, 214, 262, 299
 - pvx, 352
 - pvxvoc, 352
 - pitch-follow-stdin, 262, 299
 - pvxvoc, 352
 - pitch-lowconf-mode, 182, 203, 262, 299
 - pvx, 352
 - pvxvoc, 352
 - pitch-map, 65, 72, 182, 192, 203, 299
 - pvxvoc, 352
 - pitch-map-crossfade-ms, 182, 299
 - pvx, 352
 - pvxvoc, 352
 - pitch-map-smooth-ms, 203, 299
 - pvx, 352
 - pvxvoc, 352
 - pitch-map-stdin, 65, 299
 - pvxvoc, 352
 - pitch-mode, 65, 68, 182, 192, 214, 262, 299
 - pvxvoc, 352
 - pitch-ratio, 182
 - pitch-shift-cents, 65, 78, 262, 299
 - pvxformant, 352
 - pvxtransient, 352
 - pvxvoc, 352
 - pitch-shift-ratio, 65, 72, 182, 192, 299
 - pvxtransient, 352
 - pvxvoc, 352
 - pitch-shift-semitones, 65, 262, 299
 - pvxformant, 352
 - pvxtransient, 352
 - pvxvoc, 352
 - policy, 249, 299
 - pvx, 352
 - pre-delay-ms, 299
 - pvxrir, 352
 - prefix, 270
 - preset, 65, 68, 182, 192, 214, 256, 299
 - pvxvoc, 352
 - quality-profile, 299
 - pvxvoc, 352
 - quick, 65, 182, 192, 249, 271
 - quiet, 262, 299
 - pvx, 352
 - pvxvoc, 352
 - random-phase, 299
 - pvxfreeze, 352
 - rate, 142, 182, 192, 214, 299
 - pvxenvelope, 352
 - pvxreshape, 352
 - ratio, 214, 299
 - pvxvoc, 352
 - ratio-max, 299
 - hps-pitch-track, 352
 - pvx, 352
 - ratio-min, 299
 - hps-pitch-track, 352
 - pvx, 352
 - ratio-reference, 299
 - hps-pitch-track, 352
 - pvx, 352
 - recommend-root, 65, 182, 299
 - pvxretune, 352
 - reduction-db, 65, 68, 142, 182, 192, 203, 214, 262, 299
 - pvxdenoise, 352
 - ref-channel, 68, 182, 192, 299
 - pvxvoc, 352
 - reference-hz, 299
 - hps-pitch-track, 352
 - pvx, 352
 - refresh-baselines, 249
 - release-sec, 142, 182, 214, 299
 - pvxenvelope, 352
 - render-args, 262
 - report-json, 262
 - requested-stretch, 65, 182, 214, 299
 - pvx, 352
 - resample-mode, 299
 - pvxconform, 352
 - pvxformant, 352
 - pvxharmonize, 352
 - pvxlayer, 352
 - pvxretune, 352
 - pvxtransient, 352
 - pvxunison, 352
 - pvxvoc, 352
 - pvxwarp, 352

- resonance-decay, 142, 182, 192, 299
 - pvxring, 352
- resonance-hz, 142, 182, 192, 214, 299
 - pvxring, 352
- resonance-mix, 142, 182, 192, 214, 299
 - pvxring, 352
- resonance-q, 142, 182, 192, 214, 299
 - pvxring, 352
- response, 142, 182, 192, 214, 299
 - pvxfilter, 352
- response-gain-db, 299
 - pvxfilter, 352
- response-mix, 182, 192, 214, 299
 - pvxfilter, 352
- resume, 65, 182, 192, 214, 249, 299
 - pvx, 352
 - pvxvoc, 352
- rms-dbfs, 299
 - pvxvoc, 352
- root, 65, 182, 192, 214, 262, 299
 - pvxretune, 352
- root-hz, 65, 142, 182, 192, 214, 299
 - pvxharmpmap, 352
 - pvxretune, 352
- route, 65, 182, 192, 203, 204, 214, 299
 - pvx, 352
 - pvxvoc, 352
- rt60, 256, 299
 - pvxrir, 352
- run-benchmarks, 266
- safety-margin, 65, 299
 - pvx, 352
- sample-rate, 182, 299
 - pvx, 352
- scale, 65, 182, 192, 214, 299
 - pvxretune, 352
- scale-cents, 262, 299
 - pvxretune, 352
- seed, 65, 182, 231, 249, 256, 299
 - pvx, 352
 - pvxcodec, 352
 - pvxgain, 352
 - pvxnoise, 352
 - pvxrir, 352
 - pvxspecaugment, 352
- semitones, 299
 - pvxvoc, 352
- shift-bins, 214, 299
 - pvxfilter, 352
- silent, 299
 - pvx, 352
- pvxvoc, 352
- sine-cycles, 214, 299
 - pvxenvelope, 352
- sine-phase-rad, 299
 - pvxenvelope, 352
- smooth, 65, 68, 182, 192, 214, 299
 - pvxdenoise, 352
- smooth-frames, 299
 - hps-pitch-track, 352
 - pvx, 352
- smoothing-bins, 142, 182, 192, 299
 - pvxresponse, 352
- snr, 256, 299
 - pvxnoise, 352
- soft-clip-drive, 299
 - pvxvoc, 352
- soft-clip-level, 78, 182, 262, 299
 - pvxvoc, 352
- soft-clip-type, 262, 299
 - pvxvoc, 352
- speaker-angles, 299
 - pvxtrajectoryreverb, 352
- split, 182, 249, 299
 - pvx, 352
- split-mode, 182, 249, 299
 - pvx, 352
- sr, 299
 - pvx, 352
- start, 142, 192, 214, 299
 - pvxenvelope, 352
 - pvxtrajectoryreverb, 352
- stdout, 65, 72, 182, 192, 214, 262, 299, 320
 - pvxenvelope, 352
 - pvxmorph, 352
 - pvxreshape, 352
 - pvxvoc, 352
- stereo-mode, 65, 68, 182, 192, 214, 299
 - pvxvoc, 352
- strength, 65, 142, 182, 192, 203, 214, 262, 299
 - pvxdeverb, 352
 - pvxharmpmap, 352
 - pvxretune, 352
- stretch, 52, 65, 68, 72, 142, 182, 192, 203, 214, 256, 299
 - hps-pitch-track, 352
 - pvx, 352
 - pvxvoc, 352
- stretch-from, 299
 - hps-pitch-track, 352
 - pvx, 352

- stretch-max, 299
 - hps-pitch-track, 352
 - pvx, 352
- stretch-min, 299
 - hps-pitch-track, 352
 - pvx, 352
- stretch-mode, 65, 68, 214, 262, 299
 - pvxvoc, 352
- stretch-scale, 299
 - hps-pitch-track, 352
 - pvx, 352
- strict, 65, 182, 249, 270, 299
 - pvx, 352
- strict-manifest, 299
 - pvx, 352
- subtype, 65, 299
 - pvx, 352
 - pvxvoc, 352
- suffix, 65, 182, 262, 299
 - pvxvoc, 352
- summary-json, 299
 - pvxanalysis, 352
 - pvxresponse, 352
- sustain, 142, 182, 214, 299
 - pvxenvelope, 352
- target, 299
 - pvxgain, 352
- target-duration, 65, 182, 192, 214, 262, 299
 - pvxtransient, 352
 - pvxvoc, 352
- target-f0, 182, 299
 - pvxvoc, 352
- target-lufs, 182, 192, 262, 299
 - pvxvoc, 352
- target-max, 214, 299
 - pvxreshape, 352
- target-min, 214, 299
 - pvxreshape, 352
- target-pitch-shift-semitones, 299
 - pvxvoc, 352
- target-sample-rate, 299
 - pvxvoc, 352
- time-mask, 299
 - pvxspecaugment, 352
- time-stretch, 65, 68, 78, 182, 214, 262, 299
 - pvxtransient, 352
 - pvxvoc, 352
- time-stretch-factor, 262, 299
 - pvxvoc, 352
- tolerance-cents, 142, 182, 192, 214, 299
 - pvxharmmap, 352
- trajectory-shape, 299
 - pvxtrajectoryreverb, 352
- transform, 78, 182, 192, 262, 299
 - pvxanalysis, 352
 - pvxvoc, 352
- transforms, 262
- transient-crossfade-ms, 65, 68, 182, 214, 299
 - pvxvoc, 352
- transient-mode, 65, 68, 182, 192, 214, 256, 299
 - pvxvoc, 352
- transient-preserve, 52, 65, 78, 262, 299
 - pvxvoc, 352
- transient-protect-ms, 65, 68, 182, 214, 299
 - pvxvoc, 352
- transient-sensitivity, 65, 68, 182, 214, 299
 - pvxvoc, 352
- transient-threshold, 65, 299
 - pvxtransient, 352
 - pvxvoc, 352
- transpose-semitones, 214, 299
 - pvxfilter, 352
- true-peak-max-dbtp, 65, 68, 299
 - pvxvoc, 352
- tv-interp, 142, 182, 192, 214, 299
 - pvxfilter, 352
 - pvxring, 352
- tv-key, 142, 182, 192, 299
 - pvxfilter, 352
- tv-map, 142, 182, 192, 214, 299
 - pvxfilter, 352
 - pvxring, 352
- tv-order, 182, 192, 299
 - pvxfilter, 352
 - pvxring, 352
- variants-per-input, 65, 182, 231, 249, 256, 299
 - pvx, 352
- verbose, 299
 - pvxvoc, 352
- verbosity, 299
 - pvxvoc, 352
- versions, 270
- voices, 65, 182, 192, 262, 299
 - pvxunison, 352
- wave, 65, 182, 299
 - pvxenvelope, 352
- wavelet, 231
- wet, 256, 299
 - pvxrir, 352
 - pvxtrajectoryreverb, 352

- width, 65, 182, 192, 262, 299
 - pvxunison, 352
 - win-length, 65, 78, 142, 182, 192, 262, 299
 - pvxanalysis, 352
 - pvxvoc, 352
 - window, 142, 182, 192, 214, 262, 299
 - pvxanalysis, 352
 - pvxreshape, 352
 - pvxvoc, 352
 - windows, 262
 - work-dir, 299
 - pvx, 352
 - workers, 182, 231, 249, 256, 299
 - pvx, 352
- Clipping, 354
- Cocteau Twins, 342
- Coherent gain, 354
- COLA, 354
- commands
 - hps-pitch-track
 - backend, 352
 - confidence-floor, 352
 - emit, 352
 - feature-set, 352
 - fmax, 352
 - fmin, 352
 - frame-length, 352
 - hop-size, 352
 - mfcc-count, 352
 - output, 352
 - ratio-max, 352
 - ratio-min, 352
 - ratio-reference, 352
 - reference-hz, 352
 - smooth-frames, 352
 - stretch, 352
 - stretch-from, 352
 - stretch-max, 352
 - stretch-min, 352
 - stretch-scale, 352
 - pvx, 52, 65, 68, 70, 72, 78, 136, 142, 146, 182, 192, 203, 204, 214, 219, 231, 249, 256, 262, 270, 271, 299, 320, 322, 329, 344, 352
 - append-manifest, 352
 - audit-metrics, 352
 - backend, 352
 - batch-size, 352
 - bit-depth, 352
 - budget-path, 352
 - chunk-seconds, 352
 - confidence-floor, 352
 - context-ms, 352
 - crossfade-ms, 352
 - dedupe-by, 352
 - device, 352
 - disk-budget, 352
 - dry-run, 352
 - duration, 352
 - emit, 352
 - engine, 352
 - example, 352
 - fail-if-exceeds, 352
 - feature-set, 352
 - fmax, 352
 - fmin, 352
 - frame-length, 352
 - group-separator, 352
 - grouping, 352
 - hop-size, 352
 - intent, 352
 - json, 352
 - keep-intermediate, 352
 - label-policy, 352
 - labels-csv, 352
 - manifest-csv, 352
 - manifest-jsonl, 352
 - material, 352
 - max-length-s, 352
 - mfcc-count, 352
 - mode, 352
 - no-audit-metrics, 352
 - no-progress, 352
 - out, 352
 - output, 352
 - output-csv, 352
 - output-dir, 352
 - output-format, 352
 - output-jsonl, 352
 - output-subtype, 352
 - output-suffix, 352
 - overwrite, 352
 - pair-mode, 352
 - pipeline, 352
 - pipeline-config, 352
 - pitch, 352
 - pitch-conf-min, 352
 - pitch-lowconf-mode, 352
 - pitch-map-crossfade-ms, 352
 - pitch-map-smooth-ms, 352
 - policy, 352
 - quiet, 352

- ratio-max, 352
- ratio-min, 352
- ratio-reference, 352
- reference-hz, 352
- requested-stretch, 352
- resume, 352
- route, 352
- safety-margin, 352
- sample-rate, 352
- seed, 352
- silent, 352
- smooth-frames, 352
- split, 352
- split-mode, 352
- sr, 352
- stretch, 352
- stretch-from, 352
- stretch-max, 352
- stretch-min, 352
- stretch-scale, 352
- strict, 352
- strict-manifest, 352
- subtype, 352
- variants-per-input, 352
- work-dir, 352
- workers, 352
- pvx-cli, 231, 299
- pvxalgorithms, 70, 271
- pvxan, 142, 182, 192, 214, 299
- pvxanalysis, 70, 146, 270, 271, 299
 - analysis-channel, 352
 - cuda-device, 352
 - device, 352
 - hop-size, 352
 - json, 352
 - kaiser-beta, 352
 - n-fft, 352
 - no-center, 352
 - output, 352
 - summary-json, 352
 - transform, 352
 - win-length, 352
 - window, 352
- pvxbandamp, 70, 329
- pvxchordmapper, 70, 329
- pvxcodec, 70, 256, 299
 - bit-depth, 352
 - bits, 352
 - codec, 352
 - output, 352
 - seed, 352
- pvxconform, 70, 72, 182, 299
 - crossfade-ms, 352
 - map, 352
 - resample-mode, 352
- pvxdenoise, 70, 72, 299
 - floor, 352
 - noise-file, 352
 - noise-seconds, 352
 - reduction-db, 352
 - smooth, 352
- pvxdeverb, 70, 72, 299
 - decay, 352
 - floor, 352
 - strength, 352
- pvxenvelope, 70, 146, 299
 - amplitude, 352
 - attack-sec, 352
 - center, 352
 - cycles, 352
 - decay-sec, 352
 - duration, 352
 - duty-cycle, 352
 - end, 352
 - exp-curve, 352
 - format, 352
 - freq, 352
 - frequency-hz, 352
 - key, 352
 - max, 352
 - min, 352
 - mode, 352
 - out, 352
 - output, 352
 - peak, 352
 - phase, 352
 - phase-rad, 352
 - rate, 352
 - release-sec, 352
 - sine-cycles, 352
 - sine-phase-rad, 352
 - start, 352
 - stdout, 352
 - sustain, 352
 - wave, 352
- pvxfilter, 70, 146, 270, 271, 299, 329
 - band-gain-db, 352
 - band-width-bins, 352
 - comp-ratio, 352
 - comp-threshold-db, 352
 - dry-mix, 352
 - expand-ratio, 352

- noise-floor, 352
- operator, 352
- peak-count, 352
- response, 352
- response-gain-db, 352
- response-mix, 352
- shift-bins, 352
- transpose-semitones, 352
- tv-interp, 352
- tv-key, 352
- tv-map, 352
- tv-order, 352
- pvxformant, 70, 270, 271, 299
 - formant-lifter, 352
 - formant-max-gain-db, 352
 - formant-shift-ratio, 352
 - mode, 352
 - pitch-shift-cents, 352
 - pitch-shift-semitones, 352
 - resample-mode, 352
- pvxfreeze, 70, 72, 270, 271, 299
 - duration, 352
 - freeze-time, 352
 - phase-mode, 352
 - random-phase, 352
- pvxgain, 70, 299
 - bit-depth, 352
 - gain, 352
 - normalize, 352
 - output, 352
 - seed, 352
 - target, 352
- pvxharmmap, 70, 146, 299, 329
 - attenuation, 352
 - boost-db, 352
 - chord, 352
 - dry-mix, 352
 - inharmonic-f0-hz, 352
 - inharmonic-mix, 352
 - inharmonic-ity, 352
 - operator, 352
 - root-hz, 352
 - strength, 352
 - tolerance-cents, 352
- pvxharmonize, 70, 72, 299
 - force-stereo, 352
 - gains, 352
 - intervals, 352
 - intervals-cents, 352
 - pans, 352
 - resample-mode, 352
- pvxinharmonator, 70, 329
- pvxlayer, 70, 299
 - harmonic-gain, 352
 - harmonic-kernel, 352
 - harmonic-pitch-cents, 352
 - harmonic-pitch-semitones, 352
 - harmonic-stretch, 352
 - percussive-gain, 352
 - percussive-kernel, 352
 - percussive-pitch-cents, 352
 - percussive-pitch-semitones, 352
 - percussive-stretch, 352
 - resample-mode, 352
- pvxmorph, 70, 72, 299
 - alpha, 352
 - blend-mode, 352
 - envelope-lifter, 352
 - interp, 352
 - mask-exponent, 352
 - normalize-energy, 352
 - order, 352
 - output, 352
 - output-format, 352
 - overwrite, 352
 - phase-mix, 352
 - stdout, 352
- pvxnoise, 70, 256, 299
 - background-dir, 352
 - band, 352
 - bit-depth, 352
 - noise-type, 352
 - output, 352
 - seed, 352
 - snr, 352
- pvxnoisefilter, 70, 329
- pvxreshape, 70, 146, 299
 - exponent, 352
 - factor, 352
 - format, 352
 - input-format, 352
 - interp, 352
 - key, 352
 - max, 352
 - min, 352
 - offset, 352
 - operation, 352
 - order, 352
 - out, 352
 - output, 352
 - output-key, 352
 - rate, 352

- stdout, 352
 - target-max, 352
 - target-min, 352
 - window, 352
- pvxresponse, 70, 146, 299
 - json, 352
 - method, 352
 - normalize, 352
 - output, 352
 - phase-mode, 352
 - smoothing-bins, 352
 - summary-json, 352
- pvxretune, 70, 72, 182, 270, 271, 299
 - a4-reference-hz, 352
 - chunk-ms, 352
 - f0-max, 352
 - f0-min, 352
 - overlap-ms, 352
 - recommend-root, 352
 - resample-mode, 352
 - root, 352
 - root-hz, 352
 - scale, 352
 - scale-cents, 352
 - strength, 352
- pvxrf, 142, 146, 182, 192, 214, 299
- pvxring, 70, 146, 299, 329
 - depth, 352
 - feedback, 352
 - frequency-hz, 352
 - mix, 352
 - operator, 352
 - resonance-decay, 352
 - resonance-hz, 352
 - resonance-mix, 352
 - resonance-q, 352
 - tv-interp, 352
 - tv-map, 352
 - tv-order, 352
- pvxringfilter, 70, 329
- pvxringtvfilter, 70, 329
- pvxrir, 70, 256, 299
 - bit-depth, 352
 - cache-dir, 352
 - category, 352
 - drr, 352
 - ir-database, 352
 - ir-dir, 352
 - ir-file, 352
 - list-databases, 352
 - no-trim, 352
 - output, 352
 - pre-delay-ms, 352
 - rt60, 352
 - seed, 352
 - wet, 352
- pvxspecaugment, 70, 299
 - bit-depth, 352
 - fill, 352
 - freq-mask, 352
 - hop-length, 352
 - n-fft, 352
 - num-freq-masks, 352
 - num-time-masks, 352
 - output, 352
 - seed, 352
 - time-mask, 352
- pvxspeccompander, 70, 329
- pvxtrajectoryreverb, 70, 299
 - coord-system, 352
 - distance-law, 352
 - dry, 352
 - end, 352
 - ir, 352
 - no-normalize-gains, 352
 - normalize-gains, 352
 - speaker-angles, 352
 - start, 352
 - trajectory-shape, 352
 - wet, 352
- pvxtransient, 70, 299
 - pitch-shift-cents, 352
 - pitch-shift-ratio, 352
 - pitch-shift-semitones, 352
 - resample-mode, 352
 - target-duration, 352
 - time-stretch, 352
 - transient-threshold, 352
- pvxtvfilter, 70, 329
- pvxunison, 70, 299
 - detune-cents, 352
 - dry-mix, 352
 - resample-mode, 352
 - voices, 352
 - width, 352
- pvxvoc, 70, 72, 231, 256, 270, 271, 299, 320
 - ambient-phase-mix, 352
 - ambient-preset, 352
 - analysis-channel, 352
 - auto-profile, 352
 - auto-profile-lookahead-seconds, 352
 - auto-segment-seconds, 352

- auto-transform, 352
- bit-depth, 352
- cents, 352
- checkpoint-dir, 352
- checkpoint-id, 352
- clip, 352
- coherence-strength, 352
- compander-attack-ms, 352
- compander-compress-ratio, 352
- compander-expand-ratio, 352
- compander-makeup-db, 352
- compander-release-ms, 352
- compander-threshold-db, 352
- compressor-attack-ms, 352
- compressor-makeup-db, 352
- compressor-ratio, 352
- compressor-release-ms, 352
- compressor-threshold-db, 352
- control-stdin, 352
- cpu, 352
- cuda-device, 352
- device, 352
- dither, 352
- dither-seed, 352
- dry-run, 352
- example, 352
- expander-attack-ms, 352
- expander-ratio, 352
- expander-release-ms, 352
- expander-threshold-db, 352
- explain-plan, 352
- extreme-stretch-threshold, 352
- extreme-time-stretch, 352
- f0-max, 352
- f0-min, 352
- force, 352
- formant-lifter, 352
- formant-max-gain-db, 352
- formant-strength, 352
- fourier-sync, 352
- fourier-sync-max-fft, 352
- fourier-sync-min-fft, 352
- fourier-sync-smooth, 352
- gpu, 352
- guided, 352
- hard-clip-level, 352
- hop-size, 352
- interp, 352
- kaiser-beta, 352
- limiter-threshold, 352
- manifest-append, 352
- manifest-json, 352
- max-stage-stretch, 352
- metadata-policy, 352
- multires-ffts, 352
- multires-fusion, 352
- multires-weights, 352
- n-fft, 352
- no-center, 352
- no-onset-realign, 352
- no-progress, 352
- normalize, 352
- onset-credit-max, 352
- onset-credit-pull, 352
- onset-time-credit, 352
- order, 352
- out, 352
- output, 352
- output-dir, 352
- output-format, 352
- overwrite, 352
- peak-dbfs, 352
- phase-engine, 352
- phase-locking, 352
- phase-random-seed, 352
- pitch, 352
- pitch-conf-min, 352
- pitch-follow-stdin, 352
- pitch-lowconf-mode, 352
- pitch-map, 352
- pitch-map-crossfade-ms, 352
- pitch-map-smooth-ms, 352
- pitch-map-stdin, 352
- pitch-mode, 352
- pitch-shift-cents, 352
- pitch-shift-ratio, 352
- pitch-shift-semitones, 352
- preset, 352
- quality-profile, 352
- quiet, 352
- ratio, 352
- ref-channel, 352
- resample-mode, 352
- resume, 352
- rms-dbfs, 352
- route, 352
- semitones, 352
- silent, 352
- soft-clip-drive, 352
- soft-clip-level, 352
- soft-clip-type, 352
- stdout, 352

- stereo-mode, 352
- stretch, 352
- stretch-mode, 352
- subtype, 352
- suffix, 352
- target-duration, 352
- target-f0, 352
- target-lufs, 352
- target-pitch-shift-semitones, 352
- target-sample-rate, 352
- time-stretch, 352
- time-stretch-factor, 352
- transform, 352
- transient-crossfade-ms, 352
- transient-mode, 352
- transient-preserve, 352
- transient-protect-ms, 352
- transient-sensitivity, 352
- transient-threshold, 352
- true-peak-max-dbtp, 352
- verbose, 352
- verbosity, 352
- win-length, 352
- window, 352
- pvxwarp, 70, 72, 182, 270, 271, 299
 - crossfade-ms, 352
 - map, 352
 - resample-mode, 352
- Compact Feature Set (Smaller CSV), 202
- Comparative listening worksheet, 344
- Comparative Window Summary, 83
- Compare phase locking modes, 218
- Compatibility Shims, 70
- Complete Window Atlas, 87
- Complex number, 354
- Complex Policy Example Equivariant Music Labels, 247
- Complex Policy Example Robust Speech with Auditable Views, 246
- Complex spectrum, 354
- composers
 - Adams, John, 341
 - Anderson, Laurie, 342
 - Anonymous, 333
 - Aphex Twin, 338
 - Autechre, 339
 - Bach, Johann Sebastian, 334, 335, 337, 339
 - Barber, Samuel, 343
 - Bartok, Bela, 340
 - Basinski, William, 344
 - Beethoven, Ludwig van, 335, 337, 339, 343
 - Berio, Luciano, 334
 - Berlioz, Hector, 339
 - Bingen, Hildegard von, 333
 - Bjork, 342
 - Boards of Canada, 342
 - Brahms, Johannes, 339
 - Britten, Benjamin, 340
 - Brown, James, 338
 - Bruckner, Anton, 336
 - Bush, Kate, 342
 - Cage, John, 338, 343
 - Carlos, Wendy, 332
 - Chopin, Frederic, 335, 337
 - Cocteau Twins, 342
 - Dave Brubeck Quartet, 338
 - Debussy, Claude, 336, 340
 - Derbyshire, Delia, 341
 - Eno, Brian, 341
 - Handel, George Frideric, 334
 - Harvey, Jonathan, 331
 - Holst, Gustav, 336
 - Ives, Charles, 340
 - Kraftwerk, 338
 - Kuchera-Morin, JoAnn, 331
 - Lansky, Paul, 333
 - Ligeti, Gyorgy, 336, 340
 - Lucier, Alvin, 344
 - Mahler, Gustav, 334, 340
 - Messiaen, Olivier, 336
 - Monteverdi, Claudio, 333
 - Mozart, Wolfgang Amadeus, 334, 339
 - Murail, Tristan, 332
 - My Bloody Valentine, 342
 - Oneohtrix Point Never, 343
 - Pachelbel, Johann, 343
 - Paganini, Niccolo, 337
 - Palestrina, Giovanni Pierluigi da, 339
 - Part, Arvo, 336
 - Pink Floyd, 341
 - Prez, Josquin des, 333
 - Public Enemy, 338
 - Purcell, Henry, 333
 - Radiohead, 342
 - Ravel, Maurice, 336, 343
 - Reich, Steve, 335, 344
 - Reynolds, Roger, 331
 - Richter, Max, 344
 - Rimsky-Korsakov, Nikolai, 337
 - Risset, Jean-Claude, 332
 - Saariaho, Kaija, 332
 - Schoenberg, Arnold, 334

- Schubert, Franz, 334, 335
- Squarepusher, 338
- Stockhausen, Karlheinz, 341
- Stravinsky, Igor, 337, 340
- Tallis, Thomas, 333
- Tavener, John, 337
- The Beatles, 341
- The Orb, 342
- Varese, Edgard, 338
- Vivaldi, Antonio, 335
- Wagner, Richard, 334, 335, 343
- Williams, Ralph Vaughan, 336
- Wishart, Trevor, 330–332
- Xenakis, Iannis, 340
- Compositional histories beyond the canonical examples, 39
- Computer music and the widening of purpose, 13
- confidence - subtle pitch bend depth, 194
- Confidence-Gated External Pitch Control via Pipe, 167
- Conform CSV with per-segment ratios, 259
- Conjugate symmetry, 354
- Conservative Field-Restoration Chain, 169
- Contents, 220, 250
- Contrastive and Self-Supervised Views, 241
- Contrastive Paired Views for Self-Supervised Learning, 180
- Contrastive Planning-Only Pass, 234
- Contrastive view generation (SSL), 224
- Control-Map Authoring With Envelope + Reshape, 209
- Control-Rate Interpolation Graph Examples, 316
- Convolution, 354
- Core Command, 232
- cosine, 134
- cosine power 2, 123
- cosine power 3, 124
- cosine power 4, 125
- Create Chorus/Unison Thickness, 152
- creative spectral effects, 302
- Cross-synthesis, 354
- Crossfade, 354
- cubic, 350
- CUDA Attempt with Deterministic CPU Fallback Workflow, 166
- Custom cents map retune, 259
- CZT for Non-Power-of-Two Frame Setup, 161
- Daphnis et Chloe, Lever du jour*, 336
- Data Product Map, 318
- Dave Brubeck Quartet, 338
- dBFS, 354
- DCT timbral compaction for smooth harmonic material, 262
- De-Noise Field Recording, 152
- De-Reverb a Room Recording, 153
- Debug Workflow, 68
- Debussy, Claude, 336, 340
- Decay, 354
- Default production backend (FFT + transient protection), 261
- denoise and restoration, 303
- Denoise without musical-noise artifacts, 67
- Denoising with Phase-Consistent STFT Processing, 57
- Der Holle Rache*, 334
- Derbyshire, Delia, 341
- dereverb and room correction, 304
- Designing the Augmentation Distribution, 237
- Desintegrations*, 332
- Detecting Shortcuts and Augmentation Artifacts, 245
- Determinism and Reproducibility, 225
- Determinism and Seed Architecture, 239
- Deterministic AI Dataset Augmentation (pvx augment), 180
- Deterministic render, 354
- Development Installation, 267
- DFT, 354
- Diagnose Environment and Install Issues (pvx doctor), 178
- Dict-based map for keyed datasets, 225
- Dido's Lament*, 333
- Dies irae*, 333
- Different Trains*, 335
- Digital Moonscapes*, 332
- Docs Guide, 70
- Doctor Who Theme*, 341
- Documentation Build Graph, 79
- Documentation Build Pipeline, 320
- Documented phase-vocoder practice, 330
- Doppler transposition, 22
- Dreampaths*, 331
- Drum-safe stretch, 67
- Drums Safe Stretch, 157
- Dry signal, 354
- Dry-run output validation, 258
- DST odd-symmetry color pass, 262
- Dudley, Homer, 8
- Dynamic Analysis Resolution (N-FFT + Hop Size Maps), 172
- Dynamic Control Interpolation (CSV/JSON), 76

-
- Dynamic Pitch Ratio via JSON + Polynomial Interpolation, 170
 - Dynamic range, 355
 - Dynamic Stretch via Direct CSV Flag Input, 170
 - dynamics and loudness, 305
 - dynamics transfer curves, 346
 - Electronic studio and ambient listening, 341
 - Eltro information rate changer, 20
 - End-to-End pvxvoc Flow, 315
 - Energy normalization, 355
 - Enhancement and source separation, 243
 - Eno, Brian, 341
 - Entries Still Using Scholar Links, 323
 - Envelope, 355
 - Envelope follower, 355
 - Equalization, 355
 - ERB, 355
 - Erbarme dich*, 334
 - Erlkonig*, 334
 - Estimate Stretch Budget Before Extreme Jobs, 62
 - Etude Op. 10 No. 4*, 337
 - Evaluation Beyond One Aggregate Score, 244
 - Everything in Its Right Place*, 342
 - exact blackman, 94
 - Exaggerate movement with power law, 199
 - Example Audio Scenarios, 59
 - Example Workflows, 234
 - Experimental in 0.1.x, 69
 - exponential, 350
 - exponential 0p25, 117
 - exponential 0p5, 118
 - exponential 1p0, 119
 - Exponential interpolation, 355
 - Extra Beginner Recipes (Quick Wins), 64
 - Extreme Ambient Stretch (10-minute target), 158
 - Extreme duration and temporal form, 343
 - Extreme duration and the change of listening scale, 46
 - Extreme Stretch Multistage Strategy, 317
 - Extreme Stretch with Checkpointed Recovery, 165
 - Extreme stretch with multistage strategy, 259
 - Extreme Time Stretch Ambient Pad, 150
 - Extreme Time-Scale Madness, 205
 - f0 hz - pitch ratio, 194
 - Fairness Consent and Dataset Governance, 247
 - Fantasia on a Theme by Thomas Tallis*, 336
 - Fast End-to-End Health Check (pvx smoke), 179
 - Fast Starting Presets, 66
 - Feature Tracking for Sidechain Control, 63
 - Feature-Vector Recipes (MFCC + MPEG-7), 199
 - FFT, 355
 - FFT size, 11, 355
 - File subtype, 355
 - Filter, 355
 - First 3 Commands to Run, 55
 - First Construction in Metal*, 338
 - Flanagan and Golden in the Bell Labs setting, 25
 - Flanagan and Golden phase as transmitted information, 12
 - Flanagan s digital formulation, 10
 - Flanagan, James L., 10
 - flattop, 92
 - Flight of the Bumblebee*, 337
 - Floor, 355
 - Formant, 355
 - Formant + onset, 196
 - Formant 2 + spectral spread, 198
 - Formant preservation, 355
 - Formula Families, 80
 - Formula Version Policy, 270
 - Frame, 355
 - Frame rate, 355
 - Freeze + Morph Experiments, 206
 - Freeze a Harmonic Moment, 150
 - Frequency resolution, 355
 - frequency shifting
 - compared with pitch shifting, 23
 - From offline render to interactive instrument, 32
 - From research technique to musical tool, 11
 - From waveform to WOLA frames, 3
 - Full Audio Container List (Current Build), 71
 - Full Mastering Chain in One Render, 162
 - Function Graph Gallery, 78, 317
 - Fundamental frequency, 355
 - Funky Drummer*, 338
 - Gabor, Dennis, 22
 - Gain, 355
 - Gantz Graf*, 339
 - gaussian 0p25, 108
 - gaussian 0p35, 109
 - gaussian 0p45, 110
 - gaussian 0p55, 111
 - gaussian 0p65, 112
 - general gaussian 1p5 0p35, 113
 - general gaussian 2p0 0p35, 114
 - general gaussian 3p0 0p35, 115
 - general gaussian 4p0 0p35, 116
 - general hamming 0p50, 129
 - general hamming 0p60, 130
 - general hamming 0p70, 131
 - general hamming 0p80, 132

- Generate a Stretch Envelope (Function Stream), 173
- Generative and neural-vocoder training, 243
 - Gesang der Junglinge*, 341
- glossary, 353
- GPU Acceleration, 154
- GPU-Accelerated Transforms (PyTorch), 229
- granular and modulation, 306
- graphs
 - dynamics transfer curves, 346
 - mask exponent response, 348
 - morph blend magnitude curves, 347
 - phase-mix angle response, 348
 - pitch ratio from cents, 346
 - pitch ratio from semitones, 345
 - soft-clip transfer functions, 347
- Griffin-Lim algorithm, 356
- Group delay, 356
- Group-Level Summary, 300
- Guide A controls pitch guide B controls stretch, 201

- HAL 9000, 22
- hamming, 88
- Handel, George Frideric, 334
- hann, 87
- hann poisson 0p5, 126
- hann poisson 1p0, 127
- hann poisson 2p0, 128
- Harmonic, 356
- Harmonic analysis before electronic sound, 17
- Harmonic Chord Mapping with pvx chordmapper, 177
- Harmonic inharmonic and spatial design, 189
- harmonic ratio - harmonicity-linked pitch, 195
- Harmonic-percussive separation, 356
- Harmonize Then Widen in One Pipeline, 164
- Harmonize Triad Stack, 156
- Hartley real-basis exploratory render, 262
- Harvey, Jonathan, 331
- Hertz, 356
- Holst, Gustav, 336
- Homer Dudley the vocoder and the Voder, 18
- Hop ratio, 356
- Hop size, 356
- Horizontal coherence and vertical coherence, 14
- Host, 264
- How pvx puts the idea to work, 52
- How to use the library, 330
- hps-pitch-track, 273
- HuggingFace Datasets Integration, 223
- Hybrid Transient Engine, 315
- Hybrid Transient Mode (Manual Tuning), 158
- Hybrid transient mode (recommended on speech/-drums), 61

- I Am Sitting in a Room*, 344
- I/O Behavior by Category, 71
- Ich bin der Welt abhanden gekommen*, 334
- Identity phase locking, 356
- Immediate Next Steps, 145
- Implemented in Phase 1 and Phase 2, 144
- Implemented in Phase 3 Phase 4 and Phase 5, 144
- Implemented in Phase 6 and Phase 7, 145
- Import Paths, 216
- Impulse Response Database, 228
- Increase MFCC dimensionality to 20, 200
- Independent cents retune, 259
- Inharmonic Spectral Warping with pvx inharmonator, 178
- inharmonicity - anti-harmonic pitch detune, 196
- Inharmonique*, 332
- Inspect Feature Columns Before Routing, 193
- Installation, 220
- Instantaneous frequency, 356
- Integrated loudness targeting with limiter, 258
- Intent Preset Pipelines, 221
- Intent Presets, 62
- Intent Profiles, 232
- Interpolation, 356
- interpolation
 - cubic, 350
 - exponential, 350
 - linear, 349
 - nearest neighbour, 349
 - polynomial order 1, 351
 - polynomial order 2, 351
 - polynomial order 3, 352
 - polynomial order 5, 352
 - sample and hold, 349
 - smootherstep, 351
 - smoothstep S-curve, 350
- Interpretation and use, 87–136
- Invariance Equivariance and Invalid Transformations, 236
- Inverse FFT, 356
- Inverse STFT and weighted overlap-add, 7
- Invert and clamp, 199
- Ionisation*, 338
- IRCAM and the institutionalization of spectral transformation, 27
- IRCAM as a meeting of research and commission, 43
- Irrational Pitch Ratio Input, 160

-
- Ives, Charles, 340
 James A. Moorer and the computer-music turn, 25
 Jean Laroche and Mark Dolson explaining phasiness, 14
 JoAnn Kuchera-Morin and expanded duration, 28
 Jonathan Harvey and spectral ambiguity, 27
 JSON Manifest + Automation-Friendly Logging, 157
 Jupyter Notebook Snippets, 217
 Just-Intonation CSV Conform Map, 161
 Just-Ratio Pitch Input, 160
 kaiser, 135
 Kaiser beta, 356
 Keyframe, 356
 Keyword Spotting / Wake-Word Detection, 251
Kontakte, 341
 Koonce, Paul, 36
 Kraftwerk, 338
 Kuchera-Morin, JoAnn, 331
La mer, Dialogue du vent et de la mer, 340
 Laboratory one a reusable spectral imprint, 141
 Laboratory three spectral resonance and harmonic mapping, 142
 Laboratory two control data as compositional material, 141
Lamento d'Arianna, 333
 lanczos, 106
 Lansky, Paul, 333
 Laroche and Dolson from artifact to explanatory model, 29
Lascia ch'io pianga, 334
 Latency, 356
 Launch-Ready Helper Commands, 54
 Layer Harmonic and Percussive Paths, 156
 Leakage, 356
 Lesson five scripts are part of the instrument, 140
 Lesson four expose the processing choice that changes the sound, 140
 Lesson one small commands can form a broad language, 138
 Lesson six stable releases need an experimental edge, 140
 Lesson three intermediate artifacts support musical memory, 139
 Lesson two time-varying control belongs in the foundation, 138
 LFO, 356
Lichtbogen, 332
 Ligeti, Gyorgy, 336, 340
 linear, 349
 Linear phase, 356
 Link-Type Summary, 323
 listening guide, 330
 Adams, John
 Short Ride in a Fast Machine, 341
 Anderson, Laurie
 O Superman, 342
 Anonymous
 Dies irae, 333
 Aphex Twin
 Vordhosbn, 338
 Autechre
 Gantz Graf, 339
 Bach, Johann Sebastian
 Air on the G String, 335
 Erbarne dich, 334
 Prelude in C major, BWV 846, 337
 The Art of Fugue, Contrapunctus I, 339
 Barber, Samuel
 Adagio for Strings, 343
 Bartok, Bela
 Music for Strings, Percussion and Celesta, third movement, 340
 Basinski, William
 The Disintegration Loops 1.1, 344
 Beethoven, Ludwig van
 Piano Sonata No. 14, third movement, 337
 Symphony No. 5, opening, 343
 Symphony No. 7, Allegretto, 335
 Symphony No. 9, finale, 339
 Berio, Luciano
 Sequenza III, 334
 Berlioz, Hector
 Symphonie fantastique, Marche au supplice, 339
 Bingen, Hildegard von
 O vis aeternitatis, 333
 Bjork
 All Is Full of Love, 342
 Boards of Canada
 Roygbiv, 342
 Brahms, Johannes
 Symphony No. 4, finale, 339
 Britten, Benjamin
 The Young Person's Guide to the Orchestra, 340
 Brown, James
 Funky Drummer, 338
 Bruckner, Anton
 Symphony No. 7, Adagio, 336

- Bush, Kate
 Running Up That Hill, 342
- Cage, John
 First Construction in Metal, 338
 Organ2/ASLSP, 343
- Carlos, Wendy
 Digital Moonscapes, 332
- Chopin, Frederic
 Etude Op. 10 No. 4, 337
 Nocturne in E-flat major, Op. 9 No. 2, 335
- Cocteau Twins
 Lorelei, 342
- Dave Brubeck Quartet
 Take Five, 338
- Debussy, Claude
 La mer, Dialogue du vent et de la mer, 340
 Prelude a l'apres-midi d'un faune, 336
- Derbyshire, Delia
 Doctor Who Theme, 341
- Eno, Brian
 An Ending (Ascent), 341
- Handel, George Frideric
 Lascia ch'io pianga, 334
- Harvey, Jonathan
 Mortuos Plango, Vivos Voco, 331
- Holst, Gustav
 Neptune, the Mystic, 336
- Ives, Charles
 The Unanswered Question, 340
- Kraftwerk
 Numbers, 338
- Kuchera-Morin, JoAnn
 Dreampaths, 331
- Lansky, Paul
 *Six Fantasies on a Poem by Thomas Cam-
 pion*, 333
- Ligeti, Gyorgy
 Atmospheres, 340
 Lux Aeterna, 336
- Lucier, Alvin
 I Am Sitting in a Room, 344
- Mahler, Gustav
 Ich bin der Welt abhanden gekommen, 334
 Symphony No. 2, finale, 340
- Messiaen, Olivier
 Louange a l'Eternite de Jesus, 336
- Monteverdi, Claudio
 Lamento d'Arianna, 333
- Mozart, Wolfgang Amadeus
 Der Holle Rache, 334
 Symphony No. 40, first movement, 339
- Murail, Tristan
 Desintegrations, 332
- My Bloody Valentine
 Only Shallow, 342
- Oneohtrix Point Never
 Replica, 343
- Pachelbel, Johann
 Canon in D, 343
- Paganini, Niccolo
 Caprice No. 24, 337
- Palestrina, Giovanni Pierluigi da
 Missa Papae Marcelli, Kyrie, 339
- Part, Arvo
 Spiegel im Spiegel, 336
- Pink Floyd
 On the Run, 341
- Prez, Josquin des
 Ave Maria virgo serena, 333
- Public Enemy
 Bring the Noise, 338
- Purcell, Henry
 Dido's Lament, 333
- Radiohead
 Everything in Its Right Place, 342
- Ravel, Maurice
 Bolero, 343
 Daphnis et Chloe, Lever du jour, 336
- Reich, Steve
 Different Trains, 335
 Piano Phase, 344
- Reynolds, Roger
 Transfigured Wind I, 331
- Richter, Max
 Sleep, 344
- Rimsky-Korsakov, Nikolai
 Flight of the Bumblebee, 337
- Risset, Jean-Claude
 Inharmonique, 332
 Songes, 332
- Saariaho, Kaija
 Lichtbogen, 332
 NoaNoa, 332
- Schoenberg, Arnold
 Mondestrunken, 334
- Schubert, Franz
 Erlkonig, 334
 String Quintet in C, Adagio, 335
- Squarepusher
 My Red Hot Car, 338
- Stockhausen, Karlheinz
 Gesang der Junglinge, 341

-
- Kontakte*, 341
 - Stravinsky, Igor
 - The Firebird, Infernal Dance*, 340
 - The Rite of Spring, Augurs of Spring*, 337
 - Tallis, Thomas
 - Spem in alium*, 333
 - Tavener, John
 - The Protecting Veil*, 337
 - The Beatles
 - Tomorrow Never Knows*, 341
 - The Orb
 - Little Fluffy Clouds*, 342
 - Varese, Edgard
 - Ionisation*, 338
 - Vivaldi, Antonio
 - Winter, Largo*, 335
 - Wagner, Richard
 - Mild und leise*, 334
 - Prelude to Das Rheingold*, 335
 - Prelude to Tristan und Isolde*, 343
 - Williams, Ralph Vaughan
 - Fantasia on a Theme by Thomas Tallis*, 336
 - Wishart, Trevor
 - Red Bird*, 331
 - Supernova*, 331
 - Tongues of Fire*, 332
 - Vox 5*, 330
 - Vox Cycle*, 331
 - Xenakis, Iannis
 - Metastaseis*, 340
 - Little Fluffy Clouds*, 342
 - Long renders comparison and reproducibility, 191
 - Lorelei*, 342
 - Louange a l'Eternite de Jesus*, 336
 - Loudness, 356
 - Loudness and Dynamics, 77
 - Loudness proxy + transient mask, 198
 - Lower Pitch With Formant Preservation, 150
 - Lucier, Alvin, 344
 - Lux Aeterna*, 336
 - Machine learning enters the surrounding workflow, 32
 - Magnitude, 357
 - Magnitude spectrum, 357
 - Mahler, Gustav, 334, 340
 - Main lobe, 357
 - Maintainer Release Flow, 270
 - Managed One-Line Chain (No Manual Pipe Wiring), 169
 - Manifest Fields, 233
 - Manifest merge + stats, 235
 - Manifest Validation and Merge, 181
 - Manual Pipe Variants, 202
 - Mark Dolson and the tutorial synthesis, 13
 - Mark Dolson research software and pedagogy, 26
 - Mask, 357
 - mask exponent response, 348
 - Masking, 357
 - Mastering, 258
 - Mastering Chain Block Diagram, 317
 - Messiaen, Olivier, 336
 - Metadata, 357
 - Metastaseis*, 340
 - Metrics Printing Path, 317
 - MFCC + MPEG-7 audio spectrum envelope band, 199
 - MFCC + MPEG-7 spectral flux, 196
 - MFCC second coefficient driver, 199
 - Microtonal, 258
 - Microtonal CSV Interpolation Flow, 319
 - Microtonal Just-Ratio Map Playback, 167
 - Microtonal Map Processing, 316
 - Microtonal Retune Mapping, 77
 - Mid-side, 357
 - Mid/Side Lock with Formant-Preserving Pitch Shift, 167
 - Mid/Side Stereo Coherence Stretch, 160
 - Mild und leise*, 334
 - Minimal Pitch-Shift (Duration-Preserving), 216
 - Minimal Public-Demo Sequence (pvx quickstart), 178
 - Minimal Time-Stretch Example, 216
 - Missa Papae Marcelli, Kyrie*, 339
 - Mix online and offline augmentation, 228
 - Mixing GPU and NumPy transforms, 229
 - Moderate vocal stretch with formant preservation, 259
 - modulator signal, 2
 - Module Layout, 328
 - Mondestrunken*, 334
 - Mono, 357
 - Monteverdi, Claudio, 333
 - Morph - formant - unison, 260
 - morph blend magnitude curves, 347
 - Morph Then Freeze for Hybrid Pads, 166
 - Morph Two Sounds, 151
 - Mortuos Plango, Vivos Voco*, 331
 - Moving the frames without moving their pitch, 6
 - Mozart, Wolfgang Amadeus, 334, 339
 - MPEG-7 attack-time + MFCC driver, 198
 - MPEG-7 centroid + MPEG-7 spread, 198
 - MPEG-7 spectral crest and decrease, 200

- Multi-Feature Recipes (Different Features - Different Targets), 196
- multi-resolution analysis, 11
- Multi-Resolution Fusion Fan-In, 319
- Multi-Resolution Fusion Render, 155
- Multi-resolution fusion stretch, 260
- Multi-Resolution Fusion With Explicit Weights, 168
- Multi-Stage Pipelines Streaming and Batch Tricks, 212
- Multi-Tool Pipe with Streaming I/O, 163
- Multichannel, 357
- Multiple Guide Files (A and B both influence target), 201
- Multiresolution analysis, 357
- Murail, Tristan, 332
- Music Classification and Tagging, 253
- Music for Strings, Percussion and Celesta, third movement*, 340
- Music information retrieval, 243
- Music Information Retrieval Set, 234
- Music Source Separation, 256
- My Bloody Valentine, 342
- My Red Hot Car*, 338
- N-TET CSV Map (19-TET) with pvxconform, 161
- nearest neighbour, 349
- Neptune, the Mystic*, 336
- New Quality Modes (Beginner Version), 61
- Next Steps, 63
- NoaNoa*, 332
- Nocturne in E-flat major, Op. 9 No. 2*, 335
- Noise ambience and environmental recordings, 45
- Noise floor, 357
- Noise residuals and the limits of sinusoidal phase, 31
- Noise-aware hiss + hum, 197
- Nonstationary, 357
- Normalized frequency, 357
- Notation, 74, 80
- Notes, 262
- Numbers*, 338
- nuttall, 91
- Nyquist frequency, 357
- O Superman*, 342
- O vis aeternitatis*, 333
- Offline pre-computation (recommended for large datasets), 226
- Old vs New, 204
- On the Run*, 341
- One-liner convenience, 224
- Oneohtrix Point Never, 343
- Online augmentation with worker processes, 226
- Online Offline and Hybrid Augmentation, 241
- Only Shallow*, 342
- Onset, 357
- onset strength - attack-reactive stretch, 195
- Open tools standards and software migration, 31
- option values
 - 32f, 352
 - 16, 352
 - 24, 352
 - 32, 352
 - acf, 352
 - advanced, 352
 - aiff, 352
 - all, 352
 - allow alter, 352
 - am radio, 352
 - ambient, 352
 - arctan, 352
 - asr robust, 352
 - auto, 352
 - bandlimited, 352
 - basic, 352
 - bin, 352
 - brown, 352
 - caf, 352
 - cartesian, 352
 - contrastive2, 352
 - cpu, 352
 - csv, 352
 - cubic, 352
 - cuda, 352
 - drums, 352
 - ease-in, 352
 - ease-in-out, 352
 - ease-out, 352
 - f0 hz, 352
 - fft, 352
 - first, 352
 - flac, 352
 - FLOAT, 352
 - float, 352
 - formant-preserving, 352
 - gaussian, 352
 - hold, 352
 - hybrid, 352
 - hz, 352
 - identity, 352
 - independent, 352
 - inherit, 352

- instantaneous, 352
- interp, 352
- inv pitch ratio, 352
- inverse, 352
- inverse-square, 352
- json, 352
- label balanced, 352
- linear, 352
- lo fi, 352
- mean, 352
- median, 352
- mid side lock, 352
- mir music, 352
- mix, 352
- mp3 low, 352
- mp3 medium, 352
- mps, 352
- multistage, 352
- none, 352
- off, 352
- ogg, 352
- PCM 16, 352
- PCM 24, 352
- PCM 32, 352
- peak, 352
- pink, 352
- pitch map, 352
- pitch ratio, 352
- pitch to stretch, 352
- preserve, 352
- pvx-cli, 352
- pyin, 352
- pytorch, 352
- random, 352
- ref channel lock, 352
- reset, 352
- rms, 352
- shift, 352
- speaker balanced, 352
- speech, 352
- spherical, 352
- ssl contrastive, 352
- standard, 352
- stateful, 352
- stem-prefix, 352
- stretch map, 352
- tanh, 352
- telephone, 352
- torchaudio, 352
- unity, 352
- vocal, 352
- voip narrow, 352
- voip wide, 352
- wav, 352
- wavelet, 352
- white, 352
- wrapper, 352
- wsola, 352
- Organ2/ASLSP*, 343
- Output Policy Controls, 63
- Overlap, 357
- Overlap-add, 357
- Oversampling, 357
- Pachelbel, Johann, 343
- Paganini, Niccolo, 337
- Paired and Structured Targets, 242
- Palestrina, Giovanni Pierluigi da, 339
- Parameter Ranges That Usually Work, 68
- Part, Arvo, 336
- Partial, 357
- parzen, 105
- Peak bin, 358
- Peak locking, 358
- Per-Signal Results, 265
- Percussion and the historical challenge of the on-set, 45
- Performance Tips, 226
- Persist STFT Analysis Artifact (PVXAN), 174
- Persistent spectral artifacts, 188
- Phase, 358
- Phase 1 Coherence-aware phase propagation in core, 321
- Phase 2 Adaptive coherence policy, 321
- Phase 3 Design, 328
- Phase 3 Stereo/multichannel coherence extension, 322
- Phase 4 Benchmark and regression gates, 322
- Phase 4 Design, 329
- Phase 5 Design, 329
- Phase 5 UX and docs, 322
- Phase coherence, 358
- Phase increment, 358
- phase locking, 11
- Phase propagation, 358
- Phase Propagation and Locking, 315
- Phase reset, 358
- Phase Reset vs Hybrid Boundary Behavior, 318
- Phase unwrapping, 358
- Phase vocoder, 358
- phase vocoder, 2–52
- phase-mix angle response, 348

- Phase-Vocoder Augmentation as a Special Case, 238
- Phase-vocoder core, 259
- Phase-Vocoder Frequency and Phase Update, 76
- Phased implementation, 321
- Phasiness, 358
- phasiness, 3
- Piano Phase*, 344
- Piano Sonata No. 14, third movement*, 337
- Pink Floyd, 341
- Pipe Chaining Pattern, 318
- Pipe Multiple Tools in One Line, 154
- Pipelines, 260
- Pitch and melody estimation, 243
- pitch detection and tracking, 307
- Pitch Estimation and Melody Extraction, 254
- Pitch Harmony and Tuning Extremes, 207
- Pitch ratio, 358
- pitch ratio - stretch (inverse motion), 194
- pitch ratio from cents, 346
- pitch ratio from semitones, 345
- Pitch Shift with Formant Preservation, 158
- Pitch shifting, 358
- pitch shifting, 2
- Pitch shifting harmonising and spectral translation, 15
- pitch stability - micro-variation amount, 196
- Pitch-follow sidechain map (A controls B), 261
- Pitch-Map Confidence Gate, 319
- Plan-Only Diagnostic (No Render), 162
- Policy File Template, 235
- polynomial order 1, 351
- polynomial order 2, 351
- polynomial order 3, 352
- polynomial order 5, 352
- Polyphony and orchestral density, 339
- Portnoff and the practical digital formulation, 25
- Portnoff the FFT makes the method practical, 12
- Portnoff, Mark, 10
- Practical Recipes, 67
- Pre-echo, 358
- Prelude a l'apres-midi d'un faune*, 336
- Prelude in C major, BWV 846*, 337
- Prelude to Das Rheingold*, 335
- Prelude to Tristan und Isolde*, 343
- Preservation reconstruction and obsolete systems, 47
- Prez, Josquin des, 333
- Prime-size frame experiment with CZT backend, 261
- Processing Contracts, 328
- Public Enemy, 338
- Publication and Handoff Checklist, 247
- Puckette phase locking and real-time environments, 29
- Purcell, Henry, 333
- PVC as a working environment, 137
- PVC package, 36
- pvx, 274
- pvx Window Reference, 80
- PVXAN, 144
- pvxanalysis, 279
- PvxAugmentDataset, 222
- PvxAugmentTransform (torchvision-style), 222
- pvxcodec, 279
- pvxconform, 280
- pvxdenoise, 280
- pvxdeverb, 280
- pvxenvelope, 280
- pvxfilter, 282
- pvxformant, 283
- pvxfreeze, 283
- pvxgain, 283
- pvxharmmap, 284
- pvxharmonize, 284
- pvxlayer, 285
- pvxmorph, 285
- pvxnoise, 286
- pvxreshape, 286
- pvxresponse, 287
- pvxretune, 288
- PVXRF, 144, 358
- pvxring, 288
- pvxrir, 289
- pvxspecaugment, 290
- pvxtrajectoryreverb, 290
- pvxtransient, 291
- pvxunison, 291
- pvxvoc, 292
- pvxwarp, 299
- PyTorch Integration, 222
- QA Stress and Reproducibility Challenges, 213
- Quality Control Before Training, 245
- Quality metrics on processed speech, 257
- Quality regression check, 257
- Quality-First First Pass (pvx safe), 179
- Quality-First Optimization Loop, 319
- Quantitative Metrics, 83
- Quantization, 358
- Quick Setup (Install + PATH), 54
- Quick Start, 193
- Quick Start Pure Python, 221

-
- Radiohead, 342
 - Raise Pitch Without Changing Speed, 149
 - Ravel, Maurice, 336, 343
 - Reading frequency from phase advance, 5
 - Reading the foundational papers as historical documents, 33
 - Reading the history through software design, 50
 - Real-time factor, 358
 - Recipe 1 dialogue reconstruction from matched room tone, 183
 - Recipe 10 MFCC pitch with flux-driven time, 186
 - Recipe 11 transient-protected feature following, 187
 - Recipe 12 two guides with separate musical roles, 187
 - Recipe 13 reusable timbral imprint with an automated entrance, 188
 - Recipe 14 response-informed noise removal before extreme stretch, 188
 - Recipe 15 morph freeze and finish a hybrid pad, 188
 - Recipe 16 response-shaped dynamics and resonant afterimage, 189
 - Recipe 17 retuned harmony choir with controlled width, 189
 - Recipe 18 chord-constrained resonant cloud, 190
 - Recipe 19 inharmonic bell transformed into a frozen bed, 190
 - Recipe 2 archival speech with two-stage timing repair, 183
 - Recipe 20 stereo vocal transposition and harmony with image control, 190
 - Recipe 21 four-hour resumable installation render, 191
 - Recipe 22 deterministic delivery and null-test pair, 191
 - Recipe 23 matched transform and resolution suite, 191
 - Recipe 24 complex session with plan render and regression gate, 192
 - Recipe 3 field recording preservation with multiresolution stretch, 184
 - Recipe 4 coherent multichannel restoration, 184
 - Recipe 5 exponential expansion with a smoothed alternate take, 184
 - Recipe 6 polynomial pitch arc with formant protection, 185
 - Recipe 7 rhythmic sample-and-hold time folding, 185
 - Recipe 8 section-adaptive analysis resolution, 186
 - Recipe 9 inverse melodic breathing, 186
 - Reconstruction, 358
 - rect, 136
 - Red Bird*, 331
 - Reference Fourier baseline using explicit DFT mode, 261
 - Reference locking, 358
 - Reference-Channel Lock for Multichannel Content, 160
 - Reich, Steve, 335, 344
 - Related CLI Equivalents, 219
 - Release Checklist, 271
 - Replica*, 343
 - Reproduce, 264
 - Reproducibility Controls, 232
 - Reproducible Ambient Session With Manifest Logging, 169
 - Resampling, 359
 - Research Notes, 248
 - Research Questions Worth Testing, 248
 - Reshape and Densify a Control Map, 173
 - Resonance, 359
 - Resonant Ring Filtering with pvx ringfilter, 177
 - Response-Profile Denoising with pvx noisefilter, 175
 - Response-Shaped Spectral Dynamics with pvx spec-compander, 176
 - Restoration and delivery, 183
 - Resume + Append for Long Augmentation Runs, 181
 - Resume and append, 235
 - retune and intonation, 308
 - Retune Vocal to A440, 151
 - Reusable Response Artifact (PVXRF), 175
 - Reuse a Saved Feature Map for Fast Iteration, 202
 - Reynolds, Roger, 331
 - Richter, Max, 344
 - Rimsky-Korsakov, Nikolai, 337
 - Ring Modulation Fundamentals with pvx ring, 177
 - Ring Resonator Chordmapper Inharmonator, 211
 - Risset, Jean-Claude, 332
 - RMS, 359
 - rms norm - stretch, 194
 - Roger Reynolds and the genealogy of Transfigured Wind, 28
 - rolloff norm - pitch, 196
 - Rollout Guidance, 204
 - Room Recording Rehab (De-Reverb Then Stretch), 165
 - Room size, 359
 - rotating-head playback, 20

- Route Operator Patterns (const inv pow mul add
affine clip), 198
- Roygbiv*, 342
- Run PVC Parity Benchmark Gate, 174
- Running Any Command with uv, 55
- Running Up That Hill*, 342
- Running with uv, 248
- Runtime and CLI Flow, 79
- Saariaho, Kaija, 332
- Safe Starting Ranges, 203
- Safety Limiter + Hard Clip Guard, 162
- Sample, 359
- sample and hold, 349
- Sample rate, 359
- Sample Transform Use Cases, 76
- Sampling digital audio and the FFT threshold, 24
- Saving augmented datasets, 224
- Scale-Quantized Retune Followed by Unison
Widening, 166
- Scalloping loss, 359
- Schema Notes, 144
- Schoenberg, Arnold, 334
- Schubert, Franz, 334, 335
- Scope, 143
- Secure speech and large-scale vocoder engineering,
19
- Segment Processing Example (Python), 218
- Segment-Based Dynamic Stretch via CSV, 153
- Self-Supervised / Contrastive Learning, 255
- separation and decomposition, 309
- Sequenza III*, 334
- Severity Coverage and Curriculum, 239
- Shell Path, 269
- Short Ride in a Fast Machine*, 341
- short-time Fourier transform, *see* STFT
- Side lobe, 359
- Sidechain and feature routing, 186
- Sidechain Follow and Feature-Driven Control, 208
- Sidechain Pitch Trajectory (A Controls B), 153
- Simpler One-Line Pipelines, 62
- sine, 95
- Single-Feature Recipes, 194
- Sinusoid, 359
- Sinusoidal modeling and a neighboring lineage, 26
- Six Fantasies on a Poem by Thomas Campion*, 333
- Sleep*, 344
- Slow Down Speech, 149
- smotherstep, 351
- Smoothing, 359
- smoothstep S-curve, 350
- Soft clip and hard safety ceiling, 258
- soft-clip transfer functions, 347
- Songes*, 332
- Sorted Top-Level Command Roadmap, 145
- source modules
 - hps pitch track.py, 352
 - pvx augment.py, 352
 - pvx helpers.py, 352
 - pvx pipeline.py, 352
 - pvxanalysis.py, 352
 - pvxcodec.py, 352
 - pvxconform.py, 352
 - pvxdenoise.py, 352
 - pvxdeverb.py, 352
 - pvxenvelope.py, 352
 - pvxfilter.py, 352
 - pvxformant.py, 352
 - pvxfreeze.py, 352
 - pvxgain.py, 352
 - pvxharmmmap.py, 352
 - pvxharmonize.py, 352
 - pvxlayer.py, 352
 - pvxmorph.py, 352
 - pvxnoise.py, 352
 - pvxreshape.py, 352
 - pvxresponse.py, 352
 - pvxretune.py, 352
 - pvxring.py, 352
 - pvxrir.py, 352
 - pvxspecaugment.py, 352
 - pvxtrajectoryreverb.py, 352
 - pvxtransient.py, 352
 - pvxunison.py, 352
 - pvxwarp.py, 352
 - voc console.py, 352
 - voc parser.py, 352
 - voc.py, 352
- Source notes for the expanded history, 50
- Source paper and cited references to track, 322
- source-filter model, 2
- Spatial, 261
- spatial and multichannel, 310
- Spatial Coherence and Channel Alignment, 78
- Speaker and paralinguistic modeling, 242
- Speaker Verification / Identification, 252
- Speaker-Balanced Split Assignment for Augmenta-
tion, 180
- Spectral centroid, 359
- spectral centroid hz - brightness-linked pitch, 195
- Spectral envelope, 359
- Spectral envelopes formants and the source-filter
return, 30

- spectral flatness - noisy/tonal pitch response, 195
- Spectral flux, 359
- spectral flux - stretch, 195
- Spectral freeze, 359
- Spectral leakage, 359
- Spectral Peak Emphasis with pvx bandamp, 176
- spectral time frequency transforms, 313
- Spectral-composition context, 331
- Spectrogram, 359
- Spectrogram visualization snippet, 217
- Speech analysis before digital audio, 8
- Speech as the first difficult material, 43
- Speech Clarity Pass (Identity Lock + Conservative Window/Hop), 168
- Speech Enhancement and Denoising, 252
- Speech Natural Stretch (Hybrid Transients), 157
- Speech Robustness Set, 234
- Speech stretch with transient protection, 67
- Speech vs music probabilities, 197
- Spem in alium*, 333
- Spiegel im Spiegel*, 336
- Split Hygiene and Family Leakage, 240
- Springer machine, 20
- Springer, Anton Marian, 20
- Squarepusher, 338
- Stable in 0.1.x, 69
- Stable Installation, 267
- Stairstep (Sample-and-Hold) Control Mode, 171
- Stanford CCRMA and model-based sound, 42
- Start from a feature then scale and bias, 199
- Starter Path (Run These First), 148
- Stateful Stream Mode with Dynamic Stretch CSV, 172
- Step A Baseline stretch, 60
- Step B Add transient protection, 61
- Step C Pitch shift with formant preservation, 61
- Step D Auto profile planning, 61
- Step-by-Step Walkthrough (One Small WAV), 60
- Stereo, 359
- Stereo / Multichannel Coherence Modes, 316
- Stereo and multichannel coherence, 16
- Stereo coherence lock, 67
- Stereo coherence lock (recommended for wide stereo sources), 62
- Stereo Coherent Stretch, 157
- Stereo cues interaural level difference + interaural time difference, 197
- Stereo spatial hearing and multichannel history, 31
- STFT, 52, 359
- STFT Analysis / Synthesis Timeline, 315
- STFT Analysis and Synthesis, 74
- Stockhausen, Karlheinz, 341
- Stravinsky, Igor, 337, 340
- Streaming, 359
- Streaming Augmentation for Long-Form Audio, 227
- Stretch + Preserve Formants, 150
- Stretch vs Pitch Shift, 56
- String Quintet in C, Adagio*, 335
- Supernova*, 331
- Supported File Types, 56
- Supported Surface, 271
- Sustained instrumental sound and the discovery of coherence, 44
- Sustained tone and resonance, 335
- Symphonie fantastique, Marche au supplice*, 339
- Symphony No. 2, finale*, 340
- Symphony No. 4, finale*, 339
- Symphony No. 40, first movement*, 339
- Symphony No. 5, opening*, 343
- Symphony No. 7, Adagio*, 336
- Symphony No. 7, Allegretto*, 335
- Symphony No. 9, finale*, 339
- Synthesis frame, 360
- Synthesis hop, 360
- synthesis hop, 11
- Take Five*, 338
- Tallis, Thomas, 333
- Tape music and independent control by segmentation, 19
- Tape television and the wish to uncouple time from pitch, 10
- Task-Specific Policy Reasoning, 242
- Tavener, John, 337
- Teaching diagrams and the public life of the algorithm, 41
- Telegraphy telephony and the filter-bank imagination, 18
- Tempo + centroid, 197
- Tempophon, 20
- Temporal resolution, 360
- Tensor map function, 225
- TensorFlow / tf.data Integration, 225
- The analysis file as a new kind of score, 38
- The Art of Fugue, Contrapunctus I*, 339
- The Beatles, 341
- The Disintegration Loops 1.1*, 344
- The Firebird, Infernal Dance*, 340
- The hidden history of computation time, 35
- The historical thread, 8
- The Orb, 342
- The Protecting Veil*, 337

- The Rite of Spring, Augurs of Spring*, 337
- The STFT creates a time-frequency lattice, 4
- The Unanswered Question*, 340
- The Young Person's Guide to the Orchestra*, 340
- Three PVC-inspired laboratories, 141
- time compression
 - physical editing, 20
- Time map, 360
- time scale and pitch core, 314
- Time Stretch and Pitch Ratio, 76
- time stretching, 2
- Time-Compress Speech (Faster Playback), 149
- Time-frequency tradeoff, 360
- Time-scale modification, 360
- Time-stretch - denoise - dereverb in one pipe, 260
- Time-Varying Parameter Control (CSV/JSON), 57
- Time-Varying Spectral Filtering with pvx tvfilter, 175
- Timeline Warp with pvxwarp, 156
- TimeStretch and PitchShift Engine Selection, 226
- Tomorrow Never Knows*, 341
- Tongues of Fire*, 332
- Transfer functions, 345
- Transfer-Function and Automation Graph Atlas, 345
- Transfigured Wind I*, 331
- Transform A/B FFT vs DCT, 161
- Transform Availability and Selection (pvx transforms), 179
- Transform Backend Decision Path, 317
- Transform Backend Families and Tradeoffs, 74
- Transform Order Is Part of the Model, 237
- Transform Research A/B (FFT vs Hartley), 168
- Transform selection, 261
- Transient, 360
- Transient and rhythmic stress tests, 337
- Transient Feature Stack, 316
- Transient preservation, 360
- transient preservation, 11
- Transient Reset Mode for Cleaner Attacks, 159
- transientness - transient-safe stretch variation, 195
- Transients become a separate research object, 29
- Transients expose the limits of stationarity, 16
- Transients Stereo and Layer Split, 207
- Trevor Wishart and sound morphing, 28
- triangular, 98
- Troubleshooting, 269
- Troubleshooting Decision Tree, 318
- True envelope, 360
- tukey, 100
- tukey 0p1, 101
- tukey 0p25, 102
- tukey 0p75, 103
- tukey 0p9, 104
- UC San Diego CARL and a software culture, 26
- UCSD CARL and the command-line studio in detail, 42
- Ultra-smooth speech stretch (600x), 259
- Uninstalling, 268
- Unique Long Flags, 272
- Unvoiced, 360
- Unwrap, 360
- Upgrading, 268
- Use-Case Index (By Theme), 148
- Use-Case Matrix (Where to start quickly), 60
- Validation targets, 322
- Varese, Edgard, 338
- VBAP adaptive panning via algorithm dispatcher, 261
- Verify on Your Machine, 72
- Vibrato, 360
- Visual Mental Model of STFT, 59
- Vivaldi, Antonio, 335
- Vocal Retune by Scale/Root (pvxretune), 164
- Vocoder, 360
- vocoder, 8
 - compared with phase vocoder, 2
- Vocoder versus phase vocoder, 2
- Voice text and formants, 333
- Voiced, 360
- voicing prob - stretch, 194
- Vordhosbn*, 338
- Vox 5*, 330
- Vox Cycle*, 331
- W0, 80
- W1, 80
- W10, 82
- W11, 82
- W12, 82
- W13, 82
- W14, 82
- W15, 82
- W16, 83
- W17, 83
- W18, 83
- W2, 80
- W3, 81
- W4, 81
- W5, 81

-
- W6, 81
 - W7, 81
 - W8, 81
 - W9, 82
 - Wagner, Richard, 334, 335, 343
 - welch, 107
 - Wet signal, 360
 - Wet-dry mix, 360
 - What Is a Phase Vocoder, 2
 - What Outputs Should Sound Like, 61
 - What Problem pvx Solves, 55
 - What pvx should not copy, 141
 - What the Formula Installs, 268
 - What the historical work leaves us, 51
 - What the transform sees, 51
 - When to Keep Manual Pipes, 204
 - Why Augmentation Works, 236
 - Why basic phase propagation can still sound wrong, 8
 - Why This Change, 204
 - Why this paper matters, 321
 - Williams, Ralph Vaughan, 336
 - Window, 360
 - window function, 52
 - Window length, 360
 - Window/Overlap Tradeoff Map, 318
 - windows
 - bartlett, 96
 - bartlett hann, 99
 - blackman, 89
 - blackman nuttall, 93
 - blackmanharris, 90
 - bohman, 133
 - boxcar, 97
 - cauchy 0p5, 120
 - cauchy 1p0, 121
 - cauchy 2p0, 122
 - cosine, 134
 - cosine power 2, 123
 - cosine power 3, 124
 - cosine power 4, 125
 - exact blackman, 94
 - exponential 0p25, 117
 - exponential 0p5, 118
 - exponential 1p0, 119
 - flattop, 92
 - gaussian 0p25, 108
 - gaussian 0p35, 109
 - gaussian 0p45, 110
 - gaussian 0p55, 111
 - gaussian 0p65, 112
 - general gaussian 1p5 0p35, 113
 - general gaussian 2p0 0p35, 114
 - general gaussian 3p0 0p35, 115
 - general gaussian 4p0 0p35, 116
 - general hamming 0p50, 129
 - general hamming 0p60, 130
 - general hamming 0p70, 131
 - general hamming 0p80, 132
 - hamming, 88
 - hann, 87
 - hann poisson 0p5, 126
 - hann poisson 1p0, 127
 - hann poisson 2p0, 128
 - kaiser, 135
 - lanczos, 106
 - nuttall, 91
 - parzen, 105
 - rect, 136
 - sine, 95
 - triangular, 98
 - tukey, 100
 - tukey 0p1, 101
 - tukey 0p25, 102
 - tukey 0p75, 103
 - tukey 0p9, 104
 - welch, 107
 - Winter, Largo*, 335
 - Wishart, Trevor, 330–332
 - With explicit manifest paths, 234
 - WOLA, 360
 - WSOLA PSOLA granular methods and productive rivalry, 30
 - WSOLA Similarity Search, 316
 - WSOLA-Only Transient Handling, 159
 - Xenakis, Iannis, 340
 - YouTube searches, 330
 - Zero padding, 361